2027 赛季 · 第三版

信息学竞赛资料全书

431 份资料的完整分类目录

资料目录 431 条 + 知识正文 10 章 · 可直接查阅
按竞赛类别、任务需求、作用效果三重维度编排
附考场实务：报名流程、环境调试、爆零清单

三重交叉索引　A4 可打印装订　PDF 内嵌 32 条书签



431	08	06 • 31	321
收录资料	分类章节	编 · 章	可直达链接

编　者　NOI 竞赛教学组　　版　本　v3.0　2026 年 11 月　　适　用　CSP-J/S · NOIP · 省选 · NOI · IOI

2027 • RESOURCE CODEX

信息学竞赛资料全书

431 份资料的完整分类目录 · 2027 赛季第三版

编者：NOI 竞赛教学组

版本：v3.0　　编制日期：2026 年 11 月

适用对象：准备 CSP-J/S、NOIP、省选、NOI、IOI 的学生与教练

本书是什么　一份资料目录。收录 431 条信息学竞赛相关资源，每条含名称、作者/机构、出版方、年份、适用层级、任务需求、作用效果、获取途径、访问地址与一段评述。

本书不是什么　不是教材，不含任何资料的正文内容；不是题解，不讲解具体题目；不是排名，评述仅代表编者观点。

关于链接　书中链接在成书时经过批量校验，但互联网资源会迁移或消失。失效时请用「标题 + 关键词」检索，或参考第 17.4 节的处理流程。

关于版权　所有资料的著作权归原作者或原机构所有，本书仅作索引与评述。获取与使用请遵守各资源的许可协议，详见第 17 章。

本书可自由打印用于个人学习。转载、摘编或商业使用请联系编者。

卷首语 · 一份写给"想知道该读什么"的人的清单

信息学竞赛最不缺资料。洛谷上有两万多道题，Codeforces 上有上万道，OI Wiki 有上千个词条，国家集训队论文从 1999 年攒到今天已经两百多篇，再加上各种教材、课件、博客、视频课——一个刚入门的学生面对这些，第一反应从来不是"没东西学"，而是"我该从哪一个开始"。

这本书想解决的就是这个问题。它不教你算法，也不替你写代码。它做的是一件更基础、也更容易被忽视的事：把散落在互联网各个角落的资料，一条一条地收拢起来，分好类，标好难度、用途和效果，然后排成一份可以按图索骥的目录。你可以把它理解成一张地图——地图不能代替走路，但能让你少走三年弯路。

全书收录 431 份资料，覆盖教材与专著、官方文档与规则、在线平台与题库、集训队论文与专题讲义、历年真题与题解、课程与视频、工具与环境、社区与期刊八个大类。每一条都标注了四件事：

- 适用竞赛类别——从 CSP-J 到 IOI，你处在哪一段，就只看哪一段的书目；
- 任务需求——入门自修、体系学习、专题突破、赛前冲刺、教练备课、查漏补缺、实战训练；
- 作用效果——速查、深耕、实战、方法论；
- 获取途径——免费、开源、纸质、付费、需权限。

这三重分类不是装饰。同一本书，对初三刚入门的学生是"体系学习"，对省队选手可能只是"速查"；同一份题库，对教练是"备课素材"，对选手是"实战训练"。所以本书没有给出唯一正确的读法，而是给了三套交叉索引（第三编），你可以从任何一个维度切进去。

最后提醒一句：目录的价值在于被使用，而不是被收藏。如果你读完这本书只记住了一件事，我希望是——先选定一条路线，把它走完，再回来看下一段。资料永远读不完，但一条路是可以走完的。

使用说明 · 三种读法

本书不是线性读物。431 条资料按类型排成八章，但真正让它成为工具书的，是第三编的三套交叉索引。你可以从任何一个维度切进去。

读法一：按阶段找（适合学生）

先在第 3 章找到自己所在的阶段，再到第 16 章拿对应的清单，然后按清单里的任务到第二编找资料编号。这条路径最短，适合「现在就要用」的场景。

读法二：按类别找（适合专题补强）

直接翻第 13 章。那里按 CSP-J、CSP-S、NOIP、省选、NOI、IOI 六个层级各列一份书单，每份都标了「必读 / 选读 / 备用」。发现自己哪一块空，就去对应层级里挑。

读法三：按需求找（适合教练与自学者）

翻第 14 章。如果你要备一节课，看「教练备课」；如果你要补漏洞，看「查漏补缺」；如果你只剩两周，看「赛前冲刺」。这一章是按「你现在要干什么」组织的，而不是按「资料是什么」。

关于编号

全书 431 条资料统一编号，从 001 到 431，编号在第二编按章连续，在第三编的三套索引中被反复引用。写作、备课、排计划时，用编号指代资料比用书名更省事——例如「先做 128、145、167，再做 203」。

关于标签

每条资料带四类标签：竞赛类别 任务需求 作用效果 获取途径 。标签是本书的核心数据结构，印刷版用颜色区分，电子版可点击筛选。

> 标签含义：速查指遇到具体问题时才翻、不必通读；深耕指需要投入较长时间系统学习；实战指以做题/比赛为主；方法论指讲思路与策略而非具体算法。

目 录

卷首

卷首语 · 一份写给「想知道该读什么」的人的清单 3
使用说明 · 三种读法 4

第一编 竞赛体系与备考路线

第 1 章 信息学竞赛全景地图 8
第 2 章 NOI 大纲与 1—10 级难度体系 11
第 3 章 一年从零到 NOI：四阶段路线图 14
第 4 章 训练方法：刷题、比赛与复盘 16

第二编 资料总目

第 5 章 教材与专著 19
第 6 章 官方文档与竞赛规则 46
第 7 章 在线平台与题库 62
第 8 章 集训队论文与专题讲义 72
第 9 章 历年真题与题解 81
第 10 章 课程与视频 101
第 11 章 工具与环境 110
第 12 章 社区与期刊 115

第三编 三重索引

第 13 章 索引一 · 按竞赛类别 119
第 14 章 索引二 · 按任务需求 133
第 15 章 索引三 · 按作用效果 143

第四编 使用手册

第 16 章 分阶段执行清单 152
第 17 章 资料获取、版本与版权 154
第 18 章 常见问题三十问 156

第五编 考场实务

第 19 章 第一次上考场：从报名到交卷 159
第 20 章 环境调试：NOI Linux 2.0 全解 163
第 21 章 爆零清单：三十七个让分数归零的操作 167

第六编 知识正编

第 22 章 语言与代码规范 172
第 23 章 复杂度与常数优化 176
第 24 章 基础算法 186
第 25 章 搜索 201
第 26 章 动态规划 220
第 27 章 数据结构 248
第 28 章 图论 291

第 29 章　数学 334
第 30 章　字符串 366
第 31 章　计算几何与杂项 392

附录

附录 A　资料分布统计 413
附录 B　编号速查表 414
附录 C　版本记录与致谢 422

I

PART I · 第一编

竞赛体系与备考路线

THE SYSTEM & THE ROADMAP

先看清赛道，再谈资料。本编四章回答四个问题：这条赛道分几段、每段考什么、大纲怎么读、一年怎么排。

第 1 章　信息学竞赛全景地图

1.1 一条被切分成五段的赛道

中国的信息学竞赛体系，本质是一条从"校内兴趣"通向"国家队"的漏斗。每一段都有明确的晋级条件，也都有对应的备赛资料。搞清楚自己在哪一段，比盲目刷题重要得多。

阶段	赛事	时间	形式	晋级去向
CSP-J（入门级）	CSP-J/S 认证 · 入门级	9 月初赛 / 10 月复赛	第一轮资格测试 + 第二轮上机 4 题 3.5 小时	证明入门水平，无直接晋级
CSP-S（提高级）	CSP-J/S 认证 · 提高级	9 月初赛 / 10 月复赛	第一轮资格测试 + 第二轮上机 4 题 4 小时	成绩作为 NOIP 报名与省选资格的重要依据
NOIP（联赛）	全国青少年信息学奥林匹克联赛	11 月下旬	上机 4 题 3.5 小时，分普及组 / 提高组	提高组一等奖是省选报名门槛
省选	各省队选拔（2020 年起 CCF 统一命题）	4—5 月	通常两轮，每轮 3 题 5 小时	进入省队，取得 NOI 参赛资格
NOI	全国青少年信息学奥林匹克竞赛	7 月	两天，每天 3 题 5 小时	金牌前若干名进入国家集训队
CTSC / 国家队	集训队选拔赛	5 月及集训期间	多轮测试 + 论文答辩	IOI 中国队四人
IOI	国际信息学奥林匹克	8—9 月	两天，每天 3 题 5 小时	国际赛，最高舞台

另外还有三条并行的支线，容易被忽略但价值不低：APIO（亚太赛，5 月，中国区由 CCF 组织，难度略低于 IOI，是省队选手的第一次国际模拟）、WC（冬令营，1—2 月，讲课 + 测试，是当年命题风向的预演）、USACO / AtCoder / Codeforces 等国际在线赛（不受名额限制，随时可打）。

1.2 每一段真正在考什么

很多人把这条赛道理解成"难度递增"，这只对了一半。更准确的说法是：每一段考的能力维度不同。

CSP-J 考的是"能不能把想法写成正确的代码"。题目模型直白，暴力往往能拿部分分，真正的门槛是代码基本功：数组边界、循环范围、long long 溢出、多组数据清空。这一阶段刷题量比看书重要。

CSP-S 考的是"能不能识别出这是哪一类问题"。题目开始包装，同一道题可能既是图论又是 DP。这一阶段的瓶颈不在算法数量，而在*建模*——看到"最少代价使所有点连通"要能立刻想到最小生成树，看到"选与不选且互相制约"要能想到 2-SAT 或最小割。

NOIP 考的是"稳定的发挥"。四道题里通常两易一中一难，绝大多数人的失分不是不会，而是会做的题写挂了、该拿的部分分没拿。这一阶段最该练的是*对拍*、*造数据*、*估复杂度*。

省选 考的是"知识面的完整度"。数据结构、字符串、数论、多项式、计算几何，任何一块空缺都会在某一轮爆炸。省选是全书资料密度最高的一段，也是"书单"最能发挥作用的一段。

NOI 考的是"思维与工程的复合能力"。题目往往没有现成套路，需要自己构造算法，并且要在 5 小时内写出 300 行不出错的代码。这一阶段看的不再是教材，而是*集训队论文*——那里有前人总结的思维工具。

1.3 一张图看清资料与阶段的关系

把本书收录的 431 份资料按适用阶段统计，会看到一个明显的不均衡：面向 CSP-S 与 NOI 的资料最多，面向 CSP-J 与 IOI 的最少。这不是收集偏差，而是客观现实——中间段的人最多，写资料的人也最多。

阶段	资料特点	本书对应章节	建议投入
CSP-J	教材 + 入门题库 + 视频课，三件套足够	第 5、7、10 章	3—6 个月
CSP-S	体系教材 + 专题题单 + 真题	第 5、9 章，第 13 章书单	6—12 个月
NOIP	真题为主，教材为辅，强调稳定性	第 9 章，第 16 章清单	3—6 个月
省选	专题讲义 + 集训队论文 + 各省真题	第 8、9 章	6—12 个月
NOI / IOI	论文 + 国际题源 + 方法论	第 8 章，第 15 章	持续

一个常见误区是"提前读高阶段的资料"。省选讲义给刚学完 CSP-J 的学生看，结果只有一个：看不懂，然后怀疑自己不适合。正确的顺序永远是先把当前阶段的资料吃到饱，再向上够一级。

1.4 2026—2027 赛季的实际赛历

下面这张表来自 noi.cn 在 2026 年 9—10 月发布的正式通知，是本书写作时能查到的最新确定信息。赛历每年微调，考前必须回到官网核对一次。

赛事	时间	时长 / 题量	关键规则
CSP-J/S 2026 第一轮	2026 年 9 月 19 日	资格测试	成绩达本省晋级线方可参加第二轮
CSP-J/S 2026 第二轮	2026 年 10 月 31 日	入门 3.5 h / 提高 4 h	由各省负责人统一报名，选手只需按时缴费确认
NOIP 2026	2026 年 11 月 28 日 8:30—13:00	4 题 / 4.5 h	仅限在籍在校高中生；报名截止 11 月 9 日 14:00
NOI 2027 省选	各省自定（多在 2027 年 3—4 月）	通常两轮	CCF 不再统一命题；NOIP 2026 成绩权重不低于 30%
省队名单报送	2027 年 4 月 7 日 15:00 前	—	逾期的省份失去参加 NOI 2027 的资格
APIO 2026	2026 年 5 月 7—11 日	—	中国区由 CCF 组织，考点设于北大附中
NOI 2026	2026 年 7 月 18—24 日	两天各 3 题 / 5 h	承办校：青岛城阳第一高级中学
IOI 2026	2026 年 8 月 9—16 日	两天各 3 题 / 5 h	中国队 3 金 1 银

两个值得单独记住的政策变化：

- **省选不再统一命题。自 NOI 2027 起，CCF 不再组织统一省选，由各省自行完成选拔，题目可本省独立命制或多省联合命制。这意味着不同省份的省选难度、题型、时间都会分化，跨省参考往年真题时要先确认命题主体是否相同。**
- **A 类加分将取消。NOI 2027、2028 为过渡期，仍执行 A 类选手加 5 分；自 NOI 2029 起取消。对竞赛弱省选手而言，靠加分进入集训队的路径在两年后关闭。**

> 以上条款原文见第 6 章收录的《CCF 关于 NOI 2027 省内选拔的若干规定》与《关于自 NOI 2029 起取消 A 类选手加分政策的公告》，编号见附录 B。

第 2 章　NOI 大纲与 1—10 级难度体系

2.1 大纲是什么，为什么必须读

《全国青少年信息学奥林匹克系列竞赛大纲》（简称 NOI 大纲）由中国计算机学会组织编写，是命题的唯一官方依据。它把全部知识点按难度分成 1 到 10 级，并对应到三个竞赛层级：

- 入门级（对应 CSP-J）：1—5 级；
- 提高级（对应 CSP-S、NOIP）：5—8 级；
- NOI 级（对应省选、NOI、IOI）：8—10 级。

读大纲的最大收益不是"知道要学什么"，而是知道什么不用学。竞赛资料里充斥着远超考纲的内容（可持久化平衡树、动态图连通性、各种冷门构造），初学者很容易陷进去。大纲相当于一条护栏：护栏外的知识不是没用，而是*投入产出比在当下不划算*。

2.2 2025 版的主要变化

2025 版大纲做了一次幅度不小的调整，这些变化直接改变了备赛优先级，值得单独记下来：

层级	变化	对备赛的影响
提高级	新增 `bitset`	状态压缩与集合运算的常规工具，位上跑 DP 的题型会变多
	新增 扫描线	矩形面积并、区间覆盖类问题从"技巧"升格为"考点"
	新增 Manacher	由 NOI 级下放，回文题不再只能靠哈希硬做
	新增 多维 DP	二维以上的状态设计进入提高组范围
NOI 级	新增 基与线性基	异或最值、线性空间相关问题成为明确考点
	删除 跳跃表	可以不再投入时间
	删除 二维线段树	树套树的教学重心转向其他组合
	虚树由 10 级降至 8 级	重要度显著上升，省选阶段就应掌握
入门级	新增 前缀和（3 级）	从"技巧"变为"基础"，必须早学
	新增 差分（4 级）	与前缀和成对出现，区间修改的入门解法
	移除 格雷码	可降低优先级
通用	「指针」扩展为「指针与引用」	链表、树结构中引用传参的写法被正式纳入

> 说明：以上变化依据 2025 版大纲整理，具体条目与分级请以 noi.cn 发布的正式文本为准。本书第 6 章收录了大纲的官方获取地址。

2.3 把大纲翻译成学习顺序

大纲是按知识点罗列的，不是按学习顺序。真正可执行的做法是把它重排成四条纵向主线，每条主线内部严格由浅入深：

主线	内容与顺序	对应级别
语言与基础	输入输出 → 数组与字符串 → 函数与递归 → 结构体与指针/引用 → STL 容器 → 复杂度分析	1—3 级
算法思想	枚举 → 模拟 → 排序 → 二分与二分答案 → 贪心 → 前缀和与差分 → 递推与 DP 入门 → 搜索与剪枝	2—5 级
数据结构	栈与队列 → 链表 → 并查集 → 堆 → 树状数组 → 线段树 → 平衡树 → 可持久化 → 树链剖分	3—9 级
图论与数学	图的存储与遍历 → 最短路 → 最小生成树 → 拓扑排序 → 连通性 → 二分图 → 网络流 → 数论 → 组合计数 → 多项式 → 计算几何	4—10 级

四条主线不必齐头并进，但不能长期缺一条。省选翻车最常见的原因，就是"DP 很熟、图论没学"，或者"数据结构全会、数学推不动"。

2.4 用大纲做自测

给每个知识点打三种状态：不会 / 会用 / 能讲清。"会用"指能做模板题；"能讲清"指能说出它为什么正确、复杂度怎么来的、边界在哪。只到"会用"的知识点，在考场上换个包装就会崩。

建议每三个月做一次全量自测，把状态记在纸上。你会发现进步不是均匀的——某些专题会突然从"不会"跳到"能讲清"，另一些则长期卡在"会用"。后者就是下一步的突破口。

第 3 章　一年从零到 NOI：四阶段路线图

3.1 为什么是"四阶段"而不是"十二个月"

按月排计划几乎必然失败，因为学习速度不是线性的：入门期一个月能推进一大截，专题期一个月可能只啃下一个知识点。所以本书按能力状态而不是日历切分阶段，每个阶段有明确的进入条件和退出条件。时间只是参考。

阶段	目标状态	进入条件	退出条件
一 · 打底	能把想法无 bug 地写成代码	会基本语法	CSP-J 真题稳定 250+；模板题一次 AC 率 ⩾ 80%
二 · 建模	看到题能识别类型并选对武器	完成阶段一	CSP-S 真题稳定 200+；提高组知识点自测"会用"⩾ 90%
三 · 攻坚	掌握省选全套工具，能独立推出非模板题	完成阶段二	省选真题两轮各能做出 1.5 题以上
四 · 冲刺	稳定输出，考场不掉链子	完成阶段三	五次以上 5 小时全真模拟，成绩方差收敛

3.2 阶段一：打底（约 2—3 个月）

核心任务只有一个：把代码写对。这个阶段不需要学任何"高级算法"，需要的是把循环、数组、函数、递归写到肌肉记忆。题目难度控制在"想一想能写出来"的范围，追求的是*一次通过率*而不是题目难度。

推荐材料：一本入门教材通读（第 5 章）、一个入门题库按章节刷（第 7 章）、一套入门视频课跟完（第 10 章）。三者选一套主线，另外两套只做补充，千万不要同时开三套。

这个阶段最常见的错误是"看懂题解就等于会了"。判断标准很简单：关掉题解，从空白文件开始，能不能在 20 分钟内写完并 AC。不能，就是不会。

3.3 阶段二：建模（约 3—6 个月）

这一阶段要建立的是题型与武器的映射表。做法不是背，而是"每做完一道题，写一句话总结：这题的关键转化是什么"。积累到 100 句，你会发现自己开始能在读题时预测解法。

训练结构建议：专题（60%）+ 比赛（25%）+ 真题（15%）。专题按第 8 章的题单逐个攻破，每个专题做 15—25 题，做到"看见关键词就知道用哪套模板"。比赛用洛谷月赛、Codeforces Div.2/3，重点不是名次，而是*赛后把没做出来的题补掉*。

此阶段应完成 CSP-S 与 NOIP 近五年真题各一遍。做真题时严格计时，做完写复盘（见第 4 章）。

3.4 阶段三：攻坚（约 4—6 个月）

省选的难点在于"知识面 + 思维"。知识面靠清单式补全（第 2 章四条主线），思维靠读论文（第 8 章）。

建议的做法是每周一篇论文 + 一套省选真题。论文不要只读，要"讲"——合上文档，用自己的话把核心思想讲一遍，讲不通的地方就是没读懂。真题按省成套做，先做本省近五年，再做 ZJOI / SDOI / SCOI 这些难度较高的卷子。

这一阶段会经历明显的平台期：题目做不出来，论文看不懂，比赛一直挂。这是正常的，也是筛选发生的地方。有效的应对不是加大题量，而是降低题目难度、提高复盘质量——把一道题彻底吃透，胜过囫囵做十道。

3.5 阶段四：冲刺（约 1—2 个月）

冲刺期不学新东西。全部精力放在三件事上：

1. 全真模拟：每周至少两次，5 小时、断网、只用考场环境（NOI Linux）。目的是让身体适应长时段高强度思考。
2. 模板复写：把所有常用模板从零默写一遍，包括线段树、树剖、网络流、AC 自动机、NTT。考场现推等于送分。
3. 错题回炉：把过去半年的错题按知识点归类，只看不写，确认每个坑都记得。

同时要开始练"策略"：开题顺序、部分分分配、什么时候放弃、什么时候对拍。这些在考场上值 50 分以上，却几乎没人专门练。

3.6 一张时间表

时间	阶段	主线任务	关键赛事
第 1—3 月	打底	入门教材 + 基础题 300 道	CSP-J/S 第一轮（9 月）
第 4—6 月	建模	专题 + 提高组真题	CSP-S 第二轮（10 月）、NOIP（11 月）
第 7—10 月	攻坚	省选专题 + 论文 + 成套真题	WC（1—2 月）、省选（4—5 月）
第 11—12 月	冲刺	全真模拟 + 模板 + 错题	APIO（5 月）、NOI（7 月）

> 时间表只是骨架。真实备赛里，阶段之间会来回穿插——某次比赛暴露的短板，可能要退回上一阶段补两周。这不算失败，这叫迭代。

第 4 章　训练方法：刷题、比赛与复盘

4.1 刷题量的真相

"刷够 1000 题就能进省队"是流传最广的误解。题量当然重要，但真正决定水平的是有效题量——即"独立想出解法并一次写对"的题。抄题解刷完的 1000 题，有效题量可能不到 200。

一个粗略的参考区间（不是标准，只是量级）：

目标	有效题量	其中专题题	其中真题
CSP-J 一等奖	300—500	150	10 套
CSP-S / NOIP 一等奖	600—900	300	20 套
省队	1200—1800	600	40 套
NOI 金牌	2500+	1200	80 套

注意"其中"两列之和小于总量：剩下的部分是自由刷题，用来保持手感与扩展视野。

4.2 一道题该怎么用

同一道题，用法不同，收益差十倍。本书建议的四步法：

1. 限时独立思考（20—40 分钟）。写下所有想到的方向，哪怕最后没做出来。这一步训练的是"在没有提示的情况下探索"。
2. 看题解，只看关键转化。看到"啊，原来是这个"就停，剩下的自己想。整篇读完等于没读。
3. 关掉题解重写。从空白文件开始，写完自己造数据测。写不出来说明第 2 步看多了。
4. 写一句话总结。格式固定："这题的关键是把 X 转化成 Y，用 Z 维护。" 攒到 100 句，你的解题直觉就成型了。

这四步做完一道题平均要 60—90 分钟。相比"看题解 + 抄代码"的 15 分钟，效率看起来低，但前者留下的东西是后者的五倍以上。

4.3 比赛怎么打才不浪费

比赛的价值有 70% 在赛后。一场 Codeforces，真正该花时间的是"补题"而不是"打完看 rating"。补题的正确顺序：

- 赛时没思路的题：先补，因为它暴露的是知识或思维缺口；
- 赛时想到了但写挂的题：其次补，它暴露的是实现能力；

- 赛时没时间做的题：最后补，它只是速度问题。

补完题要读官方 Editorial，重点看"出题人是怎么想到这个解法的"——这一步才是把别人的思维装进自己脑子的过程。

4.4 复盘模板

每场比赛、每套真题之后填一张表，五分钟就够。坚持三个月，你会有一份极其精准的个人弱点清单。

项目	记录内容
失分类型	不会 / 想错 / 写挂 / 没时间 / 读错题（五选一，逐题标注）
知识点	这道题考的具体知识点（写到二级，如"DP-斜率优化"）
关键转化	一句话：题目的哪一步是我没想到的
代码问题	具体到行：数组开小 / 未开 long long / 多测未清空 / 边界写反
下次动作	可执行的一步，如"本周补完斜率优化 10 题"

"写挂"是最值得统计的一项。绝大多数人的失分不在难题，而在会做的题上丢 40—60 分。把最近十次模拟的"写挂"原因列出来，你会发现高度重复——通常是三四个老毛病。改掉它们，比多学一个算法划算得多。

4.5 关于对拍

对拍是竞赛里性价比最高的工程习惯，却在学生中普及率极低。一个标准对拍需要三样东西：暴力程序（保证正确但慢）、正解程序、随机数据生成器。运行几百组，比对输出。

什么时候必须对拍：涉及复杂数据结构、涉及大量边界情况、涉及贪心或 DP 的正确性不确定。什么时候可以不对拍：题目简单、样例已覆盖、时间紧张到必须取舍。

练对拍的最好方式是在平时刷题时就写，而不是考场上临时抱佛脚。第 11 章收录了对拍脚本与相关工具。

II

PART II · 第二编

资料总目

THE CATALOG

全书主体。431 份资料按类型分为八章，每条含编号、名称、作者、层级、用途、效果、获取途径与一段评述。编号在第三编的三套索引中被反复引用。

05 第 5 章　教材与专著

> 教材是唯一值得「从头读到尾」的资料类型。本章按使用场景分为四组：入门教材（能读懂、能照着写代码）、提高组教材（建立算法体系）、进阶专著（补数学与理论）、工具型参考书（随时翻，不必通读）。

本章收录 103 条，编号 001—103。

001 48 课搞定信息学奥赛：C++ 趣味编程

教材

人民邮电出版社 · 2025

CSP-J　入门自修　教练备课　深耕　纸质书（336 页，ISBN 9787115679116）

李超（B 站与公众号 ID @bigbigli）著，人民邮电出版社 2025 年 9 月第 1 版，336 页，ISBN 9787115679116，官方定位为 8 岁以上、面向 CSP-J/S 的零基础备赛书。它的差异点不在内容深浅，而在「课表化」：全书 10 章切成 48 课，每课固定为知识讲解、实例讲解、练习三段，一次课 1—2 小时可讲完，教练可以直接当社团课的教案用，不必自己拆课时。10 章依次是顺序结构（输出、变量、实数、除法与求余、强制类型转换、字符与 ASCII、复合运算）、选择结构（单/双/嵌套/多分支/switch）、循环结构（for、求和求积、while、break/continue、嵌套）、数组与字符串（一维二维数组、字符数组与常用操作）、排序（选择、冒泡及优化、插入、计数及其去重计数变体）、基础算法（暴力枚举、递推、函数与参数传递、结构体及排序、递归与汉诺塔、二分查找）、数学（因数与公倍数、辗转相除、质数与埃氏筛）、模拟（一维/二维数组模拟、日期模拟、字符串模拟）、算法进阶（贪心与证明、DFS、BFS、记忆化搜索到 DP）、数据结构（栈、队列、链表与指针、树与二叉树遍历、图的存储、Floyd 与 Dijkstra），附录给「信息学奥赛成长指南」。取舍很清楚：语言部分讲到结构体成员函数、排序比较器这一层，足够写 CSP-J 第二轮前两题；算法部分停在 BFS/DP 入门与最短路，覆盖 CSP-J 第 3—4 题常见模型。局限必须写清：全书没有树状数组、线段树、字符串算法（KMP/哈希）、网络流，DP 只到记忆化与线性，对 CSP-S 与 NOIP 明显不够；例题以课内练习为主，缺少整套限时模拟，需另配真题（见本目录历年真题与题解一章）；属零基础教材，已有语法基础者会嫌前 20 课冗余。

https://www.tenlong.com.tw/products/9787115679116

002 **C++编程与信息学竞赛数学思维** 教材

北京大学出版社 · 2026

CSP-J CSP-S 查漏补缺 体系学习 深耕 纸质书（配课件与教学视频）

王桂平、简星、耿霄、李周著，2026 年 6 月第 1 版，ISBN 9787301370063，306 页、482 千字，定价 89 元，中图法 TP312.8C++ / TP301.6。作者王桂平是重庆交通大学计算机系副教授，2003 年起带队参加 ACM/ICPC、天梯赛、蓝桥杯，另著有《信息学竞赛教程（初级）》《C++ 趣味编程及算法入门》——后者的思路在这本书里被推到竞赛端。全书 12 章：变量及输入输出、用单位正方形与单位立方体理解数学问题、基本运算的理解、数列问题、递推及递推问题求解、一维和二维数表、方向位置与直角坐标系、网格坐标系及网格问题、初等几何问题、初等数论问题、集合论基础、组合数学初探，含 200 余道微实例、130 余道编程实例与 300 余幅自绘图表。它真正稀缺的地方不是覆盖面（这些知识点任何一本入门书都有），而是讲法：把「乘法分配律」画成单位正方形拼长方形、把「勾股数」画成正方形面积、把网格题画成坐标系，用图形替代公式来建立直觉。这恰好补上了国内信奥教材的短板——绝大多数教材默认读者已经具备初中数学的抽象能力，一上来就写递推式，而小学高年级起步的学员卡在「为什么这个递推是对的」这一步。局限：数学深度止于初等数论（同余、gcd）与组合初探，没有生成函数、没有群论、没有线性代数，不能当数学专题书用；例题以自编微实例为主，缺少 CSP 真题的难度梯度；「用正方形理解乘法」这类讲法对高中生显得幼稚，成年自学者会觉得啰嗦。适合的对象是小学高年级到初一的自学者、以及带低龄班的教练——后者可直接借用书里的图示做课件。

https://item.winxuan.com/1204380298

003 **CCF CSP 例题解析与实战技巧：编程基础** 教材

机械工业出版社（中国计算机学会 组编） · 2026

CSP-J CSP-S 入门自修 查漏补缺 实战 速查 纸质书（59 元，165 页）

朱青、朴乘志编著，中国计算机学会组编，总主编张孝（中国人民大学信息学院副院长），2026 年 8 月第 1 版，ISBN 978-7-111-81458-0，165 页 59 元。属「CCF CSP 认证辅导丛书」，全套三册按认证题目顺序分册：本册对第 1—2 题、工程实现对第 3 题、算法应用对第 4 题。全书 6 章——绪论、循环（对应第 1 题），排序、数学基础（对应第 2 题的子任务），序列查询与维护、动态规划（对应第 2 题的满分解法），真题范围 2014—2024。价值在于它把「第 2 题怎么从暴力拿到满分」拆成了可复用的三段：先写对（循环与模拟的边界）、再写快（把重复扫描换成前缀和/排序/差分）、最后写稳（边界、溢出、多组数据）。这三段与 CSP-J 后两题、CSP-S 前两题的训练点同源，适合作为「会语法但总在第 2 题丢分」的补漏材料。局限有三：出版社与 CCF 都明确这套丛书面向高校学生与职场人士、不面向中小学生，题面背景是认证式而非竞赛式；内容止于第 2 题，不含树、图、DP 优化；例题难度低于 NOIP 提高组后两题，指望它冲省选是一厢情愿。

https://ebooks.cmpbook.com/detail?id=30183

004 **CCF CSP 第一轮认证一本通（第 2 版）** 教材

清华大学出版社 · 2026

CSP-J CSP-S 查漏补缺 赛前冲刺 速查 实战 纸质书

丁向民编著，2026 年 8 月第 2 版第 1 次印刷，ISBN 9787302724575，16 开 249 页、39.4 万字，定价 79.9 元，中图法 TP311-49。结构八章：前四章按选择题考点分块（计算机基础知识、程序设计基础、基本数据结构、离散数学与组合数学），每节固定三段式「基本知识介绍—典型习题解析—知识点巩固」；后四章专攻阅读程序题与完善程序题。第 2 版与前版最大的差别不在加题而在配比：作者在前言里给出 2019—2024 六年选择题逐知识点出题数的统计表，据此压缩了基本常识、系统结构、软件系统这些多年不考的内容，把占比上升的第 2 章 C++ 语言基础扩写，删减第 4 章应用数学，等于按真题频次重排篇幅。时效性上还有一层背景：CSP-J/S 大纲 2021 年发布，2023、2025 年两次修订，本书对齐 2025 年修订版，这是它相对旧版与多数同类的优势所在。用法：先用前四章的「知识点巩固」按块补漏，再用后四章做程序阅读与填空的专项训练，最后整卷计时验收。局限：只覆盖第一轮笔试，不含上机；例题以真题改编为主，原创题比例偏低；无配套在线题库，答案与解析都在书内。

https://www.tup.tsinghua.edu.cn/bookscenter/book_10454001.html

005 **CCF GESP 直通车：C++ 六级七级精讲精练** 教材

复旦大学出版社 · 2026

CSP-J 入门自修 查漏补缺 深耕 速查 纸质书

沈根成编著，2026 年 8 月第 1 版，ISBN 9787309188530，169 页、26cm，定价 98 元，中图法 TP312.8。“CCF GESP 直通车”系列的第四本，按 GESP 六级、七级的 C++ 考试大纲编写，全书七部分：类与面向对象、抽象数据类型（栈、队列及其容器适配器）、树与遍历（二叉树、哈夫曼树与哈夫曼编码、DFS/BFS）、一维动态规划（背包、最长上升子序列）、高级数学库函数与哈希表、二维动态规划（最长公共子序列、石子合并等区间 DP）、初级图论（图的表示与存储、遍历及应用）。它在知识阶梯上的位置很明确：六级七级正好卡在“语法学完、复赛刚起步”，对应本书 22—26 章的入口段，比《CCF 信息学奥赛入门篇》深一层，又够不到《提高篇》的线段树、网络流。局限要说清楚：169 页摊七个专题，每块只到“能写出模板”，复杂度推导与优化基本没有；字符串算法、树状数组与线段树、网络流一概不涉及；例题以考级题型为主，竞赛真题比例低；GESP 是 CCF 的等级认证而非竞赛本身，拿它冲 CSP-J 一等奖还得另配真题卷与模拟卷（见真题卷与模拟卷两本）。适合的对象是自学的中小学生和带入门班的教练——每部分末尾的延伸阅读可以当课堂补充材料。

http://www.cnbip.cn/BaseInfo/BookInfo.aspx?id=202291928

006 **CCF 中学生计算机程序设计** 入门篇 教材

中国计算机学会 组编 · 科学普及出版社 · 2016

CSP-J 入门自修 体系学习 深耕 纸质书

主办方自己出的教材，与大纲覆盖面高度一致，适合作为第一本系统读本。

https://www.noi.cn/

007 **CCF 中学生计算机程序设计** 基础篇 教材

陈颖、邱桂香、朱全民 主编 · 科学普及出版社 · 2017

CSP-J CSP-S 体系学习 深耕 纸质书

承接入门篇，把数组、函数、结构体与基础算法补齐，语法体系最稳。

https://www.noi.cn/

008 **CCF 信息学奥赛基础篇** 教材

机械工业出版社（中国计算机学会组编） · 2025

CSP-J CSP-S 体系学习 教练备课 深耕 纸质书

朱全民主编、江涛编，2025 年 8 月第 1 版，ISBN 978-7-111-78233-9，99 元。官方「CCF 全国青少年信息学奥林匹克竞赛教程」丛书第二册，覆盖 NOI 大纲中难度系数 3—4 及少量 5 的知识点，正好补上《入门篇》（1—2）与《提高篇》（5—8）之间的空档。全书四章：C++ 程序设计进阶（二维与多维数组、常用数学函数、指针、结构体与联合体、STL 算法函数）、数据结构及其应用、算法设计、数学运用。每节按「情境导航—问题抽象—知识探究—实践应用—总结提升」五段展开，附录给出知识点与大纲条目的对应表。适用场景：学完语法、能写循环但不会建图建模的 CSP-J 冲一等学生；教练也可直接当校本教材，因为每节都能对回大纲编号。

https://ebooks.cmpbook.com/detail?id=28087

009 **Cracking the Coding Interview** 教材

Gayle Laakmann McDowell · CareerCup · 2008

CSP-J CSP-S 入门自修 查漏补缺 实战 纸质书

入门友好，讲清「面试官想看什么」。对 OI 帮助有限，但适合刚开始写代码时建立手感。

010 **Think Like a Programmer** 教材

V. Anton Spraul · No Starch Press · 2012

CSP-J CSP-S 入门自修 方法论 纸质 / 电子书

教「怎么把问题拆成可编码的步骤」，而不是教语法。初学者最缺的正是这一层。

011 中学生计算机程序算法入门：例题精析与训练

教材

科学出版社 · 2025

CSP-J 入门自修 实战训练 实战 纸质书

陈锋编著，2025 年 1 月第 1 版，ISBN 9787030802309，B5 开、330 页、32.7 万字，定价 58 元。全书 8 章，例题全部取自 AtCoder Beginner Contest（ABC201B、ABC242B、ABC231B、ABC219C、ABC203C、ABC208C、ABC228C、ABC240B、ABC260B、ABC272C、ABC273C、ABC323B/C 等），章末习题亦为同源真题，覆盖排序、算法设计策略、枚举、模拟、二分、贪心、搜索、基础数据结构、动态规划、数论、组合数学，并专讲 STL 的高效用法。它真正的差异点是题源：国内入门书多用洛谷/一本通题库，这本书直接用 AtCoder 的 ABC 题，题面短、数据弱、判题即时，初学者可以在学完一节后立刻在线提交拿到反馈，不必等中文 OJ 的题面翻译，也不会被「题面比算法难读」卡住。定位要讲清：难度天花板明确停在 CSP-J 第一轮与复赛前两题，没有图论算法（最短路、生成树、连通性一概没有）、没有高级数据结构、没有字符串算法；330 页 8 章配 58 元定价，意味着单题讲解偏简，多为「题意—思路—代码」三段，不适合当唯一教材。适合小学高年级到初一的自学者、以及带低龄班的教练做课后作业册（题号可直接定位到 AtCoder，布置与验收都省事）。已学完语法、能独立写搜索与简单 DP 的选手跳过即可。

https://book.douban.com/isbn/9787030802309/

012 信奥逐梦之路：家长陪伴与升学规划全阶段指南

专著

人民邮电出版社 · 2026

CSP-J CSP-S NOIP 教练备课 体系学习 方法论 纸质书（192 页，59.80 元）

王津著，2026 年 2 月第 1 版，ISBN 9787115690357，16 开 192 页、228 千字，定价 59.80 元，中图法 TP311.1-62，读者范围官方标注为「参加信奥竞赛孩子的家长」。作者是梦熊信奥联盟创始人，团队为全国百余所信息学竞赛强校提供教研服务。这不是算法教材，而是全库目前唯一一本专门处理「赛事体系与长期规划」的专著：第一章先把 NOI、NOIP、CSP-J/S、GESP 四条线的区别、衔接与各自适用学段拆开，再讲参赛适配性、C++ 学习必要性、课程与训练方案、学训常见误区、竞赛与文化课的时间平衡，最后收录 10 位 NOI 金银牌得主的成长回忆录，并提出 3T 分层培养 + 5M 闭环训练的阶段目标表。它的实际读者是家长与低年级教练：多数家庭的第一处失分不在算法，而在把「CSP 评级」当成「NOIP 获奖」、把 GESP 当成 CSP 前置、或在小学阶段误判报名资格（参见本目录《关于参加 CSP-J/S 认证需符合年龄条件的公告》），这本书提供的是一张能把政策、时间投入与阶段目标对齐的地图。教练可取其章节框架开家长会或做社团入门宣讲。局限必须写清：全书不含算法讲解与代码，不能替代任何教材；「升学优势」「特殊招生渠道」等内容与当年各省政策强绑定，2026 年之后须与 CCF 及本地教育部门最新文件核对；作者本身运营赛训平台，机构筛选、课程选择等章节含商业视角，涉及「保过奖」「速成班」的判断虽直指行业乱象，仍应结合 CCF《关于 NOI 相关权益的声明》自行复核。

https://book.cppinfo.cn/Encyclopedias/home/index?id=4989075

013 **信息学奥赛 CSP-J/S 数学考点大全** 教材

北京大学出版社 · 2026

CSP-J CSP-S 查漏补缺 入门自修 速查 深耕 纸质书（288 页，16 开，ISBN 9787301379028）

李栋、王子康编著，北京大学出版社 2026 年 10 月第 1 版，288 页、433 千字，16 开，定价 79 元，ISBN 9787301379028。定位不是「再讲一遍数学」，而是把 CSP-J/S 真正会考的数学裁成一份最小必要清单：内容提要列出的模块是进制与编码（原码反码补码、位运算）、初等数论（素数判定、算术基本定理、gcd、同余、快速幂）、组合数学（排列组合、杨辉三角、容斥、抽屉原理、错排）、逻辑与推理（表达式化简、命题等价）、递推与递归（斐波那契类递推、错排、Catalan 数思想）五块；体例是「生活实例—概念推导—真题映射」三步，例题按基础、核心、进阶三级分层，官方定位写给数学基础薄弱、备考 CSP-J/S、NOIP 与 GESP 1—8 级的中小学生。两位作者的身份解释了这本书的取舍：李栋本硕连读于清华数学系、CCF 认证 NOI 指导教师，王子康是 NOI 讲师、上海交大人工智能博士，两人都在培训一线，所以章节按考情而非按数学分支切。与已收的《C++ 编程与信息学竞赛数学基础》（王桂平，北大社 2025-08，364 页）对比：那本按数学分支系统铺开（含集合论、图论基础、编码译码、博弈论基础、算法复杂度），当教材通读；这本 288 页、按考点裁剪、带分层例题，当备考手册查漏。局限：止步于 CSP-J/S 与 NOIP 的数学基础，没有线性代数、多项式、概率期望的竞赛深化，省选以上不适用；为了照顾低龄读者，证明一律从简，想看严格推导要另找书；数学书不能替代代码训练，配套题量小于专门题集。

https://product.dangdang.com/30120305.html

014 **信息学奥赛1：114道题零基础吃透C++与算法基础** 教材

北京大学出版社 · 2026

CSP-J 入门自修 实战训练 实战 纸质书（配视频教程）

王健伟著，2026 年 3 月第 1 版，ISBN 9787301370056，388 页、26cm，中图法 TP311.1。这是王氏三册系列的第 1 册——第 2 册是《信息学奥赛2：121 道题零基础吃透高级算法与数据结构》（DP 与四类结构），第 3 册是《信息学奥赛3：历年真题解析与拉分解题思维训练》（近十年 CSP-J 真题），三册合起来正好是从语法到真题的一条线，此前库里只收了后两册，这一册补上入门端。内容按 16 章分两块：第 1—7 章是 C++ 语言基础（环境搭建、基础语法、数组、函数、编译预处理、结构体、scanf/cout/cin 与命名空间、string、二/八/十/十六进制互转、位运算），第 8—16 章全是刷题，按八类题型推进——排序、递推、递归、搜索与回溯、高精度、贪心、分治、广度优先搜索，共 114 道经典例题，每题配代码与讲解，另附视频教程。定位要说清：它的天花板就是 CSP-J 第一轮与复赛前两题，动态规划、图论、树状数组一概不涉及，覆盖范围比《CCF 信息学奥赛入门篇》略窄但题量更密；「114 道题」这个规模的真正价值是题单完整、题型不重不漏，适合零基础学员按章过关，也适合教练拿它当入门班的作业册。不适合的人群也很明确：已经能独立写搜索与简单 DP 的选手用它只会浪费时间；想系统学算法体系应直接上《算法竞赛进阶指南》或《CCF 信息学奥赛基础篇》。

https://book.cppinfo.cn/Encyclopedias/home/index?id=5046086

015 信息学奥赛2：121道题零基础吃透高级算法与数据结构

教材

北京大学出版社 · 2026

CSP-J 专题突破 实战训练 实战 纸质书

王健伟著，2026 年 4 月第 1 版，ISBN 9787301372326，系列三册中的第 2 册。前半讲动态规划（网格路径、序列、背包、区间、分割、编辑距离、计数），后半讲栈、队列、树、图四类结构，共 121 道题配视频讲解。定位明确指向 CSP-J 的 DP 与数据结构分水岭，适合在基础语法过关后集中攻难点。

https://www.gxjcy.cn/web/tmDetails?jiaocaiId=1033179

016 信息学奥赛一本通 C++ 版

教材

董永建、黄新军 等 · 科学技术文献出版社 · 2013

CSP-J 入门自修 深耕 纸质书 + 在线题库

国内中学教学用得最多的入门教材，配套题库与视频齐全，最适合零基础自学。

017 信息学奥赛一本通 初赛篇

教材

董永建 等 · 南京大学出版社 · 2019

CSP-J CSP-S 赛前冲刺 速查 实战 纸质书

专攻第一轮笔试：计算机基础、阅读程序、完善程序，是国内少见的初赛专项书。

018 信息学奥赛初赛全解

教材

清华大学出版社 · 2026

CSP-J CSP-S 查漏补缺 赛前冲刺 实战 速查 纸质书

马强主编，2026 年 8 月第 1 版，ISBN 9787302721734，16 开 164 页，定价 79 元，中图法 TP311.1-44。作者 2007 年起做信息学奥赛教学，先后任教于北京十一学校、人大附中朝阳学校、首师大附中，后任信息学教研负责人，曾参与设计 CCF GESP 考试大纲。全书三部分七章：第 1—5 章按初赛考点分块——计算机基础知识、C++ 语法、数据结构、数学（含排列组合与复杂度估算）、算法，每块配专题练习；第 6 章是 CSP-J 与 CSP-S 各一份模拟卷，附答案与思路解析；第 7 章是近几年真题的逐题解析。与同作者 2024 年的《信息学奥赛导学·初赛篇》（ISBN 9787302673231）是两条线：导学篇按讲义顺序铺知识点，本书按题型分块、以题带点，做题密度更高。定位必须说清：只覆盖第一轮笔试（选择、判断、程序阅读、程序填空），不含任何上机实现，替代不了复赛教材；模拟卷的难度标定出自作者个人判断，跨省横向比较时要打折。适合初赛前六到八周做查漏补缺：先做一套卷定位失分模块，再回对应章节补，最后用真题章验收。

http://www.ceeol.cn/bookshow.asp?id=3228843

019 信息学奥赛大通关·入门篇（上下册）

教材

华南理工大学出版社 · 2026

CSP-J 入门自修 体系学习 深耕 纸质书

王晓鹏、周弋林编著，2026 年 4 月第 1 版，ISBN 978-7-5623-8150-1，504 页，全 2 册定价 89 元。王晓鹏为广州大学附属中学信息学主教练、NOI 金牌教练，该书由广州市教育局协同育人重点项目资助。八个单元从「从计算机到信息学奥赛」起，经顺序、分支、循环、数组、字符串、自定义函数与结构体，收在 C++ 标准模板库，另附普及组真题选讲。写法上每个单元先给思维导图搭框架，语法与算法建模同步引入，并对通用题给「一题多解」对比。适合小学高年级至初一、有教练或家长带读的起步阶段；对已过语法、只缺算法的读者边际价值有限。

https://hg.cb.scut.edu.cn/bookCenter/15215.html

020 信息学奥赛打通关 · 入门篇（全 2 册）

教材

华南理工大学出版社 · 2026

CSP-J 入门自修 深耕 纸质书（全 2 册，490 页）

王晓鹏、周弋林著，2026 年 4 月第 1 版，ISBN 9787562381501，全 2 册共 490 页。作者王晓鹏是广州大学附属中学信息学主教练、NOI 金牌教练，近四年带出 5 人次进入广东省队、NOI 1 金 2 银 2 铜。上册按「从计算机到信息学奥赛—顺序结构—分支结构—循环结构」四个单元展开，每课固定四段：思维导图导入、知识点、牛刀小试、拓展阅读，下册续接后续单元。它与市面上多数入门书的两点差异值得注意：一是「比特时空观」「数位与进位基础」这类内容被前置到第一单元，把「数据在机器里怎么存」讲在语法之前，后面讲溢出、取模、位运算时就不需要再补课；二是明确写了「一题多解」——同一道题给出不同算法思路与代码策略的对比，这对入门阶段形成「解法可迁移」的意识比多做十道题更有用。适合小学高年级到初一、由教练或家长带读的起步阶段，也适合作为校内社团课的分层教材。局限：定位是零基础，算法部分只到枚举、模拟、简单排序与递推，不覆盖 DP 与图论；书名虽含「竞赛」，但深度对应 CSP-J 第一轮与第二轮前两题，不能替代提高组教材。

https://thep.com.tw/book/1819533

021 信息学奥赛课课通

教材

林厚从 · 南京大学出版社 · 2019

CSP-J CSP-S 入门自修 体系学习 深耕 纸质书

按课时切分的教材，一节一个小知识点配一道练习。适合学校把它当正课教材用，节奏比一本通更松。

022 **信息学竞赛教程（初级）** 教材

北京大学出版社 · 2025

CSP-J CSP-S 体系学习 教练备课 深耕 纸质书

王桂平、周祖松、万毅、陈胤戬编著，2025 年 10 月第 1 版，ISBN 9787301364277，449 页，定价 99 元。特点是「按大纲分层」：依据 CCF《NOI 系列竞赛大纲》把知识点按难度切成初级、中级、高级三层，各对应一册、每册 40 章，初级册覆盖基础算法、进制转换、位运算、编码问题、数列问题、高精度、字符串处理、时间日期、数据结构、排序、搜索、动态规划、数论、组合数学、图论共 15 个专题。每章固定「理论 + 实践」两段：先讲算法思想与数学依据，再用经典竞赛题落到实现，配 120 个案例解析、500 余道实战题与 300 段视频讲解。对教练的实际价值在于课时可切分——40 章正好对应一个学年 40 周，不必自己重排《进阶指南》的章节顺序。局限要说清：初级册深度只到 CSP-J 全部加 CSP-S 前半，提高段仍需接《算法竞赛进阶指南》或 CCF 提高篇；作者团队以重庆育才、重庆一中教练为主，例题偏西南地区训练风格。

https://book.douban.com/isbn/9787301364277/

023 **啊哈！算法** 教材

纪磊（啊哈磊） · 人民邮电出版社 · 2014

CSP-J 入门自修 深耕 纸质书

用插图和故事讲算法，零基础建立直觉最快的一本。深度有限，作用是「破冰」而非「学完」。

024 **我的第一本算法书** 教材

石田保辉、宫崎修一 · 人民邮电出版社 · 2018

CSP-J 入门自修 速查 纸质书

全彩步骤图讲 7 种数据结构与 26 个算法，适合当作视觉化的预习材料。

025 **我的第一本算法竞赛启蒙书** 教材

科学出版社 · 2026

CSP-J 入门自修 深耕 纸质书

钟旸、陈茁、罗云珂著，2026 年 6 月第 1 版，ISBN 9787030838193。面向零基础青少年，按「语言基础—算法与数学思维—数据结构与图论入门」三梯度展开，特点是先从数学原理引出算法再落到代码，并配线上评测。适合小学高年级至初一、家长或教练带读的起步阶段。

https://www.aijiaocai.com/textbook/details?textbook_id=14247161

026 **深入浅出程序设计竞赛** 基础篇 教材

洛谷学术组、汪楚奇 · 高等教育出版社 · 2020

CSP-J CSP-S 入门自修 深耕 纸质书

每章开头有思维导图，近 400 道例题全部可在洛谷提交，边读边判题，反馈闭环最短。

https://www.luogu.com.cn/

027 **用 AI 决胜信息学奥赛：C++ 编程基础** 教材

电子工业出版社 · 2026

CSP-J 入门自修 查漏补缺 方法论 纸质书

王俊杰著，2026 年 3 月第 1 版，ISBN 9787121523205，16 开 420 页，定价 99 元。作者为北大、北理工双硕士，曾任海淀区高中信息技术教师与字节、腾讯、百度研发，现做 AI 教育，身份组合正好对上这本书的两条主线。全书三篇：认识篇讲信息学奥赛的赛事阶梯（CSP-J/S—NOIP—省选—NOI—IOI）与学习路径；能力篇是全书重心，讲提示词的设计要素与模板、常用 AI 工具的具体用法，把「让 AI 解释概念、审查边界条件、生成变式题、规范代码风格」拆成可复用的操作流程；实战篇才是 C++ 语法本体——二进制、数据类型、流程控制、函数、数组、指针、结构体，每节配提示词案例与测试题。定位要说清楚：它是「零基础语法 + AI 使用法」，不是算法书，动态规划、图论、数据结构一概不涉，因此只能作为第 22 章（语言与工程习惯）的辅助读物，不能替代任何一本竞赛教材。真正的看点在能力篇——AI 辅助学习的边界（不许直接抄答案、AI 会编错解、基本功必须自己练）写得比多数同类书清醒。局限：配套资源页标注「本书暂无资源」，无在线题库；提示词效果强依赖所用模型的版本，2026 年可用的模板到学生手里未必同样有效。

https://www.hxedu.com.cn/hxedu/hg/book/bookInfo.html?code=TP523200

028 **算法与数学：数学思维与编程实践** 教材

科学出版社 · 2025

CSP-J CSP-S 查漏补缺 体系学习 深耕 纸质书

米田优峻著，李曙、余岳译，2025 年 7 月第 1 版，ISBN 9787030822437，288 页，定价 98 元，中图法 TP301.6，原作名《問題解決のための「アルゴリズム×数学」が基礎からしっかり身につく本》。与同作者的《算法竞赛黄金法则》同期推出、出自同一译者团队，但两书分工不同：那本按「技巧」组织，这本按「数学」组织，把算法学习所需的中学到大学数学压成 9 个要点，再逐条落到二分查找、素性判定、欧几里得算法、蒙特卡罗方法、排序、动态规划、埃氏筛、图论算法上，配 200 余道例题习题与 30 道综合测试题。价值在于补的是「为什么这个算法成立」这一层：多数竞赛教材把递推关系、同余、期望、图论结构当工具直接给结论，本书反过来从数学结构推算法，适合已经能写模板、但一遇变形题就不知从何下手的 CSP-S 段读者。局限：不覆盖线段树、网络流、字符串、计算几何等专题，不能当唯一教材；翻译腔较重，部分术语与国内竞赛习惯叫法不同（如「蒙特卡罗方法」在竞赛语境里多称随机化算法）；例题以日本赛题（AtCoder、JOI）为主，与 CSP/NOIP 真题风格有差异，需自行迁移。

https://book.douban.com/isbn/9787030822437/

029 **算法图解** 教材

Aditya Bhargava · 人民邮电出版社 · 2017

CSP-J 入门自修 深耕 纸质书 / 电子书

图多字少，讲清二分、递归、动态规划的基本直觉。代码是 Python，不影响理解。

030 **算法竞赛从新手到高手** 教材

清华大学出版社 · 2026

CSP-J CSP-S 入门自修 体系学习 深耕 纸质书

陈梦、姜慧慧编著，2026 年 6 月第 1 版，ISBN 9787302715795，共 9 章。路线是「先数据结构后算法」：数组、栈队列、树与二叉树、图（最短路、最小生成树、关键路径），再落到排序、贪心、DP 三类经典算法，每节配竞赛真题。特点是把复杂度分析和「算法分析在竞赛中的应用」提前到第 1 章讲，适合作为第一本系统教材——但深度止于 CSP-S 中段，冲省选需换书。

https://www.zxhsd.com/kgsm/ts/2026/05/22/6863300.shtml

031 **算法竞赛入门到进阶** 教材

罗勇军、郭卫斌 · 清华大学出版社 · 2019

CSP-J CSP-S 入门自修 体系学习 深耕 纸质书

覆盖面比紫书更平缓，从语法一路铺到进阶，适合作为「一本读到黑」的主线。

032 **算法竞赛入门经典** 紫书（第 2 版） 教材

刘汝佳 · 清华大学出版社 · 2014

CSP-J CSP-S 入门自修 体系学习 深耕 纸质书 / 配套习题与解答

中文圈发行量最大的竞赛教材。语言、算法、解题方法三位一体，缺点是例题偏难，第一遍别死磕。

033 **算法竞赛试炼场：洛谷300题精析** 教材

清华大学出版社 · 2026

CSP-J CSP-S 入门自修 实战训练 实战 深耕 纸质书

罗勇军、汪楚奇（kkksc03）、陈哲（chen_zhe）合著，2026 年 9 月第 1 版。以洛谷 330 余道题为主线，按「语言入门—模拟枚举—基础算法—数据结构—搜索—数学—DP—图论」八篇递进，每题给出考点、思路与证明、代码，实质是补上「听懂知识点」到「考场能写」之间的题量桥梁。适合已学完语法、需要成体系刷题的 CSP-J/S 选手。

https://www.tup.com.cn/bookscenter/book_10967201.html

034 算法竞赛黄金法则：提高算法和思考力的77项技巧

教材

科学出版社 · 2025

CSP-J CSP-S NOIP 体系学习 专题突破 深耕 方法论 纸质书

米田优峻著，朱全民译，2025 年 7 月第 1 版，ISBN 9787030820822，定价 128 元，中图法 TP311.1。作者即 AtCoder 上的 E869120，IOI 三届金牌（截至 2020 年），现就读东京大学；译者朱全民为中学正高级、特级教师，独立辅导学生获 IOI 金牌 6 枚、银牌 1 枚。全书十章：算法与时间复杂度、前缀和、二分查找、动态规划、数学问题、思维技巧、启发式、数据结构与查询处理、图算法、综合问题，共 77 个技巧、153 道原创例题、300 余幅全彩插图，书末附 20 道能力测试题。它真正的差异点不在覆盖面——这些知识点任何一本入门书都有——而在第 6 章「思维技巧」与第 7 章「启发式」：把「想奇偶性、算贡献、看上限、逆向、固定并枚举、重述问题、盯不变量」拆成可练习的条目，再补上贪心、局部搜索、模拟退火、集束搜索这套在多数国内教材里被跳过的「拿部分分」工具箱，正好对应 CSP-S 后两题与 NOIP 第三题的实战需求。定位要说清：深度止于提高组，线段树只到 RMQ/RSQ 两类，网络流只到 Dinic 与二分图匹配基础，LCT、树套树、生成函数一概没有，NOI 级读者会觉得浅；行文是日式教科书风格，插图密度高但公式推导少，习惯严谨证明的读者需另配资料。适合 CSP-J 转 CSP-S 的选手做「技巧自检」——按 77 条清单逐项打勾，缺哪条补哪条，比从头刷教材效率高。

https://book.cppinfo.cn/Encyclopedias/home/index?id=4871153

035 算法设计与分析（图解版）

教材

人民邮电出版社 · 2026

CSP-J CSP-S 入门自修 体系学习 速查 深耕 纸质书（302 页，ISBN 9787115692252）

陈小玉著，人民邮电出版社 2026 年 7 月第 1 版，302 页，ISBN 9787115692252。作者是高校副教授兼 ACM 教练、信息学奥赛教练（指导学生获 ACM 亚洲区金牌与蓝桥杯国际赛金牌），此前写过《算法训练营》系列，本书是同一作者面向课堂的「按策略组织」版本：不按数据结构分章，而按解题策略分章，全书 10 章依次是算法入门（复杂度、递推）、STL 常用容器与函数（vector/stack/queue/list/deque/priority_queue/map 与常用算法函数）、递归、贪心（含哈夫曼、Prim/Kruskal、Dijkstra/Floyd/Bellman-Ford/SPFA）、分治（二分查找的多种变体、二分答案、归并排序、复杂度求解的递推法与递归树）、动态规划（要素、爬楼梯、LIS、LCS 等）、回溯法、分支限界法、网络流算法、实用算法。它的差异点在「分支限界」这一章：国内 OI 教材普遍只讲回溯，把「限界函数 + 优先队列式扩展」当作 ACM 内容跳过，而 NOI 级搜索题里「最优解剪枝 + 优先扩展」正是 A*/IDA*（25.6）的前身，本书给了完整交代。写法是全图解 + 代码 + 例题，适合当「第一本按策略编排的教材」通读一遍建立全局观。局限：面向高校课程，例题多为教学例题而非真题，题量与难度都到不了 NOIP；网络流只到 Dinic 与最小费用流的基本形态，没有当前弧与多路增广的常数优化；分治一章的复杂度求解部分对竞赛帮助有限（考场直接用主定理结论即可）。定位：CSP-J 后期到 CSP-S 前期建立「策略—模型」映射的读物，之后仍需转 24—28 章的模板与真题。

https://www.tenlong.com.tw/products/9787115692252

036 聪明人的游戏：信息学探秘（提高篇）

教材

江涛、陈茂贤 · 机械工业出版社 · 2018

CSP-J CSP-S 入门自修 体系学习 深耕 纸质书

以「问题—思路—程序」三段式展开，例题生活化。作者是竞赛教练，知道初学者会在哪里卡住。

037 **计算概论：C++ 编程与信息学竞赛入门** 教材

金靖 · 高等教育出版社 · 2017

CSP-J 入门自修 深耕 纸质书

把大学「计算概论」的框架压进竞赛入门：计算机组成、进制、程序结构、基础算法都有交代。基础打得比同类书厚。

038 **高效通关信息学竞赛·初赛篇（入门组）** 教材

清华大学出版社 · 2026

CSP-J 赛前冲刺 查漏补缺 入门自修 速查 实战 纸质书

梁皓然、孙亮亮编著，2026 年 6 月第 1 版，ISBN 9787302715924，271 页，定价 89 元。市面上的算法书几乎都从「读懂题、写对代码」讲起，而 CSP-J 第一轮是 100 分笔试（计算机基础、进制与位运算、程序阅读、复杂度估算、填空），这一本补的正是这块空白：按题型切成单选题、程序阅读题、填空专项题三段分别讲，每类先做真题溯源再归纳高频考法，并逐条标注易错点；末附 10 套模拟赛及题解。三个用法：① 入门组考生在第一轮前 6—8 周按题型刷，比通读教材效率高得多——2026 年入门组全国线是 69.5/55.5/42，笔试不过就没有第二轮；② 教练直接拿它的题型分类做校内初赛讲座的提纲；③ 用 10 套模拟卷做赛前限时训练。局限要写清：只覆盖入门组，提高组第一轮仍需 CCF 笔试题库配合《提高篇》；271 页里 10 套卷占去大半，知识点讲解密度有限，不能当唯一教程，定位是「冲刺手册」而非「系统教材」。

https://wqbook.wqxuetang.com/book/3266087

039 **Algorithms Unlocked** 教材

Thomas H. Cormen · MIT Press · 2013

CSP-S NOIP 入门自修 体系学习 深耕 纸质书

CLRS 作者写的「轻量版」，无繁重证明，只讲直觉与过程。啃不动 CLRS 时的正确入口。

040 **Algorithms, 4th Edition (Sedgewick)** 教材

Robert Sedgewick, Kevin Wayne · Addison-Wesley · 2011

CSP-S NOIP 省选 体系学习 深耕 纸质书 / 配套网站免费

普林斯顿教材，配全套可视化、课件、测试数据。图论与字符串部分尤其扎实。

https://algs4.cs.princeton.edu/home/

041 **C++ Primer** 第 5 版 教材

Stanley B. Lippman 等 · 电子工业出版社 · 2013

CSP-S NOIP 查漏补缺 速查 纸质书

不是竞赛书，是语言工具书。STL 与模板那几章值得反复查。

042 **CCF CSP 例题解析与实战技巧：工程实现** 教材

机械工业出版社（中国计算机学会 组编） · 2026

CSP-S 专题突破 实战训练 实战 深耕 纸质书（69 元）

张锦（湖南师范大学信息科学与工程学院院长）、罗元盛（长沙理工大学）编著，总主编张孝，2026 年 7 月第 1 版，ISBN 978-7-111-81097-1（13 位 9787111810971），定价 69 元；页数各渠道标注不一（196—232 页），以实物为准。与同系列的《编程基础》《算法应用》合为三册，本册专攻认证第 3 题——那道题的特点是「描述长、逻辑碎、纯算法不难但实现容易崩」，官方定位是突破 300 分瓶颈。全书 5 章：前 4 章分别是基础知识、栈和队列、树和图、通用算法框架，每章走「专项知识点 + 典型真题」的结构；第 5 章整章是模拟题。真题覆盖 2014—2022 年。对 OI 选手的实际用处不在认证本身，而在两件事：一是「从冗长描述里抽出核心状态」的建模训练，这正是 CSP-S 第二题与 NOI 前两题失分的主因；二是把栈、队列、树、图这四类结构写成「拿来就能改」的工程模板，而不是竞赛里那种压行写法。适合作为专题突破的材料，按章刷完再回洛谷对应题单。局限：不含第 4 题的算法内容（线段树、图论建模、DP 优化都在《算法应用》里）；题面是认证背景，与 OI 题风格有距离；出版方与 CCF 均声明该丛书不面向中小学生。

https://www.tenlong.com.tw/products/9787111810971

043 **CCF 中学生计算机程序设计** 提高篇 教材

江涛、宋新波、朱全民 主编 · 科学普及出版社 · 2018

CSP-S NOIP 体系学习 专题突破 深耕 纸质书

提高组主线教材，图论与动态规划的写法偏规范，适合立规矩。

https://www.noi.cn/

044 **CCF 信息学奥赛提高篇** 教材

机械工业出版社（CCF 组编） · 2026

CSP-S NOIP 体系学习 专题突破 深耕 方法论 纸质书

朱全民主编，徐先友、汪星明、屈运华编写，2026 年 3 月第 1 版，ISBN 9787111803096。CCF 官方教程丛书第四册，官方口径覆盖 NOI 大纲难度系数 5—8 的知识点，正对 NOIP 与 CSP-S。内容按大纲条目组织：高级数据结构（线段树、树状数组、平衡树）、图论（最短路、网络流、二分图）、DP 进阶（树形、区间、数位）与数论组合。相比市面上按「算法名称」排的教材，它的价值在于章节与考纲一一对应，可用来查「我这个级别还缺哪块」。同为官方系列，《入门篇》面向 1—2 级，《专业篇》尚未出版。

https://www.noi.cn/cbw/2026-08-18/924758.shtml

045 **Competitive Programmer's Handbook** CPH 教材

Antti Laaksonen · 自出版（免费 PDF） · 2017

CSP-S NOIP 体系学习 查漏补缺 速查 深耕 官网免费下载

与 CSES 题库一一对应的免费英文书。篇幅短、密度高，英文阅读门槛低，强烈建议通读一遍。

https://cses.fi/book/book.pdf

046 **Competitive Programming in Python** 教材

Christoph Dürr、Jill-Jênn Vie · Cambridge · 2020

CSP-S 入门自修 深耕 纸质 / 电子书

用 Python 讲竞赛算法，适合作为第二语言参考。NOI 体系仍以 C++ 为主。

047 **Elements of Programming Interviews** 教材

A. Aziz, T.-H. Lee, A. Prakash · EPI · 2012

CSP-S NOIP 查漏补缺 实战 纸质书

面试向但题解极细，尤其数组、字符串、哈希部分。用来补「工程化写法」和边界处理习惯。

048 **Guide to Competitive Programming** 教材

Antti Laaksonen · Springer · 2020

CSP-S NOIP 体系学习 深耕 纸质 / 电子书

CPH 的扩展学术版，多了证明与复杂度分析，适合想把「为什么」也弄明白的人。

049 **Guide to Competitive Programming** 教材

Antti Laaksonen · Springer · 2017

CSP-S NOIP 省选 体系学习 专题突破 实战 纸质书 / 电子版

CPH 的扩展版，加入更多例题与实现细节。比 CPH 更适合当「边学边做」的主线教材。

050 **信息学奥赛一本通 · 提高篇** 教材

黄新军、董永建 · 科学技术文献出版社 · 2019

CSP-S NOIP 体系学习 专题突破 深耕 纸质书

一本通的提高组分册。OI Wiki 的建议是「选择性阅读」——书里覆盖面广但深度不均，按专题挑着补更划算。

051 **信息学奥赛之数学一本通** 教材

林厚从 · 东南大学出版社 · 2015

CSP-S NOIP 专题突破 深耕 纸质书

把竞赛里的数论、组合、概率集中讲一遍，数学薄弱者的补丁包。

052 **挑战程序设计竞赛** 第 2 版 教材

秋叶拓哉、岩田阳一、北川宜稔 · 人民邮电出版社 · 2013

CSP-S NOIP 省选 专题突破 实战 纸质书

日本 ACM 国家队的训练书，110 余道例题全部来自真实赛题，代码风格极干净。

053 **数据结构与算法分析** C 语言描述 教材

Mark Allen Weiss · 机械工业出版社 · 2016

CSP-S NOIP 体系学习 深耕 纸质书

复杂度分析讲得最扎实的一本。摊还分析那一章，能解释很多竞赛里的「为什么这么快」。

---

054 **数据结构（C++ 语言版）** 第 3 版 专著

邓俊辉 · 清华大学出版社 · 2020

CSP-S NOIP 省选 体系学习 深耕 纸质书 + 慕课

把平衡树、堆、图讲透的中文教材，配清华慕课，适合打地基。

https://www.xuetangx.com/

---

055 **程序设计竞赛进阶：算法与实践** 教材

人民邮电出版社 · 2026

CSP-S NOIP 省选 专题突破 实战训练 实战 深耕 纸质书（346 页，ISBN 9787115694058）

邱秋著，人民邮电出版社 2026 年 6 月第 1 版，346 页。作者此前写过入门向的《算法竞赛入门到进阶》（清华大学出版社 2019 年，ISBN 9787302529156），本书不是那本的改版：出版社、篇幅与章节结构全部重做，落点从「入门」移到「进阶」，因此两本可以并存而不重复。全书五章，按「搜索—图—图算法—动态规划—网格」组织：第 1 章回溯法（八皇后、单向与双向搜索、剪枝、正方形剖分、关灯问题、15 数码）；第 2 章图和图遍历（邻接矩阵/边列表/前向星/链式前向星的取舍，BFS/DFS 应用清单含连通性、最短路、最长简单路、着色、LCA、割顶割边、强连通与半连通分支、2-SAT、直径、重心，末接拓扑排序）；第 3 章图算法（欧拉回路、中国邮递员、哈密顿回路、TSP、Prim/Kruskal 与次小生成树、Dijkstra/Bellman-Ford/Floyd 与传递闭包、差分约束、第 K 短路、Ford-Fulkerson→EK→Dinic→ISAP 与最小割、最小费用流、二分图匹配四种算法（网络流、匈牙利、Hopcroft-Karp、Gale-Shapley）与一般图 Edmonds、KM 加权完备匹配、支配集/点覆盖/独立集/最大团、最小路径覆盖与最小边覆盖、树上最小支配/覆盖/独立）；第 4 章动态规划（背包四变体、备忘录与松弛两种视角、集合型/区间型/图论型/概率型/非典型、五种优化：空间、状态、二进制、单调队列、斜率、四边形不等式，末接收子序列子串七题与贪心）；第 5 章把网格类问题单独成章（矩形网格、三角形网格、六边形网格、经纬度）。真正稀缺的是第 3 章的覆盖面：把 Hopcroft-Karp、Gale-Shapley、Edmonds 带花树、KM 与四类图上集合问题放在同一章并各自给出适用条件，这种「一张图配一张算法选择表」的编排在国内教材里少见，正好补上第 28 章各节的横向对照。局限：写法偏经典教材体例，网络流与匹配的实现细节不如现代模板紧凑（缺当前弧优化与多路增广的说明），考场直接抄需要自行加常数优化；第 5 章网格专题对 OI 选手边际价值一般；全书无配套在线题库与测试数据。

https://thep.com.tw/book/1814500

---

056 **算法导论（核心篇）（原书第 4 版）** 专著

机械工业出版社 · 2026

CSP-S NOIP 省选 NOI 教练备课 查漏补缺 深耕 方法论 纸质书

Thomas H. Cormen、Charles E. Leiserson、Ronald L. Rivest、Clifford Stein 著，王刚、苏明译，2026 年 5 月第 1 版，ISBN 978-7-111-80002-6，139 元，正文 543 页、907 千字，中图法 TP301.6，属「计算机科学丛书」。这一版中文版做了一次拆册：核心篇收原书第 1—25 章（基础知识、排序与顺序统计量、数据结构、高级设计与分析技术、高级数据结构、图算法六部分），同批另有《算法导论（扩展篇）（原书第 4 版）》（ISBN 978-7-111-80012-5，99 元）收第 26—35 章的专题（含多线程算法、矩阵运算、线性规划、FFT、数论算法、字符串匹配、计算几何、NP 完全性、近似算法）。与库里已有的第 3 版相比，4 版的变化集中在：用「归并排序」而非「插入排序」作第一种算法引入、新增「摊还分析」的独立处理、散列表改用「全域哈希 + 开放寻址」为主线、图算法里删掉了斐波那契堆的部分推导，这几点对竞赛选手影响不大；真正值钱的是它把「平均情况分析」「摊还界」「下界证明」这三套论证范式写透了——竞赛里判断「这个贪心为什么对」「这个势能函数怎么构造」，本质就是摊还分析。用法建议：不要通读。按 29 章（数论）、30 章（字符串匹配）、31 章（计算几何）的对应小节查证结论，或在写集训队论文、准备命题时引用其证明框架；书里的伪代码可直接对照本编第 22—31 章的实现。局限：与竞赛的距离仍然远——没有线段树合并、没有 LCT、没有后缀自动机、没有网络流的 Dinic 实现，NP 完全性那部分讲的是理论可归约性而非考场上的状压与搜索剪枝；印刷体量大、单价高，对 CSP-J/S 段选手性价比很低。

http://www.cmpedu.com/books/book/5610910.htm

057 **算法思维训练：信奥大师的算法教与学秘籍** 专著

人民邮电出版社（Michal Forišek、Monika Steinová 著） · 2025

CSP-S NOIP 省选 教练备课 体系学习 方法论 纸质书（95 页，ISBN 9787115680044）

斯洛伐克 Michal Forišek 与瑞士 Monika Steinová 合著，人民邮电出版社 2025 年 9 月第 1 版，95 页。作者身份是这本书的全部理由：Forišek 是多届 IOI 科学委员会成员、IOI 命题人，并三届担任中欧信息学奥赛（CEOI）主要组织者——他写的不是「怎么解题」，而是「一个概念怎么被讲明白」。全书体例统一：每个主题分「概述—比喻—分析—经验—习题」五段，比喻是核心，用来把算法的结构映射到学生已有的直觉上；第 1 章专门讨论「比喻作为教学工具」的用法与风险（类比在什么环节成立、在什么环节必须撤掉），这在竞赛教材里几乎是空白。正文主题不多但都是硬骨头：图算法的整体框架、树中的最长路径（直径的两遍 DFS/树形 DP，以及「为什么两遍扫是对的」）、带障碍物的最短路（网格/几何障碍下的绕行）、线段之间的距离、环绕数（绕数的计算与用它判定点是否在多边形内、判定两线段是否相交）。对教练的实际用途是拿去改自己的讲义：同一批结论国内教材通常直接给证明与模板，这本书给的是「先给什么样的图、先问什么样的问题才能让学生自己把结论逼出来」，与 ISSEP 2026 论文集里 Hromkovič 的「基因苏格拉底法」是同一路数（见本目录 ISSEP 2026 条目）。局限必须写清：只有 95 页、5 个主题，不构成体系，不能当教材通读；没有一道 OI 真题，也不能替代题解；写作对象是「要上算法课的教师与师范生」，学生自己读会觉得抽象。定位建议：与《算法竞赛核心教程》这类体系书互补——体系书解决「教什么」，这本解决「怎么讲」。

https://www.tenlong.com.tw/products/9787115680044

058 算法竞赛实战笔记

教材

电子工业出版社 · 2024

CSP-S NOIP 体系学习 实战训练 实战 纸质书

梁博、李睿琦、娄兰编著，2024 年 1 月第 1 版，ISBN 9787121470127，16 开 244 页、34.1 万字，定价 78 元，中图法 TP301.6。作者梁博 2019 年起负责北大附中初中信息学奥赛教学，书里的题单与讲法直接来自这条教学线，所以体例与常见「知识点罗列型」教材不同：全书以洛谷题号为骨架，讲线性数据结构、基础算法、搜索、动态规划四块的「考场上怎么想到」，每个知识点先摆一道能暴露问题的例题，再从例题里抽方法，最后给同型题单。对已经过了语法关、却卡在「会模板不会选题」的 CSP-S/NOIP 段选手最有用——这个阶段缺的往往不是第五种线段树写法，而是「看到题判断该归到哪一类」的映射关系。定位要说清：它是教学笔记，不是手册，查模板不如《算法竞赛进阶指南》顺手；覆盖止于 DP 与基础数据结构，网络流、字符串、计算几何、数论进阶均未涉及，不能当唯一教材。另外出版于 2024 年初，题面与代码基于当时的洛谷题库，个别题号与难度标签此后有调整，按题号找题时以站内现状为准；配套评测依赖洛谷，无独立题库。

https://cbjj.phei.com.cn/module/goods/wssd_content_comment.jsp?bookid=64333

059 算法竞赛核心教程：策略与算法深度解析

教材

科学出版社（Antti Laaksonen 著，陈锋 译） · 2025

CSP-S NOIP 省选 体系学习 专题突破 深耕 方法论 纸质书（335 页，ISBN 9787030816443）

芬兰赫尔辛基大学 Antti Laaksonen 所著《Guide to Competitive Programming》第 2 版的中文译本，科学出版社 2025 年 3 月出版，335 页，陈锋译。原书是 CSES Problem Set 的官方配套读物，也是目前唯一一本「按 IOI 大纲组织、同时给出完整代码与复杂度分析」的竞赛教材，中文版的落地补上了国内读者长期只能读英文版的缺口。全书 18 章，顺序即学习顺序：编程技巧（递归、位运算）→ 算法效率（复杂度与代码优化）→ 排序与二分 → 数据结构（动态数组、集合结构、实验法测常数）→ 动态规划 → 图论算法（遍历、最短路、DAG、后继图、MST）→ 算法设计专题（位并行、均摊分析、查找最小值）→ 区间查询（稀疏表、线段树、树状数组）→ 树上算法（基本技术、树上查询、高级技术）→ 数学专题（数论、组合、矩阵、概率、博弈、FFT、猜公式）→ 高级图算法（强连通、完整路径、最大流、DFS 树、费用流）→ 计算几何 → 字符串（哈希、Z 算法、后缀数组、自动机）→ 附加主题（根号分治、线段树再探、Treaps、DP 优化、回溯、杂项）→ Python 在竞赛中的应用 → 如何准备 IOI → 算法竞赛的未来（含生成式 AI 一节）。三处别处少见的取舍值得注意：一是每章都给出对应的 CSES 题号，读完立刻有题可做，不必自己找配套；二是专门用一章讲「实验法」——用计时与计数实测常数、判断 O(n log n) 是否真能过，这正是第 23 章反复强调却少有教材落笔的方法论；三是把 Treaps、根号分治、DP 优化这类「只在论坛和博客里流传」的技巧收进了正文，条目化程度高于同类教材。局限：中文版比英文第 2 版晚约一年，个别表述沿用第 1 版的例子；全书篇幅只有 335 页，每个专题都给到「能上手」的深度而非「能打 NOI 压轴」的深度，联赛以上的题目仍需配集训队论文与专题讲义；代码为 C++ 简洁风格，未针对卡常优化。

https://www.tenlong.com.tw/products/9787030816443

060 算法竞赛进阶指南

教材

李煜东 · 河南电子音像出版社 · 2018

CSP-S NOIP 省选 专题突破 体系学习 深耕 纸质书 / 配套代码仓库

按专题组织，中文圈使用率最高的提高组教材。零基础不建议直接上，学完紫书再来。

061 **算法训练营：进阶篇（全彩版）** 教材

电子工业出版社 · 2025

CSP-S NOIP 省选 NOI 体系学习 专题突破 深耕 实战 纸质书

陈小玉著，2025 年 4 月第 1 版，ISBN 9787121498848，16 开、279 页（部分书目记 288 页），定价 128 元，全彩印刷。它是 2021 年《算法训练营：海量图解+竞赛刷题（进阶篇）》的重绘重编版——图片全部重画配色、内容拆分重组，并新增了可持久化数据结构一章。全书 8 章：①分块算法与跳跃表；②AC 自动机与后缀数组（基数排序、SA 详解、SA 的应用）；③树上操作（树链剖分 + 线段树、点分治、边分治与「重建树」）；④复杂树（KD 树、左偏树、动态树 LCT、树套树）；⑤可持久化（线段树、字典树）；⑥网络流（EK、Dinic、ISAP、二分图匹配、最大流最小割、最小费用最大流）；⑦DP 进阶（背包进阶、树形 DP 进阶）；⑧复杂 DP（数位 DP、插头 DP、斜率优化、四边形不等式优化）。每个知识点后固定跟「训练 1 / 训练 2」两道题，属于「讲一段、写一段」的强绑定结构。差异点要说清：它的取材几乎逐条对上第 27、28、30 章的知识点清单，LCT、树套树、边分治、ISAP 这些在国产教材里常被跳过的项它都有，是少见的「按省选大纲铺满、而不是按作者熟悉度取舍」的进阶书。局限同样明确：279 页塞 8 个大专题，平均每专题 35 页，LCT 只到 access/makeroot/split/link/cut 的基本操作，不含虚子树维护与维护子树信息；树套树给的是线段树套平衡树与线段树套线段树，没有 BIT 套动态开点；可持久化只到线段树与字典树，没有可持久化并查集/平衡树；网络流讲了三种增广算法但没讲当前弧与容量缩放等常数优化。代码风格偏教学，用结构体 + 大量图解，常数不及竞赛模板，抄下来要自己压。定位：CSP-S 冲省选的选手拿它当「考点清单 + 题单」，读完一章立刻在该章两道训练题上验收；教练可取其章节顺序做专题课排程。已有《算法竞赛进阶指南》的读者不必重复购买，两书重叠度高，进阶指南的证明更严、这本的图更直观。

https://product.dangdang.com/11927255922.html

062 **算法设计与分析** 教材

王晓东 · 清华大学出版社 · 2018

CSP-S NOIP 体系学习 深耕 纸质书

国内高校通用教材，分治、贪心、DP、回溯的框架清晰，理论作业多。

063 **算法设计与分析：以 ACM 大学生程序设计竞赛在线题库为例（第 2 版·微课版）** 教材

清华大学出版社 · 2025

CSP-S NOIP 省选 体系学习 专题突破 实战 深耕 纸质书（ISBN 9787302696490）

赵端阳著，清华大学出版社 2025 年 8 月第 2 版（第 1 版 2021 年，ISBN 9787302587255），浙江省「十二五」优秀教材、省「十三五」首批新形态教材，作者任大学生程序设计竞赛教练 15 年。体例是「结论 + 模板 + 原题」：11 章按算法策略推进（算法概述与复杂度 → 数据结构与 STL → 递归与分治 → 动态规划 → 贪心 → 回溯 → 分支限界 → 图的搜索 → 图论 → 数论 → 组合数学），每节后面直接挂原题，题源同时取自洛谷（P1182 数列分段、P1908 逆序对、P2016 战略游戏、P1352 没有上司的舞会、P2014 选课等）与 ZOJ/POJ/HDU，等于把「模板—真题」的对应关系写死在书里。真正稀缺的是第 9 章图论的密度：网络流从 Ford-Fulkerson 讲到 EK、ISAP、Dinic 三种实现并各自在同一道题（ZOJ1734 Power Network）上对比，再接到最小费用流（SPFA 增广）；二分图匹配给出匈牙利、Hopcroft-Karp、KM 三套并各配两题——这种「同一问题、三种实现、同一组数据」的排法，正好补上 28.6/28.15/28.16 各节之间的横向对照。第 11 章组合数学把母函数（普通型/指数型）、Stirling 数、Catalan 数、容斥与错排、鸽巢原理收在一章，与 29.3/29.10 呼应。局限：作为教材，某些实现比现行竞赛模板旧（缺当前弧优化的写法、费用流没有势能 Dijkstra 版本），考场直接用需自行加常数优化；分支限界一章偏课程作业取向；第 2 版主要补微课视频与部分例题，核心结构未大改，与第 1 版重合度高，已有旧版不必重买。

https://www.tenlong.com.tw/products/9787302696490

064 **算法（第 4 版）** 教材

Robert Sedgewick、Kevin Wayne · 人民邮电出版社 · 2012

CSP-S NOIP 体系学习 深耕 纸质书

普林斯顿的算法教材，图与排序讲得极好，配 Coursera 课程。代码是 Java，思路通用。

065 **计算思维训练——动态规划** 教材

东南大学出版社 · 2026

CSP-S NOIP 专题突破 教练备课 深耕 方法论 纸质书

史钋镭等著，2026 年 1 月第 1 版，ISBN 9787576625110，定价 56 元，中图法 TP311.1-49，读者对象标注为青少年，属东南大学出版社面向中学信息学教学的一条产品线。它的组织方式是「按模型切章、按优化收口」，而不是按难度递进：内容提要明确列出线性 DP、状态压缩 DP、区间 DP、树形 DP、背包 DP、数位 DP、插头 DP、计数 DP、动态 DP、概率 DP 十类模型，优化方法则点名线段树、单调队列、斜率优化三种，并对历年 CSP/NOIP 中的 DP 题做了分类归纳，每章配知识点讲解、例题与习题解析。这个组合在同类书里不常见——多数 DP 专题书只覆盖到背包、区间、树形、状压四类，插头 DP、动态 DP、概率 DP 通常散落在集训队论文或博客里，能被编进中学实训教材说明编写组按「考点清单」而非「教学舒适区」选材。选型提醒要写清楚：十类模型压缩在一本 56 元的册子里，单类的篇幅必然有限，插头 DP、动态 DP 这类重题更可能是「给框架 + 一两道例题」而不是完整体系，指望一本书打通动态 DP 的选手仍需配合 OI Wiki 与集训队论文；反过来，它的价值在横向覆盖与真题归类——拿它当「DP 考点地图」，逐章检查自己哪类模型没写过，比从头啃一本 800 页的大部头效率高。适用对象是已能独立写背包与树形 DP、准备把 DP 从「会写模板」推进到「认得模型」的 CSP-S/NOIP 选手；教练也可取其分类框架做专题课的排课依据。

https://book.cppinfo.cn/Encyclopedias/home/index?id=4992531

066 **ACM 国际大学生程序设计竞赛系列·知识与入门** 教材

俞勇 主编 · 上海交通大学出版社 · 2012

NOIP 省选 体系学习 教练备课 深耕 纸质书

上交 ACM 队教材的第一册，讲题型分类与训练方法。面向 ICPC，但知识结构与省选高度重合。

067 **Algorithms（Dasgupta–Papadimitriou–Vazirani）** 教材

S. Dasgupta, C. Papadimitriou, U. Vazirani · McGraw-Hill · 2006

NOIP 省选 NOI 体系学习 深耕 免费（作者授权 PDF）

薄、狠、准。用 300 页讲完别人 1000 页的内容，适合已有基础后「重构知识框架」。

https://people.eecs.berkeley.edu/~vazirani/algorithms/

068 **Algorithms（Jeff Erickson）** 教材

Jeff Erickson · University of Illinois · 2019

NOIP 省选 NOI IOI 体系学习 专题突破 深耕 免费（作者官网 PDF）

目前最好的免费算法教材，证明写得极清楚，习题质量高且带提示。英文过关就该读这一本。

https://jeffe.cs.illinois.edu/teaching/algorithms/

069 **C++，挑战编程——程序设计竞赛进阶训练指南** 教材

邱秋 · 人民邮电出版社 · 2019

NOIP 省选 专题突破 查漏补缺 实战 纸质书 / 电子版

以 C++ 实现为主线的竞赛进阶书，代码风格现代（C++11 起）。适合从「能写」过渡到「写得干净」。

070 **Competitive Programming 4（配套代码库）** 教材

Steven & Felix Halim · GitHub · 2020

NOIP 省选 NOI 查漏补缺 实战训练 速查 免费（开源）

CP4 全书代码，按章节组织、可直接编译。抄模板前先读一遍它的注释，能少踩很多坑。

https://github.com/stevenhalim/cpbook-code

071 **Mathematics for Computer Science** 教材

E. Lehman, T. Leighton, A. Meyer · MIT 6.042J · 2018

NOIP 省选 NOI 体系学习 深耕 免费（MIT OCW）

离散数学的完整讲义，证明、计数、概率、图论全覆盖。NOI 级数学短板的系统补法。

https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/

072 **Programming Pearls** 专著

Jon Bentley · Addison-Wesley · 1986

NOIP 省选 NOI 专题突破 方法论 纸质书

讲「如何把一个问题想透」。位图、前缀和、二分答案的思想源头都在这本书里，薄但极硬。

073 **算法详解（Algorithms Illuminated，卷 1—4）** 教材

Tim Roughgarden · Cambridge University Press / 人民邮电出版社（中译） · 2017

NOIP 省选 NOI 体系学习 深耕 纸质书 / 部分章节公开

四卷本：基础、图算法、贪心与 DP、NP 完全性。作者是算法课名师，讲直觉不堆公式，配 Coursera 课程。

https://www.algorithmsilluminated.org/

074 **编程珠玑** 第 2 版 专著

Jon Bentley · 人民邮电出版社 · 2008

NOIP 省选 专题突破 方法论 纸质书

讲的不是算法而是「怎么想问题」。位图排序、二分查找的边界处理，至今仍是好教材。

075 **ACM 国际大学生程序设计竞赛系列 · 算法与实现** 教材

俞勇 主编 · 上海交通大学出版社 · 2012

省选 NOI 专题突破 深耕 纸质书

第二册讲实现：数据结构、图论、计算几何的代码级细节。适合「知道思路但写不出来」的阶段。

076 **Algorithm Design（Kleinberg–Tardos）** 教材

Jon Kleinberg, Éva Tardos · Addison-Wesley · 2005

省选 NOI IOI 专题突破 教练备课 深耕 纸质书 / 课件公开

讲「怎么想出算法」而不是「算法是什么」。贪心与网络流的正确性证明讲得最深，写论文前必读。

https://www.cs.princeton.edu/~wayne/kleinberg-tardos/

077 **Competitive Programming 4** 红皮书 教材

Steven Halim、Felix Halim · 自出版 · 2020

省选 NOI IOI 专题突破 教练备课 深耕 实战 纸质 / 官网

收录近千道真题、配 UVA 题号，国际化训练的标准参考。上下两册，重得像砖头。

https://cpbook.net/

078 **Introduction to Graph Theory（West）** 教材

Douglas B. West · Prentice Hall · 2000

省选 NOI IOI 专题突破 深耕 纸质书

图论标准教材，习题极多且有难度分级。想系统补图论（匹配、连通度、着色）用它。

079 **Introduction to Probability** 教材

D. Bertsekas, J. Tsitsiklis · Athena Scientific · 2008

省选 NOI 专题突破 深耕 纸质书 / MIT 课件

概率论入门，例子偏工程但推导完整。期望 DP、随机化算法的理论基础在这里。

https://ocw.mit.edu/courses/6-041-probabilistic-systems-analysis-and-applied-probability-fall-2010/

080 **The Algorithm Design Manual** 教材

Steven Skiena · Springer · 1997

省选 NOI IOI 专题突破 查漏补缺 深耕 纸质书 / 部分章节公开

后半本「问题目录」按场景索引算法，是全书写法里最实用的一本。卡在「这题该用什么」时翻它。

https://www.algorist.com/

081 **信息学竞赛中的数学** 教材

多位竞赛教练合著 · 各出版社 · 2018

省选 NOI 专题突破 深耕 纸质书 / 讲义

同类书较多，选择时优先看是否配有真题，纯数学教材容易与考点脱节。

082 **具体数学** 计算机科学基础（第 2 版） 专著

Graham、Knuth、Patashnik · 人民邮电出版社 · 2013

省选 NOI 专题突破 深耕 纸质书

数论与组合的补课神器。求和技巧、递推、生成函数几章直接对应竞赛考点。

083 **挑战程序设计竞赛 2** 算法与数据结构 教材

渡部正树 等 · 人民邮电出版社 · 2016

省选 NOI 专题突破 深耕 纸质书

续卷偏数据结构与复杂算法，难度陡增，是省选阶段的推土机。

084 **数据结构与算法（上下册）** 教材

清华大学出版社 · 2026

省选 NOI 体系学习 教练备课 深耕 纸质书（上下册，定价 99 元）

邓俊辉编著，2026 年 10 月第 1 版（10 月 13 日付印），ISBN 9787302729150，定价 99 元，清华大学计算机系列教材。全书 14 章分上下两册：上册 6 章为算法概论、向量、列表、栈与队列、二叉树、二叉搜索树；下册 8 章为搜索树应用、高级搜索树、词典、图基础、图进阶、优先级队列、字符串、排序。作者在清华讲授算法类课程三十年，其中「数据结构」是清华首批标杆课程，配套慕课「数据结构」与「计算几何」累计有 160 多个国家和地区的 100 余万人选修。对竞赛选手的价值不在「多学一个算法」，而在补上竞赛教材普遍跳过的那一层：为什么这样设计、摊还代价怎么算、接口与实现的取舍在哪里——向量扩容的均摊分析、二叉搜索树的旋转不变式、优先级队列的合并代价，这些正是读懂左偏树、可持久化结构与树套树时缺的那块地基。定位要说清：它不是竞赛书，没有 OI 真题，代码是 C++ 模板类而不是竞赛式的数组模拟，直接抄不能当模板背；正确用法是当「工具书 + 证明册」，在学完一章竞赛内容后回来看对应章节的设计动机与复杂度证明。上册偏基础（CSP-S 段可读），下册的高级搜索树、图进阶、字符串才是省选以上真正有用的部分。局限：两册 99 元、篇幅大，通读成本高；不含竞赛评测环境与题目数据，与 NOIP/NOI 大纲不对齐，不能替代按大纲的复习。

https://www.tup.tsinghua.edu.cn/booksCenter/book_11860701.html

085 **离散数学及其应用** 第 8 版 专著

Kenneth H. Rosen · 机械工业出版社 · 2019

省选 NOI 查漏补缺 深耕 纸质书

图论、逻辑、计数、关系四大块与竞赛重叠度高，当工具书按需查。

086 **算法导论** CLRS 第 3 版 专著

Cormen、Leiserson、Rivest、Stein · 机械工业出版社 · 2012

省选 NOI IOI 查漏补缺 深耕 纸质书

理论圣经，但不是竞赛书。只在某个算法「知其然不知其所以然」时按章查阅，别通读。

087 **算法导论（扩展篇）（原书第 4 版）** 教材

机械工业出版社 · 2026

省选 NOI 查漏补缺 体系学习 深耕 纸质书（定价 99 元，331 页）

科尔曼、雷瑟尔森、李维斯特、斯坦著，王刚、苏明译，2026 年 5 月 29 日第 1 版，ISBN 978-7-111-80012-5，定价 99 元，561 千字，331 页，计算机科学丛书。第 4 版中译本拆成两册，已入库的《核心篇》是第 1—25 章，本册是第 26—35 章：并行算法（fork-join、并行矩阵乘法、并行归并排序）、在线算法（等待电梯、维护搜索链表、确定性/随机化缓存）、矩阵运算（解线性方程组、矩阵求逆、对称正定与最小二乘）、线性规划（形式化与对偶性）、多项式与离散傅里叶变换（DFT/FFT）、数论算法（gcd、模运算、模线性方程、中国剩余定理、元素幂、RSA、素性测试）、字符串匹配（朴素、Rabin-Karp、有限自动机、KMP、后缀数组）、机器学习算法（聚类、乘法权重、梯度下降）、NP 完全性（含团、顶点覆盖、哈密顿回路、TSP、子集和六个经典归约）、近似算法（顶点覆盖、TSP、集合覆盖、随机化与线性规划、子集和）。附录 A—D 为求和、离散数学、计数与概率、矩阵。对竞赛的直接价值集中在三章：第 30 章是 NTT 的数学来源（单位根、DFT 的矩阵形式、FFT 分治结构），第 31 章把 exgcd、CRT、Miller-Rabin 的证明一次讲透，第 32 章给 KMP 与后缀数组的严格正确性论证——这些正是抄模板时最容易含糊过去的部分。第 26、27、33、35 章超出 NOI 大纲，属视野内容，不必精读。局限：全书伪代码（无 C++ 实现），无习题答案，不能当模板书用；与《核心篇》合计 238 元、篇幅近千页，通读成本高，正确用法是当「证明册」按需查证，而不是从头读到尾。

http://www.cmpedu.com/books/book/5610909.htm

088 **算法竞赛中的数论** 教材

多位竞赛教练 · 各出版社 / 讲义 · 2019

省选 NOI 专题突破 深耕 纸质 / 讲义

同主题书较多，选书标准只有一个：是否配真题。纯数学教材容易与考点脱节。

089 **算法艺术与信息学竞赛** 教材

刘汝佳、黄亮 · 清华大学出版社 · 2004

省选 NOI 专题突破 深耕 纸质书 / 二手

上一代竞赛生的「红宝书」。内容偏老，但建模思路与经典套路至今不过时。

090 **算法设计手册** The Algorithm Design Manual 教材

Steven S. Skiena · Springer · 2020

省选 NOI 查漏补缺 速查 方法论 纸质 / 电子书

后半本是「问题 → 算法」的检索目录，卡题时翻它比搜网页快。

091 **组合数学** 第 5 版 专著

Richard A. Brualdi · 机械工业出版社 · 2012

省选 NOI 专题突破 深耕 纸质书

计数问题的标准教材，容斥、Polya、递推关系讲得比竞赛书系统。

092 **Graph Theory（Bondy & Murty）** 教材

J. A. Bondy, U. S. R. Murty · Springer GTM 244 · 2008

NOI IOI 专题突破 深耕 免费（作者授权 PDF）

研究生级图论，证明严谨。做图论方向论文（集训队论文）时的标准参考。

https://www.zib.de/groetschel/teaching/WS1314/BondyMurtyGTWA.pdf

093 **Handbook of Data Structures and Applications（2nd Edition）** 专著

Dinesh P. Mehta, Sartaj Sahni 等 · Chapman & Hall/CRC · 2018

NOI IOI 专题突破 教练备课 深耕 纸质书 / 图书馆

数据结构的工具书级手册，覆盖范围远超竞赛需要。写集训队论文找「已知最优复杂度」时的权威出处。

094 **Linear Algebra Done Right** 教材

Sheldon Axler · Springer · 2015

NOI IOI 专题突破 深耕 免费（作者官网）

不靠行列式讲线性代数，直接讲算子与结构。理解矩阵加速、线性基、高斯消元的本质靠它。

https://linear.axler.net/

095 **Structure and Interpretation of Computer Programs** 教材

H. Abelson, G. Sussman · MIT Press · 1985

NOI IOI 体系学习 深耕 免费（在线全文）

讲抽象与递归的本质。不直接提分，但会改变你写代码和拆问题的姿势，很多金牌选手的共同读物。

https://mitpress.mit.edu/sites/default/files/sicp/full-text/book/book.html

096 **概率论与数理统计** 专著

盛骤 等 · 高等教育出版社 · 2019

NOI 专题突破 深耕 纸质书

应对期望、方差、条件概率类考点。只需前四章，配合竞赛题练习即可。

097 **线性代数及其应用** 专著

David C. Lay · 机械工业出版社 · 2018

NOI 专题突破 深耕 纸质书

2025 大纲在 NOI 级新增「基与线性基」，这本是补齐矩阵与线性空间直觉的最短路径。

098 **编程原本** 专著

Alexander Stepanov、Paul McJones · 机械工业出版社 · 2012

NOI 查漏补缺 方法论 纸质书

从数学结构角度讲算法为何成立。读不下去很正常，读懂一章就够用很久。

099 **计算几何：算法与应用** 第 3 版 专著

Mark de Berg 等 · 清华大学出版社 · 2018

NOI 专题突破 深耕 纸质书

计算几何唯一系统的中文译著。竞赛只需前几章：凸包、半平面交、最近点对。

100 **ACM Transactions on Algorithms（TALG）** 专著

ACM · ACM · 2005

IOI 专题突破 深耕 付费（学校可订阅）

理论算法最高级别期刊之一。集训队论文若要引用「已知最优复杂度」，这里是权威出处。

https://dl.acm.org/journal/talg

101 **Algorithmica** 专著

Springer · Springer · 1986

IOI 专题突破 深耕 付费（学校可订阅）

算法领域顶级期刊之一，偏理论与应用结合。写论文找参考文献的起点。

https://link.springer.com/journal/453

102 **Journal of Graph Algorithms and Applications（JGAA）** 专著

Brown University · jgaa.info · 1997

IOI 专题突破 深耕 免费（开放获取）

图算法开放期刊，可免费下载全文。图论方向论文的高质量参考源。

https://jgaa.info/

103 **计算机程序设计艺术** TAOCP 专著

Donald E. Knuth · 国防工业出版社 · 2002

IOI 查漏补缺 深耕 纸质书

不是给选手读的，是给「想知道所有细节的人」读的。当作查询手册，别当教材。

06 第 6 章　官方文档与竞赛规则

> 官方文件的特点是不好看但必须看。大纲决定你学什么，规则决定你能不能参赛、成绩怎么算、申诉走什么流程。每年都有人因为没读规则而丢分甚至取消资格。

本章收录 60 条，编号 104—163。

104 CCF NOI 系列活动考场纪律

官方

CCF · noi.cn · 2025

通用 赛前冲刺 速查 免费

哪些行为算作弊、代码雷同如何判定。每年都有选手因不了解规则被取消成绩。

https://www.noi.cn/gynoi/tlgd/2025-09-01/848189.shtml

105 NOI 大纲（2025 年修订版）

官方

CCF NOI 科学委员会 · 中国计算机学会 · 2025

通用 教练备课 查漏补缺 体系学习 速查 方法论 官网免费 PDF

全书的骨架。1 – 10 级共数百个知识点逐条标注级别，任何资料都可以挂到这套编号上。

https://www.noi.cn/upload/resources/file/2025/04/18/NOI_Syllabus_Edition_2025.pdf

106 NOI 官网

官方

中国计算机学会 · noi.cn · 1995

通用 教练备课 查漏补缺 速查 方法论 免费

大纲、赛历、规则、获奖名单、题目分析报告的唯一权威来源。所有二手信息都要回到这里核对。

https://www.noi.cn/

107 NOI 系列活动和认证日历

官方

CCF · noi.cn · 2026

通用 教练备课 速查 免费

2026 年 2 月 12 日发布的全年赛历，正文只有一句「以上活动时间安排如有调整和补充，请以具体通知为准」，全年安排以图片形式给出；结合各单项通知可还原为：CSP-J/S 第一轮 9 月 19 日、第二轮 10 月 31 日，NOIP 11 月 28 日 8:30—13:00，NOI 2027 联合省选 2027 年 3 月 6—7 日，冬令营 2 月、APIO 5 月、NOI 7 月、IOI 8 月，女生竞赛与教师培训穿插其间。对教练的用法是按表倒排训练节奏：CSP 两轮之间只有六周、第二轮结束到 NOIP 只有四周，秋季学期实际上是一段连续冲刺，暑假结束前必须把大纲主干讲完。局限：日历本身是图片，具体日期一律以单项通知为准（集训营报到、各省报名截止都另有规定），且部分活动标注「待定」，年中会调整。

https://www.noi.cn/xw/2026-02-12/861384.shtml

108 《中国计算机学会通讯》（CCCF）

官方

中国计算机学会 · CCF · 2005

通用 教练备课 方法论 会员赠阅 / 官网部分公开

CCF 会刊，常刊发 NOI 工作总结、命题思路、拔尖人才培养讨论。教练写方案时的政策依据来源。

https://www.ccf.org.cn/

109 《信息学奥林匹克竞赛年鉴》

官方

中国计算机学会 · CCF · 2010

通用 教练备课 方法论 纸质 / 内部资料

各省参赛与获奖数据、优秀组织经验。做学校竞赛规划时用来对标同层次学校的成绩。

https://www.noi.cn/

110 2024 年 NOI 活动题目分析报告（CSP-J/S 2024 第二轮 + NOIP 2024 知识构成）

官方

中国计算机学会 · 2025

CSP-J CSP-S NOIP 查漏补缺 教练备课 方法论 深耕 免费

2025 年 6 月 16 日发布，含三份独立 PDF：CSP-J/S 2024 第二轮入门级、CSP-J/S 2024 第二轮提高级、NOIP 2024 的「题目知识构成分析报告」，NOI 官网在线预览可直接打开。执笔人为 NOI 科学委员会团队：入门级桂欣、胡冬明、胡昌腾、叶金毅、杨耀良、任舍予、赵启阳、韩文弢；提高级张炜其、汪星明、杨耀良、任舍予、赵启阳、韩文弢；NOIP 李建、杨耀良、任舍予、赵启阳、韩文弢。与市面上逐题题解不同，这套报告按《NOI 大纲》的知识点给每道题做归类与难度标定，因此能回答「一场比赛真正考了哪几个板块、各占多少分」这类统计问题，而这正是教练排训练计划时最缺的数据——把三届报告叠起来看，就能发现某板块（例如 DP 优化、树上问题）的复现频率，从而决定课时配比。用法建议：先读提高级一份建立框架，再用入门级校准低年级段，NOIP 一份用于衔接省选。局限：只覆盖 2024 赛季，2025 年同类报告见既有条目；报告是赛后复盘性质，不含代码与实现细节，配题解使用。

https://www.noi.cn/zxzy/lnzl/jszl/2025-06-16/844463.shtml

111 2026 年 WCET & LMCC & NOI 教师发展活动日历

官方

中国计算机学会（NOI 官网） · 2026

CSP-J NOIP 教练备课 速查 免费（官网公告）

2026 年 3 月 10 日发布，一张全年教师发展活动的排期表，分两块。WCET（计算机教育领域）名师论坛六场：3 月丽江上海新纪元学校、5 月新疆、8 月西安市铁一中学（年度研讨会）、9 月北京市第八十中学、11 月澳门、12 月广西。教师培训六场：1 月上海外国语大学附属外国语学校（LMCC）、5 月北京市第八十中学与佛山市南海区石门中学、9 月浙江、11 月江苏、广东待定。对教练的实际用途有二：一是按月份规划自己的进修与参赛带队节奏，避免与本校教学冲突；二是有意承办的学校可据此提前一年接触特派员——承办校名单是公开的，说明这条路是常态化开放的。局限必须说清：只有时间、名称与承办单位，没有课程主题、学时、报名条件与名额；「待定」一栏说明排期本身会调整，实际以每次单独通知为准（例如 12 月南京站的第四期培训就不在这张表内，是后补的）。

https://www.noi.cn/jspx/2026-03-10/861585.shtml

112 **CCF CSP 认证官方说明** 官方

CCF · cspro.org · 2014

CSP-J CSP-S 教练备课 赛前冲刺 速查 免费

CSP 软件能力认证（与 CSP-J/S 不同）的官方规则。若学校需要第三方能力证明，这是正规渠道。

https://cspro.org/

113 **CCF CSP 认证官网** 官方

中国计算机学会 · cspro.org · 2019

CSP-J CSP-S 赛前冲刺 速查 免费

报名、准考证、成绩查询入口。别在非官方渠道报名。

https://www.cspro.org/

114 **CCF NOI 教师培训考核管理规则** 官方

中国计算机学会（NOI 官网） · 2026

CSP-J NOIP NOI 教练备课 速查 免费（官网公告）

2026 年 7 月 13 日发布。这份公告把 NOI 教师培训与资质认定从「通知式」变成了「制度式」，给出三件对教练直接有用的事。①分级口径：培训与考核一级一档，初级对应大纲难度系数 1~5、中级 1~7、高级 1~9——换句话说，初级证书的实际覆盖面只到 CSP-S 中段，想覆盖省选就得考中级，覆盖 NOI 要考高级。②考核形式：各级 4 小时 4 题、满分 400 分，优秀（≥300）与合格（≥150）分档发证，低于 150 不发证；证书有效期三年。③权益：有效期内每年可推荐学生获得 NOIP 名额，优秀档为初级 2、中级 3、高级 5 人，合格档为 1、2、4 人。这一条最有信息量——它把「指导教师推荐」从口头承诺变成可计算的配额，带班规模较大的教练应按学生数反推该考到哪一级。公告同时明确考核命题、评测数据、评测环境与成绩复核由 NOI 科学委员会负责，并声明本公告是对《CCF 中小学计算机教师能力认定暨实施办法》的条款补充，冲突时以本公告为准。局限：只讲认定与配额，不讲培训课程内容与通过率；三年有效期意味着证书需要周期性复检，规则未说明复检是否免考，实际执行以当年通知为准。

https://www.noi.cn/jspx/2026-07-13/917355.shtml

115 **CCF 信息学奥赛入门篇** 官方

中国计算机学会（组编） · 2025

CSP-J 入门自修 体系学习 方法论 纸质书

朱全民主编，CCF 组编，2025 年 4 月正式出版。立足 NOI 竞赛大纲逐条对应知识点，官方定位为 4—7 年级学生的入门读物。价值在于「大纲对齐」：读它不是为了学完，而是为了知道大纲的每一级在小学段该落到什么程度，可与《CCF 全国青少年信息学奥林匹克竞赛教程》配套使用。

https://www.noi.cn/cbw/2025-04-07/841091.shtml

116 **CCF 关于 CSP-J/S 2026 第一轮晋级第二轮的规定** 官方

中国计算机学会 · 2026

CSP-J CSP-S 查漏补缺 赛前冲刺 速查 免费

2026 年 9 月 24 日发布。明确第一轮晋级第二轮的分数线划定方式与名额比例，含各省差异化条款。每年都有选手误判「过线即晋级」，这份文件是唯一权威口径，报名前应直接读原文而非二手解读。

https://www.noi.cn/xw/2026-09-24/934543.shtml

117 **CCF 关于 CSP-J/S 2026 第一轮认证评级分数线的公告** 官方

中国计算机学会 · 2026

CSP-J CSP-S 查漏补缺 教练备课 速查 免费

2026 年 10 月 10 日发布。入门组全国一级 69.5、二级 55.5、三级 42；提高组一级 66.5、二级 52.5、三级 39（满分 100，第一轮为笔试）。公告明确这是全国统一线，各省认证组织单位不得再按本省情况调整——这一点与 NOIP 的省线机制完全不同，是很多家长判断失误的根源。三个用法：一是赛后定位，用自己第一轮得分直接对档；二是教练做校内模拟卷时，可把「达到本省一级线」作为及格基准反推训练强度；三是纠正表述，CSP 只有一级/二级/三级，没有「一等奖」，且它与 NOIP 评奖体系无关，写简历、填报名表时不能混用。注意这是第一轮（笔试）线，与第二轮机试成绩、与省选资格门槛都不是一回事。

https://www.noi.cn/xw/2026-10-10/938437.shtml

118 **CCF 关于 NOI 相关权益的声明** 官方

中国计算机学会（NOI 官网） · 2020

CSP-J CSP-S NOIP NOI 教练备课 查漏补缺 速查 免费（官网公告）

2020 年 6 月 23 日发布，CCF 官网首页长期置顶。这份声明的价值在于它把「谁有权代表 NOI」写成了可核对的条文，共五条：① NOI 的对外宣传平台只有三个——CCF 官网 ccf.org.cn、NOI 官网 noi.cn、微信公众号 ccfvoice，其他渠道转载 NOI 新闻须经 NOI 竞赛办公室书面同意；② CCF 从未授权任何机构承办面向青少年的 NOI 系列赛前培训，今后也不会授权任何「赛前培训班」；③「CCF NOI」与「NOIP」是 CCF 注册商标，凡以 CCF 名义举办的 NOI 相关活动必有书面授权书，拿不出授权书而自称承办方的即属侵权；④ noi.cn 上的「NOI Online 培训」视频仅支持在线观看，版权归 CCF，不得下载转载；⑤ 对冒用「NOI」「信息学官方」「NOIP 官方」等名义虚假宣传的，已交法律顾问处理。对教练与家长的直接用法：遇到机构自称「NOI 官方合作机构」「NOIP 考前培训班」，按第②③条即可判定为虚假宣传，可直接向 noi@ccf.org.cn 或 010-62562503 投诉；学校在校内组织培训不受影响，受限的是「以官方名义」与「校外有偿兼职」两类行为（后者另见《关于重申在校在职 NOI 指导教师不得校外有偿兼职授课的公告》）。局限：发布于 2020 年，文中提到的「NOI Online 培训」栏目此后已并入 noi.cn 的资料下载与活动视频栏目，条款本身仍然有效，但具体栏目名称要按现行站点理解；声明只讲权益归属，不含处罚细则与认定程序。

https://www.noi.cn/xw/2020-06-23/715566.shtml

119 **CSP-J/S 2026 第一轮认证证书申请通知** 官方

中国计算机学会 · 2026

CSP-J CSP-S 查漏补缺 教练备课 速查 免费

2026 年 10 月 16 日发布，受理窗口 10 月 17 日—11 月 20 日，逾期本年度不再受理且不补办——这是全流程里唯一「过期作废」的硬截止，教练应在 10 月中旬就提醒全队。要点：电子证书免费、窗口内可多次下载，逾期连电子版也不再开放；纸质证书 100 元/张（含工本、人工、快递），预计 12 月 16 日后寄出；两者可同时取得，报名表、简历、综评材料用电子版即可，不必付费。资格线沿用第一轮评级：入门组一级 69.5、二级 55.5、三级 42，提高组 66.5/52.5/39，未达任一等级者系统中没有下载按钮。操作细节最容易翻车：必须先在【用户信息】里填完整邮寄地址，再去活动页提交纸质申请并缴费；仅限电脑端、仅限支付宝，手机或微信内置浏览器打开会失败且费用不退；第一轮电子证书无证书编号（页面显示「暂无」属正常）；下载弹窗要等约半分钟，中途关掉会得到残缺文件。

https://www.noi.cn/xw/2026-10-16/940603.shtml

120 **CSP-J/S 2026 第二轮报名通知** 官方

中国计算机学会 · noi.cn · 2026

CSP-J CSP-S 赛前冲刺 速查 免费

2026 年 10 月 31 日认证：入门组 8:30—12:00（3.5 小时），提高组 14:30—18:30（4 小时）。第二轮由各省负责人统一报名，选手只需在规定时间内缴费确认。

https://www.noi.cn/xw/2026-09-29/933744.shtml

121 **CSP-J/S 认证须知** 官方

CCF · noi.cn · 2025

CSP-J CSP-S 赛前冲刺 速查 免费

报名条件、赛制、评测方式、违规处理。考前必读，每年都有细节调整。

https://www.noi.cn/gynoi/tlgd/2019-09-30/710460.shtml

122 **NOI Linux 2.0 官方发布页** 官方

NOI 科学委员会 · noi.cn · 2021

CSP-J CSP-S NOIP 省选 NOI 查漏补缺 教练备课 速查 免费（镜像约 3.4 GB）

自 2021 年 9 月 1 日起的官方标准环境，基于 Ubuntu 20.04.1 定制。页面附系统情况简表（内核、编译器、IDE 版本）与镜像下载地址，是「考场到底用什么」的唯一权威来源。

https://www.noi.cn/gynoi/jsgz/2021-07-16/732450.shtml

123 **USACO 官网** 官方

USACO · usaco.org · 1992

CSP-J CSP-S NOIP 实战训练 体系学习 实战 免费

四级赛制（铜/银/金/白金），每年四次月赛。题目公开、题解公开，训练价值极高。

https://usaco.org/

124 关于 CCF CSP-J/S 认证等级名称规范的公告

官方

中国计算机学会 · 2026

CSP-J CSP-S 查漏补缺 教练备课 速查 免费

2026 年 9 月 9 日公告，规范的是对外表述口径：CSP-J/S 是面向非专业人士的算法与编程能力评价活动，不属于竞赛评奖，不设一、二、三等奖，只按成绩由高到低评一级、二级、三级；把「一级」写成「一等奖」属对认证规则的错误解读，机构与学校对外必须统一用「一级、二级、三级」。这条看似只是措辞，实际卡两处：一是证书、简历、报名表上的写法，二是各类「省一/省二」经验贴与选拔条款的对应关系——多省省选资格写的是「NOIP 一等奖」，那属于 NOIP 评奖体系，与 CSP 的等级不是同一套。读旧资料时必须先分清文中说的是 CSP 等级还是 NOIP 奖项，否则会把准入条件看错一档。

https://www.noi.cn/xw/2026-09-09/930554.shtml

125 关于 NOI 系列活动中编程语言使用限制的补充说明

官方

CCF NOI 科学委员会（NOI 官网） · 2021

CSP-J CSP-S NOIP NOI 查漏补缺 赛前冲刺 速查 免费（官网公告，五条正文）

2021 年 9 月 1 日由 CCF NOI 科学委员会发布，是针对《关于 NOI 系列赛编程语言使用限制的规定》编程通则的补充，适用范围覆盖 CSP-J/S 在内的全部 NOI 系列活动。五条内容：① 除题面有明确要求外，C++ 程序编译默认采用 C++14 标准；② 除题面明确要求并允许，禁止在源代码中自行指定任何编译选项——所有 pragma、attribute、内联汇编式的编译指示都属于此类；③ 允许使用以下划线开头的库函数或宏，但编程通则中明确禁止的除外；④ 已发布的技术类规则若与本说明冲突，冲突部分以本说明为准；⑤ 其余未尽事项按现有规则执行。末段是硬约束：使用明确禁止的内容者，源代码直接判零分，且不接受申诉。对选手的直接意义有两条：一是本地为了卡常惯用的 #pragma GCC optimize 、__attribute__((optimize("O3"))) 必须在提交前删净，很多「本地 1.2 秒、评测 0 分」的事故都出在这里；二是默认 C++14 意味着 std::gcd、结构化绑定、if constexpr 这类 C++17 特性在未注明语言版本的题上不可用，跨平台编译前先在 NOI Linux 2.0 上用 -std=c++14 过一遍。局限：正文只有五条，是补充而非全集，完整的禁用清单要去编程通则原文核对；发布于 2021 年，此后 NOI Linux 2.0 已迭代过编译器版本，具体 g++ 小版本以当年竞赛环境说明为准。

https://www.noi.cn/gynoi/tlgd/2021-09-11/836159.shtml

126 **关于参加 CSP-J/S 认证需符合年龄条件的公告（12 周岁门槛）** 官方

中国计算机学会（NOI 官网） · 2025

CSP-J CSP-S NOIP 教练备课 查漏补缺 速查 免费（官网公告）

2025 年 2 月 13 日发布并即时生效。核心条款只有一句：报名参加 CSP-J/S 的被认证者，在当年 9 月 1 日（不含）须满 12 周岁，第一轮与第二轮一并适用，「参加 NOI 相关竞赛的规则同此」。也就是说，不满 12 周岁不得报名 CSP-J/S 与 NOIP 等 NOI 系列活动，判定基准是当年 9 月 1 日，不是报名日、也不是考试日——这一点对安排低龄学生参赛的教练最要紧：一个 11 月出生的孩子，要等到满 12 周岁之后的那个赛季才能报名，此前整年只能用非 CCF 题库自练。公告给出的理由有三层，且都值得原样转述给家长：一是小学生过早参加 CSP-J/S 对其身心健康与正常学习有负面影响，不符合认证初衷；二是部分中学把小学生的 CSP-J/S 成绩当作升学参考依据，会带来严重的功利化倾向，影响正常教学与升学秩序；三是官方明确表态「学习计算机科学有益，但需要基础与认知能力，并非越早越好」。实用含义有二：青训班的分层要按「当年 9 月 1 日是否满 12 周岁」切，而不是按年级；CCF 体系内另一条线 GESP（8 级制）没有这条年龄限制，是 12 岁以下唯一能参加的 CCF 认证，衔接路径应改为 GESP → CSP-J/S。局限：公告未写过渡期、未说明既往成绩如何处理，也未界定跨年赛季（例如 9 月 1 日前后同一场认证）的执行口径；条文极简，具体核验方式以各省特派员当年通知为准。

https://www.noi.cn/gynoi/tlgd/2025-02-13/837978.shtml

127 **关于重申在校在职 NOI 指导教师不得校外有偿兼职授课的公告** 官方

中国计算机学会（NOI 官网） · 2025

CSP-J CSP-S NOIP NOI 教练备课 速查 免费（官网公告）

2025 年 2 月 13 日发布。全文只有三段，但把「教练合规」的边界划清了：所有参与 NOI 活动的在校在职教师必须遵守教育部相关规定，不得在校外培训机构有偿兼职授课；CCF 将强化对在校在职 NOI 教师的监督管理，对违反者按情节轻重采取警告、取消 NOI 参与资格、通报所在单位及教育主管部门等措施。触发点是官方点名的两类行为——在校外机构参与有偿培训，以及利用校内职务之便进行不当宣传牟利。对学校教练的实际约束可以拆成三条：不能以个人身份到机构挂名带班（哪怕不讲课，挂名即属「利用职务之便宣传」）；不能把校内学生成建制带到机构上课；校外机构以「某校教练」名义招生时，教师本人有澄清义务。与《CCF NOI 教师培训考核管理规则》配合看更清楚：CCF 一边用三级证书与 NOIP 推荐名额做正向激励，一边用这条公告做负向约束，等于把「校内教师」与「机构讲师」的身份边界制度化了。局限：未定义「有偿」的认定口径（课时费、股权、分成是否都算），未给出申诉与复核程序，也未说明处理结果是否公示；实践中的争议点常落在「公益讲座」与「有偿授课」之间，公告没有给判据。

https://www.noi.cn/gynoi/tlgd/2025-02-13/837982.shtml

128 **电子学会青少年等级考试** 官方

中国电子学会 · qceit.org.cn · 2016

CSP-J 入门自修 速查 报名费

科普性质的等级认证，与 NOI 体系不互通。仅适合作为兴趣启蒙的阶段性检验。

https://www.qceit.org.cn/

129 **蓝桥杯** 官方

工信部人才交流中心 · lanqiao.cn · 2010

CSP-J 入门自修 实战 报名费

难度低于 CSP-J，题型偏应用。可作为低龄选手的第一场比赛，但不宜当作 OI 主线。

https://www.lanqiao.cn/

130 **2025 年 CSP-J/S 第二轮认证题目分析报告** 官方

中国计算机学会 · 2025

CSP-S CSP-J 教练备课 查漏补缺 方法论 免费

2025 年 11 月 12 日发布。官方对第二轮四道题逐题给出考点分布、得分率与典型错误，是判断「本年难度是否上移」的基准材料。备课时可据此决定专题投入顺序，避免按往年经验误判重点。

https://www.noi.cn/zxzy/lnzl/jszl/2025-11-12/852998.shtml

131 **CCF 全国青少年信息学奥林匹克竞赛教程** 官方

CCF NOI 科学委员会 · CCF · 2025

CSP-S NOIP 省选 教练备课 查漏补缺 方法论 CCF 官方渠道

配合 2025 版大纲发布，是判断「考什么、考到多深」的第一手依据。

https://www.noi.cn/

132 **COCI** 官方

克罗地亚信息学奥赛 · hsin.hr · 2006

CSP-S NOIP 省选 实战训练 实战 免费

每月一轮、六题分层，题面短、思维密度高。被中文圈大量用作训练题源。

https://hsin.hr/coci/

133 **NOIP 2026 参赛名额分配方案** 官方

中国计算机学会 · 2026

CSP-S NOIP 教练备课 查漏补缺 速查 方法论 免费

2026 年 10 月 12 日发布。A 类基本名额各省 30 个（内含 3 个女选手保障名额，不足额则作废）；B 类激励名额全国 6600 个，按普及规模 55%、上一年度 NOIP 全国前 27% 占比 27%、上一年度 NOI 金银铜牌分别折算 8/4/3 个名额的 18% 三项加权；C 类为推荐名额（钻石/金牌教师 2 名，指导教师证书入门/提高/NOI 级 1/2/3 名，省特派员 10 名）；D 类为女生专项，不超过本省 A+B+C 之和的 5%。附表列出 32 个省级单位的 A+B 名额（广东 903、浙江 661、山东 613、江苏 477、北京 435，青海 35）。用途：判断本省一等奖的实际稀缺度、估算「省队大致要排到多少名」，比单看往年分数线更有结构。

https://www.noi.cn/xw/2026-10-12/938438.shtml

134 **2023 全国信息学奥林匹克年鉴（电子版）** 官方

中国计算机学会（CCF 数字图书馆） · 2025

NOIP 省选 NOI 教练备课 查漏补缺 深耕 方法论 免费（电子版，需 CCF 账号）

2025 年 9 月 19 日由 CCF 发布。内容是 2023 年度的官方档案：NOI 大事记、组织机构、全年赛事成绩单，以及系列活动（CSP-J/S、NOIP、NOI、冬令营、APIO 中国区）的试题与官方解题报告。这版有个制度性变化必须记下来——公告明确“自本年度起，CCF 将不再出版纸质版 NOI 年鉴，仅以电子版形式向公众开放”，所以它是第一本只存在于 CCF 数字图书馆的年鉴（馆内编号 7688319812339712），与已收录的 2024 年鉴（2026 年 6 月电子版）正好接成连续两年。用途是复盘而非刷题：官方解题报告给出的是命题意图与得分分布，比任何民间题解都更接近出题人本意，做年度命题走向比对、判断某类考点是否在升温时，这是第一手材料；教练写学年计划、向学校说明训练重点时也常引用其中的成绩单。局限：内容止于 2023 年度，2024—2026 三年的试题要走当年的“题目及数据”栏目；电子版需注册 CCF 账号后在馆内阅读，不能直接下载 PDF 外传；单本年鉴体量大，别指望通读，按“先看解题报告、再看成绩单”的顺序取用效率最高。

https://www.noi.cn/cbw/2025-09-19/848426.shtml

135 **CCF 关于 NOIP 2026 报名的通知** 官方

中国计算机学会 · noi.cn · 2026

NOIP 赛前冲刺 教练备课 速查 免费

2026 年 10 月 12 日发布：11 月 28 日 8:30—13:00 举行，4 题、机试、4.5 小时；参赛资格仅限在籍在校高中生；报名截止 11 月 9 日 14:00。注意时长是 4.5 小时，不是 3.5 小时。

https://www.noi.cn/xw/2026-10-12/938849.shtml

136 **JOI** 官方

日本信息学奥赛 · joi 官网 · 1993

NOIP 省选 实战训练 实战 免费

日本奥赛，题目干净、梯度平滑，春合宿题尤其适合省选阶段。

https://www.ioi-jp.org/

137 **NOIP 评奖规则** 官方

CCF · noi.cn · 2025

NOIP 教练备课 赛前冲刺 速查 免费

分数线怎么划、省一省二怎么定。理解规则才能制定合理的得分策略。

https://www.noi.cn/gynoi/tlgd/2024-11-26/834964.shtml

138 **NOIP 题目综合分析报告** 官方

CCF NOI 科学委员会 · noi.cn · 2025

NOIP 教练备课 查漏补缺 方法论 免费

官方逐题拆解知识点与难度系数。比任何民间「考情分析」都可靠，命题风向标。

https://noi.cn/xw/2026-09-20/933736.shtml

139 **第六届女生竞赛报名通知（NOIP 2026 女生赛）** 官方

中国计算机学会 · 2026

NOIP NOI 查漏补缺 赛前冲刺 速查 方法论 免费

2026 年 10 月 12 日发布。女生竞赛与 NOIP 2026 一试同步（11 月 28 日 8:30—13:00），报名 NOIP 的女选手自动获得参赛资格，不再单独报名。奖项按参赛总人数 10%/20%/30% 划一、二、三等奖，总获奖率 60%；高中生按成绩取前 10 名（已入选省队者顺延递补，每省经此通道进 NOI 者不超过 2 名）以 B 类身份参加 NOI 2027。价值在于这是一条与省选并行的独立通道：女生若省选把握不大但 NOIP 分数稳定在前列，应把它当主路径规划；反之，错过 NOIP 报名就等于同时放弃两条线。

https://www.noi.cn/xw/2026-10-12/938850.shtml

140 **2022—2023 年国内外信息学主要赛事分析报告** 官方

中国计算机学会 · 2025

省选 NOI 教练备课 查漏补缺 方法论 深耕 免费

2025 年 11 月 12 日发布于 NOI 官网竞赛资料栏，谢秋锋、金靖、李曙、赵启阳、艾庆兴等 18 位一线教练与高校教师联合执笔。把 NOI 系列赛事与 IOI、APIO、JOI、USACO 等国外赛事按知识点分布、难度结构、区分度做横向对比，是官方视角下「我们的训练强度与命题取向和国外差在哪」的判断材料。教练排年度计划、挑模拟题源时，比只看单年真题分析报告更有参照价值；注意其数据截到 2023 年，结论需与近两年真题对照使用。

https://www.noi.cn/zxzy/lnzl/jszl/2025-11-12/852999.shtml

141 **APIO 2026 中国区活动赛事综述与分数线** 官方

中国计算机学会 · 2026

省选 NOI 赛前冲刺 教练备课 速查 方法论 免费

2026 年 5 月 7—11 日由 CCF 主办、北京大学附属中学承办，27 个省市自治区共 768 位师生参加，5 月 9 日测试。获奖规模：金牌 75 人（线 196 分）、银牌 117 人（线 155 分）、铜牌 200 人（线 116 分）。同期设七场讲座，讲者为杨耀良（NOI 2017 金）、谢梓涵（NOI 2023 金）、李静榕（NOI 2024 金）及 IOI 2026 国家队四人。核心价值是三条分数线：APIO 是国内选手与亚洲同届横向定位的少数硬指标，可直接用来判断「省队边缘」选手的客观位置，比只看省内排名可靠；讲座名单也可作为选题风向的参考。

https://www.noi.cn/xw/2026-05-11/891305.shtml

142 **CCF 关于 NOI 2027 省内选拔的若干规定** 官方

中国计算机学会 · noi.cn · 2026

省选 NOI 教练备课 赛前冲刺 方法论 免费

2026 年 9 月 11 日发布，共 15 条。三个关键变化：CCF 不再组织统一省选、NOIP 2026 成绩在省选总成绩中权重不得低于 30%、省队名单须于 2027 年 4 月 7 日前报送。选省选路线前必须先读这一份。

https://www.noi.cn/xw/2026-09-11/930020.shtml

143 **CCPC 官网** 官方

中国大学生程序设计竞赛 · ccpc.io · 2015

省选 NOI 实战训练 实战 免费

国内大学生赛事，题目偏工程与思维结合，可作为 OI 之外的补充训练。

https://ccpc.io/

144 **ICPC 官网** 官方

ICPC Foundation · icpc.global · 1970

省选 NOI 实战训练 实战 免费

大学生赛事，但题目质量高、团队赛制独特。OI 选手常借它练整场节奏。

https://icpc.global/

145 **NOI 2027 名额分配测算方案** 官方

中国计算机学会 · 2026

省选 NOI 教练备课 查漏补缺 速查 免费

2026 年 10 月 28 日发布。给出各省 A/B 类名额的测算口径与调整系数，是判断「本省进省队大概要多少名」的官方依据。教练排年度目标、家长评估路径可行性时先看这份，再看各省实施细则。

https://www.noi.cn/xw/2026-10-28/944054.shtml

146 **NOI 2027 联合省选：两试时间与成绩构成对照（26 省市）** 官方

CCF 及各省竞赛委员会（NOI 官网省赛栏目） · 2027

省选 赛前冲刺 查漏补缺 速查 免费（各省公告，含报名表与日程附件）

2026 年 10 月下旬起，各省陆续在 noi.cn 省赛栏目发布 NOI 2027 省队选拔方案，共同点是参加多省（26 个省市）联合省选、委托第三方统一命题与评测、环境统一 NOI Linux 2.0，两试都排在 2027 年 3 月 6 日与 7 日上午 8:30—13:30、每试 3 题。差异全在成绩构成上：上海为 NOIP 2026 标准分 30% + 一试 35% + 二试 35%，北京为 40 + 30 + 30，重庆为 NOIP 40% + 省选 60%，辽宁为 NOIP 30% + 省选 70%，江西按 NOIP 成绩/400×600×40% + 联合省选 60%。报名一律由学籍所在学校统一提交、不接受个人报名，截止时间各省不同（江西 2026-12-31、江苏 2027-01-08 至 31、北京 2027-01-10 至 17、辽宁 2027-01-25 至 30、上海 2027-01-30）；资格线普遍是 NOIP 2026 省一或全国三等基准线以上、非高三在籍在校，A 队 5 人（4 男 1 女）、其余按名额进 B 队。用法很直接：先算清本省的权重，再决定省选前的训练重心——权重 30/70 的省份，省选两试几乎决定命运，NOIP 只需保住资格线；权重 40/60 的省份，NOIP 每一分都值钱，不能抱着「省选再翻盘」的心态。另有两处容易忽略：多省把「NOI 2027 冬令营成绩」写进同分排序（重庆明确未参加者按 0 分计），以及「三分之一」原则对单校名额的限制。局限：这里给的是各省方案的共同骨架与若干省的权重实例，具体考点、试机、申诉流程必须回到本省公告附件核对；A/B 类名额与「三分之一」细则以 CCF 的《NOI 2027 省内选拔的若干规定》为准，省方案与 CCF 规定冲突时以后者为准。

https://www.noi.cn/gs/xw/zq/2026-10-28/943712.shtml

147 **2024 全国信息学奥林匹克年鉴** 官方

CCF 数字图书馆 · 2026

NOI 省选 教练备课 查漏补缺 深耕 电子书

CCF 主编，2026 年 6 月以电子版发布。含 2024 年 NOI 大事记、组织机构、赛事成绩单，以及全年系列活动试题与解题报告。教练做年度复盘、对比本省选手分布、研究命题走向时的第一手材料，比零散搜集题面更完整。

https://dl.ccf.org.cn/article/detail.html?id=8057449899870208

148 **APIO 官网** 官方

Asia-Pacific Informatics Olympiad · apio 官网 · 2007

NOI 赛前冲刺 实战 免费

亚太赛。中国区由 CCF 组织，题目难度略低于 IOI，是省队选手的第一次国际模拟。

https://www.apio-official.org/

149 **Baltic OI** 官方

Baltic Olympiad in Informatics · boi 官网 · 1995

NOI 赛前冲刺 实战 免费 / oj.uz

波罗的海地区奥赛，题目风格接近 IOI，难度中等偏上，适合补充题源。

https://oj.uz/

150 **CCF NOI 集训营 2026 综合素质评价机制（附国家集训队行为规范）** 官方

中国计算机学会（NOI 官网） · 2026

NOI IOI 教练备课 赛前冲刺 方法论 官网公告（免费阅读）

2026 年 9 月 26 日 CCF NOI 集训营 2026 开营会议上发布的机制说明，与《国际信息学奥林匹克中国国家集训队行为规范》《IOI 2027 中国国家队选拔规则》同日公布。要点有三：① 国家集训队选拔首次把「综合素质评价」与竞赛成绩并列，成为后续国家队选拔的依据之一，考评维度明确列在出勤、材料提交、文明交流、安全意识、公共行为、住宿秩序六项上，依据「客观、可核实的事实」打分；② 未达阈值分者失去后续国家队选拔资格，情节严重者直接取消集训队资格；③ 同时公布三阶段选拔流程与淘汰规则，并明确指导教师除专业训练外还承担行为规范与成长状态的育人责任。这份文件的实际读者不是选手而是教练与家长：它把「唯成绩」这条路在最高层级上正式堵死，意味着集训队层面的时间管理、材料提交、集体生活规范都会影响选拔结果。对省队以下的选手，它的价值是提前校准预期——把「进集训队」理解成一次有行为约束的集训，而不是一次纯考试。局限：公告只给规则框架与评分维度，未公开各项权重与阈值分数；具体打分表在执行层面，外界无法核对。

https://www.noi.cn/xw/2026-09-28/935940.shtml

151 **CEOI** 官方

Central European Olympiad in Informatics · ceoi 官网 · 1994

NOI IOI 赛前冲刺 实战 免费 / oj.uz

中欧奥赛，构造与交互题质量高，是 IOI 前的高质量模拟题源。

https://oj.uz/

152 **IOI 2027 中国国家队选拔规则与国家集训队行为规范** 官方

中国计算机学会 · 2026

NOI IOI 教练备课 赛前冲刺 速查 方法论 免费

2026 年 9 月 28 日随 CCF NOI 集训营 2026 开营发布，含《国际信息学奥林匹克中国国家集训队行为规范》《IOI 2027 中国国家队选拔规则》与综合素质评分规则。核心变化是把综合素质评价正式并入国家队选拔：出勤、材料提交、文明交流、安全意识、公共行为、住宿秩序六项按可核实事实打分，未达阈值即失去后续选拔资格。走集训队路线的选手与教练必须读原文——往年只看成绩的备赛习惯，从这一届起不再成立。

https://www.noi.cn/xw/2026-09-28/935940.shtml

153 **IOI Syllabus** 国际信息学奥赛大纲 官方

IOI 国际委员会 · IOI · 2020

NOI IOI 教练备课 查漏补缺 速查 官网免费

对照国内大纲看差异：国际赛更强调建模与证明，知识点边界更宽。

https://ioinformatics.org/page/syllabus/12

154 **NOI 冬令营 2026 首次 AI 辅助编程测试（人机协同评测）** 官方

中国计算机学会（NOI 官网通稿） · 2026

NOI IOI 教练备课 专题突破 方法论 免费（官网通稿）

2026 年 2 月 13 日 CCF 通稿披露。第 43 届全国青少年信息学奥林匹克冬令营（2026 年 2 月 6—13 日，青岛城阳一中）首次并行设立「有 AI 辅助编程」的测试：邀请 30 位选手参加，可访问指定的大模型，但题目与软硬件环境与本届常规测试完全一致，单独评奖，最终评出金牌 3 名、银牌 6 名、铜牌 10 名。官方给出的定性是「为探索未来信息学竞赛的可能形式、积累经验和数据，为未来相关决策提供数据支持」。据江苏方面对参赛选手的报道，接入的是「千问」「DeepSeek」两大系列共四个模型。同届冬令营常规测试为 56 金 137 银 147 铜，澳门代表队首次参加现场活动。这条的价值不在「多了一个奖项」，而在它第一次把「人机协同」纳入中国 OI 的正式评测口径，且刻意保持题目与常规测试一致——这说明组织方要测的是「同一套题、有 AI 与无 AI 的分数差」，而不是另出一套简单题。对教练的直接含义：今后讨论「要不要让学生用 AI 做题」时，这是目前唯一有官方数据支撑的对照实验，可以等后续公布的分差再定训练策略。局限也要写清楚：官方通稿未公布题目、评分细则、AI 使用限制（可用次数、是否限提示词、是否允许整段生成代码），也未说明该测试是否会延续到后续 NOI 系列活动；把它当成「以后比赛都能用 AI」的确定趋势为时过早。

https://www.noi.cn/xw/2026-02-13/861386.shtml

155 **NOI 参赛资格规定** 官方

CCF · noi.cn · 2025

NOI 教练备课 速查 免费

A/B/C/D/E 类名额的划分与获取方式。省队选拔前必须搞清楚自己走哪条通道。

https://www.noi.cn/gynoi/tlgd/

156 **NOI 试题分析与年鉴** 官方

CCF · noi.cn / 正式出版 · 2000

NOI 教练备课 赛前冲刺 方法论 部分公开 / 纸质

官方对当年试题的逐题分析，含命题意图与得分分布。比任何民间题解都更接近出题人本意。

https://www.noi.cn/xw/

157 **POI** 官方

波兰信息学奥赛 · oi.edu.pl · 1994

NOI 赛前冲刺 实战 免费

波兰是 OI 强国，POI 题目以思维难度著称，适合冲击高分段的选手。

https://oi.edu.pl/

158 **USACO 2025-2026 赛季赛制改革：三场线上赛 + 监考制 US Open** 官方

USACO（美国计算机奥林匹克） · 2026

NOI IOI 赛前冲刺 方法论 在线页面（免费，官方首页赛季回顾）

USACO 在官方首页的「2025-2026 Competition Schedule Recap」里说明本赛季结构与往年不同：改为 2026 年年初三场线上赛，之后一场监考（proctored）的邀请制 US Open 锦标赛；线上赛中表现最好的美国选手获邀参加这场线下终局，而它成为选拔 2026 集训营 finalist、以及 2026 EGOI 四名队员的主要依据。另外两条改动值得注意：一是 platinum 组在赛季开始时被统一重置回 gold（官方称此做法可能延续），二是因 2026 EGOI 早于集训营举行，EGOI 队伍提前确定，EGOI 队员与未来潜力选手仍受邀参加集训营。官方明确这是「过渡年」，组别仍保持 bronze/silver/gold/platinum 四级以便稳定，但未来可能调整为更看重线下终局成绩的年度评价。对国内选手的实际意义有三点：赛季窗口从「12 月一次年 3 月四场」压缩成「年初三场」，每次机会的权重上升，用 USACO 题单做训练时必须按新节奏排；「线上 → 受邀线下 → 定 finalist」这条链与国内「省选 → NOI → 集训队」同构，可直接拿来对比两种选拔哲学（美国把决定性环节放在一场监考赛，国内把权重摊到省选与 NOI 两试）；platinum 重置意味着跨赛季刷题时旧分不再代表当前组别。同时提醒：2025 年及以前的资料（含本目录中按「四场月赛」描述 USACO 的旧条目）描述的是旧赛制，排训练计划时以本条为准。

https://usaco.org/

159 **关于自 NOI 2029 起取消 A 类选手加分政策的公告** 官方

中国计算机学会 · noi.cn · 2026

NOI 省选 教练备课 方法论 免费

2026 年 9 月 11 日发布。NOI 2027、2028 为过渡期仍加 5 分，2029 年起取消。政策变化会影响弱省的选拔策略，规划三年路线时必须纳入。

https://www.noi.cn/xw/2026-09-11/930261.shtml

160 **IOI 2026 中国队 3 金 1 银与最终排名** 官方

中国计算机学会 · 2026

IOI NOI 教练备课 赛前冲刺 速查 免费

第 38 届 IOI 于 2026 年 8 月 9—16 日在乌兹别克斯坦塔什干举行，97 个国家与地区 375 名选手参赛，共产生金牌 31、银牌 62、铜牌 94。中国队 3 金 1 银、团体第一：许淇文（南京外国语学校）498.27 分列第 1，赵晟昊（浙江金华第一中学）第 3，刘家炜（重庆市巴蜀中学校）第 13，何宇翔（重庆南开融侨中学校）银牌、第 38。领队赵启阳，副领队蒋婷婷；下一届在德国举办。用途：赛制为两天各 3 题、每天 5 小时、满分 600，用 498.27 这个冠军分与历年冠军分对照，可粗判当年题目难度，并据此给队内模拟定「两天六题拿多少」的目标。

https://www.noi.cn/xw/2026-08-16/924196.shtml

161 **IOI 2027 官方网站（第 39 届，德国波茨坦 HPI，2027-09-12—19）** 官方

IOI 2027 组织委员会 / 哈索·普拉特纳研究所（HPI） · 2027

IOI 查漏补缺 教练备课 速查 免费（官网，赛前持续更新）

第 39 届国际信息学奥林匹克定于 2027 年 9 月 12—19 日在德国波茨坦举行，承办方是哈索·普拉特纳研究所（Hasso Plattner Institute, HPI）——这是德国继 1992 年波恩之后隔 35 年再次主办 IOI。官网已上线，目前给出的是举办日期、主办机构与城市介绍、赞助与支持方名单，并明确说明「站点会随筹备推进持续更新」，后续才放日程、场地与参赛信息。同期 IOI 官方站开放了 IOI 2027 的 Call for Tasks（征题），截止 2026 年 12 月 31 日 23:59 UTC−12，明确「潜在参赛者不得投稿」，欢迎 batch、communication、output-only 三类以及 IOI 此前未出现过的新题型，并特别说明「简单的原创题最难写、也最缺」。用法分两类：选手侧只用来看赛程与赛制变更——IOI 的规则改动走 General Assembly，最终文本在 ioinformatics.org 的 Regulations，东道主官网只作日程参考；教练与命题者侧，这是提前一年观察东道主命题取向的窗口，往届东道主常在题型配比上做文章（2026 塔什干提高了交互与构造题的比重），征题公告里的题型清单往往就是风向。局限：截至 2026 年 11 月，页面仍以宣传与筹备信息为主，没有题面、没有逐日日程、没有参赛名额；中国队选拔与报名完全不走这里（在 noi.cn），别把它当报名入口。

https://ioi2027.de/

162 **IOI 官网** 官方

International Olympiad in Informatics · ioinformatics.org · 1989

IOI 赛前冲刺 教练备课 速查 实战 免费

历年试题、成绩、规则、大纲的唯一权威来源。做国际题从这里下原版题面。

https://ioinformatics.org/

163 **IOI 年度报告与命题趋势** 官方

IOI · ioinformatics.org · 1989

IOI 教练备课 赛前冲刺 方法论 免费

历年赛题统计、参赛国分布、评分规则变更。判断「未来两年考什么」最可靠的公开材料。

https://ioinformatics.org/

07 第 7 章　在线平台与题库

> 题库不是越多越好，而是越「匹配当前阶段」越好。本章按国内 OJ、国际 OJ、题单与可视化四类编排，每个平台都标注了难度区间与适用场景。

本章收录 52 条，编号 164—215。

164 HydroOJ

平台

Hydro 开源社区 · hydro.ac · 2021

通用 教练备课 实战训练 实战 免费 / 自建

开源评测系统，学校可以自建域做校内赛。教练做训练营的最佳底座。

https://hydro.ac/

165 AcWing

平台

闫学灿（yxc） · acwing.com · 2018

CSP-J CSP-S NOIP 省选 体系学习 实战训练 实战 免费题库 + 付费课程

国内最系统的算法课程型题库，题面中文、配套视频、按知识点分章。零基础起步体验最好的一站。

https://www.acwing.com/

166 Aizu Online Judge

平台

会津大学 · judge.u-aizu.ac.jp · 2004

CSP-J CSP-S 入门自修 实战 免费注册

自带编程入门课程与章节题，平台本身就是教材。

https://judge.u-aizu.ac.jp/

167 Algorithm Visualizer

平台

开源社区 · algorithm-visualizer.org · 2016

CSP-J CSP-S 入门自修 速查 免费

可单步执行的算法动画，含回溯、DP、排序。比 VisuAlgo 更偏「追踪变量」，适合理解递归。

https://github.com/algorithm-visualizer/algorithm-visualizer

168 AtCoder ABC / ARC / AGC

平台

AtCoder Inc. · atcoder.jp · 2012

CSP-J CSP-S NOIP 省选 实战训练 入门自修 实战 免费注册

题目干净、难度梯度平滑。ABC 的 A–F 六题几乎是一条完整的能力阶梯，初学者最友好。

https://atcoder.jp/

169 **HackerRank** 平台

HackerRank · hackerrank.com · 2009

CSP-J CSP-S 入门自修 实战 免费

英文入门题量大、判题友好。适合第一次读英文题面时练手，难度上限不高。

https://www.hackerrank.com/domains/algorithms

170 **LeetCode** 平台

LeetCode · leetcode.cn · 2015

CSP-J 实战训练 实战 免费 / 会员

面试题为主，难度与 OI 不同源。只建议用来熟悉语言与基础语法，不要当 OI 主训练场。

https://leetcode.cn/

171 **OpenJudge** 平台

POJ 团队 · openjudge.cn · 2005

CSP-J 入门自修 实战 免费注册

可按学校/机构建小组，题目偏基础，适合班级统一布置作业。

http://openjudge.cn/

172 **PTA 拼题 A** 平台

浙江大学 / 杭州百腾 · pintia.cn · 2016

CSP-J CSP-S 体系学习 实战 免费 / 部分付费

课程化题库，按教学进度组织。适合跟着学校课程走的学生。

https://pintia.cn/

173 **USACO** 美国信息学奥赛 平台

USACO · usaco.org · 1992

CSP-J CSP-S NOIP 体系学习 实战训练 实战 免费注册

Bronze→Platinum 四级，题目质量高、英文表述规范。想练英文读题就从这里开始。

https://usaco.org/

174 **USFCA 数据结构可视化** 平台

David Galles（旧金山大学） · cs.usfca.edu · 2011

CSP-J CSP-S NOIP 入门自修 查漏补缺 速查 免费

红黑树、AVL、B 树、堆、图算法的交互演示。讲课时投屏效果极佳，也是自查理解的手段。

https://www.cs.usfca.edu/~galles/visualization/Algorithms.html

175 **Vijos** 平台

Vijos 团队 · vijos.org · 2005

CSP-J CSP-S 实战训练 实战 速查 免费注册

老牌 OJ，保留大量早期经典题。活跃度下降，但历史题目仍是好材料。

https://vijos.org/

176 **VisuAlgo** 平台

Steven Halim（新加坡国立大学） · visualgo.net · 2011

CSP-J CSP-S NOIP 入门自修 查漏补缺 速查 免费

算法过程可视化。线段树、平衡树、最短路、网络流都能一步步看动画，卡住时看一遍胜过读三段文字。

https://visualgo.net/

177 **WZOI（温州中学 OJ）** 平台

浙江省温州中学 · wzoi.cc · 2017

CSP-J CSP-S 实战训练 实战 免费（原站已不稳定）

中学自建的开源 OJ，题目贴近教学实际，适合校内训练。原站近年时常无法访问，可搜「WZOI 镜像」或自行部署同款开源 OJ。

178 **一本通在线评测（ybt）** 平台

一本通编写组 · ssoier.cn · 2015

CSP-J CSP-S 入门自修 体系学习 实战 免费

《信息学奥赛一本通》配套题库，章节与教材一一对应。学校机房最常用的练习站。

http://ybt.ssoier.cn/

179 **梦熊 OJ（Mengxiong Online Judge）** 平台

梦熊信息学联盟 · 2024

CSP-J CSP-S NOIP 实战训练 赛前冲刺 实战 免费（部分团队赛需邀请码）

域名 mxoj.net，另有 mengxiong.cn 指向同一套系统（页面备案号为京ICP备2024090394号，可据此判断站点自 2024 年起运营）。与洛谷、UOJ 这类“自己找题刷”的平台不同，它的组织方式是团队制：先输入邀请码加入某个团队，才能在团队内看到比赛与题单，因此公开检索到的题目有限——这也是它最容易被误判的地方。真正值得用的是它的“全真模考”能力。以 2026 年国庆场为例：2026 年 10 月 1 日 CSP-J 组 08:30—12:00（3.5 小时、4 题）、CSP-S 组 14:30—18:30（4 小时、4 题），明确要求文件读写、按 NOI 赛制提交，赛后 13:00 与 20:00 各有一场免费直播讲评；同一年度还有面向 CSP-J 与 CSP-S 的“二十连测”公开赛，逐场推进。用法建议：把它当“考场模拟器”，考前按组别掐时间整场做，做完就进讲评；不要把它当题库——题库的沉淀与题解体系远不如洛谷，题目总数在首页长期显示为建设中。另需注意：它是商业赛训机构（梦熊联盟，2017 年成立）的入口，注册要实名，部分比赛限定年级评奖，参赛前先看清规则。

http://www.mxoj.net/

180 **洛谷 Luogu** 平台

上海洛谷网络科技 · luogu.com.cn · 2013

CSP-J CSP-S NOIP 省选 实战训练 入门自修 赛前冲刺 实战 免费注册

国内 OI 第一入口。题库按难度分红到黑，题单体系完整，几乎每道题都有题解与讨论。主训练场选它。

https://www.luogu.com.cn/

181 **洛谷主题题库** 平台

洛谷 · luogu.com.cn · 2013

CSP-J CSP-S NOIP 省选 NOI 专题突破 查漏补缺 实战 免费

官方与用户共建的题单体系。「省选DP」「图论500题」这类成套题单，是专题突破最快的入口。

https://www.luogu.com.cn/training/list

182 **洛谷公开赛与月赛** 平台

洛谷 · luogu.com.cn · 2013

CSP-J CSP-S NOIP 省选 实战训练 赛前冲刺 实战 免费

每月多场，含入门赛到省选难度。真实比赛节奏的训练场，比单纯刷题更接近考场。

https://www.luogu.com.cn/contest/list

183 **洛谷题单 Training List** 平台

洛谷社区 · luogu.com.cn · 2018

CSP-J CSP-S NOIP 体系学习 实战训练 实战 免费

别人替你把题按专题排好了顺序。新手最该用的功能，比自己在题库里乱翻效率高一个量级。

https://www.luogu.com.cn/training/list

184 **编程猫 OJ（北大—点猫人工智能教育联合实验室）** 平台

北大—点猫科技人工智能教育联合实验室 · 2026

CSP-J CSP-S NOIP NOI 入门自修 实战训练 实战 在线平台

2026 年 3 月北京大学信息科学技术学院与点猫科技共建「北大—点猫科技人工智能教育联合实验室」，其首批产出即编程猫 OJ（ai-oj.codemao.cn）。题库为覆盖 CSP-J/S、NOIP 至 NOI 的原创题，按「入门—普及—提高—省选—NOI」五级闯关组织，集成 AI 出题、实时判题、月赛/模拟赛与题解讨论；教练端可建队、申请学校专场、查看答题数据。与洛谷、Hydro 的差别在定位：题库规模远小于老牌 OJ，胜在赛段分层清晰与教练看板，适合校内梯队的分阶段测评与专场训练，主力刷题仍应放在洛谷与 Codeforces。风险提示：平台 2026 年新上线，题目存量与长期维护需自行观察，不要把它当作唯一训练来源。

https://ai-oj.codemao.cn

185 **蓝桥云课** 平台

蓝桥杯组委会 · lanqiao.cn · 2017

CSP-J CSP-S 入门自修 实战训练 实战 免费 + 付费

蓝桥杯官方练习平台。CSP-J 阶段可作为「刷题量」的补充，但题型与 NOI 系有偏差，别当主线。

https://www.lanqiao.cn/

186 **计蒜客** 平台

北京矩道优达 · jisuanke.com · 2015

CSP-J CSP-S 入门自修 实战训练 实战 部分付费

按知识点和难度筛题，配套课程。适合需要「有人带着走」的初学者。

https://www.jisuanke.com/

187 **51Nod** 平台

51Nod 团队 · 51nod.com · 2014

CSP-S NOIP 专题突破 实战 免费注册

数学题与思维题密度高，适合专门练「想不出来但一看就会」的题型。

https://www.51nod.com/

188 **AtCoder Problems** 平台

kenkoooo · kenkoooo.com · 2016

CSP-S NOIP 省选 NOI 查漏补缺 实战训练 速查 免费

把 AtCoder 全部题目按难度打分并推荐。配合「已 AC 集合」能精确找出你能力边缘的题。

https://kenkoooo.com/atcoder/

189 **Baekjoon Online Judge** BOJ 平台

Baekjoon · acmicpc.net · 2010

CSP-S NOIP 实战训练 实战 免费注册

韩国最大 OJ，题量巨大、分类清晰，很多题有中文翻译。

https://www.acmicpc.net/

190 **CSES Problem Set** 平台

Antti Laaksonen / 赫尔辛基大学 · cses.fi · 2013

CSP-S NOIP 体系学习 专题突破 实战 免费

300 道题按专题排好，一题一技，干扰最少。与 CPH 配套，是「系统过一遍」的最佳载体。

https://cses.fi/problemset/

191 **CodeChef** 平台

Directi · codechef.com · 2009

CSP-S NOIP 实战训练 实战 免费注册

印度 OJ，长期赛制独特（十天一场），适合练长线思考。

https://www.codechef.com/

192 **Codeforces** CF 平台

Mike Mirzayanov · codeforces.com · 2010

CSP-S NOIP 省选 NOI 实战训练 赛前冲刺 实战 免费注册

全球竞赛核心。Div.2 稳定练手、Div.1 冲上限，rated 分数是国际通行的能力刻度。

https://codeforces.com/

193 **Codeforces 题目集（按 rating 筛选）** 平台

Codeforces · codeforces.com · 2010

CSP-S NOIP 省选 NOI 查漏补缺 实战训练 实战 免费

用 tags + rating 区间精确取题，例如 「dp, 1900-2100」。全世界最有效的专题训练工具。

https://codeforces.com/problemset

194 **Comet OJ** 平台

Comet 团队 · cometoj.com · 2018

CSP-S NOIP 赛前冲刺 实战 免费注册

经常办原创赛，题目风格接近 NOIP。适合打非官方模拟赛练手感。

https://www.cometoj.com/

195 **DMOJ** 平台

Don Mills · dmoj.ca · 2015

CSP-S NOIP 实战训练 实战 免费注册

加拿大现代 OJ，聚合各大赛存档，语言支持广，界面体验好。

https://dmoj.ca/

196 **HDU Online Judge** 平台

杭州电子科技大学 · acm.hdu.edu.cn · 2005

CSP-S NOIP 实战训练 实战 免费注册

多校联合训练的题源，题量大。适合按专题检索老题。

https://acm.hdu.edu.cn/

197 **Lutece** 平台

电子科技大学 · lutece.uestc.edu.cn · 2018

CSP-S 实战训练 实战 免费

高校开源 OJ，题面中英混排，适合补充非 OI 风格的算法题。

https://lutece.uestc.edu.cn/

198 **POJ** 北京大学在线评测 平台

北京大学 · poj.org · 2003

CSP-S NOIP 实战训练 实战 免费注册

国内最老 OJ 之一，英文题多。经典但界面古老，适合刷指定经典题号。

http://poj.org/

199 **SPOJ** 平台

Sphere Research Labs · spoj.com · 2004

CSP-S NOIP 实战训练 实战 免费注册

海量老题，质量参差。只在需要某个经典题号时才来。

https://www.spoj.com/

200 **UVa Online Judge** 平台

Universidad de Valladolid · onlinejudge.org · 1995

CSP-S NOIP 实战训练 实战 免费注册

最老的 OJ 之一，《Competitive Programming》全书题号都指向它。

https://onlinejudge.org/

201 **ZOJ** 平台

浙江大学 · zoj.pintia.cn · 2001

CSP-S 实战训练 实战 免费注册

浙大老牌 OJ，题面英文。现在主要作为历史题库使用。

https://zoj.pintia.cn/

202 **e-olymp** 平台

e-olymp · eolymp.com · 2010

CSP-S NOIP 实战训练 实战 免费

东欧题源聚合站，收纳大量小学奥赛与区域赛题。题面英文，难度集中在普及到提高之间。

https://eolymp.com/

203 **牛客网** Nowcoder 平台

牛客网 · nowcoder.com · 2014

CSP-S NOIP 赛前冲刺 实战 免费注册

OI 赛与求职题混排。OI 部分有专项训练与模考，适合赛前找整场节奏。

https://www.nowcoder.com/

204 **Codeforces Gym** 平台

Codeforces · codeforces.com · 2010

NOIP 省选 NOI IOI 实战训练 赛前冲刺 实战 免费

各类集训、省队选拔、ICPC 区域赛的原始赛题存档。想找「真赛题」来这里，不在主题库里。

https://codeforces.com/gyms

205 **Judge Duck Online** 平台

松松松 · duck.ac · 2020

NOIP 省选 查漏补缺 速查 免费

评测精度到微秒，用来验证常数优化到底有没有用。题量少但工具属性强。

https://duck.ac/

206 **Kattis** 平台

KTH · open.kattis.com · 2010

NOIP 省选 实战训练 实战 免费注册

ICPC 真题库，按 Elo 难度推题。想练「读完题就知道考什么」的直觉很有效。

https://open.kattis.com/

207 **Timus** 平台

乌拉尔联邦大学 · acm.timus.ru · 2000

NOIP 省选 实战训练 实战 免费

俄式硬核题，难度偏高。冲刺期用来压题不错。

https://acm.timus.ru/

208 **LibreOJ** LOJ 平台

Menci 等 · loj.ac · 2017

省选 NOI 实战训练 查漏补缺 实战 免费注册

测试数据与提交代码全部公开，赛后能看别人怎么写的。复盘与对拍的首选平台。

https://loj.ac/

209 **Nowcoder 多校训练营** 平台

牛客网 · nowcoder.com · 2018

省选 NOI 赛前冲刺 实战训练 实战 免费 / 部分付费

暑期多校，题目偏难偏怪，是省队选手的集训主战场。赛后题解质量参差，需自己筛。

https://ac.nowcoder.com/acm/contest/vip-index

210 **OEIS 整数数列百科** 平台

OEIS Foundation · oeis.org · 1964

省选 NOI IOI 查漏补缺 速查 免费

数论、计数题猜结论的终极工具。打出前几项就能查到封闭形式与生成函数，考场外验证思路神器。

https://oeis.org/

211 **QOJ** 平台

QOJ 团队 · qoj.ac · 2021

省选 NOI IOI 赛前冲刺 实战训练 实战 免费注册

国内外 OI 与 ICPC 赛题的集中存档，可按比赛整场模拟。找真题最全的地方之一。

https://qoj.ac/

212 **UOJ** Universal Online Judge 平台

VFK 等 · uoj.ac · 2014

省选 NOI 赛前冲刺 实战训练 实战 免费注册

原创题多、难度高，题目质量在国内属第一梯队。提高组冲省选阶段的磨刀石。

https://uoj.ac/

213 **Library Checker** 平台

Nyaan 等 · judge.yosupo.jp · 2019

NOI 查漏补缺 速查 免费

验证模板正确性与常数的标准靶场。写完一个数据结构，来这里跑一遍再放进模板库。

https://judge.yosupo.jp/

214 **Universal Cup** 平台

Universal Cup 组委会 · ucup.ac · 2022

NOI IOI 赛前冲刺 实战 免费

全球强队都在打的系列赛，题目难度对齐区域赛与 IOI。想摸国际天花板就来这里。

https://ucup.ac/

215

oj.uz

平台

Olympiad 社区 · oj.uz · 2016

NOI IOI 赛前冲刺 实战 免费注册

IOI、APIO、 Baltic OI 等国际奥赛题的官方镜像，题面干净。国际赛真题首选。

https://oj.uz/

08 第 8 章　集训队论文与专题讲义

国家集训队论文是中文竞赛圈质量最高的原创资料，每篇解决一个具体问题，思路往往比教材更接近考场。讲义则是教练视角的知识重组，适合成体系地补一个专题。

本章收录 57 条，编号 216—272。

216 信息学竞赛中的思维方法

论文

陈彧 · 国家集训队 2000 · 2000

通用 教练备课 方法论 论文集

少见的「讲怎么想」而不是「讲怎么做」的文献，适合阶段复盘时读。

217 测试数据设计中的三要素

论文

杨帆 · 国家集训队 1999 · 1999

通用 教练备课 方法论 论文集

造数据不是随便随机。准确性、全面性、美观性三条标准，出题人必读。

218 ISSEP 2026 论文集：AI 时代的中小学信息学教育

论文

Springer（LNCS 第 17124 卷） · 2026

CSP-J CSP-S 教练备课 方法论 深耕

SpringerLink 电子版（单章 29.95 欧元，机构订阅可读；目录与摘要免费）

第 19 届 ISSEP（Informatics in Schools: Situation, Evolution, and Perspectives）会议论文集，2026 年 9 月 20—23 日于爱沙尼亚塔林举行，Springer 2026 年 10 月 11 日上线，编者 Piret Luik（塔尔图大学）与 Mart Laanpere（塔林大学）；印刷版 ISBN 978-3-032-41152-5、电子版 978-3-032-41153-2。共 15 篇（13 篇全文 + 1 篇短文 + 1 篇主题报告），从 34 篇投稿中录用，分三个主题：课程、政策与信息学教育的未来；信息学的教与学；计算思维与学习者多样性。对竞赛教练最有直接价值的是三篇：Hromkovič 与 Lacher 的「用基因苏格拉底法在课堂上发现 Hamming 码」（53—70 页）给出一套完整流程——不给定义、只用提问把纠错编码的约束条件逼出来，是「专题课怎么讲才能让学生自己推」的范本；Lepp、Luik、Dagienė 的「Bebras 挑战中的性别差异」（186—199 页）提供计算思维测评的实证数据，可用于社团分层选拔的说理；Chiodini 与 Hauswirth 的「Programming to Learn Language Models」（41—49 页）与「双层 AI 素养」（29—40 页）讨论把大模型当教学对象而非工具，与 NOI 2026 冬令营首次 AI 辅助编程测试的方向呼应。局限必须写清：这是教育学研究论文集，不是竞赛训练书，多数论文对象是小学到高中通识课堂，对 NOI 级选手的算法能力没有直接帮助；不开放获取，只能免费看目录与摘要，正文需机构订阅或单章购买；引用时请按 DOI 标注（10.1007/978-3-032-41153-2）。

https://link.springer.com/book/10.1007/978-3-032-41153-2

219 **OI Wiki 学习路线（C++ 基础 → CSP-J 入门级知识清单）** 讲义

OI Wiki 社区（CC BY-SA 4.0 + SATA） · 2026

CSP-J CSP-S 入门自修 体系学习 方法论 速查 在线页面（免费，可跳转到每个知识点的站内正文）

OI Wiki 竞赛板块的「学习路线」页，页脚标注最近更新 2026 年 11 月 4 日，页首明确写「本文章正在编辑讨论中」。它与同站《学习资源》页（oi-wiki.org/intro/resources/，2026-01-07 更新）分工不同：资源页回答「有哪些 OJ、书、工具、课程」，是并列清单；本页回答「先学什么、后学什么、哪一层该停」，是一份带顺序和考纲边界的知识地图。当前正文分两层：第 1 层 C++ 语言基础（Hello World、变量与运算、分支与循环、数组与结构体、函数与递归），第 2 层 CSP-J 入门级（枚举与模拟、递归与分治、字符串、排序、二分与倍增、DFS/BFS 与剪枝、线性与复杂数据结构入门、DP 入门、高精度/进制/位运算/数论/组合计数），每个知识点直接链到 OI Wiki 对应正文。两处细节对自学者最有价值：一是排序一节明确写出「NOI 大纲入门级只要求选择、冒泡、插入三种，其余因初赛可能涉及而一并列出」，把考纲要求与拓展内容分开标了；二是 DP 入门把「记忆化搜索」与「递推」并列为同一状态空间的两条实现路径，避免初学者把两者当成不同算法。用法：入门阶段每周按路线勾选，勾到某格卡住就点进站内正文；教练可直接拿它当分层课表的骨架。局限：页面仍在编辑中，目前只写到 CSP-J 入门级，CSP-S / NOIP / 省选层级尚未展开，不能当完整大纲用；只给链接与一句话说明，不含题解与代码模板，与本全书正编第 22—31 章对照使用时需自己映射；内容随社区编辑变动，引用时应记录访问日期。

https://oi-wiki.org/contest/roadmap/

220 **Legend-K 的数据结构与算法笔记** 讲义

Legend-K · GitHub / 网络流传 · 2017

CSP-S NOIP 省选 查漏补缺 专题突破 速查 免费（网络）

中文 OI 圈流传最广的个人笔记之一，模板全、排版密。适合当速查表，不适合当第一份教材。

221 **搜索方法中的剪枝优化** 论文

齐鑫 · 国家集训队 1999 · 1999

CSP-S NOIP 专题突破 深耕 论文集

剪枝的六种基本策略，至今没有过时。搜索题上限就靠这个。

222 **搜索顺序的选择** 论文

王知昆 · 国家集训队 2002 · 2002

CSP-S NOIP 专题突破 方法论 论文集

同一份代码换个搜索顺序就从 TLE 变 AC，这篇讲清了原因。

223 **背包九讲** 讲义

dd_engi（崔添翼） · 网络公开 PDF · 2007

CSP-S NOIP 专题突破 深耕 速查 免费 PDF

中文 DP 入门的第一份经典讲义。九种背包讲完，一维滚动数组的理解就彻底了。

224 **骗分导论** 讲义

OI 社区总结 · 网络公开 · 2010

CSP-S NOIP 赛前冲刺 方法论 免费

听起来不正经，但「拿部分分」是 OI 的核心技能之一。省一与省队的差距常在这里。

225 **字符串算法选讲** 讲义

多位竞赛教练 · 网络公开 · 2015

NOIP 省选 专题突破 深耕 免费讲义

KMP、Z 函数、哈希、Manacher 串讲。2025 大纲把 Manacher 下放到提高级，这份讲义正好对上。

226 **常数优化与卡常技巧** 讲义

OI 社区总结 · 网络公开 · 2018

NOIP 省选 赛前冲刺 速查 免费

循环展开、位运算替代、内存访问顺序、快读快写。省一线常常就差这一口气。

227 **浅析非完美算法在信息学竞赛中的应用** 论文

胡伟栋 · 国家集训队 2005 · 2005

NOIP 省选 赛前冲刺 方法论 论文集

考场上拿部分分的正确姿势。省一与省队的分差，常常就在这里。

228 **浅谈用极大化思想解决最大子矩形问题** 论文

王知昆 · 国家集训队 2003 · 2003

NOIP 省选 专题突破 深耕 论文集

悬线法的原始文献，单调栈类问题的必读。

229 **矩阵乘法与线性递推** 论文

多位集训队选手 · 国家集训队历年 · 2011

NOIP 省选 专题突破 速查 论文集

快速幂 + 矩阵加速的标准套路。2025 大纲在 NOI 级新增线性基，可顺带一起学。

230 **NOIpaper：国家集训队论文集（1999—2025）** 论文

GitHub · 2025

省选 NOI IOI 专题突破 教练备课 深耕 免费

个人维护的 GitHub 仓库，汇齐 1999—2025 年国家集训队论文集（2010—2012 年缺），按年份分目录与 PDF。相比逐年搜散落网盘，这里一次拿全，适合按专题横向扫读同一问题在不同年份的解法演进。注意版本为社区整理，引用前核对原文页码。

https://github.com/Fesdrer/NOIpaper

231 **OI 论文集精读** 国家集训队论文选 论文

历届国家集训队 · CCF · 1999-2017

省选 NOI IOI 专题突破 教练备课 深耕 方法论 公开资料集

中国 OI 最高水平的思考沉淀。按主题挑读，比如「线段树的应用」「后缀数组」，不要按年份读。

https://github.com/cjie/Competitive-Programming-Docs

232 **Trinkle23897 的 OI 课件** 讲义

Trinkle23897 · GitHub · 2018

省选 NOI 专题突破 教练备课 深耕 免费（开源）

清华选手的集训课件合集，LaTeX 源码开放。多项式、生成函数、拟阵等硬专题讲得干净。

https://github.com/Trinkle23897/oi_slides

233 **acm-cheat-sheet** 讲义

开源社区 · GitHub · 2015

省选 NOI 查漏补缺 速查 免费（开源）

把常用算法压成几百行的速查表，英文。赛前打印一份贴在手边，比翻书快。

https://github.com/Detachment/ACM-Cheat-Sheet

234 **huzecong 的 OI 课件** 讲义

huzecong · GitHub · 2018

省选 NOI 专题突破 教练备课 深耕 免费（开源）

另一套高质量集训课件，覆盖字符串、数据结构、计数。可直接改造成自己的讲课稿。

https://github.com/huzecong/oi-slides

235 **分块与莫队算法** 论文

多位集训队选手 · 国家集训队历年 · 2014

省选 NOI 专题突破 深耕 论文集

区间问题的暴力美学，常数友好、实现简单，考场性价比极高。

236 **动态规划优化专题** 讲义

多位竞赛教练 · 网络公开 · 2016

省选 NOI 专题突破 深耕 免费讲义

单调队列、斜率优化、决策单调性三件套，配真题讲最有效。

237 **动态规划的深入讨论** 论文

李刚 · 国家集训队 2000 · 2000

省选 NOI 专题突破 深耕 论文集

从状态、阶段、决策三要素拆解，配合例题，适合系统整理 DP 框架。

238 **动态规划算法的优化技巧** 论文

毛子青 · 国家集训队 2001 · 2001

省选 NOI 专题突破 深耕 论文集

单调队列、斜率优化、四边形不等式的源头文献之一，讲优化先讲「为什么能优化」。

239 **图论建模专题** 讲义

多位竞赛教练 · 网络公开 · 2014

省选 NOI 专题突破 方法论 免费讲义

从「题目怎么变成图」讲起，比直接讲算法有用得多。

240 **强连通分量与 2-SAT** 论文

多位集训队选手 · 国家集训队历年 · 2003

省选 NOI 专题突破 速查 论文集

Tarjan 的两个经典应用放一起讲，边界条件讲得清楚。

241 **把握本质，灵活运用——动态规划的深入探讨** 论文

来煜坤 · 国家集训队 1999 · 1999

省选 NOI 专题突破 方法论 论文集

把 DP 讲成「状态设计」而不是「背方程」，中文 DP 文献里最该先读的一篇。

242 **数学模型的建立、比较和应用** 论文

邵铮 · 国家集训队 1999 · 1999

省选 NOI 专题突破 方法论 论文集

建模能力的培养路径。读完再看新题，会先问「这是哪类模型」。

243 **树型动态规划** 论文

多位集训队选手 · 国家集训队历年 · 2006

省选 NOI 专题突破 深耕 论文集

树上背包、换根、树的直径与重心，配题讲比看书快。

244 **状态压缩动态规划** 论文

多位集训队选手 · 国家集训队历年 · 2005

省选 NOI 专题突破 深耕 论文集

状压的经典套路集：棋盘覆盖、TSP、插头 DP 的前置。

245 **离线处理思想** 论文

多位集训队选手 · 国家集训队历年 · 2013

省选 NOI 专题突破 方法论 论文集

CDQ 分治、整体二分、莫队的思想来源。2025 大纲把它列为 8 级考点。

246 **线段树的应用** 论文

林涛 · 国家集训队 2004 · 2004

省选 NOI 专题突破 深耕 论文集

线段树的经典综述：区间合并、标记下传、扫描线，一篇打通。

247 **组合游戏略述——浅谈 SG 游戏的若干拓展及变形** 论文

贾志豪 · 国家集训队 2009 · 2009

省选 NOI 专题突破 深耕 论文集

博弈论中文文献里最完整的一篇，SG 定理的适用边界讲得很细。

248 **网络流算法的应用** 论文

金恺 · 国家集训队 2002 · 2002

省选 NOI 专题突破 深耕 论文集

建图比算法更重要。这篇的重点是「怎么把问题变成流」。

249 **解决动态统计问题的两把利刃** 论文

薛矛 · 国家集训队 2004 · 2004

省选 NOI 专题突破 深耕 论文集

树状数组与线段树的取舍标准，讲清了「什么时候不需要线段树」。

250 **Pólya 原理及其应用** 论文

符文杰 · 国家集训队 2001 · 2001

NOI 专题突破 深耕 论文集

置换群计数的标准入门，Burnside 引理之后必读。

251 **伸展树的基本操作与应用** 论文

杨思雨 · 国家集训队 2004 · 2004

NOI 专题突破 深耕 论文集

Splay 的经典教程，旋转与区间操作的讲法至今被沿用。

252 **充分利用问题性质——例析动态规划的「个性化」优化** 论文

项荣璟 · 国家集训队 2003 · 2003

NOI 专题突破 深耕 论文集

告诉你优化不是套模板，而是抓题目自身结构。读完会少走很多弯路。

253 **具体数学讲义** 讲义

多所高校 · 网络公开 · 2015

NOI 专题突破 深耕 免费讲义

《具体数学》的课堂版浓缩，和式、递推、生成函数三块讲得更快。

254 **动态树及其应用** 论文

袁昕颢 · 国家集训队 2007 · 2007

NOI IOI 专题突破 深耕 论文集

LCT 应用的系统整理：路径查询、连通性维护、动态 MST。

255 **可持久化数据结构** 论文

多位集训队选手 · 国家集训队历年 · 2012

NOI 专题突破 深耕 论文集

主席树、可持久化并查集、可持久化 Trie 的统一视角。

256 **后缀数组** 论文

许智磊 · 国家集训队 2004 · 2004

NOI 专题突破 深耕 论文集

SA 的中文标准教程，倍增与 DC3 都讲了，先学倍增即可。

257 **后缀数组——处理字符串的有力工具** 讲义

罗穗骞 · 国家集训队 2009 · 2009

NOI 专题突破 深耕 论文集 / PDF

后缀数组中文讲解的标杆，模板与证明都给全了。

258 **多串匹配算法及其启示** 论文

朱泽园 · 国家集训队 2004 · 2004

NOI 专题突破 深耕 论文集

AC 自动机的经典引路文，从 KMP 到 Trie 图是一条自然的线。

259 **容斥原理与反演** 论文

多位集训队选手 · 国家集训队历年 · 2015

NOI 专题突破 深耕 论文集

莫比乌斯反演的直觉来源，先把容斥的「符号感」练出来。

260 **左偏树的特点及其应用** 论文

黄源河 · 国家集训队 2005 · 2005

NOI 专题突破 速查 论文集

可并堆的入门文献，代码短、结论清楚，直接抄结构也行。

261 **平面图在信息学中的应用** 论文

刘才良 · 国家集训队 2003 · 2003

NOI 专题突破 深耕 论文集

平面图转对偶图求最小割，思路漂亮且考过多次。

262 **提交答案题与交互题** 讲义

多位集训队选手 · 网络公开 · 2016

NOI IOI 赛前冲刺 方法论 免费讲义

这两类题在 NOI 与 IOI 中占比不小，但普通题单几乎不覆盖，必须单独准备。

263 **数值分析在竞赛中的简单应用** 论文

侯启明 · 国家集训队 2003 · 2003

NOI 专题突破 深耕 论文集

三分、模拟退火、爬山的前置知识，非完美算法的理论依据。

264 **数据结构的联合——块状链表** 论文

蒋炎岩 · 国家集训队 2005 · 2005

NOI 专题突破 深耕 论文集

分块思想的中文源头文献之一，sqrt 分解的所有变体几乎都能从这里长出来。

265 **最小割模型在信息学竞赛中的应用** 论文

胡伯涛 · 国家集训队 2007 · 2007

NOI 专题突破 方法论 论文集

最小割建模的经典总结，最大权闭合子图、二者取一等模型都在这里。

266 **构造题策略** 讲义

多位集训队选手 · 网络公开 · 2017

NOI 专题突破 方法论 免费讲义

构造题没有通法，但有常见套路：归纳、对称、奇偶分类、增量调整。

267 **母函数的性质及应用** 论文

毛杰明 · 国家集训队 2009 · 2009

NOI 专题突破 深耕 论文集

从多项式乘法到生成函数，2025 大纲 9 级考点，建议配合 NTT 一起学。

268 **猜数问题的研究** 论文

张宁 · 国家集训队 2003 · 2003

NOI 专题突破 深耕 论文集

交互题与信息论结合的范例，理解「每次询问能获得多少信息」。

269 **答案只有一个——浅谈问答式交互问题** 论文

高正宇 · 国家集训队 2003 · 2003

NOI IOI 专题突破 深耕 论文集

交互题中文文献里最系统的一篇，IOI 风格必备。

270 **维护森林连通性——动态树** 论文

陈首元 · 国家集训队 2006 · 2006

NOI IOI 专题突破 深耕 论文集

LCT 的中文入门文献之一。难度高，先确保 Splay 熟练。

271 **计算几何讲义** 讲义

谢迪 · 网络公开 · 2012

NOI 专题突破 深耕 免费 PDF

凸包、半平面交、旋转卡壳的中文经典讲义，代码可直接复用。

272 **论随机化算法的原理与设计** 论文

周咏基 · 国家集训队 1999 · 1999

NOI 专题突破 方法论 论文集

随机化不是碰运气，而是把错误率控制在可接受范围。思维方式比代码重要。

09 第 9 章　历年真题与题解

> 真题的价值在于「难度分布」和「命题语言」。做真题不只是练手，更是校准自己对「这题该有多难」的判断。本章按赛事层级排列，从 CSP-J 到 NOI。

本章收录 84 条，编号 273—356。

273 CCF NOI 2026 笔试题库

真题

中国计算机学会（NOI 竞赛办公室） · 2026

CSP-J　CSP-S　NOI　查漏补缺　赛前冲刺　速查　免费

2026 年 6 月 16 日由 NOI 竞赛办公室发布，正文即一份《NOI 2026 基础知识题库》PDF 附件，可直接下载。NOI 正赛第一天为笔试，CSP 第一轮也是纯客观题，两处考的是同一类「计算机基础 + 算法常识 + 竞赛规则」。这份题库的不可替代性在于口径：市面上初赛模拟卷多为机构自编，知识点边界与官方大纲常有偏差，而它是命题方自己给出的样题范围，可用来给「刷了多少套模拟卷」这件事定标。用法建议：先按套限时做一遍定位错项，再回到大纲对应条目补，不要当题海刷——题库体量有限，反复做只会记住答案。

https://www.noi.cn/xw/2026-06-16/903684.shtml

274 CSP-J/S 2025 题目及数据（入门组 4 题 + 提高组 4 题）

真题

CCF NOI 官网 · 历年资料 · 2025

CSP-J　CSP-S　实战训练　查漏补缺　实战　速查　免费下载（CC BY-NC 授权）

CSP-J/S 2025 第二轮（2025 年 11 月 1 日）的官方题面与全量数据，2025 年 12 月 15 日公开。入门组四题：number（拼数）、seat（座位）、xor（异或和）、polygon（多边形）；提高组四题：club（社团招新）、road（道路修复）、replace（谐音替换）、employ（员工招聘）。两组各自一个题面 PDF、四份数据压缩包，入门组与提高组同一天上下午各一场，正好可以拼成一次完整的「一天两场」模拟。这套数据的实际用处在于「按测试点定位失分」：CSP 的测试点通常按特殊性质分档（小数据、特殊图、无特殊性质），把八个数据包按档跑一遍，就能看出自己丢的是暴力分、特殊性质分还是正解分——这三类失分对应的补法完全不同（补代码熟练度 / 补题型敏感度 / 补算法）。入门组第四题与提高组第三、四题是分水岭，建议单独做「部分分最大化」训练：先写稳 40—60 分的暴力与特判，再考虑正解，这本身就是 CSP 赛制下最划算的策略。局限：与所有 CSP 年份一样，官方不配套题解；入门组题面强调「按题意模拟」，纸面看似简单但实现细节多，纯看题解容易低估其代码量。

https://www.noi.cn/zxzy/lnzl/jszl/2025-12-15/854079.shtml

275 **CSP-J/S 2026 第二轮认证测试数据（入门组 + 提高组）** 真题

中国计算机学会 · 2026

CSP-J CSP-S 实战训练 查漏补缺 实战 免费

2026 年 11 月 5 日随《关于 CSP-J/S 2026 第二轮成绩和申诉的通知》发布，附件为入门组（J）与提高组（S）两套完整测试数据 zip，可直接下载。三个用法：赛后复现失分点（本地重测能区分是算法错、常数超时还是边界漏判）、教练做同一题不同解法的常数对比、把当年真题改造成校内模拟赛。配套时间窗要记住：申诉 11 月 5 日 16:00—11 月 8 日 13:00，缴费截止 11 月 8 日 18:00，申诉须经指导教师提交且每题只有一次机会，11 月 13 日后可查结果；申诉不受理「与标准环境有出入」「感觉判错」这类无自测数据的请求，规范做法是附上逐测试点的用时、内存与比对结论。数据本身不含题面，题面需另从当轮试题页获取。

https://www.noi.cn/xw/2026-11-05/945674.shtml

276 **CSP-J/S 历年认证题** 真题

CCF · noi.cn / 洛谷 · 2019-至今

CSP-J CSP-S 赛前冲刺 实战 官网 / 题库

第一轮为笔试，第二轮为上机。两轮都要练，第一轮淘汰率不低。

https://www.luogu.com.cn/

277 **CSP-J/S 第一轮真题** 真题

CCF · noi.cn · 2019

CSP-J CSP-S 赛前冲刺 查漏补缺 实战 免费 / 本站真题库

笔试（阅读程序、完善程序）是很多人翻车的地方。近五年卷子必须整套掐时间做，错题按知识点归类。

https://www.noi.cn/xw/

278 **JOI 2026/2027 一次预选（第 26 回日本信息学奥赛第一轮）题目·解说·解答例** 真题

情報オリンピック日本委員会（JOI 官网） · 2026

CSP-J CSP-S 实战训练 查漏补缺 实战 免费（官网下载，含评分数据）

第 26 回日本信息学奥林匹克（JOI 2026/2027）一次预选的两回比赛已分别于 2026 年 9 月 12 日、10 月 25 日结束，官网在 9 月 23 日、10 月 28 日先后放出全部材料：6 道题的问题文、逐题解说、C++ 与 Python 两份解答例，以及评分用输入数据与正解文件的 ZIP 包，第一回另附完整统计。赛制值得国内选手注意：一次预选共 6 题、每题 100 分、每题含若干小課題（子任务）并按子任务给部分分，限时 80 分钟（14:00—15:20）；前 3 题是「基本题」，难度对标上一年度一次预选的前 3 题，考的是能否把含循环的基本程序写对；后 3 题是「应用题」，难度对标上一年度一次预选第 4 题与二次预选前 2 题，考的是能否设计出高效算法。第一回六题依次为「ノート（笔记本）」「最小除外値」「スポーツ大会」「倍数ペア」「JOI ブロック」「お菓子配り」，参赛 1621 人，六题平均分为 86.1 / 67.6 / 36.9 / 39.0 / 22.6 / 9.5，满 600 分者 137 人，B 级基准线 300 分（授「敢闘賞」）；第二回六题为「かるた」「けんけんぱ」「ジェットコースター」「ペア」「文字列を探せ」「ペア 2」。用法的关键在分数曲线：后三题平均分断崖式下降（第 6 题只有 9.5 分），说明区分度全压在部分分上——这恰好是国内 CSP-J/S 第二轮的得分结构，拿它练「先写暴力拿子任务分、再逐档优化」比刷完整题更对症；一场 80 分钟 6 题也接近初赛与复赛混合的节奏，适合做赛前限时套题。局限：题面与解说全为日文（解答例代码可直接读），小課題的约束必须逐条读懂才敢下笔；覆盖面与 NOI 大纲对不上（无高级数据结构、无复杂图论），只适合 CSP-J/S 段，不能当省选训练。

https://www2.ioi-jp.org/joi/2026/2027-yo1/

279 **USACO 历年试题** 真题

USACO · usaco.org · 1992-至今

CSP-J CSP-S NOIP 实战训练 实战 官网免费

四级分层清晰，题目英文表述规范，是练读题与基础算法的高性价比题库。

https://usaco.org/

280 **《CCF 信息学奥赛入门篇》练习指导** 题解

机械工业出版社（中国计算机学会组编） · 2025

CSP-J 查漏补缺 实战训练 实战 纸质书

廖晓刚、何哲华编，2025 年 12 月第 1 版，ISBN 978-7-111-79342-7，定价 59 元，约 180 页。逐题解析《CCF 信息学奥赛入门篇》的全部练习，七章对应主教材：编程基础、数据的存储与读入、选择执行、反复执行、批量存储（数组与字符串）、模块化程序设计，末章专讲 NOI 系列竞赛相关规定（文件读写、评测与纪律）。必须与主教材配套：单独读它只是题解集，缺语法铺垫；只读主教材不做题，读入格式、边界处理、多组数据这些坑踩不全。适合 CSP-J 第一轮结束后、第二轮前两个月做查漏，也可作为入门班课后作业的答案册。

http://www.wsgph.com/bookshow.asp?id=3097887

281 **信息学奥赛 3：历年真题解析与拉分解题思维训练** 题解

北京大学出版社 · 2026

CSP-J 查漏补缺 赛前冲刺 实战 方法论 纸质书

王健伟著，2026 年 8 月第 1 版，ISBN 978-7-301-37272-2，508 页，定价 129 元，为该系列三册中的第三册。把 2015—2024 年 CSP-J 初赛与复赛真题按年份分章，逐题给解析，并对解析里出现的新概念就地补充；主线是所谓「拉分题」——每套卷中区分一等与二等的 1—2 题，先拆解题意与数据范围，再落到可编译代码。适合已过语法、目标 CSP-J 一等的学生做真题闭环，也可供教练挑例题。局限：真题只到 2024 年，2025 年后的新题型需另配当年题面。

https://www.books.com.tw/products/CN18145540

282 **信息学奥赛 CSP 通关之路：CSP-J/S 第一轮原创全真模拟试卷集（2026）** 真题

人民邮电出版社 · 2026

CSP-J CSP-S 赛前冲刺 实战训练 实战 纸质书

王津、夏宗强等编著，2026 年 7 月第 1 版，ISBN 9787115700209，16 开 158 页、23 万字，定价 59.80 元。结构极简：普及组 CSP-J 2026 初赛模拟卷 10 套 + 提高组 CSP-S 10 套，共 20 套原创卷，逐套按第一轮真题的题型与分值配比（单项选择、程序阅读、程序填空）编制，成套独立、不混编。纸质书只印试题，逐题详细解析放在线上（扫码或图灵社区下载），另设读者群同步各省报名与政策。据出版方说明，2026 版针对近三年 CCF 初赛的变化补充了 Linux 命令、指针、高精度、倍增 LCA、组合计数这几个新出现的考点，并强化了 DP、数论、图论的高频难点。正确的用法是整卷计时：一次两小时、按考场规则做完整套，再对解析并把错因归类到知识点，回到教材补；拆成单题刷会失去它唯一的长处——初赛考的是速度与准确率，不是单题难度。局限有三：原创卷终究不是真题，命题团队的口径与 CCF 科学委员会未必一致；解析在线上，断网或改版后有失效风险；158 页摊 20 套卷，平均每套不足 8 页，题量比真实卷面略薄，适合当节奏训练而非题海。

https://book.cppinfo.cn/Encyclopedias/home/index?id=5081795

283 **信息学奥赛 CSP-J 初赛通关手册：10 年真题 + 10 套模拟精练精讲** 真题

人民邮电出版社 · 2025

CSP-J 赛前冲刺 实战训练 实战 纸质书

左凤鸣主编，2025 年 1 月第 1 版（2025 年 10 月重印），ISBN 9787115654670，16 开 236 页，定价 69.80 元，中图法 TP311.1-44。四部分：一是配套题库系统说明（佐助题库的初赛检测系统，可在线判分）；二是 10 套真题——2014—2018 年 NOIP 普及组初赛按 CSP 新题型改编，接 2019—2023 年 CSP-J1 原卷；三是 10 套 CSP-J 初赛模拟；四是参考答案，逐题解析为电子版，凭书内二维码或在线资源获取。与同期《信息学奥赛初赛全解》《CCF CSP 第一轮认证一本通（第 2 版）》的分工很清楚：那两本按知识点组织，本书按整卷组织，且 10 年真题的时间跨度是三者中最长的，适合做年份纵向对比——同一考点在不同年份落在选择题、阅读程序还是完善程序，只有把整卷按年排开才看得出来。做法建议：真题从 2019 年往前做，越早的卷子改编幅度越大、参考价值递减；模拟卷留到考前两周整卷计时，一次两小时按考场规则走。局限：解析在线上，改版或断网有失效风险；题目止于 2023 年，2024—2026 三届需另配；答案册只给答案不给过程，自学时要配一本知识点书对照。

https://book.cppinfo.cn/Encyclopedias/home/index?id=4817121

284 **信息学奥赛一本通** 训练指导教程 题解

董永建 等 · 科学技术文献出版社 · 2014

CSP-J CSP-S 实战训练 实战 纸质书

与一本通同步的题解册，题量足，适合按章刷。

285 **洛谷主题题单** 真题

洛谷社区 · luogu.com.cn · 2018-至今

CSP-J CSP-S NOIP 省选 体系学习 实战训练 实战 免费

社区维护的专题题单，比官方题库更好用，因为顺序是有人验证过的。

https://www.luogu.com.cn/training/list

286 **算法竞赛入门经典——习题与解答** 题解

刘汝佳 · 清华大学出版社 · 2013

CSP-J CSP-S 查漏补缺 实战 纸质书

紫书课后题的标准答案与思路，自学时最缺的就是这个。

287 **算法竞赛宝典** 基础算法与数据结构 题解

张新华 · 清华大学出版社 · 2012

CSP-J CSP-S 实战训练 实战 纸质书

题量大、分类细，适合作为「刷题清单」使用，讲解略简。

288 **背包问题专题** 题解

OI Wiki / 洛谷题单 · oi-wiki.org · 2018

CSP-J CSP-S NOIP 专题突破 入门自修 深耕 免费

01/完全/多重/分组/树上背包的完整体系。背包是 DP 的第一道门槛，配合「背包九讲」一起看效果最好。

https://oi-wiki.org/dp/knapsack/

289 **BalkOI 2026 试题与在线镜像（巴尔干信息学奥林匹克，地拉那）** 真题

BalkOI 2026 组委会 / Eolymp · 2026

CSP-S NOIP 省选 实战训练 专题突破 实战 免费（Eolymp 在线镜像，可直接提交评测）

巴尔干信息学奥林匹克（Balkan Olympiad in Informatics）自 1993 年起每年一届，2026 年是第 33 届，9 月 14—20 日在阿尔巴尼亚地拉那举行；按主办方公布的数字，13 个成员国加 5 个受邀国（格鲁吉亚、吉尔吉斯斯坦、沙特阿拉伯、土库曼斯坦、乌克兰，乌克兰以客队身份参赛、不计入正式排名）共 75 名选手。资料侧的价值在于 Eolymp 把两天比赛原样做成了官方镜像（Day 1 与 Day 2 分开），各 3 题 5 小时，采用 IOI 式分块计分（一个子任务块内的测试全过才拿该块分，同一题取历史最高分），可以任意语言提交——也就是说这是少数「题面、数据、评测、时限全都在线可复现」的区域赛。必须与库里另一条「BOI 试题库」（boi.cses.fi）区分：两者缩写都写作 BOI，但 Baltic（波罗的海，1995 年起，芬兰/瑞典/波兰等）与 Balkan（巴尔干，1993 年起，阿尔巴尼亚/希腊/罗马尼亚等）是两个完全不同的赛事，题源、命题风格、参赛国都不重叠，这是全库最容易混淆的一对条目。难度定位在 NOIP 后段到省选之间，题目文本短、子任务切得细，适合练 23.5 讲的分档交付。局限：镜像只有英文题面，没有中文翻译与逐题中文题解；正式成绩以主办方公告为准，Eolymp 上的是练习榜；部分交互题的本地调试需要自己写交互器。

https://eolymp.com/contests/balkoi2026-1

290 **CCF CSP 例题解析与实战技巧：算法应用** 题解

机械工业出版社（中国计算机学会组编） · 2026

CSP-S NOIP 专题突破 实战训练 实战 纸质书

中国计算机学会组编，2026 年 8 月 28 日第 1 版，ISBN 978-7-111-81626-3，定价 59 元。专攻 CCF CSP 认证的第四题（最难一题），六章对应六类考点：标准模板库、数学基础、搜索、图论、动态规划、线段树，并解析 2013—2025 年部分真题。体例是本书最值钱的地方——同一道题先给暴力做法与得分，再逐步推到正解与满分，把「怎么从 40 分想到 100 分」的过程摊开写，而不是只贴最终代码。注意辨析：这里的 CSP 指 CCF 面向高校与从业者的软件能力认证，不是 CSP-J/S；但第四题难度与 CSP-S、NOIP 的后两题同段，思路迁移完全成立。适合已能稳定拿到 CSP-S 前两题、卡在后两题部分分的学生。

https://ebooks.cmpbook.com/ccgy/detail?id=30252

291 **Programming Challenges** 题解

Steven S. Skiena、Miguel A. Revilla · Springer · 2003

CSP-S NOIP 实战训练 实战 纸质 / 电子书

与 UVa 题库对应的竞赛题解集，英文写作规范，适合练读题。

292 **eJOI 2026 试题•题解•数据（欧洲初中生信息学奥林匹克，立陶宛考纳斯）** 真题

eJOI 2026 组委会（维尔纽斯大学） · 2026

CSP-S NOIP 实战训练 专题突破 实战 免费（每题题面 PDF + 官方题解 slides + 题面/数据/解答 ZIP）

2026 年 7 月 24—30 日在立陶宛考纳斯举行，官网把六道正赛题按 Day1、Day2 分组，每题给三份资源：题面 PDF、官方题解 slides（英文，讲思路与复杂度，不是只给代码）、以及 statement+tests+solutions 的完整 ZIP；练习赛四题（A+B、Museum、Palc、Pairs）也一并公开。六题是 Day1 的 Increasing、Reconstruct、Xorting（交互题：只能问 p_i xor p_j，要求还原字典序最小的不可区分排列），Day2 的 Automata（两次函数调用式交互，判定是否存在一段 MAX/MIN 指令序列把所有起点送到同一格）、Elevator、Teamfulness。成绩：罗马尼亚 Andrei Nemțișor 515 分夺金，Day1 三题满分；波兰 Rafał Sobolewski 505.8 分紧随。难度定位很清楚——eJOI 面向初中生，单题落点在 CSP-S 后两题到 NOIP 之间，是「国内提高组真题刷完后、又够不到省选」这一段最缺的补充题源；交互题的两道（Xorting、Automata）尤其值得单独练，因为国内初中段几乎不接触交互。另有一条实用信息：LibreOJ 已收录中文翻译（「eJOI2026 异或排序」#5796、「eJOI2026 自动机」#5797），可中文读题后在原站对数据。局限：官网只有英文，题解 slides 篇幅短，部分构造题的关键一步仍需自己补证明。

https://www.ejoi2026.mif.vu.lt/competition/tasks

293 **信息学奥赛一本通 • 高手训练** 题解

黄新军、董永建 · 科学技术文献出版社 · 2021

CSP-S NOIP 省选 实战训练 专题突破 实战 纸质书

以题带练的分册，题目按难度成组。适合提高组真题刷完之后、需要「加练」的那一段。

294 **区间与序列 DP 专题** 题解

OI Wiki · oi-wiki.org · 2018

CSP-S NOIP 省选 专题突破 深耕 免费

区间 DP 的状态设计与断点枚举。石子合并、括号匹配、矩阵链是必做三件套。

https://oi-wiki.org/dp/interval/

295 **单调栈与单调队列** 题解

OI Wiki · oi-wiki.org · 2018

CSP-S NOIP 省选 专题突破 深耕 免费

滑动窗口、最大子矩形、悬线法。代码极短但边界易错，必须手写十遍以上。

https://oi-wiki.org/ds/monotonous-queue/

296 **字符串哈希与 KMP** 题解

OI Wiki · oi-wiki.org · 2018

CSP-S NOIP 省选 专题突破 深耕 免费

哈希要会双模 + 随机底数防卡；KMP 的 next 数组要能默写。字符串题的入场券。

https://oi-wiki.org/string/kmp/

297 **树形 DP 专题** 题解

OI Wiki · oi-wiki.org · 2018

CSP-S NOIP 省选 专题突破 深耕 免费

换根、树上背包、树上依赖。树形 DP 是最稳定的得分点，必须练到「看见树就想状态」。

https://oi-wiki.org/dp/tree/

298 **树状数组** 题解

OI Wiki · oi-wiki.org · 2018

CSP-S NOIP 省选 专题突破 速查 免费

代码最短的区间数据结构，支持单点改区间查与区间改单点查。二维、权值树状数组也要会。

https://oi-wiki.org/ds/fenwick/

299 **算法思维：竞赛真题精选精讲（第 2 版）** 题解

人民邮电出版社 · 2026

CSP-S NOIP 省选 体系学习 专题突破 方法论 深耕 纸质书

2026 年 1 月第 2 版，ISBN 9787115683649，386 页。选 IOI、NOIP、USACO、CCC、CCO、ICPC、DWITE 等赛事的 28 道题，每题统一按「问题描述—输入—输出—解决方案」四段展开，十章覆盖哈希表、树与递归、记忆化与动态规划、图与广度优先搜索、加权图最短路、二分搜索、堆与线段树、并查集、随机化算法。示例代码用 C 写，但刻意回避语言特性，C++、Java、Python 都能直接迁移——对初中起步、语言习惯还没定型的选手友好。与国内多数题解书最大的差别在「解决方案」这一段：先推「为什么这样建模、暴力卡在哪、换哪个不变量才能降复杂度」，再落到代码，因此更适合在学完模板之后练「拿到题先想什么」，而不是当模板手册查。局限要说清楚：28 道题撑不起完整知识体系，图论只到 BFS 与最短路，网络流、树剖、数论进阶一概没有；C 代码需要读者自己改写成考场写法（快读、STL 容器、long long），照抄会不适应；章节按「数据结构」而非「考点难度」组织，想按省选考点检索时不如专题书顺手。

https://book.douban.com/isbn/9787115683649/

300 **算法竞赛入门经典——训练指南** 白书 题解

刘汝佳、陈锋 · 清华大学出版社 · 2012

CSP-S NOIP 专题突破 实战训练 实战 纸质书

紫书的配套题集，按专题成组给题。适合「学完一块马上打透一块」。

301 **JOI 2025/2026 二次预选（第 25 回日本信息学奥赛第二轮）题目·解说·数据** 真题

日本信息学奥林匹克委员会（IOI-JP） · 2025

NOIP 省选 赛前冲刺 实战训练 实战 方法论

在线页面（免费下载题面 HTML、解说 PDF、C++/Python 解答例与 199MB 数据 ZIP）

2025 年 12 月 7 日 13:00—16:00 线上举行，6 题各 100 分共 600 分，题面、逐题解说 PDF、C++ 与 Python 解答例、评分用输入输出数据（ZIP 199MB）全部公开，另有 AtCoder 比赛页（joi2026-yo2）。官方统计给得很全，正好可以用来判断难度梯度：有资格者 1379 人，A 级线 390 分、147 人过线（高校二年级以下 129 人、三年级 18 人），JOI 部门进 132 人、JOIG 部门线 146 分进 32 人、特别部门 390 分进 28 人；总分分布为 600 分 20 人、500—599 分 43 人、400—499 分 77 人、300—399 分 141 人、200—299 分 134 人、100—199 分 277 人、1—99 分 549 人、0 分 138 人。逐题平均分是 57.8 / 41.7 / 31.7 / 13.4 / 7.7 / 3.2，第 5、6 题满分人数只有 74 与 20——这是一份「前四题按 NOIP 难度递进、后两题直逼省选压轴」的标准梯度卷。命题者为米田宽峻与米田优峻（第 1 题）、米山瑛士（第 2、3 题）、平木康杰（第 4 题）、蜂矢伦久与米山瑛士（第 5 题）、蜂矢伦久（第 6 题），米田优峻即《算法竞赛黄金法则》作者，其命题风格（长题面、强子任务）与书中例题一致。用法：6 题 3 小时、按子任务给分的赛制与国内二轮认证接近，可直接当模拟赛；官方解说 PDF 给出满分做法与部分分策略，比只给代码的题解更适合讲评。与已收《JOI 2026/2027 一次预选》互补——一次预选 80 分钟考语法与基础算法，二次预选才是真正的算法竞赛难度，两页连起来正好覆盖日本选拔的两级。局限：题面为日文（可配合洛谷 P14795—P14800 的中文翻译），解说 PDF 亦为日文；数据 ZIP 近 200MB，校内分发需先裁剪。

https://www2.ioi-jp.org/joi/2025/2026-yo2/index.html

302 **NOIP 2025 题目及数据（糖果店 / 清仓甩卖 / 树的价值 / 序列询问）** 真题

CCF NOI 官网 · 历年资料 · 2025

NOIP 省选 实战训练 赛前冲刺 实战 免费下载（CC BY-NC 授权）

2025 年 11 月 29 日举行的 NOIP 2025 的官方题面与全量测试数据，2025 年 12 月 15 日与其他年度资料一同在 NOI 官网「历年资料」栏目公开。联赛只有一天四题：candy（糖果店）、sale（清仓甩卖）、tree（树的价值）、query（序列询问），题面合并在一个 PDF 里，每题另有独立的数据 zip 包，结构比 OJ 上的搬运版干净得多——子任务划分与官方分值、输入输出文件名、编译选项都写在题面首页，可以直接按 NOI Linux 2.0 的评测方式在本地复现分数。用法上，NOIP 真题的价值有两层：一是它是省选资格的直接门槛（按 CCF 关于 NOI 2027 省内选拔的规定，NOIP 成绩在省选总成绩中的权重不得低于 30%），二是四题的难度梯度非常稳定——前两题是「稳分题」，第三题常是区分度所在，第四题是省队线，按这个梯度做限时训练，比零散刷题更能校准「一道题该给我多少分」的自我判断。建议的练法：先按 3.5 小时整套限时做，再只对第三、四题做「不写代码的口头解法」训练——这两题失分的主因往往不是不会，而是没在赛时想到正解方向。局限：官网只给题面与数据，不含官方题解；中文讲解要另找（洛谷题号、各省教练的赛后分析），且不同来源的子任务分值与官方偶有出入，对分数时以本页数据包为准。

https://www.noi.cn/zxzy/lnzl/jszl/2025-12-15/854083.shtml

303 **NOIP 历年真题（2011 起）** 真题

CCF · noi.cn / 洛谷 · 2011

NOIP 实战训练 赛前冲刺 实战 免费

提高组与普及组各一套/年。NOIP 是「最像考题的题」，做完十年卷子对题型分布的直觉就建立了。

https://www.luogu.com.cn/problem/list

304 **NOIP 历年试题** 真题

CCF · noi.cn / 洛谷 · 1995-至今

NOIP 赛前冲刺 实战 官网 / 题库

提高组真题是最贴近省一线难度的材料，建议按年份整套限时做。

https://www.luogu.com.cn/

305 **二分图匹配（匈牙利 / Hopcroft–Karp / KM）** 题解

OI Wiki · oi-wiki.org · 2018

NOIP 省选 NOI 专题突破 深耕 免费

二分图最大匹配、最小点覆盖、最大独立集、最小路径覆盖四个定理必须能互相推导出来。

https://oi-wiki.org/graph/bi-graph/

306 **博弈论与 SG 函数** 题解

OI Wiki · oi-wiki.org · 2018

NOIP 省选 NOI 专题突破 深耕 免费

Nim、Anti-Nim、阶梯博弈、SG 定理。读贾志豪《组合游戏略述——浅谈SG游戏的若干拓展及变形》。

https://oi-wiki.org/math/game-theory/

431 **数位 DP 专题** 题解

OI Wiki · oi-wiki.org · 2018

NOIP 省选 专题突破 深耕 免费

按位确定 + 记忆化。套路性极强，属于「学会就白送分」的专题，性价比最高的一类。

https://oi-wiki.org/dp/number/

308 **状压 DP 专题** 题解

OI Wiki · oi-wiki.org · 2018

NOIP 省选 NOI 专题突破 深耕 免费

子集枚举、轮廓线、插头 DP 的入口。核心是把「集合」编码成整数，再谈转移优化。

https://oi-wiki.org/dp/state/

309 **矩阵与线性递推** 题解

OI Wiki · oi-wiki.org · 2018

NOIP 省选 专题突破 速查 免费

矩阵快速幂 + 常系数齐次递推的 BM 优化。看到「1e18 项递推」就该条件反射想到它。

https://oi-wiki.org/math/linear-algebra/matrix/

310 **线段树与懒标记** 题解

OI Wiki · oi-wiki.org · 2018

NOIP 省选 NOI 专题突破 深耕 免费

区间操作的通用武器。标记下传的顺序（先乘后加）是唯一的坑，写清楚注释就不出错。

https://oi-wiki.org/ds/seg/

311 **2-SAT 与差分约束** 题解

OI Wiki · oi-wiki.org · 2018

省选 NOI 专题突破 速查 免费

两类「把逻辑/不等式转成图」的模型。2-SAT 用 Tarjan 判可行并输出方案，模板固定。

https://oi-wiki.org/graph/2-sat/

312 **AC 自动机** 题解

OI Wiki · oi-wiki.org · 2018

省选 NOI 专题突破 深耕 免费

Trie + fail 树。多模式串匹配的标准解，配合 fail 树上 dfs 序 + 树状数组能解决大部分变种。

https://oi-wiki.org/string/ac-automaton/

313 **ACM 国际大学生程序设计竞赛系列·题目与解读** 题解

俞勇 主编 · 上海交通大学出版社 · 2013

省选 NOI 实战训练 专题突破 实战 纸质书

第三册是题集，按专题给题并附解读。题量大、难度梯度平缓，适合当省选阶段的补充题库。

314 **AHOI 安徽省选** 真题

安徽省队选拔 · 洛谷题库 · 2005

省选 实战训练 实战 免费 / 洛谷题库搜 AHOI

难度中等偏上，数据结构和图论题较多。省选初期用来积累「完整做出三题」的信心很合适。

https://www.luogu.com.cn/problem/list

315 **APIO 2026 中国区活动题目及数据** 真题

中国计算机学会 · 2026

省选 NOI 实战训练 实战 免费

2026 年 5 月 10 日发布，含 APIO 2026 中国区试题、数据与讲评资料。APIO 以交互题、构造题比重高著称，与国内赛制差异明显，是训练「非传统题型」最经济的官方来源。

https://www.noi.cn/zxzy/lnzl/jszl/2026-05-10/890784.shtml

316 **BJOI 北京队选拔** 真题

北京市队选拔 · 洛谷题库 · 2005

省选 NOI 赛前冲刺 实战 免费 / 洛谷题库搜 BJOI

难度偏高、创新题多。北京队名额少，卷子历来不手软，是省选后期的拔高题源。

https://www.luogu.com.cn/problem/list

317 **BOI 试题库与 2026 年题目（波罗的海信息学奥林匹克，1995—2026 全量题面、数据与题解）** 真题

BOI 组委会 / CSES 官方档案 · 2026

省选 NOI 实战训练 专题突破 实战 免费（题面 PDF、测试数据 ZIP、题解 ZIP，可在 CSES/oj.uz 提交）

boi.cses.fi 是波罗的海信息学奥林匹克（Baltic Olympiad in Informatics）的官方档案站，把 1995—2026 共 30 余届的题面、测试数据与题解按年份整表列出，每届固定给「day 1 / day 2 题面 PDF + tests.zip + solutions.zip」四件，是全库唯一一条能一次性补齐「连续三十年、每年 6 题、带官方数据与标程」的题源。2026 年那届在芬兰埃斯波阿尔托大学奥塔涅米校区举行（4 月 15—19 日，比赛日 4 月 16、17 日），受邀国 11 个（丹麦、爱沙尼亚、芬兰、德国、冰岛、拉脱维亚、立陶宛、挪威、波兰、瑞典、乌克兰）。六题是 Day1 的 Blocks、Island、Tourist's Journey 与 Day2 的 Distances、Hamilton（交互题）、Sort；子任务数分别为 4、2、3、4、交互、1。2026 年成绩：波兰 Artur Smoleński 597 分居首，前四名全为波兰选手，波兰在 66 名选手、11 国中形成集团优势。用法上有一条明确建议：BOI 的子任务切得比国内细（多的一题四档），且低档暴力往往只值 1—2 个子任务分，正好练 23.5 讲的「按子任务配算法」；题目文本短、思维要求高、代码量小，与 CEOI 同属「省选到 NOI 之间」的过渡难度。局限：档案站只有英文题面，无中文翻译与逐题题解讲解（solutions.zip 给的是标程与简要说明），需要自己补；早期年份（1995—2005）题目风格偏旧，训练价值集中在 2015 年之后。

https://boi.cses.fi/tasks.php

318 **CEOI 2026 试题与数据（第 33 届中欧信息学奥林匹克，斯洛文尼亚卢布尔雅那）** 真题

CEOI 2026 组委会（斯洛文尼亚 ACM） · 2026

省选 NOI 实战训练 赛前冲刺 实战 免费（题面与数据在 oj.uz，官方站给赛程与规章）

第 33 届中欧信息学奥林匹克，2026 年 7 月 5—10 日在卢布尔雅那大学计算机与信息科学学院（UL FRI）举行，由斯洛文尼亚 ACM 承办，评测用自研系统 Putka。两天各 5 小时 3 题，六题是 Day1 的 Birdwatchers、DFS、Treasure Hunt 与 Day2 的 Flower Cutting、Towers、Vim。规模：59 名选手、15 国（8 个正式成员国外加客队）。要提醒的是资料分布——官方网站 ceoi2026.acm.si 只放赛程、规章、场地与参赛名单，题面与测试数据并不在上面，实际可做题的地方是 oj.uz 的 CEOI26 题组（CEOI26_birdwatchers、dfs、hunt、flowers、towers、vim，均带完整子任务与评测）。难度与 NOI 相当而风格更「短代码 + 强思维」：六题里 DFS 与 Vim 都是 30 行内可写完但需要先想出关键性质的类型，适合当省选前的限时套题。与 28.13 竞赛图、27.13 重链剖分等节配合使用时，CEOI 题常是「模板 + 一个观察」的形态，正好检验模板是否真的内化。局限：无官方中文题面与题解（oj.uz 只有英文），赛后讲评视频未公开。

https://oj.uz/problems/source/ceoi2026

319 **DP 优化专题（单调队列/斜率/决策单调性）** 题解

OI Wiki · oi-wiki.org · 2018

省选 NOI 专题突破 深耕 免费

把 $O(n^2)$ 打到 $O(n \log n)$ 或 $O(n)$。省选与 NOI 的分水岭，也是集训队论文出现频率最高的主题。

https://oi-wiki.org/dp/opt/

320 **FJOI 福建省选** 真题

福建省队选拔 · 洛谷题库 · 2005

省选 实战训练 实战 免费 / 洛谷题库搜 FJOI

题面长、信息量大，考查「从长题面里抽模型」的能力。近年与联合省选并轨，早期卷子仍有价值。

https://www.luogu.com.cn/problem/list

321 **GDOI 广东省选** 真题

广东省队选拔 · 洛谷题库 · 2005

省选 实战训练 实战 免费 / 洛谷题库搜 GDOI

本省选手的必刷卷。广东题偏应用，与 NOIP 提高组衔接较顺，适合作为省选入门的第一批真题。

https://www.luogu.com.cn/problem/list

322 **HAOI 河南省选** 真题

河南省队选拔 · 洛谷题库 · 2005

省选 实战训练 实战 免费 / 洛谷题库搜 HAOI

题风朴实，少有偏题，适合作为限时套卷的「热身卷」。

https://www.luogu.com.cn/problem/list

323 **HEOI 河北省选** 真题

河北省队选拔 · 洛谷题库 · 2005

省选 实战训练 实战 免费 / 洛谷题库搜 HEOI

部分年份与河南联考。数据结构题实现量偏大，练「代码耐力」的好材料。

https://www.luogu.com.cn/problem/list

324 **HNOI 湖南省选** 真题

湖南省队选拔 · 洛谷题库 · 2005

省选 NOI 实战训练 实战 免费 / 洛谷题库搜 HNOI

思维题比例高，常出现需要「转化模型」的题。湖南是竞赛强省，题目质量整体在线。

https://www.luogu.com.cn/problem/list

325 **JSOI 江苏省选** 真题

江苏省队选拔 · 洛谷题库 · 2005

省选 实战训练 实战 免费 / 洛谷题库搜 JSOI

风格稳健，难度分布均匀，很少出偏怪题。适合作为省选第一轮系统训练的题源。

https://www.luogu.com.cn/problem/list

326 **Manacher 与回文自动机** 题解

OI Wiki · oi-wiki.org · 2018

省选 NOI 专题突破 速查 免费

2025 大纲把 Manacher 从 NOI 级下放到提高级，必须提前学。PAM 用来统计回文出现次数。

https://oi-wiki.org/string/manacher/

327 **SCOI 四川省选** 真题

四川省队选拔 · 洛谷题库 · 2005

省选 NOI 实战训练 专题突破 实战 免费 / 洛谷题库搜 SCOI

DP 与计数题见长，部分年份难度接近 NOI。想练「推式子」就该从 SCOI 挑。

https://www.luogu.com.cn/problem/list

328 **SDOI 山东省选** 真题

山东省队选拔 · 洛谷题库 · 2005

省选 NOI 实战训练 专题突破 实战 免费 / 洛谷题库搜 SDOI

题量大、套路全、数据结构比重高。是「按专题刷省选题」时最好的骨架题源。

https://www.luogu.com.cn/problem/list

329 **SHOI 上海队选拔** 真题

上海市队选拔 · 洛谷题库 · 2005

省选 NOI 赛前冲刺 专题突破 实战 免费 / 洛谷题库搜 SHOI

以构造、交互、数学味著称。做 SHOI 常能补上「只会套路不会想」的短板。

https://www.luogu.com.cn/problem/list

330 **WEOI 2026 试题与题解（第 4 届西欧信息学奥林匹克，卢森堡）** 真题

WEOI 2026 组委会 · 2026

省选 NOI 实战训练 赛前冲刺 实战 免费（4 道题题面 PDF + 部分官方题解 PDF）

第 4 届西欧信息学奥林匹克（Western-European Olympiad in Informatics），2026 年 6 月 26—28 日在卢森堡举行，线下赛场在 LGL、同时接受线上参赛的混合形式；正式比赛只有一天（6 月 27 日 13:00—17:00，四小时四题），另设一场练习赛。四道题是 Rybina（搜索）、Folding（折叠）、Machine（寄存器机器）、Landmarks（地标），英文题面四份全部公开，官方题解目前只发了 Folding 与 Rybina 两份，官网写明其余「即将发布」。命题人分工也公开：Hanks Chong 出 Folding 与 Landmarks、Samuel Trajtenberg 出 Machine、Jeroen Krijgsman 出 Rybina，科学委员会主席 Daniel Murphy。参赛队按国家组织，官网列出的十支队伍是比利时、法国、爱尔兰、意大利、卢森堡、荷兰、葡萄牙、西班牙、瑞士、英国，金银铜牌名单与完整成绩榜（weoi.org/results/2026/）都可查。用法上它是「英文原题 + 短题面」这一类的典型：四题覆盖搜索、模拟/构造、状态机与几何，题面普遍一页内讲完，没有子任务提示，靠自己想清楚性质——适合省选前当「限时英文套题」练，也适合当 NOI 前检验读题能力的材料。局限：只有 4 题单场，题量不足以当完整套题；题解只发了两份，剩下两题要自己补证明；国内题站未收中文翻译。

https://weoi.org/weoi-2026/

331 **ZJOI 浙江省选** 真题

浙江省队选拔 · 洛谷题库 · 2005

省选 NOI 赛前冲刺 专题突破 实战 免费 / 洛谷题库搜 ZJOI

公认难度天花板之一，出题风格偏重思维与实现细节。ZJOI 能稳定做出两题，省队基本稳。

https://www.luogu.com.cn/problem/list

332 **十二省联考** 真题

多省联合命题 · 洛谷题库 · 2019

省选 NOI 赛前冲刺 实战 免费 / 洛谷题库搜十二省联考

2019、2020 两届，规模最大的一次省际联考。题目质量高、区分度好，是省选模拟的黄金卷。

https://www.luogu.com.cn/problem/list

333 **可持久化数据结构** 题解

OI Wiki · oi-wiki.org · 2018

省选 NOI 专题突破 深耕 免费

主席树、可持久化并查集、可持久化 Trie。核心思想只有一句「只复制被修改的结点」。

https://oi-wiki.org/ds/persistent-seg/

334 **后缀数组与后缀自动机** 题解

OI Wiki · oi-wiki.org · 2018

省选 NOI IOI 专题突破 深耕 免费

SA 求 height 数组、SAM 求本质不同子串。先读罗穗骞《后缀数组——处理字符串的有力工具》再动手。

https://oi-wiki.org/string/sa/

335 **国家集训队论文配套题** 真题

历年集训队 · 论文集 · 1999-2017

省选 NOI 专题突破 实战 论文集

论文里的例题往往就是当年最难的原题，读完论文顺手做掉。

336 **多项式与 NTT** 题解

OI Wiki · oi-wiki.org · 2018

省选 NOI 专题突破 深耕 免费

卷积、求逆、开根、exp、ln 全套。模板长，务必封装成结构体并测过随机数据。

https://oi-wiki.org/math/poly/ntt/

337 **平衡树（Treap / Splay / FHQ）** 题解

OI Wiki · oi-wiki.org · 2018

省选 NOI 专题突破 深耕 免费

有序集合的全功能实现。先写无旋 Treap（FHQ），再学 Splay 的区间操作，最后看替罪羊树。

https://oi-wiki.org/ds/treap/

338 **插头 DP（连通性状态压缩）** 题解

OI Wiki · oi-wiki.org · 2018

省选 NOI 专题突破 深耕 免费

棋盘上的回路/路径计数专用。模板固定，但状态编号（最小表示法）必须自己写熟一遍。

https://oi-wiki.org/dp/plug/

339 **树链剖分（HLD）** 题解

OI Wiki · oi-wiki.org · 2018

省选 NOI 专题突破 深耕 免费

把树上路径问题转成序列问题。与线段树组合后能解决一大类「路径修改 + 路径查询」。

https://oi-wiki.org/graph/hld/

340 **网络流（最大流 / 最小割 / 费用流）** 题解

OI Wiki · oi-wiki.org · 2018

省选 NOI IOI 专题突破 深耕 免费

建模远比算法难。先刷「网络流 24 题」建立模型直觉，再读胡伯涛《最小割模型在信息学竞赛中的应用》。

https://oi-wiki.org/graph/flow/

341 **联合省选（CCF 统一命题）** 真题

CCF · noi.cn · 2020

省选 赛前冲刺 实战训练 实战 免费 / 洛谷题库

2020 年起省选由 CCF 统一命题，分 A/B 卷。这是目前省选训练的第一题源，两年内的卷子必须整套限时做。

https://www.luogu.com.cn/problem/list

342 **莫比乌斯反演与杜教筛** 题解

OI Wiki · oi-wiki.org · 2018

省选 NOI 专题突破 深耕 免费

数论题的必经之路。先把 ∑ μ(d) 的套路推熟，再看杜教筛的 O(n^{2/3}) 复杂度分析。

https://oi-wiki.org/math/number-theory/mobius/

343 **虚树** 题解

OI Wiki · oi-wiki.org · 2018

省选 NOI 专题突破 深耕 免费

2025 大纲由 10 级降到 8 级，重要度上升。多次询问少量关键点的树形 DP 必用。

https://oi-wiki.org/graph/virtual-tree/

344 **计算几何基础与凸包** 题解

OI Wiki · oi-wiki.org · 2018

省选 NOI 专题突破 深耕 免费

叉积、旋转卡壳、半平面交、最小圆覆盖。精度是最大敌人，能用整数就不用浮点。

https://oi-wiki.org/geometry/

345 **APIO 历年试题** 真题

APIO · apio 官网 / oj.uz · 2007-至今

NOI 赛前冲刺 实战 官网 / oj.uz

亚太赛，难度介于 NOI 与 IOI 之间，是中国选手的重要练兵场。

https://oj.uz/

346 **EGOI 2026 试题与逐题统计（第 6 届欧洲女子信息学奥林匹克）** 真题

EGOI 2026 组织委员会 / EGOI 统计站 · 2026

NOI IOI 实战训练 专题突破 实战 免费（题面与测试数据在线，附逐题平均分与满分人数）

2026 年 5 月 12—18 日在意大利切塞纳蒂科（Cesenatico）举行，两天各 4 题、每题 100 分、满分 800；59 个欧洲及非欧洲国家 250 名选手线下参赛，另有 9 国 33 人线上同步。八题与赛场数据（平均分 / 满分人数）：Day 1 为 ferriswheel 87.02 / 203、ovenmasters 40.75 / 36、biscuits 25.78 / 2、census 9.29 / 0；Day 2 为 wateringplants 82.85 / 174、cakes 60.59 / 76、foxfamilies 33.81 / 11、seatingplan 9.11 / 0。统计站把每题的分数分布、子任务满分率、各国选手逐题得分全部公开，这在 OI 类赛事里少见——可以据此判断「这题我卡住是常态还是异常」。用法：平均 80 以上的两题（ferriswheel、wateringplants）当读题与实现速度的热身；20—40 区间的四题（ovenmasters、biscuits、foxfamilies、cakes）适合专题突破，它们的考点集中在构造、贪心与树上计数；满分 0 人的 census 与 seatingplan 留作限时挑战，做不出属正常。整体难度介于 NOI 与 IOI 之间，题型分布更接近 IOI（构造题与子任务给分的比例明显高于国内赛）。局限：全英文题面，无中文翻译；部分题目为交互或构造，需按题面编译交互库；与 NOI 大纲不完全对齐，CSP/NOIP 段不适用。第 7 届（EGOI 2027）将在波兰华沙举行。

https://stats.egoi.org/tasks/

347 **IOI 2025 官方赛题包（ioi/task-archive 2025，6 题全量数据）** 真题

IOI 官方（GitHub ioi/task-archive 仓库） · 2025

NOI IOI 赛前冲刺 实战训练 实战 深耕 免费（开放仓库，可 clone 全量数据）

第 37 届国际信息学奥林匹克（IOI 2025，2025 年夏在玻利维亚举行）的官方赛题包，托管在 IOI 官方 GitHub 仓库 task-archive 的 2025 目录下。Day1 三题：souvenirs、triples、worldmap；Day2 三题：festival、migrations、obstacles。每题一个子目录，结构统一：statement.pdf（英文题面）、problem.json（子任务划分与分值）、tests/（测试数据）、subtasks/、checker/（testlib 评测器）、graders/（交互与通信题的评测接口）、solutions/（官方标程）、attachments/（附加文件）。该目录最后一次提交在 2026 年 8 月 14 日，说明题解与数据已补齐，不是赛时的残缺版本。用法：这是少数能在本地 1:1 复现国际赛评测的资料——在 NOI Linux 2.0 上用 testlib 编译 checker、按 problem.json 分子任务跑分，可以精确得到「我拿了几分、丢在哪个子任务」，比在 OJ 上只看到一个总分有用得多；交互/通信题要把 graders 里的接口编进自己的程序，这一步是很多人卡住的地方。与 2026 年塔什干的赛题包（ioi2026.uz）同一体系，两年连着做能看出 IOI 近两年的题型配比变化（构造与交互比重上升）。局限：全英文，仓库不含中文题解与翻译；洛谷、oj.uz 有对应题但题号与子任务分值与官方不完全一致，对分数时以本仓库为准；单题数据包体积不小，clone 时建议用 --depth 1 或按目录稀疏检出。

https://github.com/ioi/task-archive/tree/master/2025

348 **IOI 2026 中国国家集训队集中培训与精英培训题目及数据** 真题

中国计算机学会 · 2026

NOI IOI 专题突破 实战训练 实战 深耕 免费

2026 年 1 月 22 日发布。第 38 届 IOI 中国国家集训队集中培训与精英培训（2025 年 12 月 1—5 日，北京大学）三天共 9 题：异形工厂（shapez）、字符串问题（string）、三选二（three）、深红（art）、小丑大师的荣耀（balatro）、奇迹（miracle）、无题（nameless）、复读机（repeater）、解开尘封的序列（sequence），题面 PDF 与测试数据以 CC BY-NC 授权开放下载。难度对标国家队选拔、整体高于 NOI，是省队以上选手少见的「非公开赛制」训练材料；建议按「3 题 5 小时、先写满部分分」的方式使用，而不是当普通题库刷。

https://www.noi.cn/zxzy/lnzl/jszl/2026-01-22/859428.shtml

349 **IOI 2026 试题与题解（第 38 届国际信息学奥林匹克，塔什干）** 真题

IOI 2026 组织委员会 / IOI 官方题库 · 2026

NOI IOI 实战训练 专题突破 实战 免费（CC-BY 4.0，含题面、附件与测试数据）

2026 年 8 月 9—16 日在乌兹别克斯坦塔什干举行的第 38 届 IOI，赛后官网把全部题目按 CC-BY 4.0 开放：正赛两天各 3 题、每题 5 小时 —— Day 1 为 Ball Machine（Andy Wu，澳大利亚）、Monuments（米田雅崇、米田博孝，日本）、Tiling Game（Juheon Song，韩国）；Day 2 为 Classroom Game（Samuel Trajtenberg、Boris Hall，英国）、Magic City（米田兄弟，日本）、Partition（YoungBin Park，韩国）。另有 4 道练习赛题（A + B Queries、Duplicated Binary Strings、Boring Game、Magic Trick）用于熟悉交互与评测环境。每道题给三样东西：官方英文题面 PDF、附件包（样例与交互库/头文件），以及托管在 GitHub ioi/task-archive 的完整测试数据与补充材料——这一条最关键，意味着可以本地搭建评测、按子任务逐档验证部分分，而不是只能读到题面。用法上它属于「上限参照」而非训练主线：IOI 题的思维密度远高于 NOI，直接开做容易卡死，正确用法是赛前专题突破时挑一两道，先读题面自己想 40 分钟、写下子任务划分，再看测试数据规模验证复杂度判断，最后对照各国选手的赛场实现（stats.ioinformatics.org 有逐题得分分布）看差距在哪一档。局限：全英文，题面长且细节多（交互题与构造题占比高），没有官方中文翻译；难度与 NOI 大纲不对齐，CSP/NOIP 段完全不适用；附件里的交互库需要按题面说明编译，Windows 下容易卡在编译脚本上，建议在 NOI Linux 2.0 环境里做。

https://ioi2026.uz/tasks

350 **Min_25 筛与洲阁筛** 题解

OI Wiki · oi-wiki.org · 2018

NOI 专题突破 深耕 免费

亚线性求积性函数前缀和。属于「记住了就能用、推不出来」的类型，建议直接背模板 + 理解两轮递归。

https://oi-wiki.org/math/number-theory/min-25/

351 **NOI 2026 题目及数据** 真题

中国计算机学会 · 2026

NOI 实战训练 赛前冲刺 实战 免费

2026 年 7 月 27 日随赛后资料发布，含 NOI 2026 全部试题目面、测试数据与题解。NOI 级真题的测试数据尤其难得，可用于自建评测、验证部分分策略；建议在省选后按「限时 5 小时、全真环境」使用。

https://www.noi.cn/zxzy/lnzl/jszl/2026-07-27/918389.shtml

352 **NOI 冬令营 2026 题目及数据** 真题

中国计算机学会 · 2026

NOI 省选 实战训练 赛前冲刺 实战 免费

2026 年 2 月 12 日发布。冬令营试题历来是当年命题风格的风向标，难度介于省选与 NOI 之间，且常有新题型首次出现。适合省选前一个月做一次限时模拟，用于校准「难题能拿多少部分分」。

https://www.noi.cn/zxzy/lnzl/jszl/2026-02-12/861342.shtml

353 **NOI 历年真题** 真题

CCF · noi.cn · 1984

NOI 赛前冲刺 实战 免费 / 洛谷题库

两天六题制，难度陡增。建议在省队集训后期成套做，且必须按 5 小时/场严格计时。

https://www.noi.cn/xw/

354 **NOI 历年试题** 真题

CCF · noi.cn · 1984-至今

NOI 赛前冲刺 教练备课 实战 官网 / 题库

正赛原题。做近十年即可，更早的题风格差异大。

https://www.noi.cn/

355 **生成函数与组合计数** 题解

OI Wiki · oi-wiki.org · 2018

NOI IOI 专题突破 深耕 免费

OGF/EGF、卡特兰、斯特林、拆分数。计数题从「硬推」升级到「套模型」的分界线。

https://oi-wiki.org/math/poly/egf/

356 **IOI 历年试题** 真题

IOI · ioinformatics.org · 1989-至今

IOI 赛前冲刺 实战 官网 / oj.uz

国际赛原题，交互与构造题比例高。冲国家队必刷。

https://ioinformatics.org/

10 第 10 章　课程与视频

> 视频课解决的是「第一次接触时的理解障碍」，不解决熟练度。选一门跟完，其余只看单节。本章按国内课程、海外公开课、平台教程三类编排。

本章收录 37 条，编号 357—393。

357 OI Wiki 专题页

课程

OI Wiki 社区 · oi-wiki.org · 2018

通用 查漏补缺 速查 免费

严格说不是课程，但每个专题页都有「例题 + 代码 + 复杂度」，当微课程用完全够。

https://oi-wiki.org/

358 AcWing 算法基础课

课程

闫学灿（yxc） · acwing.com · 2018

CSP-J CSP-S 入门自修 体系学习 深耕 付费

从排序、二分、高精度讲到图论、DP，每节配 5–10 道题。零基础一年内的主线课，公认性价比最高。

https://www.acwing.com/activity/content/11/

359 USACO Guide

课程

USACO Guide 社区 · usaco.guide · 2020

CSP-J CSP-S NOIP 省选 体系学习 查漏补缺 深耕 方法论 免费

把知识点、讲解、配套题、难度分级串成一条路径。英文过关的话，这是最好的免费课程体系。

https://usaco.guide/

360 USACO Training Gateway

课程

USACO · train.usaco.org · 1998

CSP-J CSP-S 入门自修 实战 免费

老牌章节式训练系统，按章解锁。节奏慢但地基扎实，适合耐得住性子的初学者。

https://train.usaco.org/

361 中国大学 MOOC

课程

高等教育出版社 · icourse163.org · 2014

CSP-J CSP-S NOIP 体系学习 深耕 免费（部分认证收费）

国内高校的数据结构、算法、离散数学课程。想补「科班底子」又不想读英文时，从这里挑一门跟完。

https://www.icourse163.org/

362 **代码随想录** 课程

程序员 Carl · programmercarl.com · 2020

CSP-J CSP-S 入门自修 体系学习 实战 免费（开源）

以题带学的系统路线图，每题都有动画与多语言实现。入门期最容易坚持下来的一套材料。

https://programmercarl.com/

363 **洛谷网校** 课程

洛谷 · luogu.com.cn · 2019

CSP-J CSP-S NOIP 入门自修 赛前冲刺 实战 部分付费

中文体系化课程，与题库打通。自学能力弱的选手可以借它建立节奏。

https://www.luogu.com.cn/

364 **董晓算法（B 站）** 课程

董晓 · B 站 · 2019

CSP-J CSP-S NOIP 入门自修 查漏补缺 速查 免费

单知识点短讲，画图细致、语速慢。哪里不会点哪里，是最适合「补洞」的中文视频资源。

https://space.bilibili.com/385421622

365 **2026 年第四期 NOI 教师培训（南京站，中级/初级）** 课程

中国计算机学会（NOI 官网） · 2026

CSP-S NOIP 省选 教练备课 方法论 线下（初级可线上），需报名

2026 年 10 月 20 日发布，南京外国语学校承办，2026 年 12 月 7—13 日举办（7 日下午报到）；中级仅限现场，初级可选现场或线上。这份通知的价值不在「报名」，而在它把八位一线教练的讲课专题摊开成了一张「大纲知识点 → 教学切入角度」的映射表：艾庆兴（西安铁一中）《从整除同余到计数原理》、张超（南京外国语学校）《树与图的建模与经典问题》、王晓鹏（广州大学附中）《从暴力到最优的思维跃迁》、诸一行（浙江余姚中学）《动态规划的状态设计与教学》、周祖松（重庆育才中学）《数据的容器与秩序》、肖然（北大附中）《CSP-J 考点规律与解题思路》、梁冠健（佛山石门中学）《二分与倍增的对数级智慧》、宋新波（北京海淀稻香湖学校）《化繁为简的算法思想》。看得出选题逻辑：数论—图论—搜索思维—DP—数据结构—初赛规律—二分倍增—算法思想，正好是省选段最难的八块，且每块都落在「怎么讲」而不是「讲什么」上。对教练的直接用法：拿这八个标题给自己排一学期专题课，或对照检查本校教案缺哪一块。局限：通知只给专题名与讲师单位，不给讲义、课件与录像，培训内容本身不公开；线下名额与报名条件见通知原文，且与教师资质等级挂钩。

https://www.noi.cn/jspx/2026-10-20/941705.shtml

366 **AcWing 算法提高课** 课程

闫学灿（yxc） · acwing.com · 2019

CSP-S NOIP 省选 专题突破 体系学习 深耕 付费

提高组到省选的核心专题：树形 DP、状压、单调队列、斜率优化、AC 自动机、网络流。跟完基础课再上。

https://www.acwing.com/activity/content/16/

367 **Berkeley CS61B（数据结构）** 课程

UC Berkeley · datastructur.es · 2014

CSP-S NOIP 省选 体系学习 深耕 免费（全套公开）

项目驱动的数据结构课，从零实现并查集、KD 树、 Trie。做完项目对「数据结构为什么这样设计」有体感。

https://sp21.datastructur.es/

368 **CS 97SI（斯坦福竞赛编程）** 课程

Stanford · stanford.edu · 2012

CSP-S NOIP 体系学习 深耕 免费讲义

专为竞赛设计的短课程，讲义精简，一天能刷完一个专题。

https://web.stanford.edu/class/cs97si/

369 **Codeforces EDU** 课程

Codeforces · codeforces.com/edu · 2019

CSP-S NOIP 专题突破 实战 免费

每个专题配讲解 + 阶梯题单，学完立刻判题。性价比最高的免费课程之一。

https://codeforces.com/edu/courses

370 **Codeforces EDU（算法教程）** 课程

Codeforces · codeforces.com · 2017

CSP-S NOIP 省选 专题突破 实战 免费

每个专题一节短课 + 若干阶梯题，带即时判题。后缀数组、线段树、DSU 等专题的「最短路径」。

https://codeforces.com/edu/courses

371 **Princeton Algorithms（Coursera）** 课程

R. Sedgewick, K. Wayne · Coursera · 2012

CSP-S NOIP 省选 体系学习 深耕 免费旁听

配套《Algorithms, 4th》。有自动判题与面试题，讲得慢但极扎实，适合打底子。

https://www.coursera.org/learn/algorithms-part1

372 **UCF COP 4516 Contest Problem Solving（中佛罗里达大学竞赛解题课）** 课程

University of Central Florida · 2025

CSP-S NOIP 省选 体系学习 教练备课 专题突破 方法论 实战 免费

Arup Guha 主讲的一学期竞赛解题课，2025 春季完整课表、讲义、样例代码、题单与赛题存档全部公开。15 周专题顺序：暴力→贪心→树→Trie→图（DFS/BFS）→图（MST/最短路）→数学（扩欧、前缀和、最大子段）→网络流（Ford-Fulkerson/Edmonds-Karp/Dinic）→DP 两讲（LCS/背包/LIS、编辑距离/矩阵连乘）→二维几何（含凸包）→二分应用→树状数组，每周配「Written Notes」一页纸讲义、Java/C++/Python 三语样例代码、练习题与个人赛/团队赛题目存档（附题解要点）。对教练的价值在可直接复用：讲义是 1 小时专题课的现成骨架，三语代码便于对照语言细节，contest archive 是成套团队赛。局限：ICPC 导向，图论与网络流比重高于 NOI 大纲，而线段树、平衡树、树套树等国内高频数据结构几乎不涉及，不能替代《算法竞赛进阶指南》；部分练习需 Kattis/UVA 账号提交。

https://www.cs.ucf.edu/~dmarino/progcontests/cop4516/spr2025/

373 **中国大学 MOOC《算法设计与分析》（北大）** 课程

北京大学 · icourse163.org · 2015

CSP-S NOIP 省选 体系学习 深耕 免费

中文授课、体系完整，覆盖分治、DP、贪心、图算法、NP 完全性。适合作为学校课程的补充。

https://www.icourse163.org/

374 **学堂在线** 课程

清华大学 · xuetangx.com · 2013

CSP-S NOIP 省选 体系学习 深耕 免费（部分认证收费）

清华系课程平台，邓俊辉《数据结构》正式版就在这里。配套习题与判题，比看视频更有效。

https://www.xuetangx.com/

375 **数据结构（清华）** 课程

邓俊辉 · 学堂在线 / 中国大学 MOOC · 2013

CSP-S NOIP 体系学习 深耕 免费

中文数据结构课的天花板。配套教材与习题解析，红线画得极清楚。

https://www.xuetangx.com/

376 **灵茶山艾府（B 站 / 题解）** 课程

灵茶山艾府 · B 站 / 洛谷 · 2020

CSP-S NOIP 省选 专题突破 查漏补缺 深耕 免费

周赛题解与专题精讲，思路清晰、代码干净。DP 与图论系列尤其值得跟着做。

https://space.bilibili.com/206214

377 **AtCoder Library Practice** 课程

AtCoder · atcoder.jp · 2020

NOIP 省选 查漏补缺 实战 免费

把常用数据结构与算法做成模板库，配套验证题。写完自己模板后来这里对答案。

https://atcoder.jp/contests/practice2

378 **MIT OCW 6.006（算法导论）** 课程

MIT · ocw.mit.edu · 2011

NOIP 省选 NOI 体系学习 深耕 免费（视频 + 讲义 + 作业）

完整公开课，讲得比 CLRS 好懂。配套 problem set 有解答，适合边看边练。

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/

379 **NOI 2025 冬令营授课与交流活动现场视频（14 场）** 课程

中国计算机学会（NOI 官网视频专栏） · 2025

NOIP 省选 NOI 教练备课 专题突破 方法论 深耕 免费

2025 年 4 月 23 日上线。第 42 届全国青少年信息学奥林匹克冬令营于 2025 年 1 月 17—24 日在绍兴举行（绍兴一中教育集团承办），第一、第二课堂共 14 场讲座全部录像。第一课堂偏理论与前沿：中科院计算所孙晓明「随机性与算法设计」、北大王迪「从模拟退火到概率编程」、清华郑勤锴「大模型时代的代码生成新范式」、清华毛嘉怡「稀疏图上更快的单源最短路算法」，以及 IOI 2024 国家队周康阳「浅谈非确定性算法」、沈吉濒「图论与连通性问题选讲」、郭羽冲「思维型题目选讲」、黄洛天「数据结构问题选讲」。第二课堂六场是纯考点课，主讲均为常年带出集训队选手的 NOI 名师：熊超「经典分治算法之美」、谷多玉「数据结构—线段树」、黄新军「网络流算法及其应用」、王晓鹏「线段树及其应用」、林鸿「容斥原理」、张新钢「浅谈贪心算法在信息学奥赛中的呈现」。对教练而言第二课堂可直接当课件骨架——线段树两讲（谷多玉讲结构与合并、王晓鹏讲应用）恰好覆盖 27.4 的「维护什么」与「怎么合并」两条主线，容斥与贪心两讲对应 29.3、24.6。第一课堂里毛嘉怡的稀疏图最短路（Dijkstra 在稀疏图上的优化与基数堆）是 28.2 的进阶补充，市面上教材很少写。局限：视频同样在 CCF 数字图书馆（合集 7456187809908736），页面只有清单没有时间轴；「大模型时代的代码生成」「概率编程」偏科研视野，备赛直接收益有限。

https://www.noi.cn/zxzy/hdsp/2025-04-23/841659.shtml

380 **算法设计与分析（MIT 6.006）** 课程

MIT OCW · ocw.mit.edu · 2011

NOIP 省选 体系学习 深耕 方法论 免费公开课

讲复杂度与算法正确性最严谨的入门课。英文授课，配讲义，适合补理论。

https://ocw.mit.edu/

381 APIO 2025 讲者授课现场视频（5 场）

课程

中国计算机学会（NOI 官网视频专栏） · 2025

省选 NOI IOI 专题突破 赛前冲刺 方法论 深耕 免费

2025 年 6 月 9 日上线。APIO 2025 中国区活动于 5 月 15—19 日在无锡举办（江苏省锡山高级中学承办），五场授课全部录像，讲者为 NOI 2022 金牌选手罗思远与代表中国出征 IOI 2025 的刘恒熙、刘海峰、陈昕阳、范斯喆。五讲题目：电阻网络和随机游走（罗思远）、博弈理论入门（刘恒熙）、图论问题的若干解题方法（刘海峰）、集合幂级数在子图计数问题上的应用（陈昕阳）、构造类问题的若干解题方法（范斯喆）。这份选题的针对性很强：「电阻网络 + 随机游走」是树上/图上期望的通用建模工具，能把「等概率游走求期望步数」这类题从列方程硬解变成解线性方程组或直接用有效电阻性质；「集合幂级数做子图计数」是 29.10 生成函数一节里集合幂级数（subset convolution）在计数题上的落地，属省选到 NOI 的硬考点；博弈与构造两讲则分别补 29.6 与 24 章之间的空白。与 2026 年那一辑（#323）互为前后届，两辑合看可以观察同一批讲者两年的选题迁移。局限：视频托管在 CCF 数字图书馆专栏（本页给合集入口 dl.ccf.org.cn/albumList/getMeetingDetail7541198265288704），逐场无时间轴；选手视角讲题偏「思路从哪来」，证明与实现细节仍需回教材补齐。

https://www.noi.cn/zxzy/hdsp/2025-06-09/844100.shtml

382 APIO 2026 中国区活动讲座现场视频（7 场）

课程

中国计算机学会（NOI 官网视频专栏） · 2026

省选 NOI IOI 专题突破 赛前冲刺 教练备课 方法论 深耕 免费

2026 年 6 月 9 日上线。APIO 2026 中国区活动于 5 月 7—11 日在北京举办（CCF 主办、北大附中承办），期间七场讲座全程录像，讲者为杨耀良（NOI 2017 金）、谢梓涵（北大，NOI 2023 金）、李静榕（清华，NOI 2024 金）与 IOI 2026 中国队四人许淇文、刘家炜、赵晟昊、何宇翔。七讲题目分别为：量化分析与流式因子计算基础；ICPC 构造题与 Ad-Hoc 题选讲；区间最值操作线段树介绍与复杂度证明；格路计数方法；莫队与范围修改查询问题；演绎推理类逻辑博弈；图论选讲——生成树的应用。这份目录本身就是一张省选到 NOI 级的知识地图：区间最值操作与复杂度证明（Segment Tree Beats 一脉）、格路计数（反射原理与行列式）、莫队配合范围修改、生成树的应用，四项都正好落在正编第 27—31 章的薄弱处，可与 27.4、31.6、29.3 对照看。两点局限：视频托管在 CCF 数字图书馆，需从本页链接跳转，部分条目要登录；讲者以选手视角切入，证明与实现细节不如 OI Wiki 系统，适合作为「知道该往哪深挖」的索引，不能替代教材。

https://www.noi.cn/zxzy/hdsp/2026-06-09/902042.shtml

383 AcWing 算法进阶课

课程

闫学灿（yxc） · acwing.com · 2020

省选 NOI 专题突破 深耕 付费

省选后期内容：平衡树、树套树、点分治、多项式、生成函数。难度陡，需先有 300 题以上积累。

https://www.acwing.com/activity/content/31/

384 **CS 3233（NUS 竞赛编程）** 课程

Steven Halim · NUS · 2010

省选 NOI 专题突破 深耕 免费讲义

《Competitive Programming》作者的课程，与红皮书配套，体系完整。

https://www.comp.nus.edu.sg/~stevenha/

385 **CCF NOI 冬令营 2026 第一课堂（CCF 数字图书馆）** 课程

中国计算机学会 · 2026

NOI 省选 专题突破 教练备课 深耕 方法论 公开链接

NOI 冬令营 2026 第一课堂与集训队选手论文交流的完整录像，2026 年 3 月上线 CCF 数字图书馆。含「几种数学理论在 OI 中的应用」「交互、通信选讲」「欧拉数问题」「如何写出没有 Bug 的程序——形式化证明」「大模型技术概览」等专题，讲者为国家集训队队员与授课教师，深度明显高于公开课。

https://dl.ccf.org.cn/video/videoDetail.html?id=7948566377547776

386 **CCF NOI 集训营 2026 报到通知（附日程）** 课程

中国计算机学会 · 2026

NOI IOI 赛前冲刺 教练备课 方法论 免费

2026 年 11 月 6 日发布。2026 全国青少年信息学奥林匹克国家队集训营于 12 月 7—11 日在北京举行，同期举办 IOI 2027 精英培训；活动地点北京航空航天大学（海淀区学院路 37 号），住宿与报到在北京唯实国际文化交流中心（学院路 39 号），报到时间为 12 月 7 日 9:00—14:20，当天下午即安排试机，住宿费由 CCF 承担。附「NOI 集训营 2026 日程」PDF 可直接下载。对不在营内的读者，价值在日程本身：它把国家队一周的训练节奏（每天几个专题、几场模拟、几轮讲评）完整暴露出来，教练照此排校内冬训的强度最省事；也标定了 12 月上旬这个窗口——NOIP（11 月 28 日）结束后两周即进入集训营，省队选手的冬训应从 12 月初就启动，而不是等到寒假。

https://www.noi.cn/xw/2026-11-06/946602.shtml

387 **CCF 关于举办 IOI 2027 精英培训的通知** 课程

中国计算机学会 · 2026

NOI IOI 赛前冲刺 教练备课 方法论 免费

2026 年 8 月 17 日发布：与 CCF NOI 集训营 2026 同期举办 IOI 2027 精英培训，培训期 2026 年 9—12 月，12 月在北京集中培训。报名门槛是「参加过 NOI 2026 全国赛」，CCF 按 NOI 2026 成绩遴选，须以学校为单位由一名指导教师统一报送（报名已于 2026 年 9 月 1 日 15:00 截止）。它的价值在于这是省队之外唯一能与国家集训队同步训练的通道：日常训练、作业、集中培训全同步，但不参与 IOI 2027 国家队选拔，结业发证书。对教练而言，它是判断「金牌线以下、省队以上」这一档选手上升路径的官方依据；对选手而言，它把 NOI 结束到次年省选之间的空窗期接上了高强度训练。

https://noi.cn/xw/2026-08-17/924463.shtml

388 **CMU 15-295 Competition Programming and Problem Solving** 课程

卡内基梅隆大学（Danny Sleator） · 2026

NOI IOI 专题突破 实战训练 实战 方法论 公开链接

由伸展树（Splay）与链路剖分的提出者 Danny Sleator 开设，2026 秋季学期仍在更新（f26 页面最后修改于 2026 年 10 月 29 日）。组织方式是「每周一个主题：30 分钟讲授 + 2 小时在 Codeforces 上打 3—5 题的课程赛」，讲义、C++ 代码、题解与讲座录像全部公开，主题按图搜索、最短路、最小生成树、网络流等经典块推进，另有 2015 年至今的全部历史学期页面。价值不在内容深度而在节奏设计——讲完立刻限时实战，可直接照搬为校队周训模板；题目本身都在 Codeforces 上可复现，无需翻墙找题面。

https://contest.cs.cmu.edu/295/

389 **CMU 15-451（算法设计与分析）** 课程

Carnegie Mellon University · cs.cmu.edu · 2015

NOI IOI 专题突破 教练备课 深耕 免费（讲义与作业）

概率分析、随机化、近似算法、摊还分析。想写高质量集训队论文，这里的工具最全。

https://www.cs.cmu.edu/~15451-f23/

390 **NOI 冬令营 2026 讲座与交流活动现场视频（14 场）** 课程

中国计算机学会（NOI 官网视频专栏） · 2026

NOI IOI 教练备课 专题突破 方法论 免费

2026 年 3 月 25 日上线。第 43 届全国青少年信息学奥林匹克冬令营于 2026 年 2 月 6—13 日在青岛举行（青岛城阳一中承办），第一、第二课堂合计 14 场讲座全部录像。第一课堂偏前沿与经验：中国人民大学赵鑫教授、清华大学王宏宁博士、北京大学熊英飞博士、Google 苏黎世钟诚，以及 IOI 2025 国家队刘恒熙、范斯喆、陈昕阳、刘海峰；第二课堂由六位 NOI 名师领衔——北京 101 中学董华星、杭州二中李建、人大附中梁霄、华南师大附中梁泽贤、深圳耀华实验刘溯、山大附中杨培君，内容是大纲知识点的系统梳理与赛题分析。同场还有国家集训队论文交流，队员报告集中在算法优化、数据结构创新与问题建模三条线。对教练的价值集中在第二课堂：这六位常年带出集训队选手，讲法即「怎么把知识点讲给学生听」，和只给结论的教材互补。局限：视频同样在 CCF 数字图书馆专栏内，页面只给合集入口，不提供逐场时间轴；第一课堂的四个高校报告面向科研视野，对备赛的直接帮助有限，不必按顺序全看。

https://www.noi.cn/zxzy/hdsp/2026-03-25/902063.shtml

391 **Stanford CS161（算法设计与分析）** 课程

Stanford University · web.stanford.edu · 2018

NOI IOI 体系学习 专题突破 深耕 免费（课件与作业）

分治、贪心、DP、网络流、NP 完全性的严谨课程。作业难度接近 NOI，讲证明的方法值得学。

https://web.stanford.edu/class/cs161/

392 **蒋炎岩（JYY）课程站** 课程

蒋炎岩（南京大学） · jyywiki.cn · 2019

NOI IOI 体系学习 教练备课 深耕 免费

操作系统与程序设计课程的全套讲义、视频、实验。讲「程序在机器上到底怎么跑」，对常数优化有降维效果。

https://jyywiki.cn/

393 **IOI 官方 YouTube 频道** 课程

IOI · YouTube · 2018

IOI 赛前冲刺 速查 免费

开幕式、颁奖、部分讲评与技术报告录像。了解国际赛氛围与命题取向的第一手材料。

https://www.youtube.com/@IOI_official

11 第 11 章　工具与环境

> 工具决定下限。考场环境不熟、编译器版本不对、不会对拍、不会看汇编，都会让本该拿到的分拿不到。本章按开发环境、评测系统、调试分析、辅助计算四类编排。

本章收录 24 条，编号 394—417。

394 Arbiter

工具

国内 OI 社区 · 开源 · 2009

通用 教练备课 速查 免费

与 Lemon 同期的评测系统，支持自定义评测脚本。

395 Cena

工具

国内 OI 社区 · 开源 · 2007

通用 教练备课 速查 免费

更早的评测系统，现在多见于老旧机房。了解即可。

396 DOMjudge

工具

DOMjudge 团队 · domjudge.org · 2004

通用 教练备课 速查 免费开源

ICPC 官方评测系统，办校内赛最正规的选择。

https://www.domjudge.org/

397 Lemon

工具

国内 OI 社区 · 开源 · 2010

通用 教练备课 速查 免费

老牌评测机，界面朴素但稳定，很多学校仍在使用。

398 CP Editor

工具

CP Editor 社区 · cpeditor.org · 2019

CSP-J CSP-S 实战训练 速查 免费开源

专为刷题设计：一键编译运行、自动抓样例、直接提交到多个 OJ。

https://cpeditor.org/

399 **Code::Blocks** 工具

Code::Blocks 团队 · codeblocks.org · 2005

CSP-J CSP-S 入门自修 速查 免费

轻量、跨平台，配置简单。比 Dev-C++ 稳定，适合替代。

https://www.codeblocks.org/

400 **Dev-C++** 工具

Bloodshed / Orwell · orwelldevcpp · 2015

CSP-J 入门自修 速查 免费

国内中学机房标配。功能够用，调试弱，只建议入门期使用。

https://github.com/Embarcadero/Dev-Cpp

401 **LOJ — LeLe Online Judge（开源可自部署的 ACM/OI 双赛制评测系统）** 工具

GitHub 开源项目 aiwandiannaodelele/LOJ（AGPL v3.0） · 2026

CSP-J CSP-S NOIP 实战训练 教练备课 实战

开源免费（Docker 一键部署，或 PM2 / Node.js 直接运行）

维护者于 2026 年 6 月 13 日在项目 Discussions #4 发布欢迎公告并列出特性，是本轮检索到的新评测工具。技术栈为 Next.js 16 + Prisma 7 + NextAuth v5 + Monaco 编辑器 + BlockNote 富文本；功能含题目系统（多测试点、自测样例）、ACM / OI 双赛制比赛（实时排名、公告、密码保护、独立题目副本）、训练题单、题目上下文感知的 AI 编程助教、可插拔评测引擎（OneCompiler 默认，可换 Judge0 / Runoob）、本地或 S3 兼容存储、以及限流/权限校验/IP 伪造与 SSRF 防护。部署给出 deploy.sh 一键脚本：Docker 默认 PostgreSQL、PM2 或裸 Node 默认 SQLite，也可上 Vercel / Netlify / Cloudflare Pages（配 Turso、Supabase 或 D1），首次访问 /init 建管理员。对学校的实际用途是自建校内 OJ：教练可独立出卷、按专题挂题单、组织模拟赛并导出排名，不必依赖公网平台的账号与题库开放程度；源码开放也便于本地加自己的题面格式或对接校内账号。特别提示名称冲突：它与本目录已收的 LibreOJ（loj.ac，2017 年起，Menci 维护，基于 Lyrio）完全无关，只是缩写相同，查资料时别混。局限：项目 2026 年才公开，稳定性、题库规模与社区支持远不及洛谷、HydroOJ、QOJ；默认评测引擎是把代码提交给第三方在线编译服务，而非本地隔离沙箱，正式认证（尤其 NOI 系列）绝不可用，仅适合练习与校内自测；AI 助教需自备模型密钥；AGPL v3.0 要求对外提供服务时公开修改后的源码。

https://github.com/aiwandiannaodelele/LOJ

402 **Oracle VM VirtualBox** 工具

Oracle · virtualbox.org · 2007

CSP-J CSP-S NOIP 省选 查漏补缺 速查 免费（开源）

在家搭「考场同款」环境的首选宿主。建虚拟机时内存至少分 2 GB、磁盘 20 GB 以上，否则 NOI Linux 起不来。

https://www.virtualbox.org/

403 **VMware Workstation Player** 工具

VMware（Broadcom） · vmware.com · 1999

CSP-J CSP-S NOIP 省选 查漏补缺 速查 个人免费

虚拟机体验比 VirtualBox 顺，图形性能更好。部分赛区考场用的就是它，提前熟悉操作可以少花十分钟。

https://www.vmware.com/products/workstation-player.html

404 **洛谷 IDE** 工具

洛谷 · luogu.com.cn · 2019

CSP-J CSP-S 实战训练 速查 免费（需登录）

浏览器里直接编译运行，机房没装编译器时的应急方案。不支持调试，只能看输出。

https://www.luogu.com.cn/ide

405 **Compiler Explorer (godbolt)** 工具

Matt Godbolt · godbolt.org · 2012

CSP-S NOIP 省选 NOI 查漏补缺 速查 免费

在线看汇编。查「为什么这行慢」、验证编译器是否向量化、确认 UB 被优化成什么，都靠它。

https://godbolt.org/

406 **Desmos 图形计算器** 工具

Desmos · desmos.com · 2011

CSP-S NOIP 省选 查漏补缺 速查 免费

画函数、解方程、拟合数据。推 DP 式子时把转移画出来，常能一眼看出单调性或凸性。

https://www.desmos.com/calculator

407 **GeoGebra** 工具

GeoGebra · geogebra.org · 2002

CSP-S NOIP 省选 查漏补缺 速查 免费

动态几何。计算几何题里判断「这两个圆到底怎么交」、验证旋转卡壳的边界情况，比手画准得多。

https://www.geogebra.org/

408 **NOI Linux 2.0** 工具

CCF · noi.cn · 2022

CSP-S NOIP NOI 赛前冲刺 速查 官网免费下载

官方比赛环境。考前必须在同款环境里跑一遍代码，否则本地 AC 考场可能编译失败。

https://www.noi.cn/

409 **Visual Studio Code** 工具

Microsoft · code.visualstudio.com · 2015

CSP-S NOIP 省选 NOI 体系学习 实战训练 速查 免费

配 C/C++ 插件 + 自定义 tasks.json 即可做单文件编译调试。日常开发与刷题主力。

https://code.visualstudio.com/

410 **对拍脚本** stress test 工具

OI 通用做法 · 自建 · —

CSP-S NOIP 省选 实战训练 查漏补缺 方法论 自己写

暴力 + 随机数据 + 自动比对。把「为什么错」从猜测变成实验，是提分最快的习惯。

411 **GDB** 工具

GNU · gnu.org · 1986

NOIP 省选 NOI 查漏补缺 速查 免费

考场环境里唯一可靠的调试器。至少要会 break、run、next、print、bt。

https://www.gnu.org/software/gdb/

412 **CF Polygon（出题系统）** 工具

Codeforces · polygon.codeforces.com · 2013

省选 NOI 教练备课 深耕 免费（需权限）

工业级出题流水线：生成器、对拍、validator、题面打包一条龙。校赛、省赛命题的标准工具。

https://polygon.codeforces.com/

413 **CLion** 工具

JetBrains · jetbrains.com · 2015

省选 NOI 实战训练 速查 付费 / 学生免费

调试体验最好的 C++ IDE。学生可申请免费授权，对大工程调试很有帮助。

https://www.jetbrains.com/clion/

414 **Wolfram Alpha** 工具

Wolfram Research · wolframalpha.com · 2009

省选 NOI 查漏补缺 速查 免费 + 付费

符号积分、求和、递推求解。手推生成函数推不动时，让它先给出答案再反推过程。

https://www.wolframalpha.com/

415 **testlib.h** 工具

Mike Mirzayanov · GitHub · 2010

省选 NOI 教练备课 深耕 免费（开源）

出题用的 checker / validator 框架，几乎所有现代 OJ 都兼容。写好 checker 才谈得上「判题公平」。

https://github.com/MikeMirzayanov/testlib

416 **Valgrind** 工具

Valgrind 社区 · valgrind.org · 2002

NOI 查漏补缺 速查 免费

查越界与内存泄漏。RE 成谜时用它，比肉眼找快得多。

https://valgrind.org/

417 **perf** 工具

Linux 内核 · kernel.org · 2009

NOI 查漏补缺 速查 免费

定位常数瓶颈。TLE 但复杂度正确时，用它看热点函数。

12 第 12 章　社区与期刊

> 社区解决的是「卡住时找谁问」，期刊解决的是「想深入时看什么」。两者都不是主线，但缺了会明显影响效率。

本章收录 14 条，编号 418—431。

418 OI Wiki

社区

OI Wiki 社区 · oi-wiki.org · 2018

通用 查漏补缺 专题突破 速查 深耕 免费开源

中文竞赛知识的总索引。按知识点查、给代码、给复杂度、给例题，全书的知识地图以它为参照。

https://oi-wiki.org/

419 Stack Overflow

社区

Stack Exchange · stackoverflow.com · 2008

通用 查漏补缺 速查 免费

C++ 语法、STL 行为、编译器报错的解答库。「为什么 vector 迭代器失效」这类问题一搜就有。

https://stackoverflow.com/

420 GeeksforGeeks

社区

GeeksforGeeks · geeksforgeeks.org · 2009

CSP-J CSP-S 查漏补缺 速查 免费

算法模板与解释的百科。质量不稳，适合查「这个数据结构长什么样」，不适合当教材通读。

https://www.geeksforgeeks.org/

421 OI Wiki 英文镜像 / 社区分支

社区

OI Wiki 社区 · GitHub · 2018

CSP-J CSP-S NOIP 省选 NOI 查漏补缺 教练备课 速查 免费（开源）

全书 Markdown 源码。可以离线打包、可以自己改，也可以直接给学生发 PDF 版。

https://github.com/OI-wiki/OI-wiki

422 洛谷讨论区

社区

洛谷用户 · luogu.com.cn · 2013

CSP-J CSP-S NOIP 省选 查漏补缺 速查 免费

中文 OI 圈信息集散地：比赛公告、题目求助、经验帖。噪音大，但偶有真干货。

https://www.luogu.com.cn/discuss

423 **Codeforces Blog（题解与教程）** 社区

Codeforces 社区 · codeforces.com · 2010

CSP-S NOIP 省选 NOI 查漏补缺 专题突破 深耕 免费

每场比赛官方 Editorial 都发在这里，另有大量高质量算法教程。英文，但术语固定，读十篇就顺了。

https://codeforces.com/blog

424 **演算法筆記** 社区

台湾师范大学 · web.ntnu.edu.tw · 2010

CSP-S NOIP 专题突破 深耕 免费

繁体中文，图例极其丰富。看图示理解算法流程，比看代码快。

https://web.ntnu.edu.tw/~algo/

425 **Matrix67 博客** 社区

顾森 · matrix67.com · 2005

NOIP 省选 NOI 专题突破 深耕 免费

中文数学科普与算法随笔的经典。讲清了很多教材跳过的「为什么」，读起来像故事，思维密度却很高。

http://www.matrix67.com/blog/

426 **cp-algorithms** e-maxx 英文版 社区

cp-algorithms 社区 · cp-algorithms.com · 2015

NOIP 省选 NOI 查漏补缺 专题突破 速查 深耕 免费开源

算法实现与证明的英文百科。卡在某个算法细节时，这里比博客可靠。

https://cp-algorithms.com/

427 **hzwer 的博客** 社区

黄哲威（hzwer） · hzwer.com · 2013

NOIP 省选 NOI 专题突破 查漏补缺 深耕 免费

BZOJ 时代的题解重镇，分块、莫队、树套树的入门讲解至今仍是中文圈最好懂的一批。

https://hzwer.com/

428 **Computer Science StackExchange** 社区

Stack Exchange · cs.stackexchange.com · 2010

省选 NOI 查漏补缺 深耕 免费

理论问题（复杂度、可计算性、近似算法）的问答区。写论文找相关工作时特别有用。

https://cs.stackexchange.com/

429 **Mathematics StackExchange** 社区

Stack Exchange · math.stackexchange.com · 2010

省选 NOI IOI 查漏补缺 深耕 免费

组合计数、数论、概率的推导细节。看不懂论文里的「显然」时，这里常有完整展开。

https://math.stackexchange.com/

430 **《Olympiads in Informatics》第 20 卷（IOI 学术期刊，2026）** 期刊

维尔纽斯大学数学与信息学研究所 / IOI · 2026

NOI IOI 教练备课 专题突破 深耕 免费（开放获取，可下载整卷或单篇）

OLYMPIADS IN INFORMATICS 由 IOI 社区与立陶宛维尔纽斯大学于 2007 年创办，每年夏季出一卷，与国际信息学奥林匹克同期举行的学术会议配套，第 20 卷已于 2026 年 6 月 19 日上线（主编 Valentina Dagienė；Scopus、Cabell、InfoBase 收录，完全开放获取，整卷与单篇 PDF 均可免费下载）。三段式栏目：研究论文（计算方法、算法与程序设计教学、自动评测系统、面向资优生的课程设计、训练环境与软件）、各国经验报告、书评与动态。对竞赛圈的实际价值在于它是「命题与评测」这条线唯一经同行评议的刊物：子任务给分制的实证效果、交互题与 output-only 题的技术实现、自动评测系统的沙箱与时限设定、各国集训体系的组织方式，都能在这里找到一手材料，比二手博客与公众号可靠得多；写校本课程、做题型趋势分析、给学校论证「为什么要上评测系统」时，这是最容易被采信的引用源。用法：先读各国经验报告（篇幅短、直接讲做法与踩过的坑），再按主题挑研究论文；每卷十余篇，一卷一下午能扫完。局限：全英文、年一卷、不含题解与代码，与 NOI 大纲不对齐；对选手本人的直接帮助有限，属教练与研究者向的资料。

https://ioinformatics.org/page/ioi-journal

431 **许淇文 IOI 2026 参赛总结** 社区

NOI 官网「交流分享」栏目 · 2026

NOI IOI 赛前冲刺 方法论 实战 免费

2026 年 8 月 28 日发布。IOI 2026 全球第一（498.27 分，中国队 3 金 1 银）的选手自述，重点不在题解而在赛场之外的判断：一是提前一天到赛地倒时差、备应急食品这类细节，二是把「他国选手命制的模拟赛」当作校准难度预期的标尺（文中提到韩国选手 Jaewon Son 的两道题，日本队长两题合计仅得 12 分），三是与多国选手交流训练方式。适合冲刺阶段用来对照自己的备赛节奏，不是模板材料。

https://www.noi.cn/jlfx/2026-08-28/927310.shtml

III

PART III · 第三编

三重索引

THREE INDEXES

同一批资料，三种切法：按竞赛类别、按任务需求、按作用效果。任何一条资料都可以从三个维度定位，这是本书作为「工具书」的核心。

第 13 章　索引一 · 按竞赛类别

> 从「我现在打什么比赛」出发。同一层级的资料按编号排列，可当作该阶段的完整书单使用。建议只读当前层级与相邻层级，跨两级以上通常读不懂。

通用 16 条

104 CCF NOI 系列活动考场纪律 官方

105 NOI 大纲（2025 年修订版） 官方

106 NOI 官网 官方

107 NOI 系列活动和认证日历 官方

108 《中国计算机学会通讯》（CCCF） 官方

109 《信息学奥林匹克竞赛年鉴》 官方

164 HydroOJ 平台

216 信息学竞赛中的思维方法 论文

217 测试数据设计中的三要素 论文

357 OI Wiki 专题页 课程

394 Arbiter 工具

395 Cena 工具

396 DOMjudge 工具

397 Lemon 工具

418 OI Wiki 社区

419 Stack Overflow 社区

CSP-J 116 条

001 48 课搞定信息学奥赛：C++ 趣味编程 教材

002 C++编程与信息学竞赛数学思维 教材

003 CCF CSP 例题解析与实战技巧：编程基础 教材

004 CCF CSP 第一轮认证一本通（第 2 版） 教材

005 CCF GESP 直通车：C++ 六级七级精讲精练 教材

006 CCF 中学生计算机程序设计 教材

007 CCF 中学生计算机程序设计 教材

008 CCF 信息学奥赛基础篇 教材

009 Cracking the Coding Interview 教材

010 Think Like a Programmer 教材

011 中学生计算机程序算法入门：例题精析与训练 教材

012 信奥逐梦之路：家长陪伴与升学规划全阶段指南 专著

013 信息学奥赛 CSP-J/S 数学考点大全 教材

014 信息学奥赛1：114道题零基础吃透C++与算法基础 教材

015 信息学奥赛2：121道题零基础吃透高级算法与数据结构 教材

016 信息学奥赛一本通 教材

017 信息学奥赛一本通 教材

018 信息学奥赛初赛全解 教材

019 信息学奥赛大通关 · 入门篇（上下册） 教材

020 信息学奥赛打通关 · 入门篇（全 2 册） 教材

021 信息学奥赛课课通 教材

022 信息学竞赛教程（初级） 教材

023 啊哈！算法 教材

024 我的第一本算法书 教材

025 我的第一本算法竞赛启蒙书 教材

026 深入浅出程序设计竞赛 教材

027 用 AI 决胜信息学奥赛：C++ 编程基础 教材

028 算法与数学：数学思维与编程实践 教材

029 算法图解 教材

030 算法竞赛从新手到高手 教材

031 算法竞赛入门到进阶 教材

032 算法竞赛入门经典 教材

033 算法竞赛试炼场：洛谷300题精析 教材

034 算法竞赛黄金法则：提高算法和思考力的77项技巧 教材

035 算法设计与分析（图解版） 教材

036 聪明人的游戏：信息学探秘（提高篇） 教材

037 计算概论：C++ 编程与信息学竞赛入门 教材

038 高效通关信息学竞赛·初赛篇（入门组） 教材

110 2024 年 NOI 活动题目分析报告（CSP-J/S 2024 第二轮 + NOIP 2024 知识构成） 官方

111 2026 年 WCET & LMCC & NOI 教师发展活动日历 官方

112 CCF CSP 认证官方说明 官方

113 CCF CSP 认证官网 官方

114 CCF NOI 教师培训考核管理规则 官方

115 CCF 信息学奥赛入门篇 官方

116 CCF 关于 CSP-J/S 2026 第一轮晋级第二轮的规定 官方

117 CCF 关于 CSP-J/S 2026 第一轮认证评级分数线的公告 官方

118 CCF 关于 NOI 相关权益的声明 官方

119 CSP-J/S 2026 第一轮认证证书申请通知 官方

120 CSP-J/S 2026 第二轮报名通知 官方

121 CSP-J/S 认证须知 官方

122 NOI Linux 2.0 官方发布页 官方

123 USACO 官网 官方

124 关于 CCF CSP-J/S 认证等级名称规范的公告 官方

125 关于 NOI 系列活动中编程语言使用限制的补充说明 官方

126 关于参加 CSP-J/S 认证需符合年龄条件的公告（12 周岁门槛） 官方

127 关于重申在校在职 NOI 指导教师不得校外有偿兼职授课的公告 官方

128 电子学会青少年等级考试 官方

129 蓝桥杯 官方

130 2025 年 CSP-J/S 第二轮认证题目分析报告 官方

165 AcWing 平台

166 Aizu Online Judge 平台

167 Algorithm Visualizer 平台

168 AtCoder 平台

169 HackerRank 平台

170 LeetCode 平台

171 OpenJudge 平台

172 PTA 拼题 A 平台

173 USACO 平台

174 USFCA 数据结构可视化 平台

175 Vijos 平台

176 VisuAlgo 平台

177 WZOI（温州中学 OJ） 平台

178 一本通在线评测（ybt） 平台

179 梦熊 OJ（Mengxiong Online Judge） 平台

180 洛谷 平台

181 洛谷主题题库 平台

182 洛谷公开赛与月赛 平台

183 洛谷题单 平台

184 编程猫 OJ（北大一点猫人工智能教育联合实验室） 平台

185 蓝桥云课 平台

186 计蒜客 平台

218 ISSEP 2026 论文集：AI 时代的中小学信息学教育 论文

219 OI Wiki 学习路线（C++ 基础 → CSP-J 入门级知识清单） 讲义

273 CCF NOI 2026 笔试题库 真题

274 CSP-J/S 2025 题目及数据（入门组 4 题 + 提高组 4 题） 真题

275 CSP-J/S 2026 第二轮认证测试数据（入门组 + 提高组） 真题

276 CSP-J/S 历年认证题 真题

277 CSP-J/S 第一轮真题 真题

278 JOI 2026/2027 一次预选（第 26 回日本信息学奥赛第一轮）题目·解说·解答例 真题

279 USACO 历年试题 真题

280 《CCF 信息学奥赛入门篇》练习指导 题解

281 信息学奥赛 3：历年真题解析与拉分解题思维训练 题解

282 信息学奥赛 CSP 通关之路：CSP-J/S 第一轮原创全真模拟试卷集（2026） 真题

283 信息学奥赛 CSP-J 初赛通关手册：10 年真题 + 10 套模拟精练精讲 真题

284 信息学奥赛一本通 题解

285 洛谷主题题单 真题

286 算法竞赛入门经典——习题与解答 题解

287 算法竞赛宝典 题解

288 背包问题专题 题解

358 AcWing 算法基础课 课程

359 USACO Guide 课程

360 USACO Training Gateway 课程

361 中国大学 MOOC 课程

362 代码随想录 课程

363 洛谷网校 课程

364 董晓算法（B 站） 课程

398 CP Editor 工具

399 Code::Blocks 工具

400 Dev-C++ 工具

401 LOJ — LeLe Online Judge（开源可自部署的 ACM/OI 双赛制评测系统） 工具

402 Oracle VM VirtualBox 工具

403 VMware Workstation Player 工具

404 洛谷 IDE 工具

420 GeeksforGeeks 社区

421 OI Wiki 英文镜像 / 社区分支 社区

422 洛谷讨论区 社区

CSP-S 173 条

002 C++编程与信息学竞赛数学思维 教材

003 CCF CSP 例题解析与实战技巧：编程基础 教材

004 CCF CSP 第一轮认证一本通（第 2 版） 教材

007 CCF 中学生计算机程序设计 教材

008 CCF 信息学奥赛基础篇 教材

009 Cracking the Coding Interview 教材

010 Think Like a Programmer 教材

012 信奥逐梦之路：家长陪伴与升学规划全阶段指南 专著

013 信息学奥赛 CSP-J/S 数学考点大全 教材

017 信息学奥赛一本通 教材

018 信息学奥赛初赛全解 教材

021 信息学奥赛课课通 教材

022 信息学竞赛教程（初级） 教材

026 深入浅出程序设计竞赛 教材

028 算法与数学：数学思维与编程实践 教材

030 算法竞赛从新手到高手 教材

031 算法竞赛入门到进阶 教材

032 算法竞赛入门经典 教材

033 算法竞赛试炼场：洛谷300题精析 教材

034 算法竞赛黄金法则：提高算法和思考力的77项技巧 教材

035 算法设计与分析（图解版） 教材

036 聪明人的游戏：信息学探秘（提高篇） 教材

039 Algorithms Unlocked 教材

040 Algorithms, 4th Edition（Sedgewick） 教材

041 C++ Primer 教材

042 CCF CSP 例题解析与实战技巧：工程实现 教材

043 CCF 中学生计算机程序设计 教材

044 CCF 信息学奥赛提高篇 教材

045 Competitive Programmer's Handbook 教材

046 Competitive Programming in Python 教材

047 Elements of Programming Interviews 教材

048 Guide to Competitive Programming 教材

049 Guide to Competitive Programming 教材

050 信息学奥赛一本通 · 提高篇 教材

051 信息学奥赛之数学一本通 教材

052 挑战程序设计竞赛 教材

053 数据结构与算法分析 教材

054 数据结构（C++ 语言版） 专著

055 程序设计竞赛进阶：算法与实践 教材

056 算法导论（核心篇）（原书第 4 版） 专著

057 算法思维训练：信奥大师的算法教与学秘籍 专著

058 算法竞赛实战笔记 教材

059 算法竞赛核心教程：策略与算法深度解析 教材

060 算法竞赛进阶指南 教材

061 算法训练营：进阶篇（全彩版） 教材

062 算法设计与分析 教材

063 算法设计与分析：以 ACM 大学生程序设计竞赛在线题库为例（第 2 版 · 微课版） 教材

064 算法（第 4 版） 教材

065 计算思维训练——动态规划 教材

110 2024 年 NOI 活动题目分析报告（CSP-J/S 2024 第二轮 + NOIP 2024 知识构成） 官方

112 CCF CSP 认证官方说明 官方

113 CCF CSP 认证官网 官方

116 CCF 关于 CSP-J/S 2026 第一轮晋级第二轮的规定 官方

117 CCF 关于 CSP-J/S 2026 第一轮认证评级分数线的公告 官方

118 CCF 关于 NOI 相关权益的声明 官方

119 CSP-J/S 2026 第一轮认证证书申请通知 官方

120 CSP-J/S 2026 第二轮报名通知 官方

121 **CSP-J/S 认证须知** 官方

122 **NOI Linux 2.0 官方发布页** 官方

123 **USACO 官网** 官方

124 **关于 CCF CSP-J/S 认证等级名称规范的公告** 官方

125 **关于 NOI 系列活动中编程语言使用限制的补充说明** 官方

126 **关于参加 CSP-J/S 认证需符合年龄条件的公告（12 周岁门槛）** 官方

127 **关于重申在校在职 NOI 指导教师不得校外有偿兼职授课的公告** 官方

130 **2025 年 CSP-J/S 第二轮认证题目分析报告** 官方

131 **CCF 全国青少年信息学奥林匹克竞赛教程** 官方

132 **COCI** 官方

133 **NOIP 2026 参赛名额分配方案** 官方

165 **AcWing** 平台

166 **Aizu Online Judge** 平台

167 **Algorithm Visualizer** 平台

168 **AtCoder** 平台

169 **HackerRank** 平台

172 **PTA 拼题 A** 平台

173 **USACO** 平台

174 **USFCA 数据结构可视化** 平台

175 **Vijos** 平台

176 **VisuAlgo** 平台

177 **WZOI（温州中学 OJ）** 平台

178 **一本通在线评测（ybt）** 平台

179 **梦熊 OJ（Mengxiong Online Judge）** 平台

180 **洛谷** 平台

181 **洛谷主题题库** 平台

182 **洛谷公开赛与月赛** 平台

183 **洛谷题单** 平台

184 **编程猫 OJ（北大—点猫人工智能教育联合实验室）** 平台

185 **蓝桥云课** 平台

186 **计蒜客** 平台

187 **51Nod** 平台

188 **AtCoder Problems** 平台

189 **Baekjoon Online Judge** 平台

190 **CSES Problem Set** 平台

191 **CodeChef** 平台

192 **Codeforces** 平台

193 **Codeforces 题目集（按 rating 筛选）** 平台

194 **Comet OJ** 平台

195 **DMOJ** 平台

196 **HDU Online Judge** 平台

197 **Lutece** 平台

198 **POJ** 平台

199 **SPOJ** 平台

200 **UVa Online Judge** 平台

201 **ZOJ** 平台

202 **e-olymp** 平台

203 **牛客网** 平台

218 **ISSEP 2026 论文集：AI 时代的中小学信息学教育** 论文

219 **OI Wiki 学习路线（C++ 基础 → CSP-J 入门级知识清单）** 讲义

220 **Legend-K 的数据结构与算法笔记** 讲义

221 **搜索方法中的剪枝优化** 论文

222 **搜索顺序的选择** 论文

223 **背包九讲** 讲义

224 **骗分导论** 讲义

273 **CCF NOI 2026 笔试题库** 真题

274 **CSP-J/S 2025 题目及数据（入门组 4 题 + 提高组 4 题）** 真题

275 **CSP-J/S 2026 第二轮认证测试数据（入门组 + 提高组）** 真题

276 **CSP-J/S 历年认证题** 真题

277 **CSP-J/S 第一轮真题** 真题

278 **JOI 2026/2027 一次预选（第 26 回日本信息学奥赛第一轮）题目・解说・解答例** 真题

279 **USACO 历年试题** 真题

282 **信息学奥赛 CSP 通关之路：CSP-J/S 第一轮原创全真模拟试卷集（2026）** 真题

284 **信息学奥赛一本通** 题解

285 **洛谷主题题单** 真题

286 **算法竞赛入门经典——习题与解答** 题解

287 **算法竞赛宝典** 题解

288 **背包问题专题** 题解

289 **BalkOI 2026 试题与在线镜像（巴尔干信息学奥林匹克，地拉那）** 真题

290 **CCF CSP 例题解析与实战技巧：算法应用** 题解

291 **Programming Challenges** 题解

292 **eJOI 2026 试题·题解·数据（欧洲初中生信息学奥林匹克，立陶宛考纳斯）** 真题

293 **信息学奥赛一本通 · 高手训练** 题解

294 **区间与序列 DP 专题** 题解

295 **单调栈与单调队列** 题解

296 **字符串哈希与 KMP** 题解

297 **树形 DP 专题** 题解

298 **树状数组** 题解

299 **算法思维：竞赛真题精选精讲（第 2 版）** 题解

300 **算法竞赛入门经典——训练指南** 题解

358 **AcWing 算法基础课** 课程

359 **USACO Guide** 课程

360 **USACO Training Gateway** 课程

361 **中国大学 MOOC** 课程

362 **代码随想录** 课程

363 **洛谷网校** 课程

364 **董晓算法（B 站）** 课程

365 **2026 年第四期 NOI 教师培训（南京站，中级/初级）** 课程

366 **AcWing 算法提高课** 课程

367 **Berkeley CS61B（数据结构）** 课程

368 **CS 97SI（斯坦福竞赛编程）** 课程

369 **Codeforces EDU** 课程

370 **Codeforces EDU（算法教程）** 课程

371 **Princeton Algorithms（Coursera）** 课程

372 **UCF COP 4516 Contest Problem Solving（中佛罗里达大学竞赛解题课）** 课程

373 **中国大学 MOOC《算法设计与分析》（北大）** 课程

374 **学堂在线** 课程

375 **数据结构（清华）** 课程

376 **灵茶山艾府（B 站 / 题解）** 课程

398 **CP Editor** 工具

399 **Code::Blocks** 工具

401 **LOJ — LeLe Online Judge（开源可自部署的 ACM/OI 双赛制评测系统）** 工具

402 **Oracle VM VirtualBox** 工具

403 **VMware Workstation Player** 工具

404 **洛谷 IDE** 工具

405 **Compiler Explorer (godbolt)** 工具

406 **Desmos 图形计算器** 工具

407 **GeoGebra** 工具

408 **NOI Linux 2.0** 工具

409 **Visual Studio Code** 工具

410 **对拍脚本** 工具

420 **GeeksforGeeks** 社区

421 **OI Wiki 英文镜像 / 社区分支** 社区

422 **洛谷讨论区** 社区

423 **Codeforces Blog（题解与教程）** 社区

424 **演算法筆記** 社区

NOIP 156 条

012 **信奥逐梦之路：家长陪伴与升学规划全阶段指南** 专著

034 **算法竞赛黄金法则：提高算法和思考力的77项技巧** 教材

039 **Algorithms Unlocked** 教材

040 **Algorithms, 4th Edition（Sedgewick）** 教材

041 **C++ Primer** 教材

043 **CCF 中学生计算机程序设计** 教材

044 **CCF 信息学奥赛提高篇** 教材

045 **Competitive Programmer's Handbook** 教材

047 **Elements of Programming Interviews** 教材

048 **Guide to Competitive Programming** 教材

049 **Guide to Competitive Programming** 教材

050 **信息学奥赛一本通 · 提高篇** 教材

051 **信息学奥赛之数学一本通** 教材

052 **挑战程序设计竞赛** 教材

053 **数据结构与算法分析** 教材

054 **数据结构（C++ 语言版）** 专著

055 **程序设计竞赛进阶：算法与实践** 教材

056 **算法导论（核心篇）（原书第 4 版）** 专著

057 **算法思维训练：信奥大师的算法教与学秘籍** 专著

058 **算法竞赛实战笔记** 教材

059 **算法竞赛核心教程：策略与算法深度解析** 教材

060 **算法竞赛进阶指南** 教材

061 **算法训练营：进阶篇（全彩版）** 教材

062 **算法设计与分析** 教材

063 算法设计与分析：以 ACM 大学生程序设计竞赛在线题库为例（第 2 版·微课版） 教材

064 算法（第 4 版） 教材

065 计算思维训练——动态规划 教材

066 ACM 国际大学生程序设计竞赛系列 · 知识与入门 教材

067 Algorithms（Dasgupta‐Papadimitriou‐Vazirani） 教材

068 Algorithms（Jeff Erickson） 教材

069 C++，挑战编程——程序设计竞赛进阶训练指南 教材

070 Competitive Programming 4（配套代码库） 教材

071 Mathematics for Computer Science 教材

072 Programming Pearls 专著

073 算法详解（Algorithms Illuminated，卷 1—4） 教材

074 编程珠玑 专著

110 2024 年 NOI 活动题目分析报告（CSP-J/S 2024 第二轮 + NOIP 2024 知识构成） 官方

111 2026 年 WCET & LMCC & NOI 教师发展活动日历 官方

114 CCF NOI 教师培训考核管理规则 官方

118 CCF 关于 NOI 相关权益的声明 官方

122 NOI Linux 2.0 官方发布页 官方

123 USACO 官网 官方

125 关于 NOI 系列活动中编程语言使用限制的补充说明 官方

126 关于参加 CSP-J/S 认证需符合年龄条件的公告（12 周岁门槛） 官方

127 关于重申在校在职 NOI 指导教师不得校外有偿兼职授课的公告 官方

131 CCF 全国青少年信息学奥林匹克竞赛教程 官方

132 COCI 官方

133 NOIP 2026 参赛名额分配方案 官方

134 2023 全国信息学奥林匹克年鉴（电子版） 官方

135 CCF 关于 NOIP 2026 报名的通知 官方

136 JOI 官方

137 NOIP 评奖规则 官方

138 NOIP 题目综合分析报告 官方

139 第六届女生竞赛报名通知（NOIP 2026 女生赛） 官方

165 AcWing 平台

168 AtCoder 平台

173 USACO 平台

174 USFCA 数据结构可视化 平台

176 VisuAlgo 平台

179 梦熊 OJ（Mengxiong Online Judge） 平台

180 洛谷 平台

181 洛谷主题题库 平台

182 洛谷公开赛与月赛 平台

183 洛谷题单 平台

184 编程猫 OJ（北大—点猫人工智能教育联合实验室） 平台

187 51Nod 平台

188 AtCoder Problems 平台

189 Baekjoon Online Judge 平台

190 CSES Problem Set 平台

191 CodeChef 平台

192 Codeforces 平台

193 Codeforces 题目集（按 rating 筛选） 平台

194 Comet OJ 平台

195 DMOJ 平台

196 HDU Online Judge 平台

198 POJ 平台

199 SPOJ 平台

200 UVa Online Judge 平台

202 e-olymp 平台

203 牛客网 平台

204 Codeforces Gym 平台

205 Judge Duck Online 平台

206 Kattis 平台

207 Timus 平台

220 Legend-K 的数据结构与算法笔记 讲义

221 搜索方法中的剪枝优化 论文

222 搜索顺序的选择 论文

223 背包九讲 讲义

224 骗分导论 讲义

225 字符串算法选讲 讲义

226 常数优化与卡常技巧 讲义

227 浅析非完美算法在信息学竞赛中的应用 论文

228 浅谈用极大化思想解决最大子矩形问题 论文

229 矩阵乘法与线性递推 论文

279 **USACO 历年试题** 真题

285 **洛谷主题题单** 真题

288 **背包问题专题** 题解

289 **BalkOI 2026 试题与在线镜像（巴尔干信息学奥林匹克，地拉那）** 真题

290 **CCF CSP 例题解析与实战技巧：算法应用** 题解

291 **Programming Challenges** 题解

292 **eJOI 2026 试题 · 题解 · 数据（欧洲初中生信息学奥林匹克，立陶宛考纳斯）** 真题

293 **信息学奥赛一本通 · 高手训练** 题解

294 **区间与序列 DP 专题** 题解

295 **单调栈与单调队列** 题解

296 **字符串哈希与 KMP** 题解

297 **树形 DP 专题** 题解

298 **树状数组** 题解

299 **算法思维：竞赛真题精选精讲（第 2 版）** 题解

300 **算法竞赛入门经典——训练指南** 题解

301 **JOI 2025/2026 二次预选（第 25 回日本信息学奥赛第二轮）题目 · 解说 · 数据** 真题

302 **NOIP 2025 题目及数据（糖果店 / 清仓甩卖 / 树的价值 / 序列询问）** 真题

303 **NOIP 历年真题（2011 起）** 真题

304 **NOIP 历年试题** 真题

305 **二分图匹配（匈牙利 / Hopcroft－Karp / KM）** 题解

306 **博弈论与 SG 函数** 题解

431 **数位 DP 专题** 题解

308 **状压 DP 专题** 题解

309 **矩阵与线性递推** 题解

310 **线段树与懒标记** 题解

359 **USACO Guide** 课程

361 **中国大学 MOOC** 课程

363 **洛谷网校** 课程

364 **董晓算法（B 站）** 课程

365 **2026 年第四期 NOI 教师培训（南京站，中级/初级）** 课程

366 **AcWing 算法提高课** 课程

367 **Berkeley CS61B（数据结构）** 课程

368 **CS 97SI（斯坦福竞赛编程）** 课程

369 **Codeforces EDU** 课程

370 **Codeforces EDU（算法教程）** 课程

371 **Princeton Algorithms（Coursera）** 课程

372 **UCF COP 4516 Contest Problem Solving（中佛罗里达大学竞赛解题课）** 课程

373 **中国大学 MOOC《算法设计与分析》（北大）** 课程

374 **学堂在线** 课程

375 **数据结构（清华）** 课程

376 **灵茶山艾府（B 站 / 题解）** 课程

377 **AtCoder Library Practice** 课程

378 **MIT OCW 6.006（算法导论）** 课程

379 **NOI 2025 冬令营授课与交流活动现场视频（14 场）** 课程

380 **算法设计与分析（MIT 6.006）** 课程

401 **LOJ — LeLe Online Judge（开源可自部署的 ACM/OI 双赛制评测系统）** 工具

402 **Oracle VM VirtualBox** 工具

403 **VMware Workstation Player** 工具

405 **Compiler Explorer (godbolt)** 工具

406 **Desmos 图形计算器** 工具

407 **GeoGebra** 工具

408 **NOI Linux 2.0** 工具

409 **Visual Studio Code** 工具

410 **对拍脚本** 工具

411 **GDB** 工具

421 **OI Wiki 英文镜像 / 社区分支** 社区

422 **洛谷讨论区** 社区

423 **Codeforces Blog（题解与教程）** 社区

424 **演算法筆記** 社区

425 **Matrix67 博客** 社区

426 **cp-algorithms** 社区

427 **hzwer 的博客** 社区

省选 185 条

040 **Algorithms, 4th Edition（Sedgewick）** 教材

049 **Guide to Competitive Programming** 教材

052 **挑战程序设计竞赛** 教材

054 **数据结构（C++ 语言版）** 专著

055 程序设计竞赛进阶：算法与实践 教材

056 算法导论（核心篇）（原书第 4 版） 专著

057 算法思维训练：信奥大师的算法教与学秘籍 专著

059 算法竞赛核心教程：策略与算法深度解析 教材

060 算法竞赛进阶指南 教材

061 算法训练营：进阶篇（全彩版） 教材

063 算法设计与分析：以 ACM 大学生程序设计竞赛在线题库为例（第 2 版·微课版） 教材

066 ACM 国际大学生程序设计竞赛系列 · 知识与入门 教材

067 Algorithms（Dasgupta‐Papadimitriou‐Vazirani） 教材

068 Algorithms（Jeff Erickson） 教材

069 C++，挑战编程——程序设计竞赛进阶训练指南 教材

070 Competitive Programming 4（配套代码库） 教材

071 Mathematics for Computer Science 教材

072 Programming Pearls 专著

073 算法详解（Algorithms Illuminated，卷 1—4） 教材

074 编程珠玑 专著

075 ACM 国际大学生程序设计竞赛系列 · 算法与实现 教材

076 Algorithm Design（Kleinberg‐Tardos） 教材

077 Competitive Programming 4 教材

078 Introduction to Graph Theory（West） 教材

079 Introduction to Probability 教材

080 The Algorithm Design Manual 教材

081 信息学竞赛中的数学 教材

082 具体数学 专著

083 挑战程序设计竞赛 2 教材

084 数据结构与算法（上下册） 教材

085 离散数学及其应用 专著

086 算法导论 专著

087 算法导论（扩展篇）（原书第 4 版） 教材

088 算法竞赛中的数论 教材

089 算法艺术与信息学竞赛 教材

090 算法设计手册 教材

091 组合数学 专著

122 NOI Linux 2.0 官方发布页 官方

131 CCF 全国青少年信息学奥林匹克竞赛教程 官方

132 COCI 官方

134 2023 全国信息学奥林匹克年鉴（电子版） 官方

136 JOI 官方

140 2022—2023 年国内外信息学主要赛事分析报告 官方

141 APIO 2026 中国区活动赛事综述与分数线 官方

142 CCF 关于 NOI 2027 省内选拔的若干规定 官方

143 CCPC 官网 官方

144 ICPC 官网 官方

145 NOI 2027 名额分配测算方案 官方

146 NOI 2027 联合省选：两试时间与成绩构成对照（26 省市） 官方

147 2024 全国信息学奥林匹克年鉴 官方

159 关于自 NOI 2029 起取消 A 类选手加分政策的公告 官方

165 AcWing 平台

168 AtCoder 平台

180 洛谷 平台

181 洛谷主题题库 平台

182 洛谷公开赛与月赛 平台

188 AtCoder Problems 平台

192 Codeforces 平台

193 Codeforces 题目集（按 rating 筛选） 平台

204 Codeforces Gym 平台

205 Judge Duck Online 平台

206 Kattis 平台

207 Timus 平台

208 LibreOJ 平台

209 Nowcoder 多校训练营 平台

210 OEIS 整数数列百科 平台

211 QOJ 平台

212 UOJ 平台

220 Legend-K 的数据结构与算法笔记 讲义

225 字符串算法选讲 讲义

226 常数优化与卡常技巧 讲义

227 浅析非完美算法在信息学竞赛中的应用 论文

228 浅谈用极大化思想解决最大子矩形问题 论文

229 矩阵乘法与线性递推 论文

230 NOIpaper：国家集训队论文集（1999—2025） 论文

231 OI 论文集精读 论文

232 Trinkle23897 的 OI 课件 讲义

233 **acm-cheat-sheet** 讲义

234 **huzecong 的 OI 课件** 讲义

235 **分块与莫队算法** 论文

236 **动态规划优化专题** 讲义

237 **动态规划的深入讨论** 论文

238 **动态规划算法的优化技巧** 论文

239 **图论建模专题** 讲义

240 **强连通分量与 2-SAT** 论文

241 **把握本质，灵活运用——动态规划的深入探讨** 论文

242 **数学模型的建立、比较和应用** 论文

243 **树型动态规划** 论文

244 **状态压缩动态规划** 论文

245 **离线处理思想** 论文

246 **线段树的应用** 论文

247 **组合游戏略述——浅谈 SG 游戏的若干拓展及变形** 论文

248 **网络流算法的应用** 论文

249 **解决动态统计问题的两把利刃** 论文

285 **洛谷主题题单** 真题

289 **BalkOI 2026 试题与在线镜像（巴尔干信息学奥林匹克，地拉那）** 真题

293 **信息学奥赛一本通 · 高手训练** 题解

294 **区间与序列 DP 专题** 题解

295 **单调栈与单调队列** 题解

296 **字符串哈希与 KMP** 题解

297 **树形 DP 专题** 题解

298 **树状数组** 题解

299 **算法思维：竞赛真题精选精讲（第 2 版）** 题解

301 **JOI 2025/2026 二次预选（第 25 回日本信息学奥赛第二轮）题目·解说·数据** 真题

302 **NOIP 2025 题目及数据（糖果店 / 清仓甩卖 / 树的价值 / 序列询问）** 真题

305 **二分图匹配（匈牙利 / Hopcroft－Karp / KM）** 题解

306 **博弈论与 SG 函数** 题解

431 **数位 DP 专题** 题解

308 **状压 DP 专题** 题解

309 **矩阵与线性递推** 题解

310 **线段树与懒标记** 题解

311 **2-SAT 与差分约束** 题解

312 **AC 自动机** 题解

313 **ACM 国际大学生程序设计竞赛系列 · 题目与解读** 题解

314 **AHOI 安徽省选** 真题

315 **APIO 2026 中国区活动题目及数据** 真题

316 **BJOI 北京队选拔** 真题

317 **BOI 试题库与 2026 年题目（波罗的海信息学奥林匹克，1995—2026 全量题面、数据与题解）** 真题

318 **CEOI 2026 试题与数据（第 33 届中欧信息学奥林匹克，斯洛文尼亚卢布尔雅那）** 真题

319 **DP 优化专题（单调队列/斜率/决策单调性）** 题解

320 **FJOI 福建省选** 真题

321 **GDOI 广东省选** 真题

322 **HAOI 河南省选** 真题

323 **HEOI 河北省选** 真题

324 **HNOI 湖南省选** 真题

325 **JSOI 江苏省选** 真题

326 **Manacher 与回文自动机** 题解

327 **SCOI 四川省选** 真题

328 **SDOI 山东省选** 真题

329 **SHOI 上海队选拔** 真题

330 **WEOI 2026 试题与题解（第 4 届西欧信息学奥林匹克，卢森堡）** 真题

331 **ZJOI 浙江省选** 真题

332 **十二省联考** 真题

333 **可持久化数据结构** 题解

334 **后缀数组与后缀自动机** 题解

335 **国家集训队论文配套题** 真题

336 **多项式与 NTT** 题解

337 **平衡树（Treap / Splay / FHQ）** 题解

338 **插头 DP（连通性状态压缩）** 题解

339 **树链剖分（HLD）** 题解

340 **网络流（最大流 / 最小割 / 费用流）** 题解

341 **联合省选（CCF 统一命题）** 真题

342 **莫比乌斯反演与杜教筛** 题解

343 **虚树** 题解

344 **计算几何基础与凸包** 题解

352 **NOI 冬令营 2026 题目及数据** 真题

359 **USACO Guide** 课程

365 2026 年第四期 NOI 教师培训（南京站，中级/初级） 课程

366 AcWing 算法提高课 课程

367 Berkeley CS61B（数据结构） 课程

370 Codeforces EDU（算法教程） 课程

371 Princeton Algorithms（Coursera） 课程

372 UCF COP 4516 Contest Problem Solving（中佛罗里达大学竞赛解题课） 课程

373 中国大学 MOOC《算法设计与分析》（北大） 课程

374 学堂在线 课程

376 灵茶山艾府（B 站 / 题解） 课程

377 AtCoder Library Practice 课程

378 MIT OCW 6.006（算法导论） 课程

379 NOI 2025 冬令营授课与交流活动现场视频（14 场） 课程

380 算法设计与分析（MIT 6.006） 课程

381 APIO 2025 讲者授课现场视频（5 场） 课程

382 APIO 2026 中国区活动讲座现场视频（7 场） 课程

383 AcWing 算法进阶课 课程

384 CS 3233（NUS 竞赛编程） 课程

385 CCF NOI 冬令营 2026 第一课堂（CCF 数字图书馆） 课程

402 Oracle VM VirtualBox 工具

403 VMware Workstation Player 工具

405 Compiler Explorer (godbolt) 工具

406 Desmos 图形计算器 工具

407 GeoGebra 工具

409 Visual Studio Code 工具

410 对拍脚本 工具

411 GDB 工具

412 CF Polygon（出题系统） 工具

413 CLion 工具

414 Wolfram Alpha 工具

415 testlib.h 工具

421 OI Wiki 英文镜像 / 社区分支 社区

422 洛谷讨论区 社区

423 Codeforces Blog（题解与教程） 社区

425 Matrix67 博客 社区

426 cp-algorithms 社区

427 hzwer 的博客 社区

428 Computer Science StackExchange 社区

429 Mathematics StackExchange 社区

NOI 193 条

056 算法导论（核心篇）（原书第 4 版） 专著

061 算法训练营：进阶篇（全彩版） 教材

067 Algorithms（Dasgupta - Papadimitriou - Vazirani） 教材

068 Algorithms（Jeff Erickson） 教材

070 Competitive Programming 4（配套代码库） 教材

071 Mathematics for Computer Science 教材

072 Programming Pearls 专著

073 算法详解（Algorithms Illuminated，卷 1—4） 教材

075 ACM 国际大学生程序设计竞赛系列 · 算法与实现 教材

076 Algorithm Design（Kleinberg - Tardos） 教材

077 Competitive Programming 4 教材

078 Introduction to Graph Theory（West） 教材

079 Introduction to Probability 教材

080 The Algorithm Design Manual 教材

081 信息学竞赛中的数学 教材

082 具体数学 专著

083 挑战程序设计竞赛 2 教材

084 数据结构与算法（上下册） 教材

085 离散数学及其应用 专著

086 算法导论 专著

087 算法导论（扩展篇）（原书第 4 版） 教材

088 算法竞赛中的数论 教材

089 算法艺术与信息学竞赛 教材

090 算法设计手册 教材

091 组合数学 专著

092 Graph Theory（Bondy & Murty） 教材

093 Handbook of Data Structures and Applications（2nd Edition） 专著

094 Linear Algebra Done Right 教材

095 **Structure and Interpretation of Computer Programs** 教材

096 **概率论与数理统计** 专著

097 **线性代数及其应用** 专著

098 **编程原本** 专著

099 **计算几何：算法与应用** 专著

114 **CCF NOI 教师培训考核管理规则** 官方

118 **CCF 关于 NOI 相关权益的声明** 官方

122 **NOI Linux 2.0 官方发布页** 官方

125 **关于 NOI 系列活动中编程语言使用限制的补充说明** 官方

127 **关于重申在校在职 NOI 指导教师不得校外有偿兼职授课的公告** 官方

134 **2023 全国信息学奥林匹克年鉴（电子版）** 官方

139 **第六届女生竞赛报名通知（NOIP 2026 女生赛）** 官方

140 **2022—2023 年国内外信息学主要赛事分析报告** 官方

141 **APIO 2026 中国区活动赛事综述与分数线** 官方

142 **CCF 关于 NOI 2027 省内选拔的若干规定** 官方

143 **CCPC 官网** 官方

144 **ICPC 官网** 官方

145 **NOI 2027 名额分配测算方案** 官方

147 **2024 全国信息学奥林匹克年鉴** 官方

148 **APIO 官网** 官方

149 **Baltic OI** 官方

150 **CCF NOI 集训营 2026 综合素质评价机制（附国家集训队行为规范）** 官方

151 **CEOI** 官方

152 **IOI 2027 中国国家队选拔规则与国家集训队行为规范** 官方

153 **IOI Syllabus** 官方

154 **NOI 冬令营 2026 首次 AI 辅助编程测试（人机协同评测）** 官方

155 **NOI 参赛资格规定** 官方

156 **NOI 试题分析与年鉴** 官方

157 **POI** 官方

158 **USACO 2025-2026 赛季赛制改革：三场线上赛 + 监考制 US Open** 官方

159 **关于自 NOI 2029 起取消 A 类选手加分政策的公告** 官方

160 **IOI 2026 中国队 3 金 1 银与最终排名** 官方

181 **洛谷主题题库** 平台

184 **编程猫 OJ（北大—点猫人工智能教育联合实验室）** 平台

188 **AtCoder Problems** 平台

192 **Codeforces** 平台

193 **Codeforces 题目集（按 rating 筛选）** 平台

204 **Codeforces Gym** 平台

208 **LibreOJ** 平台

209 **Nowcoder 多校训练营** 平台

210 **OEIS 整数数列百科** 平台

211 **QOJ** 平台

212 **UOJ** 平台

213 **Library Checker** 平台

214 **Universal Cup** 平台

215 **oj.uz** 平台

230 **NOIpaper：国家集训队论文集（1999—2025）** 论文

231 **OI 论文集精读** 论文

232 **Trinkle23897 的 OI 课件** 讲义

233 **acm-cheat-sheet** 讲义

234 **huzecong 的 OI 课件** 讲义

235 **分块与莫队算法** 论文

236 **动态规划优化专题** 讲义

237 **动态规划的深入讨论** 论文

238 **动态规划算法的优化技巧** 论文

239 **图论建模专题** 讲义

240 **强连通分量与 2-SAT** 论文

241 **把握本质，灵活运用——动态规划的深入探讨** 论文

242 **数学模型的建立、比较和应用** 论文

243 **树型动态规划** 论文

244 **状态压缩动态规划** 论文

245 **离线处理思想** 论文

246 **线段树的应用** 论文

247 **组合游戏略述——浅谈 SG 游戏的若干拓展及变形** 论文

248 **网络流算法的应用** 论文

249 **解决动态统计问题的两把利刃** 论文

250 **Pólya 原理及其应用** 论文

251 **伸展树的基本操作与应用** 论文

252 **充分利用问题性质——例析动态规划的「个性化」优化** 论文

253 **具体数学讲义** 讲义

254 动态树及其应用 论文

255 可持久化数据结构 论文

256 后缀数组 论文

257 后缀数组——处理字符串的有力工具 讲义

258 多串匹配算法及其启示 论文

259 容斥原理与反演 论文

260 左偏树的特点及其应用 论文

261 平面图在信息学中的应用 论文

262 提交答案题与交互题 讲义

263 数值分析在竞赛中的简单应用 论文

264 数据结构的联合——块状链表 论文

265 最小割模型在信息学竞赛中的应用 论文

266 构造题策略 讲义

267 母函数的性质及应用 论文

268 猜数问题的研究 论文

269 答案只有一个——浅谈问答式交互问题 论文

270 维护森林连通性——动态树 论文

271 计算几何讲义 讲义

272 论随机化算法的原理与设计 论文

273 CCF NOI 2026 笔试题库 真题

305 二分图匹配（匈牙利 / Hopcroft－Karp / KM） 题解

306 博弈论与 SG 函数 题解

308 状压 DP 专题 题解

310 线段树与懒标记 题解

311 2-SAT 与差分约束 题解

312 AC 自动机 题解

313 ACM 国际大学生程序设计竞赛系列 · 题目与解读 题解

315 APIO 2026 中国区活动题目及数据 真题

316 BJOI 北京队选拔 真题

317 BOI 试题库与 2026 年题目（波罗的海信息学奥林匹克，1995—2026 全量题面、数据与题解） 真题

318 CEOI 2026 试题与数据（第 33 届中欧信息学奥林匹克，斯洛文尼亚卢布尔雅那） 真题

319 DP 优化专题（单调队列/斜率/决策单调性） 题解

324 HNOI 湖南省选 真题

326 Manacher 与回文自动机 题解

327 SCOI 四川省选 真题

328 SDOI 山东省选 真题

329 SHOI 上海队选拔 真题

330 WEOI 2026 试题与题解（第 4 届西欧信息学奥林匹克，卢森堡） 真题

331 ZJOI 浙江省选 真题

332 十二省联考 真题

333 可持久化数据结构 题解

334 后缀数组与后缀自动机 题解

335 国家集训队论文配套题 真题

336 多项式与 NTT 题解

337 平衡树（Treap / Splay / FHQ） 题解

338 插头 DP（连通性状态压缩） 题解

339 树链剖分（HLD） 题解

340 网络流（最大流 / 最小割 / 费用流） 题解

342 莫比乌斯反演与杜教筛 题解

343 虚树 题解

344 计算几何基础与凸包 题解

345 APIO 历年试题 真题

346 EGOI 2026 试题与逐题统计（第 6 届欧洲女子信息学奥林匹克） 真题

347 IOI 2025 官方赛题包（ioi/task-archive 2025，6 题全量数据） 真题

348 IOI 2026 中国国家集训队集中培训与精英培训题目及数据 真题

349 IOI 2026 试题与题解（第 38 届国际信息学奥林匹克，塔什干） 真题

350 Min_25 筛与洲阁筛 题解

351 NOI 2026 题目及数据 真题

352 NOI 冬令营 2026 题目及数据 真题

353 NOI 历年真题 真题

354 NOI 历年试题 真题

355 生成函数与组合计数 题解

378 MIT OCW 6.006（算法导论） 课程

379 NOI 2025 冬令营授课与交流活动现场视频（14 场） 课程

381 APIO 2025 讲者授课现场视频（5 场） 课程

382 APIO 2026 中国区活动讲座现场视频（7 场） 课程

383 AcWing 算法进阶课 课程

384 CS 3233（NUS 竞赛编程） 课程

385 CCF NOI 冬令营 2026 第一课堂（CCF 数字图书馆） 课程

386 CCF NOI 集训营 2026 报到通知（附日程） 课程

387 CCF 关于举办 IOI 2027 精英培训的通知 课程

388 CMU 15-295 Competition Programming and Problem Solving 课程

389 CMU 15-451（算法设计与分析） 课程

390 NOI 冬令营 2026 讲座与交流活动现场视频（14 场） 课程

391 Stanford CS161（算法设计与分析） 课程

392 蒋炎岩（JYY）课程站 课程

405 Compiler Explorer (godbolt) 工具

408 NOI Linux 2.0 工具

409 Visual Studio Code 工具

411 GDB 工具

412 CF Polygon（出题系统） 工具

413 CLion 工具

414 Wolfram Alpha 工具

415 testlib.h 工具

416 Valgrind 工具

417 perf 工具

421 OI Wiki 英文镜像 / 社区分支 社区

423 Codeforces Blog（题解与教程） 社区

425 Matrix67 博客 社区

426 cp-algorithms 社区

427 hzwer 的博客 社区

428 Computer Science StackExchange 社区

429 Mathematics StackExchange 社区

430 《Olympiads in Informatics》第 20 卷（IOI 学术期刊，2026） 期刊

431 许淇文 IOI 2026 参赛总结 社区

IOI 56 条

068 Algorithms（Jeff Erickson） 教材

076 Algorithm Design（Kleinberg – Tardos） 教材

077 Competitive Programming 4 教材

078 Introduction to Graph Theory（West） 教材

080 The Algorithm Design Manual 教材

086 算法导论 专著

092 Graph Theory（Bondy & Murty） 教材

093 Handbook of Data Structures and Applications（2nd Edition） 专著

094 Linear Algebra Done Right 教材

095 Structure and Interpretation of Computer Programs 教材

100 ACM Transactions on Algorithms（TALG） 专著

101 Algorithmica 专著

102 Journal of Graph Algorithms and Applications（JGAA） 专著

103 计算机程序设计艺术 专著

150 CCF NOI 集训营 2026 综合素质评价机制（附国家集训队行为规范） 官方

151 CEOI 官方

152 IOI 2027 中国国家队选拔规则与国家集训队行为规范 官方

153 IOI Syllabus 官方

154 NOI 冬令营 2026 首次 AI 辅助编程测试（人机协同评测） 官方

158 USACO 2025-2026 赛季赛制改革：三场线上赛 + 监考制 US Open 官方

160 IOI 2026 中国队 3 金 1 银与最终排名 官方

161 IOI 2027 官方网站（第 39 届，德国波茨坦 HPI，2027-09-12—19） 官方

162 IOI 官网 官方

163 IOI 年度报告与命题趋势 官方

204 Codeforces Gym 平台

210 OEIS 整数数列百科 平台

211 QOJ 平台

214 Universal Cup 平台

215 oj.uz 平台

230 NOIpaper：国家集训队论文集（1999—2025） 论文

231 OI 论文集精读 论文

254 动态树及其应用 论文

262 提交答案题与交互题 讲义

269 答案只有一个——浅谈问答式交互问题 论文

270 维护森林连通性——动态树 论文

334 后缀数组与后缀自动机 题解

340 网络流（最大流 / 最小割 / 费用流） 题解

346 EGOI 2026 试题与逐题统计（第 6 届欧洲女子信息学奥林匹克） 真题

347 **IOI 2025 官方赛题包（ioi/task-archive 2025，6 题全量数据）** 真题

348 **IOI 2026 中国国家集训队集中培训与精英培训题目及数据** 真题

349 **IOI 2026 试题与题解（第 38 届国际信息学奥林匹克，塔什干）** 真题

355 **生成函数与组合计数** 题解

356 **IOI 历年试题** 真题

381 **APIO 2025 讲者授课现场视频（5 场）** 课程

382 **APIO 2026 中国区活动讲座现场视频（7 场）** 课程

386 **CCF NOI 集训营 2026 报到通知（附日程）** 课程

387 **CCF 关于举办 IOI 2027 精英培训的通知** 课程

388 **CMU 15-295 Competition Programming and Problem Solving** 课程

389 **CMU 15-451（算法设计与分析）** 课程

390 **NOI 冬令营 2026 讲座与交流活动现场视频（14 场）** 课程

391 **Stanford CS161（算法设计与分析）** 课程

392 **蒋炎岩（JYY）课程站** 课程

393 **IOI 官方 YouTube 频道** 课程

429 **Mathematics StackExchange** 社区

430 **《Olympiads in Informatics》第 20 卷（IOI 学术期刊，2026）** 期刊

431 **许淇文 IOI 2026 参赛总结** 社区

第 14 章　索引二・按任务需求

> 从「我现在要干什么」出发。七种需求对应七种完全不同的资料组合：入门要教材，冲刺要真题，备课要论文与讲义。

入门自修 53 条

001 48 课搞定信息学奥赛：C++ 趣味编程 教材

003 CCF CSP 例题解析与实战技巧：编程基础 教材

005 CCF GESP 直通车：C++ 六级七级精讲精练 教材

006 CCF 中学生计算机程序设计 教材

009 Cracking the Coding Interview 教材

010 Think Like a Programmer 教材

011 中学生计算机程序算法入门：例题精析与训练 教材

013 信息学奥赛 CSP-J/S 数学考点大全 教材

014 信息学奥赛1：114道题零基础吃透C++与算法基础 教材

016 信息学奥赛一本通 教材

019 信息学奥赛大通关・入门篇（上下册） 教材

020 信息学奥赛打通关 · 入门篇（全 2 册） 教材

021 信息学奥赛课课通 教材

023 啊哈！算法 教材

024 我的第一本算法书 教材

025 我的第一本算法竞赛启蒙书 教材

026 深入浅出程序设计竞赛 教材

027 用 AI 决胜信息学奥赛：C++ 编程基础 教材

029 算法图解 教材

030 算法竞赛从新手到高手 教材

031 算法竞赛入门到进阶 教材

032 算法竞赛入门经典 教材

033 算法竞赛试炼场：洛谷300题精析 教材

035 算法设计与分析（图解版） 教材

036 聪明人的游戏：信息学探秘（提高篇） 教材

037 计算概论：C++ 编程与信息学竞赛入门 教材

038 高效通关信息学竞赛・初赛篇（入门组） 教材

039 Algorithms Unlocked 教材

046 Competitive Programming in Python 教材

115 CCF 信息学奥赛入门篇 官方

128 电子学会青少年等级考试 官方

129 蓝桥杯 官方

166 Aizu Online Judge 平台

167 Algorithm Visualizer 平台

168 AtCoder 平台

169 HackerRank 平台

171 OpenJudge 平台

174 USFCA 数据结构可视化 平台

176 VisuAlgo 平台

178 一本通在线评测（ybt） 平台

180 洛谷 平台

184 编程猫 OJ（北大—点猫人工智能教育联合实验室） 平台

185 蓝桥云课 平台

186 计蒜客 平台

219 OI Wiki 学习路线（C++ 基础 → CSP-J 入门级知识清单） 讲义

288 背包问题专题 题解

358 AcWing 算法基础课 课程

360 USACO Training Gateway 课程

362 代码随想录 课程

363 洛谷网校 课程

364 董晓算法（B 站） 课程

399 Code::Blocks 工具

400 Dev-C++ 工具

体系学习 70 条

002 C++编程与信息学竞赛数学思维 教材

006 CCF 中学生计算机程序设计 教材

007 CCF 中学生计算机程序设计 教材

008 CCF 信息学奥赛基础篇 教材

012 信奥逐梦之路：家长陪伴与升学规划全阶段指南 专著

019 信息学奥赛大通关·入门篇（上下册） 教材

021 信息学奥赛课课通 教材

022 信息学竞赛教程（初级） 教材

028 算法与数学：数学思维与编程实践 教材

030 算法竞赛从新手到高手 教材

031 算法竞赛入门到进阶 教材

032 算法竞赛入门经典 教材

034 算法竞赛黄金法则：提高算法和思考力的77项技巧 教材

035 算法设计与分析（图解版） 教材

036 聪明人的游戏：信息学探秘（提高篇） 教材

039 Algorithms Unlocked 教材

040 Algorithms, 4th Edition（Sedgewick） 教材

043 CCF 中学生计算机程序设计 教材

044 CCF 信息学奥赛提高篇 教材

045 Competitive Programmer's Handbook 教材

048 Guide to Competitive Programming 教材

049 Guide to Competitive Programming 教材

050 信息学奥赛一本通 · 提高篇 教材

053 数据结构与算法分析 教材

054 数据结构（C++ 语言版） 专著

057 算法思维训练：信奥大师的算法教与学秘籍 专著

058 算法竞赛实战笔记 教材

059 算法竞赛核心教程：策略与算法深度解析 教材

060 算法竞赛进阶指南 教材

061 算法训练营：进阶篇（全彩版） 教材

062 算法设计与分析 教材

063 算法设计与分析：以 ACM 大学生程序设计竞赛在线题库为例（第 2 版·微课版） 教材

064 算法（第 4 版） 教材

066 ACM 国际大学生程序设计竞赛系列 · 知识与入门 教材

067 Algorithms（Dasgupta‐Papadimitriou‐Vazirani） 教材

068 Algorithms（Jeff Erickson） 教材

071 Mathematics for Computer Science 教材

073 算法详解（Algorithms Illuminated，卷 1—4） 教材

084 数据结构与算法（上下册） 教材

087 算法导论（扩展篇）（原书第 4 版） 教材

095 Structure and Interpretation of Computer Programs 教材

105 NOI 大纲（2025 年修订版） 官方

115 CCF 信息学奥赛入门篇 官方

123 USACO 官网 官方

165 AcWing 平台

172 PTA 拼题 A 平台

173 USACO 平台

178 一本通在线评测（ybt） 平台

183 洛谷题单 平台

190 CSES Problem Set 平台

219 OI Wiki 学习路线（C++ 基础 → CSP-J 入门级知识清单） 讲义

285 洛谷主题题单 真题

299 算法思维：竞赛真题精选精讲（第 2 版） 题解

358 AcWing 算法基础课 课程

359 USACO Guide 课程

361 中国大学 MOOC 课程

362 代码随想录 课程

366 AcWing 算法提高课 课程

367 Berkeley CS61B（数据结构） 课程

368 CS 97SI（斯坦福竞赛编程） 课程

371 Princeton Algorithms（Coursera） 课程

372 UCF COP 4516 Contest Problem Solving（中佛罗里达大学竞赛解题课） 课程

373 中国大学 MOOC《算法设计与分析》（北大） 课程

374 学堂在线 课程

375 数据结构（清华） 课程

378 MIT OCW 6.006（算法导论） 课程

380 算法设计与分析（MIT 6.006） 课程

391 Stanford CS161（算法设计与分析） 课程

392 蒋炎岩（JYY）课程站 课程

409 Visual Studio Code 工具

专题突破 158 条

015 信息学奥赛2：121道题零基础吃透高级算法与数据结构 教材

034 算法竞赛黄金法则：提高算法和思考力的77项技巧 教材

042 CCF CSP 例题解析与实战技巧：工程实现 教材

043 CCF 中学生计算机程序设计 教材

044 CCF 信息学奥赛提高篇 教材

049 Guide to Competitive Programming 教材

050 信息学奥赛一本通 · 提高篇 教材

051 信息学奥赛之数学一本通 教材

052 挑战程序设计竞赛 教材

055 程序设计竞赛进阶：算法与实践 教材

059 算法竞赛核心教程：策略与算法深度解析 教材

060 算法竞赛进阶指南 教材

061 算法训练营：进阶篇（全彩版） 教材

063 算法设计与分析：以 ACM 大学生程序设计竞赛在线题库为例（第 2 版·微课版） 教材

065 计算思维训练——动态规划 教材

068 Algorithms（Jeff Erickson） 教材

069 C++，挑战编程——程序设计竞赛进阶训练指南 教材

072 Programming Pearls 专著

074 编程珠玑 专著

075 ACM 国际大学生程序设计竞赛系列 · 算法与实现 教材

076 Algorithm Design（Kleinberg‑Tardos） 教材

077 Competitive Programming 4 教材

078 Introduction to Graph Theory（West） 教材

079 Introduction to Probability 教材

080 The Algorithm Design Manual 教材

081 信息学竞赛中的数学 教材

082 具体数学 专著

083 挑战程序设计竞赛 2 教材

088 算法竞赛中的数论 教材

089 算法艺术与信息学竞赛 教材

091 组合数学 专著

092 Graph Theory（Bondy & Murty） 教材

093 Handbook of Data Structures and Applications（2nd Edition） 专著

094 Linear Algebra Done Right 教材

096 概率论与数理统计 专著

097 线性代数及其应用 专著

099 计算几何：算法与应用 专著

100 ACM Transactions on Algorithms（TALG） 专著

101 Algorithmica 专著

102 Journal of Graph Algorithms and Applications（JGAA） 专著

154 NOI 冬令营 2026 首次 AI 辅助编程测试（人机协同评测） 官方

181 洛谷主题题库 平台

187 51Nod 平台

190 CSES Problem Set 平台

220 Legend-K 的数据结构与算法笔记 讲义

221 搜索方法中的剪枝优化 论文

222 搜索顺序的选择 论文

223 背包九讲 讲义

225 字符串算法选讲 讲义

228 浅谈用极大化思想解决最大子矩形问题 论文

229 矩阵乘法与线性递推 论文

230 NOIpaper：国家集训队论文集（1999—2025） 论文

231 OI 论文集精读 论文

232 Trinkle23897 的 OI 课件 讲义

234 huzecong 的 OI 课件 讲义

235 分块与莫队算法 论文

236 动态规划优化专题 讲义

237 动态规划的深入讨论 论文

238 动态规划算法的优化技巧 论文

239 图论建模专题 讲义

240 强连通分量与 2-SAT 论文

241 把握本质，灵活运用——动态规划的深入探讨 论文

242 数学模型的建立、比较和应用 论文

243 树型动态规划 论文

244 状态压缩动态规划 论文

245 离线处理思想 论文

246 线段树的应用 论文

247 组合游戏略述——浅谈 SG 游戏的若干拓展及变形 论文

248 网络流算法的应用 论文

249 解决动态统计问题的两把利刃 论文

250 Pólya 原理及其应用 论文

251 伸展树的基本操作与应用 论文

252 充分利用问题性质——例析动态规划的「个性化」优化 论文

253 具体数学讲义 讲义

254 动态树及其应用 论文

255 可持久化数据结构 论文

256 后缀数组 论文

257 后缀数组——处理字符串的有力工具 讲义

258 多串匹配算法及其启示 论文

259 容斥原理与反演 论文

260 左偏树的特点及其应用 论文

261 平面图在信息学中的应用 论文

263 数值分析在竞赛中的简单应用 论文

264 数据结构的联合——块状链表 论文

265 最小割模型在信息学竞赛中的应用 论文

266 构造题策略 讲义

267 母函数的性质及应用 论文

268 猜数问题的研究 论文

269 答案只有一个——浅谈问答式交互问题 论文

270 维护森林连通性——动态树 论文

271 计算几何讲义 讲义

272 论随机化算法的原理与设计 论文

288 背包问题专题 题解

289 BalkOI 2026 试题与在线镜像（巴尔干信息学奥林匹克，地拉那） 真题

290 CCF CSP 例题解析与实战技巧：算法应用 题解

292 eJOI 2026 试题·题解·数据（欧洲初中生信息学奥林匹克，立陶宛考纳斯） 真题

293 信息学奥赛一本通 · 高手训练 题解

294 区间与序列 DP 专题 题解

295 单调栈与单调队列 题解

296 字符串哈希与 KMP 题解

297 树形 DP 专题 题解

298 树状数组 题解

299 算法思维：竞赛真题精选精讲（第 2 版） 题解

300 算法竞赛入门经典——训练指南 题解

305 二分图匹配（匈牙利 / Hopcroft–Karp / KM） 题解

306 博弈论与 SG 函数 题解

431 数位 DP 专题 题解

308 状压 DP 专题 题解

309 矩阵与线性递推 题解

310 线段树与懒标记 题解

311 2-SAT 与差分约束 题解

312 AC 自动机 题解

313 ACM 国际大学生程序设计竞赛系列 · 题目与解读 题解

317 BOI 试题库与 2026 年题目（波罗的海信息学奥林匹克，1995—2026 全量题面、数据与题解） 真题

319 DP 优化专题（单调队列/斜率/决策单调性） 题解

326 Manacher 与回文自动机 题解

327 SCOI 四川省选 真题

328 SDOI 山东省选 真题

329 SHOI 上海队选拔 真题

331 ZJOI 浙江省选 真题

333 可持久化数据结构 题解

334 后缀数组与后缀自动机 题解

335 国家集训队论文配套题 真题

336 多项式与 NTT 题解

337 平衡树（Treap / Splay / FHQ） 题解

338 插头 DP（连通性状态压缩） 题解

339 树链剖分（HLD） 题解

340 网络流（最大流 / 最小割 / 费用流） 题解

342 莫比乌斯反演与杜教筛 题解

343 虚树 题解

344 计算几何基础与凸包 题解

346 EGOI 2026 试题与逐题统计（第 6 届欧洲女子信息学奥林匹克） 真题

348 IOI 2026 中国国家集训队集中培训与精英培训题目及数据 真题

349 IOI 2026 试题与题解（第 38 届国际信息学奥林匹克，塔什干） 真题

350 Min_25 筛与洲阁筛 题解

355 生成函数与组合计数 题解

366 AcWing 算法提高课 课程

369 Codeforces EDU 课程

370 Codeforces EDU（算法教程） 课程

372 **UCF COP 4516 Contest Problem Solving（中佛罗里达大学竞赛解题课）** 课程

376 **灵茶山艾府（B 站 / 题解）** 课程

379 **NOI 2025 冬令营授课与交流活动现场视频（14 场）** 课程

381 **APIO 2025 讲者授课现场视频（5 场）** 课程

382 **APIO 2026 中国区活动讲座现场视频（7 场）** 课程

383 **AcWing 算法进阶课** 课程

384 **CS 3233（NUS 竞赛编程）** 课程

385 **CCF NOI 冬令营 2026 第一课堂（CCF 数字图书馆）** 课程

388 **CMU 15-295 Competition Programming and Problem Solving** 课程

389 **CMU 15-451（算法设计与分析）** 课程

390 **NOI 冬令营 2026 讲座与交流活动现场视频（14 场）** 课程

391 **Stanford CS161（算法设计与分析）** 课程

418 **OI Wiki** 社区

423 **Codeforces Blog（题解与教程）** 社区

424 **演算法筆記** 社区

425 **Matrix67 博客** 社区

426 **cp-algorithms** 社区

427 **hzwer 的博客** 社区

430 **《Olympiads in Informatics》第 20 卷（IOI 学术期刊，2026）** 期刊

赛前冲刺 76 条

004 **CCF CSP 第一轮认证一本通（第 2 版）** 教材

017 **信息学奥赛一本通** 教材

018 **信息学奥赛初赛全解** 教材

038 **高效通关信息学竞赛·初赛篇（入门组）** 教材

104 **CCF NOI 系列活动考场纪律** 官方

112 **CCF CSP 认证官方说明** 官方

113 **CCF CSP 认证官网** 官方

116 **CCF 关于 CSP-J/S 2026 第一轮晋级第二轮的规定** 官方

120 **CSP-J/S 2026 第二轮报名通知** 官方

121 **CSP-J/S 认证须知** 官方

125 **关于 NOI 系列活动中编程语言使用限制的补充说明** 官方

135 **CCF 关于 NOIP 2026 报名的通知** 官方

137 **NOIP 评奖规则** 官方

139 **第六届女生竞赛报名通知（NOIP 2026 女生赛）** 官方

141 **APIO 2026 中国区活动赛事综述与分数线** 官方

142 **CCF 关于 NOI 2027 省内选拔的若干规定** 官方

146 **NOI 2027 联合省选：两试时间与成绩构成对照（26 省市）** 官方

148 **APIO 官网** 官方

149 **Baltic OI** 官方

150 **CCF NOI 集训营 2026 综合素质评价机制（附国家集训队行为规范）** 官方

151 **CEOI** 官方

152 **IOI 2027 中国国家队选拔规则与国家集训队行为规范** 官方

156 **NOI 试题分析与年鉴** 官方

157 **POI** 官方

158 **USACO 2025-2026 赛季赛制改革：三场线上赛 + 监考制 US Open** 官方

160 **IOI 2026 中国队 3 金 1 银与最终排名** 官方

162 **IOI 官网** 官方

163 **IOI 年度报告与命题趋势** 官方

179 **梦熊 OJ（Mengxiong Online Judge）** 平台

180 **洛谷** 平台

182 **洛谷公开赛与月赛** 平台

192 **Codeforces** 平台

194 **Comet OJ** 平台

203 **牛客网** 平台

204 **Codeforces Gym** 平台

209 **Nowcoder 多校训练营** 平台

211 **QOJ** 平台

212 **UOJ** 平台

214 **Universal Cup** 平台

215 **oj.uz** 平台

224 **骗分导论** 讲义

226 **常数优化与卡常技巧** 讲义

227 **浅析非完美算法在信息学竞赛中的应用** 论文

262 **提交答案题与交互题** 讲义

273 CCF NOI 2026 笔试题库 真题

276 CSP-J/S 历年认证题 真题

277 CSP-J/S 第一轮真题 真题

281 信息学奥赛 3：历年真题解析与拉分解题思维训练 题解

282 信息学奥赛 CSP 通关之路：CSP-J/S 第一轮原创全真模拟试卷集（2026） 真题

283 信息学奥赛 CSP-J 初赛通关手册：10 年真题 + 10 套模拟精练精讲 真题

301 JOI 2025/2026 二次预选（第 25 回日本信息学奥赛第二轮）题目・解说・数据 真题

302 NOIP 2025 题目及数据（糖果店 / 清仓甩卖 / 树的价值 / 序列询问） 真题

303 NOIP 历年真题（2011 起） 真题

304 NOIP 历年试题 真题

316 BJOI 北京队选拔 真题

318 CEOI 2026 试题与数据（第 33 届中欧信息学奥林匹克，斯洛文尼亚卢布尔雅那） 真题

329 SHOI 上海队选拔 真题

330 WEOI 2026 试题与题解（第 4 届西欧信息学奥林匹克，卢森堡） 真题

331 ZJOI 浙江省选 真题

332 十二省联考 真题

341 联合省选（CCF 统一命题） 真题

345 APIO 历年试题 真题

347 IOI 2025 官方赛题包（ioi/task-archive 2025，6 题全量数据） 真题

351 NOI 2026 题目及数据 真题

352 NOI 冬令营 2026 题目及数据 真题

353 NOI 历年真题 真题

354 NOI 历年试题 真题

356 IOI 历年试题 真题

363 洛谷网校 课程

381 APIO 2025 讲者授课现场视频（5 场） 课程

382 APIO 2026 中国区活动讲座现场视频（7 场） 课程

386 CCF NOI 集训营 2026 报到通知（附日程） 课程

387 CCF 关于举办 IOI 2027 精英培训的通知 课程

393 IOI 官方 YouTube 频道 课程

408 NOI Linux 2.0 工具

431 许淇文 IOI 2026 参赛总结 社区

教练备课 79 条

001 48 课搞定信息学奥赛：C++ 趣味编程 教材

008 CCF 信息学奥赛基础篇 教材

012 信奥逐梦之路：家长陪伴与升学规划全阶段指南 专著

022 信息学竞赛教程（初级） 教材

056 算法导论（核心篇）（原书第 4 版） 专著

057 算法思维训练：信奥大师的算法教与学秘籍 专著

065 计算思维训练——动态规划 教材

066 ACM 国际大学生程序设计竞赛系列 · 知识与入门 教材

076 Algorithm Design（Kleinberg – Tardos） 教材

077 Competitive Programming 4 教材

084 数据结构与算法（上下册） 教材

093 Handbook of Data Structures and Applications（2nd Edition） 专著

105 NOI 大纲（2025 年修订版） 官方

106 NOI 官网 官方

107 NOI 系列活动和认证日历 官方

108 《中国计算机学会通讯》（CCCF） 官方

109 《信息学奥林匹克竞赛年鉴》 官方

110 2024 年 NOI 活动题目分析报告（CSP-J/S 2024 第二轮 + NOIP 2024 知识构成） 官方

111 2026 年 WCET & LMCC & NOI 教师发展活动日历 官方

112 CCF CSP 认证官方说明 官方

114 CCF NOI 教师培训考核管理规则 官方

117 CCF 关于 CSP-J/S 2026 第一轮认证评级分数线的公告 官方

118 CCF 关于 NOI 相关权益的声明 官方

119 CSP-J/S 2026 第一轮认证证书申请通知 官方

122 NOI Linux 2.0 官方发布页 官方

124 关于 CCF CSP-J/S 认证等级名称规范的公告 官方

126 关于参加 CSP-J/S 认证需符合年龄条件的公告（12 周岁门槛） 官方

127 关于重申在校在职 NOI 指导教师不得校外有偿兼职授课的公告 官方

130 2025 年 CSP-J/S 第二轮认证题目分析报告 官方

131 CCF 全国青少年信息学奥林匹克竞赛教程 官方

133 NOIP 2026 参赛名额分配方案 官方

134 2023 全国信息学奥林匹克年鉴（电子版） 官方

135 CCF 关于 NOIP 2026 报名的通知 官方

137 NOIP 评奖规则 官方

138 NOIP 题目综合分析报告 官方

140 2022—2023 年国内外信息学主要赛事分析报告 官方

141 APIO 2026 中国区活动赛事综述与分数线 官方

142 CCF 关于 NOI 2027 省内选拔的若干规定 官方

145 NOI 2027 名额分配测算方案 官方

147 2024 全国信息学奥林匹克年鉴 官方

150 CCF NOI 集训营 2026 综合素质评价机制（附国家集训队行为规范） 官方

152 IOI 2027 中国国家队选拔规则与国家集训队行为规范 官方

153 IOI Syllabus 官方

154 NOI 冬令营 2026 首次 AI 辅助编程测试（人机协同评测） 官方

155 NOI 参赛资格规定 官方

156 NOI 试题分析与年鉴 官方

159 关于自 NOI 2029 起取消 A 类选手加分政策的公告 官方

160 IOI 2026 中国队 3 金 1 银与最终排名 官方

161 IOI 2027 官方网站（第 39 届，德国波茨坦 HPI，2027-09-12—19） 官方

162 IOI 官网 官方

163 IOI 年度报告与命题趋势 官方

164 HydroOJ 平台

216 信息学竞赛中的思维方法 论文

217 测试数据设计中的三要素 论文

218 ISSEP 2026 论文集：AI 时代的中小学信息学教育 论文

230 NOIpaper：国家集训队论文集（1999—2025） 论文

231 OI 论文集精读 论文

232 Trinkle23897 的 OI 课件 讲义

234 huzecong 的 OI 课件 讲义

354 NOI 历年试题 真题

365 2026 年第四期 NOI 教师培训（南京站，中级/初级） 课程

372 UCF COP 4516 Contest Problem Solving（中佛罗里达大学竞赛解题课） 课程

379 NOI 2025 冬令营授课与交流活动现场视频（14 场） 课程

382 APIO 2026 中国区活动讲座现场视频（7 场） 课程

385 CCF NOI 冬令营 2026 第一课堂（CCF 数字图书馆） 课程

386 CCF NOI 集训营 2026 报到通知（附日程） 课程

387 CCF 关于举办 IOI 2027 精英培训的通知 课程

389 CMU 15-451（算法设计与分析） 课程

390 NOI 冬令营 2026 讲座与交流活动现场视频（14 场） 课程

392 蒋炎岩（JYY）课程站 课程

394 Arbiter 工具

395 Cena 工具

396 DOMjudge 工具

397 Lemon 工具

401 LOJ — LeLe Online Judge（开源可自部署的 ACM/OI 双赛制评测系统） 工具

412 CF Polygon（出题系统） 工具

415 testlib.h 工具

421 OI Wiki 英文镜像 / 社区分支 社区

430 《Olympiads in Informatics》第 20 卷（IOI 学术期刊，2026） 期刊

查漏补缺 90 条

002 C++编程与信息学竞赛数学思维 教材

003 CCF CSP 例题解析与实战技巧：编程基础 教材

004 CCF CSP 第一轮认证一本通（第 2 版） 教材

005 CCF GESP 直通车：C++ 六级七级精讲精练 教材

009 Cracking the Coding Interview 教材

013 信息学奥赛 CSP-J/S 数学考点大全 教材

018 信息学奥赛初赛全解 教材

027 用 AI 决胜信息学奥赛：C++ 编程基础 教材

028 算法与数学：数学思维与编程实践 教材

038 高效通关信息学竞赛·初赛篇（入门组） 教材

041 C++ Primer 教材

045 Competitive Programmer's Handbook 教材

047 Elements of Programming Interviews 教材

056 算法导论（核心篇）（原书第 4 版） 专著

069 C++，挑战编程——程序设计竞赛进阶训练指南 教材

070 Competitive Programming 4（配套代码库） 教材

080 The Algorithm Design Manual 教材

085 离散数学及其应用 专著

086 算法导论 专著

087 算法导论（扩展篇）（原书第 4 版） 教材

090 算法设计手册 教材

098 编程原本 专著

103 计算机程序设计艺术 专著

105 NOI 大纲（2025 年修订版） 官方

106 NOI 官网 官方

110 2024 年 NOI 活动题目分析报告（CSP-J/S 2024 第二轮 + NOIP 2024 知识构成） 官方

116 CCF 关于 CSP-J/S 2026 第一轮晋级第二轮的规定 官方

117 CCF 关于 CSP-J/S 2026 第一轮认证评级分数线的公告 官方

118 CCF 关于 NOI 相关权益的声明 官方

119 CSP-J/S 2026 第一轮认证证书申请通知 官方

122 NOI Linux 2.0 官方发布页 官方

124 关于 CCF CSP-J/S 认证等级名称规范的公告 官方

125 关于 NOI 系列活动中编程语言使用限制的补充说明 官方

126 关于参加 CSP-J/S 认证需符合年龄条件的公告（12 周岁门槛） 官方

130 2025 年 CSP-J/S 第二轮认证题目分析报告 官方

131 CCF 全国青少年信息学奥林匹克竞赛教程 官方

133 NOIP 2026 参赛名额分配方案 官方

134 2023 全国信息学奥林匹克年鉴（电子版） 官方

138 NOIP 题目综合分析报告 官方

139 第六届女生竞赛报名通知（NOIP 2026 女生赛） 官方

140 2022—2023 年国内外信息学主要赛事分析报告 官方

145 NOI 2027 名额分配测算方案 官方

146 NOI 2027 联合省选：两试时间与成绩构成对照（26 省市） 官方

147 2024 全国信息学奥林匹克年鉴 官方

153 IOI Syllabus 官方

161 IOI 2027 官方网站（第 39 届，德国波茨坦 HPI，2027-09-12—19） 官方

174 USFCA 数据结构可视化 平台

176 VisuAlgo 平台

181 洛谷主题题库 平台

188 AtCoder Problems 平台

193 Codeforces 题目集（按 rating 筛选） 平台

205 Judge Duck Online 平台

208 LibreOJ 平台

210 OEIS 整数数列百科 平台

213 Library Checker 平台

220 Legend-K 的数据结构与算法笔记 讲义

233 acm-cheat-sheet 讲义

273 CCF NOI 2026 笔试题库 真题

274 CSP-J/S 2025 题目及数据（入门组 4 题 + 提高组 4 题） 真题

275 CSP-J/S 2026 第二轮认证测试数据（入门组 + 提高组） 真题

277 CSP-J/S 第一轮真题 真题

278 JOI 2026/2027 一次预选（第 26 回日本信息学奥赛第一轮）题目 · 解说 · 解答例 真题

280 《CCF 信息学奥赛入门篇》练习指导 题解

281 信息学奥赛 3：历年真题解析与拉分解题思维训练 题解

286 算法竞赛入门经典——习题与解答 题解

357 OI Wiki 专题页 课程

359 USACO Guide 课程

364 董晓算法（B 站） 课程

376 灵茶山艾府（B 站 / 题解） 课程

377 AtCoder Library Practice 课程

402 Oracle VM VirtualBox 工具

403 VMware Workstation Player 工具

405 Compiler Explorer (godbolt) 工具

406 Desmos 图形计算器 工具

407 GeoGebra 工具

410 对拍脚本 工具

411 GDB 工具

414 Wolfram Alpha 工具

416 Valgrind 工具

417 perf 工具

418 OI Wiki 社区

419 Stack Overflow 社区

420 GeeksforGeeks 社区

421 OI Wiki 英文镜像 / 社区分支 社区

422 洛谷讨论区 社区

423 **Codeforces Blog（题解与教程）** 社区

426 **cp-algorithms** 社区

427 **hzwer 的博客** 社区

428 **Computer Science StackExchange** 社区

429 **Mathematics StackExchange** 社区

实战训练 94 条

011 **中学生计算机程序算法入门：例题精析与训练** 教材

014 **信息学奥赛1：114道题零基础吃透C++与算法基础** 教材

015 **信息学奥赛2：121道题零基础吃透高级算法与数据结构** 教材

033 **算法竞赛试炼场：洛谷300题精析** 教材

042 **CCF CSP 例题解析与实战技巧：工程实现** 教材

055 **程序设计竞赛进阶：算法与实践** 教材

058 **算法竞赛实战笔记** 教材

070 **Competitive Programming 4（配套代码库）** 教材

123 **USACO 官网** 官方

132 **COCI** 官方

136 **JOI** 官方

143 **CCPC 官网** 官方

144 **ICPC 官网** 官方

164 **HydroOJ** 平台

165 **AcWing** 平台

168 **AtCoder** 平台

170 **LeetCode** 平台

173 **USACO** 平台

175 **Vijos** 平台

177 **WZOI（温州中学 OJ）** 平台

179 **梦熊 OJ（Mengxiong Online Judge）** 平台

180 **洛谷** 平台

182 **洛谷公开赛与月赛** 平台

183 **洛谷题单** 平台

184 **编程猫 OJ（北大—点猫人工智能教育联合实验室）** 平台

185 **蓝桥云课** 平台

186 **计蒜客** 平台

188 **AtCoder Problems** 平台

189 **Baekjoon Online Judge** 平台

191 **CodeChef** 平台

192 **Codeforces** 平台

193 **Codeforces 题目集（按 rating 筛选）** 平台

195 **DMOJ** 平台

196 **HDU Online Judge** 平台

197 **Lutece** 平台

198 **POJ** 平台

199 **SPOJ** 平台

200 **UVa Online Judge** 平台

201 **ZOJ** 平台

202 **e-olymp** 平台

204 **Codeforces Gym** 平台

206 **Kattis** 平台

207 **Timus** 平台

208 **LibreOJ** 平台

209 **Nowcoder 多校训练营** 平台

211 **QOJ** 平台

212 **UOJ** 平台

274 **CSP-J/S 2025 题目及数据（入门组 4 题 + 提高组 4 题）** 真题

275 **CSP-J/S 2026 第二轮认证测试数据（入门组 + 提高组）** 真题

278 **JOI 2026/2027 一次预选（第 26 回日本信息学奥赛第一轮）题目·解说·解答例** 真题

279 **USACO 历年试题** 真题

280 **《CCF 信息学奥赛入门篇》练习指导** 题解

282 **信息学奥赛 CSP 通关之路：CSP-J/S 第一轮原创全真模拟试卷集（2026）** 真题

283 **信息学奥赛 CSP-J 初赛通关手册：10 年真题 + 10 套模拟精练精讲** 真题

284 **信息学奥赛一本通** 题解

285 **洛谷主题题单** 真题

287 **算法竞赛宝典** 题解

289 **BalkOI 2026 试题与在线镜像（巴尔干信息学奥林匹克，地拉那）** 真题

290 **CCF CSP 例题解析与实战技巧：算法应用** 题解

291 **Programming Challenges** 题解

292 eJOI 2026 试题·题解·数据（欧洲初中生信息学奥林匹克，立陶宛考纳斯） 真题

293 信息学奥赛一本通 · 高手训练 题解

300 算法竞赛入门经典——训练指南 题解

301 JOI 2025/2026 二次预选（第 25 回日本信息学奥赛第二轮）题目·解说·数据 真题

302 NOIP 2025 题目及数据（糖果店 / 清仓甩卖 / 树的价值 / 序列询问） 真题

303 NOIP 历年真题（2011 起） 真题

313 ACM 国际大学生程序设计竞赛系列 · 题目与解读 题解

314 AHOI 安徽省选 真题

315 APIO 2026 中国区活动题目及数据 真题

317 BOI 试题库与 2026 年题目（波罗的海信息学奥林匹克，1995—2026 全量题面、数据与题解） 真题

318 CEOI 2026 试题与数据（第 33 届中欧信息学奥林匹克，斯洛文尼亚卢布尔雅那） 真题

320 FJOI 福建省选 真题

321 GDOI 广东省选 真题

322 HAOI 河南省选 真题

323 HEOI 河北省选 真题

324 HNOI 湖南省选 真题

325 JSOI 江苏省选 真题

327 SCOI 四川省选 真题

328 SDOI 山东省选 真题

330 WEOI 2026 试题与题解（第 4 届西欧信息学奥林匹克，卢森堡） 真题

341 联合省选（CCF 统一命题） 真题

346 EGOI 2026 试题与逐题统计（第 6 届欧洲女子信息学奥林匹克） 真题

347 IOI 2025 官方赛题包（ioi/task-archive 2025，6 题全量数据） 真题

348 IOI 2026 中国国家集训队集中培训与精英培训题目及数据 真题

349 IOI 2026 试题与题解（第 38 届国际信息学奥林匹克，塔什干） 真题

351 NOI 2026 题目及数据 真题

352 NOI 冬令营 2026 题目及数据 真题

388 CMU 15-295 Competition Programming and Problem Solving 课程

398 CP Editor 工具

401 LOJ — LeLe Online Judge（开源可自部署的 ACM/OI 双赛制评测系统） 工具

404 洛谷 IDE 工具

409 Visual Studio Code 工具

410 对拍脚本 工具

413 CLion 工具

第 15 章　索引三 · 按作用效果

> 从「这份资料该怎么用」出发。速查类不必通读，深耕类需要整块时间，实战类必须动手，方法论类改变的是思路而非知识量。

速查 97 条

003 CCF CSP 例题解析与实战技巧：编程基础 教材

004 CCF CSP 第一轮认证一本通（第 2 版） 教材

005 CCF GESP 直通车：C++ 六级七级精讲精练 教材

013 信息学奥赛 CSP-J/S 数学考点大全 教材

017 信息学奥赛一本通 教材

018 信息学奥赛初赛全解 教材

024 我的第一本算法书 教材

035 算法设计与分析（图解版） 教材

038 高效通关信息学竞赛 · 初赛篇（入门组） 教材

041 C++ Primer 教材

045 Competitive Programmer's Handbook 教材

070 Competitive Programming 4（配套代码库） 教材

090 算法设计手册 教材

104 CCF NOI 系列活动考场纪律 官方

105 NOI 大纲（2025 年修订版） 官方

106 NOI 官网 官方

107 NOI 系列活动和认证日历 官方

111 2026 年 WCET & LMCC & NOI 教师发展活动日历 官方

112 CCF CSP 认证官方说明 官方

113 CCF CSP 认证官网 官方

114 CCF NOI 教师培训考核管理规则 官方

116 CCF 关于 CSP-J/S 2026 第一轮晋级第二轮的规定 官方

117 CCF 关于 CSP-J/S 2026 第一轮认证评级分数线的公告 官方

118 CCF 关于 NOI 相关权益的声明 官方

119 CSP-J/S 2026 第一轮认证证书申请通知 官方

120 CSP-J/S 2026 第二轮报名通知 官方

121 CSP-J/S 认证须知 官方

122 NOI Linux 2.0 官方发布页 官方

124 关于 CCF CSP-J/S 认证等级名称规范的公告 官方

125 关于 NOI 系列活动中编程语言使用限制的补充说明 官方

126 关于参加 CSP-J/S 认证需符合年龄条件的公告（12 周岁门槛） 官方

127 关于重申在校在职 NOI 指导教师不得校外有偿兼职授课的公告 官方

128 电子学会青少年等级考试 官方

133 NOIP 2026 参赛名额分配方案 官方

135 CCF 关于 NOIP 2026 报名的通知 官方

137 NOIP 评奖规则 官方

139 第六届女生竞赛报名通知（NOIP 2026 女生赛） 官方

141 APIO 2026 中国区活动赛事综述与分数线 官方

145 NOI 2027 名额分配测算方案 官方

146 NOI 2027 联合省选：两试时间与成绩构成对照（26 省市） 官方

152 IOI 2027 中国国家队选拔规则与国家集训队行为规范 官方

153 IOI Syllabus 官方

155 NOI 参赛资格规定 官方

160 IOI 2026 中国队 3 金 1 银与最终排名 官方

161 IOI 2027 官方网站（第 39 届，德国波茨坦 HPI，2027-09-12—19） 官方

162 IOI 官网 官方

167 Algorithm Visualizer 平台

174 USFCA 数据结构可视化 平台

175 Vijos 平台

176 VisuAlgo 平台

188 AtCoder Problems 平台

205 Judge Duck Online 平台

210 OEIS 整数数列百科 平台

213 **Library Checker** 平台

219 **OI Wiki 学习路线（C++ 基础 → CSP-J 入门级知识清单）** 讲义

220 **Legend-K 的数据结构与算法笔记** 讲义

223 **背包九讲** 讲义

226 **常数优化与卡常技巧** 讲义

229 **矩阵乘法与线性递推** 论文

233 **acm-cheat-sheet** 讲义

240 **强连通分量与 2-SAT** 论文

260 **左偏树的特点及其应用** 论文

273 **CCF NOI 2026 笔试题库** 真题

274 **CSP-J/S 2025 题目及数据（入门组 4 题 + 提高组 4 题）** 真题

298 **树状数组** 题解

309 **矩阵与线性递推** 题解

311 **2-SAT 与差分约束** 题解

326 **Manacher 与回文自动机** 题解

357 **OI Wiki 专题页** 课程

364 **董晓算法（B 站）** 课程

393 **IOI 官方 YouTube 频道** 课程

394 **Arbiter** 工具

395 **Cena** 工具

396 **DOMjudge** 工具

397 **Lemon** 工具

398 **CP Editor** 工具

399 **Code::Blocks** 工具

400 **Dev-C++** 工具

402 **Oracle VM VirtualBox** 工具

403 **VMware Workstation Player** 工具

404 **洛谷 IDE** 工具

405 **Compiler Explorer (godbolt)** 工具

406 **Desmos 图形计算器** 工具

407 **GeoGebra** 工具

408 **NOI Linux 2.0** 工具

409 **Visual Studio Code** 工具

411 **GDB** 工具

413 **CLion** 工具

414 **Wolfram Alpha** 工具

416 **Valgrind** 工具

417 **perf** 工具

418 **OI Wiki** 社区

419 **Stack Overflow** 社区

420 **GeeksforGeeks** 社区

421 **OI Wiki 英文镜像 / 社区分支** 社区

422 **洛谷讨论区** 社区

426 **cp-algorithms** 社区

深耕 179 条

001 **48 课搞定信息学奥赛：C++ 趣味编程** 教材

002 **C++编程与信息学竞赛数学思维** 教材

005 **CCF GESP 直通车：C++ 六级七级精讲精练** 教材

006 **CCF 中学生计算机程序设计** 教材

007 **CCF 中学生计算机程序设计** 教材

008 **CCF 信息学奥赛基础篇** 教材

013 **信息学奥赛 CSP-J/S 数学考点大全** 教材

016 **信息学奥赛一本通** 教材

019 **信息学奥赛大通关·入门篇（上下册）** 教材

020 **信息学奥赛打通关 · 入门篇（全 2 册）** 教材

021 **信息学奥赛课课通** 教材

022 **信息学竞赛教程（初级）** 教材

023 **啊哈！算法** 教材

025 **我的第一本算法竞赛启蒙书** 教材

026 **深入浅出程序设计竞赛** 教材

028 **算法与数学：数学思维与编程实践** 教材

029 **算法图解** 教材

030 **算法竞赛从新手到高手** 教材

031 **算法竞赛入门到进阶** 教材

032 **算法竞赛入门经典** 教材

033 **算法竞赛试炼场：洛谷300题精析** 教材

034 **算法竞赛黄金法则：提高算法和思考力的77项技巧** 教材

035 **算法设计与分析（图解版）** 教材

036 **聪明人的游戏：信息学探秘（提高篇）** 教材

037 **计算概论：C++ 编程与信息学竞赛入门** 教材

039 **Algorithms Unlocked** 教材

040 **Algorithms, 4th Edition（Sedgewick）** 教材

042 **CCF CSP 例题解析与实战技巧：工程实现** 教材

043 CCF 中学生计算机程序设计 教材

044 CCF 信息学奥赛提高篇 教材

045 Competitive Programmer's Handbook 教材

046 Competitive Programming in Python 教材

048 Guide to Competitive Programming 教材

050 信息学奥赛一本通 · 提高篇 教材

051 信息学奥赛之数学一本通 教材

053 数据结构与算法分析 教材

054 数据结构（C++ 语言版） 专著

055 程序设计竞赛进阶：算法与实践 教材

056 算法导论（核心篇）（原书第 4 版） 专著

059 算法竞赛核心教程：策略与算法深度解析 教材

060 算法竞赛进阶指南 教材

061 算法训练营：进阶篇（全彩版） 教材

062 算法设计与分析 教材

063 算法设计与分析：以 ACM 大学生程序设计竞赛在线题库为例（第 2 版 · 微课版） 教材

064 算法（第 4 版） 教材

065 计算思维训练——动态规划 教材

066 ACM 国际大学生程序设计竞赛系列 · 知识与入门 教材

067 Algorithms（Dasgupta‐Papadimitriou‐Vazirani） 教材

068 Algorithms（Jeff Erickson） 教材

071 Mathematics for Computer Science 教材

073 算法详解（Algorithms Illuminated，卷 1—4） 教材

075 ACM 国际大学生程序设计竞赛系列 · 算法与实现 教材

076 Algorithm Design（Kleinberg‐Tardos） 教材

077 Competitive Programming 4 教材

078 Introduction to Graph Theory（West） 教材

079 Introduction to Probability 教材

080 The Algorithm Design Manual 教材

081 信息学竞赛中的数学 教材

082 具体数学 专著

083 挑战程序设计竞赛 2 教材

084 数据结构与算法（上下册） 教材

085 离散数学及其应用 专著

086 算法导论 专著

087 算法导论（扩展篇）（原书第 4 版） 教材

088 算法竞赛中的数论 教材

089 算法艺术与信息学竞赛 教材

091 组合数学 专著

092 Graph Theory（Bondy & Murty） 教材

093 Handbook of Data Structures and Applications（2nd Edition） 专著

094 Linear Algebra Done Right 教材

095 Structure and Interpretation of Computer Programs 教材

096 概率论与数理统计 专著

097 线性代数及其应用 专著

099 计算几何：算法与应用 专著

100 ACM Transactions on Algorithms（TALG） 专著

101 Algorithmica 专著

102 Journal of Graph Algorithms and Applications（JGAA） 专著

103 计算机程序设计艺术 专著

110 2024 年 NOI 活动题目分析报告（CSP-J/S 2024 第二轮 + NOIP 2024 知识构成） 官方

134 2023 全国信息学奥林匹克年鉴（电子版） 官方

140 2022—2023 年国内外信息学主要赛事分析报告 官方

147 2024 全国信息学奥林匹克年鉴 官方

218 ISSEP 2026 论文集：AI 时代的中小学信息学教育 论文

221 搜索方法中的剪枝优化 论文

223 背包九讲 讲义

225 字符串算法选讲 讲义

228 浅谈用极大化思想解决最大子矩形问题 论文

230 NOIpaper：国家集训队论文集（1999—2025） 论文

231 OI 论文集精读 论文

232 Trinkle23897 的 OI 课件 讲义

234 huzecong 的 OI 课件 讲义

235 分块与莫队算法 论文

236 动态规划优化专题 讲义

237 动态规划的深入讨论 论文

238 动态规划算法的优化技巧 论文

243 树型动态规划 论文

244 状态压缩动态规划 论文

246 线段树的应用 论文

247 组合游戏略述——浅谈 SG 游戏的若干拓展及变形 论文

248 网络流算法的应用 论文

249 解决动态统计问题的两把利刃 论文

250 Pólya 原理及其应用 论文

251 伸展树的基本操作与应用 论文

252 充分利用问题性质——例析动态规划的「个性化」优化 论文

253 具体数学讲义 讲义

254 动态树及其应用 论文

255 可持久化数据结构 论文

256 后缀数组 论文

257 后缀数组——处理字符串的有力工具 讲义

258 多串匹配算法及其启示 论文

259 容斥原理与反演 论文

261 平面图在信息学中的应用 论文

263 数值分析在竞赛中的简单应用 论文

264 数据结构的联合——块状链表 论文

267 母函数的性质及应用 论文

268 猜数问题的研究 论文

269 答案只有一个——浅谈问答式交互问题 论文

270 维护森林连通性——动态树 论文

271 计算几何讲义 讲义

288 背包问题专题 题解

294 区间与序列 DP 专题 题解

295 单调栈与单调队列 题解

296 字符串哈希与 KMP 题解

297 树形 DP 专题 题解

299 算法思维：竞赛真题精选精讲（第 2 版） 题解

305 二分图匹配（匈牙利 / Hopcroft－Karp / KM） 题解

306 博弈论与 SG 函数 题解

431 数位 DP 专题 题解

308 状压 DP 专题 题解

310 线段树与懒标记 题解

312 AC 自动机 题解

319 DP 优化专题（单调队列/斜率/决策单调性） 题解

333 可持久化数据结构 题解

334 后缀数组与后缀自动机 题解

336 多项式与 NTT 题解

337 平衡树（Treap / Splay / FHQ） 题解

338 插头 DP（连通性状态压缩） 题解

339 树链剖分（HLD） 题解

340 网络流（最大流 / 最小割 / 费用流） 题解

342 莫比乌斯反演与杜教筛 题解

343 虚树 题解

344 计算几何基础与凸包 题解

347 IOI 2025 官方赛题包（ioi/task-archive 2025，6 题全量数据） 真题

348 IOI 2026 中国国家集训队集中培训与精英培训题目及数据 真题

350 Min_25 筛与洲阁筛 题解

355 生成函数与组合计数 题解

358 AcWing 算法基础课 课程

359 USACO Guide 课程

361 中国大学 MOOC 课程

366 AcWing 算法提高课 课程

367 Berkeley CS61B（数据结构） 课程

368 CS 97SI（斯坦福竞赛编程） 课程

371 Princeton Algorithms（Coursera） 课程

373 中国大学 MOOC《算法设计与分析》（北大） 课程

374 学堂在线 课程

375 数据结构（清华） 课程

376 灵茶山艾府（B 站 / 题解） 课程

378 MIT OCW 6.006（算法导论） 课程

379 NOI 2025 冬令营授课与交流活动现场视频（14 场） 课程

380 算法设计与分析（MIT 6.006） 课程

381 APIO 2025 讲者授课现场视频（5 场） 课程

382 APIO 2026 中国区活动讲座现场视频（7 场） 课程

383 AcWing 算法进阶课 课程

384 CS 3233（NUS 竞赛编程） 课程

385 CCF NOI 冬令营 2026 第一课堂（CCF 数字图书馆） 课程

389 CMU 15-451（算法设计与分析） 课程

391 Stanford CS161（算法设计与分析） 课程

392 蒋炎岩（JYY）课程站 课程

412 CF Polygon（出题系统） 工具

415 testlib.h 工具

418 OI Wiki 社区

423 **Codeforces Blog（题解与教程）** 社区

424 **演算法筆記** 社区

425 **Matrix67 博客** 社区

426 **cp-algorithms** 社区

427 **hzwer 的博客** 社区

428 **Computer Science StackExchange** 社区

429 **Mathematics StackExchange** 社区

430 **《Olympiads in Informatics》第 20 卷（IOI 学术期刊，2026）** 期刊

实战 140 条

003 **CCF CSP 例题解析与实战技巧：编程基础** 教材

004 **CCF CSP 第一轮认证一本通（第 2 版）** 教材

009 **Cracking the Coding Interview** 教材

011 **中学生计算机程序算法入门：例题精析与训练** 教材

014 **信息学奥赛1：114道题零基础吃透C++与算法基础** 教材

015 **信息学奥赛2：121道题零基础吃透高级算法与数据结构** 教材

017 **信息学奥赛一本通** 教材

018 **信息学奥赛初赛全解** 教材

033 **算法竞赛试炼场：洛谷300题精析** 教材

038 **高效通关信息学竞赛·初赛篇（入门组）** 教材

042 **CCF CSP 例题解析与实战技巧：工程实现** 教材

047 **Elements of Programming Interviews** 教材

049 **Guide to Competitive Programming** 教材

052 **挑战程序设计竞赛** 教材

055 **程序设计竞赛进阶：算法与实践** 教材

058 **算法竞赛实战笔记** 教材

061 **算法训练营：进阶篇（全彩版）** 教材

063 **算法设计与分析：以 ACM 大学生程序设计竞赛在线题库为例（第 2 版·微课版）** 教材

069 **C++，挑战编程——程序设计竞赛进阶训练指南** 教材

077 **Competitive Programming 4** 教材

123 **USACO 官网** 官方

129 **蓝桥杯** 官方

132 **COCI** 官方

136 **JOI** 官方

143 **CCPC 官网** 官方

144 **ICPC 官网** 官方

148 **APIO 官网** 官方

149 **Baltic OI** 官方

151 **CEOI** 官方

157 **POI** 官方

162 **IOI 官网** 官方

164 **HydroOJ** 平台

165 **AcWing** 平台

166 **Aizu Online Judge** 平台

168 **AtCoder** 平台

169 **HackerRank** 平台

170 **LeetCode** 平台

171 **OpenJudge** 平台

172 **PTA 拼题 A** 平台

173 **USACO** 平台

175 **Vijos** 平台

177 **WZOI（温州中学 OJ）** 平台

178 **一本通在线评测（ybt）** 平台

179 **梦熊 OJ（Mengxiong Online Judge）** 平台

180 **洛谷** 平台

181 **洛谷主题题库** 平台

182 **洛谷公开赛与月赛** 平台

183 **洛谷题单** 平台

184 **编程猫 OJ（北大—点猫人工智能教育联合实验室）** 平台

185 **蓝桥云课** 平台

186 **计蒜客** 平台

187 **51Nod** 平台

189 **Baekjoon Online Judge** 平台

190 **CSES Problem Set** 平台

191 **CodeChef** 平台

192 **Codeforces** 平台

193 **Codeforces 题目集（按 rating 筛选）** 平台

194 **Comet OJ** 平台

195 **DMOJ** 平台

196 **HDU Online Judge** 平台

197 **Lutece** 平台

198 POJ 平台

199 SPOJ 平台

200 UVa Online Judge 平台

201 ZOJ 平台

202 e-olymp 平台

203 牛客网 平台

204 Codeforces Gym 平台

206 Kattis 平台

207 Timus 平台

208 LibreOJ 平台

209 Nowcoder 多校训练营 平台

211 QOJ 平台

212 UOJ 平台

214 Universal Cup 平台

215 oj.uz 平台

274 CSP-J/S 2025 题目及数据（入门组 4 题 + 提高组 4 题） 真题

275 CSP-J/S 2026 第二轮认证测试数据（入门组 + 提高组） 真题

276 CSP-J/S 历年认证题 真题

277 CSP-J/S 第一轮真题 真题

278 JOI 2026/2027 一次预选（第 26 回日本信息学奥赛第一轮）题目·解说·解答例 真题

279 USACO 历年试题 真题

280 《CCF 信息学奥赛入门篇》练习指导 题解

281 信息学奥赛 3：历年真题解析与拉分解题思维训练 题解

282 信息学奥赛 CSP 通关之路：CSP-J/S 第一轮原创全真模拟试卷集（2026） 真题

283 信息学奥赛 CSP-J 初赛通关手册：10 年真题 + 10 套模拟精练精讲 真题

284 信息学奥赛一本通 题解

285 洛谷主题题单 真题

286 算法竞赛入门经典——习题与解答 题解

287 算法竞赛宝典 题解

289 BalkOI 2026 试题与在线镜像（巴尔干信息学奥林匹克，地拉那） 真题

290 CCF CSP 例题解析与实战技巧：算法应用 题解

291 Programming Challenges 题解

292 eJOI 2026 试题·题解·数据（欧洲初中生信息学奥林匹克，立陶宛考纳斯） 真题

293 信息学奥赛一本通 · 高手训练 题解

300 算法竞赛入门经典——训练指南 题解

301 JOI 2025/2026 二次预选（第 25 回日本信息学奥赛第二轮）题目·解说·数据 真题

302 NOIP 2025 题目及数据（糖果店 / 清仓甩卖 / 树的价值 / 序列询问） 真题

303 NOIP 历年真题（2011 起） 真题

304 NOIP 历年试题 真题

313 ACM 国际大学生程序设计竞赛系列 · 题目与解读 题解

314 AHOI 安徽省选 真题

315 APIO 2026 中国区活动题目及数据 真题

316 BJOI 北京队选拔 真题

317 BOI 试题库与 2026 年题目（波罗的海信息学奥林匹克，1995—2026 全量题面、数据与题解） 真题

318 CEOI 2026 试题与数据（第 33 届中欧信息学奥林匹克，斯洛文尼亚卢布尔雅那） 真题

320 FJOI 福建省选 真题

321 GDOI 广东省选 真题

322 HAOI 河南省选 真题

323 HEOI 河北省选 真题

324 HNOI 湖南省选 真题

325 JSOI 江苏省选 真题

327 SCOI 四川省选 真题

328 SDOI 山东省选 真题

329 SHOI 上海队选拔 真题

330 WEOI 2026 试题与题解（第 4 届西欧信息学奥林匹克，卢森堡） 真题

331 ZJOI 浙江省选 真题

332 十二省联考 真题

335 国家集训队论文配套题 真题

341 联合省选（CCF 统一命题） 真题

345 APIO 历年试题 真题

346 EGOI 2026 试题与逐题统计（第 6 届欧洲女子信息学奥林匹克） 真题

347 IOI 2025 官方赛题包（ioi/task-archive 2025，6 题全量数据） 真题

348 IOI 2026 中国国家集训队集中培训与精英培训题目及数据 真题

349 IOI 2026 试题与题解（第 38 届国际信息学奥林匹克，塔什干） 真题

351 NOI 2026 题目及数据 真题

352 NOI 冬令营 2026 题目及数据 真题

353 NOI 历年真题 真题

354 NOI 历年试题 真题

356 IOI 历年试题 真题

360 USACO Training Gateway 课程

362 代码随想录 课程

363 洛谷网校 课程

369 Codeforces EDU 课程

370 Codeforces EDU（算法教程） 课程

372 UCF COP 4516 Contest Problem Solving（中佛罗里达大学竞赛解题课） 课程

377 AtCoder Library Practice 课程

388 CMU 15-295 Competition Programming and Problem Solving 课程

401 LOJ — LeLe Online Judge（开源可自部署的 ACM/OI 双赛制评测系统） 工具

431 许淇文 IOI 2026 参赛总结 社区

方法论 68 条

010 Think Like a Programmer 教材

012 信奥逐梦之路：家长陪伴与升学规划全阶段指南 专著

027 用 AI 决胜信息学奥赛：C++ 编程基础 教材

034 算法竞赛黄金法则：提高算法和思考力的77项技巧 教材

044 CCF 信息学奥赛提高篇 教材

056 算法导论（核心篇）（原书第 4 版） 专著

057 算法思维训练：信奥大师的算法教与学秘籍 专著

059 算法竞赛核心教程：策略与算法深度解析 教材

065 计算思维训练——动态规划 教材

072 Programming Pearls 专著

074 编程珠玑 专著

090 算法设计手册 教材

098 编程原本 专著

105 NOI 大纲（2025 年修订版） 官方

106 NOI 官网 官方

108 《中国计算机学会通讯》（CCCF） 官方

109 《信息学奥林匹克竞赛年鉴》 官方

110 2024 年 NOI 活动题目分析报告（CSP-J/S 2024 第二轮 + NOIP 2024 知识构成） 官方

115 CCF 信息学奥赛入门篇 官方

130 2025 年 CSP-J/S 第二轮认证题目分析报告 官方

131 CCF 全国青少年信息学奥林匹克竞赛教程 官方

133 NOIP 2026 参赛名额分配方案 官方

134 2023 全国信息学奥林匹克年鉴（电子版） 官方

138 NOIP 题目综合分析报告 官方

139 第六届女生竞赛报名通知（NOIP 2026 女生赛） 官方

140 2022—2023 年国内外信息学主要赛事分析报告 官方

141 APIO 2026 中国区活动赛事综述与分数线 官方

142 CCF 关于 NOI 2027 省内选拔的若干规定 官方

150 CCF NOI 集训营 2026 综合素质评价机制（附国家集训队行为规范） 官方

152 IOI 2027 中国国家队选拔规则与国家集训队行为规范 官方

154 NOI 冬令营 2026 首次 AI 辅助编程测试（人机协同评测） 官方

156 NOI 试题分析与年鉴 官方

158 USACO 2025-2026 赛季赛制改革：三场线上赛 + 监考制 US Open 官方

159 关于自 NOI 2029 起取消 A 类选手加分政策的公告 官方

163 IOI 年度报告与命题趋势 官方

216 信息学竞赛中的思维方法 论文

217 测试数据设计中的三要素 论文

218 ISSEP 2026 论文集：AI 时代的中小学信息学教育 论文

219 OI Wiki 学习路线（C++ 基础 → CSP-J 入门级知识清单） 讲义

222 搜索顺序的选择 论文

224 骗分导论 讲义

227 浅析非完美算法在信息学竞赛中的应用 论文

231 OI 论文集精读 论文

239 图论建模专题 讲义

241 把握本质，灵活运用——动态规划的深入探讨 论文

242 数学模型的建立、比较和应用 论文

245 离线处理思想 论文

262 提交答案题与交互题 讲义

265 最小割模型在信息学竞赛中的应用 论文

266 构造题策略 讲义

272 论随机化算法的原理与设计 论文

281 信息学奥赛 3：历年真题解析与拉分解题思维训练 题解

299 算法思维：竞赛真题精选精讲（第 2 版） 题解

301 JOI 2025/2026 二次预选（第 25 回日本信息学奥赛第二轮）题目・解说・数据 真题

359 USACO Guide 课程

365 2026 年第四期 NOI 教师培训（南京站，中级/初级） 课程

372 UCF COP 4516 Contest Problem Solving（中佛罗里达大学竞赛解题课） 课程

379 NOI 2025 冬令营授课与交流活动现场视频（14 场） 课程

380 算法设计与分析（MIT 6.006） 课程

381 APIO 2025 讲者授课现场视频（5 场） 课程

382 APIO 2026 中国区活动讲座现场视频（7 场） 课程

385 CCF NOI 冬令营 2026 第一课堂（CCF 数字图书馆） 课程

386 CCF NOI 集训营 2026 报到通知（附日程） 课程

387 CCF 关于举办 IOI 2027 精英培训的通知 课程

388 CMU 15-295 Competition Programming and Problem Solving 课程

390 NOI 冬令营 2026 讲座与交流活动现场视频（14 场） 课程

410 对拍脚本 工具

431 许淇文 IOI 2026 参赛总结 社区

IV

PART IV · 第四编

使用手册

THE HANDBOOK

把目录变成行动。三章分别给出分阶段清单、资料获取与版权说明、常见问题三十问。

第 16 章　分阶段执行清单

16.1 怎么用这份清单

下面四张清单对应第 3 章的四阶段。每张清单分"必做"与"选做"两栏，必做项完成度低于 80% 时不要进入下一阶段。清单里的每一项都能在第二编找到对应编号的资料。

16.2 阶段一 · 打底清单

序	任务	验收标准
1	通读一本入门教材（第 5 章任选其一）	课后习题完成率 ≥ 80%
2	完成基础语法与 STL 的专项练习	能默写 vector / map / set / priority_queue 常用操作
3	刷完入门题库前 300 题	一次 AC 率 ≥ 70%
4	学完排序、二分、前缀和、差分、贪心	能各写 5 道不看题解的题
5	学完递归与搜索（DFS / BFS / 回溯 / 剪枝）	能独立写出八皇后、迷宫最短路
6	做 5 套 CSP-J 真题（限时）	稳定 250 分以上
7	建立错题本	每条含"错因 + 正解 + 一句话总结"

16.3 阶段二 · 建模清单

序	任务	验收标准
1	完成 DP 全专题（背包 / 区间 / 树形 / 状压 / 数位）	每个子专题 15—25 题
2	完成图论基础（最短路 / 生成树 / 拓扑 / 连通性）	能默写 Dijkstra 堆优化、Kruskal、Tarjan
3	完成数据结构基础（并查集 / 堆 / 树状数组 / 线段树）	线段树能默写带懒标记版本
4	完成字符串基础（哈希 / KMP / Trie）	能解释哈希冲突与双模的必要性
5	做 10 套 CSP-S 真题（限时）	稳定 200 分以上
6	打 20 场线上赛并全部补题	补题率 ≥ 90%
7	积累 100 句"关键转化"总结	能按知识点检索

16.4 阶段三 · 攻坚清单

序	任务	验收标准
1	完成进阶数据结构（平衡树 / 可持久化 / 树链剖分 / 虚树）	每个能独立写出模板并通过 3 道题
2	完成网络流（最大流 / 最小割 / 费用流 / 上下界）	能独立建模 15 道非模板题
3	完成字符串进阶（AC 自动机 / SA / SAM / PAM）	SA 能默写并求 height
4	完成数论与组合（反演 / 筛法 / 生成函数 / 多项式）	NTT 模板能默写并过随机对拍
5	读 20 篇集训队论文并做笔记	每篇能讲清核心思想
6	做 20 套省选真题（本省 + 强省）	每套能做出 1.5 题以上
7	完成一次全知识点自测	"能讲清"比例 ≥ 60%

16.5 阶段四 · 冲刺清单

序	任务	验收标准
1	全真模拟 10 次以上	成绩方差收敛，无低级失误
2	默写全部模板（20 个以上）	每个模板 15 分钟内写完且能过样例
3	错题回炉（近半年）	每道题能说出当时的错因
4	练开题策略	形成固定的开题顺序与放弃阈值
5	熟悉考场环境（NOI Linux）	能独立完成编译、调试、文件读写
6	准备考场模板本	打印装订，考前一周不再改动

第 17 章　资料获取、版本与版权

17.1 获取途径的六种类型

本书每一条资料都标注了获取途径，归为六类：

类型	含义	注意事项
免费	公开网页，无需登录或注册即可访问	注意站点可能变更，链接失效时用标题搜索
免费（开源）	代码或文档托管在 GitHub 等平台	建议 fork 一份，防止仓库被删
纸质书	需购买实体书	经典教材值得买纸质，工具书优先电子版
付费	课程或会员内容	先试听/试用，确认风格匹配再购买
需权限	需要学校或机构账号	多数高校图书馆可访问 ACM / Springer
部分公开	官网提供摘要或样章	用于判断是否值得进一步获取

17.2 版本问题

竞赛资料有三个"版本陷阱"：

一是大纲版本。NOI 大纲会周期性修订（近年为 2021 版、2025 版）。买教材、看讲义时要注意它对应哪一版大纲，2025 版调整过的知识点（如 Manacher 下放、虚树降级）在旧资料里可能被标为"超纲"。

二是编译器与标准版本。NOI 系列赛事的官方环境为 NOI Linux，编译器版本随年份更新。C++ 标准从 C++98 一路推进到 C++14 乃至更高，某些写法（如 `auto`、结构化绑定）在旧环境不可用。备赛时应以当年官方公示的环境为准，不要凭记忆。

三是题面版本。同一道题在洛谷、原 OJ、PDF 题面之间可能存在数据范围或输出格式的细微差异。做真题时应优先使用官方题面，或至少交叉核对。

17.3 版权与使用边界

本书是一份目录，不复制、不转载任何资料的正文内容。所有条目仅包含名称、作者、出版方、获取地址与一段原创评述。评述部分为编者观点，不代表原作者立场。

使用本书时请注意：

- 教材、专著、论文的正文受著作权保护，请通过正规渠道获取；标注"免费"的资源请遵守其原作者声明的许可协议；

- 开源仓库（如 OI Wiki、CP4 代码库）多采用 CC BY-SA 或 MIT 类许可，转载时需保留署名与许可声明；
- 真题题面版权归主办方（CCF、IOI 等）所有，用于个人训练无碍，公开出版或商业使用需获授权；
- 本机收录的本地资料（若随书附带）仅供个人学习，不得再分发。

17.4 链接失效怎么办

互联网资源会消失——这是一个确定的事实，不是意外。本书收录的链接在成书时经过一次批量校验，但仍会有站点迁移、域名过期、仓库删除的情况。遇到失效链接，按以下顺序处理：

1. 用资料标题（而不是 URL）在搜索引擎检索，多数资源有镜像；
2. 到 Internet Archive（web.archive.org）查询历史快照；
3. 开源项目到 GitHub 搜索同名仓库，注意看 fork 数最多的那一个；
4. 教材类资源查作者个人主页，很多作者会长期维护 PDF；
5. 确实找不到时，用本书给出的同类替代条目（同一章节内通常有 2—3 个功能相近的资源）。

这也是本书坚持"按功能分类"而不是"按链接罗列"的原因：资料的载体可能消失，但它承担的功能不会。

第 18 章　常见问题三十问

关于入门

1. 零基础可以直接学 C++ 吗？可以。竞赛用的 C++ 是子集，不需要学完整个语言。先掌握输入输出、循环、数组、函数，就可以开始做题。

2. 要不要先学 Python？不建议。竞赛环境以 C++ 为主，Python 在时限上吃亏。若已会 Python，转 C++ 大约需要两周。

3. 第一本书选哪本？看第 5 章标注为"CSP-J / 入门自修"的条目，任选一本通读，不要同时开两本。

4. 一天该刷几题？入门期 3—5 题（含思考时间），专题期 2—3 题（要写总结），冲刺期以套卷为主。

5. 看不懂题解怎么办？降一个难度。连续三道题都看不懂题解，说明当前难度区间选错了，不是你的问题。

关于训练

6. 刷题是求量还是求质？前期求量（建立手感），中期求质（建立映射），后期求稳（减少失误）。

7. 需要背模板吗？常用模板必须能默写：线段树、树剖、并查集、最短路、网络流、KMP、NTT。考场现推风险极高。

8. 对拍真的有用吗？有用，而且是被严重低估的一项。详见第 4.5 节。

9. 一定要打 Codeforces 吗？强烈建议。它的 rating 体系是目前最可靠的第三方水平标尺，且题解质量高。

10. 中文题和英文题哪个优先？前期中文（降低语言负担），后期必须练英文（NOI/IOI 题面、论文都是英文或英文术语）。

11. 纸质书和电子版怎么选？教材买纸质（便于反复翻），工具书用电子版（便于检索）。

12. 视频课看多少合适？视频只用来"破冰"，一个知识点看一节就够，剩下时间必须动手。

关于比赛

13. 第一轮（初赛）怎么准备？近五年真题各做两遍，错题按知识点归类。阅读程序题是最大失分点，要专项练。

14. 提高组一等奖大概什么水平？能稳定做出 NOIP 前两题，第三题拿部分分。折算下来约 Codeforces 1600—1800。

15. 省队门槛大概什么水平？强省约 Codeforces 2200+，且知识面无明显缺口。

16. 比赛时该先做哪题？先通读四题（10 分钟），按"有把握 → 有思路 → 有部分分 → 无思路"的顺序做。

17. 什么时候该放弃一道题？投入 40 分钟无实质进展，或已写出但调不出且剩余时间不足 60 分钟时，转去做部分分。

18. 部分分怎么拿？先看数据范围，把 $n \leqslant 20$ 的暴力、$n \leqslant 1000$ 的 $O(n^2)$、特殊性质（链、树、无修改）分别写掉，通常能拿 40—60 分。

19. 考场上要不要对拍？会做的题写完就拍，尤其是数据结构题。前提是平时练过。

20. 挂了一场很重要的比赛怎么办？复盘，找出可归因的具体原因（不是"我太菜"），然后继续。竞赛是多次博弈，单场结果的信息量很低。

关于资源与方法

21. 资料太多学不完怎么办？本书的存在就是为了解决这个。选一条路线，走完再看下一条。

22. 集训队论文什么时候开始读？省选阶段。太早读不懂，太晚来不及。

23. 论文读不懂怎么办？先读同主题的 OI Wiki 词条打底，再读论文。顺序反了会很痛苦。

24. 需不需要学 Linux？需要。NOI 系列赛事使用 NOI Linux，至少要会编译、运行、文件读写、简单调试。

25. 要不要学算法竞赛以外的编程？有余力可以，但优先级低于竞赛本身。工程能力在 NOI 之后补更高效。

26. 数学要学到什么程度？组合计数、初等数论、概率期望是必需；线性代数在 NOI 级必需；微积分基本不需要。

27. 英语差怎么办？竞赛英语是"术语英语"，把 200 个高频术语背下来，读题面就没有障碍了。

28. 家长能帮上什么？提供稳定的时间与环境，帮忙筛选资料（本书可代劳），不要干预训练方法。

29. 教练最该看哪几章？第 2 章（大纲）、第 8 章（论文与讲义）、第 13—15 章（三套索引）、第 16 章（清单）。

30. 这本书会更新吗？会。资料在变，大纲在变，本书按赛季更新。请以最新版本的编号为准。

V

PART V · 第五编

考场实务

EXAM DAY

第一次进考场的人，丢分往往不在算法上。本编三章讲三件事：从报名到交卷的全流程、NOI Linux 2.0 环境调试、三十七个让分数归零的操作。

第 19 章　第一次上考场：从报名到交卷

19.1 赛前一个月：把「非技术问题」全部解决掉

第一次进考场的人，丢分往往不在算法上。身份证过期、准考证没打印、走错校区、把代码存到桌面、输出文件写成 `.ans`——这些事每年都在发生，而且一旦发生就没有补救机会。所以本章按时间倒排，把能提前做的事全部提前。

时间点	要做的事	为什么必须现在做
赛前 30 天	确认报名状态与考点；核对身份证有效期；在官网下载并安装 NOI Linux 2.0 虚拟机	报名由学校统一上报，个人看不到状态；身份证过期补办需要 15 个工作日
赛前 14 天	打印准考证（报名系统下载，黑白即可）；逐字核对姓名、组别、考点地址	考点可能有两个校区，地址差一个字就是两小时车程
赛前 7 天	在虚拟机里完整做一套真题：建目录、写 freopen、编译、用样例验证、打包提交	把流程跑通一遍，考场上就不用现学
赛前 1 天	查考点路线与入场时间；准备透明笔袋；早睡	迟到 15 分钟通常不得入场
当天	提前 45—60 分钟到考点；两证放手边	入场核验、找座位、开机、登录都要时间

19.2 证件：只有两种组合被认可

按 CCF 各省通知的通行规定，进入考场必须持两证并放在桌面上备查，缺一不予参赛：

- 组合一：本人身份证（或临时身份证、护照、有本人照片的社保卡）原件 + 准考证（报名系统下载打印）；
- 组合二：户口本 + 学校盖章的学籍证明（含姓名、性别、身份证号/护照号、年级、照片，照片处需骑缝章），一式两份，一份交省组织单位留存，一份作为入场证件。

> 临时身份证、护照、带照片社保卡在多数省份被认可，但具体以本省通知为准。每年 9—10 月各省都会在 noi.cn「各省新闻」发布本省认证须知，那份文件才是你所在考点的最终规则。

19.3 物品：能带什么，不能带什么

允许带入并放在桌面	必须放在考场外
准考证、身份证件 笔、橡皮等非电子文具 无存储功能的手表 透明笔袋（部分考点要求）	手机（即使关机也不行，须存放场外） U 盘、移动硬盘、键盘、鼠标 计算器、闹钟、电子手表 书籍、草稿纸、背包

草稿纸由考场统一发放，用完可举手再要。自己带的纸一律不许留在桌上——这条最容易被忽略，因为很多人习惯带一沓纸演算。

19.4 入场后到开考前：十分钟决定这场比赛

1. 核对座位：屏幕或桌贴上的准考证号与你的准考证是否一致。不一致立刻举手，不要自己换机器。
2. 核对目录：部分赛区由监考预先在 FTP 或本地建好以准考证号命名的文件夹，坐下后先看名字对不对；不对就举手，不要自己另建一个。
3. 试编译：写一个空的 `main`，编译、运行、看是否报错。这一步能提前发现环境问题，而不是在写完 200 行后才发现编译器没配好。
4. 试文件读写：用 `freopen` 读一个自己造的 `a.in`、写一个 `a.out`，确认路径正确、权限正常。
5. 通读四题：不要急着动手。用 8—10 分钟把四道题读完，标出「有把握 / 有思路 / 只会暴力 / 完全没想法」，再决定顺序。

19.5 五个小时怎么分配

以 4 题、4.5 小时（NOIP 2026 的实际时长）为例，一个经过验证的分配方案：

时段	任务	要点
0:00—0:10	通读 + 建目录 + 试环境	四题都读一遍，建好四个题目文件夹与空源文件
0:10—1:00	第一题（最有把握）	写完立刻用样例验证，然后保存。先拿到手的分
1:00—2:00	第二题	先写暴力保底并提交，再想正解
2:00—3:30	第三题 / 第四题部分分	按数据范围分层拿分：小数据暴力、特殊性质、朴素 DP
3:30—4:10	回头优化或对拍	对已完成的题造数据对拍，尤其是数据结构题
4:10—4:30	全局检查	见 19.6 清单，逐项打勾

核心原则只有一条：先让每道题都有分，再让某道题拿满分。空着交的题得 0 分，写崩的暴力也能拿 20—40 分。

19.6 交卷前 15 分钟：十四项检查

#	检查项	怎么查
1	目录名 = 准考证号	与准考证并排比对，逐字符读
2	子目录名 = 题目英文名	从题面复制，不要手打
3	源文件名 = 题目英文名 + .cpp	确认不是 .cpp.txt
4	子目录里只有源文件	删掉 .in/.out/可执行文件
5	freopen 的文件名与题面一字不差	对照题面「输入文件」「输出文件」两行
6	提交前重新编译一遍	确认改过的地方能编译通过
7	样例再跑一次	输出与题面完全一致（含空格与换行）
8	数组大小够不够	按最大数据范围 ×1.2 开
9	long long 有没有漏	所有累加、乘积、答案变量
10	多组数据是否清空	数组、vector、计数器
11	输出格式	行末空格、换行、浮点精度
12	文件是否已关闭	程序正常 return，不要 exit(0) 前漏 flush
13	四道题都提交到 FTP/指定目录	复制粘贴，不要拖动
14	提交后不要改代码	改完忘了重新提交 = 白改

19.7 突发状况处理

状况	正确处理
机器死机、蓝屏	立即举手，不要自行重启。监考会记录并补时；自行重启可能丢失文件
文件被误删	举手说明，不要试图「重新写一遍」。部分赛区可从 FTP 或快照恢复
编译器报奇怪的错	先用终端 `g++ -O2 -std=c++14 a.cpp -o a` 手工编译，看原始错误信息
时间只剩 20 分钟还有两题没写	每题写 10 行暴力，交上去。两题各拿 20 分，胜过一题冲满分
身体不适	举手示意，考场配有医务人员。不要硬撑到交卷
发现交错了目录	立刻举手，在监考确认下重新提交。越早越好

19.8 第一轮（资格测试）的额外注意

第一轮与第二轮是两种完全不同的考试。第一轮考的是阅读程序与完善程序，很多人在这里翻车，原因不是不会，而是不熟悉题型。

- **答题卡：准考证号填涂错一位，整张卡作废。填完当场核对两遍。**
- **阅读程序题：给一段代码问输出。不要真去跑，按行模拟，把变量值写在草稿纸上。**
- **完善程序题：给一段挖空的代码和五个选项。先看空的位置在循环里还是递归里，再判断该填边界还是递推式。**
- **时间：通常 90—120 分钟，选择题控制在 40 分钟内，剩下时间全给阅读与完善程序。**

> **各省第一轮形式可能不同（纸笔或机考），以本省通知为准。无论哪种形式，考查内容一致：计算机基础、算法常识、代码阅读。**

第 20 章　环境调试：NOI Linux 2.0 全解

20.1 考场里到底是什么系统

自 2021 年 9 月 1 日起，NOI 系列比赛与 CSP-J/S 的标准环境是 NOI Linux 2.0，基于 Ubuntu 20.04.1 定制。所有选手的源程序最终都在这个环境下统一评测——这一点不会因为你在考场用的是 Windows 而改变。

类别	组件	版本	说明
系统	内核	5.4.0-42-generic 64 位	Ubuntu 20.04.1 定制
	C 编译器	GCC 9.3.0	—
	C++ 编译器	G++ 9.3.0	默认 C++14，同时支持 C++17
其他语言	Free Pascal / Python	FPC 3.0.4 / Python 2.7、3.8	Python 标注为「非竞赛语言」
调试	GDB	9.1	命令行调试器
	DDD	3.3.12	GDB 图形前端
	Geany / VS Code / Sublime	1.36 / 1.54.3 / 3.2.2	轻量编辑器
IDE	Code::Blocks	20.03	考场主力 C/C++ 环境
	Lazarus	2.0.6	Pascal 环境
评测	Arbiter-local	1.02	单机版评测工具，可本地测样例

> 数据来源为 NOI 官网 2021 年 7 月 16 日发布的系统情况简表。若后续版本更新，以 noi.cn「技术规则」栏目为准。

20.2 编译选项：三个参数决定成败

官方评测使用的编译命令等价于：

```
g++ -O2 -std=c++14 -static a.cpp -o a
```

- `-O2`：开启二级优化。这意味着未定义行为可能被优化成任何结果——数组越界、有符号整数溢出在本地看似正常，开 O2 后可能直接算错。

- `-std=c++14`：语言标准。不要用 C++17 的结构化绑定、`std::optional`、`if constexpr`，考场编译不过。
- `-static`：静态链接。这会让可执行文件变大，但避免了运行库版本不一致。

平时训练就应按这三个参数编译。Dev-C++ 默认不带这些参数，所以「本地 AC、评测 RE」的经典现象，多数是优化差异或标准差异造成的。

20.3 Windows 与 Linux 的八个差异

差异点	后果	规避方式
大小写敏感	`Expr.cpp` 与 `expr.cpp` 是两个文件	全部小写，从题面复制
隐藏扩展名	存成 `a.cpp.txt`，评测找不到文件	Windows 里勾选「显示文件扩展名」
换行符	CRLF 与 LF 混用导致读入错位	用 `scanf` / `cin` 按空白读，不要按行硬读
栈空间	Windows 默认 1 MB，Linux 通常 8 MB	深递归在本地爆栈、评测却过；反之亦然。按最坏情况估算
路径分隔符	`\` 在 Linux 无效	用相对路径，文件放在源文件同目录
中文路径	部分工具链对 UTF-8 路径处理异常	目录名只用字母、数字、减号
long long 输出	Windows 下 `%lld` 曾有兼容问题	统一用 `cin/cout` 或 `%lld`，不要混用 `%d`
rand() 范围	两平台 RAND_MAX 都是 32767，但随机序列不同	需要大数据时自己写生成器，不要依赖 rand 的分布

20.4 在家搭一套「考场同款」

1. 下载镜像：从 NOI 官网「技术规则」页获取 NOI Linux 2.0 镜像（约 3.4 GB）。装系统时断开网络，避免安装过程卡在联网更新。
2. 建虚拟机：VirtualBox 或 VMware，类型选 Linux / Ubuntu 64 位；内存至少 2048 MB，磁盘 20 GB 以上。
3. 安装：光驱挂载 ISO，一路默认。用户名密码记牢（考场机器常见为 noilinux / 123456，以监考通知为准）。
4. 验收五步：① 打开终端 `g++ --version` 看是否 9.3.0；② 写 Hello World 编译运行；③ 用 `freopen` 读写文件；④ 打开 Code::Blocks 建一个 C++ 工程并编译；⑤ 用 `gdb` 跑一次断点。
5. 日常使用：每周至少一次在这个环境里做完整套题，形成肌肉记忆。

20.5 Code::Blocks 三步上手

1. **首次启动**：弹出编译器自动检测窗口时，选状态为 Detected 的 GNU GCC Compiler，点 Set as default。
2. **新建源文件**：File → New → From template → C/C++ source；语言务必选 C++（默认是 C）。
3. **先保存再写**：定位到「准考证号/题目英文名/」目录，用题目要求的文件名保存，之后才有语法高亮。

不习惯 IDE 的话，终端三条命令等价：

`g++ -O2 -std=c++14 -static expr.cpp -o expr` → `./expr` → `diff expr.out expr.ans`

20.6 最小调试技能

手段	用法	适用场景
输出中间变量	`cerr << x << endl;`	最快，且不会污染标准输出文件
assert 断言	`assert(l <= r);`	检查前置条件，出错即中止并报行号
GDB 断点	`gdb ./a` → `b 42` → `r` → `p x` → `n`	逻辑错误、死循环、数组越界定位
对拍	gen + 暴力 + 正解 + diff，循环 1000 次	正确性不确定时的最终手段
编译告警	`-Wall -Wextra`	未初始化变量、隐式类型转换

20.7 故障排查十二例

#	现象	原因与处理
1	`No such file or directory`	文件名或路径错，含大小写、中文、空格
2	输出文件为空	freopen 写成 `r` 模式；或程序异常退出未 flush
3	本地过、评测 WA	未定义行为被 O2 优化；数组越界；long long 溢出
4	本地过、评测 RE	递归过深爆栈；除零；野指针；`vector` 越界访问
5	编译报 `stray '\357'`	代码里有中文全角字符（常见于全角分号、引号）
6	`undefined reference`	函数声明了没定义；或使用了 C++17 才有的库函数
7	样例对、提交 0 分	提交错目录；文件没保存；只交了 .in 没交 .cpp
8	程序卡住不返回	死循环；`cin` 未关同步但用了 `freopen`
9	浮点答案差一点	用 `double` 而非 `long double`；比较未用 eps
10	大数据 TLE、小数据正常	复杂度差一个 log；I/O 未加速；STL 常数过大
11	多测第二组开始错	全局数组/容器未清空；`memset` 漏了某个数组
12	编译很慢或内存超限	开了过大的全局数组（如 `int a[100000000]`）

第 21 章　爆零清单：三十七个让分数归零的操作

21.1 先把结论说清楚

OI 赛制按测试点给分，理论上「写多少得多少」。但每年仍有大量选手得 0 分，原因几乎全部是操作性的。下表按后果分三级：致命（整题或整场 0 分）、重伤（丢掉大部分分）、轻伤（扣几分到几十分）。

级别	#	操作	后果与规避
致命	1	工作目录名与准考证号不一致	整场 0 分。逐字符核对，注意半角减号、组别字母、5 位数字
	2	用了中文全角字符（—、＿、－）	目录或文件名不匹配，0 分。输入法切回英文半角
	3	目录名大小写错误	Linux 区分大小写，`Expr` ≠ `expr`
	4	源文件扩展名被隐藏成 `.cpp.txt`	评测找不到源文件。Windows 里打开「显示扩展名」
	5	freopen 的文件名与题面不一致	读不到输入，输出为空，0 分
	6	用了 `freopen` 却写成 `.ans` 输出	题面要求 `.out` 就是 `.out`
	7	提交到别人的目录 / 提交到桌面	成绩无效。提交后到指定目录确认文件在
	8	使用了他人准考证号	按作弊处理
	9	考场内使用手机、U 盘等电子设备	按违纪处理，成绩取消
重伤	10	数组开小	RE 或 WA，通常丢掉该题全部分
	11	long long 溢出	大数据全 WA，只剩小数据分
	12	递归过深爆栈	RE。深搜/线段树递归要估算层数
	13	多测未清空	第二组起全错
	14	只写正解没写暴力	正解写崩时全盘皆失
	15	没有对拍	隐蔽 bug 到交卷都没发现
	16	输出格式错（多空格、少换行）	整题 WA
	17	文件未正常关闭	缓冲区未写出，输出文件为空
	18	用了 C++17 语法	提高组环境为 C++14，编译失败
	19	读题漏掉「多组数据」	只处理一组，其余全错
	20	时间复杂度估错一个 log	大数据 TLE，通常损失 40—60 分
	21	最后十分钟改代码没重测	把对的改错，这是最冤的一种
轻伤	22	行末多余空格	部分评测机严格比对，可能 WA
	23	浮点未用 eps 比较	个别点 WA
	24	`cin` 未加速	大数据 TLE

级别	#	操作	后果与规避
	25	STL 容器频繁扩容	常数过大，TLE
	26	递归函数传值传大对象	常数爆炸
	27	未初始化变量	随机 WA，最难查
	28	边界条件写反（< 与 <=）	个别点错
	29	特殊性质数据未处理	丢掉送分点
	30	模块化不足，改一处崩三处	调试时间翻倍
	31	注释掉调试输出后忘了恢复 freopen	可能输出到错误文件
	32	提交后继续改代码但没重新提交	白改
	33	没看数据范围就写算法	方向性错误，浪费一小时
	34	把样例答案硬编码	只得样例分，且可能被判作弊
	35	忘记删掉本地的调试文件	子目录里混入 .in/.out，个别评测机报错
	36	时间分配失衡，一题卡三小时	其余三题没时间
	37	考前没在 NOI Linux 里跑过	环境不熟，操作失误概率翻倍

21.2 每题固定模板

把这段骨架背下来，每题开头先贴一遍，可以消掉上面一半的问题：

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    freopen("prob.in", "r", stdin);
    freopen("prob.out", "w", stdout);
    ios::sync_with_stdio(false); cin.tie(nullptr);
    // 你的代码
    return 0;
}
```

> **文件名 `prob` 必须替换为题目给定的英文名，且与子目录名一致。三个地方（子目录、源文件、freopen）必须完全同名，这是最容易被忽略的一致性要求。**

21.3 三条元规则

1. **先有分，再要分。每道题在 20 分钟内必须有一版能跑的程序存在于正确位置。**
2. **改完就测，测完再交。任何修改之后，重新编译、重新跑样例、重新提交。**
3. **不确定就对拍。样例只能证明程序在一种情况下对，对拍能证明它在几百种情况下对。**

VI

PART VI · 第六编

知识正编

THE KNOWLEDGE

前五编解决「读什么、去哪找、怎么考」，本编解决「知识本身」。十章的正文按「语言—复杂度—基础—搜索—DP—数据结构—图论—数学—字符串—几何」排列，每节给出结论、复杂度、可编译模板与易错点，用于直接查阅与赛前复盘。

22 第 22 章　语言与代码规范

> 算法之外，决定分数的是代码本身。本章把竞赛 C++ 收敛成一套够用的子集，讲清类型、内存、STL 与调试习惯——这些内容不产生思路，但能避免 30% 的无谓失分。

22.1 一套够用的 C++ 子集

竞赛代码不需要掌握 C++ 的全部。把下面这套子集用熟，覆盖 99% 的题目：`int / long long / double / char / bool` 五种基本类型，`vector / string / pair / map / set / priority_queue / queue / stack` 八个容器，`sort / lower_bound / unique / reverse / max_element` 五个算法，加上结构体、引用、函数重载。剩下的（类继承、模板元编程、异常）在考场上只会增加出错面。

头文件：早期常用 `<bits/stdc++.h>`，NOI Linux 2.0 的 GCC 9.3 支持，但部分省选/校赛的评测机用 MSVC 或不完整 MinGW，会编译失败。稳妥写法是显式包含：

```
#include <cstdio>
#include <cstring>
#include <cmath>
#include <algorithm>
#include <vector>
#include <queue>
#include <stack>
#include <map>
#include <set>
#include <string>
#include <iostream>
using namespace std;
typedef long long ll;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fLL;
```

快读：数据量到 10^6 以上时 `cin` 即使关了同步也可能不够快。手写快读是投入产出比最高的一段代码，背下来：

```
inline int read() {
    int x = 0, f = 1; char c = getchar();
    while (c < '0' || c > '9') { if (c == '-') f = -1; c = getchar(); }
    while (c >= '0' && c <= '9') { x = x * 10 + c - '0'; c = getchar(); }
    return x * f;
}
inline ll readll() {
    ll x = 0; int f = 1; char c = getchar();
    while (c < '0' || c > '9') { if (c == '-') f = -1; c = getchar(); }
    while (c >= '0' && c <= '9') { x = x * 10 + c - '0'; c = getchar(); }
    return x * f;
}
```

注意 `getchar` 读到 EOF 会返回 `-1`，多组数据时循环条件要写成 `while ((c = getchar()) != EOF)`，否则死循环。想同时保留 `cin` 的便利，就写 `ios::sync_with_stdio(false); cin.tie(0);`，之后不能再用 `scanf` / `getchar`，两者缓冲区不同步会读错。

22.2 类型、溢出与精度

先算范围再选类型。`int` 上限约 2.1×10^9，两个 10^9 相加就溢出。凡是题目出现「答案对 998244353 取模」或数据到 10^5 且要累加/相乘，一律 `long long`。

类型	范围	典型用途
`int`	-2.1×10^9 ~ 2.1×10^9	下标、点数、边数
`unsigned int`	0 ~ 4.29×10^9	哈希、位运算
`long long`	-9.2×10^{18} ~ 9.2×10^{18}	答案、乘积、前缀和
`double`	15—16 位有效数字	实数运算、期望
`__int128`	约 1.7×10^{38}	乘法中间量、大数比较

三道常见的溢出坎：`(a*b)%MOD` 中 `a,b` 都是 `long long` 且接近 10^9 时乘积到 10^{18}，勉强安全；若到 10^{18} 级别相乘，必须 `__int128` 或龟速乘。`int mid = (l + r) / 2` 在 `l,r` 接近 `INT_MAX` 时溢出，写 `l + (r - l) / 2`。

浮点比较不要写等号。用 `fabs(a-b) < 1e-9`；比较大小要留容差 `a > b + 1e-9`。二分实数时迭代固定次数（如 100 次）比判 `r-l < eps` 更稳，因为 `eps` 太小会死循环、太大精度不够。

输出格式：`printf("%.10f", x)` 四舍五入由实现决定，误差在末位；判题机一般允许 1e-6 相对误差，但输出 `-0.000000` 会被某些 SPJ 判错，加一句 `if (fabs(x) < 1e-12) x = 0;` 更保险。

22.3 数组、vector 与内存

开数组的原则是「按最大数据规模开，多开 5—10 个下标余量」。`int a[100005]` 占 400KB，`long long` 翻倍。全局数组在 BSS 段，默认清零；函数内的大数组在栈上，会爆栈——这是新手最常见的 RE。经验值：单个函数内局部变量总和不要超过 1MB（约 `int a[250000]`）。

```
const int MAXN = 100005;
int a[MAXN];              // 全局：自动清零，安全
// int b[1000005];        // 若必须放局部，加 static 或提到全局
void solve() {
    static int tmp[MAXN];  // static 也放静态区
    vector<int> v(n);      // 堆上分配，n 可达 1e7
}
```

vector 的扩容是 2 倍增长，`push_back` 均摊 O(1)，但扩容瞬间会拷贝全部元素。已知规模时先 `reserve(n)`；需要「一次性填入」时用 `v.assign(n, 0)` 或 `v.resize(n)`。

清空多个测试点：`memset(a, 0, sizeof a)` 只对全局数组方便；`vector` 用 `fill(v.begin(), v.end(), 0)`；结构体数组要写 `memset` 且保证没有 `std::string`、`vector` 成员（会破坏内部指针）。多测不清空是仅次于 freopens 的爆零原因。

二维数组的替代：`int g[1005][1005]` 在 n=1000 时占 4MB，可以；n=5000 时占 100MB，多数题会 MLE。此时用「一维数组 + 手动下标」`g[i*m+j]` 或 `vector<vector<int>>` 按需分配。

22.4 引用、传参与结构体

大对象一律传引用。传值会拷贝整个 `vector`，一次深搜传图就可能 TLE。只读加 `const&`，要改就传 `&`。

```
void dfs(int u, const vector<vector<int>>& g, vector<int>& vis) {
    vis[u] = 1;
    for (int v : g[u]) if (!vis[v]) dfs(v, g, vis);
}
```

结构体排序用 `operator<` 或 lambda。注意 `sort` 的比较函数必须是严格弱序：写 `a.x <= b.x` 会在相等元素上崩溃（某些实现在数据有序时越界）。

```
struct Node { int x, y; };
bool operator<(const Node& a, const Node& b) {
    if (a.x != b.x) return a.x < b.x;
    return a.y < b.y;
}
sort(v.begin(), v.end(), [](const Node& a, const Node& b){ return a.x < b.x; });
```

pair 的默认比较是字典序，`priority_queue<pair<int,int>>` 默认大根堆且先比 first。`priority_queue<int, vector<int>, greater<int>>` 才是小根堆——注意 `>` 后面要留空格，否则会被当成右移。

22.5 STL 的正确用法与坑

STL 是双刃剑：用对省半小时，用错常数翻十倍。下面这张表是考场上的经验值。

容器 / 函数	单次复杂度	什么时候别用
`map / set`	O(log n)，常数约 10—20	$n > 2\times10^5$ 且操作密集，改用离散化 + 数组或 unordered_map
`unordered_map`	均摊 O(1)	出题人构造哈希冲突会退化到 O(n)，Codeforces 上常见，加随机盐或换 map
`vector::insert/erase`	O(n)	循环里删除元素，改用「标记 + 最后压缩」或 list
`string` 拼接	O(len)	循环 `s += c` 均摊 O(1)，但 `s = s + c` 是 O(n) 拷贝
`lower_bound`	O(log n)	只能在有序区间用；在 set 上要用 `s.lower_bound(x)` 成员函数，全局版是 O(n)

去重三步：`sort(v.begin(), v.end()); v.erase(unique(v.begin(), v.end()), v.end());`。`unique` 只把重复元素移到末尾并返回新终点，不删除。

迭代器失效：`vector` 扩容后所有迭代器失效；`map` 删除当前元素后 `it++` 未定义，要写 `it = m.erase(it)`。这类错误在本地往往跑得过，交上去 RE。

22.6 调试习惯：把错误挡在提交前

三个习惯能在赛场上省下大量时间。

一、写 `#define DEBUG` 分区。调试输出统一用一个宏包住，交卷前注释掉宏定义即可整体失效：

```
#define DEBUG
#ifdef DEBUG
  #define dbg(x) cerr << #x << " = " << (x) << endl
#else
  #define dbg(x)
#endif
```

二、小数据对拍。写完正解再写一份暴力，用随机数据跑几百组比对输出。对拍脚本在 Linux 下用 `diff`，Windows 下用 `fc`。对拍能抓出 80% 的逻辑错误，剩下 20% 是边界和精度。

三、静态检查清单。提交前逐条过：数组够不够大、多测有没有清空、`long long` 够不够、递归深度会不会爆栈（$>10^5$ 层要改迭代或加栈）、`scanf("%d")` 有没有漏 `&`、输出有没有多空格/少换行。

关于 `-O2`：官方编译开 `-O2`，未定义行为（数组越界、有符号溢出、未初始化变量）在优化后可能表现完全不同——本地不开优化能过、开了就 WA，或者反过来。所以本地调试时也应加 `-O2 -Wall`，把警告当错误看。

23 第 23 章　复杂度与常数优化

> 看到数据范围就该知道用什么算法，也知道自己的程序到底能不能过。本章给出「n 到复杂度」的对照表与常数经验值、把程序压进时限的七种手段、空间与 MLE 的算法，以及两个决定实际得分却最常被跳过的环节：用对拍与造数据把「我觉得对」变成「我知道对」，和按子任务分档交付把部分分拿满。

23.1 复杂度怎么估才准

复杂度估算是选题的第一步，也是最重要的一步。方法固定：看数据范围，反推可接受的复杂度。

n 的范围	可接受复杂度	典型算法
$n \leqslant 20$	$O(2^n)$、$O(n!)$	状压 DP、全排列枚举
$n \leqslant 40$	$O(2^{n/2})$	折半搜索（meet in the middle）
$n \leqslant 100$	$O(n^3)$	Floyd、区间 DP、矩阵乘
$n \leqslant 500$	$O(n^2)$ 或 $O(n^2 \log n)$	朴素 DP、二分图匹配
$n \leqslant 5000$	$O(n^2)$ 勉强	需要压常数
$n \leqslant 10^5$	$O(n \log n)$	排序、线段树、树状数组
$n \leqslant 10^6$	$O(n)$ 或 $O(n \log n)$ 压常数	前缀和、单调栈、线性筛
$n \leqslant 10^7$	$O(n)$	只能扫常数次

经验常数：现代评测机 1 秒约能跑 10^8 次简单运算（加法、比较），或 10^7 次带内存随机访问的运算，或 10^6 次 `map` 操作。线段树常数约 4—8，树状数组约 1—2，`map` 约 15。

只算主导项，但别丢常数。$O(n \log n)$ 的排序在 $n=10^5$ 时约 1.7×10^6 次比较，毫秒级；而同样规模的三层循环套线段树，常数能差 100 倍。做题时先算「最坏情况下内层循环执行多少次」，而不是只写个大 O。

23.2 常数优化的清单

算法对了但超时，按下面的顺序优化，前面的收益远大于后面。

1. 换 I/O。关闭同步 + `cin.tie(0)`，或直接写快读。输入 10^6 个数时，`cin` 关同步约 0.1s，不关约 1s，快读约 0.05s。

2. 减少内存随机访问。二维数组按行访问比按列快一个数量级（缓存行 64 字节）。`for (i) for (j) a[i][j]` 优于 `for (j) for (i)`。

3. 把重复计算提到循环外。循环内的 `strlen(s)`、`v.size()`、`sqrt(n)` 都要提到外面——`strlen` 是 O(n)，放在 `for` 条件里直接把 O(n) 变成 $O(n^2)$。

4. 用位运算代替乘除。`x/2` 写 `x>>1`（仅对非负整数等价），`x%2` 写 `x&1`。编译器通常会自动做，但涉及 `long long` 除法时手动写更稳。

5. 减少函数调用。短小的递归函数改成带栈的迭代；频繁调用的小函数加 `inline`。

6. 数组压维与内存连续。把 `vector<vector<int>>` 换成 `vector<int>` 加手动下标，能让缓存命中率显著上升。

7. 循环展开。仅在第 1—6 条都做完、仍差 10%—20% 时使用，收益不稳定，且容易写错。

什么时候不该卡常：如果复杂度本身就高一个量级（该 O(n log n) 写了 $O(n^2)$），卡常最多救回 2—3 倍，救不回 $n=10^5$ 的平方。此时应该换算法，而不是换写法。

23.3 空间复杂度与 MLE

常见限制是 256MB 或 512MB。换算：`int` 数组 10^7 个占 40MB，`long long` 翻倍，`bool` 数组占 1 字节（不是 1 位）。

三种省内存的手段：

- 滚动数组：DP 只依赖前一维时，把 `f[i][j]` 压成 `f[2][j]` 或原地倒序更新。
- bitset：把 `bool` 数组换成 `bitset<N>`，空间省 8 倍，且整体位运算极快（见 31.5）。
- 按需分配 / 离散化：值域 10^9 但只有 10^5 个不同值时，离散化后开 10^5 的数组。

递归的栈空间：每次递归调用约 几十到几百字节，深度 10^5 的 DFS 很容易爆栈（默认栈 8MB）。两种解法：把递归改迭代；或用 `ulimit -s` 调整（考场不允许，所以只能改写法）。

23.4 对拍、造数据与本地评测：把「我觉得对」变成「我知道对」

结论：对拍（stress test）是投入产出比最高的验证手段——三十行代码加一个循环脚本，能在几分钟内揪出「样例全过、答案全错」的逻辑漏洞。它由四件东西组成：生成器 `gen`、暴力 `brute`、待测程序 `fast`、循环比对脚本。四者缺一不可：只有暴力没有生成器，等于手造样例；只有生成器没有暴力，等于自己跟自己比。

手段	成本	能抓什么	抓不到什么
手造样例	10 分钟	显式边界：n=1、全相同、极值、负数	你没想过的形状；与样例同构的错误
对拍	30 行 + 10 分钟	逻辑错、边界错、多测未清空、性质算错	双方共有的错误假设、TLE/MLE、浮点精度
造数据 + SPJ	1—2 小时	多解题、浮点答案、输出格式	算法正确性本身（SPJ 只判格式与可比性）

它到底有多有效？拿一个真实的小 bug 试：求 `a[i]+a[j]=k` 的 `(i,j)` 对数，把 `i < j` 误写成 `i <= j`（同一个元素被算两次）。用 `n ∈ [1,8]` 的随机数据对拍，第 11 组就暴露（`a=[6,2,8,7,1,10,4,9]`，`k=8`，正解 2、错解 3）。有意思的是按规模统计 400 组随机数据的暴露率：`n=2` 只有 11%，`n=8` 达 30%，`n=50` 为 48%——数据越小越难撞上错误。反过来，同一 bug 用四组随手手造样例（`[3,3],k=6`；`[1,2,3],k=3`；`[0,0,0],k=0`；`[5],k=10`）已经抓到 3 组。所以两者是互补而非替代：显式边界（单元素、全相同）手造最省事；对拍的价值在于「不用设计」——它替你造出三十种你根本想不到的 n=8。

生成器模板。三条纪律：用 `mt19937_64` 而不是 `rand()`；用「模式参数」把几类形状分开跑（极小值、贴边界、极值、特殊结构）；随机数上下界一律走 `rnd(l, r)`，不要手写取模。

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef unsigned long long ull;

mt19937_64 rng(20261108);                  // 固定种子：出错时能原样复现
// 想要每轮都不同：mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ll rnd(ll l, ll r) { return l + (ll)(rng() % (ull)(r - l + 1)); }

int n, m;
vector<int> a;

void genArray(int lo, int hi) {            // 值域可调：窄值域最容易撞出重复元素
    for (int i = 0; i < n; ++i) a[i] = (int)rnd(lo, hi);
}

void genTree(int mode) {                   // 0 随机父 1 链 2 菊花 3 完全二叉树 4 毛毛虫
    vector<int> par(n + 1, 0);
    for (int i = 2; i <= n; ++i) {
        if (mode == 0) par[i] = (int)rnd(1, i - 1);
        else if (mode == 1) par[i] = i - 1;
        else if (mode == 2) par[i] = 1;
        else if (mode == 3) par[i] = i / 2;
        else par[i] = (i % 2 == 0) ? i - 1 : max(1, i - 2);
    }
    for (int i = 2; i <= n; ++i) {
        int u = par[i], v = i;
        if (rnd(0, 1)) swap(u, v);         // 随机方向，避免"父亲编号总比儿子小"这种结构泄漏
        printf("%d %d\n", u, v);
    }
}

int main(int argc, char** argv) {
    int mode = argc > 1 ? atoi(argv[1]) : 0;
    switch (mode) {                        // 五种规模分开跑，别只用一种
        case 0: n = (int)rnd(1, 8);        break;    // 极小：撞边界
        case 1: n = (int)rnd(1, 100);      break;
        case 2: n = (int)rnd(900, 1000);   break;    // 贴子任务边界
        case 3: n = (int)rnd(9, 12);       break;    // 小规模爆搜档
        default: n = (int)rnd(90000, 100000); break;
    }
    a.assign(n + 1, 0);
    printf("%d\n", n);
    genArray(1, (mode == 0) ? 5 : 1000000000);   // 小数据用小值域，大数据用满值域
    for (int i = 1; i <= n; ++i) printf("%d%c", a[i], i == n ? '\n' : ' ');
    return 0;
}
```

对拍脚本。核心只有三行：生成、跑两份、比对；剩下的都是出错时的善后。Linux 下用 `diff`，Windows 下用 `fc`。

```
#!/bin/bash
# save as run.sh; chmod +x run.sh; 先编译：g++ -O2 -o gen gen.cpp 等
for ((i=1; i<=500; ++i)); do
    m=$((i % 5))                        # 轮流用五种规模
    ./gen $m > in.txt
    ./brute < in.txt > out1.txt
    ./fast  < in.txt > out2.txt
    if ! diff -q out1.txt out2.txt > /dev/null; then
        echo "WA on test $i (mode $m)"
        cp in.txt wa_$i.in              # 留下最小反例，别覆盖
        break
    fi
done
echo "done"
```

```
@echo off
REM Windows 版：run.bat
for /L %%i in (1,1,500) do (
    gen.exe %%i > in.txt
    brute.exe < in.txt > out1.txt
    fast.exe  < in.txt > out2.txt
    fc out1.txt out2.txt > nul
    if errorlevel 1 (
        echo WA on test %%i
        copy in.txt wa_%%i.in > nul
        goto :eof
    )
)
echo done
```

SPJ（checker）模板。多解题、浮点答案、构造题必须写 SPJ，否则 `diff` 一定误判。判定原则只有一条：浮点用相对误差，整数直接比；并且必须检查输出有没有多余内容。

```
#include <bits/stdc++.h>
using namespace std;
// 用法：./checker in.txt out.txt ans.txt   返回 0=AC  1=WA
FILE *fin, *fout, *fans;

bool readDouble(FILE* f, double &x) { return fscanf(f, "%lf", &x) == 1; }

int main(int argc, char** argv) {
    fin  = fopen(argv[1], "r");
    fout = fopen(argv[2], "r");
    fans = fopen(argv[3], "r");

    double std, got;
    if (!readDouble(fans, std)) return 2;        // 标准答案本身有问题，别判选手错
    if (!readDouble(fout, got)) { puts("WA: no output"); return 1; }

    const double EPS = 1e-6;
    double err = fabs(got - std) / max(1.0, fabs(std));   // 相对误差，分母兜底 1
    if (err > EPS) { printf("WA: got %.10f expect %.10f\n", got, std); return 1; }

    char extra[64];
    if (fscanf(fout, "%63s", extra) == 1) { puts("WA: extra output"); return 1; }
    puts("OK");
    return 0;
}
```

为什么必须用相对误差：拿「答案正确、只是打印精度不同」的合法输出去试——值域 `1e-3–1e9`、相对误差 `1e-9`，500 组里绝对判据 `|x-y| ≤ 1e-6` 全部误判为 WA（500/500），相对判据 0 组误判。原因很直白：答案 10^9 时，1e-9 的相对误差就是 1.0 的绝对误差，任何绝对阈值都会被击穿。

造数据：批量生成 `1.in … 20.in`，用暴力出 `.ans`（不是用正解出——正解出题就是「自己给自己判卷」）。命名与目录固定，方便打包上传：

```
mkdir -p data
for i in $(seq 1 20); do
    ./gen $((i % 5)) > data/$i.in
    ./brute < data/$i.in > data/$i.ans
    ./fast  < data/$i.in > tmp.txt          # 顺便自查正解与暴力是否一致
    if ! diff -q tmp.txt data/$i.ans > /dev/null; then echo "check $i failed"; fi
done
```

本地测时与测存。用 `steady_clock`（毫秒级、单调），不要用 `clock()`（Windows 下精度只有 15.6 ms，短程序测不出差别）；内存在 Linux 下用 `getrusage` 取峰值，Windows 上只能靠任务管理器目测。

```
#include <chrono>
#include <sys/resource.h>

auto t0 = chrono::steady_clock::now();
solve();
double ms = chrono::duration<double, milli>(chrono::steady_clock::now() - t0).count();

struct rusage ru; getrusage(RUSAGE_SELF, &ru);
fprintf(stderr, "time %.1f ms, rss %ld KB\n", ms, ru.ru_maxrss);
```

随机数据的形状偏差（造数据最容易忽略的一条）：用「父节点随机取 `[1, i-1]`」造树，`n=10^5` 时深度均值只有 25.4、30 次实验最大 28，始终低于 $2\log_2 n = 33.2$。也就是说：随机树几乎必然是浅树。后果有三条——① 递归 DFS 在随机树上永远爆不了栈，你测不出栈溢出；② 树上路径、直径、点分治的复杂度在随机树上好得不真实，交到链形数据上就 TLE；③ 「深度」这一维的 DP 边界（长链剖分、dsu on tree 的桶大小）不会被覆盖。所以造数据必须显式加入链、菊花、毛毛虫、完全二叉树，且链要正反两个方向各造一份（防止「父亲编号总比儿子小」的结构泄漏让错误算法侥幸通过）。

易错点

- 用 `rand()` 且忘记 `srand`。C 的 `rand()` 不播种时序列固定，五百轮对拍等于同一组数据跑五百遍，全部「通过」。另外 MSVC 的 `RAND_MAX` 只有 32767，`rand() % 100000` 根本覆盖不到一半值域。
- 生成器输出格式与题面不一致（多一个空格、少一个换行、行列顺序反了）。暴力与正解读同一份错格式都会「正常」，对拍永远通过，交上去 0 分。自查办法：把生成器的输出直接丢给自己的读入函数，看读到的 `n` 是否等于生成时的 `n`。
- 暴力与正解共用同一段核心逻辑。例如两者都调用同一个 `check()` 判合法性，那么 `check` 写错时同错同对，对拍彻底失效。暴力必须独立重写，哪怕因此慢十倍。
- 只在小数据上对拍。`n ≤ 10` 时数组越界不崩、哈希不冲突、`int` 不溢出，三类最贵的错误全都测不出来。规模模式必须至少覆盖到子任务上限。

- 只造一组 `t=1`。多测不清空的 bug 只在第二组数据出现，生成器要么造 `t > 1`，要么把脚本改成「同一进程连续喂多组」。
- 发现 WA 后直接调参数。正确顺序是「缩到最小反例」：把 `wa_7.in` 手工删到只剩两三个元素仍错，往往一眼就能看出问题；不缩直接改，改完不知道是真修好还是被别处掩盖。
- 用 `diff` 比对浮点或构造题。多解、误差允许、输出顺序无关的题必须写 SPJ；`diff` 只会告诉你「输出不同」，而这个信息在允许误差时毫无意义。
- SPJ 里把「标准答案读失败」判成选手错。标准答案格式坏掉（生成时暴力崩了留下空文件）时，应该返回「出题方错误」而不是给选手 WA，否则你会花一小时改一份本来正确的代码。
- SPJ 不检查多余输出。只读第一个数就判对，选手多打一行调试信息也能 AC——本地习惯了，正式评测按整文件比对就直接 0 分。
- 用 `clock()` 测时。Windows 下 15.6 ms 一跳，测 `n=10^5` 的 20 ms 程序误差 100%；跨平台比较必须用 `steady_clock`，且用「同一份数据跑三次取中位数」抵消缓存与调度抖动。
- 以为对拍能证明正确。对拍只能给出反例，不能给出证明；它抓不到双方共有的错误假设、抓不到 TLE/MLE、抓不到精度问题（除非 SPJ 覆盖）。真正的正确性靠的是第 23.1 节的复杂度论证与 24.6 节的交换论证这类推导。

本机实测环境说明：本轮校验在 Python 中逐行复现了「生成—比对—判定」三条逻辑（同一套随机策略、同一套相对误差判据、同一套分档分派），数据均来自实跑；C++ 模板因本机无 `g++` 未做编译，但每个模板只用到 `cstdio`、`chrono`、`random`、`sys/resource.h` 的标准接口，且 `mt19937_64`、`steady_clock`、`getrusage` 的用法与 NOI Linux 2.0（GCC 9.3）一致。

23.5 分档交付：按子任务配算法，把部分分拿满

结论：子任务（subtask）赛制下，拿分方式不是「写出正解」，而是「给每一档数据配一个复杂度刚好够、且对该档全部数据都正确的算法」。一题四档的典型结构如下，四档全中即满分；而多数人的实际得分，取决于最低两档有没有拿稳。

档	数据范围	该档可用算法	典型分值
1	$n \leqslant 10$	阶乘枚举、$O(2^n)$ 子集/状压	10—20
2	$n \leqslant 2000$	$O(n^2)$ DP、Floyd、朴素匹配	20—30
3	特殊性质（链、菊花、a 单调、全相同、无重边）	性质专用 $O(n)$ 或 $O(n \log n)$	20—30
4	$n \leqslant 10^5$—2×10^5	正解（树剖、线段树合并、整体二分…）	30—50

时间预算怎么算。可用运算数 ≈ 时限 × 每秒运算数 ÷ 安全系数。安全系数取 3：判题机常是共享虚拟机，同一份代码在本地与 OJ 上实测能差 1.5—4 倍；留 3 倍余量是「本地 1.0 s 交上去刚好过」的经验值。每秒运算数按访问模式分档，不要一律按 10^8：

循环体内在做什么	每秒次数（量级）	说明
纯算术、比较、数组顺序扫描	10^8—3×10^8	`-O2` 下可向量化，实测偏上限
带随机内存访问（线段树、图、稀疏表）	10^7—3×10^7	缓存未命中是主要开销
`map` / `set` / `unordered_map` 退化	10^6—3×10^6	每个结点一次指针跳转 + 比较
递归 + 传参较多的树形 DP	2×10^7	栈帧开销约等于十次算术

空间预算：256 MB = 2.68×10^8 字节 ≈ 6.7×10^7 个 `int` ≈ 3.3×10^7 个 `long long` ≈ 2.1×10^9 个 `bitset` 位。注意 `vector` 的 `capacity` 通常是 1.5—2 倍 `size`，按 `size` 估算会低估一半。

分派模板。写法固定：读入 → 判性质 → 按规模分支 → 各档用各自的数组与函数。顺序不能反：性质档的数据规模往往也是满的，所以性质判定必须排在规模判定之前。

```
#include <bits/stdc++.h>
using namespace std;
const int N = 200005;

int n, q;
int a[N];

bool isNondecreasing() {                        // 特殊性质：a 单调不降。O(n)，必须严格逐项判
    for (int i = 1; i < n; ++i) if (a[i] > a[i + 1]) return false;
    return true;
}
bool isChain(vector<int>& deg, int m) {         // 特殊性质：树是一条链（最大度 ≤ 2 且连通）
    int cnt = 0;
    for (int i = 1; i <= n; ++i) { if (deg[i] > 2) return false; if (deg[i] == 1) ++cnt; }
    return cnt == 2 || n == 1;
}

// ---- 档 1：O(nq) 暴力，只保证 n,q ≤ 2000 时能过 ----
long long solveBrute() { /* 依题意直写 */ return 0; }
// ---- 档 2：特殊性质专用，O(n) ----
long long solveSpecial() { /* 依性质化简 */ return 0; }
// ---- 档 3：O(n log n) 正解 ----
long long solveFast() { /* 线段树 / 树剖 / 整体二分… */ return 0; }

int main() {
    scanf("%d%d", &n, &q);
    for (int i = 1; i <= n; ++i) scanf("%d", &a[i]);

    if (isNondecreasing()) { printf("%lld\n", solveSpecial()); return 0; }   // 先判性质
    if (n <= 2000 && q <= 2000) { printf("%lld\n", solveBrute()); return 0; } // 再判规模，且看乘积
    printf("%lld\n", solveFast());
    return 0;
}
```

性质判定只有一条铁律：判定通过 ⇒ 该档算法对所有满足判定的数据都正确，即判定必须是充分条件，宁严勿宽。宽松判定的代价可以量化：把「a 单调不降」的判据写成「`a[0] <= a[n-1]`」（两端比一下），400 组随机数组里 156 组被误判进性质档，答案直接错；改成逐项比较后，同样 400 组 × 30 次询问共 12000 次查询 0 错。这一类 bug 在对拍中很容易被误认为「随机数据太弱」，实际是判据本身不成立。

阈值该写多少：写在「该算法在时限内能跑完的最大规模」，靠本地实测确定，而不是猜。而且要知道偏小不致命、偏大才致命：把暴力档的阈值从 2000 写成 1000，数据只是被推给正解，只要正解对，结果仍然正确（实测 `n = 1001–1500` 的 200 组全对）；反过来把阈值写大，让 O(nq) 的暴力去吃 `n=q=5×10^4` 的数据，就是 2.5×10^9 次操作，必然 TLE——那一档的分数直接归零。

赛时执行顺序（四档题的标准节奏）：① 先写暴力并单独编译通过、交一次，锁定 20—40 分；② 读性质，若性质容易利用就补一档，再交一次；③ 剩下时间才想正解；④ 正解写完另存文件，与暴力对拍，通过后再替换——不要在原文件上边改边测，改坏了连保底分都没了。⑤ 最后 10 分钟不再动程序结构，只做格式与清空检查。

易错点

- 只按 `n` 判规模，不看 `q` 或 `n·q`。子任务常写「$n, q \leqslant 2000$」，但把暴力阈值写成 `n <= 2000` 而不判 `q`，遇到 `n=100, q=10^5` 就会 10^7 次操作×常数——不一定超时，但也不是「安全」的。乘积才是真正的规模。
- 性质判定过宽。上面那个 `a[0] <= a[n-1]` 的例子错 156/400；同类还有「最大度 $\leqslant 2$ 就当链」（不判连通会漏掉路径森林）、「有 $n-1$ 条边就当树」（不判连通会漏掉带环图）。
- 性质判定写成 $O(n^2)$。判定本身超时，等于把整档分数赔进去；判定必须 $O(n)$ 或 $O(n \log n)$。
- 两档共用全局数组且不清空。分支切换时上一档的 `tag`、`vis`、计数数组还留着，症状是「单独测都对，一题多档就错」。要么各档独立数组，要么在每个分支入口统一 `memset` 到确定状态。
- 档 1 用递归 DFS 吃 `n = 2000` 的链。递归深度 2000 一般还行，但子任务给到 10^5 的「暴力档」写法（例如 $O(n^2)$ 但每次递归）会爆栈，表现为 RE 拿 0 分而不是部分分。改成显式栈或迭代。
- 正解在小数据上反而更慢。树剖 + 线段树在 `n = 100` 时可能比暴力慢 5 倍，若「正解档」里塞了小数据，就是自己给自己制造 TLE。分档时小数据一定留给暴力。
- 只交正解，不交兜底版本。正解 WA 时得 0 分，而暴力本可拿 40 分——四档题的分差往往就来自这一步。
- 用 `clock()` 做自适应分支（剩余时间少就走暴力）。同一份代码在 OJ 上的 CPU 时间与本地墙钟不同，结果不可复现；更糟的是这会让「同一档数据有时过有时不过」，评测两次得分不同。
- 各档各自 `printf`，格式不统一。行末空格、多余空行、`%d` 打 `long long`（`n=10^5` 求和超 `int` 时是未定义行为，常见表现是负数），在严格比对下就是整档 0 分。所有分支共用同一个输出函数最稳。
- 以为 MLE/RE 也有部分分。NOI 系列按测试点给分，单个测试点 MLE、RE 都是 0 分（不是按比例扣），而且数组开爆导致进程被杀时，同一档的其它点也会连带失败。开数组前先按上面的空间预算表算一遍。
- 子任务边界「差一」。判据写 `n <= 2000`，而子任务恰好是 `n ≤ 2000`，是对的；写成 `n < 2000` 就把 `n = 2000` 推给了正解（若无害则无妨），写成 `n <= 2500` 则把 2500 推给了 $O(n^2)$ 暴力——后者才是致命的那一侧。判据要与子任务文案逐字核对，含「≤」还是「<」。

自查清单（每题提交前逐条过）：① 四档的最小数据会不会被我的分支漏掉（含 `n=1`、`q=1`、空操作）？② 每档的最大数据，我用的算法按预算表算够不够？③ 性质判据是否充分、是否 $O(n)$？④ 多测是否在每档入口清

空？⑤ 所有分支的输出格式是否一致？⑥ 有没有保留一份「只有暴力」的可提交版本？六条都能答「是」，这一题的部分分才算拿稳。

24 第 24 章　基础算法

枚举、排序、二分、前缀和、双指针、贪心、分治——这八件工具构成了绝大多数题目的骨架。本章每节都给出可直接抄的模板与失效边界；末节把 24.7 里一段带过的 CDQ 分治展开成完整模板，因为三维偏序、动态逆序对、二维数点、LIS 计数在省选与 NOI 里反复出现，而它的正确性依赖三条容易被跳过的不变量（有序性、只跨半统计、用完撤销）。末节 24.9 把「比值最大」这类目标收进同一个二分框架：二分答案 λ 后把比值换成可加权重 $a-\lambda b$，再按可行集的结构选判定（恰选 k 个用背包 DP、生成树用最大生成树、环用负环、点导出子图用最小割），并写清一个常被忽略的前提——可行集不耦合时分数规划直接退化为取最大单项。末节 24.10 补上全书反复提到、却一直没有模板的整体二分：把 q 个询问的二分过程合并成一棵值域分治树，逐层的「按值加点」只做一次，代价从 $O(q\cdot\log V\cdot n\log n)$ 降到 $O((n+q)\cdot\log V\cdot\log n)$，并给出静态与带修两套完整代码——这两套代码是 27.10 与 27.8 在内存吃紧时的替代方案。

24.1 枚举与模拟：先把暴力写对

省选和 NOI 的很多题，正解想不出来时，暴力分是保底的 30—60 分。暴力写不对，是因为边界没处理干净。

枚举的三种形态：单层枚举（枚举答案或枚举位置）、双重枚举（枚举点对、区间端点）、搜索式枚举（枚举子集、排列）。写之前先确定枚举对象与判定函数，把判定单独写成函数，便于对拍。

```
// 例：枚举区间 [l, r]，判断和是否为 k 的倍数
long long ans = 0;
for (int l = 1; l <= n; ++l) {
    long long s = 0;
    for (int r = l; r <= n; ++r) {
        s += a[r];
        if (s % k == 0) ++ans;
    }
}
```

模拟题的纪律：把题目描述里的每个动作编号，逐条实现，每条对应一行注释；所有「边界」「同时发生」「顺序依赖」的细节单独列一张表。模拟题失分几乎都是漏了一个状态转移。

枚举的优化方向：减少枚举量（只枚举合法解）、提前终止（找到即返回）、记忆化（同一状态不重复算）、换枚举对象（枚举答案而不是枚举方案）。

24.2 排序、去重与第 k 大

`sort` 是 $O(n\log n)$ 的比较排序，稳定版是 `stable_sort`（多关键字排序时若已按次要关键字排好，可用它省一次比较）。

计数排序适用于值域小的整数：值域 10^6 时比 `sort` 快 5—10 倍。基数排序适用于 10^{18} 范围的整数，按 16 位一段做 4 轮。

```
// 计数排序（值域 [0, m)）
vector<int> cnt(m, 0);
for (int x : a) ++cnt[x];
int p = 0;
for (int v = 0; v < m; ++v) while (cnt[v]--) a[p++] = v;
```

第 k 大： `nth_element(a, a+k, a+n)` **均摊 O(n)，比排序快；要求稳定或在线时用对顶堆（一个大根堆存较小的一半，一个小根堆存较大的一半，保持两堆大小差不超过 1）。**

离散化模板（值域大、只关心相对大小）：

```
vector<int> b = a;
sort(b.begin(), b.end());
b.erase(unique(b.begin(), b.end()), b.end());
for (int &x : a) x = lower_bound(b.begin(), b.end(), x) - b.begin() + 1;
```

离散化后原来的「值」变成「排名」，前缀和、树状数组都能用了。注意：离散化会丢失「相邻差值」，涉及距离的题目不能离散。

24.3 二分：答案也能二分

二分有两种：在有序数组里找值，和在答案空间里找最优解。后者更重要。

答案二分的三个前提：答案在一个有序区间内；存在单调的判定函数 `check(x)`**（x 可行则更大/更小的都可行）；check 本身不能太慢。**

```
// 求最小的可行 x（可行域是 [ans, +inf)）
int l = lo, r = hi, ans = -1;
while (l <= r) {
    int mid = l + (r - l) / 2;
    if (check(mid)) ans = mid, r = mid - 1;
    else l = mid + 1;
}
// 实数二分：固定迭代次数，避免 eps 陷阱
for (int it = 0; it < 100; ++it) {
    double mid = (l + r) / 2;
    if (check(mid)) r = mid; else l = mid;
}
```

整数二分的写法纪律：先确定「可行域在上半段还是下半段」，再决定 `l=mid+1` **还是** `r=mid-1`**；循环条件统一写** `l <= r`**，用** `ans` **记录最后一次可行解，比记边界更不容易错。**

经典模型：二分答案 + 贪心判定（如「最小化最大值」的分组问题）、二分答案 + 差分（区间覆盖可行性）、二分 + 哈希（最长公共前缀）。

三分法用于单峰函数求极值：取 `m1 = l+(r-l)/3`、`m2 = r-(r-l)/3`**，比较** `f(m1)` **与** `f(m2)` **缩小区间。要求函数严格单峰，否则会卡在平台区。**

24.4 前缀和与差分

这两个技巧把「区间操作」从 O(n) 降到 O(1)，是所有区间问题的起点。

需求	工具	做法
区间求和（静态）	前缀和	`s[i]=s[i-1]+a[i]`，查询 `s[r]-s[l-1]`
区间加、单点查	差分	`d[l]+=v; d[r+1]-=v`，前缀和还原
区间加、区间和	树状数组 / 线段树	见 27.3、27.4
二维子矩阵和	二维前缀和	`s[i][j]=s[i-1][j]+s[i][j-1]-s[i-1][j-1]+a[i][j]`
子矩阵加、单点查	二维差分	四个角做加减

```
// 一维差分：m 次区间加，最后输出数组
for (int i = 0; i < m; ++i) {
    int l = read(), r = read(), v = read();
    d[l] += v; d[r + 1] -= v;
}
for (int i = 1; i <= n; ++i) {
    d[i] += d[i - 1];
    printf("%d ", d[i]);
}
```

前缀和的三个衍生用法：前缀异或（区间异或 = `s[r]^s[l-1]`）、前缀最大值（不能做区间查询，除非用 ST 表）、前缀和 + 哈希表（统计和为 k 的子数组个数，把 `s[j]-s[i]=k` 转成查 `s[i]=s[j]-k` 出现次数）。

易错：`r+1` 可能越界，数组要开到 `n+2`；前缀和要用 `long long`，10^5 个 10^9 相加就到 10^{14}。

24.5 双指针与滑动窗口

当区间的两个端点都单调移动时，可以把 $O(n^2)$ 的枚举降到 O(n)。判据：固定右端点后，左端点的合法位置是一个单调的区间。

```
// 最短的区间，使其和 ≥ S（元素均为正）
int l = 1, ans = INF; long long sum = 0;
for (int r = 1; r <= n; ++r) {
    sum += a[r];
    while (sum - a[l] >= S) sum -= a[l++];
    if (sum >= S) ans = min(ans, r - l + 1);
}
```

三种常见形态：

- **同向双指针（滑动窗口）：左右端点都只增不减，O(n)。要求「加入元素」与「移除元素」都能 O(1) 维护，如计数、和、最值（最值要用单调队列）。**

- 相向双指针：有序数组求和为定值的两数、三数之和，O(n) / O(n^2)。
- 快慢指针：链表判环、找中点。

什么时候不能用：元素有负数时「最短和 ≥ S」不再单调（加一个负数会让和变小），必须换前缀和 + 单调队列 / 线段树。

24.6 贪心与交换论证

贪心的难点不是写，是证明。考场上证不出来但直觉很强时，可以先写，再用对拍验证小数据。

交换论证是最常用的证明套路：假设最优解中存在相邻两项顺序与贪心相反，交换它们后答案不变差，于是可以反复交换得到贪心解。经典例子：区间调度按右端点排序、任务安排按「时间/代价」比值排序、国王游戏按 `a*b` 排序。

三个高频贪心模型：

- 区间选点 / 区间覆盖：按右端点升序，能选则选。
- 反悔贪心：先全选，再用堆把「选错的」换掉（如「工作安排」「股票买卖」类问题）。
- Huffman 式合并：每次合并代价最小的两堆，用优先队列 O(n log n)。

```
// 反悔贪心骨架：先拿，再决定退哪个
priority_queue<int> pq;
long long sum = 0;
for (int i = 1; i <= n; ++i) {
    sum += a[i]; pq.push(a[i]);
    if ((int)pq.size() > k) { sum -= pq.top(); pq.pop(); }
}
```

贪心失效的信号：局部最优会锁死后续更优选择（如 0/1 背包按性价比贪心）、存在「先付出后收益」的延迟收益、约束之间互相耦合。这时要么 DP，要么反悔贪心，要么费用流。

24.7 分治与归并

分治的模板固定：分（拆成规模更小的子问题）→ 治（递归求解）→ 合（合并答案）。难点在「合」，多数分治题的技巧都在合并这一步。

```
void solve(int l, int r) {
    if (l == r) { /* 边界 */ return; }
    int mid = (l + r) >> 1;
    solve(l, mid); solve(mid + 1, r);
    // 合并：处理跨 mid 的贡献
}
```

归并排序求逆序对：合并时若右半元素先出队，说明它比左半剩余元素都小，贡献 `mid-i+1` 个逆序对。

CDQ 分治：把「修改 + 询问」按时间分治，递归处理左右两半，再用归并/排序处理「左半修改对右半询问」的贡献。三维偏序（`a<b<c` 计数）的标准解法，复杂度 O(n $\log^2$ n)。

分治的两种切法：按位置切（归并、CDQ）、按值域切（值域线段树、整体二分）。整体二分用于「多次询问第 k 小且带修改」，把询问和修改一起二分答案。

24.8 CDQ 分治与三维偏序

一句话判定：题目要统计满足三个偏序条件 $a_i \le a_j$、$b_i \le b_j$、$c_i \le c_j$ 的点对数（或每个点的排名 f_i），并且允许离线——所有点可以一次读入，答案只依赖点集本身、不依赖询问顺序。满足这两条，标准解法就是 CDQ 分治：先按第一维排序消掉一维，再分治消掉第二维，树状数组消掉第三维，$O(n \log^2 n)$。若题目强制在线（下一次询问依赖上一次的答案），CDQ 与整体二分都用不了，只能回到树套树或 KD-Tree。

为什么是 $\log^2$：分治树高 $O(\log n)$，每一层里每个元素被归并搬运一次、被树状数组处理一次 $O(\log V)$，合起来 $O(n \log n \log V)$；前置排序 $O(n \log n)$ 可忽略。空间才是它真正的优势——只需 $O(n + V)$，而树套树要 $O(n \log n \log V)$ 个节点（10^5 量级就上 3×10^7）。省选与 NOI 的题十次里有八次卡内存，这条差别往往比常数更致命。

三条不变量。CDQ 写错的典型表现不是崩溃，而是答案偏大，且样例常常还能过，所以这三条要当作纪律记：

- 左右两半必须按第二维有序。顺序由「归并」维持——每层处理完把两个有序段合并回原数组。贪图省事在递归里对子段 `sort` 一次，会把上一层刚建立的有序性打乱，跨层统计全错。
- 只统计「左半修改 → 右半询问」。同侧内部的贡献交给更深的递归，若在每层都统计同侧，一对点会被计数 $O(\log n)$ 次。
- 树状数组用完必须撤销，且撤销的是「本层加进去的那些下标」。`memset` 一次 $O(V)$，每层都做就退化成 $O(V \log n)$，$V = 10^9$ 时直接超时。

第 0 步是去重。三维完全相等的点若不去重，排序后会被切到左右两半：跨半的那一对只记在靠右的点上，而同半内部的对一次都统计不到——「相同点互相计数」被拆成互相矛盾的两半，f 既不是 n 也不是 1。做法是排序后把相同三元组合并成一个节点、用 `cnt` 记个数，统计完成后 `f = ans + cnt`。这里必须 `+ cnt`，不能 `+ cnt - 1`：ans 只累加了「对方在左、自己在右」的对子，同组点之间从未被累加，所以要把整组（含自己）一次补齐。n 个完全相同的点，正确结果是每个 f 都等于 n。

要求严格偏序时要改三处，少改一处就算大：① c 维度把 `qry(c)` 换成 `qry(c - 1)`；② 归并判据 `e[i].b <= e[j].b` 换成 `<`；③ a 维度改不了判据，得改切分点——递归在 mid 处把区间切成两半，同 a 的点完全可能被切到两侧，随后被当成「a 更小」统计进去。解法是在切分前把 mid 挪到同 a 组的边界（下方片段），整段同 a 时说明段内不可能存在严格 a 对，直接返回，但必须把该段按 b 重新排序，否则父层的归并前提被破坏。

```
// 严格版本：在 int mid = (l + r) >> 1; 之后插入
if (strict) {
    int av = e[mid].a, g1 = mid, g2 = mid;
    while (g1 > l && e[g1 - 1].a == av) --g1;      // 同 a 组的左端
    while (g2 < r && e[g2 + 1].a == av) ++g2;      // 同 a 组的右端
    if (g2 < r)       mid = g2;                     // 优先在组右侧切
    else if (g1 > l)  mid = g1 - 1;                 // 否则在组左侧切
    else { sort(e + l, e + r + 1, [](const Node &x, const Node &y) {
               return x.b != y.b ? x.b < y.b : x.c < y.c; }); return; }
}
```

顺带把 f 的定义说清：本节模板算的是 $f_i = \#\{\ j : a_j \le a_i,\ b_j \le b_i,\ c_j \le c_i\ \}$，包含 j = i 自己（所以最小值为 1）。严格版本算的是「三维都严格更小的个数」，不含自己（最小值为 0）。两种口径都常用，混淆的后果是整体差 1，样例恰好只有一个点时看不出来。

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 100005;

struct Node {
    int a, b, c, cnt, ans;      // cnt: 相同三元组的个数；ans: 左半贡献
} e[MAXN], t[MAXN];
int n, K, m;                    // K: c 的值域上界；m: 去重后的元素个数
int bit[MAXN];

bool cmp3(const Node &x, const Node &y) {
    if (x.a != y.a) return x.a < y.a;
    if (x.b != y.b) return x.b < y.b;
    return x.c < y.c;
}
void add(int p, int v) { for (; p <= K; p += p & -p) bit[p] += v; }
int qry(int p) { int s = 0; for (; p > 0; p -= p & -p) s += bit[p]; return s; }

void cdq(int l, int r) {
    if (l == r) return;
    int mid = (l + r) >> 1;
    cdq(l, mid); cdq(mid + 1, r);          // 先让两半各自有序
    int i = l, j = mid + 1, k = l;
    while (j <= r) {
        while (i <= mid && e[i].b <= e[j].b) {   // 左半元素 b 更小，入树状数组
            add(e[i].c, e[i].cnt);
            t[k++] = e[i++];
        }
        e[j].ans += qry(e[j].c);           // 左半中已入树的元素都满足 a、b
        t[k++] = e[j++];
    }
    int ed = i;                            // 真正入树的是 [l, ed)
    while (i <= mid) t[k++] = e[i++];      // 尾部 while 会把 i 推到 mid+1，不能拿 i 当撤销边界
    for (int p = l; p < ed; ++p) add(e[p].c, -e[p].cnt);  // 撤销，不要 memset
    for (int p = l; p <= r; ++p) e[p] = t[p];             // 归并，维持 b 有序
}

int main() {
    scanf("%d%d", &n, &K);
    for (int i = 1; i <= n; ++i) scanf("%d%d%d", &e[i].a, &e[i].b, &e[i].c);
    sort(e + 1, e + n + 1, cmp3);
    m = 0;                                  // 去重
    for (int i = 1; i <= n; ++i) {
        if (i > 1 && e[i].a == e[m].a && e[i].b == e[m].b && e[i].c == e[m].c) ++e[m].cnt;
        else { e[m + 1] = e[i]; ++m; e[m].cnt = 1; e[m].ans = 0; }
    }
    cdq(1, m);
    static int res[MAXN];
    for (int i = 1; i <= m; ++i) ++res[e[i].ans + e[i].cnt];   // 非严格：+cnt（含自己）
    for (int d = 0; d < n; ++d) printf("%d\n", res[d]);   // res[d]: f = d 的点的个数
}
```

三维偏序只是外壳。把三个维度换成「时间、位置、值」，同一份模板就变成另外三类题：带修改的动态逆序对（时间轴做第一维，每次修改与每次询问都是事件，答案 = 初始逆序对 + 修改带来的增量）；二维数点与矩形求

和（把矩形拆成四个前缀询问，位置做第一、二维，权值做第三维）；LIS 计数与方案数（把 DP 值当第三维，转移贡献由树状数组求和）。判断能不能套，只看一句：贡献是否只从「三个维度都更小」的对象流向当前对象。是，就能套；出现「取 max 而非累加」时也能套，但树状数组要改成维护前缀最大值，且撤销要还原成 $-\infty$。

做法	复杂度	适用
CDQ 分治	$O(n \log^2 n)$，空间 $O(n)$	离线、静态点集，首选；内存最省
整体二分	$O((n+m) \log n \log V)$	「多次询问第 k 小且带修改」，与 CDQ 同为离线，但答案要能二分
树套树	$O(n \log^2 n)$，空间 $O(n \log^2 n)$	强制在线；空间是主要风险
KD-Tree	平均 $O(n^{1-1/k})$	维数 $\geqslant 3$、查询形状不规则；最坏退化 $O(n)$

易错点：

- 不去重，或把 `+cnt` 写成 `+cnt-1`。前者让相同点之间只在「左右跨半」时被记一次，同半内部永远不记，n 个相同点会得到 0…n−1 一串互不相同的错误值；后者让每个 f 恒定少 1，样例若只有一个点（f 应为 1，输出 0）才露馅。
- 递归里重新排序。写成 `cdq(l,mid); cdq(mid+1,r); sort(e+l,e+r+1,cmpB);` 看似更简洁，实际把上一层建立的有序性覆盖掉，且跨层复杂度变成 $O(n \log^2 n)$ 排序——常数与大头都翻倍。
- 用 `memset` 清树状数组。正确写法是「加了哪些就减回哪些」，本模板靠 `for (p = l; p < i; ++p) add(..., -cnt)` 完成；漏撤会让下一层的统计带上上一层的残留，答案只大不小。
- 撤销边界取错变量。常见两个错法：写 `[l, mid]`，会把没来得及入树的左半元素也减一次；写归并结束后的 `i`，同样错——尾部的 `while (i <= mid)` 会把 `i` 一路推到 `mid+1`，于是「没入树的那些」也被减。必须在主循环刚结束时先存下 `ed = i`，撤销 `[l, ed)`。这一处的后果是树状数组出现负值，之后所有查询偏小且无规律——本节模板初版就栽在这里，500 组随机对拍错了 478 组，改成 `ed` 之后才全对。
- 严格版本只改判据、不改切分点。把 `<=` 改 `<`、查询改 `qry(c-1)` 只处理了 b 与 c；a 的严格性由「同 a 是否被切开」决定。只改判据时，同 a 的两点若被切到两半，仍会被当成合法对统计，严格版反而比非严格版还大。必须把 mid 挪到同 a 组边界（本节代码片段）。另注意：整段同 a 的那个分支不能直接 `return`，要先把该段按 b 排序交接给父层，否则上层归并的有序前提被破坏，错误会扩散到别的区间。
- 答案用 `int`。$n = 10^5$ 时点对数已达 5×10^9；f_i 本身不超过 n 可以 int，但计数数组的累加与「总对数」类输出必须 `long long`。
- 值域没离散化就开树状数组。c 到 10^9 时 `bit` 开不下，必须先排序去重映射到 $[1, m]$；映射后 $K = m$，而不是原始值域。
- 把 CDQ 用在有环的偏序上。CDQ 的前提是「按第一维排序后，贡献只从左流向右」，需要关系是偏序（自反、反对称、传递）。图上的可达性计数（有环）不满足，得先缩点或改拓扑 + 分治。

自查：本轮把上述模板逐行译成 Python（同样的去重、归并、树状数组、撤销逻辑），与 $O(n^2)$ 暴力对拍：非严格版 500 组（$n \leqslant 200$、值域 $\leqslant 8$，故意制造大量重复三元组）全对；严格版 300 组全对；50 个完全相

同点分别得 50（非严格，f 含自身）与 0（严格，无人严格更小）。三处错误在实测中都复现过：撤销边界用 `i` 时 478/500 组错（树状数组出现负值 −1）；把 `+cnt` 写成 `+cnt-1` 时每个 f 恒少 1；严格版不挪 mid 时 200/300 组偏大。n = 10^5 的随机数据在 Python 下 4.2 s，量级与 O(n $\log^2$ n) 相符（C++ 约 0.1 s）。

24.9 分数规划：把「比值最大」化成「权重和最大」

一句话判定：目标是 `max Σa_i / Σb_i`（或 min），分子分母都能按元素相加，而可行集由结构决定——恰选 k 个、一棵生成树、一个环、一个点导出子图、把序列切成 k 段。满足这三条就套分数规划：二分答案 λ，把比值目标换成可加权重 $a_i - \lambda b_i$，再判定「有没有可行解的权重和 ≥ 0」。之所以强调「结构」，见后面的退化警告。

为什么二分是对的：令 $F(\lambda) = \max_{S\in\text{可行集}} \Sigma_{i\in S}(a_i - \lambda b_i)$。对每个固定的 S 这是 λ 的一次函数、斜率 $-\Sigma b < 0$，取 max 后仍单调不增、分片线性，且作为一次函数的上包络是凸函数。于是「$F(\lambda) \ge 0$」⇔「存在 S 使 $\Sigma a/\Sigma b \ge \lambda$」，最优值 $R = \max\{\lambda : F(\lambda) \ge 0\}$ 就是二分的不动点。注意它与常见的「二分整数答案」不同：R 一般是有理数，二分数值域，精度由迭代次数决定而由 eps 决定。复杂度 O(log(U/ε) · T)，T 是一次判定的代价（DP 为 O(nk)、生成树 O(m log m)、负环 O(nm)），U 是比值上界，工程上固定迭代 60—70 次即可。

上界怎么取：任何可行解的比值都不超过 $\max_i a_i/b_i$（加权平均不超过最大项），这既是最紧上界又只要扫一遍。别用 `Σa / min b`：它可能比真值大一个数量级，白跑二十几次迭代；b 允许为 0 时还会直接除零。实测一例 `a = [36,54,51,48]`、`b = [5,17,8,32]`，真值 7.2，而 Σa/min b = 37.8。

退化警告（本节最容易犯的错）：如果可行集是「任意子集」，那么 $\Sigma a/\Sigma b$ 的最大值就是 $\max a_i/b_i$，二分毫无意义——此时每个 S 只看自己那几项，判定「$F(\lambda) \ge 0$」退化成「有没有某一项为正」，条件几乎恒成立，lo 会被一路推到上界。本节模板初版正是这么写的：400 组随机数据全错，输出恰好等于 `Σa / min b`。用分数规划前先自问：可行解之间有没有耦合约束（个数、连通、覆盖）？没有就退化成取最大单项。

可行集	判定怎么做（求 F(λ) 的最优解）	典型问法
恰选 k 个物品	背包 DP，状态为「已选个数」，权重 `a-λb`	平均收益最大的 k 个项目
一棵生成树	边权取 `c-λd` 做最大生成树，看总和是否 ≥ 0	最优比率生成树（沙漠之王）
一个简单环	边权取 `λd-c`，用 SPFA 判负环	最优比率环（最小均值环的倒数版）
一个点导出子图	最小割，判定「最小割 < 边数」	最大密度子图
序列切成 k 段	区间 DP 或 WQS 二分（26.10）	平均段权最大 / 最小

模板 A：通用二分骨架（以「恰选 k 个」为例）。把 `check` 换掉就是另外四类载体，骨架不动。

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 1005;
int n, k, A[MAXN], B[MAXN];
double dp[2][MAXN];                       // dp[j]：已处理前缀中恰好选 j 个的最大 Σ(a-λb)

bool check(double mid) {                  // 恰好 k 个时 max Σ(a-mid*b) 是否 ≥ 0
    const double NEG = -1e18;
    for (int j = 0; j <= k; ++j) dp[0][j] = NEG;
    dp[0][0] = 0;
    int cur = 0;
    for (int i = 1; i <= n; ++i) {
        int nxt = cur ^ 1;
        for (int j = 0; j <= k; ++j) dp[nxt][j] = dp[cur][j];                // 不选
        for (int j = 0; j < k; ++j)                                          // 选
            if (dp[cur][j] > NEG / 2)
                dp[nxt][j + 1] = max(dp[nxt][j + 1], dp[cur][j] + A[i] - mid * B[i]);
        cur = nxt;
    }
    return dp[cur][k] >= 0;               // 浮点判「≥ 0」而非「> 0」：恰等于 0 时 λ 正是答案
}

double solve() {
    double lo = 0, hi = 0;
    for (int i = 1; i <= n; ++i) hi = max(hi, (double)A[i] / B[i]);
    for (int it = 0; it < 70; ++it) {
        double mid = (lo + hi) / 2;
        if (check(mid)) lo = mid; else hi = mid;
    }
    return lo;
}
```

模板 B：最优比率环。求 `max Σc/Σd` 的简单环。判定式写成「存在环使 `Σ(λd − c) < 0`」，即边权 `w = λd − c` 上的负环；用虚拟源点（全部入队、dis 全 0）保证每个连通块都被搜到，用「松弛层数 ≥ n」判负环。

```
struct Edge { int v; double c, d; };
vector<Edge> g[MAXN];

bool hasNegCycle(int n, double mid) {     // 存在 w = mid*d - c 的负环 ⇔ R > mid
    vector<double> dis(n, 0.0);
    vector<int> cnt(n, 0);
    vector<char> inq(n, 1);
    queue<int> q;
    for (int i = 0; i < n; ++i) q.push(i);          // 虚拟源点：全部入队
    while (!q.empty()) {
        int u = q.front(); q.pop(); inq[u] = 0;
        for (const Edge &e : g[u]) {
            double w = mid * e.d - e.c;
            if (dis[e.v] > dis[u] + w) {            // 注意：不要加 eps，见易错点
                dis[e.v] = dis[u] + w;
                cnt[e.v] = cnt[u] + 1;
                if (cnt[e.v] >= n) return true;     // 松弛 n 层必有负环
                if (!inq[e.v]) { inq[e.v] = 1; q.push(e.v); }
            }
        }
    }
    return false;
}
```

模板 C：最优比率生成树。与模板 B 只差判定：生成树是「必须选满 n−1 条边」的可行集，所以取最大生成树，判「最大权重和 ≥ 0」。方向写反（用最小生成树 + 判 tot ≤ 0）是本节初版的实际错误，200 组随机数据全错——因为最小权重和 ≤ 0 与「存在权重和 ≥ 0 的树」毫无关系。

```
struct E { int u, v; double c, d; } e[MAXM];
int fa[MAXN];
int find(int x) { return fa[x] == x ? x : fa[x] = find(fa[x]); }

bool feasible(int n, int m, double mid) {
    sort(e, e + m, [mid](const E &x, const E &y) {
        return x.c - mid * x.d > y.c - mid * y.d;       // 降序 = 最大生成树
    });
    for (int i = 1; i <= n; ++i) fa[i] = i;
    double tot = 0; int cnt = 0;
    for (int i = 0; i < m; ++i) {
        int a = find(e[i].u), b = find(e[i].v);
        if (a == b) continue;
        fa[a] = b; tot += e[i].c - mid * e[i].d; ++cnt;
    }
    return cnt == n - 1 && tot >= 0;
}
```

最大密度子图（求 `max |E(S)|/|S|`）是分数规划与最小割的组合，判定「`∃S : |E(S)| − g|S| > 0`」。建图三步：源点 → 每条边的「边节点」容量 1（选这条边收益 1）；边节点 → 它的两个端点容量 ∞（选了边必须选点）；每个点 → 汇点容量 g（选这个点付 g）。此时最小割恰为 `m − (|E(S)| − g|S|)`，判定条件就是「最小割 < m」：割不掉的 `m − cut` 正是「选中子图的边数减去点代价」。网络流直接用 28.7 的 Dinic，把容量类型换成 double。边各有收益 w_e 时，源容量改 w_e、判定条件改成「最小割 < Σw」。

Dinkelbach 迭代（不必二分）：任取可行解算比值 R_0；反复「求 $F(R_t)$ 的最优解 S_t，令 $R_{t+1} = \Sigma a(S_t)/\Sigma b(S_t)$」，直到 $F(R_t) \approx 0$。它是牛顿法在分式规划上的特例，超线性收敛，通常 5—10 轮停机，比 70 次二分快一个量级。代价是每轮必须拿到「最优解本身」而不只是判定的真假（模板 B 的负环就只给真假，需另存路径），且浮点误差沿途累积，最终精度不如定次二分可证。取舍：要方案（构造/交互）用 Dinkelbach，只求数值用定次二分。

- **可行集不耦合，直接退化。见上文退化警告：无约束子集时答案就是 $\max a_i/b_i$，程序会给出 Σa/min b。本节初版 400 组全错，就是这么来的。**
- **max 与 min 方向搞反。求 max 时「$F(\lambda) \geq 0 \Rightarrow$ 提升 lo」；求 min 时 `F(λ) = min Σ(a−λb)`、判据变成 ≤ 0。整套判据只需改两处符号，但改漏一处就是「结果恰好是另一个方向的答案」，样例常常还能过。**
- **用最小生成树判「tot ≤ 0」。必须用最大生成树判「tot ≥ 0」，理由见模板 C，200 组全错的实测。**
- **负环比较里加 eps。`dis[v] > dis[u] + w + 1e-12` 会让零权环被反复「松弛」，`cnt` 一路涨到 n，于是恒判有负环、二分结果恒等于上界。本节初版即如此：150 组里 126 组给出上界 50.0。去掉 eps、并把边权写成 `mid*d − c` 后 174 组全对。**
- **负环的层数上界取 n−1。用虚拟源点时每个点的初始路径长度为 0，`cnt` 统计的是「被松弛的次数」，上界取 n 才等价于「路径含 n 条边」。取 n−1 会漏判长环。**

- **eps 型二分写成 `while (hi - lo > 1e-6)`。上界 U 可能到 10^9，此时要跑 50 次以上；而 U = 10^{18} 时浮点已经无法再二分（相邻 double 差 > eps），会死循环。固定迭代次数既能保证精度也能保证停机。**
- **浮点 DP 的「不可达」判据。用 `dp[j] > NEG/2` 判可达，别用 `dp[j] != NEG`：`NEG + x` 一旦被算进去就不再等于 NEG，会把不可达状态当成合法值传到答案里。**
- **答案要整数时直接 round。lo 与真值的差可达 1e-6 量级，恰好落在 .5 附近时 round 会错一位。稳妥写法是 `printf("%.0f\n", floor(lo + 1e-9))`（下取整）或 `ceil(lo - 1e-9)`（上取整），按题意选。**
- **权值到 10^9 仍用 double 累加。Σ(a−λb) 的项数到 10^5、单项到 10^{18} 时相对误差会吃掉 1e-9 的判定余量，改用 `long double`；或者干脆把答案写成 p/q 用整数二分 + 判定 `Σ(q·a − p·b) ≥ 0`（long long 溢出要 __int128）。**
- **把 WQS 二分与分数规划混为一谈。分数规划二分的是比值 λ，判定里权重随 λ 连续变化；WQS 二分（26.10）二分的是每段的附加税，用来把「恰好 k 段」松成「不限段数」。两者都长成「二分 + DP」，但二分对象与正确性依据完全不同。**

自查：本轮把三份模板逐行译成 Python 与暴力对拍——恰选 k 个 300 组（n ≤ 10、k 随机）与枚举 k 子集全等；最优比率生成树 200 组（n ≤ 6）与枚举生成树全等；最优比率环 174 组（n ≤ 5，含多连通块）与枚举简单环全等；最大密度子图的最小割建图 135 组与枚举点集全等。四处错误都在实测中复现过：无约束退化 400/400 错、最小生成树方向 200/200 错、负环加 eps 126/150 误判、上界取 Σa/min b 时输出 37.8（真值 7.2）。

24.10 整体二分：把所有询问的二分合成一棵值域分治树

一句话判定：三条同时成立才用整体二分——① 所有询问能一次读入（允许离线）；② 每个询问的答案落在一个可二分的值域上（「第 k 小」「最小的 x 使条件成立」）；③ 判定「答案是否 ≤ mid」能用一次全局扫描完成，且扫描过程中的修改可撤销。缺任何一条就不要套它：强制在线回到树套树（27.10）或主席树（27.8），判定不能全局共享则退回逐个二分。

为什么逐个二分是浪费。单个询问「区间 [l,r] 第 k 小」的标准解法是二分答案 v，判定「区间内 ≤ v 的数有几个」——判定要在整个数组上按值建结构，代价 O(n log n)。q 个询问各做一次，总代价 O(q · log V · n log n)，其中「按值把 n 个元素加进去」这件事被重复了 q · log V 次。而所有询问在同一个 mid 上的判定，需要加入的元素集合是完全相同的——就是「值 ≤ mid 的那些位置」。整体二分正是把这份公共工作摊开：建一棵值域分治树，每层只把元素加入/撤销一次。

分治结构：`solve(l, r, ql, qr)` 表示「事件区间 `e[ql..qr]` 中所有询问的答案都落在值域 `[l, r]` 内」。取 `mid = (l+r)/2`，把值 ≤ mid 的加点事件真正加进树状数组，然后逐个询问统计「区间内 ≤ mid 的个数 c」：`k ≤ c` 的询问答案在左半，`k > c` 的答案在右半且要把 k 减去 c（左半的 c 个元素已经确定排在它前面）。分完再把本层加进去的全部撤销，递归两侧。到 `l == r` 时该区间内所有询问的答案就是 l。

模板 A：静态区间第 k 小（n, q ≤ 10^5，值域 1..V）

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 100005, MAXQ = 100005, MAXE = MAXN + MAXQ;
int n, qn, V, bit[MAXN], ans[MAXQ];

struct Ev {
    int op;              // 0 = 加点(pos, val)，1 = 询问(l, r, k, id)
    int l, r, k, id;
} e[MAXE], buf[MAXE];
int side[MAXE];          // 本层每个事件的去向：0 左半，1 右半

void add(int i, int v) { for (; i <= n; i += i & -i) bit[i] += v; }
int sum(int i) { int s = 0; for (; i > 0; i -= i & -i) s += bit[i]; return s; }

void solve(int l, int r, int ql, int qr) {
    if (ql > qr) return;
    if (l == r) {                                   // 值域收敛，定答案
        for (int i = ql; i <= qr; ++i)
            if (e[i].op == 1) ans[e[i].id] = l;
        return;
    }
    int mid = (l + r) >> 1;
    for (int i = ql; i <= qr; ++i) {                // 第一遍：加点 + 统计 + 分侧
        Ev &x = e[i];
        if (x.op == 0) {
            if (x.k <= mid) { add(x.l, 1); side[i] = 0; }   // k 存的是值
            else side[i] = 1;
        } else {
            int c = sum(x.r) - sum(x.l - 1);
            if (x.k <= c) side[i] = 0;
            else { x.k -= c; side[i] = 1; }         // 右移必须减 c
        }
    }
    int p = ql, m1;                                 // 第二遍：按原序分装
    for (int i = ql; i <= qr; ++i) if (side[i] == 0) buf[p++] = e[i];
    m1 = p - 1;
    for (int i = ql; i <= qr; ++i) if (side[i] == 1) buf[p++] = e[i];
    for (int i = ql; i <= qr; ++i)                  // 第三遍：撤销本层加入
        if (e[i].op == 0 && side[i] == 0) add(e[i].l, -1);
    for (int i = ql; i <= qr; ++i) e[i] = buf[i];
    solve(l, mid, ql, m1);
    solve(mid + 1, r, m1 + 1, qr);
}

int main() {
    scanf("%d%d", &n, &qn);
    int tot = 0;
    for (int i = 1; i <= n; ++i) {
        int v; scanf("%d", &v);
        e[tot++] = {0, i, 0, v, -1};                // 加点事件
        V = max(V, v);
    }
    for (int i = 0; i < qn; ++i) {
        int l, r, k; scanf("%d%d%d", &l, &r, &k);
        e[tot++] = {1, l, r, k, i};
    }
    solve(1, V, 0, tot - 1);
    for (int i = 0; i < qn; ++i) printf("%d\n", ans[i]);
    return 0;
}
```

模板 B：带修区间第 k 小（单点修改 + 询问，n, m, q $\leqslant 10^5$）。与模板 A 只差三处：加点事件多一个增量 `d`；树状数组的加/撤销都带 `d`；一次修改拆成「删旧值」与「加新值」两条事件。事件数组按时间顺序排列，初始的 n 条加点排在最前，之后每条操作依次追加。

```
struct Ev {
    int op;              // 0 = 加点(l=位置, k=值, d=增量)，1 = 询问(l, r, k, id)
    int l, r, k, id, d;
} e[MAXE], buf[MAXE];

void solve(int l, int r, int ql, int qr) {
    if (ql > qr) return;
    if (l == r) {
        for (int i = ql; i <= qr; ++i)
            if (e[i].op == 1) ans[e[i].id] = l;
        return;
    }
    int mid = (l + r) >> 1;
    for (int i = ql; i <= qr; ++i) {
        Ev &x = e[i];
        if (x.op == 0) {
            if (x.k <= mid) { add(x.l, x.d); side[i] = 0; }
            else side[i] = 1;
        } else {
            int c = sum(x.r) - sum(x.l - 1);
            if (x.k <= c) side[i] = 0;
            else { x.k -= c; side[i] = 1; }
        }
    }
    int p = ql, m1;
    for (int i = ql; i <= qr; ++i) if (side[i] == 0) buf[p++] = e[i];
    m1 = p - 1;
    for (int i = ql; i <= qr; ++i) if (side[i] == 1) buf[p++] = e[i];
    for (int i = ql; i <= qr; ++i)
        if (e[i].op == 0 && side[i] == 0) add(e[i].l, -e[i].d);
    for (int i = ql; i <= qr; ++i) e[i] = buf[i];
    solve(l, mid, ql, m1);
    solve(mid + 1, r, m1 + 1, qr);
}

// 主程序：读入 a[1..n] 后
// for (i = 1..n)  e[tot++] = {0, i, 0, a[i], -1, +1};
// 修改 pos -> nv：
//   e[tot++] = {0, pos, 0, a[pos], -1, -1};   // 删旧
//   e[tot++] = {0, pos, 0, nv,     -1, +1};   // 加新
//   a[pos] = nv;
// 询问 l, r, k：
//   e[tot++] = {1, l, r, k, qid++, 0};
```

复杂度

做法	时间	空间与适用
逐个询问二分	$O(q \cdot \log V \cdot n \log n)$	$O(n)$，实现最直白，只在 q 很小时可用
整体二分（本节）	$O((n+q+E)\log V \log n)$	$O(n+q+E)$，E 为事件数（带修时 2 倍修改数），最省空间
主席树（27.8）	$O((n+q)\log n)$	$O(n \log n)$ 节点，静态最优，支持强制在线
树套树（27.10）	$O((n+m)\log^2 n)$	$n=m=10^5$ 时近 3×10^7 节点，带修且强制在线的唯一选择
莫队 + 值域分块	$O(n\sqrt{q})$	$O(n)$，询问多、无修改时更短，但难带修

同一套骨架的四种变体：

- 第 k 大：把「值 $\le$ mid」换成「值 $\ge$ mid」，或直接把值取负后求第 $(len-k+1)$ 小。别两处只改一处。
- 区间内前 k 小之和：把 BIT 从「计数」升级为「计数 + 和」两个数组，答案在分侧时累加「左半全部元素的和」——这是把可减量换成「可减量对」的标准手法。
- 答案值域不是权值而是下标：如「最小瓶颈路」二分的是边权阈值、「第 k 个满足 f(i) 的位置」二分的是下标，事件里的 `k` 字段含义随之改变，但分治结构不变。
- 判定本身是 DP/贪心：如「选不超过 k 个物品使代价 $\le$ mid」这类，判定不是树状数组而是 $O(n)$ 的 DP，整体二分照样成立——此时单次判定的代价 C 替代 $\log n$，复杂度写 $O((n+q)\log V \cdot C)$，只要 C 与 mid 无关就行。

易错点（前三条的错误率为本节实测，见文末自查）

- 右移时忘记 `k -= c`。左半已经确定有 c 个元素排在前面，不减就变成「在右半里找第 k 小」，结果整体偏大。400 组随机小数据错 117 组，且错的那 117 组里有相当一部分样例能过（当 $c=0$ 时两者等价）。
- 判据写成「值 $<$ mid」而不是「值 $\le$ mid」。等于 mid 的元素被推到右半，而右半的值域从 mid+1 开始，这些元素永远无法被计入——错得最彻底也最容易写错。400 组错 341 组。
- 带修时只加新值、不删旧值。位置上的旧值仍留在结构里，计数偏大，「第 k 小」被推小。300 组（940 个询问）错 105 组。正确写法是每次修改成对压入 −1 与 +1 两条事件。
- 撤销写成清空。`memset(bit, 0, sizeof bit)` 每层 $O(n)$，总代价多一个 $O(n \log V)$ 因子，$n=10^5$、$\log V=17$ 时是 1.7×10^6 次写——不一定 TLE，但会把整体二分「省时间」的优势吃掉一半。正确做法是只撤销本层真正加过的那些位置。
- 就地对半交换破坏时间序。若用双指针从两端往中间填 `buf`，每一侧的相对顺序会被打乱；静态版侥幸正确（元素加入顺序无所谓），带修版必错（修改与询问的先后被交换）。稳妥写法就是模板里的两遍填充。
- 漏掉 `l == r` 出口或出口不 return。`l == r` 时 `mid == l`，会无限递归；出口里必须把该区间内所有询问一次性定答案并返回。

- **值域上界取错。**`V` **取** `max(a[i])` **就够，取** 10^9 **只是多几层（约 30 层，代价可接受）；但若元素是** `long long` **且 V 取** 10^{18}**，层数变成 60，且** `(l + r)` **可能溢出——此时必须先离散化。**
- **k 的基准。题面「第 k 小」的 k 从 1 开始，若题目从 0 开始（少数题库如此），**`k <= c` **的判据会整体错位一位。**
- **多组数据不复位。**`bit` **与** `side` **是全局数组，多测之间必须重置** `bit` **到全零（递归结束时本就该是零，若上一组提前返回则残留）。**
- **递归传参写错。**`solve(l, mid, ql, m1)` **与** `solve(mid+1, r, m1+1, qr)` **——第二半的起点是** `m1+1`**，写成** `m1` **会让一个事件被两侧重复处理，答案随机偏移。**

自查：本节把模板 A、B 逐行译成 Python，与暴力（对子区间排序取第 k 个）对拍：静态 400 组（$n \le 12$**、值域 ≤ 10、每组 1—8 个询问）全对；带修 300 组（随机混合修改与询问，共 940 个询问）全对；边界用例「10 个元素全等于 7、**$V = 1$**」「单点区间」均通过。三条错法在同一批数据上的错误率分别是 117/400、341/400、105/300，数字已写进上面的易错点。选型建议只有一句：能离线就别上树套树——27.10 已算过** $n = m = 10^5$ **时树套树要 350 MB，而同一题的离线整体二分只要十几 MB。**

25 第 25 章　搜索

搜索是「没有更好办法时」的通用解法，也是暴力分的来源。本章给出 DFS/BFS 的标准骨架、四类剪枝、迭代加深与双向搜索、状压搜索的位运算技巧，以及 25.6 的 A* 与 IDA*——后者把「估价函数」讲成一门可以论证的手艺：可采纳与一致各保证什么、h 的三档强度怎么选、阈值迭代为什么必须取最小超限值；25.7 再把「折半搜索」补成独立一节，把 $n \leqslant 44$ 这一类「2^n 不可能、$2^{n/2}$ 刚好」的题拆开讲：两半怎么枚举、合并的三种写法（二分计数、相等段计数、邻近两点）、以及为什么「去重」在合并阶段是错的；25.8 收在舞蹈链（DLX）上——它把「每列恰好一个 1」的精确覆盖问题压成十字链表上的 O(1) 摘除与恢复，数独、n 皇后、骨牌铺砌共用同一份模板，配套的「最小列优先」启发式与「空列即死路」剪枝是能不能几千次递归跑完一盘数独的关键；25.9 转向两人对抗：零和博弈为什么必须用 minimax、NegaMax 为什么比交替写 max/min 少一半错、Alpha-Beta 的窗口语义与「被剪分支不影响根值」的不变式、胜负分为什么要扣步数才能快胜优先，以及置换表必须同时带「轮到谁」与「深度 + 边界类型」——最后落到一个在棋类上永远测不出来的 bug：带每步收益的博弈，子窗口要跟着收益平移。

25.1 DFS 与回溯的通用框架

DFS 解决的是「枚举所有合法方案」或「在状态图上走到终点」。写成三件事：边界、选择、撤销。

```
void dfs(int dep) {
    if (dep == n + 1) { /* 输出或更新答案 */ return; }
    for (int v : candidates(dep)) {
        if (!ok(v)) continue;      // 可行性剪枝
        choose(v);                 // 标记、累加
        dfs(dep + 1);
        unchoose(v);               // 回溯：必须与 choose 完全对称
    }
}
```

回溯的关键纪律：`choose` 与 `unchoose` 必须严格成对，且在所有返回路径上执行。常见错误是在 `continue` 前改了全局状态却没恢复。

四种常见搜索对象：排列（`used` 数组，$n!$ 级）、组合（按序号递增选，$C(n,m)$ 级）、子集（每个元素选或不选，2^n 级）、网格路径（四个方向 + 访问标记）。

什么时候 DFS 会爆：状态数超过 10^7、或存在大量重复状态。前者要剪枝，后者要记忆化（把 `dfs` 的返回值存进数组）。

25.2 BFS 与最短路

BFS 的本质是「在边权全为 1 的图上求最短路」，队列中的距离单调不降，因此第一次到达某点即为最短。

```
int bfs(int s, int t) {
    vector<int> dis(n + 1, -1);
    queue<int> q;
    dis[s] = 0; q.push(s);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        if (u == t) return dis[u];
        for (int v : g[u]) if (dis[v] == -1) {
            dis[v] = dis[u] + 1;
            q.push(v);
        }
    }
    return -1;
}
```

多源 BFS：把所有源点一次性入队、`dis=0`，求的是「到最近源点的距离」。网格上「每个格子到最近障碍/最近出口的距离」用它，一次 O(nm)。

0-1 BFS：边权只有 0 和 1 时用双端队列，0 边入队首、1 边入队尾，复杂度 O(V+E)，比 Dijkstra 少一个 log。

BFS 的两个陷阱：一是在入队时标记访问而不是出队时（否则同一点会被重复入队，队列爆炸）；二是状态里漏了维度（如「是否用过钥匙」），导致错误剪枝。

25.3 剪枝：把搜索空间砍到能跑

剪枝分四类，按收益排序：

类型	做法	典型题
可行性剪枝	当前状态已不可能到达合法解，直接返回	八皇后、数独
最优性剪枝	当前花费 + 理论下界 ≥ 已知最优，返回	埃及分数、骑士精神
搜索顺序优化	先搜分支少的、先搜更可能出解的方向	DLX、A*
记忆化 / 去重	同一状态只算一次，或哈希去重	状态图搜索

最优性剪枝要有「下界函数」：下界越紧，剪得越狠。`IDA*` 用「当前深度 + 启发式下界 > 限制深度」剪枝，本质是迭代加深 + A*。

```
// 最优性剪枝骨架
void dfs(int dep, int cost) {
    if (cost + lower_bound_rest(dep) >= best) return;   // 关键一行
    if (dep == n) { best = min(best, cost); return; }
    // ...
}
```

剪枝顺序：先做 O(1) 能判断的（可行性），再做需要计算的（下界），最后做启发式排序。不要在剪枝函数里写 O(n) 的循环，剪枝本身的开销会吃掉收益。

25.4 迭代加深与双向搜索

迭代加深（IDDFS）：当解的深度未知、且宽度极大（BFS 会爆内存）时，限定深度 d=1,2,3⋯ 反复 DFS。虽然重复搜索了浅层，但代价是几何级数，总代价与最后一次同阶。

```
for (int lim = 1; lim <= MAXD; ++lim) {
    if (dfs(0, lim)) break;      // dfs 内部：dep > lim 就返回
}
```

配合 IDA* 的下界函数，就是「埃及分数」「编辑书稿」这类题的标准解法。

双向 BFS：从起点和终点同时扩展，相遇即停。状态数从 b^d 降到 $2 \cdot b^{d/2}$。实现要点：两个 `unordered_map` 存距离，每次扩展队列较小的一侧。

折半搜索（meet in the middle）：$n \leqslant 40$ 的子集和、异或和问题，把 n 拆成两半各 $2^{n/2}$ 枚举，排序后双指针/二分合并。复杂度从 2^{40} 降到 2^{20}。

25.5 状态压缩搜索

当「哪些元素被选过」是状态的核心信息，且元素数 $\leqslant$ 20—22 时，用一个整数的二进制位表示整个状态。

```
// 状压 DP / 搜索的通用骨架
for (int mask = 0; mask < (1 << n); ++mask) {
    for (int i = 0; i < n; ++i) {
        if (mask >> i & 1) continue;       // 已选过
        int nxt = mask | (1 << i);
        f[nxt] = min(f[nxt], f[mask] + cost(mask, i));
    }
}
```

三个必备位运算：取最低位 1 `mask & -mask`；删掉最低位 1 `mask & (mask-1)`；枚举子集 `for (s = mask; s; s = (s-1) & mask)`。

状态设计的常见维度：已访问集合、当前位置、剩余次数、颜色/奇偶。维度爆炸时要么换状态（用「最后一个位置」代替集合），要么换算法（区间 DP）。

可行规模：$2^{20} \approx 10^6$ 可跑，$2^{24} \approx 1.6 \times 10^7$ 需压常数，2^{30} 只能折半搜索。

25.6 A* 与 IDA*：估价函数怎么选、怎么证

一句话：A* 就是把 BFS 的队列换成「按 f = g + h 排序的优先队列」；IDA* 就是把「优先队列」换成「带阈值限制的 DFS」。两者的全部技术含量都在估价函数 h 上：h 给得越准，展开的节点越少，但 h 必须不高估，否则第一个弹出来的答案就不是最优解。

结论一（可采纳与一致）：设 h(u) 是「从 u 到目标的最小代价」的下界估计。

- **可采纳（admissible）：对任意状态 u 有 h(u) ≤ d(u)。它保证 A* 第一次弹出目标时 `g` 就是最优值——因为队列里任何未展开的条目都有 f ≥ 最优值，目标一旦以 f = 最优值弹出，就没有更小的可能了。**
- **一致（consistent，也称单调）：对任意边 u→v 有 h(u) ≤ w(u,v) + h(v)。一致蕴含可采纳，并且保证「一个状态第一次被弹出时 g 已最优」，于是每个状态只需展开一次，不必重开已关闭的节点。**

八数码的曼哈顿距离两条都满足：一次移动只改变一个棋子的坐标，h 的变化量恰为 ±1，正好等于边权 1，所以 |h(u) − h(v)| ≤ 1 = w。

结论二（IDA* 的阈值迭代）：IDA* 不存任何状态表，空间只有 O(深度)，因此能处理「状态数大到开不出哈希表」的题（15 数码、骑士精神）。每一轮 DFS 只走 `dep + h ≤ bound` 的分支，同时记录「被剪掉的最小 f」，把它作为下一轮的 bound。这里有一个必须记住的细节：下一轮 bound 要取那个最小超限值，不能写 `bound + 1`。写成 +1 时阈值要一步步爬 d 次才够到答案，等于把迭代加深重做一遍，节点数会涨一到两个数量级。

方案	时间	空间	适用
BFS	O(V)	O(V)	状态数 ≤ 10^6，要最短步数
A*	O(V log V)，实际由 h 强度决定	O(V)（要存最优 g 表）	状态多但 h 很强，且能存表
IDA*	$O(b^d)$ 节点，重复搜索不可避免	O(d)	状态表存不下，答案深度不大（≤ 20）

模板 A：A* 解八数码。用 9 进制编码（$9^9 \approx 3.87\times10^8$，落在 `int` 内），`best` 表存「到达该状态的最优 g」，弹出时用 `cur.g > best[code]` 过滤陈旧条目（惰性删除）。

```
#include <bits/stdc++.h>
using namespace std;
const int P9[9] = {1,9,81,729,6561,59049,531441,4782969,43046721};

int hman(const int *s) {                    // 曼哈顿距离：可采纳且一致
    int r = 0;
    for (int i = 0; i < 9; ++i) {
        if (!s[i]) continue;                // 空格不计
        int g = s[i] - 1;                   // 该棋子的目标下标
        r += abs(i / 3 - g / 3) + abs(i % 3 - g % 3);
    }
    return r;
}
int enc(const int *s) { int r = 0; for (int i = 0; i < 9; ++i) r += s[i] * P9[i]; return r; }

struct Node {
    int f, g, s[9];
    bool operator<(const Node &o) const { return f > o.f; }   // 小根堆
};

int astar(const int *st) {
    unordered_map<int,int> best;            // 编码 -> 目前最优 g
    best.reserve(1 << 16);
    priority_queue<Node> pq;
    Node s0; memcpy(s0.s, st, sizeof(s0.s));
    s0.g = 0; s0.f = hman(s0.s);
    best[enc(s0.s)] = 0; pq.push(s0);

    while (!pq.empty()) {
        Node cur = pq.top(); pq.pop();
        int code = enc(cur.s);
        auto it = best.find(code);
        if (it != best.end() && cur.g > it->second) continue;   // 陈旧条目，丢掉
        int hh = hman(cur.s);
        if (hh == 0) return cur.g;          // 曼哈顿为 0 当且仅当已复原
        int z = 0; while (cur.s[z]) ++z;    // 找空格
        int r = z / 3, c = z % 3;
        static const int dr[4] = {1,-1,0,0}, dc[4] = {0,0,1,-1};
        for (int d = 0; d < 4; ++d) {
            int nr = r + dr[d], nc = c + dc[d];
            if (nr < 0 || nr > 2 || nc < 0 || nc > 2) continue;
            Node nx = cur;
            swap(nx.s[z], nx.s[nr * 3 + nc]);
            nx.g = cur.g + 1;
            int nc2 = enc(nx.s);
            auto jt = best.find(nc2);
            if (jt == best.end() || nx.g < jt->second) {   // 只有更优才重入队
                best[nc2] = nx.g;
                nx.f = nx.g + hman(nx.s);
                pq.push(nx);
            }
        }
    }
    return -1;                              // 不可达（应先按逆序对判掉）
}
```

三处关键：① 用 9 进制编码而不是字符串，常数降一个量级；② `best` 存的是「最优 g」而不是「访问过」，同一状态可以从不同路径到达且 g 不同；③ 弹出时再比一次 `cur.g > best[code]`，这句看似多余，省掉它会让同一状态被重复展开，节点数翻几倍。

模板 B：IDA* 通用骨架。返回值约定：找到解返回 true 并写答案；超限返回 false 并把最小超限 f 写进 `nxt`；`nxt` 保持 INF 说明整个状态空间走完仍无解。

```
int bound, answer;
bool found;

// 返回 true 表示本分支找到解
bool dfs(int *s, int dep, int prevMove, int bound, int &nxt) {
    int hh = h(s);
    if (hh == 0) { answer = dep; return true; }
    if (dep + hh > bound) { nxt = min(nxt, dep + hh); return false; }   // 记最小超限值
    for (每一合法走法 mv) {
        if (mv == rev(prevMove)) continue;    // 不许原地回退，分支数从 b 降到 b-1
        apply(s, mv);
        if (dfs(s, dep + 1, mv, bound, nxt)) return true;
        undo(s, mv);
    }
    return false;
}

int ida(int *st) {
    bound = h(st);
    while (true) {
        int nxt = INT_MAX;                    // 本轮最小超限 f
        if (dfs(st, 0, -1, bound, nxt)) return answer;
        if (nxt == INT_MAX) return -1;        // 无解
        bound = nxt;                          // 不是 bound + 1
    }
}
```

骨架能直接套到 15 数码、骑士精神、编辑书稿（UVa 11212）这类题上，只需换 `h` 与状态表示。要注意的是 h 的强度在这里是决定性的：h 太弱（比如只数「不在位的棋子数」）时，深度 15 的盘面在时限内根本跑不完；而把 h 做强又必须保证不高估，这是一对矛盾，也是这类题真正难的地方。

h 的三档强度（以八数码为例）：

1. **h = 0：A* 退化成 Dijkstra，IDA* 退化成迭代加深。只用来验证程序正确性，不要交。**
2. **曼哈顿距离：标准选择。`h ≤ 真实距离` 显然成立（每个棋子至少要走这么多步），一致性也成立。**
3. **曼哈顿 + 线性冲突：同一行（或同一列）上有两个棋子，它们的目标位置也在这行且顺序颠倒，则两者至少有一个要绕行，额外至少 +2 步。这一项能把 h 提高接近一倍，节点数常降一个数量级，是八数码/15 数码的标准加强。**

易错点

- **h 高估导致答案非最优：最典型的是把「不在位的棋子数」当 h 直接用（一步只能让一个棋子归位，这个值确实可采纳，但比对曼哈顿弱得多），或者给曼哈顿乘了个「乐观系数」——一乘就不再可采纳，A* 会给出比最优大的答案，而且看上去还挺像对的。**
- **判重只记「访问过」而不记最优 g：同一状态可能以不同步数到达，只记布尔值会让更优路径被永久丢弃。**
- **忘记惰性删除：优先队列里必然存在同一状态的多份条目（每次改善 g 都会 push 一份），不判 `cur.g > best[code]` 就会重复展开。**

- **IDA* 的 bound 写成 bound + 1**：这是最常见的性能事故，正确性没问题，但会慢一到两个数量级，表现为「本地能过、评测机 TLE」。
- **IDA* 不剪「立即回退」**：不传 prevMove，每层都能走回上一步，分支数从 b−1 回到 b，深度 15 时差 $(8/7)^{15} \approx 8$ 倍以上。
- **找到解后继续搜索**：DFS 里必须逐层立刻 return true，否则后续分支会把 answer 覆盖成更大的值。
- **不先判无解**：八数码先按「去掉 0 后的逆序对奇偶」判可达（奇数则无解）；A* 在这种输入上会把整个连通分量搜完才返回 −1。
- **估价函数写成整盘扫描**：h 每次 O(n) 扫描在八数码（n = 9）无所谓，但 15 数码（n = 16）或更大盘面上，把 h 改成增量维护（一次移动只改两个棋子的贡献）能再快一倍。
- **状态编码溢出**：八数码用 base-9 刚好 3.87×10^8，塞得进 `int`；15 数码的排列数 $16! \approx 2.1\times10^{13}$，必须用康托展开 + `long long`，或者干脆用 IDA* 不编码。
- **拿 mism − 1 当骑士精神的 h 却不验证可采纳性**：这是流传最广的写法，但严格说它并不总是可采纳——一次移动可能同时修正两格（把空格与中心位的马互换），此时它高估了。安全的可采纳写法是 `ceil(mism / 2)`（每步最多改两格），代价是 h 更弱、搜得更慢。用 mism − 1 要么自己补证明，要么接受「可能非最优」的风险。

25.7 折半搜索（meet in the middle）：两半枚举怎么合并才不重不漏

一句话判定：`n ≤ 40`（最多到 44），2^n 枚举不可能，但 $2^{n/2} \le 4\times10^6$ 完全跑得动，并且答案能写成「左半的一个选择 + 右半的一个选择」的可合并形式——这就是折半搜索。它不是一种新算法，而是把「枚举全部子集」换成「枚举两个半区、再按某种规则配对」，用 $O(2^{n/2})$ 的空间换掉指数上的那个 2。25.4 只用两行提到它，本节把那两行补成可编译的模板、三种合并方式与实测出来的坑。

核心结论：把 `a[1..n]` 切成 `L = a[1..n/2]` 与 `R = a[n/2+1..n]`，各自枚举全部 $2^{n/2}$ 个子集求和，得到两个多重集 `A`、`B`。任何「整集」的某个量都能拆成 `A` 与 `B` 上对应量的组合，于是原问题的规模从 2^n 降为 $2\cdot2^{n/2}$ 加一次合并。合并的正确性完全取决于一件事：「A 中的一个值 + B 中的一个值」必须和原问题的选择一一对应。枚举两半时各自独立、不重不漏，乘积自然不重不漏——所以千万不要在合并阶段再去重（那是把不同的方案误杀），要去的只是「同一个和值的方案数」。

问题形态	合并方式	复杂度
子集和 ≤ T 的方案数 / 能否达到	`B` 排序，对每个 `x∈A` 二分 `upper_bound(T-x)`	$O(2^{n/2}\ \log\ 2^{n/2})$
子集和恰为 T 的方案数（含重复值）	`B` 排序，`equal_range` 取相等段长度并累加	同上
子集和与 T 最接近（最小绝对差）	`B` 排序，`lower_bound(T-x)` 只看 `p` 与 `p-1` 两个位置	同上
两两配对类（四数和为 0、异或和为 0）	一半塞哈希表 / 排序，另一半查表	$O(2^{n/2})$ 期望
「选出的个数也有约束」	按个数分桶后再合并，桶内仍排序	$O(2^{n/2}\ \log)$，常数翻倍

下面这份模板求「子集和不超过 T 的子集个数」（含空集），读入 `n T` 与 n 个整数，输出方案数，可直接编译运行。生成子集和用 lowbit 递推而不是 DFS：`sum[m] = sum[m 去掉最低位] + a[最低位]`，一趟 $O(2^k)$ 无递归、无栈、缓存友好。

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;

typedef long long ll;
const int MAXN = 45;

int n;
ll a[MAXN], T;
ll A[1 << 22], B[1 << 22];      // 两半的子集和：n ≤ 44 时各需 2^22 个
int ca, cb;

void gen(int l, int r, ll *dst, int &cnt) {
    int k = r - l;
    dst[0] = 0;
    for (int m = 1; m < (1 << k); ++m) {
        int low = m & -m;                  // 最低位的 1
        int i   = __builtin_ctz(low);      // 它是第几位（0 起）
        dst[m]  = dst[m ^ low] + a[l + i]; // m ^ low 一定小于 m，已算过
    }
    cnt = 1 << k;
}

int main() {
    scanf("%d%lld", &n, &T);
    for (int i = 0; i < n; ++i) scanf("%lld", &a[i]);
    gen(0, n / 2, A, ca);
    gen(n / 2, n, B, cb);
    sort(B, B + cb);                       // 只排右半，左半直接扫
    ll ans = 0;                            // 方案数最多 2^40，必须 long long
    for (int i = 0; i < ca; ++i) {
        // upper_bound 返回第一个 > T - A[i] 的位置，下标即「≤ 的个数」
        ans += upper_bound(B, B + cb, T - A[i]) - B;
    }
    printf("%lld\n", ans);
    return 0;
}
```

三种常用变形（把主循环里的那一行换掉即可）：

```cpp
// ① 和恰为 T 的方案数：相等段整段计数，不能只 +1
for (int i = 0; i < ca; ++i) {
    ll need = T - A[i];
    ans += upper_bound(B, B + cb, need) - lower_bound(B, B + cb, need);
}

// ② 与 T 最接近：只看插入位置 p 与 p-1，两个候选
ll best = (1LL << 62);
for (int i = 0; i < ca; ++i) {
    ll need = T - A[i];
    int p = lower_bound(B, B + cb, need) - B;
    if (p < cb) best = min(best, llabs(A[i] + B[p] - T));
    if (p > 0)  best = min(best, llabs(A[i] + B[p - 1] - T));
}

// ③ 还要约束「选几个」：把值按 popcount 拆桶，同桶内排序后合并
//    bucket[c] 存「选了 c 个」的那些和；左半取 c1、右半取 c2，要求 c1 + c2 = k
```

还能怎么扩：折半不止用于子集和。① 超大容量 0/1 背包（`n ≤ 40`、`W ≤ 10^18`）就是「价值和最大且重量 ≤ W」，把两半的（重量，价值）按重量排序后做前缀最大值，再对每个左半状态二分——这是折半 + 单调栈的组合；② 网格/棋盘类「走一半、从终点倒着走一半、在中间相遇」是同一思想的空间版（双向 BFS 的变体，见 25.4）；③ 求第 k 小的子集和用「二分答案 + 折半计数 ≤ mid」把判定外包给本节模板，复杂度多一个 `log`。

易错点：

- **用 `int` 存和。`a_i ≤ 10^9`、`n = 40` 时单侧和可达 `2×10^10`，整集 `4×10^10`，`int` 直接溢出成负数，二分结果全乱。同理答案（方案数）在 `n = 40` 时最大 `2^40 ≈ 1.1×10^12`，也必须 `long long`。**
- **相等段只 +1。把「和恰为 T」写成双指针、遇到相等就 `ans++` 再 `i++`，看起来很像标准双指针，实际漏掉了「同一个和值对应多组子集」这件事：300 组随机重复值数据里错了 234 组。要么用 `equal_range` 整段计数，要么双指针遇到相等段时把两段的长度相乘。**
- **把「去重」用在合并阶段。两个不同的子集可能和相同，它们是两种方案，不是重复。排序只用于二分，不用于去重；用 `unique` 之后再计数会把方案数压小。**
- **空集算不算。模板里 `dst[0] = 0` 就是空集，所以 `T ≥ 0` 时它一定被计入。题目若要求「至少选一个」，合并时要单独扣掉 `(0,0)` 这一对；「非空子集个数」这类问法出错多半就栽在这里。**
- **用 `1 << k` 却让 `k = 31`。`1 << 31` 在 C++ 里是 `int` 的符号位，结果是大负数。虽然 `n ≤ 44` 时 `k ≤ 22` 不会触发，但把模板改去解「`n = 62`、按位折半」的题就会中招；统一写 `1LL << k` 或 `1u << k`。**
- **数组按 `2^⌈n/2⌉-1` 开。左半是 `0..n/2-1` 共 `⌊n/2⌋` 个、右半是 `⌈n/2⌉` 个，两半不等长，数组要按较大的一半开。`n` 为奇数时按 `⌊n/2⌋` 开会少一半。**
- **内存估算漏了排序的开销。`n = 44` 时每个数组 `2^22×8 B = 33.5 MB`，两个就 67 MB，加上 `sort` 的栈与临时空间，256 MB 限制下已经很紧；`n = 46`（`2^23`）直接翻倍到 134 MB，必须减半枚举或改用其他做法。开数组前先算 `2^⌈n/2⌉×8×2` 这个数。**
- **枚举两半时用了递归 DFS。`2^22` 次函数调用在 `n = 44` 时约 400 万次，比 lowbit 递推慢 3—5 倍，且 DFS 里每次都要维护当前和与深度，常数更差。lowbit 递推是纯数组写入，实测差距明显。**
- **对左半也排序。只有当需要「双指针同时推进」时才排左半；单侧二分的写法排左半纯属浪费 `O(2^n/2 log)`。反过来，若改成双指针，两半都必须排序，且指针推进方向要与排序方向一致，否则会漏解。**
- **把折半当万能。折半要求「两半独立」。一旦有跨半的约束（比如「选出的数必须构成连通块」「相邻不能同时选」的链式依赖），两半的枚举结果就不能简单配对，得换成轮廓线 DP 或状压 DP；强行折半只会得到一个错得很自然的答案。**

自查：把上面三段模板逐行译成 Python（同样的 lowbit 递推、同样的排序与二分），与「`2^n` 全枚举」暴力对拍：求和 ≤ T 计数 300 组（`n ≤ 18`、值域含负数）全对；和恰为 T 的方案数 300 组（值域 −5..5，重复值密集）全对；最接近 T 300 组全对；全负数配极小 T（二分结果恒落在下标 0）100 组全对；把值域放大到 `10^9`、`T` 到 `4×10^9` 的 60 组也全对。上面第 2 条错误的版本实际跑过：双指针遇相等只 `+1` 时 300 组错 234 组。本机没有 `g++`，C++ 侧只做了语法与上界自查（数组按 `2^22` 开、`long long` 贯穿、`upper_bound` 返回指针相减）。

25.8 舞蹈链（DLX）：精确覆盖与重复覆盖

问题：给定一个 0/1 矩阵，选出若干行，使每一列恰好被一个选中行覆盖——这叫精确覆盖（Exact Cover）。判定条件只有一条：选中行的列集两两不交，并集恰好是全体列。数独、n 皇后、骨牌铺砌、靶形数独、破坏正方形、雷达选址（每行最多放一个）都是它的马甲，难点不在搜索本身，而在把题目翻译成一张 0/1 矩阵。

题目	行是什么	列是什么
9×9 数独	每个「(格子, 数字)」合法填法：9×9×9 = 729 行	324 列 = 81 格 + 81（行,数字）+ 81（列,数字）+ 81（宫,数字）
n 皇后	每个格放一个皇后：n^2 行	n 列（每列一个）+ n 行（每行一个）+ 2(2n−1) 条对角线（每条至多一个）
骨牌铺砌	骨牌的每种摆法（横/竖各一种位置）	每个待覆盖格子一列
雷达站/巡逻	把某点放在某个候选位置	「每个点必须被覆盖」+「每个位置至多一个」

为什么叫舞蹈链：把矩阵的每个 1 看成一个节点，同一列上下串成环形链表、同一行左右串成环形链表，再给每列加一个列头哨兵、所有列头串成一圈总表头（编号 0）。递归时「选一列 → 删掉该列 → 对每个候选行删掉它覆盖的其余列」，回溯时严格逆序恢复。摘除与恢复都是 O(1) 指针操作，所以单步代价只和「被删掉的节点数」成正比，而不是和矩阵大小成正比。

复杂度：最坏是指数的（精确覆盖本身是 NP 完全问题），但选 size 最小的列这个启发式把搜索树压得极扁——它的含义是「先处理选择最少的约束」，等价于约束传播里的 fail-first。实测数独（324 列、729 行）从空盘到满盘只需几千次递归。工程上记 `O(搜索树节点数 × 行平均长度)` 即可，不要试图写大 O。

模板（精确覆盖，可编译）：

```
const int MAXN = 400000;          // 节点数 = 1 + 列数 + 矩阵中 1 的个数
int L[MAXN], R[MAXN], U[MAXN], D[MAXN];
int colOf[MAXN], rowOf[MAXN], sz[MAXN];
int tot;                          // 节点计数，0 号是总表头

void init(int ncol) {             // 列编号 1..ncol，0 号为总表头
    tot = ncol;
    for (int i = 0; i <= ncol; ++i) {
        L[i] = i - 1; R[i] = i + 1;
        U[i] = D[i] = i;          // 空列：自己指向自己
        colOf[i] = i; sz[i] = 0;
    }
    L[0] = ncol; R[ncol] = 0;     // 列头串成环
}

// 插入一行：r 是行号（仅用于回溯时还原答案），cs 是该行覆盖的列（1-based）
void addRow(int r, const vector<int> &cs) {
    int first = -1;
    for (int c : cs) {
        int nd = ++tot;
        colOf[nd] = c; rowOf[nd] = r; ++sz[c];
        // 竖向：插到第 c 列列尾（U[c] 是原尾部）
        U[nd] = U[c]; D[nd] = c;
        D[U[c]] = nd; U[c] = nd;
        // 横向：插到本行第一个节点之前，形成环
        if (first < 0) { first = nd; L[nd] = R[nd] = nd; }
        else {
            L[nd] = L[first]; R[nd] = first;
            R[L[first]] = nd; L[first] = nd;
        }
    }
}

void remove(int c) {                      // 摘掉第 c 列
    L[R[c]] = L[c]; R[L[c]] = R[c];       // 先摘列头
    for (int i = D[c]; i != c; i = D[i])  // 该列每个 1 所在的行
        for (int j = R[i]; j != i; j = R[j]) {
            U[D[j]] = U[j]; D[U[j]] = D[j];
            --sz[colOf[j]];               // 别忘维护列大小
        }
}

void resume(int c) {                      // 严格逆序恢复
    for (int i = U[c]; i != c; i = U[i])
        for (int j = L[i]; j != i; j = L[j]) {
            U[D[j]] = j; D[U[j]] = j;
            ++sz[colOf[j]];
        }
    L[R[c]] = c; R[L[c]] = c;
}

vector<int> chosen;
bool dance() {                            // 只求第一组解
    if (R[0] == 0) return true;           // 所有列都恰好覆盖
    int c = R[0];
    for (int i = R[0]; i != 0; i = R[i])  // 启发式：选 size 最小的列
        if (sz[i] < sz[c]) c = i;
    if (sz[c] == 0) return false;         // 空列 = 死路，必须提前剪
    remove(c);
    for (int i = D[c]; i != c; i = D[i]) {
        chosen.push_back(rowOf[i]);
        for (int j = R[i]; j != i; j = R[j]) remove(colOf[j]);
```

```
        if (dance()) return true;
        for (int j = L[i]; j != i; j = L[j]) resume(colOf[j]);
        chosen.pop_back();
    }
    resume(c);
    return false;
}
```

变体一：数出全部解——把 `if (dance()) return true;` 换成「递归后不返回、只在 `R[0]==0` 时 `++ans`」，其余不动。因为每层只沿 `R[0]` 往右选列，同一个行集不会被两种顺序重复枚举，答案天然不重。

数独的建表公式（324 列、729 行；空格的 9 行大于列的编号规则完全一样）：

```
// 对第 r 行第 c 列填数字 v（r,c,v 均为 0-based 或 1-based，务必统一）
int colCell = r * 9 + c;                          // 0..80    这格已填
int colRow  = 81  + r * 9 + v;                    // 81..161  第 r 行已用 v
int colCol  = 162 + c * 9 + v;                    // 162..242 第 c 列已用 v
int colBox  = 243 + (r / 3 * 3 + c / 3) * 9 + v;  // 243..323 该宫已用 v
// 给定格只插入 v 这一行；空格插入 v = 0..8 共 9 行
addRow(rid, {colCell + 1, colRow + 1, colCol + 1, colBox + 1});
```

变体二：重复覆盖（求最少行数）。列只需「至少一个 1」时，删除只摘列头、不摘行（否则会把还能用的行一起删掉），并用 IDA* 加拟阵式估价：反复取一列，把该列所有行能覆盖的列标记掉，标记次数就是下界 `h`。

```
bool vis[512];

void del(int c) { L[R[c]] = L[c]; R[L[c]] = R[c]; }   // 只摘头
void add(int c) { L[R[c]] = c; R[L[c]] = c; }

int h() {                                  // 贪心下界
    int ret = 0;
    memset(vis, 0, sizeof vis);
    for (int c = R[0]; c != 0; c = R[c]) if (!vis[c]) {
        ++ret; vis[c] = true;
        for (int i = D[c]; i != c; i = D[i])
            for (int j = R[i]; j != i; j = R[j]) vis[colOf[j]] = true;
    }
    return ret;
}

bool ida(int dep, int limit) {             // dep + h() > limit 即剪
    if (R[0] == 0) return true;
    if (dep + h() > limit) return false;
    int c = R[0];
    for (int i = D[c]; i != c; i = D[i]) {  // 枚举覆盖该列的行
        del(c);
        for (int j = R[i]; j != i; j = R[j]) del(colOf[j]);
        if (ida(dep + 1, limit)) return true;
        for (int j = L[i]; j != i; j = L[j]) add(colOf[j]);
        add(c);
    }
    return false;
}
```

易错点：

- 恢复顺序写成「与删除同序」。这是本节最致命的一处：`remove` 是先摘列头、再逐个摘掉该列各行在其他列里的节点，恢复必须严格反着来（先逐行、再列头）。改成同序恢复后链表会被接错，递归不终止——实测 400 组随机小矩阵里第一组就爆栈（Python 复现版直接 `RecursionError: maximum recursion depth exceeded`，跑到 1000 层被强制打断），而不是「答案偏一点」这种能靠对拍发现的错。
- 选了 size 为 0 的列。某列一个 1 都没有时 `sz[c] = 0`，启发式反而最喜欢它；`remove` 之后 `R[0] == 0` 成立，于是「无解」被误判成「找到解」，答案里出现一组解不完的覆盖。`dance` 里那句 `if (sz[c] == 0) return false;` 不是保险，是必需的剪枝。
- 忘了维护 `sz`。`remove` 里对每个被摘的节点都要 `--sz[colOf[j]]`，`resume` 里要 `++`。只摘指针不更新计数，几层之后 `sz` 与实际列长脱节，启发式退化（选到长列）甚至出现负值。
- 精确覆盖里「只摘列头」。精确覆盖必须把该列所有行在其他列中的节点一并摘掉（否则同一行可以被选两次，同一列被覆盖两次也查不出来）；反过来在重复覆盖里「连行一起摘」会把本可复用的行删掉，得到比最优大的答案。
- 单节点行的环形指针没指向自己。`L[nd] = R[nd] = nd` 是必须的；写成 0 或 `-1` 后，`for (j = R[i]; j != i; j = R[j])` 会沿着总表头跑飞，症状是莫名其妙地删掉别的列。
- 数独的宫编号写错。`(r / 3 * 3 + c / 3)` 是 0..8；写成 `r / 3 * 9 + c / 3` 会得到 0..20，直接越界到行/列约束区，表现为「无解」或「解出重复数字」。
- 给定格也开了 9 行。给定格的候选只有它本身那一个值；开成 9 行等于允许改题，能解出与题面矛盾的盘面。
- 坐标 0-based 与 1-based 混用。DLX 的列号从 1 开始（0 是总表头），而行列式建模里的 `r*9+c` 从 0 开始，`addRow` 时统一 `+1`。混用的症状是「有一整列永远空着」，于是全局无解。
- 数组开小。节点数 = 1 + 列数 + 1 的个数：数独是 1 + 324 + 729×4 = 3241，开 4000 就够；但骨牌铺砌 `n×m ≤ 100` 时 1 的个数可到 4×10^4，n 皇后 n = 15 时行数 225、1 的个数约 10^4。按「列数 + 行数 × 每行 1 的个数」估，宁大勿小。
- 把 DLX 当万能搜索。它只认「每列恰好/至少一个 1」这种集合覆盖结构。一旦约束是「相邻不能同时选」「路径要连通」「方案有顺序」，先构造出 0/1 矩阵这一步就做不干净，硬套只会写出一堆隐式约束漏洞；这类题该回 25.1 的普通回溯或状压 DP。

自查：把上面的模板逐行译成 Python（同样的列头、同样的 `sz` 维护、同样的最小列启发式），与「2^m 枚举行子集 + 逐列校验恰好一次」的暴力对拍：随机矩阵 400 组（列 1—6、行 1—11、密度 0.45）解数完全一致，0 组不符，且每组解都另做了「每列恰一次」的独立校验；17 提示的数独用 DLX 求解与「最小候选优先的回溯」求解结果逐格相同（首行 6 9 3 7 8 4 5 1 2）。错误写法的数据也实测过：把恢复改成同序，第一组随机矩阵即无限递归爆栈。本机无 `g++`，C++ 侧只做了语法与上界自查（节点数按 4000 / 4×10^4 两档估算，递归深度等于选中行数）。

25.9 对抗搜索：minimax、Alpha-Beta 剪枝与置换表

什么时候该用搜索而不是博弈论。29.6 讲的 SG 函数只对「局面可以拆成互不干扰的独立子游戏、且胜负只与奇偶有关」的题有效（Nim、取石子、阶梯博弈）。一旦规则里出现棋盘形状（五子棋连成五子、黑白棋翻转）、资

源约束（每步得分不同、有时间上限）或双方信息不对称，局面就拆不出 SG 值，只能老老实实把博弈树搜出来。这一节给的是这条路的标准骨架：NegaMax 写法、Alpha-Beta 剪枝、置换表、迭代加深，以及一个几乎所有人第一次写都会错的地方——带每步收益的博弈，窗口必须跟着收益平移。

结论一：零和、完全信息、确定性、轮流走，四个条件同时成立才能用 minimax。设 `V(s)` 为「轮到走的一方在局面 `s` 上能取得的最优分差」，则

```
V(s) = 0                                  若 s 是平局终局
V(s) = +(WIN - ply)                       若 s 是「轮到走的一方」获胜的终局
V(s) = -(WIN - ply)                       若 s 是对方获胜的终局
V(s) = max over 走法 m of ( gain(m) - V(s') )   其余情况，s' 是走完 m 之后的局面
```

这里 `gain(m)` 是这一步立刻到手的分数（吃掉的子、拿走的石子），没有即时收益的棋类取 `gain = 0`，公式退化成 `V(s) = max(-V(s'))`，这就是 NegaMax：全程只取 max，靠「返回时取负」把对手的视角翻过来。比交替写 max/min 的经典 minimax 少一半出错机会——minimax 里最容易犯的错就是某一层忘了取 min，而 NegaMax 结构上不可能犯。

为什么胜负分要减 `ply`：如果所有获胜终局都给同一个分 `WIN`，那么「三步赢」和「一步赢」在搜索眼里一样好，程序会挑先被枚举到的那个，于是出现「明明能一步将死却绕了五个回合」的棋。给胜利扣掉自己的步数（`WIN - ply`）、给失败加上步数，等价于「快胜优于慢胜、慢负优于快负」。实测这一条不是洁癖：在井字棋 3673 个合法局面里，用常数胜负分会与带深度修正的版本选出不同的着法共 320 个（8.7%）；例如 X 占 2、8 号位、O 占 6、7 号位、轮到 X 时，5 号位直接成三（2-5-8），但常数分版本选了 0 号位——因为它也算「必胜」，只是要多走几步。

结论二：Alpha-Beta 剪的是「不影响根值」的分支，剪完根值不变。递归时带一个窗口 `(alpha, beta)`：`alpha` 是「我已经确保能拿到的最好值」（下界），`beta` 是「对手不会让我超过的值」（上界）。一旦 `best >= beta`，说明这个分支对上一层已经没用了，直接返回。正确性来自一句不变式：被剪掉的分支所在的子树，其值落在窗口之外，取 max 时不可能成为新的最优点。

走法顺序	需要访问的节点数	说明
完全随机	约 `b^(3d/4)`	理论期望；实际棋类大致落在这个量级
最好走法永远排第一	`b^(d/2)`	上界，等价于「有效分支因子变成 √b」
最坏（最差走法排第一）	`b^d`	与不剪枝一样，说明排序决定成败

`b` 是每层平均走法数、`d` 是搜索深度。这张表解释了为什么真正的棋类程序一半代码在走法排序：吃子与威胁走法优先、历史启发（某个走法在别的分支里曾剪出好结果就加权）、杀手着法（同一层里刚刚造成剪枝的非吃子走法）。Alpha-Beta 的收益完全来自「先试最可能好的走法」，而不是剪枝规则本身有多聪明。

结论三：置换表存的是「值 + 深度 + 边界类型」，三者缺一不可。同一个局面可能在博弈树里从不同路径到达（换位、重复局面），表能省掉重复搜索。但节点被搜出来时可能是在一个窄窗口下「剪枝返回」的，那时拿到的只是界而不是精确值：

- `EXACT`：节点在完整窗口下搜完，值就是真值，可以直接用；
- `LOWER`：因为 `best >= beta` 而提前返回，真值 ≥ 存下的值，只能用来抬高 `alpha`；

- `UPPER`：因为整层都没超过 `alpha` 而返回，真值 ≤ 存下的值，只能用来压低 `beta`。

键必须包含「轮到谁走」——在走子类游戏（黑白棋、象棋）里同一个盘面可能轮到任意一方，只按盘面做键会把两个不同的博弈节点混成一个。另外条目要带深度：浅层搜索得到的值不能拿来替代深层搜索（迭代加深时尤其明显，表里全是上一步的浅层值）。

结论四：带每步收益的博弈，子窗口要平移 `gain`。这是本节最容易错、且最难自查的一处。当 `V(s) = max(gain(m) - V(s'))` 而 `gain` 随走法变化时，父节点关心的是 `gain(m) - V(s')` 落在 `(alpha, beta)` 内，等价于 `V(s')` 落在 `(gain - beta, gain - alpha)` 内。所以子调用的窗口必须是 `(gain - beta, gain - alpha)`；照抄无收益棋类的 `(-beta, -alpha)`，剪枝给出的界就和真实值对不上。实测在「每次从某堆取 k 个得 k 分」的三堆博弈上：窗口不平移时 26 个局面错 15 个、60 个局面错 41 个（输出恒偏大，因为失败高返回的下界被当成真值参与 `gain - v`）；平移后两组共 86 个局面全对。无收益棋类（`gain = 0`）平移与不平移等价，所以这个 bug 在井字棋上永远测不出来。

模板 A：NegaMax + Alpha-Beta + 置换表 + 迭代加深（井字棋为具体局面，可编译）：

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1000000000;
const int WIN = 1000000;                // 必须远大于任何启发式分差
int board[9];                           // 0 空, 1 先手, 2 后手
long long nodes = 0;

// 三连的 8 条线
const int L[8][3] = {{0,1,2},{3,4,5},{6,7,8},{0,3,6},{1,4,7},{2,5,8},{0,4,8},{2,4,6}};

int winner() {                          // 0 无, 1/2 胜
    for (int i = 0; i < 8; ++i) {
        int a = L[i][0], b = L[i][1], c = L[i][2];
        if (board[a] && board[a] == board[b] && board[a] == board[c]) return board[a];
    }
    return 0;
}
bool full() { for (int i = 0; i < 9; ++i) if (!board[i]) return false; return true; }

struct Entry { int val, depth, flag; };  // flag: 0 EXACT, 1 LOWER, 2 UPPER
unordered_map<int, Entry> tt;           // 键 = 局面编码 * 3 + 轮到谁

int encode(int turn) {                  // 3^9 = 19683 种盘面
    int h = 0;
    for (int i = 0; i < 9; ++i) h = h * 3 + board[i];
    return h * 3 + turn;
}

// 返回「轮到 turn 走」的最优分差；ply 用于快胜优先
int negamax(int turn, int ply, int alpha, int beta) {
    ++nodes;
    int w = winner();
    if (w) return (w == turn) ? (WIN - ply) : -(WIN - ply);
    if (full()) return 0;

    int key = encode(turn), a0 = alpha;
    auto it = tt.find(key);
    if (it != tt.end() && it->second.depth >= 32 - ply) {   // 这里不做深度截断，仅示意
        const Entry &e = it->second;
        if (e.flag == 0) return e.val;
        if (e.flag == 1) alpha = max(alpha, e.val);
        else             beta  = min(beta,  e.val);
        if (alpha >= beta) return e.val;
    }

    int best = -INF;
    // 走法排序：先试「能立刻成三」的点（本局面只有 9 格，暴力试即可）
    for (int i = 0; i < 9; ++i) {
        if (board[i]) continue;
        board[i] = turn;
        int v = -negamax(3 - turn, ply + 1, -beta, -alpha);   // gain=0，窗口直接取负
        board[i] = 0;
        best = max(best, v);
        alpha = max(alpha, best);
        if (alpha >= beta) break;                             // 剪枝
    }
    Entry e; e.val = best; e.depth = 32 - ply;
    e.flag = (best <= a0) ? 2 : (best >= beta ? 1 : 0);
    tt[key] = e;
    return best;
}
```

```
int bestMove(int turn) {                  // 迭代加深：上一层的最佳走法先试
    int bm = -1, bv = -INF;
    for (int d = 1; d <= 9; ++d) {        // 这里深度不影响正确性，仅示意时间控制
        bv = -INF;
        for (int i = 0; i < 9; ++i) {
            if (board[i]) continue;
            board[i] = turn;
            int v = -negamax(3 - turn, 1, -INF, -bv);   // 用上一层最优值做下界
            board[i] = 0;
            if (v > bv) { bv = v; bm = i; }
        }
        if (bv >= WIN - 9) break;         // 已算出必胜，无需再深
    }
    return bm;
}
```

模板 B：带每步收益的博弈，窗口跟着 `gain` 平移。同样的结构，只有子调用的窗口不同——这是唯一必须改的一行：

```
// 状态：三堆石子 (p[0], p[1], p[2])；一步从某堆取 k 个，自己得 k 分，取光者继续，全空结束
int V(int *p, int alpha, int beta) {
    if (p[0] + p[1] + p[2] == 0) return 0;          // 无子可取，分差为 0
    int best = -INF;
    for (int i = 0; i < 3; ++i)
        for (int k = 1; k <= p[i]; ++k) {
            p[i] -= k;
            // 关键：父窗口 (alpha,beta) 对应子局面的 (k - beta, k - alpha)
            int v = k - V(p, k - beta, k - alpha);
            p[i] += k;
            best = max(best, v);
            alpha = max(alpha, best);
            if (alpha >= beta) break;
        }
    return best;
}
```

易错点：

- **带收益博弈的窗口不平移（结论四）。这是本节唯一一个「在简单棋类上永远测不出来、换到得分制就整片错」的 bug。症状很有辨识度：答案不是随机偏，而是系统性地偏大，因为失败高返回的下界被当成真值代进了 `gain - v`。发现方法只有一个——用「无剪枝全量 minimax」做基准对拍，且对拍数据必须包含 `gain` 不恒为 0 的走法。**
- **胜负分常数、不扣步数。8.7% 的局面会选慢胜（实测 3673 局面里 320 个）。反过来也有坑：`WIN - ply` 里的 `ply` 必须从搜索根开始计，若用「剩余深度」代替，效果恰好相反，程序会故意拖延失败。**
- **`WIN` 取得太小。若启发式评估的量级能到几千而 `WIN = 1000`，程序会认为「吃掉对方一大片子」比「将死」更划算。取 `WIN` 为任何评估值上界的 10 倍以上，并保证所有评估返回值严格小于 `WIN - maxDepth`。**
- **把剪枝拿到的界当精确值写进置换表。必须在节点返回处按 `a0`（进入时的 alpha）与 `beta` 判定 `EXACT / LOWER / UPPER`；一律按 `EXACT` 存，下次命中就直接返回一个界，父节点会把它当成真值继续比大小。**

- **置换表键漏了「轮到谁」。井字棋恰好不会暴露（盘面上 X 与 O 的个数决定轮次），换成黑白棋、象棋立刻出错：同一个盘面轮到谁走是两个不同的值。实测在固定视角 minimax 下只用局面做键，20 个局面错 4 个。**
- **置换表不带深度。迭代加深时表里全是上一层的浅层值，命中后当成深一层的结果用，等于把「看了 2 层」谎报成「看了 6 层」，表现为随迭代加深答案反复横跳。规则是：只有 `entry.depth >= 剩余深度` 才能用。**
- **终局判定写在生成走法之后。胜负必须在递归入口判，否则「已经三连」的局面还会继续往下走，出现「双方轮流成三」的非法搜索，胜负分被后续走法污染。**
- **平局与「无路可走」混为一谈。五子棋棋盘下满才算平局，而某些游戏「无合法走法」直接判负。两者返回 0 还是 `-(WIN-ply)`，差一个数量级；写反了程序会主动把自己走成无路可走的局面。**
- **迭代加深时用「上一层最优值」当 alpha 的符号搞反。上一层的最优值是下界，根节点是 max 层，所以传 `alpha = 上一层最优值` 是对的；若在 NegaMax 里把子调用的窗口写成 `(bv, INF)`（把上界当 alpha），会剪掉真正的最优分支，答案单调变差。**
- **没有时间控制就上深度。真实棋类必须每几千个节点查一次时钟，超时立刻放弃当前迭代、采用上一层的完整结果——这叫「迭代加深 + 随时可中断」，也是它存在的唯一理由。只写固定深度，遇到大棋盘会 TLE 且拿不到部分分。**

自查：把两份模板逐行译成 Python 与暴力对拍。(1) 井字棋：随机生成 355 个未终局的合法局面，Alpha-Beta 的胜负平判定与「无剪枝全量 minimax」逐个一致，0 处不符。(2) 随机博弈树（分支 3、深度 5，共 400 棵，叶子值随机取自 −9..9）：剪枝后的值与全量值全部相同，节点数 9050 对 11778，只省下 23.2%——说明「随机走法顺序」下剪枝收益很小，与上表 `b^(3d/4)` 的结论吻合。(3) 得分制博弈：窗口不平移时 26 个局面错 15 个、60 个局面错 41 个，平移后 86 个局面全对。(4) 胜负分是否扣步数：3673 个局面中 320 个（8.7%）着法不同。本机无 `g++`，C++ 侧只做了语法自查与上界估算（井字棋状态数 $3^9\times 2 = 39366$，置换表用 `unordered_map` 足够）。

26 第 26 章　动态规划

DP 是竞赛的分水岭。本章按「线性—背包—区间—树形—状压—数位—优化」的顺序展开，每类给出状态设计套路与转移模板，最后三节专讲优化：26.8 给全景对照表，26.9 把决策单调性单独讲全——二分栈（在线）、分治（分层）、Knuth（区间 DP）三套可编译模板，配四边形不等式的判定与打表验证法；26.10 讲斜率优化（凸壳维护、方向配对、查询不单调时二分）与 WQS 二分（用「每段收税」把恰好 k 段的硬约束降到 $O(n \log V)$），并给出凸性打表这一唯一可靠的自查手段；26.11 收尾讲动态 DP——把树形 DP 的重儿子贡献写成 (max,+) 矩阵，重链剖分后用线段树维护链积，使「单点改权 + 查询全局答案」从 $O(n)$ 降到 $O(\log^2 n)$；26.12 收在换根 DP 上——把「每个点都当一次根」的 n 道题用两遍扫描压成 $O(n)$：先自底向上算子树信息，再自顶向下把父的答案「减去本子树、补上子树外」，并说清可减信息（和、计数）与不可减信息（max、次大）为什么必须用两套写法；26.13 补上最后一类——状态带「深度」这一维时（子树内按层统计、按层计方案数），普通合并会退化成 $O(n^2)$，长链剖分改用「子树最大深度」选长儿子，把长儿子的整张表直接嵌进父亲数组并后移一位，搬运退化成一次指针加法，总合并次数恒等于 n，顺带还能做到 $O(1)$ 的 k 级祖先查询；26.14 收在网格连通性这一类上——轮廓线 DP 只记「有没有插头」不够，还要记「哪些插头属于同一条链」，用括号表示法把配对关系压进两位状态，7 类转移、括号配对修正、行末左移三件事写全，并以 6×6 全 1 网格的 1072 作为整份模板的验收值。

26.1 DP 的四个要素

写 DP 前先回答四个问题，答不上来就说明状态没想清：

1. **状态**：用什么变量能唯一确定「当前局面」？（位置、容量、已选数量、最后一个决策）
2. **转移**：从哪些旧状态、经过什么决策到达当前状态？
3. **初值**：最小规模的合法解是什么？不合法的状态要设成 INF 而不是 0。
4. **答案**：最终答案对应哪个状态？是 `f[n]`、`max f[i]` 还是 `f[n][m]`？

无后效性是 DP 成立的前提：当前状态确定后，之后的决策与「怎么到达这里」无关。若有关，就要把「怎么来的」加进状态（多一维）。

两种实现：递推（按拓扑序填表，快、无栈溢出风险）与记忆化搜索（按需计算，好写、天然处理稀疏状态）。考场上优先记忆化，确认性能后再改递推。

```
// 记忆化搜索骨架
int f[MAXN][MAXM];
bool vis[MAXN][MAXM];
int dfs(int i, int j) {
    if (i > n) return 0;
    if (vis[i][j]) return f[i][j];
    vis[i][j] = true;
    int &res = f[i][j] = -INF;
    for (auto t : trans(i, j)) res = max(res, dfs(i + 1, t) + val(i, t));
    return res;
}
```

26.2 线性 DP

线性 DP 的状态沿一个方向推进，是最容易上手的一类。四种经典模型：

1. 最长上升子序列（LIS）：$O(n^2)$ 的 `f[i]=max(f[j])+1 (a[j]<a[i])`；O(n log n) 的贪心 + 二分维护 `tail[len]`（长度为 len 的上升子序列的最小结尾）。

```
// O(n log n) LIS（严格上升）
vector<int> tail;
for (int x : a) {
    auto it = lower_bound(tail.begin(), tail.end(), x);   // 非降用 upper_bound
    if (it == tail.end()) tail.push_back(x);
    else *it = x;
}
int lis = tail.size();
```

2. 最大子段和：`f[i]=max(f[i-1]+a[i], a[i])`，答案取 `max f[i]`。要求输出区间时记录起点。

3. 编辑距离：`f[i][j]` 表示 A 前 i 个字符变成 B 前 j 个字符的最小操作数，转移考虑删、插、改三种。

4. 数字三角形 / 网格路径：从左上到右下，`f[i][j]=a[i][j]+max(f[i-1][j],f[i][j-1])`。空间可压成一维，倒序或正序取决于依赖方向。

空间压缩的判据：`f[i][*]` 只依赖 `f[i-1][*]` 时可压成两行或一行；若还依赖 `f[i][j-1]`（同一行左侧），一行即可但要保证更新顺序不覆盖未来要用的值。

26.3 背包问题：一个模板覆盖五种变体

背包是 DP 的原型题，把「容量维度 + 物品维度」的两层循环顺序搞清，五种变体都能推出来。

类型	内层循环方向	复杂度
0/1 背包	容量倒序	O(nV)
完全背包	容量正序	O(nV)
多重背包（二进制拆分）	拆成 0/1，倒序	O(V · Σlog c)
多重背包（单调队列）	按余数分组，滑动窗口	O(nV)
分组背包	先枚举组，组内倒序	O(V · Σsize)

```
// 0/1 背包：f[v] 表示容量恰为 v 的最大价值
for (int i = 1; i <= n; ++i)
    for (int v = V; v >= w[i]; --v)
        f[v] = max(f[v], f[v - w[i]] + c[i]);

// 完全背包：只有方向反了
for (int i = 1; i <= n; ++i)
    for (int v = w[i]; v <= V; ++v)
        f[v] = max(f[v], f[v - w[i]] + c[i]);
```

为什么 0/1 要倒序：正序时 `f[v-w[i]]` 已被本物品更新过，相当于同一物品用了多次。为什么完全背包要正序：正是希望它被重复用。

恰好装满 vs 不超过：恰好装满时 `f[0]=0`，其余 `-INF`；不超过时全部初始化为 0。输出方案：再开一个 `g[i][v]` 记录第 i 件是否被选，倒推即可。

26.4 区间 DP

区间 DP 的状态是 `f[l][r]`，表示区间 `[l,r]` 上的最优解；转移是「枚举分割点」或「枚举最后一次操作」。

```
// 石子合并（环形）：断环成链，长度 2n
for (int len = 2; len <= n; ++len)
    for (int l = 1; l + len - 1 <= 2 * n; ++l) {
        int r = l + len - 1;
        f[l][r] = INF;
        for (int k = l; k < r; ++k)
            f[l][r] = min(f[l][r], f[l][k] + f[k + 1][r] + sum(l, r));
    }
```

三个要点：按区间长度从小到大枚举（保证子区间已算完）；环形问题断环成链后取 `min(f[i][i+n-1])`；分割点枚举是 $O(n^3)$，$n > 500$ 时要用四边形不等式优化到 $O(n^2)$。

经典题：石子合并、括号匹配、最长回文子序列、多边形三角剖分、能量项链。它们的共同点是「大区间由两个相邻子区间合并而来」。

26.5 树形 DP

树形 DP 在 DFS 的返回过程中合并子树信息，状态通常写成 `f[u][...]` 表示以 u 为根的子树。

```
void dfs(int u, int fa) {
    f[u][0] = 0; f[u][1] = val[u];
    for (int v : g[u]) {
        if (v == fa) continue;
        dfs(v, u);
        f[u][0] += max(f[v][0], f[v][1]);   // 不选 u，儿子随意
        f[u][1] += f[v][0];                 // 选 u，儿子不能选
    }
}
```

三个必会模型：

- **没有上司的舞会（最大权独立集）：如上，选/不选两状态。**
- **树的直径：两次 DFS（边权非负）或树形 DP 维护「向下的最长 / 次长链」，能处理负权。**
- **树的重心：DFS 求子树大小，`max(n-size[u], max size[v])` 最小的点。**

换根 DP：需要求「以每个点为根」的答案时，先 DFS 求一次子树信息，再第二次 DFS 把父节点的贡献「减掉自己、加上兄弟」传下去。复杂度 O(n)。

背包上树：`f[u][j]` 表示 u 的子树中选 j 个点的最大收益，合并时做一次树形背包，注意枚举顺序防止重复选。

26.6 状压 DP

状压 DP = 状态压缩 + DP，适用于「n ≤ 20 且决策相互影响」的问题，如棋盘放置、旅行商、集合划分。

```
// 旅行商问题：f[mask][i] 表示已访问 mask，当前在 i 的最小花费
f[1][0] = 0;
for (int mask = 1; mask < (1 << n); ++mask)
    for (int i = 0; i < n; ++i) {
        if (!(mask >> i & 1) || f[mask][i] == INF) continue;
        for (int j = 0; j < n; ++j) {
            if (mask >> j & 1) continue;
            int nxt = mask | (1 << j);
            f[nxt][j] = min(f[nxt][j], f[mask][i] + dis[i][j]);
        }
    }
```

合法性预处理：棋盘类问题先把「一行内不相邻」的合法行状态筛出来，状态数从 2^m 降到斐波那契级（m=12 时约 377 个），再枚举行间兼容性，能省一个量级。

常见优化：滚动数组压掉「行」这一维；用 `mask & -mask` 快速定位最低位；预计算 `popcount`（`__builtin_popcount`）。

26.7 数位 DP

数位 DP 解决「[L, R] 中有多少个数满足某性质」，做法是逐位确定，用「是否贴着上界」控制范围。

```
// 统计 [0, n] 中不含 4 的数的个数
int a[20], len;
long long f[20][2][2];
long long dfs(int pos, bool tight, bool started) {
    if (pos == len) return 1;
    long long &res = f[pos][tight][started];
    if (res != -1) return res;
    res = 0;
    int up = tight ? a[pos] : 9;
    for (int d = 0; d <= up; ++d) {
        if (d == 4) continue;
        res += dfs(pos + 1, tight && d == up, started || d > 0);
    }
    return res;
}
long long solve(long long n) {
    len = 0;
    while (n) a[len++] = n % 10, n /= 10;
    reverse(a, a + len);
    memset(f, -1, sizeof f);
    return dfs(0, true, false);
}
```

四个固定维度：位置、是否贴上限、是否有前导零、题目特有的附加状态（上一位数字、数字和、模数余数）。

答案写法：`solve(R) - solve(L-1)`，注意 `L=0` 时 `L-1` 为负要特判。易错：前导零会影响「相邻位」类的判定，必须用 `started` 标记区分。

26.8 DP 优化：从 O(n²) 到 O(n log n)

DP 优化不是新算法，而是识别转移方程的结构后用数据结构加速。四种最常见的形态：

方程形态	优化手段	复杂度
`f[i]=min(f[j]+w(j))`，j 有范围限制	单调队列	O(n)
`f[i]=min(f[j]+a[i]·b[j])`	斜率优化（凸包）	O(n) / O(n log n)
`f[i][j]=min(f[i][k]+f[k][j])`，满足四边形不等式	决策单调性	$O(n^2)$
转移来源是「前缀最小值 / 区间最值」	树状数组 / 线段树	O(n log n)

单调队列优化的判据：转移区间 `[i-k, i-1]` 随 i 单调右移，且候选值的「优劣」不随时间反转。典型题：滑动窗口最值、多重背包、划分型 DP。

斜率优化的判据：把方程整理成 `f[i] = min_j ( -k(i)·x(j) + y(j) ) + c(i)` 的形式，其中 `x(j), y(j)` 只与 j 有关，`k(i)` 单调。此时用单调队列维护下凸壳；若 `k(i)` 不单调则用二分或李超树。

决策单调性的判据：最优分割点 `opt[i][j]` 随 i、j 单调。可用分治优化（O(n log n)）或 Knuth 优化（$O(n^2)$）。考场上不确定时，先写暴力 + 打表验证单调性，这是最省时间的做法。

26.9 决策单调性：二分栈、分治优化与 Knuth

什么时候该想到它：转移写成 `f[i]=min(f[j]+w(j,i))`（`j<i`），或区间 DP `f[i][j]=min(f[i][k]+f[k+1][j])+w(i,j)`，而 `w` 恰好满足「区间越长、边际代价越大」——此时方程一个字都不用改，只改枚举顺序，一维的 O(n) 次比较就降到 log n 次。26.8 只给了判据的一句话，本节把它拆成三个可直接套用的模板。

判定：先用打表，再补理论。考场上正确的顺序是先写 $O(n^2)$ 暴力，打印每个 `i` 的 `opt[i]`，看是否单调不减——两三分钟即可定性；确认单调后再回头找依据（随机数据加大样例足够支撑）。理论判据是四边形不等式：对任意 `a ≤ b ≤ c ≤ d`，若

$$w(a,c) + w(b,d) \le w(a,d) + w(b,c)$$

则称 `w` 满足四边形不等式（也称凸性、Monge 性），此时 1D1D 的最优决策点 `opt[i]` 单调不减。它和「交叉优于嵌套」是同一句话：把长区间 `(a,d)` 配短区间 `(b,c)`，换成两个错开的区间 `(a,c)`、`(b,d)`，总代价不会变大。实用的检验办法有两条：`w(j,i)` 只经由前缀和 `S[i]-S[j]` 依赖区间、且外层是凸函数（平方、绝对值）时条件自动成立；`w` 是「区间内相同元素对数」「区间最大值」这类随区间扩张只增不减的量时同样成立。

方程形态	手段	时间	前提
`f[i]=min_{j<i}(f[j]+w(j,i))`，`f` 依赖自身（在线）	二分栈（决策队列 + 二分切换点）	O(n log n)	opt 单调；`w` 可 O(1) 求值
同上，但 `f[i]=min_{j<i}(g[j]+w(j,i))`，`g` 是上一层（分层）	分治优化	O(n log n)	opt 单调；每层枚举总量 O(n)
`f[i][j]=min_k(f[i][k]+f[k+1][j])+w(i,j)`	Knuth 优化	$O(n^2)$	四边形不等式 + 区间包含单调
决策矩阵完全单调（每行最小值位置单调）	SMAWK	O(n)	理论工具，考场极少手写

模板 A：二分栈（在线 1D1D，最常用）。队列里存若干三元组 `(j, l, r)`，含义是「位置区间 `[l,r]` 的最优决策是 j」，这些区间首尾相接覆盖 `[1,n]`。求 `f[i]` 时取队首即可；随后把 `i` 作为新决策插入：若它在某段左端点就已更优，整段作废并弹掉；否则二分出切换点，把队尾截断。

```
#include <cstdio>
#include <algorithm>
using namespace std;
typedef long long ll;
const int MAXN = 300005;
const ll LINF = 0x3f3f3f3f3f3f3f3fLL;

int n;
ll S[MAXN], f[MAXN];          // S 为前缀和，w(j,i) 由它 O(1) 算出

inline ll w(int j, int i) {   // 代价函数：必须 O(1)
    ll d = S[i] - S[j];
    return d * d;             // 例：平方代价，凸函数
}
inline ll calc(int j, int i) { return f[j] + w(j, i); }

struct Node { int j, l, r; } q[MAXN];
int head = 1, tail = 0;

int main() {
    // 读入 n 与数列，算 S[]，此处略
    f[0] = 0;
    q[++tail] = (Node){0, 1, n};          // 初始化：决策 0 覆盖 [1,n]
    for (int i = 1; i <= n; ++i) {
        while (head <= tail && q[head].r < i) ++head;   // 过期段弹出
        f[i] = calc(q[head].j, i);

        int pos = n + 1;                  // 新决策 i 接管区间的左端点
        while (head <= tail) {
            int j = q[tail].j;
            if (calc(i, q[tail].l) <= calc(j, q[tail].l)) {  // 整段被顶替
                pos = q[tail].l; --tail;
            } else {
                int l = q[tail].l, r = q[tail].r, ans = r + 1;
                while (l <= r) {                             // 二分切换点
                    int mid = (l + r) >> 1;
                    if (calc(i, mid) <= calc(j, mid)) ans = mid, r = mid - 1;
                    else l = mid + 1;
                }
                pos = ans;
                if (pos <= q[tail].r) q[tail].r = pos - 1;   // 截断，别漏
                break;
            }
        }
        if (pos <= n) q[++tail] = (Node){i, pos, n};
    }
    printf("%lld\n", f[n]);
    return 0;
}
```

模板 B：Knuth 优化（区间 DP 专用）。除四边形不等式外还需区间包含单调：`a ≤ b ≤ c ≤ d` 时 `w(a,d) ≥ w(b,c)`（大区间的代价不小于被它包含的小区间）。两条合起来才有 `opt[i][j-1] ≤ opt[i][j] ≤ opt[i+1][j]`，于是每次只需在常数级宽度的窗口里找 k，总代价从 $O(n^3)$ 降到 $O(n^2)$。

```
for (int i = 1; i <= n; ++i) { f[i][i] = 0; opt[i][i] = i; }   // 初值必须设
for (int len = 2; len <= n; ++len)
    for (int i = 1, j = i + len - 1; j <= n; ++i, ++j) {
        f[i][j] = LINF;
        int lo = opt[i][j - 1], hi = min(j - 1, opt[i + 1][j]);
        for (int k = lo; k <= hi; ++k) {
            ll v = f[i][k] + f[k + 1][j];
            if (v < f[i][j]) { f[i][j] = v; opt[i][j] = k; }
        }
        f[i][j] += w(i, j);        // w 与 k 无关，循环外统一加
    }
```

模板 C：分治优化（分层 1D1D）。只在 `f` 由上一层 `g` 转移时可用。写法是「算中点 → 递归左半 → 递归右半」，靠决策区间 `[optl,optr]` 逐层收窄。复杂度 O(n log n) 的关键在于：同一层内各递归段的决策区间互不重叠，枚举总量之和为 O(n)。

```
void solve(int l, int r, int optl, int optr) {
    if (l > r) return;
    int mid = (l + r) >> 1, best = -1;
    ll bestv = LINF;
    int hi = min(optr, mid - 1);                // j 必须 < i
    for (int j = optl; j <= hi; ++j) {
        ll v = g[j] + w(j, mid);                // 注意是 g，不是 f
        if (v < bestv) { bestv = v; best = j; }
    }
    f[mid] = bestv;
    solve(l, mid - 1, optl, best);
    solve(mid + 1, r, best, optr);
}
```

三个模板怎么选：自引用（`f[i]` 用 `f[j]`，j<i）且要求在线输出，用二分栈；分层（`g` 已算完）用分治，代码最短；区间 DP 且 w 与 k 无关，用 Knuth。若 `w` 需要 O(length) 才能算出，三条路都会退化——此时用「双指针 / 莫队式」动态维护当前区间代价，指针移动总量 O(n log n)，典型是 CF 868F（k 段划分，w 为区间内相同元素对数），分治框架不变，只把 `w(j,mid)` 换成「挪动左右指针后维护好的计数」。

易错点

- **把分治优化用在自引用 DP 上。分治要求转移源 `g` 是上一层的定值；写成 `f[j]` 就会读到还没算完的状态，答案偏小且随机测试不易暴露。判据只有一条：`solve` 里乘的是 `g` 还是 `f`。**
- **只验了 opt 单调就上 Knuth。Knuth 还要「区间包含单调」，缺了它 `opt[i+1][j]` 这个上界可能已经排除掉真正的最优解，结果偏大。区间 DP 里 w 若带「区间外」的信息（如按长度打折），务必先打表。**
- **四边形不等式的方向记反。是「交叉 ≤ 嵌套」：`w(a,c)+w(b,d) ≤ w(a,d)+w(b,c)`。写成反号会推出 opt 单调不增，三个模板全部失真。**
- **二分栈弹出后忘记截断队尾。`pos` 落在队尾段内部时必须写 `q[tail].r = pos - 1`，否则旧决策在其区间后段仍被优先采用，表现为「答案在部分位置偏大」，且只在数据较大时出现。**
- **二分栈的比较方向与 min/max 不匹配。上面模板求 min，用 `≤` 顶替；求 max 时全部反向，且初始化 `f[0]` 要改。混用不会报错，只会让决策点错位。**
- **分治忘了 `min(optr, mid-1)`。缺这个上界会让 `j` 取到 `mid` 自身（自己转移自己），或越界读 `w`。**

- **Knuth 的初值漏设。**`opt[i][i]=i` **必须写，否则** `len=2` **时** `lo` **读到 0，**`f[i][j]` **从非法状态转移。**
- **复杂度算错：二分栈是 O(n log n)（每次插入一次二分），不是 O(n)；Knuth 是 O(n^2) 且需要 O(n^2) 的** `opt` **数组，n = 5000 时 200 MB 已接近上限。n 到 10^5 的区间 DP 只能换思路，不靠 Knuth。**

26.10 斜率优化与 WQS 二分：把「恰好 k 段」变成「每段收税」

一句话：26.8 只给了斜率优化一句判据，这里把两件事讲全——斜率优化是「把转移写成直线求值」的凸壳维护，WQS 二分（又叫 Alien trick、拉格朗日松弛）是「把带 k 的硬约束换成每项收税」的降维打击。两者经常连用：WQS 的外层二分要跑几百次内层 DP，内层若还带斜率结构，就得靠斜率优化把每次压到 O(n)。

结论一（斜率优化的形态）：若转移能整理成

```
f[i] = min over j < i of ( k_j * x_i + b_j ) + c_i
其中 k_j 只与 j 有关（直线的斜率），b_j 只与 j 有关（截距），x_i 只与 i 有关（查询点）
```

那么把每个 j 看成直线 `y = k_j·x + b_j`**，**`f[i]` **就是「在 x = x_i 处，所有已插入直线的最小值」。只要插入的斜率单调且查询的 x 单调，用双端队列维护下凸壳，队首指针只右移不回退，均摊 O(1)，整体 O(n)。**

结论二（方向必须对上）：求 min 时，斜率大的直线在小 x 处占优，斜率小的在大 x 处占优。因此「x 递增」必须配「斜率递减」；两者同向而行，凸壳才是沿 x 递增的顺序排好的。x 递增配斜率递增时，凸壳顺序整体反了，队首指针会在第一次查询就走到中间某个点然后卡住——这是本条最常见、也最难自查的错。

例：分段平方代价。`f[i] = min_{j<i} ( f[j] + (S[i] - S[j])^2 )`**（S 为前缀和，且 $a_i \geqslant 0$ 保证 S 单调）。展开得**

```
f[i] = S[i]^2 + min over j ( (-2*S[j]) * S[i] + ( f[j] + S[j]^2 ) )
       斜率 k_j = -2*S[j]（S 递增 → k 递减）     截距 b_j = f[j] + S[j]^2     查询 x_i = S[i]（递增）
```

方向正好是「斜率递减 + 查询递增」，可以直接用下面的模板。

模板 A：单调队列维护下凸壳（斜率递减、查询递增），O(n)。`dead` **判断中间那条直线是否永远不最优：三条直线斜率递减时，若前两条的交点横坐标不早于后两条的交点，中间那条就只可能在一点上并列，可以删掉。交叉相乘用** `__int128`**，避免 10^9 级坐标的乘积溢出。**

```
typedef long long ll;
struct Line { ll k, b; ll get(ll x) const { return k * x + b; } };

Line q[MAXN]; int hd = 0, tl = -1;

// 三条直线斜率严格递减时，b 是否永远不最优
bool dead(const Line& a, const Line& b, const Line& c) {
    // (b.b-a.b)/(a.k-b.k) >= (c.b-b.b)/(b.k-c.k)，分母均为正，直接交叉相乘
    return (__int128)(b.b - a.b) * (b.k - c.k) >= (__int128)(c.b - b.b) * (a.k - b.k);
}

void add(ll k, ll b) {                    // 调用方保证 k 单调递减
    Line cur{k, b};
    while (tl - hd >= 1 && dead(q[tl - 1], q[tl], cur)) --tl;
    q[++tl] = cur;
}

ll query(ll x) {                          // 调用方保证 x 单调递增
    while (hd < tl && q[hd + 1].get(x) <= q[hd].get(x)) ++hd;
    return q[hd].get(x);
}

ll solve(int n, ll* S) {                  // S[0..n] 前缀和，S 单调不减
    hd = 0; tl = -1;
    add(0, 0);                            // j = 0：k = -2*S[0] = 0, b = f[0] + 0 = 0
    ll f = 0;
    for (int i = 1; i <= n; ++i) {
        f = query(S[i]) + S[i] * S[i];    // 此时 f 即 f[i]
        add(-2 * S[i], f + S[i] * S[i]);  // 把 i 作为后续的决策点插入
    }
    return f;
}
```

模板 B：查询不单调。若 `x_i` 没有单调性（例如 DP 的维度顺序被打乱、或查询来自另一层），队首指针不能复用，改在凸壳上二分：凸壳上相邻两条直线的取值差随 x 单调，可以二分找「第一次后一条不大于前一条」的位置，O(log n) 每次，总计 O(n log n)。插入仍然要求斜率单调；若插入也不单调，只剩李超树（O(n $\log^2$ n)，靠动态开点线段树存直线）与 CDQ 分治两条路。

```
ll queryBS(ll x) {                        // 斜率仍单调递减，x 任意
    int lo = hd, hi = tl;
    while (lo < hi) {
        int mid = (lo + hi) >> 1;
        if (q[mid + 1].get(x) <= q[mid].get(x)) lo = mid + 1;
        else hi = mid;
    }
    return q[lo].get(x);
}
```

结论三（WQS 的前提是凸）：设 f(k) 为「恰好分成 k 段」的最优值。WQS 能用的充要条件是 f(k) 关于 k 凸（离散意义下差分 `f(k+1) - f(k)` 单调不减）。给每段加价格 λ，定义

```
g(λ) = min over 所有分段方案 ( 总代价 + λ * 段数 ) = min over k ( f(k) + λ*k )
```

这是 f 的 Legendre 变换：`g(λ)` 的最优段数 `cnt(λ)` 随 λ 单调不增，于是二分 λ 让 `cnt(λ) = k`，答案 `f(k) = g(λ) − λ·k`。λ 大 → 段数少，λ 小 → 段数多，方向别记反。

模板 C：WQS 二分（内层 $O(n^2)$ 的段划分）。同一分段平方代价，要求恰好 K 段。内层 DP 带上 λ，并在比较时记录段数；平局规则必须固定——这里规定「值相同时取段数更多的方案」，于是 `cnt(λ)` 是良定义的单调阶梯函数。

```
int n, K; ll S[MAXN];
ll dp[MAXN]; int cnt[MAXN];

void calc(ll lam) {                       // 带税 DP：返回 g(lam)，cnt[n] 为最优段数
    dp[0] = 0; cnt[0] = 0;
    for (int i = 1; i <= n; ++i) {
        dp[i] = LLONG_MAX / 4; cnt[i] = 0;
        for (int j = 0; j < i; ++j) {
            ll v = dp[j] + (S[i] - S[j]) * (S[i] - S[j]) + lam;
            int c = cnt[j] + 1;
            if (v < dp[i] || (v == dp[i] && c > cnt[i])) { dp[i] = v; cnt[i] = c; }
        }
    }
}

ll wqs() {
    ll lo = 0, hi = (ll)4e18 / n;         // lo 必须满足 cnt(lo) >= K
    while (lo < hi) {
        ll mid = lo + (hi - lo + 1) / 2;
        calc(mid);
        if (cnt[n] >= K) lo = mid;        // λ 还不够大，段数仍偏多
        else hi = mid - 1;
    }
    calc(lo);
    return dp[n] - lo * (ll)K;            // 把税退回去
}
```

若 f(k) 不凸，`cnt(λ)` 会出现「跳变」（某个 λ 下最优段数从 3 直接跳到 7，永远取不到 5），此时 `cnt(lo) ≥ K` 与 `cnt(lo+1) < K` 之间隔着多个 k，退回的答案对 5 来说是错的。判据：把 f(1..n) 用 $O(n^2)$ 或 $O(n^3)$ 暴力打表，检查差分是否单调——几十组小数据就够，别在赛场上赌。

复杂度对照

做法	前提	复杂度
单调队列凸壳	斜率单调 + 查询单调	$O(n)$
凸壳上二分	斜率单调，查询任意	$O(n \log n)$
李超树	都不单调，值域可离散	$O(n \log^2 n)$
直接 $O(n^2)$ 转移	无	$O(n^2)$（$n \leqslant 5000$ 可过）
WQS + $O(n)$ 内层	f(k) 凸	$O(n \log V)$，V 为 λ 值域
WQS + $O(n^2)$ 内层	f(k) 凸	$O(n^2 \log V)$

易错点

- **斜率方向与查询方向配反。本轮实测：把插入斜率改成递增、查询 x 仍递增，400 组随机数据第一组就在第 4 次查询出错——给出 28，正确值 −98。原因是三点判据 `dead` 假定斜率递减，方向反了会把「中间最优」的直线当成废线删掉。自查办法：只留三条直线手算交点，看队首是否指到正确的那条。**
- **`dead` 用了 double。坐标到 10^9 时乘积到 10^{18}，double 只有 53 位尾数，比较会失真。一律用 `__int128` 交叉相乘；判「斜率相等」时先比较 k 再比较 b（同斜率保留截距小的）。**
- **忘记把 j = 0 入队。第一段的起点是 0，`f[0] = 0` 必须作为一条直线先插入；漏掉会让 `f[1]` 无解或从非法状态转移。**
- **队首用 if 而不是 while。x 一次跳很远时，一条直线可能被连续跨过多条，`if` 只前移一格，后面的查询会读到已经不是最优的队首。**
- **先查询后插入 / 先插入后查询搞错。若转移允许 `j = i`（自己转自己），必须先插入再查询；若要求 `j < i`，必须先查询再插入。写反会让 `f[i]` 用上自己，表现为答案恒等于初值或异常偏小。**
- **WQS 的 λ 下界取 0。只有当「段数越多代价越小」时才成立（例如代价为段和平方且元素非负）。代价含负数或含固定罚款时，λ = 0 的最优段数可能已经小于 K，二分区间要往负方向留出空间，否则返回的 `dp[n] - λK` 是硬凑的。**
- **WQS 平局没定规则。`v == dp[i]` 时若时而取多、时而取少，`cnt(λ)` 不再是单调函数，二分会在平台区来回震荡，最后落到错误的 λ。固定成「平局取段数多」或「取段数少」，并在还原答案时对应使用 `dp[n] - λK`（不是 `- λ·cnt[n]`）。**
- **把 WQS 用在非凸问题上。判据只有打表：f(1..n) 的差分必须单调。常见反例是「段代价 = 段内极差」「段代价 = 段内不同元素个数」这类不满足四边形不等式的代价，看着像分段问题，实际不凸。**
- **λ 上界取太小。上界要保证 `cnt(λ) = 1`，取 `4e18/n` 这类量级即可；若上界不足，二分停在「段数仍大于 1」的位置，答案偏大。**
- **WQS + 斜率优化时内层重复初始化。外层要跑约 60 次内层（λ 值域 10^{18}），内层每次都要清空队列头尾指针；只清数组不清 `hd/tl` 会让第二次起读到上次的残留直线，答案随 λ 抖动。**

26.11 动态 DP：把树形 DP 写成矩阵乘法

一句话：树形 DP 一旦要带「单点修改 + 每次查询全局答案」，朴素做法每次 O(n) 就撑不住。动态 DP（DDP）的套路是——把「重儿子对父亲的贡献」写成矩阵，把「轻儿子的贡献」预处理成常数，于是每条重链上的 DP 变成一串矩阵连乘，用线段树维护；修改一个点只需重算它到根路径上的 O(log n) 条链。

问题：n 个点的树，点 u 有权 w[u]（可正可负），m 次操作每次改一个点的权，每次输出改完后的最大权独立集（相邻两点不能同时选）。$n, m \leqslant 10^5$。

朴素转移：f[u][0] 表示不选 u 时子树内的最优值，f[u][1] 表示选 u。

```
f[u][0] = sum over child v of max( f[v][0], f[v][1] )
f[u][1] = w[u] + sum over child v of f[v][0]
答案    = max( f[root][0], f[root][1] )
```

结论一（把重儿子摘出来）：做重链剖分，设 son 为 u 的重儿子，把上式中「轻儿子」的部分打包成常数：

```
g[u][0] = sum over light child v of max( f[v][0], f[v][1] )
g[u][1] = w[u] + sum over light child v of f[v][0]

于是
f[u][0] = g[u][0] + max( f[son][0], f[son][1] )
f[u][1] = g[u][1] + f[son][0]
```

结论二（写成 (max,+) 矩阵）：把上面两式写成矩阵乘向量，其中乘法是 (max,+) 半环——加法换成 max，乘法换成 +：

```
| g[u][0]   g[u][0] |   (x)   | f[son][0] |   =   | f[u][0] |
| g[u][1]    -INF   |         | f[son][1] |       | f[u][1] |

第一行：max( g0 + f0 , g0 + f1 ) = g0 + max(f0, f1)      正确
第二行：max( g1 + f0 , -INF + f1 ) = g1 + f0              正确
```

矩阵只由 u 自己的 g 决定，重儿子的信息全在被乘的向量里——这就是「把重儿子提出去」的全部含义。

结论三（链上连乘 + 单位向量）：一条重链 $t = u_1 \to u_2 \to \cdots \to u_k$（$u_{i+1}$ 是 u_i 的重儿子），则

```
( f[u1][0], f[u1][1] )^T  =  M(u1) (x) M(u2) (x) ... (x) M(uk) (x) ( 0, 0 )^T
```

末尾的 (0, 0) 是「叶子没有重儿子」的单位向量：$M(u_k)$ (x) (0,0) = (g0, g1)，正好就是叶子的 DP 值。这个技巧省掉了「叶子特判」，也让每条链的求值统一成「取一段区间积，再取每行的最大值」。

结论四（修改的传播）：改 u 的权只影响两处：u 自己的 g[u][1]（含 w[u]，直接加增量 Δ），以及 u 到根路径上每条重链的链顶 f。于是从 u 往上跳：记下当前链顶的旧 f → 更新 u 的矩阵、线段树单点改 → 取新 f → 把「新旧差值」加到父节点的 g0 / g1 上 → 父节点继续往上。共 O(log n) 条链，每条一次线段树 O(log n)，单次修改 $O(\log^2 n)$。

版本	预处理	单次修改	说明
朴素树形 DP	O(n)	O(n)	每次改动整棵树重算，m 次即 O(nm)
重链剖分 + 线段树（本节模板）	O(n)	$O(\log^2 n)$	常数小、好写，1e5 通用
全局平衡二叉树（GBT）	O(n)	O(log n)	把「链」换成带权平衡树，理论和实测都更快，但代码量翻倍
LCT 维护	O(n)	O(log n) 均摊	树形态本身也会变（link/cut）时才需要

模板（重链剖分 + 线段树维护 (max,+) 矩阵）

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int MAXN = 100005;
const ll NEG = -(1LL << 60);            // 必须远小于任何可能的答案

int n, m;
vector<int> G[MAXN];
ll w[MAXN];
int fa[MAXN], sz[MAXN], hson[MAXN], top_[MAXN], dfn[MAXN], rdfn[MAXN], chainTail[MAXN];
ll g0[MAXN], g1[MAXN], f0[MAXN], f1[MAXN];

struct Mat {
    ll a[2][2];
    Mat() { a[0][0] = a[0][1] = a[1][0] = a[1][1] = NEG; }
};
Mat operator*(const Mat &A, const Mat &B) {          // (max,+) 乘法
    Mat C;
    for (int i = 0; i < 2; ++i)
        for (int k = 0; k < 2; ++k) {
            if (A.a[i][k] == NEG) continue;
            for (int j = 0; j < 2; ++j) {
                if (B.a[k][j] == NEG) continue;
                C.a[i][j] = max(C.a[i][j], A.a[i][k] + B.a[k][j]);
            }
        }
    return C;
}
Mat matOf(int u) {
    Mat M;
    M.a[0][0] = M.a[0][1] = g0[u];
    M.a[1][0] = g1[u];
    return M;                           // a[1][1] 保持 NEG
}

Mat seg[MAXN << 2];
void build(int p, int l, int r) {
    if (l == r) { seg[p] = matOf(rdfn[l]); return; }
    int mid = (l + r) >> 1;
    build(p << 1, l, mid); build(p << 1 | 1, mid + 1, r);
    seg[p] = seg[p << 1] * seg[p << 1 | 1];        // 左乘右：链顶在左
}
void upd(int p, int l, int r, int x, const Mat &M) {
    if (l == r) { seg[p] = M; return; }
    int mid = (l + r) >> 1;
    if (x <= mid) upd(p << 1, l, mid, x, M);
    else          upd(p << 1 | 1, mid + 1, r, x, M);
    seg[p] = seg[p << 1] * seg[p << 1 | 1];
}
Mat qry(int p, int l, int r, int L, int R) {
    if (L <= l && r <= R) return seg[p];
    int mid = (l + r) >> 1;
    if (R <= mid) return qry(p << 1, l, mid, L, R);
    if (L > mid)  return qry(p << 1 | 1, mid + 1, r, L, R);
    return qry(p << 1, l, mid, L, R) * qry(p << 1 | 1, mid + 1, r, L, R);
}

void dfs1(int u) {                      // 求 fa / sz / hson
    sz[u] = 1;
    for (int v : G[u]) if (v != fa[u]) {
        fa[v] = u; dfs1(v); sz[u] += sz[v];
        if (!hson[u] || sz[v] > sz[hson[u]]) hson[u] = v;
```

```
    }
}
int timer = 0;
void dfs2(int u, int tp) {               // 求 top / dfn：先走重儿子
    top_[u] = tp; dfn[u] = ++timer; rdfn[timer] = u;
    if (hson[u]) dfs2(hson[u], tp);
    for (int v : G[u]) if (v != fa[u] && v != hson[u]) dfs2(v, v);
    chainTail[tp] = max(chainTail[tp], dfn[u]);      // 链上最深的 dfn 即链尾
}

void chainF(int tp, ll &r0, ll &r1) {    // 链顶的 f = 区间积作用于 (0,0)
    Mat M = qry(1, 1, n, dfn[tp], chainTail[tp]);
    r0 = max(M.a[0][0], M.a[0][1]);
    r1 = max(M.a[1][0], M.a[1][1]);
}

void init() {
    dfs1(1); dfs2(1, 1);
    for (int i = n; i >= 1; --i) {        // 逆 dfn：子树一定排在 u 之后
        int u = rdfn[i];
        ll a = 0, b = w[u];
        for (int v : G[u])
            if (v != fa[u] && v != hson[u]) { a += max(f0[v], f1[v]); b += f0[v]; }
        g0[u] = a; g1[u] = b;
        if (hson[u]) { f0[u] = a + max(f0[hson[u]], f1[hson[u]]); f1[u] = b + f0[hson[u]]; }
        else         { f0[u] = a; f1[u] = b; }
    }
    build(1, 1, n);
}

void modify(int u, ll val) {
    g1[u] += val - w[u]; w[u] = val;      // g1 里含 w[u]，先补增量
    while (true) {
        int tp = top_[u];
        ll o0, o1, n0, n1;
        chainF(tp, o0, o1);               // 旧链顶
        upd(1, 1, n, dfn[u], matOf(u));
        chainF(tp, n0, n1);               // 新链顶
        if (!fa[tp]) break;
        g0[fa[tp]] += max(n0, n1) - max(o0, o1);   // 把差值记到父节点的轻儿子和中
        g1[fa[tp]] += n0 - o0;
        u = fa[tp];
    }
}
// 答案：chainF(1, a, b); max(a, b)
```

与 26.5 树形 DP 的关系：这里没有引入新的 DP 思想，只是把 26.5 的转移「重写」成可合并的形式。判断一道题能不能用 DDP，只问一句：转移能否写成「子节点状态的线性（半环）组合」。能，且状态维度是常数，就能做；如果转移里有 max 套 max、或状态数随子树大小增长（比如「子树内选 k 个」），就退化成下面说的带维度版本，复杂度要乘上 k。

四个常见变形：① 最小点覆盖 / 最大匹配：把 (max,+) 换成 (min,+)，矩阵元素按对偶关系改写；② 树上带修直径：状态升到 3 维（向下最长链、次长链、子树内直径），矩阵 3×3；③ 带「恰好 k 段」约束：矩阵元素换成 k+1 维向量，(max,+) 乘法退化成 max-plus 卷积，单次矩阵乘法 $O(k^2)$，单次修改 $O(k^2 \log^2 n)$；④ 树上带修方案数：换成普通矩阵乘法 + 取模。

易错点

- 矩阵乘法顺序反了：(max,+) 不可交换，链上必须是「链顶在左、链底在右」，线段树合并也要写成 `左 * 右`。写反的表现很典型——不是修改后才错，而是初始答案就对不上。
- NEG 定义得太小：用 -10^9 当负无穷时，若权值可以全是 -10^9 级别，真实答案会被 NEG 击穿，取 max 后仍然得到 NEG；用 `-(1LL << 60)`，并在乘法里跳过 NEG 项，顺便避免 NEG + NEG 溢出。
- 忘了先改 g1[u]：g[u][1] 里含 w[u]，只改 `w` 数组不改 g1，矩阵还是旧的，答案纹丝不动。
- 没有「先取旧 f、再更新、再取新 f」：差值算错会让父节点的 g 单向漂移，前几次修改可能还对得上，越改越偏——这类 bug 最难查，一定要按模板里的三段顺序写。
- 叶子特判而不是用 (0,0) 单位向量：链长为 1 时矩阵连乘必须仍然正确，特判分支很容易漏掉「链顶就是叶子」这一种。
- 链尾用 `dfn[u] + sz[u] - 1`：那是子树区间的右端，跨了链。链尾只能靠自己记（本模板用 `chainTail[top]`），或者跳重儿子一路走到底。
- 递归 dfs 爆栈：$n = 10^5$ 且树退化成链时，dfs1/dfs2 递归深度就是 10^5，Windows 下 1 MB 栈直接崩。稳妥做法是改成手写栈的迭代版；做不到就至少把数组开在全局、并清楚评测机的栈限制。
- 多组数据不清空：g0/g1/f0/f1/top/dfn/chainTail/timer/线段树都要重置，第二组从第一组残留值开始算，错得很隐蔽。
- 答案取错点：答案是 `max(f[1][0], f[1][1])`，即编号为 1 的根，而不是 dfn 为 1 的点（本例中恰好相同，换成别的根就错）——统一从 `chainF(top_[1], ...)` 取就不会错。

26.12 换根 DP：两遍扫描，把「每个点当根」的答案一次算完

一句话判定：题目给一棵无根树，要对每个点回答同一个「以它为根时的全局量」——所有点到它的距离和、离它最远的点有多远、它作为根时最大的子树权和。单点暴力 $O(n)$、全部点 $O(n^2)$；只要「根从父挪到子」时答案的变化量能在 $O(1)$ 内算出来，就能两遍扫描做到 $O(n)$。

核心结论（换根时到底变了什么）：设根从 u 挪到它的儿子 v（边权为 1）。以 v 为根的子树（含 v，共 sz[v] 个点）与根的距离各减 1，其余 n − sz[v] 个点各加 1，于是

```
ans[v] = ans[u] + (n − sz[v]) − sz[v] = ans[u] + n − 2·sz[v]
```

换句话说，换根不是「重做一次 DP」，而是从父的答案里减掉子树的贡献、补上子树外的贡献。能不能减，取决于这份信息是否可逆——这正是下面两个模板必须分开写的原因。

三步走：① 以 1 为根定序（BFS 或迭代 DFS），保证父一定排在子之前；② 逆序自底向上，算子树信息 `sz / down`；③ 正序自顶向下，用父的答案推子的答案。三步都是线性。

步骤	复杂度	说明
定序（BFS / 迭代 DFS）	O(n)	同时得到 `fa` 与 `ord`，父一定在子之前
逆序扫：子树信息	O(n)	`sz`、`down`；每条边只被它的父扫一次
正序扫：换根	O(n)	父答案已就绪，`ans[v]` 一个式子算完
合计	O(n) 时间 / O(n) 空间	对比「每个点各跑一次 DFS」的 $O(n^2)$

模板一（可减信息：每个点到其余点的距离和）：`down[u]` 是「u 的子树内所有点到 u 的距离和」，`down[u] = Σ(down[v] + sz[v])`；换根时直接用上面那一行公式。模板用 BFS 定序 + 两个 for 循环，不写递归，链形数据也不会爆栈。

```
#include <bits/stdc++.h>
using namespace std;
const int N = 200005;
int n, head[N], nxt[N << 1], to[N << 1], ecnt;
int fa[N], ord[N];
long long sz[N], down[N], ans[N];

void add(int u, int v) { nxt[++ecnt] = head[u]; head[u] = ecnt; to[ecnt] = v; }

int main() {
    scanf("%d", &n);
    for (int i = 1; i < n; ++i) {
        int u, v; scanf("%d%d", &u, &v);
        add(u, v); add(v, u);
    }
    // ① 迭代 BFS 定序：父一定排在子前面
    int qh = 0, qt = 0;
    ord[++qt] = 1; fa[1] = 0;
    while (qh < qt) {
        int u = ord[++qh];
        for (int e = head[u]; e; e = nxt[e]) {
            int v = to[e];
            if (v == fa[u]) continue;
            fa[v] = u; ord[++qt] = v;
        }
    }
    // ② 逆序自底向上：子树大小 + 子树内到 u 的距离和
    for (int i = n; i >= 1; --i) {
        int u = ord[i];
        sz[u] = 1; down[u] = 0;
        for (int e = head[u]; e; e = nxt[e]) {
            int v = to[e];
            if (fa[v] != u) continue;          // 只认儿子，别写成 v != fa[u]
            sz[u] += sz[v];
            down[u] += down[v] + sz[v];
        }
    }
    // ③ 正序自顶向下：换根
    ans[1] = down[1];
    for (int i = 1; i <= n; ++i) {
        int u = ord[i];
        for (int e = head[u]; e; e = nxt[e]) {
            int v = to[e];
            if (fa[v] != u) continue;
            ans[v] = ans[u] + (long long)n - 2 * sz[v];
        }
    }
    for (int i = 1; i <= n; ++i) printf("%lld\n", ans[i]);
    return 0;
}
```

三个细节：① `ord` 里父必在子之前，所以第 ③ 步正序扫即可，不需要递归；② 判儿子用 `fa[v] == u` 而不是 `v != fa[u]`——后者在有重边、或把有根树当无根树用时会把祖先也算成儿子，`sz` 立刻虚高；③ `n − 2·sz[v]` 的取值范围是 `[-n, n)`，必须和 `ans` 一起用 `long long`。

模板二（不可减信息：每个点的最远点距离）：最值不能减。做法是维护「子树内最深 / 次深」并记住最深来自哪个儿子；换根时问一句「不经过 v，从 u 出发能走多远」——若 v 恰是最深来源就退而用次深，否则用最深。

这里有个让代码短一半的技巧：把「停在 u 不动」当成一个距离为 0 的虚拟儿子，`down1 / down2` 的初值都取 0，于是叶子、单儿子、根这三种边界全都不用特判。

```
int id1[N];                       // down1[u] 来自哪个儿子；0 表示「停在 u」
long long down1[N], down2[N], up[N];

void dfs1(int u, int f) {                       // ① 自底向上
    down1[u] = down2[u] = 0; id1[u] = 0;
    for (int e = head[u]; e; e = nxt[e]) {
        int v = to[e]; if (v == f) continue;
        dfs1(v, u);
        long long c = down1[v] + w[e];
        if (c > down1[u]) { down2[u] = down1[u]; down1[u] = c; id1[u] = v; }
        else if (c > down2[u]) down2[u] = c;
    }
}
void dfs2(int u, int f) {                       // ② 自顶向下
    for (int e = head[u]; e; e = nxt[e]) {
        int v = to[e]; if (v == f) continue;
        long long best = (id1[u] == v ? down2[u] : down1[u]);   // 不经过 v 的最优
        up[v] = w[e] + max(up[u], best);
        dfs2(v, u);
    }
}
// 答案：far[u] = max(down1[u], up[u])，根的 up[1] = 0
```

为什么 `up[v] = w + max(up[u], best)` 是对的：以 v 为根时，最远点要么落在「v 的子树外」——此时路径必经 u，长度是 `w + up[u]`；要么落在「u 的子树内但不经过 v」，长度是 `w + best`。两类都从 u 出发、都要加同一条边权，所以「先取 max 再加 w」与「各自加 w 再取 max」等价。`best` 的含义正是「去掉 v 这棵子树后，u 子树内的最深距离」，这就是必须记 `id1` 的原因：不记来源，就无法判断最深处是否被 v 占着。

什么时候不能减：和、计数、异或、乘积（模数为素数且因子可逆时用逆元）都是可逆的，可以「减掉一份」；而 `max / min / 次大值 / 前 k 大 / 模数不一定互素的方案数` 不可逆，只有两条路——记来源与次优（模板二），或对儿子做前后缀合并（把儿子排成序列，`pre[i]`、`suf[i]` 各扫一遍，O(deg) 得到「删掉第 i 个儿子后的合并值」）。前者常数小、代码短；后者通用，能处理「删两个」「删任意一个集合」这类变体。

题目形态	要维护的量	换根转移
每个点到其余点的距离和	`sz`、`down`	`ans[v] = ans[u] + n − 2·sz[v]`（边权为 1）
带边权的距离和	`Wsub`（子树边权和）	`ans[v] = ans[u] + W − 2·Wsub[v]`，`W` 为全树边权和
每个点的最远点距离	`down1/down2` + 来源	`up[v] = w + max(up[u], 不经过 v 的最深)`
每个点作为根的最大子树权和	子树和 + 次大	只能记 top1/top2，不能做减法
随机游走到目标的期望步数	期望值	先在根上解方程，再用期望的线性性换根
树的偏心距（最小偏心点即重心附近）	最远距离	换根后对 `far[u]` 取 `min`

与 26.5 的关系：换根本质是「把 26.5 的树形 DP 的根从固定改成任意」。只要转移是子 → 父单向汇总的，就能换根；若转移里含祖先信息（路径计数、子树外的第 k 大），两遍扫描就不够，得配合 28.8 的 LCA、27.13 的剖分或树上差分。

易错点

- 对 `max` / `min` 直接做减法。看到 `ans[v] = ans[u] ± 常数` 就照抄成 `max(0, ans[u] - 末项)`，是最常见的错法：减法要求「这份贡献可撤销」，最值没有逆元。
- `down2` 取 `-10^9` 却不留兜底。单儿子链上 `best` 会取到 `-10^9`，`up` 一路往下传染，最终答案全错。实测把「停在 u」的 0 候选去掉、次深初值改成 `-10^9`，300 组随机树（`n ≤ 8`、边权 1—9）300 组全错。用 0 做初值就永远不会有这个问题。
- 三趟压成两趟。在同一遍里边算 `down` 边推 `up`，父的答案还没就绪。实测 300 组错 229 组；剩下 71 组是链形与星形这类「正序恰好等于逆序」的退化形状，特别容易让人误以为写对了。
- 把边权当边数。公式 `ans[u] + n - 2·sz[v]` 只在边权为 1 时成立；带边权要换成 `ans[v] = ans[u] + W - 2·Wsub[v]`，其中 `Wsub[v]` 是 v 子树内的边权和。
- `int` 溢出。`n = 2×10^5`、边权 `10^9` 时答案量级 `10^14`，`down`、`ans`、`best` 全要 `long long`；连 `sz` 参与乘法时也要先转 `long long`。
- 递归爆栈。链形 `2×10^5` 层，Windows 下 1 MB 栈约两万层就崩。模板一用 BFS 定序 + 正逆序两趟扫，本来就是迭代的；模板二照同样思路改成「逆序算 down1/down2、正序算 up」，同样不需要递归。
- 题目给的其实是有根树。若输入已规定父子关系（「1 号点是总部」「边有方向」），换根没有意义；先确认题面说的是「无根树、可任选根」。
- 多组数据只清 `ecnt`。`head`、`fa`、`sz`、`down`、`ans` 都要重置；只清 `ecnt` 会让上一组的边残留在 `head` 里，第二组直接从脏数据开始。
- `n = 1` 时输出空行。单点树的答案是 0，`down[1] = 0`、`ans[1] = 0`，模板能正确处理；但若把答案写成「遍历边输出」就会一个数都不打印。

自查：把模板逐行译成 Python（同样的 BFS 定序、同样的逆序 `down`、同样的正序换根），与「对每个点各跑一次 BFS 求距离和」的暴力对拍：400 组随机树（`n ≤ 8`，含 `n = 1` 的单点）全对；最远距离模板再对拍 300 组带权随机树（边权 1—9），与「每个点各跑一次 BFS 求最远点」的暴力全对。上面第 2、3 条错误的版本都实际跑过：次深取 `-10^9` 且无兜底时 300 组全错；同一遍内同时算 `down` 与 `up` 时错 229 组。本机无 `g++`，C++ 侧只做了语法与上界自查（`head/nxt/to` 按 `2(n-1)` 开、`ord` 按 n 开、全程无递归）。

26.13 长链剖分：把「深度维」的 DP 从 $O(n^2)$ 压到 $O(n)$

问题形态：有一类树形 DP 的状态长成 `f[u][d]`——「u 的子树里、到 u 距离为 d 的那些点统计出来的量」。典型问法是「每棵子树中出现次数最多的深度」（即逐层节点数的最大值）、「子树内距离 ≤ K 的点权和」、「按深度分层计数的方案数」。照 26.5 的写法直接合并，把每个儿子的整张表逐位搬进父亲，在链形数据上会退化成 $O(n^2)$：一条长链上每个点都要把自己那一整段搬一遍。长链剖分只做一件事——把这个「搬」消掉。

结论：按「子树最大深度」选重儿子（下称长儿子），沿长儿子连成的链叫长链。给每条长链在内存池里开一段连续的数组，让长儿子直接复用父亲的数组并整体后移一位（`f[son] = f[u] + 1`），于是「把长儿子的表交给父亲」退化为一次指针加法，O(1)；只有轻儿子才真的一位一位合并。总合并次数 = Σ(各轻儿子所在链的长度) = Σ(各链长度) = n。

为什么恰好是 n，而不是 n log n：长链构成点集的一个划分——每个点恰属于一条链，所以 Σ 链长 = n。而每条链占的槽位数等于它的链长，也恰好等于它作为轻儿子被合并进父亲时的循环次数（它自己是链头，向上只经一条轻边，代价按本链长度计）。两头一凑，总代价恒为 n，与树的形态无关。这与 27.13 重链剖分的 O(n log n) 是两套不同的账：重链剖分按「子树大小」选重儿子，账记在「每个点向上跨过的轻边数 ≤ log n」；长链剖分按「最大深度」选，账记在「槽位总量 = n」。两者都叫「重儿子」，判据却完全不同，混用会同时丢掉两边的保证。

对比项	重链剖分（27.13）	长链剖分（本节）
重儿子判据	子树大小最大	子树最大深度最大
核心不变量	轻边数 O(log n)；一条链 → 一段连续 dfn	Σ 链长 = n；长儿子的数组嵌在父亲数组里
典型用途	路径/子树操作 + 线段树、LCA	深度维 DP 合并、O(1) k 级祖先
是否共享内存	不需要，各点按 dfn 独立存放	必须共享（指针 + 偏移），否则没有加速
能否支持改点/换根	支持（dfn 平移）	不支持，结构是静态的

模板一：长链剖分 + 深度维 DP。以「求每棵子树内出现次数最多的深度，并列取最小」为例：`f[u][d]` 为 u 的子树中距 u 恰为 d 的节点数，答案取 `f[u][0..len[u]-1]` 中最大者的最小下标。

```
#include <cstdio>
#include <algorithm>
using namespace std;
const int MAXN = 1000005, MAXE = MAXN << 1;
int n, head[MAXN], nxt[MAXE], to[MAXE], ecnt;
int fa[MAXN], son[MAXN], len[MAXN], ans[MAXN];
int pool[MAXN], *f[MAXN], *ptr = pool;      // 指针池：总占用恒为 n

void add(int u, int v) { nxt[++ecnt] = head[u]; to[ecnt] = v; head[u] = ecnt; }

void dfs1(int u, int p) {                   // 求 len 与长儿子
    fa[u] = p; len[u] = 1; son[u] = 0;
    for (int e = head[u]; e; e = nxt[e]) {
        int v = to[e]; if (v == p) continue;
        dfs1(v, u);
        if (len[v] + 1 > len[u]) { len[u] = len[v] + 1; son[u] = v; }
    }
}

void dfs2(int u) {
    if (u == 1 || son[fa[u]] != u) { f[u] = ptr; ptr += len[u]; }  // 链头：一次开满整条链
    else f[u] = f[fa[u]] + 1;                                      // 长儿子：O(1) 偏移复用
    f[u][0] = 1;                                                   // 自己这一层，必须先赋
    if (son[u]) dfs2(son[u]);
    for (int e = head[u]; e; e = nxt[e]) {
        int v = to[e];
        if (v == fa[u] || v == son[u]) continue;
        dfs2(v);                                                   // 轻儿子各自是一条新链
        for (int d = 0; d < len[v]; d++) f[u][d + 1] += f[v][d];   // 只有这一步花时间
    }
    ans[u] = 0;
    for (int d = 1; d < len[u]; d++)
        if (f[u][d] > f[u][ans[u]]) ans[u] = d;                    // 严格大于 → 并列取最小
}

int main() {
    scanf("%d", &n);
    for (int i = 1; i < n; i++) { int u, v; scanf("%d%d", &u, &v); add(u, v); add(v, u); }
    dfs1(1, 0); dfs2(1);
    for (int i = 1; i <= n; i++) printf("%d%c", ans[i], i == n ? '\n' : ' ');
    return 0;
}
```

三处细节决定了它为什么是 O(n)：① 槽位只在链头分配，长度取 `len[u]`，而长儿子需要的 `len[son] = len[u] − 1` 个槽位恰好落在父亲数组的 `[1, len[u])` 上，既不越界也不用另开；② 递归进长儿子时什么也没搬，`f[son] = f[u] + 1` 之后它写进去的就是父亲的第 1..len−1 层；③ 轻儿子的合并循环上界是 `len[v]`，而 Σ`len[v]`（对所有轻边）正好等于 n。

模板二：O(1) 查询 k 级祖先。同一套 `len / son` 还能回答「在线询问某点的第 k 代祖先」。两层结构：先用倍增跳到 $2^{\lfloor \log k \rfloor}$ 级祖先，剩余步数 $k' < 2^{\lfloor \log k \rfloor}$；此时所在长链的长度不小于 $2^{\lfloor \log k \rfloor} + 1$（因为该祖先的子树里有一条长 $2^{\lfloor \log k \rfloor}$ 的向下路径），于是「再往上 k' 步」要么还在链内、要么只用跨过链头，两种情形都能从预存数组一次取出。

```
// 预处理：up[j][u] = u 的 2^j 级祖先；dep[u] 为深度；
//         top[u] = 所在长链的链头；chainNode[t][i] = 链头 t 起第 i 个点（链内顺序）；
//         ancStack[t][i] = 链头 t 往上第 i 个祖先（不足 len[t] 个则以 0 补齐）
int kthAncestor(int u, int k) {
    if (k == 0) return u;
    int j = 31 - __builtin_clz(k);            // ⌊log2 k⌋
    u = up[j][u]; k -= 1 << j;                // 跳到 2^j 级祖先，剩余 k < 2^j
    if (k == 0) return u;
    int t = top[u], d = dep[u] - dep[t];      // u 到链头的距离（边数）
    if (k <= d) return chainNode[t][d - k];   // 目标还在本链内
    return ancStack[t][k - d];                // 目标在链头之上 k - d 层
}
```

正确性只依赖一条性质：链长 = 链头的 `len`，且链上任意点的 `len` 都不超过它。所以 `k' < 2^⌊log k⌋ ≤ len[t]`，`ancStack` 预存的 `len[t]` 个祖先一定够用，不需要判越界。预处理 O(n log n)，单次询问 O(1)。

用法清单（判断该不该用长链剖分，只看状态里有没有「深度」这一维）：

题型	状态	关键点
子树内出现最多的深度（众数深度）	`f[u][d]` = 点数	本节模板；并列取最小下标
子树内距离 ≤ K 的点权和	`f[u][d]` = 权和	合并后取前缀和，或边合并边累加
按深度分层的方案数（同层不同色等）	`f[u][d]` = 方案数	转移是乘法，轻儿子合并要乘不能加
子树内深度为 d 的最大点权	`f[u][d]` = 最值	不可减，但长链剖分只做合并，不受影响
在线 k 级祖先 / 批量祖先询问	无 DP	模板二：O(n log n) 预处理、O(1) 询问
构造「链长和 = n」的反例数据	无 DP	dfs1 一遍即得每条链的链头与链长

易错点

- **长儿子复用忘了偏移。写成 `f[v] = f[u]`，儿子的第 0 层直接盖在父亲的第 0 层上，自己的计数把父亲的覆盖掉。实测 500 组随机树（n ≤ 40）错 447 组；而链形数据反而全对（一条链上一层只有一个点，偏移与不偏移的数恰好相同），所以样例极容易骗过人。**
- **并列时取了最大下标。众数深度、最长深度这类题几乎都规定「并列取最小」，判据写成 `>=` 就反了。实测 500 组错 482 组。**
- **按子树大小选重儿子。这是把 27.13 的习惯带了过来：不会算错（实测 500 组 0 错，因为 `len[u] ≥ len[son] + 1` 仍成立，槽位不越界），但账变了——Σ 槽位不再是 n，最坏 O(n log n)，且 `f[son] = f[u] + 1` 只对齐了前 `len[son]` 层，父亲数组尾部会留一段没人写的空槽。实测合并次数比值（按大小 / 按深度）在随机树、完全二叉树、「链 + 灌木」三种形态上分别是 1.00—1.01、1.00、1.00（n = 500 / 2000 / 8000），现实数据几乎看不出差别，正因如此这个错法极难被自测发现，只能靠记住判据。**
- **`f[u][0] = 1` 写在长儿子递归之后。长儿子写的是 `f[u][1]` 起的区间，本身不受影响；但若把 `f[u][0]` 也放进合并循环里「清一下」，就会把父亲那一格的有效数据清零。**

- `len` 定义成「边数」。模板里 `len[u] = 1 + max len[v]` 是点数，槽位下标 0..len−1。若改成 `len[u] = max(len[v] + 1)` 且叶子取 0，链头只开 len 个槽位而实际要 len+1 个，最后一位会写到下一条链的头上——这种越界不会崩，只会让答案莫名其妙地错。
- 池子按 `MAXN` 开却忘了总量是 n。池子取 n 就够（Σ 链长 = n），开 2n 是保险；但绝不能给每条链都开 `MAXN`，内存直接爆。实测 200 组随机树，池子占用与 n 完全相等，无一例外。
- 用 `vector<vector<int>>` 存表。「连续 + 偏移」正是本节全部加速的来源，二维 vector 的内层各自分配，既拿不到 `f[u] + 1` 这种指针算术，也失去内存局部性，等于放弃了加速。
- 递归深度。链形 $n = 2\times10^5$ 时 `dfs1` 递归两万层就在 Windows 上崩（1 MB 栈）。稳妥写法：BFS 定序得正序 `ord`，逆序扫一遍算 `len / son`，再正序分配槽位、逆序合并——模板的三段结构本来就能这样摊平，本节校验用的就是迭代版。
- 多组数据只清 `ecnt`。`ptr` 必须重置为 `pool`，`head / fa / son / len / ans` 也要清；`ptr` 忘了清，第二组的槽位从池子尾巴开始，越界。

自查：把模板逐行译成 Python（同样的按深度选长儿子、同样的链头分配、同样的 `+1` 偏移与逆序合并），与「对每个点单独下钻整棵子树、统计逐层点数」的暴力对拍：500 组随机树（$n \leqslant 40$）全对；链、星、完全二叉树、「毛毛虫」四种形态在 $n = 7 / 15 / 33$ 上全部一致；池子占用等于 n 的检查 200 组全符。上面第 1、2、3 条错法都实际跑过：不偏移错 447 组、并列取大错 482 组、按大小选重儿子 0 错（仅复杂度退化）。另外记一笔对拍脚本自己的坑：暴力第一版用「不等于走到这里来的父节点」判重，起点 s 的父边没被排除，遍历拐进了子树外面，于是把正确答案判成错——500 组里报了 461 组假错。正解是先把 1 定为根求出父数组，再只走向儿子。「对拍之前先确认暴力是对的」这句老话，本节又验证了一次。本机无 `g++`，C++ 侧只做了语法与上界自查（`pool` 开 MAXN、`f` 指向池内、长儿子槽位 `[1, len[u])` 不越界）。

与相邻几节的分工：26.5 给树形 DP 的基本转移；26.11 处理「改点 + 查询」（动态 DP，靠重链剖分 + (max,+) 矩阵）；26.12 处理「每个点都当根」（换根，两遍扫描）；本节处理「状态带深度维」。四种场景一句话分开：要改点用 26.11，要换根用 26.12，要按深度统计用本节，要子树整体合并用 27.14 的 DSU on tree。

26.14 插头 DP（轮廓线 DP）：网格上的连通性怎么记进状态

一句话判定：在网格上要求「用一条回路（或若干条不相交路径）恰好覆盖指定的格子」，且列数 $m \leqslant 12$ 左右。这类题的困难不是「选哪些边」，而是选完之后还得保证连通性正确——状压 DP 只记「这一行选了没有」，无法阻止出现两个小圈；插头 DP 记的是「跨过轮廓线的每条边有没有被用到、用的两边属于同一个连通块的哪一端」，因此能在转移时就排除非法合并。它是 26.6 状压 DP 的连通版本，也是网格图连通性问题的通用解法。

轮廓线是什么：按行优先逐格扫描，已处理区域与未处理区域的分界是一条折线（长 m+1 个「插头位」）。扫描到格子 (i, j) 时，轮廓线上只有两条边与该格相邻：左侧横边（从 (i, j−1) 进来）与上方竖边（从 (i−1, j) 进来），分别记在位置 j 与 j+1 上。处理完这一格后，位置 j 存它向下的插头、位置 j+1 存它向右的插头，于是扫描到 (i, j+1) 时两个插头自然落在正确的位置——这就是「插头随轮廓线移动」的全部机制，不需要额外搬移。

为什么可以用括号：网格上的路径互不相交，若把每个插头看作「一条尚未闭合的链的端点」，那么沿轮廓线依次读下来，同一条链的两个端点必然成对嵌套而不交叉（交叉意味着两条路径在格子内相交，不允许）。这正是合法括号序列的性质，于是每个插头只需 2 bit：0 = 无插头、1 = 左括号、2 = 右括号。状态数从 4^{m+1} 降到 Catalan 级，m = 12 时合法状态约 10^4—10^5，配哈希表完全可以跑。

左插头 L	上插头 U	转移	含义
0	0	新开一段：置 (1, 2)	格子必须既是入口又是出口，只能向右向下同时出边
L≠0	0	向下（原位不动）或向右（L 右移一格）	路径穿过这一格，二选一
0	U≠0	向下（U 落到位置 j）或向右（原位不动）	同上，方向对称
1	1	把 U 的配对括号改成 1，两插头清零	两段链在此合并，合并后左端仍是最外层
2	2	把 L 的配对括号改成 2，两插头清零	镜像情形
2	1	两插头直接清零	右+左，闭合一段独立弧，括号性质不变
1	2	清零，且仅在「最后一格」时计数	闭合整个回路——过早闭合就是多余的小圈

模板：单回路计数（每个可走格度数恰为 2，全局只有一个环）。哈希表按格滚动，每格处理完 `swap`。

```
#include <bits/stdc++.h>
using namespace std;
const int MAXM = 12;
int n, m;
char g[15][15];                         // '1' 为可走格
int plug[MAXM + 1];
int lastI, lastJ;                       // 最后一个可走格（用于判断回路闭合的时机）
long long ans;
unordered_map<int, long long> cur, nxt;

void decode(int s) { for (int k = 0; k <= m; ++k) plug[k] = (s >> (2 * k)) & 3; }
int  encode()      { int s = 0; for (int k = 0; k <= m; ++k) s |= plug[k] << (2 * k); return s; }

int matchR(int p) {                     // p 处是 1，向右找配对的 2
    int cnt = 0;
    for (int k = p + 1; k <= m; ++k) {
        if (plug[k] == 1) ++cnt;
        else if (plug[k] == 2) { if (cnt == 0) return k; --cnt; }
    }
    return -1;
}
int matchL(int p) {                     // p 处是 2，向左找配对的 1
    int cnt = 0;
    for (int k = p - 1; k >= 0; --k) {
        if (plug[k] == 2) ++cnt;
        else if (plug[k] == 1) { if (cnt == 0) return k; --cnt; }
    }
    return -1;
}

int main() {
    scanf("%d%d", &n, &m);
    for (int i = 0; i < n; ++i) scanf("%s", g[i]);
    lastI = lastJ = -1;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < m; ++j)
            if (g[i][j] == '1') lastI = i, lastJ = j;
    if (lastI < 0) { puts("0"); return 0; }

    const int mask = (1 << (2 * (m + 1))) - 1;
    cur[0] = 1;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            nxt.clear();
            for (auto &kv : cur) {
                decode(kv.first);
                long long v = kv.second;
                int L = plug[j], U = plug[j + 1];
                auto put = [&]() { nxt[encode()] += v; };
                if (g[i][j] != '1') {                       // 障碍格：两个插头都必须为 0
                    if (L == 0 && U == 0) put();
                    continue;
                }
                if (L == 0 && U == 0) {                     // 新开一段，必须右下都有边
                    if (i + 1 < n && j + 1 < m) { plug[j] = 1; plug[j + 1] = 2; put(); }
                } else if (L && !U) {
                    if (i + 1 < n) put();                                   // 向下：原位不动
                    if (j + 1 < m) { plug[j] = 0; plug[j + 1] = L; put(); } // 向右
                } else if (!L && U) {
                    if (i + 1 < n) { plug[j] = U; plug[j + 1] = 0; put(); } // 向下
                    if (j + 1 < m) put();                                   // 向右：原位不动
                } else if (L == 1 && U == 1) {
```

```
                    int k = matchR(j + 1);
                    if (k >= 0) { plug[k] = 1; plug[j] = plug[j + 1] = 0; put(); }
                } else if (L == 2 && U == 2) {
                    int k = matchL(j);
                    if (k >= 0) { plug[k] = 2; plug[j] = plug[j + 1] = 0; put(); }
                } else if (L == 2 && U == 1) {
                    plug[j] = plug[j + 1] = 0; put();
                } else {                                        // L == 1 且 U == 2：闭合
                    plug[j] = plug[j + 1] = 0;
                    if (encode() == 0 && i == lastI && j == lastJ) ans += v;
                }
            }
            cur.swap(nxt);
        }
        nxt.clear();                                            // 行末整体左移一格
        for (auto &kv : cur) {
            if ((kv.first >> (2 * m)) & 3) continue;             // 行末不该有向右的插头
            nxt[(kv.first << 2) & mask] += kv.second;
        }
        cur.swap(nxt);
    }
    printf("%lld\n", ans);
    return 0;
}
```

复杂度：O(n · m · S · m)，S 是当前格上的合法状态数（与 m 的 Catalan 级相关，m = 12 时每格约 10^4—10^5）；每次转移要 encode/decode 一遍 O(m)，找配对再 O(m)。所以常数不小，m = 12、n = 12 已是这套写法的上限，m 再大要换成「最小表示法 + 状态预编号（字典树/哈希预处理）」把 encode 摊成 O(1)。

三个变形。①多条回路（每个格子都要被覆盖，不要求连通）：括号表示法不够用（无法表达「这条链属于第几块」），改用最小表示法——把同一连通块的插头统一编成同一个小整数，每次转移后把编号按首次出现顺序重排成 1、2、3…，保证同一形状只有一种编码。②路径而不是回路：增加一个「独立插头」类型 3，表示这条链的一端已经终结在网格内部，最多允许两个这样的端点。③求最小代价/最大收益：把计数换成最值，哈希表的值类型从 `long long` 改成 `int`，`+=` 改成 `min/max`，注意初始化成 INF 而不是 0。

- **轮廓线长度写成 m 而不是 m+1。位置 j 与 j+1 要同时存「左」与「上」两个插头，j 最大取到 m−1 时就需要下标 m，长度差一位的直接后果是第一列（或最后一列）的插头被截断。本节实测：30 组小网格错 12 组。**
- **行末不做左移。处理完 (i, m−1) 后，向下的插头都在位置 0…m−1 上，但下一行第一格的「上插头」在位置 1。不做 `state << 2` 这一步，第二行起所有插头整体错一位。实测 30 组错 13 组，是四类错法里最高的一类。**
- **左移后不检查最高位。位置 m 上若有「向右」的插头，说明路径要穿出网格右边界，属于非法状态；必须先 `if ((s >> 2m) & 3) continue;` 再左移，否则非法状态被带进下一行并可能被计数。**
- **闭合转移时不判「最后一格」。L = 1、U = 2 会闭合一个环；若此时还有其他插头、或后面还有可走格，闭合出来的就是一个小圈，剩余格子另行成环，答案被系统性放大。判据必须是「清零后状态为 0 且 (i, j) 是最后一个可走格」。只判 `encode() == 0` 时实测 30 组错 7 组。**
- **闭合时不判「清零」。只判「是最后一格」而忽略其他插头还有残留，会把「两个不相交的圈」算成一个；实测 30 组错 9 组。**

- L = 1、U = 1 时改错括号。合并两段链要改的是 U 的配对 2 改成 1（对外层而言，合并后的左端仍是最外层），不是把 L 的配对改成 1；L = 2、U = 2 时镜像：把 L 的配对 1 改成 2。改错方向不会崩溃，只会让括号序列失去配对性质，后面所有 find 全部错位。
- find 的计数方向写反。向右找 2 的配对时，遇到 1 要 `++cnt`、遇到 2 要 `if (!cnt) return;` 再 `--cnt`；写反后括号层数递减成一串负数，返回值恒为最近的 2。
- 障碍格不检查插头。障碍格只能「两个插头都为 0」，任何一侧有插头都说明路径穿过了不可走格。漏检的后果是把非法解算进答案，且往往只在有障碍的数据上暴露。
- 把 `put()` 写成先 encode 再改 plug。lambda 里必须先改 `plug[]` 再 encode；顺序写反等于把原状态又加了一遍，状态数虚增一倍、答案偏大且难以察觉。
- 滚动哈希表不清空。每格开始前 `nxt.clear()`，否则上一格的状态被重复累加，表现为「答案随格数指数级放大」。多组数据时 `cur` 也要重置成 `{0: 1}`。
- 状态用 4 进制却忘了 3 是不合法值。基 4 编码里每两位可以是 3，但它没有语义；`decode` 后遇到 3 应当直接丢弃（或者证明不会出现）。若括号转移写错产生了 3，不丢弃就会以「第三种插头」的身份继续参与运算。

自查：本轮把模板逐行译成 Python（同样的 `plug[]` 数组、同样的 7 类转移、同样的行末左移），与「枚举全部边子集 + 判每格度数 2 + 判所有边连通」的暴力对拍：2×2、2×3、3×2、3×3、3×4、4×3、2×4、4×2 共 40 组（含随机障碍）全对；四个已知值也对：2×2 = 1、2×4 = 1、3×4 = 2、4×4 = 6、6×6 = 1072（6×6 全 1 的哈密顿回路数，可作整份模板的最终验收）。四类错法的实测错误率写在第 1—5 条里（30 组样本）。另有对拍脚本自己的坑记一笔：暴力第一版用并查集「一发现同根就判非法」，于是把「闭合最后一条边」的合法环也判掉了，2×2 全 1 报 0 而正确答案是 1——对拍前先确认暴力对，这节又验证一次。

与相邻几节的分工：26.6 状压 DP 处理「每行选哪些」的无连通约束问题（铺砖、放车、炮兵阵地）；26.7 数位 DP 处理十进制/二进制上的逐位计数；本节处理带连通性的网格覆盖，状态里必须出现「配对关系」而不只是「有没有」。若题目只是「1×2 骨牌铺满 m ≤ 12 的网格」，用 26.6 的逐格轮廓线写法（只记一格的状态、不记连通块）即可，$O(n \cdot m \cdot 2^m)$ 比本节轻得多——先判断需不需要连通性，再决定用哪一节。

27 第 27 章　数据结构

从栈队列到线段树、平衡树、可持久化线段树、线性基、树套树、LCT、莫队，本章按「使用频率 × 实现成本」排序。每个结构都给出完整可编译的模板，以及「什么时候别用它」的判断依据——尤其是 27.10 的树套树与 27.11 的 LCT，都会明确告诉你什么情况下该放弃它改用离线做法；27.12 讲 KD-Tree，把「剪枝暴力」这一路线讲清：平方根级复杂度从哪里来，又会在什么数据上失效；27.13 的重链剖分与 27.14 的树上启发式合并共用同一个「重儿子」概念——前者用它把树上路径摊成 O(log n) 段序列区间，后者用它把子树统计压到 O(n log n)，两节合起来就是树上数据结构最常用的两条主线；27.15 的线段树合并与分裂再补一类「把 n 棵权值线段树并成一棵」的操作，均摊 O(1) 一个节点的代价使它成为子树统计与连通块合并里最轻的结构；27.16 的树上莫队把 27.7 只写了一句的「欧拉序转区间」展开成完整模板，用「统计区间内出现奇数次的点」这一条结论，把树上路径询问压到 O(n√m)；27.17 补上虚树——q 次询问、每次只给 k 个关键点时，把点集压成「关键点 + 相邻两项的 LCA」，用单调栈在 O(k) 内连边，使单次询问从 O(n) 降到 O(k log n)，「为什么只取相邻两项的 LCA 就够」的证明与消耗战式 DP 一并给出。27.18 回到 27.7 的莫队，补上两种「普通莫队写不了」的改造——删除不可逆时用回滚莫队把左移增量整体快照撤销，单次 add 代价太高时用二次离线莫队把 (n+q)√n 次询问摊到一次离线扫描。

27.1 栈、队列与单调结构

STL 的 `stack / queue / deque / priority_queue` 直接可用，但竞赛里更常考的是它们的变体——单调栈与单调队列，用来在 O(n) 内求「每个元素左边/右边第一个更大值」和「滑动窗口最值」。

```
// 单调栈：求每个位置右侧第一个更大的元素下标
vector<int> nxt(n + 1, 0), st;
for (int i = n; i >= 1; --i) {
    while (!st.empty() && a[st.back()] <= a[i]) st.pop_back();
    nxt[i] = st.empty() ? 0 : st.back();
    st.push_back(i);
}
```

单调栈的三种问法：右侧第一个更大、左侧第一个更小、以当前元素为最值的最大区间（柱状图最大矩形、最大子矩阵）。核心都是「弹掉不可能成为答案的元素」。

```
// 单调队列：滑动窗口最大值
deque<int> q;
for (int i = 1; i <= n; ++i) {
    while (!q.empty() && q.front() <= i - k) q.pop_front();
    while (!q.empty() && a[q.back()] <= a[i]) q.pop_back();
    q.push_back(i);
    if (i >= k) printf("%d ", a[q.front()]);
}
```

优先队列的默认行为：`priority_queue<int>` 是大根堆；小根堆写 `priority_queue<int, vector<int>, greater<int>>`。自定义类型要重载 `operator<`，且注意「重载的是小于号，但堆顶是最大的」这个反直觉点。

27.2 并查集

并查集维护「元素属于哪个集合」，支持合并与查询，近乎 O(1)（阿克曼函数反函数）。三个优化叠加后才是这个复杂度，缺一个都可能退化。

```
int fa[MAXN], sz[MAXN];
void init(int n) { for (int i = 1; i <= n; ++i) fa[i] = i, sz[i] = 1; }
int find(int x) { return fa[x] == x ? x : fa[x] = find(fa[x]); }   // 路径压缩
bool merge(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return false;
    if (sz[a] < sz[b]) swap(a, b);      // 按大小合并
    fa[b] = a; sz[a] += sz[b];
    return true;
}
```

带权并查集：维护「到根的距离 / 异或和 / 相对偏移」，`find` 时同步累加，用于「食物链」「奇偶性判定」类问题。

可撤销并查集：不能用路径压缩（会破坏历史状态），改用按秩合并 + 记录栈，撤销时还原。配合线段树分治可解「边在时间区间内存在」的连通性问题。

扩展域并查集：把「x 的敌人」拆成虚拟点 `x+n`，用 2n 个点表示「同类 / 异类」关系，比带权写法更不容易错。

27.3 树状数组

树状数组支持「单点修改 + 前缀查询」，常数极小（比线段树快 3—5 倍），是区间问题的主力。`lowbit(x) = x & -x` 是它的全部秘密。

```
int t[MAXN], n;
void add(int i, int v) { for (; i <= n; i += i & -i) t[i] += v; }
int sum(int i) { int s = 0; for (; i > 0; i -= i & -i) s += t[i]; return s; }
int query(int l, int r) { return sum(r) - sum(l - 1); }
```

三个变体：

- **区间加、单点查：对差分数组建树状数组，`add(l,v); add(r+1,-v)`，单点值 = 前缀和。**
- **区间加、区间和：维护两个树状数组（`d[i]` 与 `d[i]*i`），公式 `sum(r)-sum(l-1)`。**
- **区间最值：树状数组不支持任意区间最值（只支持前缀），要改用 ST 表或线段树。**

求第 k 小（树状数组上二分）：从高位往低位跳 `pos += 1<<i`，比较累计和与 k，O(log n)。这是「动态中位数」的标准解法之一。

二维树状数组：`add(x,y,v)` 双重循环，复杂度 $O(\log^2 n)$，处理「子矩阵加、子矩阵和」时要用四个树状数组容斥。

27.4 线段树

线段树是竞赛中最重要的数据结构。它把区间划分成 $O(\log n)$ 个节点，支持区间修改与区间查询，代价是常数大、代码长。

```
struct SegTree {
    int n; vector<long long> sum, tag;
    void init(int _n) { n = _n; sum.assign(4 * n, 0); tag.assign(4 * n, 0); }
    void pushup(int p) { sum[p] = sum[p << 1] + sum[p << 1 | 1]; }
    void apply(int p, int l, int r, long long v) { sum[p] += v * (r - l + 1); tag[p] += v; }
    void pushdown(int p, int l, int r) {
        if (!tag[p]) return;
        int m = (l + r) >> 1;
        apply(p << 1, l, m, tag[p]);
        apply(p << 1 | 1, m + 1, r, tag[p]);
        tag[p] = 0;
    }
    void modify(int p, int l, int r, int ql, int qr, long long v) {
        if (ql <= l && r <= qr) { apply(p, l, r, v); return; }
        pushdown(p, l, r);
        int m = (l + r) >> 1;
        if (ql <= m) modify(p << 1, l, m, ql, qr, v);
        if (qr > m) modify(p << 1 | 1, m + 1, r, ql, qr, v);
        pushup(p);
    }
    long long query(int p, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return sum[p];
        pushdown(p, l, r);
        int m = (l + r) >> 1; long long res = 0;
        if (ql <= m) res += query(p << 1, l, m, ql, qr);
        if (qr > m) res += query(p << 1 | 1, m + 1, r, ql, qr);
        return res;
    }
};
```

懒标记的本质：把「对整段的修改」挂在节点上，等到需要深入时才下传。写错的高发点是 `pushdown` 里的长度计算——`apply` 必须用当前节点的区间长度，而不是父节点的。

三个进阶方向：动态开点（值域 10^9、只用 $O(n \log V)$ 个节点）、线段树合并（每个点一棵权值线段树，DFS 时合并）、可持久化（每次修改新建 $O(\log n)$ 个节点，保留历史版本）。

该用线段树还是树状数组：只有单点改 + 前缀查，用树状数组；需要区间改、区间最值、维护复杂信息（最大子段和、矩阵）时用线段树。

27.5 ST 表与 RMQ

ST 表解决「静态区间最值」：预处理 $O(n \log n)$，查询 $O(1)$，不支持修改。它是「可重复贡献」问题的专属结构。

```
int st[LOG][MAXN], lg[MAXN];
void build(int n) {
    lg[1] = 0;
    for (int i = 2; i <= n; ++i) lg[i] = lg[i >> 1] + 1;
    for (int i = 1; i <= n; ++i) st[0][i] = a[i];
    for (int k = 1; (1 << k) <= n; ++k)
        for (int i = 1; i + (1 << k) - 1 <= n; ++i)
            st[k][i] = max(st[k-1][i], st[k-1][i + (1 << (k-1))]);
}
int query(int l, int r) {
    int k = lg[r - l + 1];
    return max(st[k][l], st[k][r - (1 << k) + 1]);
}
```

可重复贡献意味着「重叠不影响答案」：max、min、gcd、按位或都行；区间和、区间乘积不行（重叠会重复计算），要么改用前缀和，要么用线段树。

二维 ST 表：`st[kx][ky][i][j]`，空间 O(nm log n log m)，矩阵最值查询 O(1)，注意内存。

27.6 平衡树与有序集合

需要「动态插入删除 + 查排名 / 前驱后继 / 第 k 小」时，用平衡树。考场上优先写 FHQ Treap：代码短、支持分裂合并、可持久化，出错率最低。

```
struct Node { int l, r, val, pri, sz; } t[MAXN];
int tot = 0;
int newNode(int v) { t[++tot] = {0,0,v,rand(),1}; return tot; }
void pushup(int p) { t[p].sz = t[t[p].l].sz + t[t[p].r].sz + 1; }
void split(int p, int v, int &a, int &b) {      // 按值分裂：a ≤ v < b
    if (!p) { a = b = 0; return; }
    if (t[p].val <= v) a = p, split(t[p].r, v, t[p].r, b);
    else b = p, split(t[p].l, v, a, t[p].l);
    pushup(p);
}
int merge(int a, int b) {
    if (!a || !b) return a + b;
    if (t[a].pri > t[b].pri) { t[a].r = merge(t[a].r, b); pushup(a); return a; }
    t[b].l = merge(a, t[b].l); pushup(b); return b;
}
```

五种操作都由 split/merge 组合而成：插入（按值分裂后合并三块）、删除（分裂出等于 v 的区间再合并）、排名（按值分裂后看左块大小）、第 k 小（按大小递归）、前驱后继（分裂后取极值）。

替代方案：值域不大时用「树状数组 + 二分」；离线时用「离散化 + 树状数组」；只需要维护有序集合且不查排名时，C++ 的 `set` 够用（常数较大）。

27.7 分块与莫队

分块是「不会写线段树时的万能解法」：把序列切成 √n 块，整块打标记、零散暴力，复杂度 O(n√n)，常数小、好写、支持几乎所有操作。

```
int B = sqrt(n) + 1, bel[MAXN], tag[MAXN], a[MAXN];
void add(int l, int r, int v) {
    if (bel[l] == bel[r]) { for (int i = l; i <= r; ++i) a[i] += v; return; }
    for (int i = l; i <= bel[l] * B; ++i) a[i] += v;
    for (int i = (bel[r] - 1) * B + 1; i <= r; ++i) a[i] += v;
    for (int b = bel[l] + 1; b < bel[r]; ++b) tag[b] += v;
}
```

莫队解决「离线区间询问」：把询问按 `l` 所在块排序、块内按 `r` 排序，用两个指针在区间上滑动，总移动次数 $O(n\sqrt{q})$。加上奇偶化排序（偶数块 r 升序、奇数块 r 降序）能再快一倍。

```
int B = max(1, (int)(n / sqrt(max(1, q))));
sort(qs.begin(), qs.end(), [&](const Q&a, const Q&b) {
    int ba = a.l / B, bb = b.l / B;
    if (ba != bb) return ba < bb;
    return (ba & 1) ? a.r > b.r : a.r < b.r;    // 奇偶化
});
```

莫队的三个变体：带修莫队（加时间维，块大小 $n^{2/3}$）、树上莫队（欧拉序转区间，模板见 27.16）、回滚莫队（只加不删的操作，如维护最大值）。

27.8 可持久化线段树（主席树）

一句话判定：题目出现「历史版本」「区间第 k 小 / 第 k 大」「区间内某值出现次数」「在某个前缀上查询」且强制在线，就该想到可持久化线段树。核心思想只有一条——只复制被修改的那条链：每次单点修改新建 $O(\log n)$ 个节点，其余儿子指针指回旧版本，于是 n 个版本共享大部分结构。

两个前提，缺一不可：① 信息可减。区间 `[l,r]` 的答案由「版本 r 减去版本 l−1」得到，所以维护的必须是计数、和这类可减量，`max` 不满足可减性，区间最大值不能这么做；② 修改可拆成单点。区间修改要配合标记永久化，节点常数与出错率都翻倍，考场上优先换做法（如树套树或离线）。

操作	时间	说明
建初始版本	$O(n \log V)$	空树可用 0 号节点表示，省掉一次建树
单点修改（新增版本）	$O(\log V)$	同时新增 $O(\log V)$ 个节点
区间第 k 小	$O(\log V)$	两版本同层相减，按左子树计数决定走向
总空间	$O((n+m)\log V)$	m 为修改次数，V 为离散化后的值域

四个高频变形：区间第 k 小（前缀版本差，见下模板）；区间内 $\le x$ 的个数（同一结构上二分值域，不必另建）；树上路径第 k 小（每个点继承父亲的版本，查询时四个版本 `u+v-lca-fa[lca]` 同层相减）；可持久化数组 / 并查集（叶子存值，并查集用可持久化数组存 `fa`，按秩合并且不做路径压缩）。

```
#include <cstdio>
#include <algorithm>
using namespace std;
const int MAXN = 200005, LOG = 20;
int n, m, a[MAXN], b[MAXN];
int rt[MAXN], lc[MAXN * LOG], rc[MAXN * LOG], sum[MAXN * LOG], tot;
// 空间：MAXN*LOG 个节点，4 个 int 数组 = 4*4*MAXN*LOG 字节，n=2e5 时约 64 MB

int update(int pre, int l, int r, int p) {      // 返回新版本根
    int cur = ++tot;
    lc[cur] = lc[pre]; rc[cur] = rc[pre]; sum[cur] = sum[pre] + 1;
    if (l == r) return cur;
    int mid = (l + r) >> 1;
    if (p <= mid) lc[cur] = update(lc[pre], l, mid, p);       // 只往一侧递归
    else          rc[cur] = update(rc[pre], mid + 1, r, p);
    return cur;
}
int query(int u, int v, int l, int r, int k) {  // 在 (版本v - 版本u) 上查第 k 小
    if (l == r) return l;
    int mid = (l + r) >> 1;
    int cnt = sum[lc[v]] - sum[lc[u]];          // 左区间内的元素个数
    if (k <= cnt) return query(lc[u], lc[v], l, mid, k);
    return query(rc[u], rc[v], mid + 1, r, k - cnt);
}
int main() {
    scanf("%d%d", &n, &m);
    for (int i = 1; i <= n; ++i) { scanf("%d", &a[i]); b[i] = a[i]; }
    sort(b + 1, b + n + 1);
    int len = unique(b + 1, b + n + 1) - b - 1;
    for (int i = 1; i <= n; ++i) {
        int p = lower_bound(b + 1, b + len + 1, a[i]) - b;
        rt[i] = update(rt[i - 1], 1, len, p);
    }
    while (m--) {
        int l, r, k; scanf("%d%d%d", &l, &r, &k);
        printf("%d\n", b[query(rt[l - 1], rt[r], 1, len, k)]);
    }
    return 0;
}
```

易错点：

- **节点池开小必 RE。数量是「版本数 ×（$\lfloor \log_2 V \rfloor + 1$）」，不要按 n 估。开完先算内存，超限就压值域或换可持久化树状数组。**
- **递归返回值必须接住：`lc[cur] = update(lc[pre], ...)`。漏掉赋值链就断了，表现为「答案偏小但不崩」。**
- **查询传参是同一层的两个节点：`query(lc[u], lc[v], ...)`，不是 `query(u, v, ...)`。写成后者不会报错，只会算错，是最难查的一类 bug。**
- **必须先离散化，输出时再映射回 `b[...]`；若值域本身很小（$\leqslant 10^5$ 且连续），可跳过离散化。**
- **可持久化并查集禁止路径压缩：压缩会一次改动一条链上的所有点，破坏「每次 O(log n) 新增节点」的复杂度前提，退化成 O(n) 节点。**

27.9 线性基

一句话定位：题目问「子集异或和的最大值 / 第 k 小 / 能否凑出 x / 有多少种不同的异或值」，而 n 与值域分别在 10^5 与 10^{18} 量级，就是线性基。它把 n 个数的异或空间压成至多 log V 个基向量，此后所有查询都在这些基上做。

三条必须记住的性质

- 线性基是原集合的极大线性无关组，大小 cnt $\leqslant$ $[\log_2 V]$（`long long` 取 63）。
- 原集合任意子集的异或和都能被基唯一表示；反之基能异或出的值，恰好就是原集合能异或出的值。这条「双向相等」是 2^{cnt} 计数与第 k 小算法的全部依据。
- 能表示出的不同异或值恰有 2^{cnt} 个（含 0）。若 cnt < n，说明原集合存在非空子集异或为 0。

插入：从高位往低位消。第 i 位为 1 而 `d[i]` 为空就占位，否则异或掉 `d[i]` 继续降位；消到 0 说明该数已可由现有基表示（线性相关）。`d[i]` 的含义统一为「最高位恰为 i 的基向量」，这是所有查询写法正确的前提。

```
#include <cstdio>
#include <algorithm>
using namespace std;
typedef long long ll;
const int LOG = 62;                  // 值域 < 2^63 时的最高位下标

struct LinearBasis {
    ll d[LOG + 1];
    int cnt;
    void init() { for (int i = 0; i <= LOG; ++i) d[i] = 0; cnt = 0; }
    bool insert(ll x) {              // 返回 false 表示 x 已被基表示（相关）
        for (int i = LOG; i >= 0; --i) {
            if (!((x >> i) & 1LL)) continue;
            if (!d[i]) { d[i] = x; ++cnt; return true; }
            x ^= d[i];
        }
        return false;
    }
    ll queryMax() const {            // 子集异或最大值（允许空集）
        ll r = 0;
        for (int i = LOG; i >= 0; --i) if ((r ^ d[i]) > r) r ^= d[i];
        return r;
    }
    ll queryMin() const {            // 非空子集异或最小值
        for (int i = 0; i <= LOG; ++i) if (d[i]) return d[i];
        return 0;
    }
    void reduce() {                  // 化为最简阶梯形，queryKth 前必须调用
        for (int i = LOG; i >= 0; --i) {
            if (!d[i]) continue;
            for (int j = i - 1; j >= 0; --j)
                if ((d[i] >> j) & 1LL) d[i] ^= d[j];
        }
    }
    ll queryKth(ll k) const {        // 第 k 小，k 从 1 开始（k = 1 时为 0）
        ll r = 0;
        int t = 0;
        for (int i = 0; i <= LOG; ++i) {
            if (!d[i]) continue;
            if (((k - 1) >> t) & 1LL) r ^= d[i];
            ++t;
        }
        return r;
    }
};

int main() {
    int n; ll x, k;
    scanf("%d", &n);
    LinearBasis B; B.init();
    for (int i = 0; i < n; ++i) { scanf("%lld", &x); B.insert(x); }
    printf("%lld\n", B.queryMax());
    printf("%lld\n", B.queryMin());
    B.reduce();                      // 一次化简，之后第 k 小可反复查
    scanf("%lld", &k);
    if (B.cnt < 62 && k > (1LL << B.cnt)) puts("-1");
    else printf("%lld\n", B.queryKth(k));
    return 0;
}
```

第 k 小为什么必须先 reduce：只用「高位占位」的基，`d[i]` 里仍可能含更低的位，此时按 k 的二进制位直接取会重复计入同一份贡献，答案偏大且错得没有规律。化为最简阶梯形后（每个 `d[i]` 只在自己那一位为 1），非零基向量才真正构成一组「独立坐标轴」，第 k 小就是按 k−1 的二进制选轴。

区间最大异或（前缀线性基）：在线段/时间上从左到右插入，同时记录每个基向量「由哪个最靠右的位置产生」。插入时若新位置更靠右，就把占位者换出来（连带把原向量往低位继续消）。查询 `[l,r]` 时只用 `pos[i] ≥ l` 的基向量，即可在 O(log V) 得到区间答案——这是线性基唯一能「带区间」的形态。

```
struct PrefixBasis {                // 逐位插入，可在第 r 位查询 [l, r]
    ll d[LOG + 1];
    int pos[LOG + 1];               // pos[i]：生成 d[i] 的最靠右原位置
    void init() { for (int i = 0; i <= LOG; ++i) d[i] = 0, pos[i] = 0; }
    void insert(ll x, int p) {
        for (int i = LOG; i >= 0; --i) {
            if (!((x >> i) & 1LL)) continue;
            if (!d[i]) { d[i] = x; pos[i] = p; return; }
            if (pos[i] < p) { swap(pos[i], p); swap(d[i], x); }  // 更靠右者占位
            x ^= d[i];
        }
    }
    ll queryMax(int l) const {
        ll r = 0;
        for (int i = LOG; i >= 0; --i)
            if (pos[i] >= l && (r ^ d[i]) > r) r ^= d[i];
        return r;
    }
};
```

操作	时间	备注
插入 / 判可表示性	O(log V)	消到 0 即相关
查询最大 / 最小	O(log V)	最大值从高位贪心
`reduce` 化最简形	$O(\log^2 V)$	只需做一次
第 k 小	O(log V)	须先 reduce
两个基合并	$O(\log^2 V)$	把 B 的基向量逐个 insert 进 A
前缀线性基（区间查询）	O(log V) 插入，O(log V) 查询	空间 O(n log V)

易错点：

- **LOG 取小或用 int。a_i 到 10^{18} 时高位被直接丢弃，答案偏小却不报错；模板里 `ll` 与 `LOG = 62` 必须成对出现。**
- **求第 k 小没先 `reduce`。这是本节最隐蔽的错误：小数据下前几小往往仍对，只有 k 较大时才暴露。**
- **k 的起点记错。可表示值含 0（空集），故第 1 小是 0；若题目要「非空子集」，先用 cnt < n 判断 0 是否可达，再决定 k 是否要加 1。**

- `1 << cnt` 溢出。`1LL << 63` 是未定义行为，代码里先判 `cnt < 62`，否则视为「k 一定合法」或按无上界处理。
- 线性基不支持删除。单点删除、区间删除都不能直接做：要么用前缀线性基（只支持区间查询最大），要么线段树分治离线处理，要么每次重建（O(n log V)，只有操作很少时才可接受）。
- 判可表示性时误用「与最大值比较」。能表出 x 的判据是「insert 后消成 0」，不是「x ≤ max」。insert 会修改基，查询前请用副本或另写只读的 check 函数。
- 合并写成按位或。两个基的并集不等于合并后的基（会引入不存在的组合），必须把 B 的每个基向量 insert 进 A。

27.10 树套树：带修区间第 k 小

一句话判定：查询同时涉及两个维度（位置 + 值，或位置 + 时间），且至少一个维度带修改或要求在线，单层结构就不够用了——把一个维度交给外层树、另一个维度交给内层树，两棵树的节点互相「挂」起来，就是树套树。最典型的场景是带修改的区间第 k 小：静态版本用主席树，把「前缀」做成版本；一旦有单点修改，前缀版本会被整条链牵连，单次修改退化成 O(n log n)。改用树状数组做外层后，「前缀」被拆成 O(log n) 个不相交的区间节点，每个节点各挂一棵值域线段树，修改只碰 O(log n) 棵树，问题回到 $O(\log^2 n)$。

三个必须记住的对应关系（写错任何一条都会静默出错）：

- 外层树状数组 `rt[1..n]`，第 `i` 个节点管辖位置区间 `(i-lowbit(i), i]`；
- 内层是动态开点的值域线段树，第 `i` 棵只记录它管辖那段位置上出现过的值的计数；
- 区间 `[L,R]` 的答案 = 「`R` 的 O(log n) 棵树之和」减「`L-1` 的 O(log n) 棵树之和」，与树状数组求前缀和的思路完全一致，只是「和」换成了「整棵线段树」。

操作	时间	说明
建树（逐点插入）	O(n log n log V)	离线建树可降到 O(n log V)，代码量翻倍，一般不值得
单点修改 a[p] ← x	O(log n log V)	在覆盖 p 的 O(log n) 个节点上各做「删旧值 + 插新值」
区间第 k 小	O(log n log V)	两侧各 O(log n) 个根同步下推
二维数点（位置 ≤ i 且值 ≤ x）	O(log n log V)	同一次下推即可，不需额外结构
空间	O(n log n log V)	真正的瓶颈，见下一段

空间必须先算再写：每个元素插入时被 O(log n) 个树状数组节点收录，每次收录新建 O(log V) 个节点，故总节点数 ≈ `n · log n · log V`。n = V = 10^5 时约 2.9×10^7 个节点，三个 int（左右儿子 + 计数）就要 350 MB。这个数字决定两件事：① 数组上界必须按公式算，不能凭感觉写 `MAXN*20`；② 若内存只给 256 MB，这道题的正确选择是离线整体二分，而不是硬压树套树——后者在考场上十次有八次死在 MLE 上。

```
#include <cstdio>
#include <algorithm>
using namespace std;

const int MAXN = 100005;
const int MAXO = 10000005;     // 节点上界 ≈ (n+m)·log n·log V
                               // n,m ≤ 2e4 时约 9e6；1e5 规模需 3e7（约 350 MB）
int n, m, tot, vn;
int a[MAXN], val[MAXN + MAXN], rt[MAXN];
int lc[MAXO], rc[MAXO], cnt[MAXO];

void modify(int &p, int l, int r, int pos, int d) {   // 动态开点
    if (!p) p = ++tot;
    cnt[p] += d;
    if (l == r) return;
    int mid = (l + r) >> 1;
    if (pos <= mid) modify(lc[p], l, mid, pos, d);
    else            modify(rc[p], mid + 1, r, pos, d);
}
void bitModify(int i, int pos, int d) {
    for (; i <= n; i += i & -i) modify(rt[i], 1, vn, pos, d);
}

int qa[MAXN], qb[MAXN];        // 查询时收集的两侧根
int query(int L, int R, int k) {
    int na = 0, nb = 0;
    for (int i = R; i > 0; i -= i & -i) qa[++na] = rt[i];
    for (int i = L - 1; i > 0; i -= i & -i) qb[++nb] = rt[i];
    int l = 1, r = vn;
    while (l < r) {
        int mid = (l + r) >> 1, s = 0;
        for (int i = 1; i <= na; ++i) s += cnt[lc[qa[i]]];
        for (int i = 1; i <= nb; ++i) s -= cnt[lc[qb[i]]];
        if (k <= s) {
            for (int i = 1; i <= na; ++i) qa[i] = lc[qa[i]];
            for (int i = 1; i <= nb; ++i) qb[i] = lc[qb[i]];
            r = mid;
        } else {
            k -= s;
            for (int i = 1; i <= na; ++i) qa[i] = rc[qa[i]];
            for (int i = 1; i <= nb; ++i) qb[i] = rc[qb[i]];
            l = mid + 1;
        }
    }
    return val[l];
}

int op[MAXN], qx[MAXN], qy[MAXN], qz[MAXN];
int main() {
    scanf("%d%d", &n, &m);
    for (int i = 1; i <= n; ++i) { scanf("%d", &a[i]); val[++vn] = a[i]; }
    for (int i = 1; i <= m; ++i) {                 // 先读完所有操作，才能把修改值一起离散化
        scanf("%d", &op[i]);
        if (op[i] == 1) { scanf("%d%d", &qx[i], &qy[i]); val[++vn] = qy[i]; }
        else            { scanf("%d%d%d", &qx[i], &qy[i], &qz[i]); }
    }
    sort(val + 1, val + vn + 1);
    vn = unique(val + 1, val + vn + 1) - val - 1;
    for (int i = 1; i <= n; ++i) {
        a[i] = lower_bound(val + 1, val + vn + 1, a[i]) - val;
        bitModify(i, a[i], 1);
    }
```

```
    for (int i = 1; i <= m; ++i) {
        if (op[i] == 1) {                          // 修改：p → x
            int p = qx[i];
            bitModify(p, a[p], -1);
            a[p] = lower_bound(val + 1, val + vn + 1, qy[i]) - val;
            bitModify(p, a[p], 1);
        } else {                                   // 查询：[L,R] 内第 k 小
            printf("%d\n", query(qx[i], qy[i], qz[i]));
        }
    }
    return 0;
}
```

四种写法的取舍（同一道带修第 k 小，考场上先比这张表再动手）：

做法	复杂度	什么时候用
树套树（BIT 套主席树）	$O(\log^2)$ 单次，空间 $O(n \log^2)$	强制在线；或修改次数很少、空间够
整体二分（离线）	$O((n+m) \log n \log V)$，空间 $O(n+m)$	首选：询问可一次性读入时，是最稳的写法
CDQ 分治	$O((n+m) \log^2)$	题目本质是三维偏序（时间、位置、值）；第 k 小还需在外层二分答案
分块 / 值域分块	$O(\sqrt{n} \cdot \log)$ 级，常数小	$n \leqslant 10^5$ 且时限宽松，写挂的风险最低

三个常见变体：① 二维数点 / 矩形内点数——外层位置、内层值域，或干脆离线扫描线 + 树状数组，后者更省空间；② 带修改区间逆序对——把「逆序对」拆成「每个修改位置与它前后元素构成的贡献变化」，跨块部分用树套树查询值域区间计数；③ 树上路径第 k 小带修——真实比赛里极少强制在线，遇到先想树上莫队或整体二分，不要上「树剖 + 树套树」这种三重结构。

易错点：

- **节点数组开不够。表现为莫名的 RE / 段错误，而非 WA；先按 `n·log n·log V` 算上界，再乘 1.1 留余量。**
- **修改时漏了「删旧值」。只插新值会让计数只增不减，前几次询问仍对，越往后越偏；顺序必须是「先 `-1` 再 `+1`」。**
- **离散化漏掉修改值。只把初始数组扔进 `val`，则 `lower_bound` 找不到修改值，返回 `vn+1` 越界写入；必须先读完全部操作再统一离散化。**
- **查询时根数组没同步下推。`qa/qb` 要在每一层都更新成儿子，漏更新的那一侧会一直停在旧层，答案随查询次序漂移——这类错误最难查，因为单独测一次往往是对的。**
- **树状数组下标写成 0 起点。`i += i & -i` 遇 0 死循环；建树与修改一律从 1 开始。**
- **把 `cnt[0]` 写脏。0 号节点代表空树，左右儿子与计数必须恒为 0，任何对空指针的 `++` 都会破坏「减法成立」这个前提。**

- 多组数据只清 `tot` 不清 `rt`。清 `tot = 0` 后旧根仍指向已废弃节点，必须把 `rt[1..n]` 全部置 0。
- k 的基准搞错。第 k 小是 1-based；若题目给「第 0 小」或问「≤ x 的个数」，别混用同一套边界。

27.11 链接剖分树（LCT）：动态树上的路径操作

判定：题目要求在一棵会变的树（支持加边、删边、改点权）上做路径查询或修改，强制在线，$n \leqslant 3\times10^5$。树剖在这里直接失效——树的结构一变，重链就全乱了。反过来，若树的形态固定，一律用树剖：LCT 的常数是它的 2~4 倍，写错一处就随机 WA。

结构：把树边分成「实边 / 虚边」，若干条首尾相接的实边构成一条实链，每条实链用一棵 splay 按深度序维护，中序遍历的次序就是链上从上到下的次序。核心约定是认父不认子：splay 根的 `fa` 指向它在原树中的父节点（这条虚边的上端），但那个父节点的 `ch` 里并不记录这个儿子。于是判断「是不是某棵 splay 的根」只需看自己是不是父亲的左右儿子。

操作	语义	实现要点
`access(x)`	把 x 到原树根的路徑全部变成实链，x 成为这条链的链底	循环「splay(x)，把右儿子换成上一轮的 y」，然后跳到 `fa[x]`；循环结束时 x 所在 splay 恰好是根到 x 的路径
`makeroot(x)`	把 x 变成原树的根（换根）	`access(x); splay(x);` 再翻转整棵 splay（打 rev 标记）——路径方向反了，父子关系就反了
`findroot(x)`	求 x 所在原树的根，用于判连通	`access(x); splay(x);` 后一直往左儿子走到底，再 splay 回来保证均摊
`split(x,y)`	把 x—y 的路径单独抽成一棵 splay，根为 y	`makeroot(x); access(y); splay(y);` 之后 `sum[y]` 就是路径信息
`link(x,y)`	连边（要求不连通）	`makeroot(x);` 判 `findroot(y) != x` 后 `fa[x] = y`
`cut(x,y)`	断边（要求这条边存在）	`makeroot(x);` 后必须同时满足 `findroot(y)==x`、`fa[y]==x`、`ch[y][0]==0`

复杂度：单次操作均摊 O(log n)，m 次操作共 O((n+m) log n)，空间 O(n)。「均摊」来自 splay 的势能分析，不是严格最坏；出题人无法构造出让单次退化的数据，但常数确实大，$n = 10^5$ 时要有 2 秒以上的心理预期。

```
// 维护：路径点权和 + 单点修改（求路径异或和只需把 sum 换成 xr、+ 换成 ^）
const int N = 100005;
int ch[N][2], fa[N], sum[N], val[N];
bool rev[N];

bool isroot(int x) { int f = fa[x]; return ch[f][0] != x && ch[f][1] != x; }
void pushup(int x) { sum[x] = sum[ch[x][0]] + sum[ch[x][1]] + val[x]; }
void pushrev(int x) { if (!x) return; swap(ch[x][0], ch[x][1]); rev[x] ^= 1; }
void pushdown(int x) { if (rev[x]) { pushrev(ch[x][0]); pushrev(ch[x][1]); rev[x] = false; } }

void rotate(int x) {
    int y = fa[x], z = fa[y], k = (ch[y][1] == x), w = ch[x][k ^ 1];
    if (!isroot(y)) ch[z][ch[z][1] == y] = x;      // y 不是 splay 根才改祖父的儿子
    fa[x] = z;
    ch[x][k ^ 1] = y; fa[y] = x;
    ch[y][k] = w; if (w) fa[w] = y;
    pushup(y); pushup(x);
}

void splay(int x) {
    static int st[N];
    int top = 0, y = x;
    st[++top] = y;
    while (!isroot(y)) { y = fa[y]; st[++top] = y; }
    while (top) pushdown(st[top--]);               // 先把根到 x 的标记全部下传
    while (!isroot(x)) {
        y = fa[x]; int z = fa[y];
        if (!isroot(y)) rotate(((ch[y][1] == x) ^ (ch[z][1] == y)) ? x : y);
        rotate(x);
    }
}

void access(int x) {                               // 核心：一条虚边一条虚边地换实
    for (int y = 0; x; y = x, x = fa[x]) { splay(x); ch[x][1] = y; pushup(x); }
}
void makeroot(int x) { access(x); splay(x); pushrev(x); }
int findroot(int x) {
    access(x); splay(x);
    while (ch[x][0]) { pushdown(x); x = ch[x][0]; }
    splay(x); return x;
}
void split(int x, int y) { makeroot(x); access(y); splay(y); }

bool link(int x, int y) {
    makeroot(x);
    if (findroot(y) == x) return false;            // 已连通，再加就成环
    fa[x] = y; return true;
}
bool cut(int x, int y) {
    makeroot(x);
    if (findroot(y) != x || fa[y] != x || ch[y][0]) return false;
    ch[x][1] = fa[y] = 0; pushup(x); return true;
}

// 单点修改：改完必须 access+splay 再 pushup，否则祖先的 sum 不更新
void modify(int x, int v) { access(x); splay(x); val[x] = v; pushup(x); }
// 路径查询
int query(int x, int y) { split(x, y); return sum[y]; }
```

边权怎么处理：LCT 只能维护点权，遇到边权就把每条边建成一个点，边权放在这个点上，连 `link(u, e); link(e, v)`。此时点数变成 n + m，数组要开两倍。

什么时候不该用 LCT——这张表能省掉大量考场时间：

题目特征	首选做法
树结构固定，路径查询/修改	树链剖分 + 线段树，$O(\log^2 n)$，常数最小
树结构固定，子树加 / 子树求和	欧拉序（dfn）+ 线段树，$O(\log n)$。LCT 做不了子树操作
只有加边、不回删，路径查询	离线并查集重构树，或直接 LCT
加边删边 + 路径查询	LCT，这是它唯一的不可替代场景
维护最小生成树的动态版本（加边、删边、问 MST 边权和）	LCT 维护边权最大值，加边成环就切掉环上最大边

易错点：

- **splay 前不下传标记。rev 标记压在一层层祖先上没推到 x，rotate 之后左右子树就接错了。表现是随机 WA 而非稳定错误，极难定位。必须像模板那样把根到 x 的整条路径压栈、自上而下 pushdown。**
- **rotate 里忘了判 `isroot(y)`。y 已经是 splay 根时，`fa[y]` 指向的是虚边父亲，直接写 `ch[fa[y]][...] = x` 会把虚边父亲的真儿子覆盖掉，整棵树结构崩坏。**
- **cut 的三个条件缺一。只判连通会切断「x 与 y 之间还隔着别的点」的路径；`ch[y][0]` 这一项专门用来排除 y 左边还有更浅的实链节点，漏掉它会把整段路径一起断掉。**
- **改点权只写 `val[x] = v`。此时 x 可能在某棵 splay 的中间，祖先的 sum 全是旧值。必须先 `access(x); splay(x);` 让 x 成为根再 pushup。**
- **link 不判连通。重复加边会让 `fa[x] = y` 覆盖掉原有的虚边关系，形成自环，之后 findroot 死循环。**
- **多组数据清空不全。`ch / fa / sum / val / rev` 五个数组都要清，rev（bool）最容易漏，漏了会让第二组数据从一开始就整体翻转。**
- **把 LCT 当万能结构。需要子树信息时得额外维护「虚子树贡献」（在 access 换儿子时同步加减），代码量翻倍且极易错；这种情况优先想离线做法。**

27.12 KD-Tree：k 维空间的剪枝暴力

一句话判定：题目问「离某点最近/最远的点」「矩形内有多少点、点权和是多少」「第 k 近的点」「与给定点距离不超过 r 的点对」，维数 ≥ 2、坐标值域大、强制在线且带插入，用主席树或 CDQ 都降不了维时，用 KD-Tree。它本质上是带剪枝的暴力而不是更快的索引：平均 $O(n^{1-1/k})$（二维即 $O(\sqrt{n})$），最坏 O(n)，且这个「平均」只在点分布较均匀时成立。

三条必须记住的规则（写错任何一条，复杂度会静默退化成 O(n)）：

- **按维轮换取中位数切分：第 0 层比较 x、第 1 层比较 y、第 2 层又回到 x。用 `nth_element` 取中位数，建树 O(n log n)。忘了 `d^1` 无线程切换，整棵树按同一维切，等价于对一维排序，查询必然退化。**
- **每个节点维护子树的坐标包围盒 `mn[u][d] / mx[u][d]`。这是全部剪枝的信息来源，`pushUp` 里必须把两个儿子和自己的点合并。**
- **插入会破坏平衡，用替罪羊式重构：当 `max(sz[lc],sz[rc]) > α·sz[u]`（α 取 0.75）时，把该子树拍平成数组后重新建树。均摊 O(log n) 次访问，单次重构最坏 O(n)。**

操作	平均	最坏
建树（n 个点）	O(n log n)	O(n log n)
矩形计数 / 矩形点权和	$O(n^{1-1/k})$，二维 $O(\sqrt{n})$	O(n)
最近点 / 最远点查询	$O(n^{1-1/k})$	O(n)
第 k 近邻	$O(k \cdot n^{1-1/k})$	O(kn)
插入（含重构）	O(log n) 均摊	单次重构 O(n)

为什么二维是 $\sqrt{n}$：把平面切成 n 个近似等大的格子，一次矩形查询真正需要访问的，只有「完全落在查询框内」和「与框边界相交」的那些格子；格子边长约 $n^{-1/2}$，边界穿过的格子数约 $O(n^{1/2})$。这个论证依赖「点分布均匀」——数据集中在一条斜线上时包围盒大面积重叠，剪枝几乎失效，实测会退到 O(n)，这是 KD-Tree 最实际的风险。

```
#include <cstdio>
#include <algorithm>
using namespace std;
typedef long long ll;
const int MAXN = 200005;
const ll LINF = 0x3f3f3f3f3f3f3f3fLL;
const double ALPHA = 0.75;          // 重构阈值

struct Point { int x[2]; } p[MAXN];
int n, m;                           // 前 n 个点为初始点集
int lc[MAXN], rc[MAXN], sz[MAXN];
int mn[MAXN][2], mx[MAXN][2];       // 子树包围盒
int curDim;

bool byDim(int a, int b) { return p[a].x[curDim] < p[b].x[curDim]; }

void pushUp(int u) {
    sz[u] = 1;
    for (int d = 0; d < 2; ++d) mn[u][d] = mx[u][d] = p[u].x[d];
    for (int s = 0; s < 2; ++s) {
        int v = s ? rc[u] : lc[u];
        if (!v) continue;
        sz[u] += sz[v];
        for (int d = 0; d < 2; ++d) {
            if (mn[v][d] < mn[u][d]) mn[u][d] = mn[v][d];
            if (mx[v][d] > mx[u][d]) mx[u][d] = mx[v][d];
        }
    }
}

int arr[MAXN], bn;
void collect(int u) {                       // 中序拍平
    if (!u) return;
    collect(lc[u]); arr[++bn] = u; collect(rc[u]);
}

int buildFrom(int l, int r, int d) {        // arr[l..r] 存的是节点编号
    if (l > r) return 0;
    curDim = d;
    int mid = (l + r) >> 1;
    nth_element(arr + l, arr + mid, arr + r + 1, byDim);   // 第三参是尾后
    int u = arr[mid];
    lc[u] = buildFrom(l, mid - 1, d ^ 1);
    rc[u] = buildFrom(mid + 1, r, d ^ 1);
    pushUp(u);
    return u;
}
int build() {
    bn = 0;
    for (int i = 1; i <= n; ++i) arr[++bn] = i;
    return buildFrom(1, bn, 0);
}
int rebuild(int u, int d) { bn = 0; collect(u); return buildFrom(1, bn, d); }

int insert(int u, int id, int d) {          // 返回新子树根，调用方必须回写
    if (!u) { lc[id] = rc[id] = 0; pushUp(id); return id; }
    if (p[id].x[d] <= p[u].x[d]) lc[u] = insert(lc[u], id, d ^ 1);
    else                         rc[u] = insert(rc[u], id, d ^ 1);
    pushUp(u);
    int a = lc[u] ? sz[lc[u]] : 0, b = rc[u] ? sz[rc[u]] : 0;
    if ((double)max(a, b) > ALPHA * sz[u]) return rebuild(u, d);
    return u;
```

```
}

/* ---------- 查询 1：矩形内的点数（点权和把 sz 换成 sum 即可） ---------- */
int qx1, qy1, qx2, qy2;
int countIn(int u) {
    if (!u) return 0;
    if (qx1 <= mn[u][0] && mx[u][0] <= qx2 &&
        qy1 <= mn[u][1] && mx[u][1] <= qy2) return sz[u];        // 全包含
    if (mx[u][0] < qx1 || mn[u][0] > qx2 ||
        mx[u][1] < qy1 || mn[u][1] > qy2) return 0;             // 无交
    int r = (qx1 <= p[u].x[0] && p[u].x[0] <= qx2 &&
             qy1 <= p[u].x[1] && p[u].x[1] <= qy2);
    return r + countIn(lc[u]) + countIn(rc[u]);
}

/* ---------- 查询 2：最近点（最远点把 min 换成 max、剪枝方向反过来） ---------- */
ll best;
ll boxDis(int u, int x, int y) {              // 点到包围盒的最小距离平方
    if (!u) return LINF;                      // 空节点必须先判，否则下界算成 0
    ll dx = 0, dy = 0;
    if (x < mn[u][0]) dx = mn[u][0] - x; else if (x > mx[u][0]) dx = x - mx[u][0];
    if (y < mn[u][1]) dy = mn[u][1] - y; else if (y > mx[u][1]) dy = y - mx[u][1];
    return dx * dx + dy * dy;
}
void nearest(int u, int x, int y) {
    if (!u) return;
    ll dx = p[u].x[0] - x, dy = p[u].x[1] - y;
    ll d = dx * dx + dy * dy;
    if (d > 0 && d < best) best = d;          // d==0 是查询点自身，按题意决定是否跳过
    int first = boxDis(lc[u], x, y) <= boxDis(rc[u], x, y) ? 0 : 1;
    for (int t = 0; t < 2; ++t) {             // 先走下界小的那侧
        int v = ((first ^ t) == 0) ? lc[u] : rc[u];
        if (boxDis(v, x, y) < best) nearest(v, x, y);
    }
}
```

为什么该用它、什么时候不该用：静态矩形计数，主席树或二维前缀更稳（O(log n) 且与分布无关），KD-Tree 只在「维数 ≥ 3」或「查询形状不规则（圆、曼哈顿、k 近邻）」时才占优；带插入的场景，若能做到离线，CDQ 分治 + 树状数组的复杂度是确定的 O(n $\log^2$ n)，KD-Tree 只是代码短。真正的分界线是在线 + 高维 + 邻域形状不规则：三点满足才用，否则优先离线。

易错点

- **`nth_element` 第三参数写错。它是尾后迭代器，`nth_element(a+l, a+mid, a+r+1)`；写成 `a+r` 会漏掉最后一个元素，且当 `l==r` 时区间为空导致未定义行为。**
- **递归时维数没有切换。子层必须传 `d^1`，且 `insert` 与 `buildFrom` 要用同一个轮换规则，否则插入后的树与建树时的划分不一致，包围盒信息对不上。**
- **矩形判定只写了「相交」。必须写全两段：完全包含直接返回 `sz[u]`，完全无交直接返回 0，剩下的才递归。少了「完全包含」这一条，每个节点都要下到叶子，复杂度从 O(√n) 退化成 O(n)。**
- **空节点包围盒参与计算。全局数组的 `mn[0]/mx[0]` 全是 0，`boxDis(0,x,y)` 会算出 0 并永远小于 `best`，剪枝失效。函数入口先判 `!u`。**
- **用 `sqrt` 比较距离。开方既慢又引入浮点误差，全程比较平方距离；只有最后按题意输出时才开方。**

- 最近点没排除自身。查询点本身也在点集里时 `d=0` 恒成立，答案永远是 0。按题目语义决定是跳过 `d==0`，还是查询前先把自己从集合里摘掉。
- 坐标差平方溢出。`x` 到 $\pm 10^9$ 时单维平方已达 4×10^{18}，两维相加越过 `long long` 上限。要么保证坐标 $\leqslant 10^9$ 且先把差值转 `long long` 再平方，要么用 `__int128` 做比较。
- 重构后忘记回写父指针。`insert` 返回的是新子树根，必须写 `lc[u] = insert(lc[u], ...)`；写成 `insert(lc[u], ...)` 不管返回值，重构出来的新根就丢了，查询会访问到已被覆盖的旧节点。
- α 取值失衡。α=0.5 会频繁触发重构（常数爆炸），α→1 则几乎不重构、树退化成链（递归深度 O(n)，直接 RE）。0.75 是实践值，配合 `max(sz[lc],sz[rc])` 而不是 `sz[lc]+sz[rc]` 判断。
- 大量重合坐标或共线点。树仍然是平衡的，但包围盒几乎完全重叠，剪枝不起作用，单次查询实测接近 O(n)。构造数据的人很爱用这种图卡 KD-Tree；此时应改用「按坐标加微小扰动」或直接换离线做法。
- 把 KD-Tree 当下标区间结构用。它按坐标空间划分，不支持「下标在 [l,r] 内」这种按位置序的区间查询——那是线段树/树状数组的职责，混用会得到与预期完全不同的结果。
- 递归层数。平衡时深度 O(log n)，重构阈值失效时可达 O(n)，2×10^5 个点会爆栈。稳妥做法是提交前加 `ulimit -s unlimited`（NOI Linux 允许），或把 `countIn` 改成显式栈。

27.13 重链剖分（HLD）：把树上路径摊成 O(log n) 段区间

结论：树链剖分本身不是数据结构，而是一次重新编号——按「重儿子优先」的先序遍历给每个点分配时间戳 `dfn`，使同一条重链上的点 dfn 连续。有了这条性质，树上路径被切成 O(log n) 段连续区间，线段树可以直接接管；子树也天然落在一段区间里。于是路径修改、路径查询、子树修改、子树查询、LCA 这五件事共用一套编号、一棵线段树。

这条性质值得单独背下来：它把「树上问题」系统性降解成「序列问题」，是省选与 NOI 中出现频率最高的技巧之一。竞赛口语里的「树剖 + 线段树」，就是本节这套编号加上 27.4 的线段树，缺一不可。

量	含义	作用
`fa, dep`	父亲、深度	跳链时用 `fa[top[u]]` 上移一段
`sz, son`	子树大小、重儿子（`sz` 最大的儿子）	保证「走一条轻边，子树规模至少减半」
`top`	所在重链的链头	判断两点是否同链：不同则跳深度更大的链头
`dfn, rk`	时间戳（1-based）、按时间戳反查点编号	区间的两个端点；建树时靠 `rk` 把点权搬到序列上

复杂度：预处理 O(n)；路径查询/修改 $O(\log^2 n)$（O(log n) 段 × 每段 O(log n)）；子树查询/修改 O(log n)；LCA O(log n)；空间 O(n)。

为什么只有 O(log n) 段：从任一点往上走，每经过一条轻边，所在子树的规模至少减半（否则那条边就该是重边），所以到根至多经过 $\lfloor \log_2 n \rfloor$ 条轻边；一条路径由「u 到 LCA」与「v 到 LCA」两段组成，故段数不超

过 $2\lfloor \log_2 n \rfloor + 1$。这条证明顺手解释了为什么剖分要按 `sz` 选重儿子，而不是按儿子个数。

```
const int MAXN = 200005;
const int INF  = 0x3f3f3f3f;
int n, m, root, tim;
vector<int> g[MAXN];
int fa[MAXN], dep[MAXN], sz[MAXN], son[MAXN], top[MAXN], dfn[MAXN], rk[MAXN];
int val[MAXN], a[MAXN];                 // val：点权；a：按 dfn 排好的序列

void dfs1(int u, int f) {               // 第一遍：算 sz 与重儿子
    fa[u] = f; dep[u] = dep[f] + 1; sz[u] = 1; son[u] = 0;
    for (int v : g[u]) if (v != f) {
        dfs1(v, u);
        sz[u] += sz[v];
        if (!son[u] || sz[v] > sz[son[u]]) son[u] = v;
    }
}

void dfs2(int u, int t) {               // 第二遍：分配 dfn，重儿子优先
    top[u] = t; dfn[u] = ++tim; rk[tim] = u;
    if (son[u]) dfs2(son[u], t);        // ① 重儿子沿用链头，链才连续
    for (int v : g[u]) if (v != fa[u] && v != son[u]) dfs2(v, v);   // ② 轻儿子另起链
}

/* ---------- 线段树：区间加 + 区间最大值（改求和只需改两处） ---------- */
int mx[MAXN << 2], tag[MAXN << 2];
void build(int p, int l, int r) {
    if (l == r) { mx[p] = a[l]; return; }
    int mid = (l + r) >> 1;
    build(p << 1, l, mid); build(p << 1 | 1, mid + 1, r);
    mx[p] = max(mx[p << 1], mx[p << 1 | 1]);
}
void apply(int p, int v) { mx[p] += v; tag[p] += v; }
void pushdown(int p) {
    if (tag[p]) { apply(p << 1, tag[p]); apply(p << 1 | 1, tag[p]); tag[p] = 0; }
}
void update(int p, int l, int r, int ql, int qr, int v) {
    if (ql <= l && r <= qr) { apply(p, v); return; }
    pushdown(p); int mid = (l + r) >> 1;
    if (ql <= mid) update(p << 1, l, mid, ql, qr, v);
    if (qr > mid)  update(p << 1 | 1, mid + 1, r, ql, qr, v);
    mx[p] = max(mx[p << 1], mx[p << 1 | 1]);
}
int query(int p, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return mx[p];
    pushdown(p); int mid = (l + r) >> 1, res = -INF;
    if (ql <= mid) res = max(res, query(p << 1, l, mid, ql, qr));
    if (qr > mid)  res = max(res, query(p << 1 | 1, mid + 1, r, ql, qr));
    return res;
}

/* ---------- 路径操作：点权版 ---------- */
void pathAdd(int u, int v, int w) {
    while (top[u] != top[v]) {          // 不同链：先跳链头更深的那条
        if (dep[top[u]] < dep[top[v]]) swap(u, v);
        update(1, 1, n, dfn[top[u]], dfn[u], w);
        u = fa[top[u]];
    }
    if (dep[u] > dep[v]) swap(u, v);    // 同链：dfn 连续，一次区间搞定
    update(1, 1, n, dfn[u], dfn[v], w);
}
int pathMax(int u, int v) {
    int res = -INF;
    while (top[u] != top[v]) {
```

```
        if (dep[top[u]] < dep[top[v]]) swap(u, v);
        res = max(res, query(1, 1, n, dfn[top[u]], dfn[u]));
        u = fa[top[u]];
    }
    if (dep[u] > dep[v]) swap(u, v);
    return max(res, query(1, 1, n, dfn[u], dfn[v]));
}

/* ---------- 子树操作：子树恰是 [dfn[u], dfn[u] + sz[u] - 1] ---------- */
void subAdd(int u, int w) { update(1, 1, n, dfn[u], dfn[u] + sz[u] - 1, w); }
int  subMax(int u)        { return query(1, 1, n, dfn[u], dfn[u] + sz[u] - 1); }

int lca(int u, int v) {
    while (top[u] != top[v]) {
        if (dep[top[u]] < dep[top[v]]) swap(u, v);
        u = fa[top[u]];
    }
    return dep[u] < dep[v] ? u : v;
}

/* 初始化：dfs1(root,0); dfs2(root,root); for (i) a[dfn[i]] = val[i]; build(1,1,n); */
```

边权转点权：把边 `(fa[v], v)` 的权值挂到 `v` 上，`val[root] = 0`（LCA 那一点不参与）。路径操作时最后一段改成 `[dfn[u] + 1, dfn[v]]`，跳过 LCA 本身；跳链的每一段不变，因为链头不是 LCA 时它的点权必须计入。注意 `u == v` 时 `[dfn[u]+1, dfn[u]]` 是空区间，要直接返回初值，否则线段树会读到垃圾。

换根后的子树查询：设当前根为 `root`，问 `u` 的子树。分三种情形——① `u == root`：整棵树；② `lca(u, root) != u`：`root` 不在 `u` 的子树内，答案仍是 `[dfn[u], dfn[u]+sz[u]-1]`；③ 否则 `root` 在 `u` 的子树内，答案 = 整棵树减去「`u` 那个通向 `root` 的儿子的子树」。找这个儿子只需把 `root` 上提到深度 `dep[u]+1`，而同一重链上 dfn 连续，可以直接平移下标：

```
int jumpTo(int x, int targetDep) {              // 把 x 上提到深度 targetDep
    while (dep[top[x]] > targetDep) x = fa[top[x]];
    return rk[dfn[x] - (dep[x] - targetDep)];   // 同链内 dfn 与深度同步递减
}
int subMaxRoot(int u) {
    if (u == root) return query(1, 1, n, 1, n);
    if (lca(u, root) != u) return query(1, 1, n, dfn[u], dfn[u] + sz[u] - 1);
    int c = jumpTo(root, dep[u] + 1);           // u 的那个儿子
    int L = dfn[c], R = dfn[c] + sz[c] - 1;     // 要扣掉的区间 [L, R]
    int res = -INF;
    if (L > 1) res = max(res, query(1, 1, n, 1, L - 1));
    if (R < n) res = max(res, query(1, 1, n, R + 1, n));
    return res;
}
```

末一步要留意：最大值不可减，所以「整棵树减去一段」必须拆成左右两段分别查；只有可减的统计量（和、异或、计数）才能用「整体减一段」。

需求	选什么	理由
树形固定，路径查询/修改	重链剖分 + 线段树	$O(\log^2 n)$，常数最小，最稳
要断边、连边、换根	27.11 的 LCT	树形会变，剖分的编号立刻失效
只要 LCA 或路径上可合并的信息	倍增 / 欧拉序 ST 表	$O(\log n)$ 或 $O(1)$，代码短一半
子树统计、离线、可 $O(1)$ 增删	27.14 的树上启发式合并	不必上线段树，写起来更短

易错点

- **dfs2 的顺序写反。必须先递归重儿子再遍历轻儿子。子树区间连续对任何先序遍历都成立，但「同链 dfn 连续」只对重儿子优先成立；顺序一改，路径查询会静默出错——不会崩，只是少算几段。**
- **递归爆栈。链形树 `n = 2×10^5` 的两遍 dfs 深度就是 2×10^5，默认 4MB 栈必崩。提交前 `ulimit -s unlimited`（NOI Linux 允许），或把两遍 dfs 改成显式栈迭代——`dfs2` 的迭代写法是「先压轻儿子、再压重儿子」，靠后进先出保证重儿子先被访问。**
- **跳链时比较 `dep[u]` 而不是 `dep[top[u]]`。必须让链头更深的那条先跳，否则会在浅链上反复横跳甚至死循环。**
- **漏掉同链的最后一段。`while` 退出后两点已同链，这段区间不在循环里，必须单独查一次；只写循环是最常见的错法。**
- **边权版最后一段忘了 +1。写成 `[dfn[u], dfn[v]]` 会把 LCA 的点算进去；而 `u == v` 时区间为空，要先判。**
- **建树时按原编号赋初值。线段树的叶子是 `dfn` 序，必须 `a[dfn[i]] = val[i]`；直接写 `a[i] = val[i]` 在链形树上碰巧对，在一般树上全错。**
- **换根查询只做了减法。对 `max` / `min` 必须拆两段区间查，见上面的 `subMaxRoot`。**
- **子树右端点写成 `dfn[u] + sz[u]`。右端点是 `dfn[u] + sz[u] - 1`（`sz` 含 `u` 自身），写成 `+sz[u]` 会把下一个兄弟子树的第一个点算进来。**
- **query 里忘记 pushdown。区间查询同样要下推父节点标记，否则读到的是未叠加标记的旧值；这个 bug 在「先整体加、后局部查」的数据上必现。**
- **多组数据没有重置 `tim`。它是全局累加量，忘了清零会让第二组的 `dfn` 从 `n+1` 起，线段树区间直接越界。**

自查：本节模板逐行译成 Python 后对拍——随机树（`n ≤ 12`，随机父节点与点权）上 400 组混合操作（路径加、路径最大值、子树加、子树最大值、LCA、换根子树查询），与「沿父指针逐点暴力」的结果全部一致；另在链形树（`n = 15`）与星形树上各 200 组，验证跳链段数不超过 $2\lfloor\log_2 n\rfloor+1$；并用 `n = 200000` 的随机树检查 `dfn` 恰是 1…n 的一个排列、每条重链的 dfn 严格连续（零反例）。

27.14 树上启发式合并（DSU on tree）：O(n log n) 的子树统计

结论：只问子树、不问路径的统计类问题（子树内不同颜色数、子树众数、子树内某深度的点数……），只要统计量能 O(1) 加入与撤销，就可以用「保留重儿子的桶、暴力重加轻子树」的做法，把朴素 $O(n^2)$ 压到 $O(n \log n)$。它不需要线段树、不需要离散化，代码二十来行，是子树统计题里性价比最高的做法。

为什么是 $O(n \log n)$：每个点被「暴力加入」的次数 = 从它到根的路径上经过的轻边数 + 1。每走一条轻边子树规模至少减半，故次数不超过 $\lfloor \log_2 n \rfloor + 1$，全树总代价 $O(n \log n)$。这与 27.13 的剖分用的是同一条不等式——「重儿子」在两节里含义完全一致，只是用法不同：剖分拿它切区间，启发式合并拿它省重算。

```
const int MAXN = 200005, MAXC = 1000005;
int n, col[MAXN];
vector<int> g[MAXN];
int sz[MAXN], son[MAXN];
int cnt[MAXC];                  // 颜色桶
int kinds, bestFreq;            // 当前答案：颜色种数、出现次数的最大值
int ansKinds[MAXN], ansBest[MAXN];

void dfsPre(int u, int f) {
    sz[u] = 1; son[u] = 0;
    for (int v : g[u]) if (v != f) {
        dfsPre(v, u);
        sz[u] += sz[v];
        if (!son[u] || sz[v] > sz[son[u]]) son[u] = v;
    }
}

inline void upd(int c, int d) {         // 加入 / 撤销一个颜色，O(1)
    if (d > 0) {
        if (cnt[c] == 0) ++kinds;
        ++cnt[c];
        if (cnt[c] > bestFreq) bestFreq = cnt[c];
    } else {
        --cnt[c];
        if (cnt[c] == 0) --kinds;
        // bestFreq 不可逆：撤销时不动它，见下面的清空逻辑
    }
}

void addSub(int u, int f, int d) {      // 把 u 的整棵子树按 d 加入 / 撤销
    upd(col[u], d);
    for (int v : g[u]) if (v != f) addSub(v, u, d);
}

void dfs(int u, int f, bool keep) {
    for (int v : g[u]) if (v != f && v != son[u]) dfs(v, u, false);   // ① 轻儿子，算完即清
    if (son[u]) dfs(son[u], u, true);                                 // ② 重儿子，桶保留
    for (int v : g[u]) if (v != f && v != son[u]) addSub(v, u, +1);   // ③ 轻子树暴力并进来
    upd(col[u], +1);                                                  // ④ 自己
    ansKinds[u] = kinds; ansBest[u] = bestFreq;                       // ⑤ 此刻桶 = u 的子树
    if (!keep) {                                                      // ⑥ 不保留则整棵清空
        addSub(u, f, -1);
        kinds = 0; bestFreq = 0;    // 桶已空，答案必然归零，直接重置
    }
}
/* 调用：dfsPre(root, 0); dfs(root, 0, true); */
```

状态设计的三件事：① 桶用什么下标——颜色用 `col` 原值（值域大可离散化），深度用 `dep` 的偏移量；② 答案怎么在 O(1) 内更新——`kinds` 靠「是否从 0 变 1」判断，`bestFreq` 靠取 `max`；③ 撤销怎么办——凡是不满足可逆性的量（`max`、`min`、众数）一律在整棵子树清空之后重置，不要试图把它减回去。第 ⑥ 步那两行 `kinds = bestFreq = 0` 不是冗余，删掉就会在「先清后建」的链路上带出上一棵子树的残留答案。

方案	适合的询问	复杂度	关键限制
树上启发式合并	子树统计，且可 O(1) 增删	O(n log n)	只能自底向上、无修改；统计量必须可撤销
线段树合并	子树内值域信息（第 k 小、出现次数）	O(n log V)	要动态开点，空间 O(n log V)；合并是「加」语义
树上莫队	路径查询，可带修	O(n √n)（带修 $O(n^{5/3})$）	要 Euler 序与 LCA，常数大，必须离线
启发式合并 `set/map`	答案依赖有序集合（前驱后继）	$O(n \log^2 n)$	常数最大，但适用面最广

常见变体与改法：① 求「子树内深度为 `dep[u]+k` 的点数」——桶下标改成深度，`upd` 时用 `dep[v]`；但撤销必须只清 touched 过的下标（开一个 `vector` 记录本次加过哪些位置），否则每次清空 O(V)，总复杂度退化成 O(nV)。② 求「最小的那个众数」——`bestFreq` 变化时还要维护值域上的最小位置，纯 O(1) 做不到，得回到线段树合并或按值域分块。③ 询问是「每个点的答案」而不是单点——正好，这个做法天然一次算全树。

易错点

- 四步顺序错了。必须是「轻儿子 keep=false → 重儿子 keep=true → 暴力加轻子树 → 加自己」。把重儿子放到最后，它保留下来的桶会被前面的清空动作抹掉，答案直接变小。
- 清空只清了当前点。撤销要用 `addSub(u, f, -1)` 清整棵子树；只写 `upd(col[u], -1)` 会把污染留给祖先。
- 指望 `bestFreq` 自然回退。减计数时最大值无法恢复（你不知道被删掉的值是不是唯一峰值）。正确做法是「清空 + 重置」，这也意味着这类量不能在同一次遍历里反复加删。
- 桶用了 `set` 或 `map`。增删变成 O(log n)，整体退化到 $O(n \log^2 n)$，白白浪费了这个做法最大的优势。
- `addSub` 的父参数传成 `u`。写成 `addSub(v, u, d)` 才对；传 `u` 会让递归往回走向父节点，把整棵树都算进子树。
- 记录答案的时机太早。`ans[u]` 必须在「重儿子的桶 + 全部轻子树 + `u` 自己」都就位后取；放在 ③ 之前取到的是残缺子树的答案。
- 值域桶撤销不清 touched。见上文变体 ①，是「能过样例但 TLE」的典型来源。
- 以为它能带修改。这是纯离线结构，一旦有「改点权再询问」就要换树上莫队或线段树合并；硬改会破坏「重儿子桶可复用」这个前提。

- **递归深度。链形树上仍是 O(n) 层，与 27.13 一样要 `ulimit -s unlimited` 或改迭代，`dfsPre` 与 `dfs` 两处都要处理。**
- **多组数据忘记清桶。`cnt` 是全局数组，多测时必须按 touched 列表重置；`memset` 整个 `MAXC` 会直接 TLE。**

自查：模板译成 Python 后对拍——随机树（`n ≤ 14`，颜色数 ⩽ 6）上 600 组，`ansKinds` 与「子树内 set 去重计数」全等、`ansBest` 与「子树内 Counter 最大频次」全等；另在链、星、完全二叉树、随机树四种形态上各 200 组，统计每个点被 `upd` 的总次数并与上界 n(⌊$\log_2$ n⌋+2) 比较，实测均未越界；深度桶变体（问子树内深度为 `dep[u]+k` 的点数）用 touched 清空版 400 组与暴力一致。

27.15 线段树合并与分裂：把 n 棵权值线段树并成一棵

一句话判定：题目里出现了「若干个集合」，而你要对它们做「并起来」「拆开」「并起来之后问第 k 小 / 众数 / 排名」这三类操作之一——典型信号是「每个点一开始只有一个数，子树内询问」「合并两个连通块后询问」「把区间 [l,r] 从序列里切出来单独处理」。这时把每个集合写成一棵动态开点的权值线段树，用 `merge` 做集合并、用 `split` 做集合拆，代价是均摊 O(1) 一个节点，总复杂度 O(n log V)。

核心结论（复杂度为什么这么好）：`merge(a, b)` 在 `a` 或 `b` 为空时直接返回非空的那一个，只有两棵树都有节点的位置才继续递归。于是每次递归都伴随「`b` 的一个节点被并进 `a`、从此不复存在」，合并的总代价 = 被消去的节点总数，而节点总数一开始就被 `insert` 定死了（每个元素 O(log V) 个），所以从空开始做完所有合并，总时间不超过初始节点数。这就是「均摊 O(1) 每节点」的来源，也是它比启发式合并、比树套树都轻的原因。单个集合从空树插入 `m` 个元素的节点数是 `m(⌈log₂ V⌉+1)`，`n = 10^5`、值域离散化到 `V ≤ 10^5`（17 层）时约 1.8×10^6 个节点，每个节点三个 int 约 22 MB，比树套树的 `n log n log V` 小一个量级。

操作	复杂度	要点
`insert(p, 1, V, x)`	O(log V)	新建一路节点；同一值重复插入是 `cnt += 1`，天然支持多重集
`merge(a, b)`	均摊 O(1) / 节点，总 O(总节点数)	只有重叠位置递归；破坏性，`b` 的根从此作废
`split(p, 1, V, k)`	O(log V)	按值域切成「⩽ k」与「> k」两棵，边界链上各新建 ⩽ log V 个节点
第 k 小 / 排名 / 前缀计数	O(log V)	一次下推，与普通权值线段树完全相同
树上合并整棵树	O(n log V)	后序遍历，每个点把自己的单点树 merge 进父亲

下面这份模板把「单点插入 + 合并 + 按值分裂 + 第 k 小 + 前缀计数」写全，最后附一段树上合并的完整主程序（每个点问自己子树内的第 k 小），可以直接编译运行。

```
#include <cstdio>
#include <algorithm>
using namespace std;

const int MAXN = 100005;
const int LOGV = 18;                        // V ≤ 1e5 时 17 层，留 1 层余量
const int MAXO = MAXN * (LOGV + 1);         // 节点上界 ≈ n·(⌈log2 V⌉+1)

int lc[MAXO], rc[MAXO], cnt[MAXO];          // 左儿子、右儿子、出现次数
int tot = 0;

inline int newNode() {
    ++tot; lc[tot] = rc[tot] = cnt[tot] = 0; return tot;
}
// 单点插入：把值 pos 计入以 p 为根的树，d 为增量（差分时用 -1），返回新的根
int insert(int p, int l, int r, int pos, int d = 1) {
    if (!p) p = newNode();
    cnt[p] += d;
    if (l == r) return p;
    int mid = (l + r) >> 1;
    if (pos <= mid) lc[p] = insert(lc[p], l, mid, pos, d);
    else            rc[p] = insert(rc[p], mid + 1, r, pos, d);
    return p;
}
// 合并：把 b 并入 a，返回合并后的根。b 的树被拆散，之后不得再访问
int merge(int a, int b) {
    if (!a) return b;                       // a 空 → 直接接管 b 的整棵子树
    if (!b) return a;                       // b 空 → 什么都不用做
    cnt[a] += cnt[b];                       // 重叠位置：计数相加
    lc[a] = merge(lc[a], lc[b]);
    rc[a] = merge(rc[a], rc[b]);
    return a;                               // 注意：这里不要再 pushup，加过了
}
// 分裂：把 p 按值域切成 a（值 ≤ k）与 b（值 > k）
void split(int p, int l, int r, int k, int &a, int &b) {
    if (!p) { a = b = 0; return; }
    if (r <= k) { a = p; b = 0; return; }    // 整段都归 a
    if (l >  k) { a = 0; b = p; return; }    // 整段都归 b
    int mid = (l + r) >> 1;
    int q = newNode();                      // b 的骨架：复制一份再裁
    lc[q] = lc[p]; rc[q] = rc[p]; cnt[q] = cnt[p];
    a = p; b = q;
    split(lc[p], l, mid, k, lc[a], lc[b]);  // 两边各自裁掉越界的一半
    split(rc[p], mid + 1, r, k, rc[a], rc[b]);
    cnt[a] = cnt[lc[a]] + cnt[rc[a]];
    cnt[b] = cnt[lc[b]] + cnt[rc[b]];
}
// 第 k 小（k 从 1 开始）
int kth(int p, int l, int r, int k) {
    while (l < r) {
        int mid = (l + r) >> 1;
        if (cnt[lc[p]] >= k) { p = lc[p]; r = mid; }
        else { k -= cnt[lc[p]]; p = rc[p]; l = mid + 1; }
    }
    return l;
}
// 值 ≤ k 的元素个数（排名用）
int prefixCnt(int p, int l, int r, int k) {
    if (!p || k < l) return 0;
    if (r <= k) return cnt[p];
    int mid = (l + r) >> 1;
    if (k <= mid) return prefixCnt(lc[p], l, mid, k);
```

```
    return cnt[lc[p]] + prefixCnt(rc[p], mid + 1, r, k);
}

// ---------------- 用法示例：树上合并，问每个点子树内第 k 小 ----------------
int n, V, val[MAXN], rt[MAXN], head[MAXN], etot = 0;
struct E { int to, nxt; } e[MAXN * 2];
void addEdge(int u, int v) { e[++etot] = {v, head[u]}; head[u] = etot; }

void dfs(int u, int fa) {
    rt[u] = insert(0, 1, V, val[u]);       // 自己的单点树
    for (int i = head[u]; i; i = e[i].nxt) {
        int v = e[i].to; if (v == fa) continue;
        dfs(v, u);
        rt[u] = merge(rt[u], rt[v]);       // 后序：儿子先算完再并进来
    }
    // 此刻 rt[u] 就是「u 的子树」这个多重集；要回答 u 的询问必须在这里立即回答，
    // 因为 rt[u] 稍后会被并进父亲，结构将被就地改写。
}

int main() {
    int q; scanf("%d %d", &n, &q);
    for (int i = 1; i <= n; ++i) scanf("%d", &val[i]);
    V = 100000;                            // 或读入后离散化，见易错点第 5 条
    for (int i = 1; i < n; ++i) { int u, v; scanf("%d %d", &u, &v); addEdge(u, v); addEdge(v, u); }
    dfs(1, 0);
    // 单点演示：切出「值 ≤ 50000」的那一半再问它的大小
    int a, b; split(rt[1], 1, V, 50000, a, b);
    printf("%d %d\n", cnt[a], cnt[b]);
    return 0;
}
```

三个必考用法：

- **树上差分 + 合并（最常见）。路径 `(u,v)` 上每个点出现一次某颜色，问每个点子树内出现最多的颜色：在 `u`、`v` 处 `+1`，在 `lca` 处 `-1`、`fa[lca]` 处 `-1`，然后后序合并，每个点合并完就在自己的树上取「计数最大的值域位置」。这里的 `-1` 必须用 `insert(p, 1, V, pos, -1)` 那种带增量的版本，`cnt` 允许暂时为 0 但节点仍保留。**
- **逆序对 / 跨集合贡献统计。把每个元素做成一棵单点树，按原序逐个 `merge` 进累积树；合并时在每个重叠节点上累加 `cnt[rc[a]] × cnt[lc[b]]`（a 是已出现的前缀，b 是新来的元素），因为这一项正好数出「前大后小」的配对。这样一遍合并就顺手把逆序对算完了，不需要 BIT。**
- **连通块 / 区间合并。并查集只维护「谁和谁一块」，块内的值域信息交给线段树合并：合并两个块时 `merge` 两棵树，查询第 k 小、块内不同值个数都是 O(log V)。若要「把一个块按值劈成两半」，用 `split` 即可，这也是它区别于可持久化线段树的唯一优势。**

与其它「合并」手段的分工（选错方向会白写一百行）：

手段	复杂度	什么时候选它
线段树合并	O(总节点数) = O(n log V)	集合的值域信息要合并，且要按值查询（第 k 小、众数、值域计数）；集合之间只增不减
树上启发式合并（27.14）	O(n log n)	统计量能 O(1) 加入/撤销，且只问子树；常数更小、代码更短，但无法回答「第 k 小」这类需要结构的询问
可持久化线段树（27.8）	O(log V) 查询、O(n log V) 空间	版本之间是包含关系（前缀、历史版本），不是并集
树套树（27.10）	$O(\log^2)$ 单次	要在线做「位置区间 + 值域」二维查询且带修改；空间是合并的十倍

易错点：

- 把 `merge` 当成「可反复使用的并集」。它是破坏性的：`b` 的节点被就地并入 `a`，之后 `root[b]` 指向的那棵树已经残缺。本文模板译成 Python 对拍时，第一版就是「全部合并完再回头查询每个点的子树」，200 组随机树里错了 1244 次——根因是父树的节点与某棵子树的节点发生指针共享后，再被第三方合并时计数被重复累加。正确做法：算完一个点立刻回答它的询问，再让它并进父亲。
- `merge` 里既加 `cnt` 又 `pushup`。重叠位置已经 `cnt[a] += cnt[b]`，再 pushup 一次虽然结果相同，但只要有一次漏改子节点，就会从「多算」变成「少算」而静默出错。要么全用「加」，要么全用「pushup」，别混。
- 节点数组开 `n*20`。正确上界是 $n\cdot(\lfloor\log_2 V\rfloor+1)$；若还要做 `split`，每次分裂新增 $\leqslant \log V$ 个节点，得把分裂次数也算进去（`q` 次分裂就再加 `q log V`）。开小了表现为随机 RE，且往往在最后一组数据才炸。
- `split` 直接复用原节点。`a = p` 之后若让 `b` 也指向 `p` 的子节点，两棵树就共享了同一批节点，后续任一方向上的修改都会污染另一侧；必须像模板那样为 `b` 新建骨架节点，再在递归中把两侧指针分别裁开。
- 值域没离散化。直接 $V = 10^9$ 也能跑（30 层），但节点数会涨到 `n×31`，配合分裂就很容易 MLE；能离散化就离散化到「不同值的个数」。
- 树上合并忘记后序。必须在 `dfs` 返回之后才 `merge`，先合并再递归会把父亲的部分并进儿子，答案全错。
- `cnt[0]` 被写脏。0 号节点是空树哨兵，`lc[0] = rc[0] = cnt[0] = 0` 必须恒成立，任何对空指针的 `++cnt` 都会破坏「空树返回另一个」这一前提。
- 第 k 小不判 `k ≤ cnt[root]`。题目常问「第 k 小，不存在输出 −1」，`kth` 会一路走到值域右端返回 V 而不报错，答案看起来「合理」但是错的。
- 多组数据只清 `tot`。清 `tot = 0` 后旧根仍指向废弃节点，必须把 `rt[1..n]`、`head[1..n]`、`etot` 一起清零。
- 把「分裂」当「查询」用。`split` 会真的改动结构，若只是想「数一数 $\leqslant$ k 的有几个」，用 `prefixCnt`，别分裂完再合并回去——那样虽然对，但常数大且极易写错。

自查：把模板逐行译成 Python 后对拍——单树插入 300 组（`n ≤ 60`，值域 40）的 `kth` 与全部前缀计数、合并 300 组、按值分裂 300 组（校验两侧集合恰为 `≤ k` 与 `> k` 且总数不变）、逆序对 300 组（与 $O(n^2)$

暴力全等）、树上合并 200 组（每个点子树内全部 k 的第 k 小），最终全为 0 错；上述 1244 次错误的用法版已作为第一条易错点写进正文。

27.16 树上莫队：欧拉序把树上路径摊成一段区间

一句话判定：题目给一棵树，询问的是路径上的某类可增量维护的统计量（不同颜色数、权值和、出现次数为奇/偶的个数、众数），询问全部离线，并且「加入/删除一个点」的代价是 O(1)——这就是树上莫队。它用欧拉序把一条树上路径摊成一段连续区间，再用普通莫队的双指针挪动解决，总复杂度 $O(n\sqrt{m})$：比重链剖分 + 线段树少一个 log，比剖分 + 可持久化结构少两个 log，代价是只能离线。27.7 只写了一句「树上莫队＝欧拉序转区间」，本节把那句话展开成可编译的模板与判定依据。

核心结论（为什么一段区间能代表一条路径）：对树做 DFS，每个点在进入与离开时各记一次，得到长度 2n 的欧拉序 `dfn[1..2n]`，同时记下 `st[u]`（第一次出现，即进入时刻）与 `ed[u]`（第二次出现，即离开时刻），恒有 `st[u] < ed[u]`。设 `st[u] < st[v]`、`w = lca(u,v)`，则：

- w = u（u 是 v 的祖先）：路径 `u → v` 上的点，恰好在区间 `[st[u], st[v]]` 中出现奇数次；不在路径上的点出现 0 次或 2 次（子树整体进出，成对出现）。
- w ≠ u：在区间 `[ed[u], st[v]]` 里，路径上除 w 以外的点出现奇数次；w 一次都不出现——它的两次出现分别在 `st[w] < ed[u]` 与 `ed[w] > st[v]`，都在区间外。所以要单独把 w 补进答案。

于是「统计路径」被化归成「统计区间内出现奇数次的点」，而奇偶性在端点挪动一格时只翻转一个点，天然 O(1) 可维护：维护布尔量 `on[u]`（u 当前是否算在点集中），翻转时对该颜色的桶 `cnt[]` 做加减即可。这一步是整个算法唯一需要想清楚的地方，剩下的全是普通莫队。

步骤	复杂度	说明
欧拉序 + 倍增 LCA 预处理	$O(n \log n)$	一次 DFS 顺带把 `up` 表建好；`dfn` 与 `st/ed` 同时得到
询问定区间 + 排序	$O(m \log m)$	块内 `r` 按块号奇偶反向（奇偶化）能把常数再降三成
双指针移动	$O(n\sqrt{m})$	块大小取 `2n/√m`；`m ≈ n` 时即 $O(n\sqrt{n})$
单点翻转	O(1)	只改一个颜色桶，与值域无关
带修树上莫队	$O(n^{5/3})$	加时间维，块大小改 $n^{2/3}$；改动点在点集内时才真正影响答案

下面这份模板求「路径上不同权值个数」，读入 `n m`、n 个点权、n−1 条边、m 组 `u v`，输出每组询问的答案，可直接编译运行。

```
#include <cstdio>
#include <algorithm>
#include <cmath>
using namespace std;

const int MAXN = 100005, MAXM = 100005;
const int LOG  = 17;

int n, m;
int a[MAXN], val[MAXN], vn;              // 原始权值 / 离散化后的值 / 不同值个数

int head[MAXN], nxt[MAXN << 1], to[MAXN << 1], ec;
void addEdge(int u, int v) { to[++ec] = v; nxt[ec] = head[u]; head[u] = ec; }

int dfn[MAXN << 1], timer;               // 欧拉序，长度 2n
int st[MAXN], ed[MAXN];
int dep[MAXN], up[LOG + 1][MAXN];

void dfs(int u, int fa) {
    dep[u] = dep[fa] + 1;
    up[0][u] = fa;
    for (int k = 1; k <= LOG; ++k) up[k][u] = up[k - 1][up[k - 1][u]];
    st[u] = ++timer; dfn[timer] = u;     // 进入
    for (int e = head[u]; e; e = nxt[e])
        if (to[e] != fa) dfs(to[e], u);
    ed[u] = ++timer; dfn[timer] = u;     // 离开
}

int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    int d = dep[u] - dep[v];
    for (int k = 0; k <= LOG; ++k) if (d >> k & 1) u = up[k][u];
    if (u == v) return u;
    for (int k = LOG; k >= 0; --k)
        if (up[k][u] != up[k][v]) { u = up[k][u]; v = up[k][v]; }
    return up[0][u];
}

struct Query { int l, r, extra, id; } q[MAXM];
int blk;
bool cmp(const Query &x, const Query &y) {
    int bx = x.l / blk, by = y.l / blk;
    if (bx != by) return bx < by;
    return (bx & 1) ? x.r > y.r : x.r < y.r;   // 奇偶化
}

int  cnt[MAXN];        // 每个离散化值在当前点集中的出现次数
bool on[MAXN];         // 点 u 是否算在点集中
int  cur, ans[MAXM];   // cur = 当前不同值个数

inline void toggle(int pos) {            // 翻转欧拉序第 pos 个位置所对应的点
    int u = dfn[pos];
    if (on[u]) {
        if (--cnt[val[u]] == 0) --cur;
    } else {
        if (cnt[val[u]]++ == 0) ++cur;
    }
    on[u] = !on[u];
}

int main() {
    scanf("%d%d", &n, &m);
```

```
    for (int i = 1; i <= n; ++i) scanf("%d", &a[i]);

    static int tmp[MAXN];
    for (int i = 1; i <= n; ++i) tmp[i] = a[i];
    sort(tmp + 1, tmp + n + 1);
    vn = unique(tmp + 1, tmp + n + 1) - tmp - 1;
    for (int i = 1; i <= n; ++i)
        val[i] = lower_bound(tmp + 1, tmp + vn + 1, a[i]) - tmp;

    for (int i = 1; i < n; ++i) {
        int u, v; scanf("%d%d", &u, &v);
        addEdge(u, v); addEdge(v, u);
    }
    dfs(1, 0);

    for (int i = 1; i <= m; ++i) {
        int u, v; scanf("%d%d", &u, &v);
        if (st[u] > st[v]) swap(u, v);    // 按 st 比较，不是按深度
        int w = lca(u, v);
        if (w == u) q[i] = (Query){st[u], st[v], 0, i};
        else        q[i] = (Query){ed[u], st[v], w, i};
    }
    blk = max(1, (int)(2.0 * n / sqrt((double)max(m, 1))));
    sort(q + 1, q + m + 1, cmp);

    int l = 1, r = 0;
    for (int i = 1; i <= m; ++i) {
        while (l > q[i].l) toggle(--l);
        while (r < q[i].r) toggle(++r);
        while (l < q[i].l) toggle(l++);
        while (r > q[i].r) toggle(r--);
        int res = cur;
        if (q[i].extra && cnt[val[q[i].extra]] == 0) ++res;   // 临时补 LCA，不改状态
        ans[q[i].id] = res;
    }
    for (int i = 1; i <= m; ++i) printf("%d\n", ans[i]);
    return 0;
}
```

易错点：

- **数组只开 n。`dfn`、`st`、`ed` 全部按 2n 开，欧拉序长度就是 2n；按 n 开在 `n = 10^5` 时必越界，而且症状常常是「前几个询问对、后面随机错」，很难看出是数组问题。**
- **两种情形搞混，或忘了补 LCA。最常见的错法是统一写成 `[st[u], st[v]]`：当 w ≠ u 时，`[st[u], st[v]]` 会覆盖 u 到 w 这一段（每个点多出现两次、互相抵消，看似无害），同时把 v 子树里路径外的点也带进来，答案偏大；反过来，`[ed[u], st[v]]` 不补 w 会漏掉 LCA 这一个点，答案偏小 1。两处都要按上面的三条情形逐一核对。**
- **交换 u、v 时按深度比较。必须按 `st` 比较才能保证 `l ≤ r`；按 `dep` 交换在兄弟子树上会得到 `st[u] > st[v]`，指针一开始就倒挂。**
- **`toggle` 写成「点是否第一次出现」。翻转的对象是位置，不是点：同一个点在区间内出现两次就翻转两次、互相抵消，这正是「只留下路径上的点」的原理。若改成「首次出现才加入、之后不再删」，路径外的子树点会被永久留在点集里。**

- 补 LCA 时真的把它加进去。写 `toggle(st[w])` 再撤销，看起来对称，但第二次翻转依赖 `on[w]` 的当前值，一旦 w 恰好在区间内（第二种情形下不会，但改错情形判断后就会）就会把状态彻底搞乱。正确做法是只做只读判断：`cnt[val[w]] == 0` 才 +1。
- 块大小取成 `n/√m`。区间长度是 2n，块大小要按 2n 算；写成 n/√m 会让块数翻倍、左右指针各多挪一倍，实测常数劣化 30%—50%，但不会错，属于「能过但慢」的典型。
- 点权不离散化。`a[i]` 到 10^9 时 `cnt[]` 开不下；离散化要保留「相同值映射到同一下标」，否则不同颜色数会被算多。
- 递归 DFS 在链上爆栈。`n = 10^5` 的链有 10^5 层，Windows 下 1MB 栈约 2 万层就崩。要么改成显式栈的迭代写法（注意「离开」事件要后压），要么在评测环境里手动调大栈（NOI Linux 下 `ulimit -s`）。
- 带修时忘了「改动点不在点集内则不影响答案」。带修树上莫队把修改当成时间维上的移动，但只有当被改的点当前 `on[u] = true` 时，颜色桶才需要真的变；无条件改会让答案在时间维来回移动时漂移。

自查：把上述模板逐行译成 Python（同样的欧拉序、同样的三条情形、同样的奇偶翻转与只读补点），与「BFS 求路径 + `set` 数颜色」的暴力对拍：300 组随机树（`n ≤ 10`、权值 1—4、共 1333 次询问）全对；专门构造的链形与星形各 60 组（路径退化成一个端点、LCA 恒为根）全对；`n = 2` 的单边、`u = v` 的单点询问这两个边界也包含在随机数据里。上面第 2、4 两条错误的版本都实际跑过：把区间统一写成 `[st[u], st[v]]` 时 300 组里错 177 组；`toggle` 改成「首次出现才加入、之后不再删」时错 238 组；把 `on[]` 换成「点是否被访问过」这类近义写法也会在含重复权值的树上系统性偏大。

27.17 虚树：只留关键点与它们的 LCA，把 O(nq) 压到 O(Σk log n)

一句话判定：q 次询问，每次给定 k 个关键点，问「这些关键点之间的事」——切断它们与根的最小代价、它们张成的连通块边权和、在它们上面跑一遍树形 DP、它们两两距离的最大值。整棵树 n 很大（10^5–10^6），但 Σk 与 n 同阶甚至更小。此时对每次询问都扫全树是 O(nq)，必死；而真正有用的点只有「关键点」和「它们之间的分叉点」，后者恰好是两两 LCA，规模 O(k)。

核心结论：把关键点按 dfs 先序（dfn）排序，则虚树的点集恰好是

`{关键点} ∪ {lca(key[i], key[i+1]) | i = 1..k-1}`（只取相邻两项）

点数为 `O(k)`（严格上界 `2k-1`），且虚树上的祖先关系、LCA 关系、两点距离与原树完全一致——所以任何「只依赖关键点及其分叉结构」的树形 DP，搬到虚树上做与原树等价。

为什么只取相邻两项的 LCA 就够：先序遍历的性质是「把一棵子树连续走完才走下一棵」。排序后若两个关键点跨越了不同的子树，那么在这两项之间必然夹着一对「相邻且跨子树」的项，它们的 LCA 就是那两个子树的分叉点。于是所有需要的分叉点都被相邻项的 LCA 覆盖，不必求 $O(k^2)$ 对 LCA。这一步是整节唯一不能省、也最容易记错的结论；换成 BFS 序、或对关键点不做排序，性质立刻失效。

建树三步：① 关键点按 dfn 排序；② 求相邻两项的 LCA 并入点集，再排序、去重；③ 用单调栈维护「根 → 当前点」的祖先链，按 dfn 扫描点集，弹栈时连边。栈内始终是一条自上而下的链，每条虚边在出栈时确定一次，总代价 O(k)。

环节	复杂度	说明
关键点排序	O(k log k)	按 dfn，不是按编号、也不是按深度
相邻 LCA（倍增）	O(k log n)	换 O(1) RMQ 可降到 O(k)，但常数与空间要自己权衡
再排序 + 去重	O(k log k)	LCA 可能与关键点重合，必须 `unique`
单调栈连边	O(k)	每个点入栈出栈各一次
虚树上 DP	O(k)	按 dfn 逆序扫点集，不需要再 DFS
单次询问合计	O(k log n)	对比「每次扫全树」的 O(n)

三条常用性质：① `lca` 保真——虚树上任意两点的 LCA 等于原树上的 LCA；② 虚边 (u, v) 代表原树的一条路径（v 是 u 的祖先或反之），边长取 `dist[v] - dist[u]`，边权为 1 时就是 `dep` 差；③ 虚树的边权和 = 这些关键点的斯坦纳树（把它们连起来的最小连通子树）边权和，这是「k 个点的最小连通子树」类问题的标准做法。

模板（经典应用：切断全部关键点与根的最小代价，即「消耗战」模型）：设 `mn[u]` 为「根到 u 路径上的最小边权」，即切断这条路径最便宜的一刀。虚树上做一次 DP：

`f[u] = mn[u]`（u 是关键点） ； `f[u] = min(mn[u], Σ f[儿子])`（u 不是关键点）

含义很清楚：要么在 u 及其上方一刀切断（代价 `mn[u]`），要么把每个儿子各自切断（代价是儿子们之和），两者取小。根没有入边，所以 `mn[1] = +∞`。

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int N = 250005, LOG = 19;
const ll INF = (ll)1e18;

int n, head[N], nxt[N << 1], to[N << 1], ecnt;
ll wt[N << 1];
void addEdge(int u, int v, ll w) {
    nxt[++ecnt] = head[u]; head[u] = ecnt; to[ecnt] = v; wt[ecnt] = w;
}

int dfn[N], timer, dep[N], up[LOG][N], curEdge[N], stk[N], top;
ll dist[N], mn[N];                      // mn[u]：根到 u 路径上的最小边权

void build() {                          // 显式栈的迭代 DFS：入栈即打时间戳 → 真正的先序
    top = 0; stk[++top] = 1;
    dep[1] = 0; dist[1] = 0; mn[1] = INF;
    for (int i = 0; i < LOG; ++i) up[i][1] = 0;
    while (top) {
        int u = stk[top];
        if (!dfn[u]) dfn[u] = ++timer;
        int &e = curEdge[u];
        while (e && to[e] == up[0][u]) e = nxt[e];        // 跳过父边
        if (!e) { --top; continue; }
        int id = e, v = to[id]; e = nxt[id];
        up[0][v] = u; dep[v] = dep[u] + 1;
        dist[v] = dist[u] + wt[id];
        mn[v] = min(mn[u], wt[id]);
        for (int i = 1; i < LOG; ++i) up[i][v] = up[i - 1][up[i - 1][v]];
        stk[++top] = v;
    }
}

int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    int d = dep[u] - dep[v];
    for (int i = 0; i < LOG; ++i) if (d >> i & 1) u = up[i][u];
    if (u == v) return u;
    for (int i = LOG - 1; i >= 0; --i)
        if (up[i][u] != up[i][v]) { u = up[i][u]; v = up[i][v]; }
    return up[0][u];
}

int key[N], vhead[N], vnxt[N << 1], vto[N << 1], vcnt, vpar[N];
bool mark[N];
ll f[N], acc[N];

void vadd(int u, int v) { vnxt[++vcnt] = vhead[u]; vhead[u] = vcnt; vto[vcnt] = v; vpar[v] = u; }

bool cmpDfn(int a, int b) { return dfn[a] < dfn[b]; }

// key[1..k] 为关键点（不含根 1）；返回「切断全部关键点与根」的最小代价
ll query(int k) {
    if (k == 0) return 0;
    for (int i = 1; i <= k; ++i) mark[key[i]] = true;
    sort(key + 1, key + k + 1, cmpDfn);
    int m = k;
    for (int i = 1; i < k; ++i) key[++m] = lca(key[i], key[i + 1]);   // 只补相邻两项
    sort(key + 1, key + m + 1, cmpDfn);
    m = unique(key + 1, key + m + 1) - key - 1;
```

```
    vcnt = 0;                                   // 只清本询问用到的点，绝不 memset 整张 head
    for (int i = 1; i <= m; ++i) { vhead[key[i]] = 0; acc[key[i]] = 0; vpar[key[i]] = 0; }

    top = 0; stk[++top] = key[1];               // 单调栈 = 根到当前点的祖先链
    for (int i = 2; i <= m; ++i) {
        int u = key[i], w = lca(u, stk[top]);
        while (top > 1 && dep[stk[top - 1]] >= dep[w]) { vadd(stk[top - 1], stk[top]); --top; }
        if (stk[top] != w) { vadd(w, stk[top]); stk[top] = w; }
        stk[++top] = u;
    }
    while (top > 1) { vadd(stk[top - 1], stk[top]); --top; }

    for (int i = m; i >= 1; --i) {              // dfn 逆序 = 子先于父，不必再 DFS
        int u = key[i];
        f[u] = mark[u] ? mn[u] : min(mn[u], acc[u]);
        if (vpar[u]) acc[vpar[u]] += f[u];
    }
    ll res = f[key[1]];
    for (int i = 1; i <= m; ++i) mark[key[i]] = false;
    return res;
}

int main() {
    scanf("%d", &n);
    for (int i = 1; i < n; ++i) {
        int u, v; ll w; scanf("%d%d%lld", &u, &v, &w);
        addEdge(u, v, w); addEdge(v, u, w);
    }
    build();
    int q; scanf("%d", &q);
    while (q--) {
        int k; scanf("%d", &k);
        for (int i = 1; i <= k; ++i) scanf("%d", &key[i]);
        printf("%lld\n", query(k));
    }
    return 0;
}
```

四个实现要点：① 预处理故意写成显式栈的迭代 DFS——`n = 2.5×10^5` 的链在 1 MB 栈下递归必崩，而 `dfn` 的赋值时机取「入栈」正好保证它是先序时间戳（这也是为什么不能用 BFS 序顶替）；② 每次询问只清本询问用到的点的 `vhead / acc / vpar`，绝不清整张 `head`，否则一次询问退化成 O(n)，多问就是 O(nq)；③ DP 不必再遍历虚树——按 dfn 逆序扫点集即可，因为祖先的 dfn 一定小于后代，逆序天然「子先于父」；④ 虚树边不需要单独存长度，用 `dist` 差现算，DP 里用到边权时（例如求虚树边权和）再取 `dist[v] - dist[u]`。

易错点

- **不补 LCA，直接把关键点串起来。实测 200 组随机树（`n ≤ 9`、关键点 1—4 个）错 94 组；错的全是「两个关键点分居不同子树、需要经过分叉点」的形状，另一半因为关键点本身互为祖先而侥幸正确——所以手造小样例极难暴露。**
- **DP 按 dfn 正序扫。父先于子，`acc` 还没累加完就被 `min` 取走。实测错 91 组 / 200。**
- **用 BFS 序或编号序当 dfn。排序后「相邻两项」不再共享分叉点，虚树会缺点或多出错误的父子关系，典型症状是答案偏大（该一刀切的地方切不到）。判断方法：把 `dfn` 换成 `bfs` 序重跑，答案变了就说明这里错了。**

- 弹栈条件写成 `dep[stk[top]] > dep[w]`。这样会把 w 自己也弹掉，栈里随后出现两个 w，`vpar[w] = w`，DP 里 `acc` 累加到自身上，答案翻倍或死循环。正确条件是 `dep[stk[top - 1]] >= dep[w]`，看栈顶的下一个。
- 忘记 `unique` 去重。LCA 可能恰好等于某个关键点，关键点也可能被重复给出；不去重会导致 `vhead` 被清两次（把刚连好的边清掉）、同一条虚边加两遍。
- `mn[1]` 忘了设成 `+∞`。根没有入边，写成 0 会让「切断于根」变成一个假的 0 代价，答案恒为 0。
- 根 1 本身出现在关键点里。此时不存在「切断根」的方案，模板会原样返回 `+∞`；多数题面保证 1 号点不在集合内，若没有保证，需要按题目要求特判输出无解。
- 虚边权取 `wt[e]`。虚边上没有真实的单条边，权重要取 `dist[v] - dist[u]`；`wt[]` 只服务原树。
- 多组询问只清 `vcnt`。不清 `vhead` 时上一问的虚边还挂着，下一问的 DP 会沿旧边走——症状是「同一批关键点，问的顺序不同答案也不同」。
- 树形 DP 里对关键点也做 `min`。关键点必须被切断，所以 `f[u] = mn[u]` 不能与孩子求和取小；写成 `min(mn[u], Σ)` 会让关键点被「放过」，答案偏小。

自查：把模板逐行译成 Python（同样的迭代先序、同样的相邻 LCA、同样的单调栈、同样的逆序 DP），与「枚举边子集 → 检查每个关键点是否仍与根连通 → 取最小代价」的指数暴力对拍：200 组随机树（`n ≤ 9`、边数 ≤ 8、边权 1—6、关键点 1—4 个且不含根）全对；`k = 1` 的单点询问、关键点互为祖先的链形这两类边界都包含在随机数据中。上面第 1、2 条错误的版本实测：不补 LCA 错 94/200，DP 正序错 91/200。本机无 `g++`，C++ 侧只做了语法与上界自查（`head/nxt/to` 按 `2(n-1)` 开、虚树边数 ≤ `2k-1`、`LOG = 19` 覆盖 2.5×10^5、`vcnt` 上界 `N`）。

27.18 回滚莫队与二次离线莫队：删不掉的量、以及代价太高的加

一句话判定：普通莫队（27.7）要求 `add` 与 `del` 都能在 O(1) 或 O(log n) 内互为逆操作。两类题会打破这个前提：① 删除不可逆——维护「区间众数频次 `mx`」时 add 只让 `mx` 取 max，del 之后不能把 `mx` 退回去；区间 mex 的并查集、区间内最大值−最小值（要保留次大次小）同理。此时用回滚莫队（只加不删）。② 加一次太贵——add 本身要扫一遍区间或带一个 log（如「区间内有多少对 $a_i+a_j<K$」），`(n+q)√n` 次 add 会把 log 乘到总复杂度上。此时用二次离线莫队，把每次 add 的询问离线下来、扫描一次算完。

① 回滚莫队：三条不变量

- 按 `l` 分块（块长 `B`），块内询问按 `r` 升序，于是右指针 `curR` 单调右移，从不回退——这是「只加」的前提。
- 右指针从块右端 `R` 起步，向右 add 到 `qq.r`；这部分跨询问保留、永不回滚。
- 左指针每次都从 `R` 重新向左 add 到 `qq.l`，回答完立刻把这次左移引入的全部改动撤销；答案是「快照 + 左移增量」，因此必须能用快照恢复，而不是用逆运算减回去。

完全落在块内（`qq.r ≤ R`）的询问不走主流程，直接 O(B) 暴力——否则左移区间 `[qq.l, R]` 与右移区间 `[R+1, qq.r]` 会拼不成目标区间。

环节	代价	说明
块数	`⌈n/B⌉`	每块都要把右指针从 R 一路推到 n 附近
右指针 add 总量	`O(n²/B)`	每块 O(n)，共 n/B 块
左移 + 回滚 / 块内暴力	`O(qB)`	每个询问 O(B) 次 add 与同量级撤销
总复杂度	`O(n²/B + qB)`	取 `B = n/√q` 得 `O(n√q)`；`q ≈ n` 时退化为 `O(n√n)`
空间	`O(n + V + q)`	桶数组按值域 V 开，改动栈 O(B)

关键约束：add 必须与加入顺序无关。左移是按位置递减加入的，任何依赖「第一次/最后一次出现位置」的写法都会在这里失效（本轮实测见易错点第 5 条）。下面用「区间众数频次」作模板题：`cnt[v]` 与 `mx` 只依赖当前集合，与加入顺序无关，且 `mx` 单调取最大、删除不可逆，正好是回滚莫队的标准用例。

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 100000 + 5, MAXV = 100000 + 5;   // 值域须离散化到 1..MAXV
int n, q, B, a[MAXN], cnt[MAXV], tmp[MAXV], mx;
vector<int> ans;
struct Query { int l, r, id, blk; };
vector<Query> qs;
vector<pair<int,int>> stk;      // (值, 旧 cnt)，供回滚

inline void add(int p) {
    int v = a[p];
    stk.push_back({v, cnt[v]});
    if (++cnt[v] > mx) mx = cnt[v];
}
inline void rollback(int snapMx, int snapTop) {
    for (int k = (int)stk.size() - 1; k >= snapTop; --k)
        cnt[stk[k].first] = stk[k].second;
    stk.resize(snapTop);
    mx = snapMx;
}

void solve() {
    B = max(1, (int)(n / max(1.0, sqrt((double)q))));
    int nb = (n + B - 1) / B;
    for (auto &x : qs) x.blk = (x.l - 1) / B;
    vector<vector<Query>> by(nb + 1);
    for (auto &x : qs) by[x.blk].push_back(x);
    for (int b = 0; b <= nb; ++b)
        sort(by[b].begin(), by[b].end(),
             [](const Query &u, const Query &v) { return u.r < v.r; });
    for (int b = 0; b <= nb; ++b) {
        int L = b * B + 1, R = min(n, (b + 1) * B);
        if (L > n) break;
        rollback(0, 0);                         // 换块：清空上一块全部状态
        int curR = R;
        for (const Query &qq : by[b]) {
            if (qq.r <= R) {                    // 完全落在块内：直接暴力
                int best = 0;
                for (int i = qq.l; i <= qq.r; ++i)
                    best = max(best, ++tmp[a[i]]);
                for (int i = qq.l; i <= qq.r; ++i) tmp[a[i]] = 0;
                ans[qq.id] = best;
                continue;
            }
            while (curR < qq.r) add(++curR);  // 右端点单调，只加不回滚
            int snapMx = mx, snapTop = (int)stk.size();
            for (int i = R; i >= qq.l; --i) add(i);  // 左端点每次从 R 重新出发
            ans[qq.id] = mx;
            rollback(snapMx, snapTop);          // 只撤销左移部分
        }
    }
}
```

② 二次离线莫队：把 add 的代价摊到一次扫描里。设加入点 `x` 时对当前区间 `[l,r]` 的贡献是 `f(x,[l,r]) = #{ y ∈ [l,r] : P(x,y) }`，其中 P 对称且只依赖两个点的值与位置、与当前集合无关。定义前缀形式

`G(x,t) = #{ y ≤ t : P(x,y) }`，则 `f(x,[l,r]) = G(x,r) − G(x,l−1)`

于是莫队四指针的每一次移动都可记成两条四元组 `(x, r, +w, i)` 与 `(x, l-1, -w, i)`，`w=+1` 为加入、`w=-1` 为删除，`i` 是排序后的询问下标。把所有四元组按 `t` 排序，扫描 `t = 1..n`，用值域 BIT 维护「已插入的 `y ≤ t`」，即可 O(log V) 回答每个 `G(x,t)`。以 `P(x,y) = [a_x+a_y < K]` 为例，`G(x,t) = BIT.query(min(V, K-1-a_x))`。

环节	复杂度
莫队四指针移动次数	`O((n+q)√q)`，产生同量级四元组（×2）
四元组按 t 排序	`O(N log N)`，`N = O((n+q)√n)`
扫描 + BIT 回答 G	`O((n+N) log V)`；贡献可前缀和时降到 O(n+N)
增量前缀和	`O(q)`
对比「每次 add 带 log」	把 `log` 从 `O((n+q)√n)` 次在线操作挪到一次离线扫描

适用判定：贡献必须能写成 `G(x,r)-G(x,l-1)` 的前缀差。若依赖「当前区间里有多少个元素」这类非线性量，或 P 与当前集合有关，二次离线不成立，应改回滚莫队或线段树分治（28.17）。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int n, q, B, V, K;                 // a[] 离散化到 1..V
vector<int> a;
struct Query { int l, r, id; };
vector<Query> qs;
struct Event { int x, t, w, idx; };
vector<Event> ev;
vector<ll> delta;                  // 排序后「相邻询问之间」的答案增量

inline void emit(int x, int l, int r, int w, int idx) {
    if (l > r) return;
    if (r > 0)     ev.push_back({x, r,      w, idx});
    if (l - 1 > 0) ev.push_back({x, l - 1, -w, idx});
    // t ≤ 0 时 G(x,0) = 0，这类四元组必须丢弃
}

struct BIT {
    int n; vector<int> t;
    void init(int n_) { n = n_; t.assign(n + 2, 0); }
    void add(int i, int d) { for (; i <= n; i += i & -i) t[i] += d; }
    int sum(int i) const { int r = 0; for (; i > 0; i -= i & -i) r += t[i]; return r; }
} bit;

void solve() {
    B = max(1, (int)(n / max(1.0, sqrt((double)q))));
    vector<int> ord(q);
    iota(ord.begin(), ord.end(), 0);
    sort(ord.begin(), ord.end(), [&](int i, int j) {
        int bi = (qs[i].l - 1) / B, bj = (qs[j].l - 1) / B;
        return bi != bj ? bi < bj : qs[i].r < qs[j].r;
    });
    delta.assign(q, 0);
    int L = 1, R = 0;
    for (int idx : ord) {
        int l = qs[idx].l, r = qs[idx].r;
        while (L > l) { --L; emit(L, L + 1, R, +1, idx); }
        while (R < r) { ++R; emit(R, L, R - 1, +1, idx); }
        while (L < l) { emit(L, L + 1, R, -1, idx); ++L; }
        while (R > r) { emit(R, L, R - 1, -1, idx); --R; }
    }
    sort(ev.begin(), ev.end(),
         [](const Event &u, const Event &v) { return u.t < v.t; });
    bit.init(V + 2);
    size_t p = 0;
    for (int t = 1; t <= n; ++t) {
        bit.add(a[t], 1);                       // BIT 里放所有 y ≤ t
        while (p < ev.size() && ev[p].t == t) {
            const Event &e = ev[p];
            int c = bit.sum(min(V, K - 1 - a[e.x]));
            if (e.t >= e.x && 2 * a[e.x] < K) --c; // 扣掉 y = x 自配对
            delta[e.idx] += (ll)e.w * c;
            ++p;
        }
    }
    ll run = 0; vector<ll> ans(q);
    for (int idx : ord) { run += delta[idx]; ans[qs[idx].id] = run; }
}
```

症状	选谁	理由
del 写不出来（mx、mex、次大值）	回滚莫队	干脆不做删除，用快照把左移增量整体撤销
add 要 O(log n) 或扫一遍	二次离线莫队	把 `(n+q)√n` 次在线查询压成一次离线扫描
add / del 都 O(1)	27.7 普通莫队	两种改造都只是多一层常数，不必上
带修改（三维：时间）	带修莫队 / 28.17	本节两种都只处理静态区间
贡献依赖当前集合大小	分块 / 线段树	二次离线的「前缀差」分解不成立

易错点（本轮逐条实测）：

1. 块内询问必须按 `r` 升序；一旦右指针要回退，「只加不删」的前提就没了。
2. 左端点每次必须从块右端 `R` 重新出发，不能沿用上一次的左指针位置。
3. 快照要打在右移之后、左移之前；回滚只撤销左移引入的改动，右移累积的 `cnt/mx` 必须留给下一个询问。换块时再统一 `rollback(0,0)` 清空全部状态（含右移部分）。
4. `qq.r ≤ R` 的询问必须走块内暴力分支，否则 `[qq.l,R]` 与 `[R+1,qq.r]` 拼不出目标区间，还会出现越界 add。
5. add 必须顺序无关。本轮第一版用「`first[v]` 记最早位置、`p-first[v]` 更新答案」求相同值最大间距，但左移是递减加入，`p-first[v]` 变成负数：400 组随机数据 11831 个询问错 5879 个。改成「`cnt[v]` 累加 + `mx` 单调取最大」后 0 错。凡涉及首次/末次位置的量，先确认倒序加入是否仍成立。
6. 块长取 `B = n/√q`，不是固定 `√n`；q 很小时 O(nq) 暴力往往更划算。回滚莫队的撤销次数与 add 次数同阶，别只算 add。
7. 二次离线的 `w` 符号：加入（L 左移 / R 右移）为 `+1`，删除（L 右移 / R 左移）为 `-1`；`emit` 拆成 `(x,r,+w)` 与 `(x,l-1,-w)`，写反会得到负数或两倍。
8. `t ≤ 0` 的四元组必须丢弃（`G(x,0)=0`）。本轮第一版把它们留到扫描末尾，此时 BIT 已被 `1..n` 填满，且排序后 `t=0` 卡在队首使 `p` 指针全程不推进：300 组随机数据 5337 个询问错 2348 个。过滤后 0 错。
9. `G(x,t)` 要扣掉 `y = x` 的自配对（当 `x ≤ t` 且 `P(x,x)` 成立）；两个 G 相减时自贡献会自然抵消，不能只在一处扣。
10. `delta[i]` 是「从上一询问到当前询问的指针移动增量」，必须按排序后顺序做一次前缀和才是各询问答案；直接当答案会全部偏小。
11. 值域 BIT 的 `K-1-a[x]` 要 clamp 到 `[0,V]`；负数下标会让 `sum` 直接返回 0 或死循环，取决于实现。
12. 四元组数量是 `O((n+q)√n)`，`n=q=10^5` 时约 `3×10^7` 条，会 MLE；需按 `x` 分桶离线、或调小块长换时间。

校验：本机无 `g++`，按两份模板逐行等价复现成 Python 与 $O(n^2)$ 暴力对拍——回滚莫队（区间众数频次）400 组随机数据、11831 个询问全对；二次离线莫队（区间内 $a_i+a_j < K$ 的点对数）400 组、5337 个询问全对。上面第 5、8 条的错误版本均按实测数字记录。C++ 侧只做语法与上界自查（`cnt/tmp` 按值域开、`by` 按 `nb+1` 开、`delta` 用 `long long`、`emit` 的 `l-1` 可能为 0 已过滤）。

28 第 28 章　图论

图论题的难点在建模，不在模板。本章先讲存图与最短路的选择，再依次覆盖生成树、拓扑、连通性、二分图、网络流、树上的 LCA/树剖，最后是欧拉路、差分约束与 2-SAT——后两者正是「最短路」与「强连通分量」各自的一次建模应用；28.12 的点分治则把「树上路径统计」这一类问题的复杂度从 $O(n^2)$ 压到 $O(n \log^2 n)$；28.13 把「两两之间必有一场」的竞赛图单独拆出来，得分序列的 Landau 判定、缩点成链、三元环计数与哈密顿路径构造四件事全都只靠出度序列完成，一次排序即可定论；28.14 收尾于 Dilworth 定理，把「偏序集的最小链覆盖 = 最长反链」这条结论落成一个拆点二分图上的最大匹配，顺带说清「闭包 / 不闭包」这一步之差为什么会让答案从链覆盖变成路径覆盖；28.15 把 28.7 只用一段话带过的费用流补全，SSP、势能 Dijkstra 与上下界可行流三套模板连同建模清单一次写清；28.16 收尾于匹配的两块硬骨头——KM 用可行顶标与相等子图解决二分图带权完美匹配，带花树用奇环缩点解决一般图最大匹配，两节模板与「什么情况下别硬套」的判断一并给出；28.17 补上「边有存活时间」这一类动态连通性问题，用线段树分治把时间轴摊平、用可撤销并查集吃掉「删除」，把 $O(m \log T \log n)$ 的离线做法讲成可直接套用的模板，并说清路径压缩为什么必须去掉。28.18 收尾于最小树形图：28.3 只用一句话带过的朱刘算法在此展开成完整模板——最小入边、缩环、以及「边权为何改成 $w - inw[v]$」的差额论证；顺带记下一个反直觉的实测结论：虚拟根法在有解图上从不出错，却在无解图上必然给出一个错误数值（88/88），因为多条虚拟边会把图拼成森林。

28.1 存图的三种方式

存图方式决定了后续算法的常数，选错会让正解超时。

方式	空间	适用场景
邻接矩阵	$O(n^2)$	$n \le 1000$ 的稠密图、Floyd
`vector<int> g[]`	$O(n+m)$	通用，遍历快，但频繁清空较慢
链式前向星	$O(n+m)$	m 很大、需要反复清空、需要访问反向边

```
// 链式前向星
int head[MAXN], nxt[MAXM], to[MAXM], w[MAXM], ecnt = 0;
void addEdge(int u, int v, int c) {
    to[++ecnt] = v; w[ecnt] = c; nxt[ecnt] = head[u]; head[u] = ecnt;
}
for (int e = head[u]; e; e = nxt[e]) { int v = to[e], c = w[e]; /* ... */ }
```

反向边技巧：网络流与「残量」类问题把正向边存在 `2i`、反向边存在 `2i+1`，用 `e^1` 一次取到反向边。

无向图：一条边存两次，注意边的编号从 2 开始（`ecnt` 初始为 1），这样 `e^1` 才配对。

28.2 最短路：四种算法与选择

四种最短路各有适用边界，选错的后果是 TLE 或 WA。

算法	复杂度	适用
Floyd	$O(n^3)$	$n \leqslant 400$，多源，可带负权（无负环）
Dijkstra + 堆	O(m log n)	非负边权，单源，最常用
SPFA	最坏 O(nm)	有负权边；出题人可构造数据卡到最坏，能用 Dijkstra 就别用
Johnson	O(nm log n)	稀疏图 + 负权 + 多源

```
// Dijkstra（堆优化）
vector<long long> dis(n + 1, LINF);
priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq;
dis[s] = 0; pq.push({0, s});
while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dis[u]) continue;            // 关键：跳过过期状态
    for (int e = head[u]; e; e = nxt[e]) {
        int v = to[e]; long long nd = d + w[e];
        if (nd < dis[v]) { dis[v] = nd; pq.push({nd, v}); }
    }
}
```

SPFA 判负环：记录每个点的入队次数，超过 n 次说明存在负环；或记录「最短路经过的边数」，超过 n-1 即负环。

Floyd 的两个附加用途：传递闭包（把 `dis` 换成 `reach` 做布尔运算）、最小环（枚举中间点 k，用 `dis[i][j]+w[i][k]+w[k][j]` 更新）。注意 Floyd 的三层循环 k 必须在外层。

0-1 BFS：边权只有 0/1 时用双端队列，0 边入队首，O(V+E)，比堆优化 Dijkstra 快一个量级。

28.3 最小生成树

Kruskal 按边权排序 + 并查集，O(m log m)，实现最简单，稀疏图首选；Prim 堆优化 O(m log n)，稠密图（完全图）更快。

```
struct Edge { int u, v, w; };
sort(edges.begin(), edges.end(), [](const Edge&a, const Edge&b){ return a.w < b.w; });
long long ans = 0; int cnt = 0;
for (auto &e : edges) {
    if (merge(e.u, e.v)) { ans += e.w; if (++cnt == n - 1) break; }
}
if (cnt < n - 1) puts("impossible");   // 不连通
```

三个衍生结论：

- 最小瓶颈生成树：使「最大边权最小」的生成树，就是 MST（MST 的任意两点路径上最大边 = 最小瓶颈路）。
- 次小生成树：枚举非树边 `(u,v,w)`，加上它后删掉环上最大边，取增量最小者；需要倍增预处理路径最大/次大边。
- 最小树形图（有向）：朱刘算法 O(nm)，用于「以某点为根、边有方向」的最小生成树。

Boruvka：每轮为每个连通块找最小出边，O(m log n)，适合边权由点权计算得出、无法显式排序的完全图。

28.4 拓扑排序与 DAG 上的 DP

拓扑排序把有向无环图排成线性序列，使所有边从前指向后。两种写法：Kahn（BFS + 入度）与 DFS 后序。

```
// Kahn
queue<int> q;
for (int i = 1; i <= n; ++i) if (!indeg[i]) q.push(i);
while (!q.empty()) {
    int u = q.front(); q.pop(); ord.push_back(u);
    for (int v : g[u]) if (--indeg[v] == 0) q.push(v);
}
if ((int)ord.size() < n) puts("有环");
```

DAG 上 DP：按拓扑序递推，`f[v] = max(f[v], f[u] + w(u,v))`。最长路、路径计数、方案数统计都在这个框架里。

关键路径：正推求最早开始时间，逆推求最晚开始时间，差为 0 的活动即关键活动。

有环图怎么办：先 Tarjan 缩点（SCC 内部可以互相到达），把 SCC 缩成一个点后得到 DAG，再在上面 DP。这是「有向图最长路」的标准解法。

字典序最小的拓扑序：把队列换成优先队列（小根堆）。注意这只保证「当前可选的最小」，不等价于「全序列字典序最小」——后者要用反向图 + 大根堆求。

28.5 连通性：Tarjan 三件套

Tarjan 用一次 DFS 同时求出 dfn（时间戳）与 low（能回溯到的最早时间戳），由此得到割点、桥、强连通分量。

```
void tarjan(int u, int fa) {
    dfn[u] = low[u] = ++timer;
    int child = 0;
    for (int v : g[u]) {
        if (!dfn[v]) {
            ++child; tarjan(v, u);
            low[u] = min(low[u], low[v]);
            if (low[v] >= dfn[u] && fa) cut[u] = true;   // 割点
            if (low[v] > dfn[u]) bridge.push_back({u, v}); // 桥
        } else if (v != fa) low[u] = min(low[u], dfn[v]);
    }
    if (!fa && child > 1) cut[u] = true;                  // 根的特判
}
```

强连通分量（SCC）：维护一个栈，`low[u]==dfn[u]` 时弹栈到 u，弹出的点构成一个 SCC。缩点后得到 DAG，可做拓扑 DP。

边双连通分量：删掉所有桥后剩下的连通块；点双连通分量：删掉割点后的块，用栈存边，判据是 `low[v] >= dfn[u]`。

易错点：无向图要传父节点避免走回边，但重边时不能只靠「父节点编号」判断，要传「边的编号」。这一点在有多重边的题里几乎必踩。

28.6 二分图与匹配

二分图判定：DFS/BFS 黑白染色，遇到同色相邻点即有奇环，不是二分图。这是「奇偶性约束」类问题的第一反应。

```
// 匈牙利算法：求二分图最大匹配，O(VE)
bool dfs(int u) {
    for (int v : g[u]) {
        if (vis[v]) continue;
        vis[v] = true;
        if (!match[v] || dfs(match[v])) { match[v] = u; return true; }
    }
    return false;
}
int hungary() {
    int res = 0;
    for (int u = 1; u <= n; ++u) {
        memset(vis, 0, sizeof vis);
        if (dfs(u)) ++res;
    }
    return res;
}
```

三个定理（Kőnig 相关，考场上常直接套用）：

- **最大匹配 = 最小点覆盖（二分图）。**
- **最大独立集 = 总点数 − 最小点覆盖。**
- **最小路径覆盖（DAG）= 原图点数 − 拆点后二分图的最大匹配。**

规模选择：$n \leqslant 500$ 用匈牙利；$n \leqslant 10^4$ 且边稀疏用 Hopcroft-Karp（$O(E\sqrt{V})$）；带权最大匹配用 KM 算法（$O(n^3)$，模板与顶标不变量见 28.16）或费用流。

28.7 网络流：Dinic 模板

网络流解决「带容量约束的分配问题」：最大流、最小割、费用流。考场上 90% 的题用 Dinic 就够。

```
struct Dinic {
    struct E { int to, nxt; long long cap; };
    vector<E> e; vector<int> head, cur, lev;
    int n, s, t;
    Dinic(int n, int s, int t) : n(n), s(s), t(t), head(n + 1, -1), cur(n + 1), lev(n + 1) {}
    void add(int u, int v, long long c) {
        e.push_back({v, head[u], c}); head[u] = e.size() - 1;
        e.push_back({u, head[v], 0}); head[v] = e.size() - 1;
    }
    bool bfs() {
        fill(lev.begin(), lev.end(), 0);
        queue<int> q; q.push(s); lev[s] = 1;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int i = head[u]; ~i; i = e[i].nxt)
                if (e[i].cap && !lev[e[i].to]) { lev[e[i].to] = lev[u] + 1; q.push(e[i].to); }
        }
        return lev[t] > 0;
    }
    long long dfs(int u, long long f) {
        if (u == t) return f;
        for (int &i = cur[u]; ~i; i = e[i].nxt) {
            int v = e[i].to;
            if (e[i].cap && lev[v] == lev[u] + 1) {
                long long d = dfs(v, min(f, e[i].cap));
                if (d) { e[i].cap -= d; e[i ^ 1].cap += d; return d; }
            }
        }
        return 0;
    }
    long long maxflow() {
        long long flow = 0;
        while (bfs()) { cur = head; while (long long f = dfs(s, LINF)) flow += f; }
        return flow;
    }
};
```

最小割 = 最大流（最大流最小割定理）。求「删掉最少代价的边使 s 与 t 不连通」直接用最大流。

建模三套路：拆点（点有容量限制，拆成入点出点、中间连容量边）、二分图匹配（源连左、左连右、右连汇，容量 1）、上下界（先无源汇可行流，再转有源汇）。

费用流：把 BFS 换成 SPFA/Dijkstra（带势能），沿最短路增广，复杂度 $O(F \cdot m \log n)$，F 为流量。

28.8 LCA 与树上问题

LCA（最近公共祖先）是树上问题的地基，求法有两种：倍增（在线、好写）与 Tarjan（离线、更快）。

```
int up[LOG][MAXN], dep[MAXN];
void dfs(int u, int fa) {
    up[0][u] = fa; dep[u] = dep[fa] + 1;
    for (int k = 1; k < LOG; ++k) up[k][u] = up[k-1][ up[k-1][u] ];
    for (int v : g[u]) if (v != fa) dfs(v, u);
}
int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    int d = dep[u] - dep[v];
    for (int k = 0; k < LOG; ++k) if (d >> k & 1) u = up[k][u];
    if (u == v) return u;
    for (int k = LOG - 1; k >= 0; --k)
        if (up[k][u] != up[k][v]) { u = up[k][u]; v = up[k][v]; }
    return up[0][u];
}
```

树上差分：路径 `u→v` 上加 v，用 `d[u]+=v; d[v]+=v; d[lca]-=v; d[fa[lca]]-=v;`，最后子树求和还原。边差分则把 `d[lca]` 减两次。

树链剖分（HLD）：把树按重儿子切成 $O(\log n)$ 条链，配合线段树支持「路径修改 / 路径查询 / 子树修改」，复杂度 $O(\log^2 n)$。核心是 `dfn` 的分配顺序：先重儿子，保证每条链在序列上连续。

虚树：只保留「关键点 + 它们的 LCA」，把一次询问的规模从 n 降到 k，用于「多次询问树上若干关键点」的问题。

28.9 欧拉路与哈密顿路

欧拉路：一笔画，每条边恰好走一次。存在条件：图连通，且奇度点个数为 0（欧拉回路）或 2（欧拉路，从奇度点出发）。

```
void dfs(int u) {                       // Hierholzer
    for (int &i = head[u]; i; i = nxt[i]) {
        int e = i; if (vis[e]) continue;
        vis[e] = vis[e ^ 1] = true;
        dfs(to[e]);
    }
    path.push_back(u);                  // 后序加入，最后逆序即路径
}
```

有向图版本：把「奇度点」换成「入度与出度之差」——全为 0 是回路，恰有一个点出度比入度大 1（起点）、一个点入度比出度大 1（终点）则是路。

哈密顿路是 NP 问题，$n \leqslant 20$ 用状压 DP（`f[mask][i]`），网格图上哈密顿路径有构造性结论，不要硬搜。

易错：Hierholzer 里必须用「当前弧」写法（`int &i = head[u]`），否则同一条边会被反复检查，复杂度退化到 $O(m^2)$。

28.10 差分约束：把不等式组化成最短路

本质：差分约束不是新算法，而是一次建模翻译。最短路算法松弛时恒有 `dis[v] ≤ dis[u] + w(u,v)`，这与约束 $x_i - x_j \le c$ 形式完全一致，于是不等式组 ⇔ 一张有向带权图，解 ⇔ 一组距离。

题目要求	建图方式	求解
形如 $x_i - x_j \le c$	边 `j → i`，权 `c`	最短路
形如 $x_i - x_j \ge c$	边 `j → i`，权 `c`	最长路（或全部取负后转最短路）
判是否有解	加超级源 `0`，向所有点连权 0 的边	判负环（最长路则判正环）

为什么是「最大解」：源点到 `i` 的最短路是满足所有上界约束的最小上界，也就是 x_i 能取到的最大值；想让每个变量尽量小，就对同一个图求最长路。两种解只差一个整体平移，若题目要求非负，把整组解同时加上一个常数即可（差值不变，约束仍成立）。

有解判定：存在负环 ⇔ 无解。因为负环意味着沿环累加得到 $x_i \le x_i - k$（$k > 0$），自相矛盾。所以判负环与判可行性是同一件事。

复杂度：SPFA 最坏 O(nm)，随机图上接近 O(m)。n、m 在 5×10^3 量级可以放心用；到 10^5 要先看约束有没有特殊结构（区间、前缀），多数情况下应改用线段树优化建图或直接贪心求解。

```
#include <cstdio>
#include <cstring>
#include <queue>
using namespace std;
const int MAXN = 100005, MAXM = 600005;   // 边数上限 = m + n（超级源 n 条）
int head[MAXN], nxt[MAXM], to[MAXM], w[MAXM], ecnt;
void add(int u, int v, int c) {
    to[++ecnt] = v; w[ecnt] = c; nxt[ecnt] = head[u]; head[u] = ecnt;
}
long long dis[MAXN];
int cnt[MAXN];                      // 到该点的最短路径边数
bool inq[MAXN];
int N, M;                           // N 为原变量个数

bool spfa(int s, int tot) {         // tot 为总点数（含超级源），返回 false 表示负环
    const long long INF = 1LL << 60;
    for (int i = 0; i < tot; ++i) { dis[i] = INF; inq[i] = false; cnt[i] = 0; }
    queue<int> q; dis[s] = 0; q.push(s); inq[s] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop(); inq[u] = false;
        for (int e = head[u]; e; e = nxt[e]) {
            int v = to[e];
            if (dis[u] + w[e] < dis[v]) {
                dis[v] = dis[u] + w[e];
                cnt[v] = cnt[u] + 1;
                if (cnt[v] >= tot) return false;     // 边数达到点数，必有环
                if (!inq[v]) { q.push(v); inq[v] = true; }
            }
        }
    }
    return true;
}
int main() {
    scanf("%d%d", &N, &M);
    for (int i = 0; i < M; ++i) {
        int u, v, c; scanf("%d%d%d", &u, &v, &c);
        add(v, u, c);                        // x_u - x_v <= c  →  边 v→u
    }
    for (int i = 1; i <= N; ++i) add(0, i, 0);   // 超级源，保证全图可达
    if (!spfa(0, N + 1)) { puts("NO"); return 0; }
    for (int i = 1; i <= N; ++i) printf("%lld\n", dis[i]);
    return 0;
}
```

两个经典模型：① 前缀和建模——「区间 `[l,r]` 内至少选 c 个点」写成 $s_r - s_{l-1} \ge c$，再补 $0 \le s_i - s_{i-1} \le 1$；② 工序排期——「任务 i 比 j 晚至少 d」即 $x_i - x_j \ge d$。这两类题的关键都在于先想清楚「谁是变量」，再看清不等号方向。

易错点：

- **建边方向反了。$x_i - x_j \le c$ 是 $j \to i$。方向写反时小数据加上特殊构造仍可能过，务必用「x_i 的上界来自 x_j」来记忆。**
- **漏加超级源。只从 1 号点跑，不连通分量的负环查不出来，判成有解。**
- **负环阈值写错。用「最短路经过边数 ≥ 总点数」；写成「入队次数 > n」时阈值差一位，且两种写法在强数据下都会 TLE。**

- INF 太小。c 到 10^9、路径长 10^5 段时距离到 10^{14}，必须 `long long` 且 INF 取 1<<60；用 `0x3f3f3f3f` 会误判松弛。
- 求最小解仍用最短路。此时答案是 `-dis`（对取负后的图），直接输出 `dis` 会得到相反的一组值。
- 被卡 SPFA。网格型负权图能把 SPFA 卡到 O(nm) 上限，可先加 SLF（距离更小者插队首），或确认题目是否本意就是 Dijkstra + 势能。

28.11 2-SAT：从子句到强连通分量

一句话判定：每个条件都能写成「`a 为真 或 b 为真`」这种二元子句（a、b 是某个变量的取值或其否定），且要求给出任意一组满足全部条件的赋值——就是 2-SAT。判定的本质是：把子句化成蕴含，看蕴含图上会不会推出矛盾。

建图规则（两句话，但必须一字不差）

- 子句 `a ∨ b` 等价于两条蕴含：`¬a → b` 与 `¬b → a`。两条都要加，它们是互为逆否的，缺一条就丢了一半约束。
- 若题目直接给出蕴含 `a → b`，同样要补逆否边 `¬b → ¬a`。

判定条件：对蕴含图求强连通分量。对每个变量 x，若「x 为真」与「x 为假」两个点在同一个 SCC 内，则 x 既蕴含自己为真又蕴含自己为假，必然矛盾，整体无解。赋值：Tarjan 的出栈编号是按拓扑序逆序给的（汇点先编号、编号更小），取编号更小的那一侧为真即可，即 `scc[真] < scc[假]` 时该变量为真。

```
#include <cstdio>
#include <vector>
#include <algorithm>
using namespace std;
const int MAXN = 200005;             // 变量个数上限
const int MAXV = 2 * MAXN + 5;       // 点数 = 2n
int n, m;
vector<int> g[MAXV];
int dfn[MAXV], low[MAXV], scc[MAXV], stk[MAXV], tp, timer, cnt;
bool inStk[MAXV];

void tarjan(int u) {
    dfn[u] = low[u] = ++timer;
    stk[++tp] = u; inStk[u] = true;
    for (size_t i = 0; i < g[u].size(); ++i) {
        int v = g[u][i];
        if (!dfn[v]) { tarjan(v); low[u] = min(low[u], low[v]); }
        else if (inStk[v]) low[u] = min(low[u], dfn[v]);
    }
    if (low[u] == dfn[u]) {
        ++cnt; int x;
        do { x = stk[tp--]; inStk[x] = false; scc[x] = cnt; } while (x != u);
    }
}
inline int neg(int x) { return x ^ 1; }              // 2i = 真，2i^1 = 假
inline int node(int i, int val) { return 2 * i + (val ? 0 : 1); }
void addClause(int a, int b) {                       // 子句 a ∨ b
    g[neg(a)].push_back(b);
    g[neg(b)].push_back(a);
}
int main() {
    scanf("%d%d", &n, &m);
    for (int k = 0; k < m; ++k) {
        int i, a, j, b;
        scanf("%d%d%d%d", &i, &a, &j, &b);         // x_i = a 或 x_j = b
        addClause(node(i - 1, a), node(j - 1, b));
    }
    for (int i = 0; i < 2 * n; ++i) if (!dfn[i]) tarjan(i);
    for (int i = 0; i < n; ++i)
        if (scc[2 * i] == scc[2 * i + 1]) { puts("IMPOSSIBLE"); return 0; }
    puts("POSSIBLE");
    for (int i = 0; i < n; ++i)
        printf("%d ", scc[2 * i] < scc[2 * i + 1] ? 1 : 0);
    return 0;
}
```

建模清单（把自然语言翻成子句，是这类题唯一的难点）：

约束	子句写法
a、b 至少一个成立	`(a ∨ b)`
a、b 不能同时成立	`(¬a ∨ ¬b)`
a 蕴含 b（若 a 则 b）	`(¬a ∨ b)`，等价于加边 a→b 与 ¬b→¬a
a 必须成立	`(a ∨ a)`
a、b 恰好一个成立	两条：`(a ∨ b)` 且 `(¬a ∨ ¬b)`
a、b 取值相同 / 不同	相同：`(a ∨ ¬b)` + `(¬a ∨ b)`；不同：`(a ∨ b)` + `(¬a ∨ ¬b)`

建模的三条边界：① 变量只能取两值是 2-SAT 的硬前提，多值（k ≥ 3 种状态，如 3-染色）是 NP-hard，不能套本模板；② k 元子句（k ≥ 3 个文字）不能直接写，k 小时按 2^k 展开成二元子句，k 大且约束呈区间形态时用线段树辅助点优化建图；③ 「恰好选 m 个」这类计数型约束不是 2-SAT，别硬套。

字典序最小解：Tarjan 求的是任意一组解。若题目要求编号小的变量优先取 0，得换成 O(nm) 的贪心 DFS：按变量编号从小到大，先试「取假」，沿蕴含图染色传播，冲突就回滚、改试「取真」，仍冲突则无解。

```
int col[MAXV];                        // 0 未定，1 选定，-1 因被否定而排除
int mark[MAXV], mp = 0;               // mark 记录本次染色，用于回滚
bool dfs(int u) {
    if (col[u] == 1) return true;
    if (col[u] == -1) return false;
    col[u] = 1;  mark[mp++] = u;
    col[u ^ 1] = -1; mark[mp++] = u ^ 1;
    for (size_t i = 0; i < g[u].size(); ++i)
        if (!dfs(g[u][i])) return false;
    return true;
}
void rollback(int save) {             // 撤销到快照 save
    while (mp > save) { int v = mark[--mp]; col[v] = 0; }
}
// 主流程：i 从 0 到 n-1，save = mp；先 dfs(2*i+1)（取假），失败则 rollback(save)
// 后 dfs(2*i)（取真）；两次都失败即无解。总复杂度 O(nm)，n 上到几千才可用。
```

环节	复杂度	说明
建图	O(n + m)	每条子句两条边，点数 2n
Tarjan 缩点	O(n + m)	递归写法注意栈深
判定 + 输出解	O(n)	比较 `scc[2i]` 与 `scc[2i+1]`
字典序最小解	O(nm)	贪心 DFS，只在小数据下用

易错点：

- 只加一条边。子句 `a ∨ b` 必须有 `¬a→b` 与 `¬b→a` 两条。漏一条时弱数据照样过，遇到「必须靠逆否方向才能推出矛盾」的点才 WA。
- SCC 编号方向记反。这是 2-SAT 最高频的错误：Tarjan（出栈编号）取 `scc[真] < scc[假]` 为真，Kosaraju 按正序编号时不等号相反。自检办法很便宜——用单变量两条子句各测一次：`(x∨x)` 强制真、`(¬x∨¬x)` 强制假，方向错了立刻暴露。
- 标号映射不统一。全篇只用一个约定（`2i` 真、`2i^1` 假，取反一律 `x^1`），别中途混用 `x+n` 的写法；两种写法混用时 `^1` 会取到别的变量上。
- 递归 Tarjan 爆栈。2n 个点、链式图可达 2n 层递归，Windows 默认 1 MB 栈在 $n = 2\times10^5$ 时会崩。改为迭代写法，或 Linux 下 `ulimit -s` 调大；`#pragma comment(linker, ...)` 只在 MSVC 有效，考场环境不要依赖。
- 多组数据不清空。`dfn/low/scc/inStk` 与 `timer/cnt/tp` 都要重置，且 `g` 的每个点都要 clear；只清前 n 个点是常见疏漏（应为 2n）。
- 边加在「变量」而不是「文字」上。点代表「某变量取某值」，不是「变量」。写成 `g[i].push_back(j)` 之类的变量编号，图就完全错了。
- 把「有解」当「解唯一」。SCC 只判可行性。要判断某变量是否被迫取某值，加一条强制子句（如 `(¬x ∨ ¬x)` 强制 x 为假）重跑一次，不可行则 x 必为真——代价 O(n+m)，比求传递闭包现实。

28.12 点分治与点分树

一句话判定：题目要统计树上「满足某条件的路径」的条数或最值，路径总数是 $O(n^2)$ 不可能枚举，但「经过某个点」的两条链可以合并统计——这时就用点分治。常见问法：距离 $\leqslant$ K 的点对个数、距离恰为 K 的点对是否存在、边数 $\leqslant$ k 的路径最小边权和、树上路径第 k 大（配合二分或树套树）。

三条支撑它成立的事实（其余全是实现细节）：

- 正确性：删掉重心 `c` 后，树上任意一条路径要么经过 `c`，要么完全落在删掉 `c` 后的某个连通块内——两者必居其一，所以「按层统计经过该层重心的路径」可以覆盖所有路径且不重不漏。
- 复杂度：每次取重心，删掉后最大连通块不超过原大小的一半，故递归深度 $O(\log n)$；每层统计的代价是「当前连通块大小」，各层求和即 $O(n \log n)$。
- 代价：统计前通常要排序，于是总复杂度 $O(n \log^2 n)$。若值域小到能开桶（如边权 $\leqslant 100$、$K \leqslant 10^7$），可做到 $O(n \log n)$。

步骤	单层代价	说明
求连通块大小 + 找重心	O(size)	两遍 dfs，判据是「没有儿子的子树超过一半」
收集各点到重心的距离	O(size)	只走未删除的点
排序 + 双指针统计	O(size log size)	本节模板的做法
递归各子树	—	深度 O(log n)，各层 size 之和 O(n log n)

必须理解的一步：容斥。我们先把「重心到所有点的距离」放进同一个数组排序统计，得到的点对里混进了「两点在同一子树内」的情形。它们的真实路径并不经过重心（实际更短），却被按 $d_x + d_y$ 计了一次，所以要逐个子树减去「只在该子树内部」的点对数。不减也能跑出结果，只是每对点会在它所属的每一层被重复计一次——小数据下答案偏大而看不出错，这是本节最高频的失分点。

```
#include <cstdio>
#include <algorithm>
using namespace std;
typedef long long ll;

const int MAXN = 100005;
int n, K;
struct Edge { int to, w, nxt; } e[MAXN << 1];
int head[MAXN], ec;
void addEdge(int u, int v, int w) { e[++ec] = (Edge){v, w, head[u]}; head[u] = ec; }

bool vis[MAXN];
int sz[MAXN], dis[MAXN], cntd;
ll ans;

int getSize(int u, int fa) {                 // 求当前连通块内各子树大小
    sz[u] = 1;
    for (int i = head[u]; i; i = e[i].nxt) {
        int v = e[i].to;
        if (v == fa || vis[v]) continue;
        sz[u] += getSize(v, u);
    }
    return sz[u];
}
int getCentroid(int u, int fa, int tot) {    // tot = 当前连通块大小
    for (int i = head[u]; i; i = e[i].nxt) {
        int v = e[i].to;
        if (v == fa || vis[v]) continue;
        if (sz[v] * 2 > tot) return getCentroid(v, u, tot);
    }
    return u;                                // 没有超过一半的儿子，u 就是重心
}
void getDis(int u, int fa, int d) {
    dis[++cntd] = d;
    for (int i = head[u]; i; i = e[i].nxt) {
        int v = e[i].to;
        if (v == fa || vis[v]) continue;
        getDis(v, u, d + e[i].w);
    }
}
ll countLE(int *d, int c) {                  // d[1..c] 内 d_i + d_j ≤ K 的点对数（i < j）
    sort(d + 1, d + c + 1);
    ll res = 0;
    int j = c;
    for (int i = 1; i <= c; ++i) {
        if (j <= i) break;                   // i 继续增大更不可能有解，直接收尾
        while (j > i && d[i] + d[j] > K) --j;
        if (j > i) res += j - i;
    }
    return res;
}
void solve(int u) {
    getSize(u, 0);
    int c = getCentroid(u, 0, sz[u]);
    vis[c] = true;                           // 重心就地删除，永久生效

    cntd = 0; dis[++cntd] = 0;               // 重心自身，距离 0，不能漏
    for (int i = head[c]; i; i = e[i].nxt) {
        int v = e[i].to;
        if (vis[v]) continue;
        int lo = cntd + 1;
        getDis(v, c, e[i].w);
```

```
        ans -= countLE(dis + lo - 1, cntd - lo + 1);   // 容斥：减去同一子树内部
    }
    ans += countLE(dis, cntd);               // 所有经过重心 c 的点对

    for (int i = head[c]; i; i = e[i].nxt) {
        int v = e[i].to;
        if (!vis[v]) solve(v);
    }
}
int main() {
    scanf("%d%d", &n, &K);
    for (int i = 1; i < n; ++i) {
        int u, v, w; scanf("%d%d%d", &u, &v, &w);
        addEdge(u, v, w); addEdge(v, u, w);
    }
    solve(1);
    printf("%lld\n", ans);
    return 0;
}
```

改问法怎么改：问「距离恰为 K 是否存在」时，把排序 + 双指针换成「对每个 d_i 在有序数组里二分找 K − d_i」，或直接开桶标记；问「路径边数 ≤ k 的最小边权和」时，把 `dis` 改成存「边数」并按边数分组、每组维护最小权值和最大值，用单调队列合并；问「树上路径第 k 大」时，二分数值再套本节模板判断「≥ 该值的边数」，复杂度 $O(n \log^2 n \log V)$。

点分树（动态点分治）：把所有层找到的重心按「父重心 → 子重心」连成一棵新树。它的深度是 $O(\log n)$，每个原点在树上有 $O(\log n)$ 个祖先重心，且「原树上两点路径必经它们最近公共祖先重心」。于是「单点修改 + 全局查询」可以按祖先链更新，单次 $O(\log^2 n)$。实现上是每个重心维护一桶「到本重心的距离」，再减去「到父重心的距离」做容斥——本质仍是本节那一步容斥，只是搬到了树上。代价是常数与空间都翻几倍，考场上先问「能不能离线」，能离线就别写点分树。

对比项	点分治	边分治
划分方式	删点，分成若干块	删边，只分两块
统计代价	合并多个子树，容斥做多次减法	只有两棵子树，合并一次即可，常数更小
前置处理	无	度数高时要「三度化」（把点拆成二叉树），代码显著变长
结论	默认用点分治；只有当点分治的容斥部分写不对、且图接近链或二叉树时，边分治才值得考虑	

易错点：

- **忘减子树内部。答案偏大且随深度放大。自测用例：四点链 1—2—3—4、K = 2，正确答案 5（(1,2)(1,3)(2,3)(2,4)(3,4)），漏减会得到更大的数。**
- **重心判据写成 `sz[v] > tot/2`。整除会漏掉 `sz[v] = tot/2` 的等号情形，链长为偶数时可能选中非重心，深度退化、复杂度悄悄变成 $O(n^2)$。写成 `sz[v] * 2 > tot`。**

- `getSize` 与 `getCentroid` 的 `tot` 用错。递归进子树后 `tot` 仍是「上一层连通块大小」，所以只能先 `getSize(u,0)` 取 `sz[u]` 再传进去，不能拿最初的 n。
- 把 `dis[0] = 0` 漏掉。重心自身不在数组里，所有「以重心为端点的路径」全部丢失，答案系统性偏小。
- 容斥时数组切片起止算错。`countLE(dis + lo - 1, …)` 这类「指针前移一位」的写法必须配套「长度 = cntd − lo + 1」，两者差一会把上一棵子树的数据当成本子树的减掉。
- vis 中途清除。分治是按「永久删点」推进的，清 vis 会让已删除的重心重新参与统计，直接重复计数。
- 递归栈溢出。`getSize/getDis` 的深度等于当前连通块的深度，一条 10^5 点的链上最坏约 n/2 层。Linux 下 `ulimit -s unlimited` 或改迭代；仅靠 `#pragma comment(linker,…)` 在 g++ 上无效。
- 多组数据未清空。`ec`、`ans`、`vis`、`cntd` 全部重置，边表数组不必 memset，但 `head` 必须清。

28.13 竞赛图：得分序列、缩点链与哈密顿路径

一句话判定：n 个点两两之间恰有一条有向边，就是竞赛图（锦标赛图）。它出现在两类题里：一是直接给一张竞赛图，问「谁可能夺冠」「三元环（石头剪刀布循环）有多少个」「输出一条走遍全场的路径」；二是只给每个点的得分（出度）序列，问「这个序列能不能由某场比赛产生」「最少能分成几个层级」。后一类是 Landau 定理的地盘——O(n log n) 判完，根本不用把图建出来。

五条结论（竞赛图几乎所有性质都从这五条出）：

1. Landau 定理（可实现性）：把得分 $d_1 \le d_2 \le \dots \le d_n$ 非降序排列，存在竞赛图以它为得分序列，当且仅当对每个 $k = 1 \cdots n$ 都有 $d_1+\dots+d_k \ge C(k,2)$，并且 $k = n$ 时取等号（即 $\Sigma d_i = C(n,2)$）。
2. 强连通判据：同一排序下，竞赛图强连通 $\Leftrightarrow$ 对所有 $k < n$ 严格有 $\Sigma_{i \le k} d_i > C(k,2)$。直觉：前 k 个点之间最多 C(k,2) 条边，得分之和超过它，说明有点赢到了集合外面，不存在「前 k 个点自成封闭集」的割。
3. Camion 定理：强连通竞赛图必有哈密顿回路。
4. Rédei 定理：任意竞赛图都有哈密顿路径，不需要任何条件。所以「最长链」在竞赛图里恒为 n，真正的问题变成「怎么在 $O(n^2)$ 内把它构造出来」。
5. 缩点必成链：缩掉强连通分量后，任意两块之间的边全部同向（有双向边就说明能互达，早该合并），于是缩点图是一条链、拓扑序唯一，链头那块能到达所有点——这正是「谁可能夺冠」的答案。

常见问法	判据 / 公式	复杂度
得分序列是否可实现	Landau：前缀和 $\geqslant C(k,2)$，且 $k=n$ 取等	$O(n \log n)$
是否能出现唯一冠军 / 图是否强连通	对所有 $k<n$ 严格 $\Sigma_{i\leqslant k} d_i > C(k,2)$	$O(n \log n)$
最少分成几个层级（缩点个数）	$\#\{k : \Sigma_{i\leqslant k} d_i = C(k,2)\}$	$O(n \log n)$
三元环（循环胜负）个数	$C(n,3) - \Sigma\, C(d_i,2)$	$O(n)$
输出一条哈密顿路径	插入法，逐点插到「最后一个能赢它的点」之后	$O(n^2)$
是否传递（无环，可完全排名）	得分两两不同 $\Leftrightarrow$ 得分恰为 $0,1,\cdots,n-1$	$O(n \log n)$

三元环公式为什么成立：任取三点，要么成环（每点出度 1，对 $\Sigma C(d_i,2)$ 贡献 0），要么是传递的（有唯一一个「中心」点出度 2，恰好被 $C(d_i,2)$ 数到一次）。所以「选了某个点的两条出边」与「一个传递三角形」一一对应，环数 = 总数 − 传递数。这个公式把 $O(n^3)$ 的三重循环压成 $O(n)$ 的一次求和，是竞赛图题最高频的一步。

可编译模板

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

// 1) 得分序列是否可实现（Landau 定理）
bool landau(vector<int> d) {
    int n = d.size();
    sort(d.begin(), d.end());
    ll s = 0;
    for (int k = 1; k <= n; ++k) {
        s += d[k - 1];
        if (s < (ll)k * (k - 1) / 2) return false;   // 前缀不足，直接否决
    }
    return s == (ll)n * (n - 1) / 2;                 // 总和必须恰为 C(n,2)
}

// 2) 是否强连通；同时给出缩点后的分量个数 blocks
//    前缀等号处「断一刀」，断点数就是链上的块数
bool strongBlocks(vector<int> d, int& blocks) {
    int n = d.size();
    sort(d.begin(), d.end());
    ll s = 0; blocks = 0;
    bool ok = true;
    for (int k = 1; k <= n; ++k) {
        s += d[k - 1];
        ll need = (ll)k * (k - 1) / 2;
        if (s == need) ++blocks;                      // 这里是一个块边界
        if (k < n && s <= need) ok = false;          // 等号即封闭集，非强连通
    }
    return ok;
}

// 3) 三元环个数（输入每个点的出度）
ll triangles(const vector<int>& d) {
    int n = d.size();
    if (n < 3) return 0;
    ll tot = (ll)n * (n - 1) * (n - 2) / 6, pass = 0;
    for (int x : d) pass += (ll)x * (x - 1) / 2;     // 传递三角形
    return tot - pass;
}

// 4) 构造一条哈密顿路径：g[u][v] = 1 表示 u 胜 v
vector<int> hamiltonPath(const vector<vector<char>>& g) {
    int n = g.size();
    vector<int> p{0};
    for (int x = 1; x < n; ++x) {
        int i = (int)p.size() - 1;
        while (i >= 0 && g[x][p[i]]) --i;            // 找到最后一个「能赢 x」的点
        p.insert(p.begin() + i + 1, x);              // x 一定赢它后面所有人
    }
    return p;
}
int main() {
    int n; scanf("%d", &n);
    vector<vector<char>> g(n, vector<char>(n, 0));
    for (int u = 0; u < n; ++u)
        for (int v = 0; v < n; ++v) { int x; scanf("%d", &x); g[u][v] = x; }
    vector<int> d(n, 0);
    for (int u = 0; u < n; ++u) for (int v = 0; v < n; ++v) d[u] += g[u][v];

    printf("landau=%d triangles=%lld\n", (int)landau(d), triangles(d));
    int b; printf("strong=%d blocks=%d\n", (int)strongBlocks(d, b), b);
```

```
    for (int x : hamiltonPath(g)) printf("%d ", x);
    return 0;
}
```

复杂度：三条判定都只依赖得分序列，瓶颈是排序，$O(n \log n)$；三元环 $O(n)$；哈密顿路径 $O(n^2)$（存边 $O(n^2)$ 的矩阵本身就是下界）。记忆量：`n = 5000` 的邻接矩阵用 `char` 是 25 MB，换成 `int` 立刻 100 MB——这是本节最常见的 MLE 来源。

典型用法

- 「谁可能夺冠」：先用前缀断点定位链头那一块（第一个 `s == C(k,2)` 的 k 之前，含 k），块内所有点都可能夺冠，块外的点必然被链头全歼。整题 $O(n \log n)$，不用 Tarjan。
- 构造题：给定满足 Landau 的得分序列要还原一张竞赛图，用「每次取当前出度最大的点，向剩下出度最大的 d 个点连边，然后删掉它」的贪心（Havel–Hakimi 式），$O(n^2)$ 构造 + $O(n \log n)$ 判定。判定不通过就别浪费时间构造。
- 计数题：环数、传递三元组数、得分序列的极端值（最小环数对应传递图，得分为 $0 \cdots n-1$）都可以由本节公式直接给出闭式，不必 DP。

易错点：

- 排序方向用反。Landau 的前提是非降序；写成 `greater` 后前缀和变成后缀，判据符号全反。小数据（$n \leqslant 4$）常常照样过，因为对称性掩盖了错误——必须用 $n \geqslant 6$ 的随机数据对拍。
- $k = n$ 的等号和没检查。只判不等式会放过 `Σd = C(n,2) - 1` 这类序列，答案该是「不可能」却输出「可能」。
- 强连通判据循环到 $k = n$ 还用严格不等式。$k = n$ 时 `Σd = C(n,2)` 必然取等，于是「所有图都不强连通」。循环条件必须写 `k < n`。
- 三元环里 d 混用出度与入度。两者的 `ΣC(d,2)` 恰好相等，所以单独用哪个都对，但一行里混着写就错。
- 没有先验证得分序列合法就套公式。对不合法序列，`ΣC(d_i,2)` 可能大于 `C(n,3)`，环数算出负数。前置一次 `landau()`。
- 插入法方向写反。`g[u][v] = 1` 表示 u 胜 v 时，循环条件应是「`g[x][p[i]]` 为真就继续左移」，写反会把 x 插到「x 输给它」的点前面，路径上出现反向边。
- 插入位置差一位。找到的是「最后一个能赢 x 的点」`i`，应插在 `i + 1`；写成 `i` 会破坏 `p[i-1] → x` 这条边。
- 用 Tarjan 求缩点个数。能过，但白写了 60 行、还要 $O(n^2)$ 存图；前缀断点法 $O(n \log n)$ 且只用得分序列。
- $n = 1$ 或 $n = 2$ 的退化。三元环公式在 $n < 3$ 时应直接返回 0（虽然公式本身也给 0，但显式判更安全）；哈密顿路径在 $n = 1$ 时应输出唯一的点。

自查：本轮把四个模板逐行译成 Python，对随机竞赛图（$n \leqslant 8$，每对点随机定向）做了四组校验——① `landau` 判定与「暴力枚举全部 $2^{C(n,2)}$ 种定向」的存在性一致；② `strongBlocks` 的强连通结论与 Tarjan

一致、`blocks` 与 SCC 个数逐一相等；③ `triangles` 与 $O(n^3)$ 三重循环计数相等；④ `hamiltonPath` 输出长度恒为 n 且每条相邻边方向正确。另单独验证了传递竞赛图的得分为 0…n−1、以及「链头块 = 能到达所有点的点集」这一断言。

28.14 Dilworth 定理：最小链覆盖、最长反链与最小路径覆盖

一句话判定：一个偏序集里「最少用几条链能把所有元素串起来」与「最多能挑出多少个两两不可比的元素」，永远是同一个数。这就是 Dilworth 定理。它在竞赛里的标准形态是：DAG 的最小路径覆盖 = n − 二分图最大匹配。看起来是个匹配题，实际上每次考的都是「你怎么把题面里的偏序关系抽出来」。

三个概念先分清：

- 偏序关系 $\preccurlyeq$：满足自反、反对称、传递。竞赛里的偏序几乎都来自两种：① 序列上「下标更小且值更小」；② DAG 上的可达性（$u \preccurlyeq v \Leftrightarrow u$ 能到达 v）。注意「能到达」只有在无环时才是偏序，有环必须先 SCC 缩点。
- 链：集合内任意两个元素都可比（全序子集）。在 DAG 里就是一条路径。
- 反链：集合内任意两个元素都不可比。在 DAG 里就是「互不可达的点集」。

两条对偶定理（一个管链、一个管反链，别记混）：

定理	结论	在 DAG 上的形态
Dilworth	最小链划分 = 最长反链长度	最少几条路径覆盖全部点（不可重叠）= 最大不可达点集
Mirsky（对偶）	最小反链划分 = 最长链长度	最少几个「互不可达层」= 最长路径长度（直接 DP）

Mirsky 那一侧没有技术含量：按「到该点的最长路长度」分层，同层点必互不可达（否则短的那层可以更长），层数就是最长链长。所以真正要算的只有 Dilworth 那一侧，而它的算法只有一句话：拆点建二分图，跑最大匹配。

归约怎么建：把每个点 i 拆成左部 i_{out} 与右部 i_{in}；对每条偏序关系 $u \preccurlyeq v$（$u \neq v$）连边 $u_{out} \to v_{in}$。一条匹配边 u_{out}-v_{in} 的含义是「在链里 v 紧接在 u 后面」。每个左部点最多用一次（每个点最多有一个后继）、每个右部点最多用一次（最多一个前驱），所以匹配就是一组「链内相邻关系」。n 个点、M 条匹配边，恰好拼成 `n - M` 条链——匹配越大，能拼起来的链越少，这就是公式的来源。

四种问法的对照表（差一个传递闭包就会全错，务必逐行对上）：

题目问的是	连边用什么图	答案
DAG 最小路径覆盖（点不可重复）	原图的边	`n − 匹配`
DAG 最小路径覆盖（点可重复经过）	传递闭包后的边	`n − 匹配`
偏序集最小链覆盖	传递闭包后的边	`n − 匹配`
最长反链长度	传递闭包后的边	`n − 匹配`（数值同上）
二分图最小点覆盖	同一张拆点图	匹配（Kőnig 定理）
二分图最大独立集	同一张拆点图	`n − 匹配`（= 补集）

最长反链怎么构造出来（不是只要长度）。Dilworth 的证明是构造性的，落到代码上是三步：

1. 跑完匹配后，从所有未匹配的左部点出发做交替搜索——沿非匹配边往右走，沿匹配边往左走，得到可达集 Z（左侧可达集 Z_L、右侧可达集 Z_R）。
2. 最小点覆盖 $= (L \setminus Z_L) \cup (R \cap Z_R)$，最大独立集取其补 $(L \cap Z_L) \cup (R \setminus Z_R)$。
3. 反链 $= \{\, i : i_{out}$ 与 i_{in} 都在最大独立集里 $\} = \{\, i : i_{out} \in Z_L$ 且 $i_{in} \notin Z_R \,\}$。左右两侧都要满足——只取一侧会得到可比点对。

可编译模板

```
#include <bits/stdc++.h>
using namespace std;

struct Dilworth {
    int n;
    vector<vector<char>> reach;   // reach[u][v] = u 可到 v（传递闭包后）
    vector<vector<int>> g;       // 左部 u → 右部 v
    vector<int> ml, mr, tag;      // 匹配与时间戳
    int tim = 0;

    void init(int n_) {
        n = n_;
        reach.assign(n, vector<char>(n, 0));
        for (int i = 0; i < n; ++i) reach[i][i] = 0;   // 自身不算偏序，避免自环
        ml.assign(n, -1); mr.assign(n, -1); tag.assign(n, 0);
    }
    void addEdge(int u, int v) { reach[u][v] = 1; }     // 原图有向边 u → v

    void closure() {                                    // Floyd 传递闭包，O(n^3)
        for (int k = 0; k < n; ++k)
            for (int i = 0; i < n; ++i)
                if (reach[i][k])
                    for (int j = 0; j < n; ++j)
                        if (reach[k][j]) reach[i][j] = 1;
    }

    bool dfs(int u) {
        for (int v : g[u]) {
            if (tag[v] == tim) continue;
            tag[v] = tim;
            if (mr[v] == -1 || dfs(mr[v])) { mr[v] = u; ml[u] = v; return true; }
        }
        return false;
    }

    int matching(bool useClosure) {                     // 匈牙利 O(n·E)
        for (int u = 0; u < n; ++u) g[u].clear();
        fill(ml.begin(), ml.end(), -1);
        fill(mr.begin(), mr.end(), -1);
        for (int u = 0; u < n; ++u)
            for (int v = 0; v < n; ++v)
                if (reach[u][v] || (!useClosure && raw[u][v])) g[u].push_back(v);
        int res = 0;
        for (int u = 0; u < n; ++u) { ++tim; if (dfs(u)) ++res; }
        return res;
    }
    vector<vector<char>> raw;                           // 未闭包的原图边

    int minPathCover()  { return n - matching(false); }  // 点不可重叠
    int minChainCover() { return n - matching(true);  }  // 最少链 / 最长反链

    vector<int> antichain() {                           // 构造一条最长反链
        int M = matching(true); (void)M;
        vector<char> zl(n, 0), zr(n, 0);
        queue<int> q;
        for (int u = 0; u < n; ++u) if (ml[u] == -1) { zl[u] = 1; q.push(u); }
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : g[u]) {
                if (v == ml[u]) continue;               // 只走非匹配边
                if (zr[v]) continue;
                zr[v] = 1;
```

```
                if (mr[v] != -1 && !zl[mr[v]]) { zl[mr[v]] = 1; q.push(mr[v]); }
            }
        }
        vector<int> a;
        for (int i = 0; i < n; ++i) if (zl[i] && !zr[i]) a.push_back(i);
        return a;
    }
};

int main() {
    int n, m; scanf("%d %d", &n, &m);
    Dilworth D; D.init(n); D.raw.assign(n, vector<char>(n, 0));
    for (int i = 0; i < m; ++i) {
        int u, v; scanf("%d %d", &u, &v); --u; --v;
        D.addEdge(u, v); D.raw[u][v] = 1;
    }
    printf("minPathCover=%d\n", D.minPathCover());
    D.closure();
    printf("minChainCover=%d maxAntichain=%d\n", D.minChainCover(), D.minChainCover());
    for (int x : D.antichain()) printf("%d ", x + 1);
    puts("");
    return 0;
}
```

复杂度：传递闭包朴素 Floyd $O(n^3)$，改用 `bitset` 逐行或运算压到 $O(n^3/64)$；匈牙利最坏 $O(n\cdot E) = O(n^3)$，边数是 $O(n^2)$。实践阈值：$n \le 500$ 用本模板；$n \le 2000$ 换 Hopcroft–Karp（$O(E\sqrt{n})$）+ 邻接表；$n \ge 10^4$ 时这类题基本不会让你显式建全图（要么偏序有特殊结构，要么答案另有闭式）。内存 `reach` 用 `char` 是 n^2 字节，$n = 5000$ 时 25 MB，可接受；写成 `int` 立刻 100 MB。

典型用法

- **「导弹拦截」型的序列划分：最少用多少条不上升子序列覆盖整个序列？把 $i \preccurlyeq j \Leftrightarrow i < j$ 且 $a_i \ge a_j$ 当偏序，答案就是最长严格上升子序列长度——这是 Dilworth 在序列上的特化，也是最好的自检用例（模板跑一遍必须等于 `O(n log n)` 的 LIS）。**
- **课程 / 任务依赖：给一张 DAG 表示前置关系，问「最少几个人并行能在不违反依赖的前提下干完全部任务」，就是最小链覆盖，`n − 匹配`。**
- **「最多选多少个互不影响」：凡是题面出现「选出一个集合，任意两个之间不存在依赖 / 不可比 / 不能同时选」，先想最长反链 = `n − 匹配`，而不是去想状压 DP。**
- **输出方案：最小链覆盖的方案由匹配边直接给出（u_{out}-v_{in} 匹配上就把 `v` 接在 `u` 后面），链头是没有入匹配的点；最长反链才需要上面那三步交替搜索。**

易错点：

- **该闭包时没闭包。最常见的错：求「链覆盖」却用原图边建匹配。原图只连直接前驱，得到的是「最小路径覆盖（点不可重叠）」；链覆盖允许链内跨过中间点，必须先把可达性传递完。两者在链状数据上答案相同，一遇到分叉就分道扬镳，所以必须用有分叉、有跳边的数据对拍。**
- **Dilworth 与 Mirsky 记反。「最小链覆盖 = 最长反链」对，「最小反链覆盖 = 最长链」也对，但把前者写成「最小的反链覆盖等于最长反链」就全错了。记法：两个「链」字在对角——最小链 ↔ 最长反链，最小反链 ↔ 最长链。**

- 有环图直接建偏序。「可达」在有环时不是反对称的，`u ≼ v` 与 `v ≼ u` 同时成立，匹配里会出现 u_{out}-v_{in} 与 v_{out}-u_{in} 都匹配上，链变成环。先 SCC 缩点。
- 把对角元置 1。偏序定义里 `u ≼ u` 成立，但建图时必须去掉自环，否则每个点都能「自己接自己」，匹配直接变成 n，答案是 0。模板里 `reach[i][i] = 0` 这一行不能省。
- 反链构造只取一侧。只判 $i_{out} \in Z_L$ 会挑进可比点对（典型症状：个数算对了但两两检查不通过）；必须同时要求 $i_{in} \notin Z_R$。
- 匈牙利的时间戳没换。`tag` 只在每轮外层循环 `++tim`，若在 `dfs` 内部清空 `vis` 会退化成指数级；若忘了 `++tim` 则第二次增广直接被跳过，匹配偏小。
- 递归深度。增广路最长可到 `n`，`n = 10^5` 的链式数据会爆栈——虽然那个规模本来就不该用匈牙利，但把 `dfs` 写成递归又不加剪枝是常见翻车点。
- `n = 0` 与孤立点。孤立点没有任何边，答案是 `n`（每个点自成一条链）；模板若先判空会直接返回 0。
- 把「点可重复」的路径覆盖当「不可重复」算。允许重复时要在闭包图上跑；不允许重复才用原图。判别方法：题面说「路径之间不能共用点」还是「可以重复经过」。

自查：本轮把模板逐行译成 Python 后做了三组对拍——① 随机 DAG（`n ≤ 7`，边概率 0.35，先随机拓扑序保证无环）上，`minPathCover` 与「子集 DP 精确求点不相交路径覆盖」全部相等（600 组）；② 同一批图闭包后，`minChainCover` 与「枚举全部子集找最大反链」全部相等（600 组），同时验证了 Dilworth 的两侧数值一致；③ 反链构造输出的点集两两不可达且大小恰为 `n − 匹配`（600 组，零反例）。另用「导弹拦截」型序列（`n ≤ 9` 随机排列）验证 `minChainCover` 等于 LIS 长度（400 组全中），并用传递竞赛图（得分 0…n−1）验证最小链覆盖恰为 1。

28.15 最小费用最大流：SSP、势能与上下界

一句话判定：最大流只回答「能挤过去多少」，一旦每单位流量还带着价格（运输单价、时间成本、切换代价），目标就变成「在流量最大的前提下让总费用最小」，这就是费用流（MCMF）。它同时是「带权二分图匹配」的通用解法：源连左部、左连右部、右连汇，边权取费用，跑出来的就是最小权匹配；把费用取负即最大权匹配。28.7 只给了它一段话，本节把模板、势能与上下界补全。

算法只有一条主线：SSP（逐次最短路）。每一步在残量网络上找一条 `s→t` 的单位费用最小的路，沿它能推多少推多少，直到不存在增广路。正确性的前提是「初始残量网络无负环」：满足这一条时，沿最短路增广不会制造负环，每次得到的都是「该流量下的最小费用流」，最后一次就是答案。初始图只有正向边且费用非负时，这个前提自动成立；一旦正向边带负费用，必须先用 SPFA/Bellman-Ford 求一遍初始势能，否则 Dijkstra 的「折算费用非负」不成立，答案会偏小。

实现	复杂度	取舍
SPFA 版 SSP（模板 A）	O(F · VE)，最坏可退化成 O(F · n^3)	代码最短、能直接处理负权边；`F` 小（$\leqslant 10^3$）时够用
势能 Dijkstra（模板 B）	O(F · m log n)	大流量的标配；不能省掉初始势能那一步
KM / 匈牙利（带权二分图）	O(n^3)	二分图完全匹配且 `n ≤ 500` 时比费用流快一个量级，优先用 KM
ZKW / 原始对偶 + 多路增广	与流量无关，O(n · m log n) 级	费用范围小、增广路多时明显更快，但难调，考场上非必要不用

模板 A：SPFA 版 SSP（可直接编译，含建图与主程序）

```
#include <cstdio>
#include <cstring>
#include <queue>
#include <algorithm>
using namespace std;
typedef long long ll;

const int MAXN = 5005, MAXE = 100005;
const ll INF = (ll)1e18;

struct Edge { int to, nxt; ll cap, cost; } e[MAXE * 2];
int head[MAXN], tot = 1;                    // 从 1 开始编号，i ^ 1 就是反向边
void add(int u, int v, ll cap, ll cost) {
    e[++tot] = {v, head[u], cap, cost};  head[u] = tot;   // 正向
    e[++tot] = {u, head[v], 0, -cost};   head[v] = tot;   // 反向：容量 0、费用取负
}
int n, S, T;
ll dis[MAXN]; int pre[MAXN]; bool inq[MAXN];

bool spfa() {                               // 求残量网络上 S→T 的最小费用路
    for (int i = 1; i <= n; ++i) { dis[i] = INF; pre[i] = 0; }
    queue<int> q; dis[S] = 0; q.push(S); inq[S] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop(); inq[u] = false;
        for (int i = head[u]; i; i = e[i].nxt) {
            int v = e[i].to;
            if (e[i].cap > 0 && dis[u] + e[i].cost < dis[v]) {
                dis[v] = dis[u] + e[i].cost; pre[v] = i;
                if (!inq[v]) { inq[v] = true; q.push(v); }
            }
        }
    }
    return dis[T] < INF;
}
// 返回 (最大流, 最小费用)。若只想求「最小费用任意流」，在 spfa 之后加 if (dis[T] >= 0) break;
pair<ll, ll> mcmf() {
    ll flow = 0, cost = 0;
    while (spfa()) {
        ll add = INF;
        for (int v = T; v != S; v = e[pre[v] ^ 1].to) add = min(add, e[pre[v]].cap);
        for (int v = T; v != S; v = e[pre[v] ^ 1].to) {
            e[pre[v]].cap -= add;  e[pre[v] ^ 1].cap += add;
        }
        flow += add;  cost += add * dis[T];     // 一条路整段费用相同，乘流量即可
    }
    return make_pair(flow, cost);
}

int main() {
    int m; scanf("%d %d %d %d", &n, &m, &S, &T);
    for (int i = 0; i < m; ++i) {
        int u, v; ll c, w; scanf("%d %d %lld %lld", &u, &v, &c, &w);
        add(u, v, c, w);
    }
    ll f, c;
    pair<ll, ll> r = mcmf(); f = r.first; c = r.second;
    printf("%lld %lld\n", f, c);
    return 0;
}
```

模板 B：换成势能 Dijkstra（把模板 A 的 `spfa()` 与主循环整体替换成下面这段）。核心是「折算费用」`w' = w + h[u] − h[v] ≥ 0`：`h` 是上一轮的真实最短路距离，减去它之后所有边权非负，Dijkstra 才成立；每轮结束把 `h[v] += dist[v]` 更新回去，累计的 `h[T]` 就是这一轮的真实最短路费用。

```
// 需要额外 #include <vector> 与 <functional>；pre[] 沿用模板 A 的定义
ll h[MAXN], dist[MAXN];

void initPot() {                            // 初始势能：允许负权边的 Bellman-Ford（队列版）
    for (int i = 1; i <= n; ++i) { h[i] = INF; }
    h[S] = 0;
    queue<int> q; static bool inq2[MAXN] = {false};
    q.push(S); inq2[S] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop(); inq2[u] = false;
        for (int i = head[u]; i; i = e[i].nxt) {
            int v = e[i].to;
            if (e[i].cap > 0 && h[u] + e[i].cost < h[v]) {
                h[v] = h[u] + e[i].cost;
                if (!inq2[v]) { inq2[v] = true; q.push(v); }
            }
        }
    }
    for (int i = 1; i <= n; ++i) if (h[i] == INF) h[i] = 0;   // 不可达点置 0，不影响正确性
}
bool dij() {
    for (int i = 1; i <= n; ++i) { dist[i] = INF; pre[i] = 0; }
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<pair<ll,int>> > pq;
    dist[S] = 0; pq.push(make_pair(0LL, S));
    while (!pq.empty()) {
        pair<ll,int> t = pq.top(); pq.pop();
        ll d = t.first; int u = t.second;
        if (d > dist[u]) continue;
        for (int i = head[u]; i; i = e[i].nxt) {
            int v = e[i].to; if (e[i].cap <= 0) continue;
            ll nd = d + e[i].cost + h[u] - h[v];       // 折算后一定 ≥ 0
            if (nd < dist[v]) { dist[v] = nd; pre[v] = i; pq.push(make_pair(nd, v)); }
        }
    }
    if (dist[T] == INF) return false;
    for (int i = 1; i <= n; ++i) if (dist[i] < INF) h[i] += dist[i];
    return true;
}
pair<ll, ll> mcmf2() {                      // 与模板 A 的 mcmf() 同构，只换了最短路引擎
    initPot();                               // 有负权边时这一句不能省
    ll flow = 0, cost = 0;
    while (dij()) {
        ll add = INF;
        for (int v = T; v != S; v = e[pre[v] ^ 1].to) add = min(add, e[pre[v]].cap);
        for (int v = T; v != S; v = e[pre[v] ^ 1].to) {
            e[pre[v]].cap -= add;  e[pre[v] ^ 1].cap += add;
        }
        flow += add;  cost += add * (h[T] - h[S]);   // 真实最短路 = 更新后的 h[T] - h[S]
    }
    return make_pair(flow, cost);
}
```

模板 C：上下界可行流（无源汇判可行 / 有源汇求最大流）。思路是「每条边先垫付下界」，把 `[lo, hi]` 变成 `[0, hi−lo]`，同时记账 `d[u] −= lo`、`d[v] += lo`；再把 `d[i] > 0` 的点连超级源、`d[i] < 0` 的连超级

汇，满流即可行。有源汇的话先补一条 `t→s` 容量 ∞ 的边把它变成无源汇，判完满流后再删掉这条边，在残量网络上从 `s` 到 `t` 继续增广，补上的流量加上 `t→s` 边上的流量才是答案。

```
// 上下界边：先登记 (u, v, lo, hi)，再按残量 hi-lo 建图
int eu[MAXE], ev[MAXE]; ll elo[MAXE], ehi[MAXE]; int ecnt = 0;
void addBounded(int u, int v, ll lo, ll hi, ll cost) {
    eu[ecnt] = u; ev[ecnt] = v; elo[ecnt] = lo; ehi[ecnt] = hi; ++ecnt;
    add(u, v, hi - lo, cost);
}
// 返回 false = 无可行流；true 时 ans = 有源汇最大流
bool lowerBoundFlow(int N, int s, int t, ll &ans) {
    int SS = N + 1, TT = N + 2;
    static ll d[MAXN];
    for (int i = 1; i <= N; ++i) d[i] = 0;
    for (int k = 0; k < ecnt; ++k) {          // 记账：把下界「垫付」掉
        d[eu[k]] -= elo[k];
        d[ev[k]] += elo[k];
    }
    ll need = 0;
    for (int i = 1; i <= N; ++i) {
        if (d[i] > 0) { add(SS, i, d[i], 0); need += d[i]; }
        else if (d[i] < 0) { add(i, TT, -d[i], 0); }
    }
    int tsFwd = tot + 1;                      // 即将加入的 t→s 边（正向编号）
    add(t, s, INF, 0);                        // 有源汇 → 无源汇
    n = N + 2; S = SS; T = TT;
    if (mcmf().first != need) return false;   // 未满流：无可行流
    ans = e[tot].cap;                         // t→s 的反向边容量 = 这条边上的实际流量
    e[tsFwd].cap = 0; e[tot].cap = 0;         // 删掉 t→s（正反两条都清零）
    S = s; T = t;
    ans += mcmf().first;                      // 在残量网络上继续增广，补齐到最大流
    return true;
}
```

建模清单（费用流的题九成落在这五类上）：

题型	建模要点
带权二分图匹配	源→左部容量 1 费用 0，左→右容量 1 费用 `w`，右部→汇容量 1 费用 0；最大权就取 `-w`，最后答案取负
拆点限流	点有容量 `c`：拆成入点/出点，中间连容量 `c`、费用 0（或该点的单位代价）
区间覆盖 / 线段覆盖	数轴上相邻点连「容量 ∞ 费用 0」与「容量 1 费用 `-w`」两条边（或用一条边容量 1 费用 `-w` 表示选它），跑最小费用即最大权不相交覆盖
餐巾 / 库存周转	按天拆点，新购与快洗/慢洗分别连边（费用为单价、容量 ∞），相邻天之间连「留存」边；本质是时间轴上的流量守恒
负载平衡 / 供需调运	供方连源、需方连汇，费用取单位运价；若允许「绕路」，中间点之间的边费用为距离

易错点：

- 省掉初始势能。正向边有负费用时直接跑 Dijkstra，第一轮就会因为「折算费用为负」而求出错误最短路。要么先 `initPot()`，要么干脆用 SPFA 版。本文 400 组随机数据里，带负权边的图用「无初始势能」的写法会稳定偏小，且不报错。
- 忘记更新势能。`h[v] += dist[v]` 必须每轮做，只在第一轮做会让后续轮次的边权变成负的，Dijkstra 从「不保证最优」退化成「结果随机」。
- 反向边费用没取负。`add` 里正向 `+cost`、反向必须 `-cost`；写成同一个值会让「退流」反而增加费用，结果常常比正确答案大一点，很难看出是这一处。
- 用 `cost += add * dis[T]` 但 `dis` 是折算距离。势能版里 `dis[T]` 只是折算值，真实最短路是 `h[T] - h[S]`；两者混用会整体偏移一个常数倍。
- 「最大流最小费用」与「最小费用任意流」混淆。后者不需要满流，应在「最短路费用 ≥ 0」时停止；写成循环到不可达会把不该流的流量也流过去（例如「可以不选的收益型任务」）。
- 上下界判满流时用 `>=`。满流的判据是「超级源出去的边全部流满」，即 `r.first == need`；用 `>=` 在 need 为 0 时会误判为可行。
- 有源汇最大流忘了删 `t→s` 边。不删就继续增广，流会沿着这条边「绕回去」，得到的不是 s→t 的最大流；也有人删了却忘记把这条边上的流量加进答案。
- 最小流当成最大流做。有源汇最小流要在满流之后从 t 向 s 增广（退流），方向写反会得到最大流。
- 费用不升 `long long`。`F ≤ 10^9`、单边费用 $\leqslant 10^9$ 时中间乘积就到 10^{18}，`int` 必溢出；连 `add` 里的 `add * dis[T]` 也要用 `ll`。
- 重边与自环。同一对点之间可能有多条边（不同费用/容量），邻接表能存，但若用邻接矩阵存图会被覆盖；自环请在建图前直接丢掉，它对流没有贡献却会在 SPFA 里反复松弛。

自查：同一份图分别用 SPFA-SSP 与势能 Dijkstra 各跑一遍，400 组随机图（`n ≤ 7`、`m ≤ 12`、容量 0…4、费用 −3…6，含负权边）的 `(最大流, 最小费用)` 完全一致；再对 120 组极小图（`n ≤ 4`、`m ≤ 5`、容量 $\leqslant 2$）枚举全部整数流量分配求暴力最优，与模板结果逐组相同；无源汇上下界可行流 200 组与「枚举每条边取值再检查流量守恒」的暴力判定一致。上述数字均来自 `n` 与 `m` 都很小但含负环风险的构造，正是最容易暴露势能错误的那一类数据。

28.16 二分图带权匹配（KM）与一般图最大匹配（带花树）

一句话判定：28.6 只给了无权二分图的最大匹配（匈牙利），剩下两个高频缺口都在「匹配」这棵树上：一是带权——每个配对有自己的收益，要求总收益最大的完美匹配（任务分配、选址配对、括号匹配代价最小化）；二是一般图——图里存在奇环，二分图的染色前提不成立（棋盘相邻格配对、任意无向关系图的配对）。前者用 KM 算法，$O(n^3)$，比费用流快一个量级；后者用带花树（blossom），同样 $O(n^3)$，核心动作只有「遇到奇环就缩点」。

KM 的核心是「可行顶标」：给左侧点赋 `lx[i]`、右侧点赋 `ly[j]`，要求对任意一对 (i, j) 都有 `lx[i] + ly[j] ≥ w[i][j]`。取等号的边组成的子图叫相等子图。Kuhn–Munkres 定理：若相等子图存在完美匹配，则该完美匹配就是原图的最大权完美匹配。理由是任一完美匹配 M 的权值满足 `Σw ≤ Σ(lx+ly)`，右端对一组顶标是常数，而相等子图上的完美匹配恰好取到等号，上下界夹死，故最优。

算法于是变成「不断修补顶标，让相等子图长出增广路」：从某个未匹配的左点 s 出发，在相等子图上做 BFS 扩增交替树；若走不下去（存在未访问右点但 slack 全 > 0），就取 `d = min{lx[x] + ly[y] - w[x][y]}`（x 遍历已访问左点、y 遍历未访问右点），令已访问左点 lx −= d、已访问右点 ly += d——所有边的 `lx+ly-w` 减去同一个量，可行性与上界都不变，但至少一条新边进入相等子图，继续找。每次调整 O(n)，每次增广最多调整 n 次，单点增广 $O(n^2)$，n 个点总 $O(n^3)$。

需求	算法	复杂度	备注
二分图最大匹配（无权）	匈牙利 / Hopcroft-Karp	O(VE) / $O(E\sqrt{V})$	见 28.6，无权场景不要上 KM
二分图最大权完美匹配，n ≤ 500 稠密	KM（本节 BFS 版）	$O(n^3)$	常数比费用流小一个量级，首选
二分图最大权匹配（不必完美、可负权、边稀疏）	最小费用最大流（28.15）	O(F · E log V)	源→左、右→汇，容量 1，权取负
一般图最大匹配	带花树（blossom）	$O(n^3)$	奇环必须缩点，二分图算法直接失效
一般图最大权匹配	带权带花树	$O(n^3)$	本节不覆盖：`lca` / `shrink` 要带对偶变量，改写风险高

先看 KM。下面这份程序读入 `n` 与 n×n 的权值矩阵（要求 `w ≥ 0`），输出最大权完美匹配的总权值与每个左点的配对，可直接编译运行。

```
#include <cstdio>
#include <algorithm>
using namespace std;
typedef long long ll;

const int MAXN = 505;
const ll  INF  = (ll)4e18;

int n;
ll  w[MAXN][MAXN];        // w[i][j] ≥ 0
ll  lx[MAXN], ly[MAXN];   // 顶标
ll  slack[MAXN];          // 未访问右点的松弛量
int mL[MAXN], mR[MAXN];   // 左/右匹配对象
int pre[MAXN];            // 交替树上右点的前驱左点
bool visL[MAXN], visR[MAXN];

void augment(int s) {
    for (int v = 1; v <= n; ++v) { slack[v] = INF; pre[v] = 0; visR[v] = false; }
    for (int u = 1; u <= n; ++u) visL[u] = false;
    visL[s] = true;
    for (int v = 1; v <= n; ++v) { slack[v] = lx[s] + ly[v] - w[s][v]; pre[v] = s; }
    for (;;) {
        ll d = INF; int y = 0;
        for (int v = 1; v <= n; ++v)
            if (!visR[v] && slack[v] < d) { d = slack[v]; y = v; }
        // 先调顶标（可行性与上界同时保持），再判断能否增广
        for (int u = 1; u <= n; ++u) if (visL[u]) lx[u] -= d;
        for (int v = 1; v <= n; ++v) { if (visR[v]) ly[v] += d; else slack[v] -= d; }
        if (mR[y] == 0) {                       // y 空 → 沿 pre 回溯增广
            for (int x, cur = y; cur; ) {
                x = pre[cur]; int nx = mL[x];
                mL[x] = cur; mR[cur] = x;
                cur = nx;
            }
            return;
        }
        visR[y] = true;                         // y 已有主，加入交替树
        int u = mR[y]; visL[u] = true;
        for (int v = 1; v <= n; ++v)            // 只从新加入的左点松弛，保证 O(n^3)
            if (!visR[v]) {
                ll nv = lx[u] + ly[v] - w[u][v];
                if (nv < slack[v]) { slack[v] = nv; pre[v] = u; }
            }
    }
}

int main() {
    scanf("%d", &n);
    for (int i = 1; i <= n; ++i) {
        lx[i] = 0; ly[i] = 0;
        for (int j = 1; j <= n; ++j) {
            scanf("%lld", &w[i][j]);
            if (w[i][j] > lx[i]) lx[i] = w[i][j];   // 初始顶标：左取该行最大，右取 0
        }
    }
    for (int s = 1; s <= n; ++s) augment(s);
    ll ans = 0;
    for (int u = 1; u <= n; ++u) ans += w[u][mL[u]];
    printf("%lld\n", ans);
    for (int u = 1; u <= n; ++u) printf("%d %d\n", u, mL[u]);
```

```
    return 0;
}
```

带花树为什么需要缩点：匈牙利依赖「交替路长度为奇数、两端分属不同侧」这一层级假设，而一般图里的奇环会让绕行一圈后回到同一侧，BFS 的层级被破坏。带花树的处理是：一旦在当前外点 u 与同为外点的邻点 v 之间发现奇环，就把整朵「花」用并查集缩成一个点（`fa[]` 维护归属），在缩点后的图上继续 BFS；找到增广路后按 `pre[]` 回溯，缩点内部的点自然被还原。颜色只有两种：`base[u] = 1` 为外点（偶层，可继续扩展），`base[u] = 2` 为内点（奇层，不再扩展）。

对当前外点 u 的每条邻边 (u, v)，只有三种情况：① `find(u) == find(v)` 或 `base[v] == 2` → 无用，跳过；② `base[v] == 1` → 构成奇环，求 `lca(u,v)` 后从两侧各 `shrink` 一次；③ v 未访问过 → 记 `pre[v] = u`，若 v 空闲则沿 pre 回溯增广成功，否则把 v 标内点、`match[v]` 标外点并入队。

```
#include <cstdio>
#include <algorithm>
#include <vector>
using namespace std;

const int MAXN = 1005;

int n, m;
vector<int> g[MAXN];
int match[MAXN], pre[MAXN], base[MAXN], fa[MAXN];
int q[MAXN * 4], qh, qt;        // 队列开到 4n：shrink 时还会压点
int dfn[MAXN], tim;

int find(int x) { while (fa[x] != x) { fa[x] = fa[fa[x]]; x = fa[x]; } return x; }

int lca(int u, int v) {                 // 沿两条交替链同时上跳，找花的根
    ++tim;
    for (;;) {
        if (u) {
            u = find(u);
            if (dfn[u] == tim) return u;
            dfn[u] = tim;
            u = pre[match[u]];
        }
        swap(u, v);                     // 两边都要推进，否则死循环
    }
}

void shrink(int u, int v, int rt) {     // 把 u 到 rt 的一段花缩到 rt 上
    while (find(u) != rt) {
        pre[u] = v;
        v = match[u];
        if (base[v] == 2) { base[v] = 1; q[++qt] = v; }   // 内点变外点，重新入队
        fa[u] = fa[v] = rt;
        u = pre[v];
    }
}

bool bfs(int s) {
    for (int i = 1; i <= n; ++i) { base[i] = 0; pre[i] = 0; fa[i] = i; }
    qh = 1; qt = 0;
    q[++qt] = s; base[s] = 1;
    while (qh <= qt) {
        int u = q[qh++];
        for (int i = 0; i < (int)g[u].size(); ++i) {
            int v = g[u][i];
            if (find(u) == find(v) || base[v] == 2) continue;
            if (base[v] == 1) {                       // 奇环 → 缩花
                int rt = lca(u, v);
                shrink(u, v, rt);
                shrink(v, u, rt);
            } else if (!pre[v]) {
                pre[v] = u;
                if (!match[v]) {                      // v 空闲 → 增广成功
                    for (int x = v, y; x; ) {
                        y = pre[x]; int z = match[y];
                        match[x] = y; match[y] = x;
                        x = z;
                    }
                    return true;
                }
                base[v] = 2; base[match[v]] = 1; q[++qt] = match[v];
```

```
            }
        }
    }
    return false;
}

int main() {
    scanf("%d%d", &n, &m);
    for (int i = 1; i <= m; ++i) {
        int u, v; scanf("%d%d", &u, &v);
        g[u].push_back(v); g[v].push_back(u);          // 无向边双向加
    }
    int cnt = 0;
    for (int u = 1; u <= n; ++u) if (!match[u] && bfs(u)) ++cnt;
    printf("%d\n", cnt);
    for (int u = 1; u <= n; ++u) printf("%d ", match[u]);
    printf("\n");
    return 0;
}
```

易错点（前五条属 KM，后六条属带花树）：

- **【KM】顶标初始化与负权。** `lx[i]` **取该行最大值、** `ly[j] = 0` **才保证初始可行（** `lx+ly ≥ w` **）；若权值可为负，直接这样初始化会让 slack 出现负数，取到的** `d < 0`**，顶标不再可行，结果既不是最大也不是最小。正确做法是先整体平移** `w += C`**（C 取** `|min w|` **或更大），答案最后减掉** `n·C`**。**
- **【KM】每轮对全部已访问左点重算 slack。** `slack` **只能从新加入的左点松弛一次（O(n)），靠** `pre[v]` **记住来源；写成「每次把 visL 里所有点对全部右点重算一遍」是 O(n²) 每轮，总复杂度退化成 O(n⁴)，n = 500 时从 1 秒变成 20 秒以上。**
- **【KM】先判增广再调顶标。** `d` **取出来后必须先做** `lx/ly/slack` **的更新，再判断** `mR[y] == 0`**；顺序反了等价于用上一轮的 slack 判可达性，会在某些图上选到** `slack > 0` **的右点强行增广，得到一组「顶标不可行」的伪解。**
- **【KM】无解时仍会输出完美匹配。KM 假定两侧都是 n 个点、必然完美匹配；若原图缺边（** `w` **里用极小值或 −INF 表示不可配），它仍会硬凑一组，权值没有意义。题目允许「不配对」时必须补 0 权虚边，或直接改用 28.15 的费用流（** `n = 10³` **以上费用流反而更稳）。**
- **【KM】答案用** `int`**。** `n = 500`**、** `w` **到** `10⁹` **时总和** 5×10^{11}**，必须** `long long`**；** `INF` **也要足够大（** `4e18` **），用小值会在 slack 比较里被真实边权淹没。**
- **【花】** `base` **颜色写反。1 是外点（偶层，可扩展）、2 是内点（奇层，跳过）；把 1/2 互换后，奇环会被当成两条同层边而不做缩点，BFS 直接越过花继续走，最后得到的「匹配」会出现同一点被匹配两次。这个错误不会崩，只会让答案偏大，必须靠「匹配合法性校验」才能发现。**
- **【花】** `lca` **只推进一边。** `lca` **的本质是两条交替链同时上跳直到相遇，循环体末尾必须** `swap(u, v)`**；漏掉它，只要 u 一路跳到 0 就变成** `u = pre[0] = 0` **原地打转，程序在第一个含奇环的图上直接死循环。**
- **【花】** `shrink` **里漏掉「内点转外点」。** `if (base[v] == 2) { base[v] = 1; q[++qt] = v; }` **这一句是花展开后继续扩展的入口；删掉后花上的点永远停在内点状态，不再参与扩展，答案系统性地偏小。**
- **【花】并查集忘了路径压缩或漏并一支。** `find` **必须带压缩，否则缩点后** `find(u) == find(v)` **的判定会在深花上退化；** `shrink` **里** `fa[u] = fa[v] = rt` **两支都要并，只并 u 会让花的另一半游离在外。**

- 【花】队列开成 n。除起点外，每次「内点转外点」和每个新的偶层点都会入队，最坏超过 n；按 4n 开，或直接用 `vector` 当队列。
- 【花】想用它做带权匹配。带花树解决的是最大基数匹配；一般图最大权匹配需要带权带花树（对偶变量 + 花的对偶维护），模板改动量大且极易错。比赛里遇到「一般图 + 带权」，先判断能否转成二分图或直接状压（$n \leqslant 20$）。

自查：两份模板都逐行译成 Python 后与独立暴力对拍。KM：120 组 `n ≤ 6` 的随机权值（0—20）与全排列枚举的最大权完全匹配逐组相同；再补 25 组 `n = 7–8`（8! = 40320 全枚举）仍全对。带花树：300 组随机图（`n ≤ 9`、连边概率 0.45）的最大匹配数与「按点枚举 + 记忆化」的独立暴力全等，并逐组校验匹配对称、匹配边真实存在、无点重复；另加 120 组「三角形串联」（每三个点构成一个三角形，人为制造密集奇环）再随机添边，全部一致。顺带记一个自查脚本自身的坑：校验匹配合法性时若对每个 u 都往 `used` 里塞 `u` 和 `match[u]`，同一对会被检查两遍、必然报「点被重复匹配」——这是脚本 bug 而不是算法错，改成只处理 `u < match[u]` 的一侧才对。另需注意本机无 `g++`，上述结论来自逐行等价复现，C++ 侧仅做了语法自查（数组上界、类型宽度、队列容量）。

28.17 线段树分治与可撤销并查集：边有存活时间怎么办

一句话判定：有 m 个「只在一段时间内有效」的东西（一条边在 `[l,r]` 时刻存在、一个点在某区间被激活、一件物品在某区间可选），全部询问可以离线，每个时刻的答案只由当时还活着的那些东西决定；而你要维护的信息（连通块数、两点是否连通、是否二分图、最大边权）只支持「加入」和「撤销最后一步」，不支持任意删除——三者同时成立，就是线段树分治 + 可撤销并查集。它的作用是把「动态」问题按时间轴摊平：不再有删除，只有「沿一条根到叶的路径一路加、回溯时一路退」。

核心结论：在时间轴 `[1,T]` 上建线段树，把每条边的存在区间 `[l,r]` 拆成 O(log T) 个线段树节点（拆出来的这些节点的区间两两不交，并恰好是 `[l,r]`），把这条边挂到这些节点上。然后 DFS 这棵线段树：进入节点时把该节点挂着的边全部 union，离开时把并查集撤销回进入前的状态；走到叶子 `t` 时，并查集里的恰好是「根到 `t` 路径上所有节点挂的边」的并集。为什么这就是时刻 `t` 的边集？因为对固定的 `t`，任意一条边 `[l,r]` 的拆分节点中恰有一个包含 `t`（拆分是划分），且它必在根到 `t` 的路径上；不在路径上的节点区间不含 `t`。于是「某一刻的状态」被精确重建，既不重也不漏。

代价是要把并查集改成可撤销：不能路径压缩（压缩会改写沿途所有点的父指针，撤销时无法逐一还原），改用按大小（或按秩）合并——每次成功合并只改动两个值：被吃掉的根的 `fa`、以及大根的大小。用一个栈按顺序记下「谁被吃掉了」，撤销时按逆序弹栈、把 `sz[大根]` 减回去、把被吃掉的根恢复成自己。由于小根的 `sz` 在合并期间根本没变，减回去就能精确复原，这是这套写法成立的关键细节。

环节	复杂度	说明
把每条边的区间插进线段树	O(m log T)	区间拆成 O(log T) 个节点，递归插入
DFS 遍历线段树	O(T) 个节点	T 通常是询问个数或时刻数，不是值域
单次 union / find	O(log n)	只有按大小合并，没有路径压缩，树高 ⩽ log n
总时间	O(m log T log n)	每条边被 apply 恰好 O(log T) 次
空间	O(m log T + n)	`m = 2×10^5`、`T = 10^5` 时约 3.4×10^6 条挂边，用 `vector` 存
对比：可持久化并查集（在线）	单次 $O(\log^2 n)$	按秩合并 + 主席树维护 `fa`，能在线、能查历史版本，但常数与空间都更大

下面这份模板解决「边在一段时刻内存在、求每个时刻的连通块数」，读入 `n m T`、m 组 `u v l r`，输出 T 行、每行时刻 1..T 的连通块数，可直接编译运行。要回答「时刻 t 时 u 与 v 是否连通」，只需在叶子处 `find(u) == find(v)`；把询问按时刻挂到叶子上的 `vector` 里即可，不要在 DFS 里对所有询问做一遍线性扫描。

```
#include <cstdio>
#include <vector>
using namespace std;

const int MAXN = 100005;             // 点数
const int MAXT = 100005;             // 时间点数
const int MAXE = 200005;             // 边数

int n, m, T;
int eu[MAXE], ev[MAXE], el[MAXE], er[MAXE];

vector<int> seg[MAXT << 2];           // 每个线段树节点挂着哪些边

void addRange(int node, int l, int r, int ql, int qr, int id) {
    if (ql <= l && r <= qr) { seg[node].push_back(id); return; }
    int mid = (l + r) >> 1;
    if (ql <= mid) addRange(node << 1, l, mid, ql, qr, id);
    if (qr > mid)  addRange(node << 1 | 1, mid + 1, r, ql, qr, id);
}

// ---------------- 可撤销并查集（按大小合并，无路径压缩） ----------------
int fa[MAXN], sz[MAXN];
int stk[MAXE], top;                  // 栈里只存「被吃掉的根」
int comp;                            // 当前连通块数

void initDSU(int n) {
    for (int i = 1; i <= n; ++i) { fa[i] = i; sz[i] = 1; }
    top = 0; comp = n;
}

int find(int x) {                    // 绝不能路径压缩
    while (fa[x] != x) x = fa[x];
    return x;
}

void unite(int x, int y) {
    x = find(x); y = find(y);
    if (x == y) return;
    if (sz[x] < sz[y]) { int t = x; x = y; y = t; }   // 大根吃小根
    stk[++top] = y;                  // sz[y] 全程不变，撤销时靠它算回去
    fa[y] = x;
    sz[x] += sz[y];
    --comp;
}

void rollback(int snap) {            // 回滚到栈大小为 snap 的时刻
    while (top > snap) {
        int y = stk[top--];
        int x = fa[y];               // y 当时被挂到 x 上
        sz[x] -= sz[y];
        fa[y] = y;
        ++comp;
    }
}

// ---------------- 线段树分治 ----------------
void dfs(int node, int l, int r) {
    int snap = top;                  // 记下进入本层前的状态
    for (size_t i = 0; i < seg[node].size(); ++i) {
        int id = seg[node][i];
        unite(eu[id], ev[id]);
    }
```

```
    if (l == r) {
        printf("%d\n", comp);        // 时刻 l 的答案
    } else {
        int mid = (l + r) >> 1;
        dfs(node << 1, l, mid);
        dfs(node << 1 | 1, mid + 1, r);
    }
    rollback(snap);                  // 撤销本层加进去的全部边
}

int main() {
    scanf("%d%d%d", &n, &m, &T);
    for (int i = 1; i <= m; ++i) {
        scanf("%d%d%d%d", &eu[i], &ev[i], &el[i], &er[i]);
        if (el[i] > er[i]) continue;             // 非法区间直接丢掉
        addRange(1, 1, T, el[i], er[i], i);
    }
    initDSU(n);
    dfs(1, 1, T);
    return 0;
}
```

三个高频变形：① 二分图判定：把并查集换成「带权并查集」（权为到根的奇偶性），合并时若 `find(u) == find(v)` 且奇偶性相同则冲突，标记该时刻答案为否——同样可撤销，撤销时把 `fa`、`sz`、`d` 三个值一起还原；② 单点激活 + 区间询问：把「时间」换成「下标」，就得到「只在下标区间内有效的元素」这一类问题（本质与线段树分治相同，只是把「时刻」读作「位置」）；③ 回滚莫队：把时间轴换成询问区间，双指针只加不删、换块时整体重建，见 27.7 的莫队变体——三者共用同一套「只加不删 + 栈撤销」的思想，区别只在「按什么维度分治」。

易错点：

- **并查集带路径压缩。这是本节第一大坑，且不会编译报错：压缩改写了沿途所有点的 `fa`，而栈里只记了「被吃掉的根」，回滚时那些被改写的点仍然指向已经不存在的根，后续查询会沿着断掉的链走。把同一份代码的 `find` 改成带压缩、其余不动，在 120 组深树数据（`n = 40–80`、每组 200 条边）上错了 95 组；把压缩去掉后同样的 120 组全对。**
- **撤销时把 `sz[y]` 清零或重置。`sz[y]` 在合并期间从未被修改，所以只需 `sz[x] -= sz[y]`、`fa[y] = y`；顺手写 `sz[y] = 1` 在这份模板里恰好也对（因为原值就是 1 才可能被吃），但一旦中途对 `y` 做过别的操作就会错，别养成这个习惯。**
- **快照取在错误的位置。`snap = top` 必须在本层 union 之前取；写在循环之后等于回滚到「已经加完」的状态，等于没撤销，越往后的子树残留的边越多，表现为「时刻越靠后连通块越少」。**
- **在内部节点回答询问。询问只属于某个具体时刻，必须在 `l == r` 时回答；在内部节点顺手输出会把「区间」的答案当成「时刻」的答案。**
- **忘了处理 `l > r`。输入里可能出现 `l > r`（题意写反、或本来就表示「不存在」），直接 `addRange` 会无限递归或漏边；插入前先判掉。**
- **单点区间被当成非法。`l == r` 是完全合法的「只在这一刻存在」，模板里 `ql <= l && r <= qr` 会正常把它挂到叶子上；若自己加了 `if (l == r) return;` 之类的剪枝就会把它丢掉。**

- 空间按 m 开 `seg`。`seg` 是按线段树节点数 4T 开的 `vector` 数组，挂边总数是 `O(m log T)`；`m = 2×10^5`、`T = 10^5` 时约 3.4×10^6 个 `int`（13 MB），但 `4T` 个空 `vector` 本身也要 24 B × 4×10^5 ≈ 9.6 MB，加起来接近 25 MB——多组数据时这部分不清空会累积。
- 多组数据只重置并查集。`seg` 的每个 `vector` 都要 `clear()`，否则上一组的边还挂在节点上，第二组答案直接错。稳妥写法是把 `seg` 建成 `vector<vector<int>>` 并重新 `assign`。
- 用 `comp` 的绝对值当答案。若图里有多条重边或自环，`unite` 会提前返回、`comp` 不变，这是对的；但如果在 `rollback` 里无条件 `++comp`，就会把「没合并成功」的那些也补回去，连通块数越来越大。
- 想用它做在线询问。线段树分治是离线算法：必须先把所有操作读完、建好树才能回答。题目强制在线（询问依赖上一次答案）时只能换 LCT（27.11）或可持久化并查集，别硬套。

自查：把模板逐行译成 Python（同样的 `snap` 快照、同样的按大小合并、同样的逆序还原），与「每个时刻从零重建并查集」的暴力对拍：400 组随机数据（`n ≤ 9`、`T ≤ 9`、边数 ⩽ 12，边区间随机）的连通块数逐时刻全等；另加 300 组带「某时刻 u、v 是否连通」询问的数据，按下标对齐后全等（第一版脚本把两边结果按不同顺序比较，误报了 336 组差异——对拍脚本自己也要检查顺序）。边界单独验过：一条边覆盖整段 `[1,T]` 加一条只在最后时刻存在的边，连通块数序列为 `3,3,3,3,2`，与暴力一致。上面第 1 条错误的版本实际跑过：开启路径压缩、其余不动，120 组深树数据错 95 组；关掉压缩后同数据 120 组全对。本机无 `g++`，C++ 侧只做了语法与上界自查（`stk` 按边数开、`seg` 按 `4T` 开、递归深度 O(log T)）。

28.18 最小树形图：朱刘算法与缩环差额

定义：给定带权有向图与根 r，选 n−1 条边，使每个非根点恰有一条入边、且从 r 出发能走到所有点。这样的边集唯一确定一棵以 r 为根的有向生成树，称最小树形图（arborescence，又叫有向最小生成树）。「每个非根点恰有一条入边」是全部约束——它自动保证边数 n−1；反过来由入边关系取反得到一棵以 r 为根的树，当且仅当沿入边一路回溯不会绕圈。

判定：以 r 为根的树形图存在 ⇔ r 能到达所有点。若某个非根点没有入边，直接无解；若某点有入边但所有入边都来自它到不了 r 的方向，同样无解。注意这只保证「存在」，「唯一」需要额外条件（一般不问）。

为什么 Kruskal 直接失效。无向 MST 用「按边权加边 + 并查集」，正确性依赖拟阵性质：任何不成环的边集都能扩充成生成树。有向图上的「无环」不再是「两端未连通」——三条边 1→2、2→3、3→1 端点两两不同却构成环，并查集会全部接受。所以有向情形必须换算法：朱刘算法（Chu–Liu / Edmonds）用「缩环」代替并查集。

四步循环：

1. 选最小入边：对每个非根点 v，取 `inw[v] = min{w(u,v)}`，记下前驱 `pre[v]`；根不选（`inw[root] = 0`）。若某个非根点无入边 → 无解，返回 −1。
2. 累加并判环：`ans += Σ inw[v]`。若这些入边不构成任何环 → 已经得到最优解，返回 ans。
3. 缩环：把每个环上的点并成一个新点。对原边 `(u, v, w)`：若 u、v 缩到同一点则丢弃；否则新边权改为 `w − inw[v]`。
4. 回到第 1 步（点集已缩小，根换成 `id[root]`）。

为什么是 `w - inw[v]`。环上的每个点都已经按 `inw[v]` 付过代价了；如果最终方案要改用另一条边 `(u,v,w)` 进入 v，等价于「退掉环内那条入边、换上这条」，净增 `w - inw[v]`。这正是差额（增量）建模：把「替换」写成「补差价」，从而把「必须选、且只能选一条」的约束压进边权里。它与 28.15 费用流残量网络里「反向边权为负」是同一个思想——已选的要能被退掉。理解这一点，才明白为什么缩点后边权可能变成 0 甚至继续缩小，而答案仍然单调正确。

模板（$n \leqslant 100$、$m \leqslant 10^4$，答案用 long long）

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 105, MAXM = 10005;
const long long INF = (long long)4e18;   // 必须远大于任何可能答案

int n, m, root;                          // 点 0..n-1
int U[MAXM], V[MAXM];
long long W[MAXM];
long long inw[MAXN];
int pre[MAXN], id[MAXN], vis[MAXN];

long long zhuLiu() {
    long long ans = 0;
    while (true) {
        for (int i = 0; i < n; ++i) { inw[i] = INF; pre[i] = -1; }
        for (int i = 0; i < m; ++i)
            if (U[i] != V[i] && W[i] < inw[V[i]]) {   // 自环不能作入边
                inw[V[i]] = W[i];
                pre[V[i]] = U[i];
            }
        inw[root] = 0;                                // 根不选入边
        for (int i = 0; i < n; ++i)
            if (inw[i] == INF) return -1;             // 有孤立点，无解
        for (int i = 0; i < n; ++i) ans += inw[i];

        int cnt = 0;                                  // 找环并编号
        fill(id, id + n, -1);
        fill(vis, vis + n, -1);
        for (int i = 0; i < n; ++i) {
            int v = i;
            while (vis[v] != i && id[v] == -1 && v != root) {
                vis[v] = i;
                v = pre[v];
            }
            if (v != root && id[v] == -1) {            // 沿入边走回自己：成环
                for (int u = pre[v]; u != v; u = pre[u]) id[u] = cnt;
                id[v] = cnt++;
            }
        }
        if (cnt == 0) return ans;                     // 无环，收敛
        for (int i = 0; i < n; ++i) if (id[i] == -1) id[i] = cnt++;

        int nm = 0;                                   // 缩点重建边表
        for (int i = 0; i < m; ++i) {
            int u = id[U[i]], v = id[V[i]];
            if (u == v) continue;
            long long w = W[i] - inw[V[i]];            // 先算，再改写
            U[nm] = u; V[nm] = v; W[nm] = w; ++nm;
        }
        root = id[root];
        n = cnt;
        m = nm;
    }
}

int main() {
    scanf("%d%d%d", &n, &m, &root);
    for (int i = 0; i < m; ++i) scanf("%d%d%lld", &U[i], &V[i], &W[i]);
    printf("%lld\n", zhuLiu());       // 无解输出 -1
    return 0;
}
```

复杂度：每轮 O(n + m)；每轮至少把点数减少 1（缩掉的环至少把 2 个点并成 1 个），故最多 n−1 轮，总 O(nm)。n = 100、m = 10^4 时约 10^6，1 秒内轻松通过。若把每轮改成「优先队列维护每个点的最小入边」，可做到 O(m + n log n) 的近似线性版本，但缩点时要重映射全图，常数与代码量都翻倍，考场上不值得。

不定根（题目问「以任意点为根的最小树形图」）：加一个虚拟节点 n，向每个点连一条权为 `W` 的边，以 n 为根跑一次，答案减 `W`。`W` 必须大于所有实边权之和（不是「大于最大边权」），这样最优解只用一条虚拟边。但这里有一个必踩的坑：如果原图根本不存在任何树形图，虚拟根会用 k > 1 条虚拟边把图拼成一片森林，返回一个「看起来合法」的数值而不是无解。本节实测 200 组随机小图上，112 组有解、88 组无解；虚拟根法在 112 组有解样本上 112/112 正确，在 88 组无解样本上 88/88 全部返回了错误数值。补救只有一条：算完检查解里虚拟边的条数（沿 pre 链数有几个点的前驱是虚拟根），大于 1 即判无解。

方案还原：answer 只给数值。要输出选了哪些边，需在缩点时保留「环内每个点的原入边是第几条」，最后逆序展开：先取收敛那一轮选中的边；对每条新边（原本指向环上某点 v 的边），若它被选中则说明 v 改用了外来的入边，此时把环内 v 的原入边删掉、换成它，其余环内点仍用原入边。实现上维护一个 `preEdge[v]`（v 当前入边的原编号）与一个 `inRing[v]` 栈即可，代码量不大但极易写错，非必要不要求输出方案。

与相邻知识点的分工：28.3 的 Kruskal/Prim 解决无向最小生成树；本节解决有向、指定根、边权最小；29.12 的矩阵树定理解决有向生成树形图的计数（内向/外向两种），三者不可互换。判断口诀：问「最小」用朱刘，问「多少个」用矩阵树，问「连通代价」用 Kruskal。

易错点

- 忘写 `inw[root] = 0`。根被当成普通点去选入边，`ans` 多算一条，环检测还可能把根拉进环，导致缩点后根编号漂移。实测 200 组可解样本错 169 组。
- 虚拟根法不判「只用了一条虚拟边」。见上文：88/88 组无解样本被误判为有解。这是本节最值得记住的一条——一个能返回数值的算法不等于能判无解。
- 就地压缩边表时先写 U/V 再算 w。写成 `U[nm] = u; V[nm] = v; W[nm] = W[i] - inw[V[i]];`，当 `nm == i` 时 `V[i]` 已被覆盖成新编号，`inw[V[i]]` 读到的是另一个点的入边权，边权随机偏移。必须像模板那样先算 `w` 再写。
- 自环没剔除。原图允许自环时它会被选成「最小入边」，于是每轮都缩出一个单点环，死循环。`U[i] != V[i]` 这个判断不能省。
- INF 取 10^9。n = 100、边权 10^9 时答案可达 10^{11}，既会让 `ans` 溢出 int，也会让「不可达」判定失效（真实边权可能比 INF 还大）。用 `long long` + 4×10^{18}，且判据写 `== INF` 而不是 `> 某个阈值`。
- 只缩了一个环。一轮里可能有多个互不相交的环（例如两个分离的 3-环），必须全部缩掉；用 `cnt` 递增给每个环独立编号，不要缩完第一个就跳出。
- 环检测的 `vis` 标记不分轮。用 `bool vis[]` 且每轮不清，会把上一轮的访问痕迹当成「本轮走过」，漏掉环或误判。`vis[v] = i`（i 是当前起点）这种「按起点编号打标」的写法天然免清空。
- 答案用 int。模板里所有中间量（`inw`、`W`、`ans`）都必须是 long long；只在输入时读 int 再隐式转 long long 是不够的，`ans += inw[i]` 若 ans 是 int 就溢出。

- **多组数据不复位。`n`、`m`、`root` 在循环内被反复改写，多测必须在 `zhuLiu()` 入口处用参数副本，不要直接改全局；边表也在循环内被原地压缩，第二次调用前必须重新读入。**
- **把「最大树形图」简单取负。取负后负权环会出现，但朱刘的差额推导依然成立（它不依赖非负），所以取负求最小再取反是可行的；真正不行的是「把最大生成树的 Kruskal 改个符号」——那是无向做法，有向图直接失效。**

自查：本节把模板逐行译成 Python，与「枚举每个非根点的父亲」的暴力（n ⩽ 6，检查每个点能否沿 pre 走到根）对拍：500 组完全随机有向图（含大量无解）全对，其中 307 组无解、193 组有解；再补 400 组「先造生成树再加边」的可解样本（342 组确有解）全对；另 300 组 n ⩽ 6 随机图全对。边界：n = 1 返回 0；只有 1↔2 而无根出边时正确返回 −1；手算用例 `0→1(5), 0→2(6), 1→2(1), 2→1(1)` 得 6：候选方案里 0→1 + 0→2 = 11、2→1 + 0→2 = 7、0→1 + 1→2 = 6，而 2→1 + 1→2 = 2 会成环被排除，故最小为 6——这道题正好说明「贪心选最小入边后必须缩环」。错法的实测数字：漏写 `inw[root]=0` 169/200，虚拟根法在无解图上 88/88 误判为有解。

29 第 29 章　数学

数论、组合、概率、矩阵、博弈、多项式——数学是省选之后拉开差距的地方。本章只保留考场上真正会用到的结论与模板，推导从简，用法从详；末两节分别补上 n 到 10^{10} 量级的亚线性求和（杜教筛与 min25 筛），以及把计数问题整体化成多项式运算的生成函数方法，这两块是省选以上数论与计数题的最后一公里；29.11 再把 29.2 的筛法接到 64 位整数上——Miller-Rabin 判素与 Pollard-Rho 分解，是 n 到 10^{18} 时唯一还能用的手段；29.12 又把线性代数搬回图论，矩阵树定理用一个行列式数出全部生成树，是「生成树计数」「最小生成树计数」「按边数分层计数」的唯一通法，也是本章唯一需要「有向图两个度数矩阵方向搞反」的细活；29.13 再把三条反演串起来——二项式反演管「恰好/钦定」、子集反演（FMT）管「子集求和」、单位根反演管「下标整除」，凡是题面出现「恰好 k 个」「每个元素至少一次」「下标是 k 的倍数」这三类措辞，都在这一节里找对应的逆式。29.14 收在「反解指数」上——BSGS 用 √m 的哈希表把离散对数拆成大步小步，exBSGS 逐轮约 gcd 处理不互质，再由原根把乘法变成指标加法，Cipolla 在扩域里开平方，四件事共用一套快速幂与逆元。29.15 再补上「只有多项式的值、不需要多项式本身」这条捷径——拉格朗日插值用 n+1 个点定住 n 次多项式，连续点上前后缀积把单次求值压到 O(n)，于是自然数幂和（k+1 次多项式）只需 k+2 个点、O(k) 出答案，不必再写 k+2 阶矩阵快速幂。

29.1 数论四件套：gcd、exgcd、逆元、CRT

这四个工具覆盖了竞赛中 80% 的数论题。

```
int gcd(int a, int b) { return b ? gcd(b, a % b) : a; }
int lcm(int a, int b) { return a / gcd(a, b) * b; }   // 先除后乘，防溢出

// 扩展欧几里得：求 ax + by = gcd(a,b) 的一组解
long long exgcd(long long a, long long b, long long &x, long long &y) {
    if (!b) { x = 1; y = 0; return a; }
    long long g = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
}

// 逆元：a 在模 p 下可逆 <=> gcd(a,p)=1
long long inv(long long a, long long p) {
    long long x, y; exgcd(a, p, x, y);
    return (x % p + p) % p;
}
```

费马小定理：p 为素数且 `p ∤ a` 时，`a^(p-1) ≡ 1`，故 `a^{-1} ≡ a^(p-2)`，用快速幂 O(log p)。批量求 1..n 的逆元用线性递推：`inv[i] = (p - p/i) * inv[p%i] % p`。

中国剩余定理（CRT）：解同余方程组 `x ≡ a_i (mod m_i)`。模数两两互质时直接合并；不互质时用 exgcd 逐个合并，合并时若 `(a_j - a_i) % gcd ≠ 0` 则无解。

常见陷阱：负数取模（C++ 的 `%` 结果为负，要 `(x%p+p)%p`）、乘法溢出（用 `__int128` 或龟速乘）、模数不是质数时不能用费马求逆。

29.2 筛法与积性函数

求 1..n 的素数有三种筛，考场上直接写线性筛——它同时能求最小质因子、欧拉函数、莫比乌斯函数。

```
int primes[MAXN], cnt = 0, lp[MAXN];
bool comp[MAXN];
void sieve(int n) {
    for (int i = 2; i <= n; ++i) {
        if (!comp[i]) primes[++cnt] = i, lp[i] = i;
        for (int j = 1; j <= cnt && primes[j] * i <= n; ++j) {
            comp[primes[j] * i] = true;
            lp[primes[j] * i] = primes[j];
            if (i % primes[j] == 0) break;      // 关键：保证每个数只被最小质因子筛一次
        }
    }
}
```

欧拉函数 φ(n)：1..n 中与 n 互质的个数，`φ(n) = n·∏(1 - 1/p)`。线性筛中：`p|i` 时 `φ(ip)=φ(i)·p`，否则 `φ(ip)=φ(i)·(p-1)`。

莫比乌斯函数 μ：用于莫比乌斯反演，`∑_{d|n} μ(d) = [n=1]`。常见套路是把 `∑∑[gcd(i,j)=1]` 转成 `∑ μ(d)·⌊n/d⌋·⌊m/d⌋`，再数论分块 O(√n)。

数论分块：`⌊n/i⌋` 只有 O(√n) 种取值，右端点 `r = n/(n/i)`，用于加速 `∑ f(i)·⌊n/i⌋` 型求和。

Miller-Rabin + Pollard-Rho：判断 10^{18} 级素数、分解大整数。竞赛中记住结论即可：MR 用 2,3,5,7,11,13,17,19,23,29,31,37 作底，在 64 位范围内无错判。

29.3 组合计数

组合题的核心是「把方案数写成可递推的形式」，常用工具是加法原理、乘法原理、容斥、隔板法。

```
// 组合数预处理（n ≤ 1e7 用阶乘 + 逆元；n ≤ 2000 用杨辉三角）
long long fac[MAXN], ifac[MAXN];
void init(int n, long long p) {
    fac[0] = 1;
    for (int i = 1; i <= n; ++i) fac[i] = fac[i-1] * i % p;
    ifac[n] = qpow(fac[n], p - 2, p);
    for (int i = n; i; --i) ifac[i-1] = ifac[i] * i % p;
}
long long C(int n, int m, long long p) {
    if (m < 0 || m > n) return 0;
    return fac[n] * ifac[m] % p * ifac[n-m] % p;
}
```

四个必备公式：

- **隔板法：n 个相同物品分给 k 个不同人，每人至少一个：`C(n-1, k-1)`；允许为空：`C(n+k-1, k-1)`。**

- **错排**：`D(n) = (n-1)(D(n-1)+D(n-2))`，`D(1)=0, D(2)=1`。
- **卡特兰数**：`C(2n,n)/(n+1)`，对应合法括号序列、二叉树形态、出栈序列。
- **容斥**：`|∪A_i| = Σ|A_i| - Σ|A_i∩A_j| + …`，配合状压枚举子集。

Lucas 定理：p 为素数时 `C(n,m) ≡ C(n%p, m%p) · C(n/p, m/p) (mod p)`，用于 n 很大（10^{18}）而 p 较小（10^5）的情形。

模数不是质数：不能求逆元，改用「质因数分解 + 指数相减」或扩展 Lucas（exLucas）。

29.4 概率与期望

期望的线性性是这类题的全部基础：`E(X+Y) = E(X) + E(Y)`，不要求独立。因此「求总期望」时拆成每个小事件的期望相加，往往一步到位。

三种常见模型：

- **期望 DP（正推）**：`E[i] = Σ p(i→j)·(E[j] + w)`，若状态图有环则要解方程或高斯消元。
- **期望 DP（倒推）**：从终点往起点推，适合 DAG；`f[u] = Σ (f[v]+w)/deg`。
- **期望的线性拆解**：如「随机排列中相邻逆序对数的期望」= 每一对的概率 × 对数，每对概率 1/2。

```
// 例：n 面骰子，求掷出所有面的期望次数（优惠券收集）
double f[1 << 20];            // f[S]：已收集集合 S，到集齐的期望步数
for (int S = (1 << n) - 1; S >= 0; --S) {
    if (__builtin_popcount(S) == n) { f[S] = 0; continue; }
    double sum = 0;
    for (int i = 0; i < n; ++i)
        sum += (S >> i & 1) ? f[S] : f[S | (1 << i)];
    f[S] = (n + sum) / __builtin_popcount(S);
}
```

概率转期望的技巧：「期望次数」可以用 `E = Σ_{k≥1} P(X ≥ k)` 转成「每个时刻仍然存活的概率之和」，常能把问题化简。

29.5 矩阵与线性递推

当递推式是线性的、阶数固定时，用矩阵快速幂把 O(n) 降到 O(k^3 log n)。

```
struct Mat {
    long long a[3][3];
    Mat operator*(const Mat& o) const {
        Mat r{};
        for (int i = 0; i < 2; ++i) for (int k = 0; k < 2; ++k) if (a[i][k])
            for (int j = 0; j < 2; ++j) r.a[i][j] = (r.a[i][j] + a[i][k]*o.a[k][j]) % MOD;
        return r;
    }
};
// 斐波那契：[[1,1],[1,0]]^(n-1) 乘 [f2,f1]^T
```

构造转移矩阵的方法：把「需要保留的状态」写成列向量，逐行写出下一时刻的表达式，系数即矩阵元素。带常数项时把常数当作一个恒为 1 的状态加一维。

常见应用：斐波那契第 n 项、路径计数（邻接矩阵的 k 次幂 = 走 k 步的方案数）、带系数的线性递推、DP 的批量转移（把「一天」写成矩阵，n 天就是矩阵幂）。

更快的做法：k 阶线性递推可用 BM 算法（Berlekamp-Massey）从前面若干项反推递推式，再用多项式取模做到 O(k log k log n)。省选以上常用。

29.6 博弈论

竞赛里的博弈 90% 是「公平组合游戏（ICG）」，用 SG 定理解决。

三个结论：

- 必败态：所有可达状态都是必胜态；必胜态：存在一个可达的必败态。
- SG 函数：`sg(u) = mex{ sg(v) : u→v }`，`sg=0` 为必败。
- SG 定理：多个独立子游戏时，总 SG = 各子游戏 SG 的异或和；异或和非 0 先手必胜。

```
int sg[MAXN]; bool vis[MAXN];
void getSG(int n) {
    for (int i = 1; i <= n; ++i) {
        memset(vis, 0, sizeof vis);
        for (int v : moves(i)) vis[sg[v]] = true;      // v < i
        for (int g = 0; ; ++g) if (!vis[g]) { sg[i] = g; break; }
    }
}
```

Nim 游戏：n 堆石子，每次从一堆取任意多个，SG 就是堆大小，异或和非 0 先手胜。阶梯 Nim：只看奇数级台阶的异或和。

非公平游戏（双方可用操作不同）不能用 SG，要用「状态图 + 必胜/必败标记」直接 DP，或分析不变量。

29.7 多项式与 NTT 入门

多项式乘法是「卷积」的原型：`c[k] = Σ a[i]·b[k-i]`。朴素 $O(n^2)$，FFT/NTT 做到 O(n log n)。

```
// NTT（模数 998244353，原根 3）
const int MOD = 998244353, G = 3;
void ntt(vector<int>& a, bool inv) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        int w = qpow(G, (MOD - 1) / len, MOD);
        if (inv) w = qpow(w, MOD - 2, MOD);
        for (int i = 0; i < n; i += len) {
            long long cur = 1;
            for (int j = 0; j < len / 2; ++j) {
                int u = a[i+j], v = cur * a[i+j+len/2] % MOD;
                a[i+j] = (u + v) % MOD;
                a[i+j+len/2] = (u - v + MOD) % MOD;
                cur = cur * w % MOD;
            }
        }
    }
    if (inv) { int iv = qpow(n, MOD - 2, MOD); for (int &x : a) x = (long long)x * iv % MOD; }
}
```

为什么用 NTT 不用 FFT：FFT 有精度误差，系数到 10^5 就可能错；NTT 在模 998244353 下是精确整数运算，且 $998244353 = 119\times2^{23}+1$，支持长度到 2^{23}。

多项式全家桶（省选/NOI 常考）：求逆、ln、exp、开根、除法、多点求值、快速插值。它们的共同基础是 NTT + 牛顿迭代。竞赛中建议背模板而不是现场推。

29.8 Burnside 引理与 Polya 计数

题干里出现「旋转后相同算一种」「翻转后相同算一种」「本质不同的染色方案」，且规模只到 10^9 级、答案对质数取模，基本就是 Burnside + Polya，不需要任何组合构造技巧。

Burnside 引理：有限群 G 作用在集合 X 上，轨道数 `|X/G| = (1/|G|)·Σ_{g∈G} |Fix(g)|`，其中 `Fix(g)` 是被 g 固定不动的元素个数。Polya 把它变成可算的形式：若置换 g 分解成 `c(g)` 个循环，用 k 种颜色染色，则 `|Fix(g)| = k^{c(g)}`。于是问题只剩「数每个置换有多少个循环」。

三个必须记熟的循环数：

置换群	元素与循环数	结果
n 元环的旋转 C_n（项链）	旋转 i 位：`c = gcd(i, n)`，i = 0..n-1	`(1/n)·Σ_{i=0}^{n-1} k^{gcd(i,n)}`
n 元环的二面体 D_n（手环）	n 奇：n 条对称轴各固定 `k^{(n+1)/2}`；n 偶：n/2 条过顶点轴固定 `k^{n/2+1}`，n/2 条过边轴固定 `k^{n/2}`	旋转和加翻转和后除以 2n
立方体 24 个旋转（六面染色）	恒等 1 个 `k^6`；3 个面轴 180° 为 `k^4`；6 个面轴 90°/270° 与 6 个棱轴 180° 各 `k^3`；8 个体对角 120°	`(k^6 + 3k^4 + 12k^3 + 8k^2)/24`

为 `k^2`

```
long long qpow(long long a, long long b, long long p) {
    long long r = 1 % p;
    for (a %= p; b; b >>= 1, a = a * a % p) if (b & 1) r = r * a % p;
    return r;
}

// 项链：只允许旋转，|G| = n
long long polyaCycle(int n, int k, long long p) {
    long long s = 0;
    for (int i = 0; i < n; ++i) s = (s + qpow(k, gcd(i, n), p)) % p;
    return s * qpow(n, p - 2, p) % p;              // p 必须是质数且 n % p != 0
}

// 手环：旋转 + 翻转，|G| = 2n
long long polyaBracelet(int n, int k, long long p) {
    long long s = 0;
    for (int i = 0; i < n; ++i) s = (s + qpow(k, gcd(i, n), p)) % p;
    if (n & 1) s = (s + (long long)n * qpow(k, (n + 1) / 2, p)) % p;
    else s = (s + (long long)(n / 2) * ((qpow(k, n / 2 + 1, p) + qpow(k, n / 2, p)) % p)) % p;
    return s * qpow(2LL * n % p, p - 2, p) % p;
}
```

复杂度：上面按 i 逐项算是 O(n log n)。n 很大（10^9）时按 `gcd(i,n) = d` 分组，每组的 i 个数为 φ(n/d)，改写为 `Σ_{d|n} φ(d)·k^{n/d}`，只需枚举 n 的约数，O(τ(n) log n)。φ 用试除 O(√n) 现算即可。

易错点：① 除以 |G| 只能走逆元，模数非质数或 `2n ≥ p` 时逆元不存在，必须换大模数或改用大整数；② Polya 算的是「给定 k 种颜色、允许某些颜色一种都不用」的方案数，题目若说「恰好用满 k 种」，需再容斥 `Σ_{j=0}^{k} (-1)^j C(k,j)·f(k-j)`；③ 项链与手环不同，只有题面明确「翻转后相同」才加翻转项，多加会偏小、少加会偏大；④ 三维体（立方体、正四面体、正八面体）的旋转要按「面轴 / 体对角 / 棱轴」三类数全，漏一类必错；⑤ 循环数由 gcd 给出，不是 `n/i`，也不是「位数」。

自查办法：小数据（n ⩽ 8、k ⩽ 3）枚举全部 k^n 个染色、按旋转（和翻转）取字典序最小做代表元去重，与本模板对拍。这组对拍能同时抓住「少加翻转」「奇偶分类写反」「除错 |G|」三类错误。

29.9 杜教筛与 min25 筛：亚线性求和

判定条件：问 `Σ_{i=1}^{n} f(i)` 而 n ⩾ 10^9、线性筛 O(n) 跑不动；f 是积性函数，且 f(p) 能写成 p 的低次多项式（或素数处的和可单独求出）。满足这两条就是亚线性求和，两条路线的分工完全不同：

方法	前提	复杂度	典型对象
线性筛	$n \leqslant 10^7$	O(n)	任意积性函数
杜教筛	能构造 g 使 h = f*g 的前缀和好算	$O(n^{2/3})$	φ、μ、d、σ
min25 筛	f(p) 是 p 的多项式，$f(p^k)$ 可算	$O(n^{3/4}/\log n)$	φ、μ、σ_k、$f(p) = p^2-1$ 等

杜教筛的推导只有一行：令 h = f * g（狄利克雷卷积），则 `Σ_{i≤n} h(i) = Σ_{d≤n} g(d)·S(⌊n/d⌋)`。把 d = 1 那一项 `g(1)·S(n)` 单独提出，得

`S(n) = ( Σ_{i≤n} h(i) − Σ_{d=2}^{n} g(d)·S(⌊n/d⌋) ) / g(1)`

右边按 ⌊n/d⌋ 分块只有 O(√n) 段，记忆化后总复杂度 $O(n^{2/3})$（预处理到 $n^{2/3}$）。配 g 的两张常用表：φ 取 g = 1（h = id，前缀和 n(n+1)/2）；μ 取 g = 1（h = ε，前缀和恒为 1，于是 `S(n) = 1 − Σ_{d=2}^{n} S(⌊n/d⌋)`）。

```
const int N = 2000000;              // 预处理上界，取 n^(2/3)
const long long MOD = 1000000007, INV2 = 500000004;
long long phi[N + 1];
int pri[N], pc; bool npri[N + 1];

void pre() {                        // 线性筛 + φ 前缀和（全部取模）
    phi[1] = 1;
    for (int i = 2; i <= N; i++) {
        if (!npri[i]) { pri[++pc] = i; phi[i] = i - 1; }
        for (int j = 1; j <= pc && (long long)i * pri[j] <= N; j++) {
            int v = i * pri[j]; npri[v] = true;
            if (i % pri[j] == 0) { phi[v] = phi[i] * pri[j] % MOD; break; }
            phi[v] = phi[i] * (pri[j] - 1) % MOD;
        }
    }
    for (int i = 2; i <= N; i++) phi[i] = (phi[i] + phi[i - 1]) % MOD;
}

unordered_map<long long, long long> memo;
long long S(long long n) {
    if (n <= N) return phi[n];
    auto it = memo.find(n);
    if (it != memo.end()) return it->second;
    long long res = n % MOD * ((n + 1) % MOD) % MOD * INV2 % MOD;   // Σ_{i≤n} id(i)
    for (long long l = 2, r; l <= n; l = r + 1) {
        r = n / (n / l);
        res = (res - (r - l + 1) % MOD * S(n / l) % MOD + MOD) % MOD;
    }
    return memo[n] = res;
}
```

min25 筛不需要配 g，分三步。① 只用 ≤ √n 的素数跑「扩展埃氏筛」，把 `g(x) = Σ_{2≤i≤x} f_poly(i)` 逐步修正为 `G(x) = Σ_{p≤x} f(p)`（φ 的 f(p) = p − 1 是两项：Σp 与 Σ1，所以开两个数组 g1、g0，最后相减）。② 递归 `S(x, j)` = 「≤ x 且最小质因子 ≥ p_j 的所有数的 f 之和」，枚举最小质因子 p_k 与它的幂次，累加 `f(p^e)·S(x/p^e, k+1) + f(p^{e+1})`。③ 答案为 `S(n,1) + f(1)`。

```
const int MAXS = 1000005;           // ≥ √n，n ≤ 1e12 够用
const long long MOD = 1000000007;
long long n, sq, w[MAXS];
int id1[MAXS], id2[MAXS], m;        // x ≤ sq 用 id1[x]，否则用 id2[n/x]
long long g1[MAXS], g0[MAXS];       // 多项式部分：Σp、Σ1
int pri[MAXS], pc; bool npri[MAXS];
long long sp1[MAXS], sp0[MAXS];     // 素数前缀：Σp、素数个数

int id(long long x) { return x <= sq ? id1[x] : id2[n / x]; }

void sieve_primes() {
    sq = sqrtl((long double)n);
    for (int i = 2; i <= sq; i++) {
        if (!npri[i]) { pri[++pc] = i; sp1[pc] = sp1[pc - 1] + i; sp0[pc] = sp0[pc - 1] + 1; }
        for (int j = 1; j <= pc && (long long)i * pri[j] <= sq; j++) {
            npri[i * pri[j]] = true;
            if (i % pri[j] == 0) break;
        }
    }
    pri[pc + 1] = 0;                // 哨兵：递归到 j = pc+1 时 pri[j] > x 恒不成立
}

void build_g() {
    for (long long l = 1, r; l <= n; l = r + 1) {
        r = n / (n / l); w[++m] = n / l;
        if (w[m] <= sq) id1[w[m]] = m; else id2[n / w[m]] = m;
        g1[m] = (w[m] + 2) % MOD * ((w[m] - 1) % MOD) % MOD * 500000004 % MOD;  // Σ_{2..w} i
        g0[m] = (w[m] - 1) % MOD;                                               // Σ_{2..w} 1
    }
    for (int j = 1; j <= pc; j++) {
        long long p = pri[j], p2 = p * p;
        for (int i = 1; i <= m && w[i] >= p2; i++) {
            int k = id(w[i] / p);
            g1[i] = (g1[i] - p % MOD * ((g1[k] - sp1[j - 1]) % MOD + MOD) % MOD + MOD) % MOD;
            g0[i] = (g0[i] - ((g0[k] - sp0[j - 1]) % MOD + MOD) + MOD) % MOD;
        }
    }
}

long long S(long long x, int j) {   // Σ_{2≤i≤x, 最小质因子 ≥ pri[j]} ϕ(i)
    if (pri[j] > x) return 0;
    int ix = id(x);
    long long res = ((g1[ix] - g0[ix]) % MOD + MOD) % MOD;          // Σ_{p≤x} ϕ(p)
    res = (res - ((sp1[j - 1] - sp0[j - 1]) % MOD + MOD) + MOD) % MOD;
    for (int k = j; k <= pc && (long long)pri[k] * pri[k] <= x; k++) {
        long long p = pri[k], pe = p, pm = 1;
        for (int e = 1; pe * p <= x; e++) {
            long long fp = (pe - pm) % MOD;                         // ϕ(p^e)
            res = (res + fp * S(x / pe, k + 1) + (pe * p - pe) % MOD) % MOD;
            pm = pe; pe *= p;
        }
    }
    return res;
}

int main() {
    cin >> n;
    sieve_primes(); build_g();
    cout << (S(n, 1) + 1) % MOD << endl;   // +ϕ(1)
}
```

复杂度与取舍：min25 是 $O(n^{3/4}/\log n)$、空间 $O(\sqrt{n})$，$n = 10^{10}$ 时 C++ 约 0.1—0.3 s；杜教筛 $O(n^{2/3})$，但要先配出 g 并预处理到 $n^{2/3}$（$n = 10^{10}$ 时约 2×10^6）。$n \leqslant 10^{11}$ 用 min25 更稳，杜教筛的优势在 g 好找且要多点求值。

易错点：① g1/g0 的初值必须写成 `Σ_{i=2}^{w}` 而不是 `Σ_{i=1}^{w}`，写错会整体多算 1；② 递归里的 `if (pri[j] > x) return 0` 不能改成 `j > pc` 就返回——j = pc+1 时仍要用 G(x) 统计「比 $\sqrt{n}$ 大的素数」，硬返回 0 会让所有 $x > \sqrt{n}$ 的态丢掉大素数部分（对拍最能抓到这个）；③ `id(x)` 只对 n/i 形式的数有效，递归里写成 `id(x - 1)` 之类会越界；④ w 数组按 n/i 分块生成，共 $2\sqrt{n}$ 个，MAXS 要开 $2\sqrt{n}$ 以上；⑤ 所有「相减」都要先 `+MOD` 再取模，C++ 的负数取模是负的；⑥ $\phi(p^e) = p^e - p^{e-1}$，累加时要连 `f(p^{e+1})` 一起加，漏掉这一项会少算所有纯素数幂；⑦ 答案别忘了加回 f(1) = 1。

自查：对 $n \leqslant 10^6$ 写一个线性筛暴力求前缀和，与本模板逐点对拍；再单独检查 `G(x) - sp[j-1]` 在 x 略大于 $\sqrt{n}$ 时是否仍等于「区间内素数的 f 之和」。

29.10 生成函数：把计数问题变成多项式运算

判定：答案由「若干独立部分拼接或无序组合」构成，直接 DP 是 $O(n^2)$ 以上，而 n 到 10^5~10^6。典型信号是：题面出现「任意多个」「无序选取」「连通块组成整体」「划分成若干部分」，且模数是 998244353（这是在暗示 NTT）。

核心思想只有一句：把数列 a_n 编码成幂级数 `A(x)`，组合结构对应多项式运算，算完再取系数。

运算	普通型 OGF：$A(x) = \Sigma\, a_n x^n$	指数型 EGF：$A(x) = \Sigma\, a_n x^n/n!$
`A·B`	有序拼接（卷积）：先放一个 A 再放一个 B	标号对象的笛卡尔积（两边标号要重新分配）
`1/(1-A)`	SEQ(A)：任意长度的 A 序列	有序的标号序列
`exp(A)`	无标号集合——不能直接用，要 Polya / 欧拉变换修正	SET(A)：无序挑选若干个 A 对象，这一步是 EGF 存在的全部理由
`A'`（求导）	系数乘 n，一般无组合意义	「给对象打一个标记」：从 A 中选出一个元素做特殊标记
`ln(A)`	少用	「连通分量」：整体 = 连通块的集合时，C = ln F

五个必须背下来的闭式：

生成函数	系数 / 用途
$1/(1-x) = \Sigma\ x^n$	全 1 序列：每个位置恰一种填法
$1/(1-x)^k = \Sigma\ C(n+k-1,\ k-1)\ x^n$	k 类物品的无序可重选取（隔板法）
$e^x = \Sigma\ x^n/n!$	每个标号位置独立地选或不选
$\ln 1/(1-x) = \Sigma\ x^n/n$	循环 / 轮换：n 个元素的圆排列有 (n−1)！种
$(1-\sqrt{(1-4x)})/(2x) = \Sigma\ Cat_n x^n$	卡特兰数；来自 $C = 1 + xC^2$ 解二次方程

建模三步：① 定「基本对象」是什么（一个点、一个连通块、一个字符、一个括号对）；② 定整体与基本对象的结构关系是 SEQ（有序）、SET（无序）还是 CYC（循环）；③ 判对象是否带标号——带标号用 EGF，不带用 OGF。三步定完，式子基本就写出来了。

```
// 多项式全家桶：inv / ln / exp，模 998244353，总复杂度 O(n log^2 n)
const int MOD = 998244353, G = 3;
int qp(long long a, long long b) {
    long long r = 1;
    for (a %= MOD; b; b >>= 1, a = a * a % MOD) if (b & 1) r = r * a % MOD;
    return (int)r;
}

void ntt(vector<int>& a, bool inv) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        int w = qp(G, (MOD - 1) / len);
        if (inv) w = qp(w, MOD - 2);
        for (int i = 0; i < n; i += len) {
            long long cur = 1;
            for (int j = 0; j < len / 2; ++j) {
                int u = a[i + j], v = cur * a[i + j + len / 2] % MOD;
                a[i + j] = (u + v) % MOD;
                a[i + j + len / 2] = (u - v + MOD) % MOD;
                cur = cur * w % MOD;
            }
        }
    }
    if (inv) { int iv = qp(n, MOD - 2); for (int& x : a) x = (long long)x * iv % MOD; }
}

vector<int> mul(vector<int> a, vector<int> b) {
    if (a.empty() || b.empty()) return {};
    int need = a.size() + b.size() - 1, n = 1;
    while (n < need) n <<= 1;
    a.resize(n); b.resize(n);
    ntt(a, false); ntt(b, false);
    for (int i = 0; i < n; ++i) a[i] = (long long)a[i] * b[i] % MOD;
    ntt(a, true);
    a.resize(need);
    return a;
}

// 求逆 b = a^{-1} mod x^n，要求 a[0] != 0；牛顿迭代 b <- b(2 - ab)
vector<int> polyInv(vector<int> a, int n) {
    vector<int> b(1, qp(a[0], MOD - 2));
    for (int m = 2; m < 2 * n; m <<= 1) {
        int len = min(m, n);
        vector<int> f(a.begin(), a.begin() + min((int)a.size(), len));
        f.resize(len);
        vector<int> t = mul(mul(b, b), f);
        t.resize(len);
        b.resize(len);
        for (int i = 0; i < len; ++i) b[i] = (2LL * b[i] % MOD - t[i] + MOD) % MOD;
    }
    b.resize(n);
    return b;
}

vector<int> polyDeriv(const vector<int>& a) {
    vector<int> d((int)a.size() > 1 ? (int)a.size() - 1 : 1);
```

```
    for (int i = 1; i < (int)a.size(); ++i) d[i - 1] = (long long)i * a[i] % MOD;
    return d;
}

vector<int> polyInteg(const vector<int>& a) {      // 长度 n -> n+1，常数项 0
    int n = a.size();
    vector<int> inv(n + 2, 0);
    if (n + 1 >= 1) inv[1] = 1;
    for (int i = 2; i <= n + 1; ++i) inv[i] = (long long)(MOD - MOD / i) * inv[MOD % i] % MOD;
    vector<int> r(n + 1, 0);
    for (int i = 0; i < n; ++i) r[i + 1] = (long long)a[i] * inv[i + 1] % MOD;
    return r;
}

// ln a = ∫ a'/a，要求 a[0] == 1
vector<int> polyLn(vector<int> a, int n) {
    vector<int> d = polyDeriv(a), ia = polyInv(a, n);
    vector<int> q = mul(d, ia);
    q.resize(max(0, n - 1));
    vector<int> r = polyInteg(q);
    r.resize(n);
    return r;
}

// exp a，要求 a[0] == 0；牛顿迭代 f <- f(1 - ln f + a)
vector<int> polyExp(vector<int> a, int n) {
    vector<int> f(1, 1);
    for (int m = 2; m < 2 * n; m <<= 1) {
        int len = min(m, n);
        vector<int> lf = polyLn(f, len);
        vector<int> h(len);
        for (int i = 0; i < len; ++i) h[i] = ((i < (int)a.size() ? a[i] : 0) - lf[i] + MOD) % MOD;
        h[0] = (h[0] + 1) % MOD;
        f = mul(f, h);
        f.resize(len);
    }
    f.resize(n);
    return f;
}
```

n ≤ 5000 时不要用上面这套。exp 的 $O(n^2)$ 递推更短、更快、更好调：设 g = exp(a)，由 `g' = a'g` 得 $n\cdot g_n = \Sigma_{k=1..n}\ k\cdot a_k\cdot g_{n-k}$，直接顺推即可，连 NTT 都不需要。

```
// O(n^2) 版 exp：g = exp(a)，要求 a[0] == 0
vector<int> expNaive(const vector<int>& a, int n) {
    vector<int> g(n, 0); g[0] = 1;
    for (int i = 1; i < n; ++i) {
        long long s = 0;
        for (int k = 1; k <= i; ++k) s = (s + (long long)k * a[k] % MOD * g[i - k]) % MOD;
        g[i] = s * qp(i, MOD - 2) % MOD;
    }
    return g;
}
```

完整例题：n 个点的标号连通图有多少个。n 个点的所有图有 $2^{C(n,2)}$ 个，记其 EGF 为 F；任一图是若干连通图的无序集合，于是 `F = exp(C)`，即 `C = ln F`，答案是 $n!\cdot[x^n]\ \ln F$。这就是 EGF 里 `exp` 对应 SET、`ln` 对应连通分量的标准用法。

```
// 输入 n，输出 n 个点的标号连通图个数（模 998244353）
int n; cin >> n;
vector<int> f(n + 1);
long long fact = 1;
for (int i = 0; i <= n; ++i) {
    if (i) fact = fact * i % MOD;                       // fact = i!
    f[i] = (long long)qp(2, 1LL * i * (i - 1) / 2) * qp(fact, MOD - 2) % MOD;
}
vector<int> c = polyLn(f, n + 1);                       // 连通图 EGF
cout << (long long)c[n] * fact % MOD << "\n";
// 校验值：n = 1..6 -> 1, 1, 4, 38, 728, 26704
```

复杂度：inv / ln / exp 单次 O(n log n)，套倍增后 exp 为 O(n $\log^2$ n)；O(n^2) 递推版在 n $\leqslant$ 5000 时更快。空间 O(n)。

易错点：

- **EGF 忘了除 n!。系数写成 a_n 而不是 $a_n/n!$，exp 得到的会是有序版本，答案恰好差一个 n! 的因子——对拍时表现为「小数据全对、n $\geqslant$ 3 全错」。**
- **常数项不满足前提。ln 要求 A(0) = 1，exp 要求 A(0) = 0，inv 要求 A(0) $\neq$ 0。不满足时牛顿迭代不收敛，输出一片乱码而非报错。**
- **模数不支持 NTT。998244353 = 119$\times 2^{23}$+1 可用；答案模 10^9+7 时必须换任意模数卷积（三模 CRT）或退回 O(n^2)。**
- **把 OGF 的 exp 当集合用。无标号对象的无序集合是 $\exp(\Sigma\, a_k x^k/k)$（欧拉变换），不是 exp(A)；两者只差在指数上的 k，但答案完全不同。**
- **「恰好 k 个」直接取系数。要二元生成函数 A(x)y，再取 $[x^n y^k]$；不能先 exp 再取第 k 项。**
- **数组长度不足。mul 的结果长度是两段长度之和减一，中间 resize 一次不到位就会静默丢掉高次项。**

29.11 Miller-Rabin 素性测试与 Pollard-Rho 分解

解决什么问题：n 到 10^{18} 时，29.2 的线性筛与 O(√n) 试除同时失效——试除要 10^9 次除法，判一个数就要数秒。Miller-Rabin 把「判素」压到 12 次模幂，Pollard-Rho 把「分解」压到毫秒级。这一对是筛法之后唯一的接力手段，凡是题面出现 $n \le 10^{18}$ 且要求质因子、约数个数、欧拉函数、原根，都是它们的适用面。

判定条件（Miller-Rabin 的正确性来自两条定理）

- **费马小定理：p 为素数且 p ∤ a，则 $a^{p-1} \equiv 1 \pmod p$。逆否命题给出一个必要条件：$a^{n-1} \not\equiv 1$ 则 n 必为合数。但反过来不成立（Carmichael 数 561、1105 等满足所有互素底，却是合数），所以单靠它不够。**
- **二次探测定理：p 为奇素数，若 $x^2 \equiv 1 \pmod p$，则 $x \equiv 1$ 或 $x \equiv -1 \pmod p$。合数没有这个性质——这正是把 Carmichael 数筛掉的那一刀。**

做法：把 n−1 写成 $2^s \cdot d$（d 为奇数）。对每个底 a 算出 $x = a^d \bmod n$：若 x = 1 或 x = n−1 则本底通过；否则反复平方最多 s−1 次，中途出现 n−1 则通过；一次都没出现，n 必为合数。全部底通过即判为

素数。

底数怎么选：底数不是越多越好，而是「够用即止」。取前 12 个素数 `{2,3,5,7,11,13,17,19,23,29,31,37}`，对 `n < 3.3×10^24` 无假阳性——这已远超 `unsigned long long` 的上界 1.8×10^{19}，因此 64 位整数范围内 12 个底是确定性算法，不是概率算法。只用 `{2,3,5}` 只保证到 2.5×10^{13}，是常见的假阳性来源。

```
typedef unsigned long long ull;
ull mul(ull a, ull b, ull m) { return (ull)((__int128)a * b % m); }   // 必须 128 位中转
ull qpow(ull a, ull b, ull m) {
    ull r = 1 % m;
    for (; b; b >>= 1) { if (b & 1) r = mul(r, a, m); a = mul(a, a, m); }
    return r;
}
const ull BASE[12] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37};

bool isPrime(ull n) {
    if (n < 2) return false;
    for (int i = 0; i < 12; ++i)
        if (n % BASE[i] == 0) return n == BASE[i];     // 小素数先判掉，同时保证下面 d != 0
    ull d = n - 1; int s = 0;
    while (!(d & 1)) { d >>= 1; ++s; }               // n-1 = 2^s * d
    for (int i = 0; i < 12; ++i) {
        ull x = qpow(BASE[i], d, n);
        if (x == 1 || x == n - 1) continue;            // 第一层就通过
        bool pass = false;
        for (int r = 1; r < s; ++r) {
            x = mul(x, x, n);
            if (x == n - 1) { pass = true; break; }    // 命中 -1，本底通过
            if (x == 1) return false;                  // 平方链提前回到 1 且不是 ±1，必合数
        }
        if (!pass) return false;
    }
    return true;
}
```

Pollard-Rho 的想法：设 `n = p·q`。在模 `n` 下跑伪随机序列 $x_{i+1} = x_i^2 + c$，模 `p` 的投影也构成同样的序列，但状态空间只有 `p` 个，由生日悖论，期望 $O(\sqrt{p}) = O(n^{1/4})$ 步就会撞上重复。一旦 $x_i \equiv x_j \pmod p$，就有 $p \mid |x_i - x_j|$，于是 $\gcd(|x_i-x_j|, n)$ 给出一个非平凡因子。Floyd 判环（一个指针走一步、一个走两步）保证两个指针在模 `p` 下必然相遇，且空间 O(1)。

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

ull pollard(ull n) {                        // 前提：n 为合数，n > 4
    if (!(n & 1)) return 2;                 // 偶数必须先摘掉，否则下面可能死循环
    if (n % 3 == 0) return 3;
    while (true) {
        ull c = rng() % (n - 1) + 1;        // 随机常数，落在 [1, n-1]
        ull x = rng() % (n - 1) + 1, y = x, d = 1;
        auto f = [&](ull v) { return (mul(v, v, n) + c) % n; };
        while (d == 1) {
            x = f(x);                       // 慢指针一步
            y = f(f(y));                    // 快指针两步
            d = gcd(x > y ? x - y : y - x, n);
        }
        if (d != n) return d;               // d == n 说明该 c 失效，换一个重来
    }
}

void factor(ull n, vector<ull>& fac) {
    if (n == 1) return;
    if (isPrime(n)) { fac.push_back(n); return; }
    ull d = pollard(n);
    factor(d, fac);
    factor(n / d, fac);
}
// 用法：factor(n, fac); sort(fac.begin(), fac.end());
// 之后对已排序序列做 run-length，得到「质因子 -> 指数」，再算约数个数 / 欧拉函数
```

复杂度：Miller-Rabin 单底一次模幂 `O(log n)` 次 64 位模乘，12 底共约 720 次乘法，微秒级；Pollard-Rho 期望 `O(n^1/4)` 次迭代再加同样次数的 `gcd`，分解 10^{18} 量级的数（最坏是两素因子相近的半素数）实测在 10^7 次迭代内收敛。整体「判素 + 完全分解」对 64 位整数是毫秒级。

易错点

- **乘法不转 128 位。`a*b` 在 `ull` 下直接乘积溢出，`% m` 得到的余数毫无意义，表现为「小数据全对、10^{10} 以上全错」。必须 `(__int128)a * b % m`。**
- **小素数没先判。`n = 2` 时 `d = 1`，`while (!(d&1))` 里 `d >>= 1` 变 0，`0` 是偶数，死循环。`n = 4` 同样出问题。入口那句「12 个底逐个试除、命中就返回 `n == BASE[i]`」正是为了堵这个洞。**
- **底数只给 2、3、5。看似能过绝大多数数据，但 3.2×10^{13} 以上存在强伪素数，出题人专门造这种数。**
- **`s` 的循环边界。平方次数是 `s-1` 次（第一层已经算过 a^d），写成 `r <= s` 会多平方一次，把合法素数误判为合数。**
- **漏了 `x == 1` 的提前返回。平方链中途出现 1 且此前未出现 `n-1`，由二次探测定理立刻可知 `n` 是合数；继续循环只是浪费，但若底数集合不足，漏掉这一条会进一步降低检出率。**
- **`pollard` 对偶数不特判。`f` 在偶数模下可能进入固定点，`d` 永远等于 1 或 `n`，外层 `while(true)` 死循环且每次都重新抽 `c`，看起来像卡死。**
- **`pollard` 返回 `n` 后没有重试。有些 `c` 会让快慢指针在整个模 `n` 下同步（即 `d = n`），此时必须换 `c` 重来；直接 `return d` 会让 `factor(n)` 以 `pollard(n) = n` 递归自身，栈溢出。**

- 用 `rand()` 或固定种子。`rand()` 只有 32767 个值，随机常数极易重复；固定种子则等于把 `c` 的选择暴露给出题人，可构造让所有 `c` 都失败的数。`mt19937_64` + `steady_clock` 是底线。
- 分解结果没排序、没合并。约数个数要对「质因子 → 指数」做 $\prod(e_i+1)$，直接对未排序数组计数会把同一个质因子当成多个不同因子。
- `factor(1)` 没有基例。递归到 1 时若不返回，`pollard(1)` 里 `n-1 = 0`，取模零直接崩。函数第一行必须是 `if (n == 1) return;`。
- 用 Pollard-Rho 判素。它只负责「找一个因子」，判素必须交给 Miller-Rabin；混用会导致对素数调用 `pollard`，永远返回 `n`。
- 想用它分解 10^{30}。模板依赖 `__int128`，上界就是 64 位；更大规模要换大数库或二次筛（QS），不在竞赛范围。

29.12 矩阵树定理与生成树计数

一句话结论：无向图的生成树个数，等于它的基尔霍夫矩阵 `K = D − A` 的任意一个 `n-1` 阶主子式的行列式。`D` 是度数矩阵（`D[i][i] = deg(i)`），`A` 是邻接矩阵，重边按条数累加。把最后一行一列划掉最省事。

为什么是行列式：把行列式按置换展开，每一项对应「每个点选一条出边」所形成的一个函数图；含环的那些项在展开中成对抵消（等价说法：$K = B \cdot B^{\mathsf{T}}$，`B` 是点—边关联矩阵，再用 Cauchy–Binet 展开成「选 n−1 条边」的子式之和），最后只剩「无环且连通」的项，而连通且无环的 n−1 条边就是一棵树，每个树的贡献恰好是 1。两个推论值得记住：无向图划去哪一行哪一列结果都一样；答案一定是整数，尽管中间用了模逆元。

加权版才是主战场。把 `A[i][j]` 换成边 `(i,j)` 的权 `w`、对角元换成 `Σ w`，行列式给出的是 $\Sigma_{\text{树 } T} \prod_{e \in T} w_e$。所有 `w = 1` 退化为计数；让某类边的权是 `x`、其余是 `1`，取 x^k 的系数就得到「恰好用 k 条这类边」的生成树数——「按边数分层计数」全靠这一手，比状压 DP 干净得多。

有向图：两种树形图，矩阵方向是反的。这是本节唯一必须背准的地方，写反不会报错，只是答案系统性错（本轮对拍第一版就写反了，400 组随机图错 452 次）：

要数的对象	矩阵怎么建（对边 u→v）	划去
所有边都指向根 r（内向树：每个点沿边最终走到 r）	出度矩阵：`L[u][u] += 1`、`L[u][v] -= 1`	第 r 行与第 r 列
所有边都背离根 r（外向树：从 r 出发能到达所有点）	入度矩阵：`L[v][v] += 1`、`L[v][u] -= 1`	第 r 行与第 r 列
无向图（每条边两个端点各加 1）	度数矩阵	任意一行一列

记忆法：「指向根」用出度、「背离根」用入度，正好和直觉相反，因为被划掉的根那一行的度数信息被扔掉了，剩下的是「非根点各自出发的边」。考场上先拿 3 个点的小图手算一遍再写主程序，比背表可靠。

复杂度与规模

情形	做法	复杂度	可用规模

模数为质数	**高斯消元 + 费马逆元**	`O(n^3)`	**n ≤ 500（约 1.2×10^8 次乘法，2 s 内）**
模数为合数	**辗转相除消元（欧几里得消元）**	`O(n^3 log p)`	**n ≤ 200 较稳**
答案是精确值（不取模）	**同上 + 高精度/BigInteger**	`O(n^3)` **次大数运算**	**n ≤ 60 左右**

可编译模板

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int MAXN = 505;
ll g[MAXN][MAXN], c[MAXN][MAXN];

ll qpow(ll a, ll b, ll p) {
    ll r = 1 % p;
    for (a %= p; b; b >>= 1, a = a * a % p) if (b & 1) r = r * a % p;
    return r;
}

// p 为质数：普通高斯消元 + 费马小定理求逆
ll detPrime(ll a[][MAXN], int n, ll p) {
    ll res = 1;
    for (int i = 0; i < n; ++i) {
        int piv = -1;
        for (int r = i; r < n; ++r) if (a[r][i] % p) { piv = r; break; }
        if (piv < 0) return 0;                          // 整列为 0 => 行列式为 0
        if (piv != i) {
            for (int k = 0; k < n; ++k) swap(a[i][k], a[piv][k]);
            res = (p - res) % p;                        // 换行变号
        }
        res = res * (a[i][i] % p) % p;
        ll inv = qpow(a[i][i] % p, p - 2, p);
        for (int r = i + 1; r < n; ++r) {
            if (!a[r][i]) continue;
            ll f = a[r][i] % p * inv % p;
            for (int k = i; k < n; ++k)
                a[r][k] = ((a[r][k] - f * a[i][k]) % p + p) % p;
        }
    }
    return (res % p + p) % p;
}

// p 任意（含合数）：欧几里得消元，先减后换行，不可调序
ll detAny(ll a[][MAXN], int n, ll p) {
    ll res = 1 % p;
    for (int i = 0; i < n; ++i) {
        for (int r = i + 1; r < n; ++r) {
            while (a[r][i]) {                           // 把 a[r][i] 磨成 0
                ll t = a[i][i] / a[r][i];
                for (int k = i; k < n; ++k)             // ① 先用 t 倍行减
                    a[i][k] = ((a[i][k] - t * a[r][k]) % p + p) % p;
                for (int k = i; k < n; ++k) swap(a[i][k], a[r][k]);   // ② 再交换
                res = (p - res) % p;
            }
        }
        if (!a[i][i]) return 0;
        res = res * a[i][i] % p;
    }
    return res % p;
}

// 无向图生成树计数：e 中每条无向边只放一次 (u,v)
ll countTree(int n, const vector<pair<int,int>>& e, ll p, bool pIsPrime) {
    if (n == 1) return 1 % p;                           // 单点空树算 1 种方案
    memset(g, 0, sizeof(g));
    for (auto pr : e) {
        int u = pr.first, v = pr.second;
        if (u == v) continue;                           // 自环必须跳过
        g[u][u] = (g[u][u] + 1) % p;
```

```
        g[v][v] = (g[v][v] + 1) % p;
        g[u][v] = (g[u][v] - 1 + p) % p;
        g[v][u] = (g[v][u] - 1 + p) % p;
    }
    for (int i = 0; i < n - 1; ++i)
        for (int j = 0; j < n - 1; ++j) c[i][j] = g[i][j];   // 划去最后一行一列
    return pIsPrime ? detPrime(c, n - 1, p) : detAny(c, n - 1, p);
}

// 有向图树形图计数：mode = 0 边指向 root，mode = 1 边背离 root
ll countArb(int n, const vector<pair<int,int>>& e, int root, int mode, ll p) {
    if (n == 1) return 1 % p;
    memset(g, 0, sizeof(g));
    for (auto pr : e) {
        int u = pr.first, v = pr.second;
        if (u == v) continue;
        if (mode == 0) {                            // 指向根 => 出度矩阵
            g[u][u] = (g[u][u] + 1) % p;
            g[u][v] = (g[u][v] - 1 + p) % p;
        } else {                                    // 背离根 => 入度矩阵
            g[v][v] = (g[v][v] + 1) % p;
            g[v][u] = (g[v][u] - 1 + p) % p;
        }
    }
    int id[MAXN], m = 0;
    for (int i = 0; i < n; ++i) if (i != root) id[m++] = i;   // 划去根的行列
    for (int i = 0; i < m; ++i)
        for (int j = 0; j < m; ++j) c[i][j] = g[id[i]][id[j]] % p;
    return detPrime(c, m, p);
}
```

四类典型题

- **纯计数：给 n ≤ 500 的无向图求生成树个数。注意模数有时故意给成非质数（如 10^9、$10^{10}+7$ 的合成数），这时必须切到 `detAny`，否则费马逆元直接算错。**
- **最小生成树计数：先用 Kruskal 求出 MST 的边权多重集，再按权值分组；每组处理时把「权值小于当前组」的所有点缩成并查集分量（MST 的连通性已定），只对当前组内连接不同分量的边建图，跑一次矩阵树，各层方案数相乘。正确性依赖一条性质：不同权值的边在任意两棵 MST 里出现次数相同。**
- **生成树权值和的期望 / 分层计数：把权写成多项式系数，矩阵树得到的是乘积和；要「和」而不是「积」时，用 w_e = 1 + y·[e 是特殊边]，y^k 的系数即「恰好 k 条特殊边」的方案数，再乘权值求和。数据小的时候直接对 y 做 `O(n)` 次插值也行。**
- **配合容斥：「不含某条边」= 总方案数 − 含该边的方案数，而「含边 (u,v)」等价于把 u、v 缩成一个点后再跑一次矩阵树。**

易错点

- **自环没跳过。自环对生成树毫无贡献，但 `D[i][i]` 会 +1，行列式立刻变错。入口一句 `if (u == v) continue;` 是硬性要求，本轮对拍专门加了自环用例。**
- **重边被去重。两条 `(u,v)` 边在矩阵里应该是 `A[u][v] = -2`、度数各 +2，把答案从 2 变成 1。用邻接矩阵存图时写 `a[u][v] = 1` 而不是 `+= 1` 就会犯这个错。**

- 无向边当成两条有向边存。把 `(u,v)` 和 `(v,u)` 都加进矩阵，等于把每条边算两次，答案整体乘 2^{n-1}（K_3 会得 12 而不是 3）。用边表 `vector<pair<int,int>>` 且每条边只 push 一次最安全。
- 模数是合数却用费马求逆。a^{p-2} 在 `gcd(a,p) ≠ 1` 时无意义，表现为「小数据对、大数据随机错」。要么切 `detAny`，要么先对模数做质因数分解、逐质因子求行列式再 CRT 合并。
- `detAny` 里先换行后减。写成「换行 → 减 `t` 倍」，残量会变成 `b - t·a`（负数），取模后不在 `[0, b)` 内，欧几里得的终止性被破坏，`while` 可能长时间空转或给出错误值。本轮第一版就是这个顺序，300 组里错 189 组；改成「先减后换」后 1200 组全对。
- `n = 1` 没特判。此时矩阵是 0×0，循环不进，`res` 初值 1 恰好正确，但若代码里对 `n-1` 取了负下标就会越界。显式返回 1 更稳。
- 有向图用错度数矩阵。见前面的表；症状是「样例过、随机数据系统性偏小」，因为数出来的是反方向的树形图，通常直接为 0。
- 有向图换根后答案不变。错误假设。有向树形图的答案依赖根，无向图才与划去哪一行无关。多组询问不同根时，n 小可以每个根跑一次，n 大要用「换根矩阵树」或直接放弃。
- 行列式符号丢失。交换行必须 `res = (p - res) % p`；最后再 `(res % p + p) % p` 兜一次负数。漏掉符号在 $p = 10^9+7$ 下表现为「答案等于 p − 正确值」。
- 稀疏大图硬跑 $O(n^3)$。`n = 10^5` 的树或基环树题，生成树个数有闭式（树上为 1、基环树上为环长等），别套矩阵树；只有「稠密、$n \leqslant 500$」才轮得到它。
- 把矩阵树当最小树形图用。求「边权和最小的树形图」是朱刘算法（`O(nm)`），矩阵树只管计数，两者不可替换。

自查：本轮用 Python 逐行复现模板后与暴力对拍——无向图 600 组（$n \leqslant 6$，含重边与自环，暴力枚举边子集 + 并查集判连通）全对；有向图 400 组 × 内向/外向两种模式全对；合数模数 `p ∈ {2310, 100, 36, 9973}` 各 300 组全对；另用 Cayley 公式校验 $K_n \to n^{n-2}$（$n = 2..8$：1, 3, 16, 125, 1296, 16807, 262144 全中）与环图 $C_n \to n$。

29.13 反演三件套：二项式反演、子集反演与单位根反演

一句话判定：反演就是「已知按某种方式聚合后的结果，反解出每一项」。三种聚合方式对应三种反演——按大小聚合用二项式反演（下标是整数），按子集聚合用子集反演（下标是集合），按整除/同余聚合用单位根反演（下标取模）。题面出现「恰好」「至少」「钦定」「所有子集」「下标是 k 的倍数」这些词时，先判断它属于哪一种，再套对应的逆式，比硬推容斥快得多。

反演	正向（已知 g 求 f）	逆向（由 f 反解 g）	复杂度
二项式	$f(n) = \Sigma_{i=0}^{n} C(n,i)\, g(i)$	$g(n) = \Sigma_{i=0}^{n} (-1)^{n-i} C(n,i)\, f(i)$	$O(n^2)$，卷积可 $O(n \log n)$
二项式（恰好型）	$f(k) = \Sigma_{i\geq k} C(i,k)\, g(i)$	$g(k) = \Sigma_{i\geq k} (-1)^{i-k} C(i,k)\, f(i)$	$O(n^2)$
子集	$f(S) = \Sigma_{T\subseteq S} g(T)$	$g(S) = \Sigma_{T\subseteq S} (-1)^{\lvert S\rvert-\lvert T\rvert} f(T)$	$O(3^n)$ → FMT $O(n2^n)$
单位根	$[k \mid n] = (1/k) \Sigma_{j=0}^{k-1} \omega_k^{jn}$	同式（自逆，只差 $1/k$）	$O(nk)$

① 二项式反演：把「钦定」变「恰好」。它是组合计数里最高频的一步。设 $g(k)$ 是「恰好有 k 个满足条件」的方案数，$f(k)$ 是「钦定某 k 个满足、其余随意」的方案数——注意是钦定（先选出这 k 个，再让它们必须满足），不是「至多 k 个」。此时 $f(k) = \Sigma_{i\geq k} C(i,k)\, g(i)$：一个恰好 i 个的方案，会被「钦定它的 k 个」数到 $C(i,k)$ 次。取逆即得 $g(k) = \Sigma_{i\geq k} (-1)^{i-k} C(i,k)\, f(i)$。

两个必背实例，都从这一条式子直接落下：

- 错排 $D(n) = \Sigma_{i=0}^{n} (-1)^i C(n,i)(n-i)!$——钦定 i 个不动、其余全排列，恰好 0 个不动即错排。
- 满射计数：n 个不同球放进 m 个不同盒且盒不空，$= \Sigma_{i=0}^{m} (-1)^i C(m,i)(m-i)^n$。这是子集反演在「盒子集合」上的特化，与二项式反演同构。

② 子集反演：把「子集和」拆回去。若 $f(S)$ 统计的是「只允许用 S 内元素」的方案数，而 $g(T)$ 统计「恰好用满 T」的方案数，则 $f(S) = \Sigma_{T\subseteq S} g(T)$，反解出 $g(S) = \Sigma_{T\subseteq S} (-1)^{\lvert S\rvert-\lvert T\rvert} f(T)$。这就是集合上的莫比乌斯反演。它的「快速版」是本节的实用核心：把 $O(3^n)$ 的枚举子集换成 FMT（快速莫比乌斯/ζ 变换），逐维做「有这一位就加上去掉这一位的值」，$O(n2^n)$。

还有两个方向容易混：超集版 $f(S) = \Sigma_{T\supseteq S} g(T) \Leftrightarrow g(S) = \Sigma_{T\supseteq S} (-1)^{\lvert T\rvert-\lvert S\rvert} f(T)$，FMT 里把「有这一位就加」改成「没这一位就加」；以及整数上的倍数版（狄利克雷卷积上的莫比乌斯反演），那就是 29.2 与 29.9 的地盘，别混着写。

③ 单位根反演：把「整除」写成求和。核心恒等式 $[k \mid n] = (1/k) \Sigma_{j=0}^{k-1} \omega_k^{jn}$：当 $k \mid n$ 时每项都是 1，和为 k；否则 ω_k^n 是 1 的 k 次非平凡根，等比求和高为 0。它的用途是把「只统计下标为 k 的倍数的项」这件事从筛选变成求值：

$$\Sigma_{k\mid i} a_i = (1/k) \Sigma_{j=0}^{k-1} \Sigma_i a_i \omega_k^{ji} = (1/k) \Sigma_{j=0}^{k-1} F(\omega_k^j)$$，其中 $F(x) = \Sigma\, a_i x^i$。

于是只要能在 k 个单位根处求值，就能一次抽出所有整除项。求「$i \equiv r \pmod k$」的项同理，把 ω^{ji} 换成 $\omega^{j(i-r)}$ 即可。它和 NTT 的关系：NTT 在 ω_n 的全部 n 个幂上求值，单位根反演是它的「部分求值」版本；循环卷积（长度 n 卷成长度 n）也靠它把溢出部分折回来。

可编译模板

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int MOD = 998244353, G = 3;
int N;
vector<ll> fac, ifac;

ll qpow(ll a, ll e) {
    ll r = 1; a %= MOD;
    while (e) { if (e & 1) r = r * a % MOD; a = a * a % MOD; e >>= 1; }
    return r;
}
void initFac(int n) {                       // 阶乘表，二项式系数 O(1)
    N = n; fac.assign(n + 1, 1); ifac.assign(n + 1, 1);
    for (int i = 1; i <= n; ++i) fac[i] = fac[i - 1] * i % MOD;
    ifac[n] = qpow(fac[n], MOD - 2);
    for (int i = n; i >= 1; --i) ifac[i - 1] = ifac[i] * i % MOD;
}
ll C(int n, int k) {
    if (k < 0 || k > n) return 0;
    return fac[n] * ifac[k] % MOD * ifac[n - k] % MOD;
}

// ① 二项式反演「恰好型」：f[k] = Σ_{i≥k} C(i,k) g[i]  ⟹  反解 g[k]
vector<ll> binomInv(const vector<ll>& f) {
    int n = (int)f.size() - 1;
    vector<ll> g(n + 1, 0);
    for (int k = 0; k <= n; ++k) {
        ll s = 0;
        for (int i = k; i <= n; ++i) {
            ll t = C(i, k) * f[i] % MOD;
            if ((i - k) & 1) s = (s - t + MOD) % MOD;
            else             s = (s + t) % MOD;
        }
        g[k] = s;
    }
    return g;
}

// ② 子集变换：zeta = 子集和；mobius = 其逆。n 是位数，f 长 2^n
void zeta(vector<ll>& f, int n) {
    for (int i = 0; i < n; ++i)
        for (int s = 0; s < (1 << n); ++s)
            if (s >> i & 1) f[s] = (f[s] + f[s ^ (1 << i)]) % MOD;
}
void mobius(vector<ll>& f, int n) {
    for (int i = 0; i < n; ++i)
        for (int s = 0; s < (1 << n); ++s)
            if (s >> i & 1) f[s] = (f[s] - f[s ^ (1 << i)] + MOD) % MOD;
}

// ③ 单位根反演：返回 Σ_{i: k|i} a[i]，要求 k | (MOD-1)
ll divIndexSum(const vector<ll>& a, int k) {
    ll wn = qpow(G, (MOD - 1) / k);         // 本原 k 次单位根
    ll s = 0, w = 1;
    for (int j = 0; j < k; ++j) {
        ll cur = 1, t = 0;                  // cur = ω^{j·i}
        for (int i = 0; i < (int)a.size(); ++i) {
            t = (t + a[i] * cur) % MOD;
            cur = cur * w % MOD;
        }
        s = (s + t) % MOD;
```

```
        w = w * wn % MOD;
    }
    return s * qpow(k, MOD - 2) % MOD;
}

int main() {
    // 错排自检：D(n) = Σ (-1)^i C(n,i) (n-i)!
    initFac(20);
    for (int n = 1; n <= 10; ++n) {
        ll d = 0;
        for (int i = 0; i <= n; ++i) {
            ll t = C(n, i) * fac[n - i] % MOD;
            d = ((i & 1) ? d - t + MOD : d + t) % MOD;
        }
        printf("D(%d)=%lld\n", n, d);
    }
    // 子集反演自检：g 随机，f = zeta(g)，再 mobius 应还原
    int n = 4; vector<ll> f(1 << n);
    for (int s = 0; s < (1 << n); ++s) f[s] = (s * 37 + 11) % MOD;
    vector<ll> g = f; zeta(g, n); mobius(g, n);
    bool ok = (g == f); printf("fmt roundtrip=%d\n", (int)ok);
    // 单位根反演自检：a[i]=i，k=4，Σ_{4|i} a[i] (i<16) = 0+4+8+12 = 24
    vector<ll> a(16); for (int i = 0; i < 16; ++i) a[i] = i;
    printf("divSum=%lld\n", divIndexSum(a, 4));
    return 0;
}
```

复杂度：二项式反演 $O(n^2)$（$n \le 5000$ 够用；再大就把 $C(i,k)$ 拆成 $i!/(k!(i-k)!)$，后半段是卷积，NTT 优化到 $O(n \log n)$）；子集反演朴素 $O(3^n)$、FMT $O(n2^n)$，$n = 20$ 时 2×10^7 次加法；单位根反演 $O(nk)$，k 通常很小（2、3、4），若 k 大且 n 大，就把每个 $F(\omega^j)$ 用 NTT 算，总复杂度 $O(k\ n \log n)$。

典型用法

- **「恰好 k 个」类计数：先写「钦定 k 个」的 $f(k)$（往往是一行组合数乘幂），再二项式反演拿 $g(k)$。凡是答案要求「恰好」，而正向枚举做不到、反向钦定很简单，就走这条。**
- **「每个元素至少出现一次」类：$f(S)$ = 只从 S 里选、允许空集；$g(S)$ = 恰好用满 S。一次 FMT + 一次 mobius 就得到 g，比逐个容斥快一个数量级。**
- **「下标为 k 的倍数」求和：直接套 $divIndexSum$。典型题：给定多项式，求所有次数为 3 的倍数的系数和；或「走 n 步、每步长度互质/同余」，用单位根把同余类分离。**
- **循环卷积：长度 n 的循环卷积 = 普通卷积后把 $i \ge n$ 的项折回 $i - n$；用单位根语言说就是「在 ω_n 处取值」，所以循环卷积直接对 n 个点做 NTT 即可，不必卷成 2n 再折。**

易错点：

- **$f(k)$ 写成「至多 k 个」。二项式反演要求 $f(k)$ 是「钦定 k 个」即 $\Sigma_{i\ge k} C(i,k)g(i)$；写成「至多」后上三角关系反转，反演公式完全不同（那是前缀和求逆，不是带符号反演）。这是本节第一大错源。**
- **组合数下标方向反。是 $C(i,k)$（从 i 个里选出被钦定的 k 个），不是 $C(n,k)$ 也不是 $C(k,i)$。写错后在 $n = k$ 的样例上照样对，一测多组就崩。**

- 子集反演符号指数写成 `|S|+|T|` 后又在别处写 `|S|-|T|`。两者奇偶性相同、结果等价，但同一个式子里混用会引入额外符号。统一用 `|S|-|T|`。
- FMT 只做了一部分维。变换必须对全部 n 个二进制位各做一轮；漏一位等于只在子格上求和，结果看起来「接近但不相等」。
- mobius 里忘了 `+MOD`。减法会出负数，`(f[s] - f[s^bit]) % MOD` 在 C++ 里是负数，后续乘法直接错。
- 单位根反演忘乘 `1/k`。答案恰好是正确值的 k 倍，且在所有样例上都是整数倍，很容易被误认为「公式差一个常数」。
- `k` 不整除 `MOD-1`。`998244353 - 1 = 2^23 × 7 × 17`，所以 `k` 只能是这些因子的乘积（常用 2 的幂、7、14、17、119）；`k = 3` 在这个模数下没有三次单位根。要 `k = 3` 就换模数（如 10^9+8 不行，`469762049` 有 7 次根），或扩域。
- ω^j 累乘写错。内层是 `cur *= ω^j`（固定 j，随 i 变），外层才是 `w *= ω`；把两者混在一起会让第二项之后全错。
- `a[i]` 为负或超过模数。单位根反演里 `a[i] * cur` 必须是模意义下的值，输入先 `((x % MOD) + MOD) % MOD`。
- 把 FMT 当 FWT 用。FMT 做的是「子集和」，FWT 做的是「异或/与/或卷积」；子集反演配 FMT，卷积题配 FWT，两者代码形似但语义不同，混用会得到无意义的结果。

自查：本轮把三个模板逐行译成 Python 后在 `998244353` 下对拍——① 二项式反演：随机 `g[0..n]`（$n \le 40$）按 $f(k)=\Sigma_{i\ge k}C(i,k)g(i)$ 正向聚合，再反演，`g` 逐项还原（300 组零错）；② 错排 `D(1..12)` 与递推 $D(n)=(n-1)(D(n-1)+D(n-2))$ 全等，满射数与「第二类斯特林数 × $m!$」在 $n,m \le 8$ 上全等；③ 子集反演：随机 `g`（$n \le 12$）做 zeta、mobius 往返，600 组全部还原，并用「恰好用满 `S`」的暴力枚举验证 `g` 的数值；④ 单位根反演：$k \in \{2,4,8,7,14\}$、`a` 长度 ≤ 20、元素随机，与「直接按 `i % k == 0` 累加」比对 500 组全等；另验证 $k \mid (MOD-1)$ 时 $\Sigma_j \omega^{jn}$ 在 $n \equiv 0$ 时恰为 k、否则恰为 0。

29.14 离散对数与原根：BSGS、exBSGS、指标与二次剩余

一句话判定：题面出现「求最小的 x 使 $a^x \equiv b \pmod m$」「$x^k \equiv b \pmod p$ 有几个解」「模 p 下 b 有没有平方根、平方根是多少」「把乘法转加法」，就是本节。它是 29.1 的逆元、29.2 的欧拉函数之后，数论里最后一类「反解指数」的通用工具；`m` 到 10^9 量级时暴力枚举必死，$\sqrt{m}$ 的 BSGS 是唯一可行解。

问题	判定条件	主算法	时间 / 空间
$a^x \equiv b \pmod m$，$\gcd(a,m)=1$	有解 ⇔ $b \in \langle a \rangle$（b 落在 a 生成的子群里）；有解则解在 $\bmod \mathrm{ord}_m(a)$ 下唯一，最小非负解 $< \mathrm{ord} \le \phi(m)$	BSGS（大步小步）	$O(\sqrt{m})$ 期望 / $O(\sqrt{m})$
同上但 $\gcd(a,m) \neq 1$	逐轮提 $d=\gcd(a,m)$；某轮 $d \nmid b$ 且 $b \neq 1$ 即无解	exBSGS（约 gcd 后转 BSGS）	$O(\sqrt{m} + \log^2 m)$ / $O(\sqrt{m})$
求模 p 的最小原根	原根存在 ⇔ $m \in \{2, 4, p^e, 2p^e\}$（p 奇素数）；判定 g：$g^{\phi/q} \not\equiv 1 \pmod m$ 对 ϕ 的每个不同质因子 q 成立	试除分解 ϕ + 从 2 起逐个试 g	$O(\sqrt{p} + r \cdot \omega(\phi) \cdot \log p)$ / $O(\omega(\phi))$
$x^2 \equiv n \pmod p$	p 奇素数；$n^{(p-1)/2} \equiv 1$（Euler 准则）⇔ n 是二次剩余，$n \neq 0$ 时恰有 2 个根 $\pm r$	Cipolla（扩域 $w^2 = a^2-n$）	期望 $O(\log p)$ / $O(1)$
$x^k \equiv b \pmod p$，g 为原根	令 $d=\gcd(k, p-1)$；$\mathrm{ind}(b) \bmod d = 0$ 时有 d 个解，否则无解	指标 $\mathrm{ind}(b)=\mathrm{BSGS}(g,b,p)$ + 线性同余	$O(\sqrt{p} + \log p)$ / $O(\sqrt{p})$

① BSGS 怎么把指数拆开。取 $t = \lceil\sqrt{m}\rceil$，把解写成 $x = i \cdot t - j$（$1 \le i \le t+1$，$0 \le j < t$）。因 $\gcd(a,m)=1$，两边可同乘 a^j：

$$a^{i \cdot t} \equiv b \cdot a^j \pmod m$$

预计算全部 $b \cdot a^j$ 存哈希表（O(t)），再从小到大枚举 $a^{i \cdot t}$（每步乘一次 a^t，O(t)）。任一 $x \le m$ 都能写成这个形状，所以命中即返回 $i \cdot t - j$。关键细节：同一个 key 可能对应多个 j（当 $\mathrm{ord}(a) < t$），而 $x = i \cdot t - j$ 中 j 越大 x 越小，所以插入时必须覆盖式保留最大的 j（或把 j 从 t−1 递减、只在 key 不存在时插入），再让 i 从小到大、命中且验算通过即返回——这样得到的才是最小非负解。

② exBSGS 怎么约掉不互质。若 $d = \gcd(a,m) > 1$，则 $x \ge 1$ 时 a^x 必被 d 整除，故 $d \nmid b$ 直接无解（$b=1$ 时 x=0 已单独返回）。否则同除 d：

$$(a/d) \cdot a^{x-1} \equiv b/d \pmod{m/d}$$

重复 k 轮直到 $\gcd(a, m_k)=1$，此时方程化为 $\mathrm{coef} \cdot a^{x-k} \equiv b' \pmod{m_k}$，其中 $\mathrm{coef} = \prod(a/d_i) \bmod m_k$、$b' = b / \prod d_i$（整数除）。因 a/d_i 与当时的模数互质，coef 可逆，于是 $a^{x-k} \equiv b' \cdot \mathrm{coef}^{-1}$ 交给 BSGS，答案加回 k。每轮都要检查 $\mathrm{coef} == b'$（对应 $x = k$），否则会漏掉被约分丢掉的小指数解。

③ **指标（离散对数）。若 g 是模 p 的原根，则** `ind(b) = BSGS(g,b,p)` **满足** $b \equiv g^{ind(b)}$，**且** $ind(xy) \equiv ind(x)+ind(y) \pmod{p-1}$ **——乘法变加法。于是** $x^k \equiv b$ **化为** $k \cdot ind(x) \equiv ind(b) \pmod{p-1}$：**设** `d=gcd(k,p-1)`，$d \nmid ind(b)$ **则无解，否则** $ind(x) \equiv (ind(b)/d) \cdot (k/d)^{-1} \pmod{(p-1)/d}$，**共 d 个解** $x = g^{ind(x)+j \cdot (p-1)/d}$。

④ **Cipolla。找一个 a 使** $W = a^2-n$ **是二次非剩余（用 Euler 准则判，期望两次命中），在扩域里取** $w^2 = W$，**则** $(a+w)^{(p+1)/2}$ **的虚部必为 0、实部是** $\sqrt{n}$。**复数乘法只需二维** `(x,y)`，**所有运算模 p。**

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128_t;

inline ll mul_mod(ll a, ll b, ll m) { return (ll)((i128)a * b % m); }

ll qpow(ll a, ll e, ll m) {
    a %= m; if (a < 0) a += m;
    ll r = 1 % m;
    while (e) { if (e & 1) r = mul_mod(r, a, m);
                a = mul_mod(a, a, m); e >>= 1; }
    return r;
}
ll exgcd(ll a, ll b, ll &x, ll &y) {
    if (!b) { x = 1; y = 0; return a; }
    ll x1, y1; ll g = exgcd(b, a % b, x1, y1);
    x = y1; y = x1 - (a / b) * y1; return g;
}
ll inv_mod(ll a, ll m) {
    ll x, y; if (exgcd(a % m, m, x, y) != 1) return -1;
    x %= m; if (x < 0) x += m; return x;
}

// (1) BSGS：a^x = b (mod m)，要求 gcd(a,m)=1；最小非负解，无解 -1
ll BSGS(ll a, ll b, ll m) {
    a %= m; if (a < 0) a += m;
    b %= m; if (b < 0) b += m;
    if (m == 1 || b == 1) return 0;
    ll t = (ll)ceill(sqrtl((long double)m));
    while ((i128)t * t < m) ++t;
    unordered_map<ll, ll> tab; tab.reserve(t * 2 + 8);
    ll cur = b;
    for (ll j = 0; j < t; ++j) {
        tab[cur] = j;                  // 覆盖式：保留最大 j → 最小 x
        cur = mul_mod(cur, a, m);
    }
    ll at = qpow(a, t, m), p = at;
    for (ll i = 1; i <= t + 1; ++i) {
        auto it = tab.find(p);
        if (it != tab.end()) {
            ll x = i * t - it->second;
            if (x >= 0 && qpow(a, x, m) == b) return x;
        }
        p = mul_mod(p, at, m);
    }
    return -1;
}

// (2) exBSGS：gcd(a,m) 任意
ll exBSGS(ll a, ll b, ll m) {
    a %= m; if (a < 0) a += m;
    b %= m; if (b < 0) b += m;
    if (m == 1 || b == 1) return 0;
    ll cnt = 0, coef = 1 % m;
    while (true) {
        ll d = std::gcd(a, m);
        if (d == 1) break;
        if (b % d) return -1;
        m /= d; b /= d; ++cnt;
        coef = mul_mod(coef, a / d, m);   // 注意：在 m 缩小之后取模
        if (coef == b) return cnt;        // x = cnt 是一个解
```

```
    }
    ll ic = inv_mod(coef, m);
    if (ic < 0) return -1;
    ll x0 = BSGS(a, mul_mod(b, ic, m), m);
    return x0 < 0 ? -1 : x0 + cnt;
}

// (3) 最小原根（p 为奇素数；p=2 返回 1，p=4 返回 3）
ll primitiveRoot(ll p) {
    if (p == 2) return 1;
    if (p == 4) return 3;
    ll phi = p - 1, n = phi; vector<ll> fac;
    for (ll d = 2; d * d <= n; ++d) if (n % d == 0) {
        fac.push_back(d); while (n % d == 0) n /= d;
    }
    if (n > 1) fac.push_back(n);
    for (ll g = 2; ; ++g) {
        bool ok = true;
        for (ll q : fac) if (qpow(g, phi / q, p) == 1) { ok = false; break; }
        if (ok) return g;
    }
}

// (4) Cipolla：x^2 = n (mod p)，p 奇素数；返回较小根，无解 -1
ll W, P;
struct C { ll x, y; };                        // x + y·w，w^2 = W
C cmul(const C &u, const C &v) {
    return C{(ll)((mul_mod(u.x, v.x, P) + mul_mod(mul_mod(u.y, v.y, P), W % P, P)) % P),
             (ll)((mul_mod(u.x, v.y, P) + mul_mod(u.y, v.x, P)) % P)};
}
C cpow(C b, ll e) {
    C r{1 % P, 0};
    while (e) { if (e & 1) r = cmul(r, b); b = cmul(b, b); e >>= 1; }
    return r;
}
ll cipolla(ll n, ll p) {
    n %= p; if (n < 0) n += p;
    if (p == 2) return n;
    if (n == 0) return 0;
    if (qpow(n, (p - 1) / 2, p) != 1) return -1;
    ll a = 1;
    while (qpow((ll)(((i128)a * a - n + p) % p), (p - 1) / 2, p) == 1) ++a;
    W = (ll)(((i128)a * a - n + p) % p); P = p;
    ll r = cpow(C{a % p, 1}, (p + 1) / 2).x;
    return min(r, (p - r) % p);
}
```

易错点（本轮逐条实测）：

1. **哈希表保留最大 j，不是最小 j。第一版写成「key 不存在才插入」以保留最小 j，在 $\mathrm{ord}_m(a) < t$ 时会漏掉更优的 j：m ≤ 120 全量 351176 组里错 2956 组，最小反例 `a=4,b=4,m=5` 返回 3 而正解是 1。原因是 `x = i·t − j` 里 j 越大 x 越小。**
2. **`x = 0` 必须提前返回（`b ≡ 1` 或 `m = 1`）；BSGS 的 i 从 1 起，永远给不出 0。**
3. **i 的枚举范围是 `1..t+1` 而不是 `0..t-1`；写成后者会让 `x = i·t − j` 出现负数，且覆盖不到 $[t^2, m)$ 段的解。命中后仍要用一次快速幂验算。**

4. 所有乘法走 `__int128`：m 到 10^{18} 时 `a·b` 必溢出 `long long`；`qpow` 的指数用 `long long`（`p-1` 量级）。
5. exBSGS 的 `coef` 要在模数缩小之后取模，每次累积的是 `a/d` 而不是 `a`；最后一轮 BSGS 传入的 `a` 仍是原始 a（从未被除过）。
6. exBSGS 每轮都要检查 `coef == b'`（对应 `x = cnt`）；只靠开头的 `b==1` 会漏掉 `x < k` 的解。约分终止条件是 `gcd(a, 当前 m)=1`，不是原模数。
7. 原根只对 `2, 4, p^e, 2p^e` 存在；`m=2` 的原根是 1、`m=4` 是 3，从 `g=2` 起试会死循环或直接判错。
8. 原根判定必须试 ϕ 的每一个不同质因子 q，条件是 $g^{\phi/q} \equiv /1$；只验 $g^{\phi} \equiv 1$ 是费马小定理，对所有与 p 互质的 g 都成立，等于没验。
9. Cipolla 的 $W = a^2-n$ 必须是二次非剩余，判据用 Euler 准则；若恰好 $a^2 \equiv n$ 则 `W=0`，ω 退化为 0，结果仍是 a，不必特判但不能把 W=0 当异常丢弃。扩域乘法里 `W` 也要先约到 `[0,p)`。
10. Euler 准则 $n^{(p-1)/2} \equiv 1$ 只对奇素数 p 成立；合数模用它判二次剩余必然误判（如 mod 8）。`n=0` 与 `p=2` 单独处理，返回的两个根是 `r` 与 `p-r`。
11. `unordered_map` 在构造数据下可被卡到 $O(n^2)$，正式比赛建议手写开放寻址哈希或换 `std::map`（多一个 log 但稳定）；`reserve` 只能缓解不能根治。
12. 指标法混用了两个模：`ind` 的等式在 mod (p−1) 下成立，底数与结果在 mod p 下。`k/d` 的逆元要对 `(p-1)/d` 取，写成对 `p-1` 取会漏解；解数恰为 `d=gcd(k,p-1)`。

校验：本机无 `g++`，按模板逐行等价复现成 Python 与暴力枚举对拍——BSGS（gcd=1）m ≤ 120 全量 351176 组、exBSGS（gcd 任意）m ≤ 120 全量 583219 组、大模数 120 组（m ≤ 30000）全部 0 错；最小原根 p < 300 逐个验阶 = p−1、Cipolla p < 200 全量 4225 组与逐点平方枚举一致，均 0 错。C++ 侧只做语法与上界自查（`__int128` 贯穿、`tab.reserve(2t+8)`、`i ≤ t+1` 覆盖 `x ≤ m+t`）。

29.15 拉格朗日插值与自然数幂和：用 k+2 个点算出第 n 项

问题形态：题面说「f 是次数不超过 n 的多项式」，或者干脆只给前若干项，要求第 k 项（k 到 10^9、10^{18}）。另一类更隐蔽：某计数题的答案被证明是「关于 k 的 d 次多项式」，于是每个 k 都要问一次。这两类都不必推通项——只要能把「第 i 项」算出来，就能把「第 k 项」插出来。

结论（拉格朗日插值）：给定 n+1 个互不相同的 x_i 与对应的 y_i，存在唯一一个次数 ≤ n 的多项式穿过它们，且

$$f(x) = \Sigma_{i=0..n}\ y_i \cdot \Pi_{j\neq i}\ (x - x_j) \,/\, (x_i - x_j)$$

唯一性来自一句老话：两个次数 ≤ n 的多项式若在 n+1 个点上相等，它们的差有 n+1 个根，而次数 ≤ n，只能恒为零。这条唯一性是所有「插值代替推式子」做法的合法性来源——你不需要证明自己构造的式子对不对，只需要确认「它确实是那个多项式」。

连续点：把 $O(n^2)$ 压到 $O(n)$。取 $x_i = i$（i = 0..n）时，分母化成 $\Pi_{j\neq i}(i-j) = i! \cdot (-1)^{n-i} \cdot (n-i)!$，分子拆成前缀积与后缀积：

$$pre[i] = \prod_{j<i}(x - j),\quad suf[i] = \prod_{j\ge i}(x - j)$$

$$f(x) = \sum_{i=0..n} y_i \cdot pre[i] \cdot suf[i+1] \cdot invfact[i] \cdot invfact[n-i] \cdot (-1)^{n-i}$$

预处理阶乘与阶乘逆元 O(n)，单次求值 O(n)。这是考场上 99% 的用法：只要能把每个 i = 0..n 的答案用暴力算出来，就能插。整节的技术含量其实只有两个下标变换：`suf[i+1]`（跳过 i 自己）与 $(-1)^{n-i}$（分母的符号）。

自然数幂和：$S_k(n) = 1^k + 2^k + \dots + n^k$ 是 n 的 k+1 次多项式。理由一行就够：一阶差分 $S_k(n) - S_k(n-1) = n^k$ 是 k 次多项式，而「差分降一次」反过来就是「求和升一次」。于是只要求出 $S_k(0..k+1)$ 这 k+2 个点，就能 O(k) 回答任意 n。$k \le 10^5$、$n \le 10^{18}$ 的幂和题，从此不必再写 k+2 阶矩阵快速幂（$O(k^3 \log n)$）。

模板一：连续点插值（`y[i] = f(i)`，i = 0..n，返回 f(k)）。

```
#include <cstdio>
#include <vector>
using namespace std;
typedef long long ll;
const int MOD = 998244353;

ll qpow(ll a, ll b) { ll r = 1; for (; b; b >>= 1, a = a * a % MOD) if (b & 1) r = r * a % MOD; return r; }

// y[i] = f(i)，i = 0..n（n 为多项式次数）；返回 f(k) mod MOD，k >= 0
ll lagrange(const vector<ll> &y, ll k) {
    int n = (int)y.size() - 1;
    k %= MOD;
    if (k <= n) return y[k];                    // 落在已知点上，直接返回（见易错点 3）
    vector<ll> fact(n + 2), invf(n + 2), pre(n + 2), suf(n + 2);
    fact[0] = 1;
    for (int i = 1; i <= n + 1; i++) fact[i] = fact[i - 1] * i % MOD;
    invf[n + 1] = qpow(fact[n + 1], MOD - 2);
    for (int i = n + 1; i >= 1; i--) invf[i - 1] = invf[i] * i % MOD;
    pre[0] = 1;
    for (int i = 0; i <= n; i++) pre[i + 1] = pre[i] * ((k - i + MOD) % MOD) % MOD;
    suf[n + 1] = 1;
    for (int i = n; i >= 0; i--) suf[i] = suf[i + 1] * ((k - i + MOD) % MOD) % MOD;
    ll res = 0;
    for (int i = 0; i <= n; i++) {
        ll t = y[i] * pre[i] % MOD * suf[i + 1] % MOD * invf[i] % MOD * invf[n - i] % MOD;
        if ((n - i) & 1) res = (res - t + MOD) % MOD;   // (-1)^(n-i)
        else             res = (res + t) % MOD;
    }
    return res;
}
```

模板二：自然数幂和（k 固定、n 巨大）。

```
// 返回 1^k + 2^k + ... + n^k (mod MOD)，n 可达 1e18，k 可达 1e5
ll powerSum(ll n, int k) {
    if (n <= k + 1) {                          // 点数不够，直接累加
        ll s = 0;
        for (ll i = 1; i <= n; i++) s = (s + qpow(i, k)) % MOD;
        return s;
    }
    vector<ll> y(k + 2);                       // k+2 个点：0..k+1
    y[0] = 0;
    for (int i = 1; i <= k + 1; i++) y[i] = (y[i - 1] + qpow(i, k)) % MOD;
    return lagrange(y, n % MOD);               // k+1 次多项式，用 k+2 个点
}
```

把 `n % MOD` 传进去是合法的，不是偷懒：$S_k(n)$ 作为 n 的多项式，其系数在模 MOD 下有意义，因此值只依赖 `n mod MOD`（前提 MOD 大于 k+2，保证阶乘可逆；998244353 对 $k \leqslant 10^5$ 满足）。这也是模板一里那句 `k %= MOD` 的由来。

点不连续时：重心形式。若题目给的是任意 x_i（比如树上节点编号），把公式整理成

$$f(x) = (\Pi_j(x - x_j)) \cdot \Sigma_i \, w_i / (x - x_i)，\text{其中 } w_i = y_i / \Pi_{j \neq i}(x_i - x_j)$$

则单点求值 O(n)、预处理 $O(n^2)$，且每个 w_i 只依赖点集本身——加减一个点时其余权重只需 O(n) 更新，适合「点集动态、查询多次」的场合。$n \leqslant 3000$ 时它比「每次重算全部 Π」快一个常数级。

题型	次数怎么定	要点
题面直接说「f 是 n 次多项式」，求 f(k)	已知 n	模板一；$k \leqslant n$ 时直接查表
$\Sigma_{i=1..n} i^k$	k + 1	模板二；k+2 个点
$\Sigma_{i=1..n} C(i, m)$ 一类组合和	m + 1	先算前 m+2 项，再插
「答案是关于 k 的 d 次多项式」，多组询问 k	打表求差分定阶	第 d 阶差分变常数 → d 次
矩阵快速幂题（k 阶线性递推、n 巨大）	一般不是多项式	特征根不全为 1 时插值无效，仍用矩阵
树上路径长度恰好为 k 的点对计数	随 k 增长可能是分段多项式	分段插，每段单独取点

易错点

- **点数少一个。n 次多项式要 n+1 个点，写成「取 n 个点」得到的是 n−1 次的那条曲线，除个别点外处处不等。实测 400 组随机多项式（次数 $\leqslant 12$，$k \in [n+5, 5000]$）400 组全错。幂和同理：它是 k+1 次，要 k+2 个点，只取 k+1 个点实测 10 组全错。**
- **漏掉 $(-1)^{n-i}$。这个符号来自分母 $\Pi_{j \neq i}(i-j)$：比 i 大的 j 贡献负号，共 $n-i$ 个。实测 400 组全错，且错误结果仍是「像模像样」的整数，不写对拍根本发现不了。**

- 删掉 `k ≤ n` 的早返回。这一句不只是优化：当模数小于等于点数（MOD ⩽ n+1）时，`fact[n+1] ≡ 0`，阶乘逆元全部失效，公式退化成 0。实测去掉早返回后，p = 5 / 7 / 11 三档各跑 200 组，分别错 170 / 170 / 183 组。反直觉的是：保留早返回后，即使 p 只有 5（点数 5 个，阶乘在模 5 下为 0）也一组不错（p = 5 / 7 / 11 / 13 各 300 组，0 错）——因为 k 先对 p 取模，结果必然落在 [0, n] 内被早返回接住，而「f(k) mod p 只依赖 k mod p」本身就是多项式恒等式。这条性质连续点插值独有，换任意点插值就不成立。
- 把幂和的次数当成 k。它是 k+1 次：$S_0(n) = n$ 是一次，$S_1(n) = n(n+1)/2$ 是二次。写模板时 `y` 的长度必须是 k+2，少了必错、多了不亏（多点插同一个多项式仍然一致）。
- k 不归一化。`k` 可能是负数（坐标系题）或超过 MOD。`k %= MOD` 之后再算 `k - i`，仍可能为负，要写 `(k - i + MOD) % MOD`。`k - i + MOD` 的取值范围是 (0, 2MOD)，一次加法就够，不必再取两次模。
- 把「递推」直接当「多项式」。k 阶常系数线性递推的通项是「幂和 × 多项式」的混合，只有当特征根全为 1（等价于 f 的 k 阶差分为常数）时才是多项式。判据：连续求差分，若第 d 阶差分出现非零常数，则 f 是 d 次多项式；若差分呈指数增长，插值一定错。
- n 很小时硬插。当 `n ≤ k+1` 时插值公式的 `y` 根本凑不齐（点数不够），模板二里第一句 `if (n ≤ k+1)` 的暴力分支不能省；同理，模板一在 `k ≤ n` 时直接返回 `y[k]`，比走一遍公式既快又稳。
- 拿不连续的点硬套连续公式。连续点公式的分母是 `i!·(n−i)!`，只对 $x_i = i$ 成立；节点编号是 1..n 时要先平移成 0..n−1，稠密但跳号的点集必须回到 O(n^2) 版或重心形式。
- k 到 10^6 仍逐个快速幂。模板二里每个 i^k 一次 `qpow`，共 O(k log k)；k 上到 10^6 时改用线性筛预处理 i^k（i^k 是完全积性函数，$(p\cdot m)^k = p^k\cdot m^k$），降到 O(k)。

自查：模板一逐行译成 Python 与「先构造随机多项式、再逐点求值」对拍：400 组（次数 ⩽ 12，k ⩽ 5000）全对；再取 k ≈ 10^9—2×10^9 的 200 组全对。幂和与两条独立算法对拍：小规模（K ⩽ 6、N ⩽ 1000）24 组；大规模（K ⩽ 7、N 取 10^9+7 / 123456789 / 2×10^9）与「斯特林数展开 Σ_j `S(k,j)·j!·C(n+1, j+1)`」对比 24 组，全对（该对照式在 K = 0 时要减去 i = 0 那一项，第一版漏了它，三条 K = 0 用例全报错——错的又是对照式本身）。上面四条错法的实际数据：少一个点 400/400 错、漏符号 400/400 错、去掉早返回在 p ⩽ n 时错 170/170/183（各 200 组）、幂和少一个点 10/10 错。本机无 `g++`，C++ 侧只做了语法与范围自查（`pre/suf` 长度 n+2、下标最大 `n+1`、`invf` 由 `fact[n+1]` 逆推）。

与相邻几节的分工：29.7 与 29.10 处理「多项式本身」（乘法、求逆、exp/ln，O(n log n)）；本节处理「只有一个多项式的值」——不需要整套多项式运算，O(n) 就能出答案，常数小得多。29.5 的矩阵快速幂与本节是替代关系而非并列关系：递推的特征根全为 1 时用插值（O(k)），否则必须回到矩阵（O(k^3 log n)）。多组询问同一点集时，把 `pre/suf` 换成重心形式的权重，O(n) 预处理后可反复用。

30 第 30 章　字符串

哈希能解决九成字符串题，剩下的用 KMP、Trie、AC 自动机；再往上才是 SA 与 SAM。本章按这个梯度排列，每个结构都给出可编译模板与典型问法；中段把哈希从序列推广到树（树同构与换根判定），并用回文自动机把「所有本质不同回文」压成一棵树、配 series link 做回文划分；30.10 转向「循环移位」这一类问题——最小表示法与 Lyndon 分解共用一条失配即整段排除的双指针，O(n) 解决循环同构与最小标号；30.11 补上 30.6 概览留下的空白，把 SAM 读成 endpos 等价类加 link 树，给出出现次数、第 k 小子串、最长公共子串与多串广义 SAM 的完整写法；30.12 从同一处空白出发走另一条路，把「任意两后缀 LCP = height 区间最小值」这条恒等式讲到能用：倍增构建、ST 表、二分分组、字典序第 k 子串与单调栈求和五套代码；30.13 回到 30.4 留下的那句「AC 自动机 + DP」，把它拆成四种问法：不含禁用词的计数、n 极大时的矩阵快速幂、关键词必须全部出现的状压、以及在 fail 树上自底向上汇总的出现次数——四种问法共用同一个前提：终止标记必须沿 fail 下传。

30.1 字符串哈希

哈希把字符串映射成整数，使「判断两个子串是否相等」从 O(len) 降到 O(1)。竞赛中 90% 的字符串题可以用哈希水过。

```
typedef unsigned long long ull;
const ull B = 131;                // 或 13331
ull h[MAXN], pw[MAXN];
void init(const string& s) {      // 下标从 1 开始
    pw[0] = 1;
    for (int i = 1; i <= (int)s.size(); ++i) {
        h[i] = h[i-1] * B + s[i-1];
        pw[i] = pw[i-1] * B;
    }
}
ull get(int l, int r) { return h[r] - h[l-1] * pw[r-l+1]; }   // 无符号自然溢出
```

防卡：单模哈希会被构造数据卡掉（Codeforces 上有成熟的 hack 手段）。两种加固：用 `unsigned long long` 自然溢出 + 随机 base；或双模（10^9+7 与 10^9+9）比较两个值。

常见用法：判断回文（正反两遍哈希）、最长公共子串（二分长度 + 哈希集合）、字符串匹配（模式串哈希在文本上滑动）、去重（把哈希塞进 set）。

注意：哈希是概率算法，理论上可能冲突。NOI 级别的题若明确卡哈希，必须换 SA/SAM。

30.2 KMP 与 Z 函数

KMP 在 O(n+m) 内求「模式串在文本中出现的所有位置」，核心是 `next`（前缀函数）数组：`next[i]` 表示 `s[1..i]` 的最长相等前后缀长度。

```
vector<int> getNext(const string& p) {      // p 下标从 0
    int m = p.size();
    vector<int> nxt(m, 0);
    for (int i = 1, j = 0; i < m; ++i) {
        while (j && p[i] != p[j]) j = nxt[j-1];
        if (p[i] == p[j]) ++j;
        nxt[i] = j;
    }
    return nxt;
}
vector<int> kmp(const string& s, const string& p) {
    vector<int> nxt = getNext(p), res;
    for (int i = 0, j = 0; i < (int)s.size(); ++i) {
        while (j && s[i] != p[j]) j = nxt[j-1];
        if (s[i] == p[j]) ++j;
        if (j == (int)p.size()) { res.push_back(i - j + 1); j = nxt[j-1]; }
    }
    return res;
}
```

next 数组的三个衍生用途：求最小循环节（`len - next[len-1]` 若能整除 len 则为循环节）、求所有 border（沿 next 链跳）、统计每个前缀出现次数（从后往前累加）。

Z 函数：`z[i]` 表示 `s` 与 `s[i..]` 的最长公共前缀长度，同样 O(n)。求法与 KMP 类似，用于「每个后缀与原串的匹配长度」。

30.3 Trie 与 01-Trie

Trie 把字符串按字符逐层存进树，支持插入、查询、前缀统计，复杂度 O(L)。

```
int ch[MAXN][26], cnt[MAXN], tot = 0;
void insert(const string& s) {
    int p = 0;
    for (char c : s) {
        int &q = ch[p][c - 'a'];
        if (!q) q = ++tot;
        p = q; ++cnt[p];
    }
}
int query(const string& s) {          // 返回以 s 为前缀的串数
    int p = 0;
    for (char c : s) { p = ch[p][c - 'a']; if (!p) return 0; }
    return cnt[p];
}
```

01-Trie：把整数按二进制从高到低位插入，用于「最大异或对」「区间异或最值」。查询时贪心走相反位。`MAXN` 要开到 `n × 位数`，开小了必 RE。

可持久化 Trie：每个版本继承上一版本，配合前缀和可求「区间内与 x 异或的最大值」，是「最大异或和」类问题的标准解法。

30.4 AC 自动机

AC 自动机 = Trie + 失配指针（fail），解决「多个模式串同时在一个文本中匹配」。

```
void build() {
    queue<int> q;
    for (int c = 0; c < 26; ++c)
        if (ch[0][c]) q.push(ch[0][c]);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int c = 0; c < 26; ++c) {
            int v = ch[u][c];
            if (v) { fail[v] = ch[fail[u]][c]; q.push(v); }
            else ch[u][c] = ch[fail[u]][c];      // 路径压缩，省掉跳 fail 的时间
        }
    }
}
```

fail 树的用法：所有 fail 指针构成一棵树，一个节点被匹配到，意味着它到根路径上的所有节点都被匹配。因此「统计每个模式串出现次数」= 在 fail 树上做子树求和。

复杂度：建树 O(总长×字符集)，匹配 O(文本长 + 匹配次数)。字符集 26 时空间是 26×节点数，1e5 个节点要 260 万个 int，注意内存。

AC 自动机 + DP：在自动机上跑 DP（`f[i][j]` 表示文本长 i、状态 j），用于「长度 n 且不含任何模式串的串有多少个」。

30.5 Manacher 与回文

Manacher 在 O(n) 内求每个中心的最长回文半径。技巧是插入分隔符，把奇偶回文统一处理。

```
string manacher(const string& s) {
    string t = "^#";
    for (char c : s) { t += c; t += '#'; }
    t += '$';
    int n = t.size(), C = 0, R = 0, best = 0, pos = 0;
    vector<int> p(n, 0);
    for (int i = 1; i < n - 1; ++i) {
        p[i] = (R > i) ? min(R - i, p[2 * C - i]) : 1;
        while (t[i + p[i]] == t[i - p[i]]) ++p[i];
        if (i + p[i] > R) { C = i; R = i + p[i]; }
        if (p[i] > best) { best = p[i]; pos = i; }
    }
    int st = (pos - best) / 2, len = best - 1;
    return s.substr(st, len);
}
```

回文的三种问法：最长回文子串（Manacher）、回文子串个数（`Σ p[i]/2`）、最少分割次数（DP + 回文判定，配合 Manacher 预处理 $O(n^2)$ 或哈希 O(n log n)）。

回文自动机（PAM）：在线加入字符，维护所有本质不同的回文串，节点数 O(n)。用于「本质不同回文串个数」「每个回文串出现次数」。

30.6 后缀数组与后缀自动机（概览）

这两个结构是字符串题的「重型武器」，省选以上必须会其一。

后缀数组（SA）：把 s 的所有后缀排序，得到 `sa[i]`（第 i 小的后缀起点）与 `rk[i]`（后缀 i 的排名），以及 `height[i] = LCP(sa[i-1], sa[i])`。倍增实现 O(n log n)，SA-IS 可到 O(n)。

```
// height 数组（Kasai）：O(n)
for (int i = 1, k = 0; i <= n; ++i) {
    if (rk[i] == 1) { k = 0; continue; }
    int j = sa[rk[i] - 1];
    while (i + k <= n && j + k <= n && s[i+k] == s[j+k]) ++k;
    height[rk[i]] = k;
    if (k) --k;
}
```

SA 能解决的问题：最长重复子串（height 最大值）、本质不同子串数（`n(n+1)/2 - Σheight`）、两个串的最长公共子串（拼接后求 height）、多串公共子串（二分 + 分组）。

后缀自动机（SAM）：状态数 ⩽ 2n-1，边数 ⩽ 3n-4，在线构建 O(n)。每个状态代表一组 endpos 相同的子串，`len` 为最长长度，`link` 指向后缀链接。

怎么选：只需要「子串出现次数 / 最长公共子串」用 SAM；需要「按字典序第 k 小子串 / 与排名相关的统计」用 SA。两者可以互相替代，但代码量都大，建议赛前各背一份模板。

30.7 周期、Border 与 fail 树

「最短循环节」「所有 border」「每个前缀在原串中出现几次」看起来是三个问题，其实共用同一个数组：前缀函数 π（即 KMP 的 next）。把它们收成一套工具，赛场上就不必每次重推。

两条等价（判定条件）：长度 p 是 s 的周期（即对所有 `i ≤ n-p` 有 `s[i] = s[i+p]`） ⇔ 长度 `n-p` 是 s 的 border（既是前缀又是后缀）。由此得最小周期 `p_min = n - π[n-1]`；若 `p_min | n`，则 s 恰好由 `n/p_min` 个相同块拼成（整周期）；否则 s 的最小整周期只能是 n 本身。

所有周期：对 border 链上的每个长度 b（`π[n-1], π[π[n-1]-1], …`），`n-b` 就是一个周期，再加上 n。链长最坏 O(n)（全同字符），单次枚举 O(链长)；若要按「第 k 短周期」反复询问，用倍增在 fail 树上跳。

周期引理（Fine–Wilf）：p、q 都是 s 的周期且 `p + q ≤ n + gcd(p,q)`，则 `gcd(p,q)` 也是周期。实用推论：`p + q ≤ n` 时 gcd(p,q) 必为周期。判断「两个周期能否同时存在」「合并周期约束」时直接用，不必构造。

fail 树：把 i 的父亲设为 `π[i-1]`（i = 1..n），由于 π[i-1] < i，节点编号本身就是拓扑序。前缀 `s[0..i-1]` 在原串中的出现次数 = 它在 fail 树上的子树大小，因为「某个位置的后缀等于该前缀」当且仅当该位置沿 fail 链能跳到 i。

```
int pi[MAXN], cnt[MAXN];                // s 下标 0..n-1

void build(const string& s) {
    int n = s.size();
    pi[0] = 0;
    for (int i = 1; i < n; ++i) {
        int j = pi[i-1];
        while (j && s[i] != s[j]) j = pi[j-1];   // 先回跳，再比较
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    for (int i = 1; i <= n; ++i) cnt[i] = 1;
    for (int i = n; i >= 1; --i) cnt[pi[i-1]] += cnt[i];   // 逆序合并到父亲
}

int minPeriod(int n) { return n - pi[n-1]; }              // 最小周期（不必整除 n）
bool isPower(int n)  { int p = minPeriod(n); return n % p == 0; }   // 是否整周期

// 所有周期，n 与 n-border 全部列出（降序）
void allPeriods(int n, vector<int>& out) {
    out.clear(); out.push_back(n);
    for (int b = pi[n-1]; b; b = pi[b-1]) out.push_back(n - b);
}
```

复杂度：前缀函数 O(n)（均摊，内层 while 的总回跳次数不超过 n）；fail 树统计 O(n)；枚举全部 border O(#border) ⩽ O(n)。全部是线性，常数极小。

用途对照：最短循环节 → `n - π[n-1]`；判断「串是否为某个串重复 k 次」→ `k | n` 且 `π[n-1] ≥ n - k`；统计每个前缀出现次数 → fail 树子树和；求最长的「前缀 = 后缀且长度 ⩽ n/2」→ 从 π[n-1] 沿链下跳到第一个 ⩽ n/2；把串按周期补齐成无限循环串做匹配 → 复制一遍 s 再跑 KMP。

易错点：① `while` 里必须先 `j = pi[j-1]` 再比较，写成「先比较、失败后不回跳」会退化成 $O(n^2)$；② `π[n-1]` 只是最长 border，不是最小周期本身，问「整周期」一定要做整除判断；③ 全同串有 n 个 border，若在每个位置都重新跳链就是 $O(n^2)$，必须先建 fail 树一次算完；④ `cnt[0]` 是「空前缀」的计数，输出时不能算进去；⑤ 下标从 1 开始时 `π[0] = 0` 是哨兵，fail 树数组要整体错一位，混用会静默算错；⑥ 多测要清 `pi[0]` 与 `cnt` 数组，只清前 n 位。

30.8 树哈希与树同构判定

结论：有根树同构 ⇔ 根的两棵子树集合（连同各自的结构）完全相同，归纳即得「哈希根的子树多重集即可」。所以树哈希的本质是：把子树的哈希做成一个无序多重集，再折叠成一个数。无根树没有指定根，先取重心（1 个或 2 个）：重心在任意标号下都是同构不变量，两个重心时取「以各自为根算出的哈希」的较小值作为规范形。

判定条件：题目问「两棵树是否同构」「以每个点为根是否同构」「本质不同的子树个数」，或需要给一棵树做与标号无关的指纹（用于哈希表去重、随机化构造）。

复杂度：单棵树 O(n log n)（每个点对子节点的哈希排序）；求全部 n 个点为根的哈希，排序版 $O(n \log^2 n)$，加法合并版 O(n)。

做法一：排序折叠（推荐，碰撞率最低）。先 splitmix64 把值打散，避免乘法折叠的结构被构造数据利用；子节点哈希排序后按 `h = h·B + x + 1` 折叠。求和那两个 +1 很关键：它让「空子树」与「哈希恰为 0 的子树」区分开。

```
typedef unsigned long long ull;
ull mix(ull x) {                        // splitmix64：把输入彻底打散
    x += 0x9e3779b97f4a7c15ULL;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}
const ull B = 0x9e3779b97f4a7c15ULL;    // 乘数取奇数即可

int n; vector<int> g[MAXN];
int par[MAXN]; ull h[MAXN];

// 迭代剥叶求重心：剩下的 1 或 2 个点就是重心
vector<int> centers() {
    vector<int> d(n + 1), q, nxt;
    for (int u = 1; u <= n; u++) { d[u] = (int)g[u].size(); if (d[u] <= 1) q.push_back(u); }
    int rest = n;
    while (rest > 2) {
        nxt.clear();
        for (int u : q) {
            rest--;
            for (int v : g[u]) if (--d[v] == 1) nxt.push_back(v);
        }
        q.swap(nxt);
    }
    return q;                           // 1 个或 2 个
}

// 以 root 为根求整棵树哈希（迭代，避免链状数据爆栈）
ull hash_rooted(int root) {
    vector<int> order; order.reserve(n);
    order.push_back(root); par[root] = 0;
    for (size_t i = 0; i < order.size(); i++) {
        int u = order[i];
        for (int v : g[u]) if (v != par[u]) { par[v] = u; order.push_back(v); }
    }
    for (int i = (int)order.size() - 1; i >= 0; i--) {
        int u = order[i];
        vector<ull> c;
        for (int v : g[u]) if (v != par[u]) c.push_back(h[v]);
        sort(c.begin(), c.end());
        ull val = 1;
        for (ull x : c) val = val * B + x + 1;
        h[u] = mix(val);
    }
    return h[root];
}

ull tree_hash() {
    vector<int> c = centers();
    ull r = hash_rooted(c[0]);
    if (c.size() == 2) r = min(r, hash_rooted(c[1]));   // 双重心必须都算
    return r;
}
```

做法二：换根求「每个点为根」的哈希。把折叠换成与顺序无关的加法合并 `h(u) = mix(1 + Σ mix(h_child))`，并约定 `mix(0) = 0` 表示空。这样剔除一个子树只需减去它的贡献，两遍 DFS 即可得到全部 n 个答案，同时天然支持「以 u 为根时某棵子树是否与给定树同构」。

```
ull down[MAXN], up[MAXN], sum[MAXN], H[MAXN];
ull f(ull x) { return x ? mix(x) : 0ULL; }   // 约定：空子树贡献 0

// 任取 1 为根，先得到拓扑序 order 与父指针 par
vector<int> order;
void build_order(int root) {
    order.clear(); order.push_back(root); par[root] = 0;
    for (size_t i = 0; i < order.size(); i++) {
        int u = order[i];
        for (int v : g[u]) if (v != par[u]) { par[v] = u; order.push_back(v); }
    }
}

void dfs_down(const vector<int>& order) {
    for (int i = (int)order.size() - 1; i >= 0; i--) {
        int u = order[i]; ull s = 0;
        for (int v : g[u]) if (v != par[u]) s += f(down[v]);
        sum[u] = s; down[u] = mix(1 + s);
    }
}

void dfs_up(const vector<int>& order) {
    up[order[0]] = 0;
    for (size_t i = 0; i < order.size(); i++) {
        int u = order[i];
        for (int v : g[u]) if (v != par[u])
            up[v] = mix(1 + f(up[u]) + sum[u] - f(down[v]));   // 无符号减法自动回绕
    }
    for (int u = 1; u <= n; u++) H[u] = mix(1 + f(up[u]) + sum[u]);
}
```

精确判定（对拍用）：AHU 算法把子树递归写成括号串 `"(" + 排序后的儿子串拼接 + ")"`，无根树取重心为根后取字典序最小者，得到 O(n log n) 的零碰撞规范形。哈希模板有理论碰撞概率，写题时用 AHU 在小数据上对拍，比手工构造更能查出「两重心只取一个」「漏 mix」这类错误。

易错点：① 用子树大小或度数序列当哈希必错——同 size 序列不同构的树大量存在（例如两条长度不同的链各接一个三叉点）；② 单模或自然溢出的「乘加哈希」会被构造数据卡，必须至少叠一层 splitmix64，且折叠前排序；③ 双重心只取一个会漏：同一棵树换标号后 centers() 的返回顺序可能不同，必须两个都算再取 min；④ 有序折叠（排序后乘加）与无序合并（加法）不能混用，同一个程序里两种哈希互相比较必然全错；⑤ 空子树哈希要约定为 0 且 mix(0) = 0，否则「一个儿子哈希恰为 0」与「没有儿子」无法区分；⑥ n = 10^5 的链递归 DFS 会爆栈（NOI Linux 默认 8 MB），上面的模板已全部改成迭代，若自己写递归记得加栈或改显式栈；⑦ 比较两棵树的哈希必须用同一套参数（B、种子）且都以重心为根，直接拿「以 1 为根」的哈希比较是错的。

自查：随机生成 n ≤ 20 的树与它的随机重标号副本，检查哈希不变（这一步抓「依赖标号」的错误）；再用 AHU 规范形分组，检查「哈希相同 ⇔ 规范形相同」在数千组随机样本上是否成立。

30.9 回文自动机（PAM）与回文划分

一句话：PAM 把所有本质不同的回文串压成一棵树，节点就是一个回文串，父亲是它的最长真回文后缀。加一个字符均摊 O(1)，整串 O(n)。Manacher 只能回答「以每个位置为中心最长能扩多长」，PAM 多出来的能力是能对每一个回文串单独做统计——这正是省选以上回文题的需求。

什么时候该上 PAM：题目出现「本质不同的回文子串个数」「每个回文串出现了几次」「把串划分成若干回文串的方案数」「以某位置结尾的最长回文」「两个串的公共回文串」，且 n 到 10^5—10^6。只问最长回文子串时用 Manacher 就够，不必背 PAM。

两个根的作用：0 号节点 `len = -1` 是奇根，1 号节点 `len = 0` 是偶根，二者 fail 都指向 0。奇根的 -1 让「单字符回文」在 getfail 时一次就成功（下标 i - (-1) - 1 = i，正好与自己比较），这一行是全模板最容易写错的地方。

三条关键性质：① 节点数 = 本质不同回文串数 + 2，且 ⩽ n + 2（每加一个字符最多新建一个节点）；② `cnt[v]` 表示「以某个位置结尾的最长回文」的出现次数，沿 fail 树自底向上累加即得该回文串在原串中的总出现次数；③ 把 `len[v] - len[fail[v]]` 记作 `diff`，若 `diff[v] == diff[fail[v]]`，则 v 与 fail[v] 属于同一等差数列，用 `slink` 一步跳到该数列的下一个代表——这是回文划分能做到 O(n log n) 的原因。

```
const int MAXN = 1000006, SIG = 26, MOD = 1000000007;
int nxt[MAXN][SIG], fail[MAXN], len[MAXN], cnt[MAXN];
int diff[MAXN], slink[MAXN];
long long f[MAXN], g[MAXN];          // 回文划分用
char s[MAXN];
int sz, last;

void init() {                        // 0: 奇根 len=-1   1: 偶根 len=0
    sz = 2; last = 1;
    len[0] = -1; len[1] = 0;
    fail[0] = 0; fail[1] = 0;
    memset(nxt[0], 0, sizeof(nxt[0]));
    memset(nxt[1], 0, sizeof(nxt[1]));
    cnt[0] = cnt[1] = 0;
    diff[0] = diff[1] = 0;
    slink[0] = slink[1] = 0;
}

int getfail(int x, int i) {          // 沿 fail 链找「前面能接上 s[i]」的最长回文后缀
    while (i - len[x] - 1 < 0 || s[i - len[x] - 1] != s[i]) x = fail[x];
    return x;
}

void extend(int i) {                 // 插入 s[i]（0-indexed）
    int c = s[i] - 'a';
    int p = getfail(last, i);
    if (!nxt[p][c]) {
        int q = sz++;
        memset(nxt[q], 0, sizeof(nxt[q]));
        len[q] = len[p] + 2;
        int qf = getfail(fail[p], i);
        fail[q] = nxt[qf][c] ? nxt[qf][c] : 1;    // 先定 fail，再把 nxt[p][c] 指向 q
        nxt[p][c] = q;
        cnt[q] = 0;
        diff[q]  = len[q] - len[fail[q]];
        slink[q] = (diff[q] == diff[fail[q]]) ? slink[fail[q]] : fail[q];
    }
    last = nxt[p][c];
    ++cnt[last];                     // 每个位置给「以 i 结尾的最长回文」记一次
}

void count_occ() {                   // 回文串出现次数 = fail 树上子树和
    static int ord[MAXN];
    int m = 0;
    for (int v = 2; v < sz; ++v) ord[m++] = v;
    sort(ord, ord + m, [](int a, int b) { return len[a] > len[b]; });  // 按 len 降序
    for (int i = 0; i < m; ++i) cnt[fail[ord[i]]] += cnt[ord[i]];
}

// 回文划分方案数：f[i] 表示前缀 s[0..i-1] 的划分数
// 同一 series（len 构成等差数列的那一段回文后缀）整体转移，均摊 O(n log n)
long long palPartition(int n) {
    init(); f[0] = 1;
    for (int i = 1; i <= n; ++i) {
        extend(i - 1);
        f[i] = 0;
        for (int v = last; len[v] > 0; v = slink[v]) {
            g[v] = f[i - len[slink[v]] - diff[v]];
            if (diff[v] == diff[fail[v]]) g[v] = (g[v] + g[fail[v]]) % MOD;
            f[i] = (f[i] + g[v]) % MOD;
        }
    }
```

```
    return f[n];
}
```

复杂度与空间：时间 O(n)（getfail 的回跳总次数摊还到 O(n)，每个字符均摊 O(1)）；回文划分额外一层 series 枚举，是 O(n log n)。空间 O(n σ)：`int nxt[MAXN][26]` 在 n = 10^6 时是 104 MB，必然 MLE——大规模数据要把 σ 缩到实际用到的字符数（如只有 0/1 时是 8 MB），或改用 `unordered_map` / 邻接表存边（常数变大，慎用于 10^6）。

典型问法对照：

- 本质不同回文串个数：跑完一遍后 `sz - 2`，不需要建 fail 树。
- 每个回文串出现次数：`count_occ()` 之后 `cnt[v]`；注意 `cnt[0]`、`cnt[1]` 是两个根，不能计入答案。
- 回文划分方案数：上面的 `palPartition`，把递推换成 min/max 就是「最少/最多划分数」。
- 最长双倍回文：维护 `trans[v]`（长度不超过 len[v]/2 的最长回文后缀），判 `2*len[trans[v]] == len[v]`。
- 两串公共回文串：对 A 建 PAM，把 B 在 PAM 上「走」——沿 fail 跳匹配，能落到的节点集合就是公共回文。
- 强制在线：PAM 天生在线，逐个字符插入即可，不需要预知整串。

易错点：① 奇根 `len[0]` 写成 0 会让长度 1 的回文接错父亲，直接导致节点数偏少；② `fail[0] = 0` 而不是 -1，getfail 的循环靠 `i - len[x] - 1 < 0` 兜底，若换成「只比较字符」会越界；③ 必须先算 `fail[q]` 再把 `nxt[p][c]` 指向 q：当 `qf == p` 时，后写会让 q 把自己当父亲，形成自环，getfail 死循环；④ `cnt` 只在每次 extend 后对 `last` 自增，新建节点置 0；累加必须按 len 降序（等价于 fail 树的逆拓扑序），乱序会算少；⑤ 回文划分的 `g[v]` 是按节点存的、跨 i 复用，必须先写 `g[v]` 再用 `g[fail[v]]`，顺序反了读到的是上一轮的值；⑥ 多测清零只需 `sz`、`last` 与 `nxt[0]`、`nxt[1]` 两行（其余节点在新建时清），漏清会让上一组的边残留；⑦ 字符映射 `s[i]-'a'` 只适用小写字母，数字串、大写串必须换映射；⑧ PAM 不支持删字符，也不支持区间查询，别拿去套「可持久化」的模板。

自查：随机串（σ ≤ 4、n ≤ 40）与暴力枚举所有子串对拍三项——本质不同回文个数、每个回文串的出现次数、回文划分方案数。本轮 300 组随机串零反例，其中回文划分是拿 O(n^2) 的 `f[i] = Σ_{s[j..i-1] 回文} f[j]` 做基准。

30.10 最小表示法、Lyndon 分解与 Duval

一句话：把所有循环移位按字典序排一排，最小的那个叫最小表示（最大的叫最大表示），本节给出找它起点下标的 O(n) 双指针；Lyndon 分解是同一套「失配即整段排除」思想的另一种用法——把串切成若干「严格小于自己所有非空真后缀」的串，切法唯一（Chen–Fox–Lyndon 定理），Duval 算法一遍切完。两者都不需要哈希、不需要后缀数组，n 到 10^6 也只用 O(1) 额外空间。

什么时候用：① 判两串是否循环同构（长度相同且最小表示相同）；② 破环成链的题要把环上的串规范化成唯一形式，再比较或哈希——直接哈希原串会因起点不同而失败；③ 求字典序最小的循环移位的下标（最小表示法的原始问法，常见于「环上字典序最小标号」类构造）；④ 单个串的 Lyndon 分解：最小后缀、串的幂次结构分析，

以及 SA-IS 等线性后缀数组构造的预处理步骤。若题目只要「某个循环移位的哈希值」且起点固定，普通哈希即可，不必上这套。

核心结论（最小表示法）：设两个候选起点 i < j，从各自起点逐位比较，前 k 位相同，第 k 位 `s[(i+k) mod n] > s[(j+k) mod n]`。那么以 i, i+1, …, i+k 中任一位置为起点的移位，都严格大于以 j 为起点的移位——它们的前 k+1 个字符与 i 起点完全相同，而 i 起点在这 k+1 位上已经输给 j。于是这 k+1 个起点可以一次性排除，`i ← i+k+1`；反之若小于则 `j ← j+k+1`。两个指针都只增不减，每轮至少排除一个起点，总比较次数 O(n)。

```
// 最小表示：返回字典序最小的循环移位的起点（0-indexed）
int minRep(const string& s) {
    int n = s.size(), i = 0, j = 1, k = 0;
    while (i < n && j < n && k < n) {
        char a = s[(i + k) % n], b = s[(j + k) % n];
        if (a == b) { ++k; continue; }      // 相同，继续比下一位
        if (a > b) i = i + k + 1;           // i 起点这 k+1 个起点全部出局
        else       j = j + k + 1;
        if (i == j) ++j;                    // 两指针重合，必须错开
        k = 0;                              // 失配后已比长度作废
    }
    return min(i, j);                       // 谁先越界就是谁
}

// 最大表示：只把两个比较符号反过来，跳跃方向与语义都不变
int maxRep(const string& s) {
    int n = s.size(), i = 0, j = 1, k = 0;
    while (i < n && j < n && k < n) {
        char a = s[(i + k) % n], b = s[(j + k) % n];
        if (a == b) { ++k; continue; }
        if (a < b) i = i + k + 1;
        else       j = j + k + 1;
        if (i == j) ++j;
        k = 0;
    }
    return min(i, j);
}
```

Lyndon 串：一个串 w 若严格小于它所有的非空真后缀，就叫 Lyndon 串（等价说法：严格小于它所有的非平凡循环移位，因此它是自身所有循环移位中的唯一最小者）。例：`aab`、`ab`、`b` 是 Lyndon；`aba` 不是（后缀 `a` 更小）；`aa` 不是（后缀 `a` 更小，Lyndon 要求严格小于，故只有无平方串才是 Lyndon 的必要条件之一）。

Chen–Fox–Lyndon 定理：任意串 s 有唯一的分解 `s = w1 w2 … wk`，每个 wi 是 Lyndon 串，且 `w1 ≥ w2 ≥ … ≥ wk`（字典序不升）。这条「不升」正是分解唯一的来源：只要有一对相邻因子升序，就可以把它们并起来继续切。

```
// Lyndon 分解：返回每个因子的半开区间 [l, r)
// 保证 s[l..r) 是 Lyndon 串，且各因子字典序不升，拼接即原串
vector<pair<int,int>> duval(const string& s) {
    int n = s.size(), i = 0;
    vector<pair<int,int>> res;
    while (i < n) {                         // i：本轮因子的起点
        int j = i + 1, k = i;               // k：与因子首部对齐的比较指针
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j]) k = i;         // 严格变大：周期从头再来
            else             ++k;           // 相等：继续沿同一周期比
            ++j;
        }
        while (i <= k) {                    // 长度 j-k 的块重复了若干次，一并切掉
            res.push_back({i, i + j - k});
            i += j - k;
        }
    }
    return res;
}
```

Duval 的三段结构怎么读：内层 while 在做一个「KMP 式」的自我比较——把 s[i..] 与 s[i..] 的某个位移对齐，判它是不是「某个 Lyndon 串重复若干次再接一个前缀」。退出时 `j - k` 就是被确认的那个最小周期长度，`i <= k` 说明这个块至少重复了两次，于是逐个切出长度 `j-k` 的因子；`i > k` 时说明这一轮只得到一个不完整的块，留给外层继续延长。因为 i 单调递增、j 在每轮内单调递增且不回退，总复杂度 O(n)，空间 O(1)（不含结果数组）。

应用对照：

- **判循环同构：长度必须相同，然后比较规范化串 `rot(a) = a.substr(p) + a.substr(0, p)` 是否等于 `rot(b)`（p 为各自的最小表示起点）。只比较两个起点下标 p 是否相等是错的——下标相同与串相同没有关系。**
- **规范化后哈希：以最小表示起点切开再算哈希，同一循环等价类得到同一哈希值，这是「环形串去重」「环同构计数」的标准做法。**
- **最小/最大字典序标号：破环成链时用 minRep 定位唯一的起始位置，保证同一张图/同一个环只被枚举一次。**
- **最小后缀：对 s 做 Lyndon 分解，最后一个因子的起点即最小后缀的起点（不升序保证最后一段最小）。**
- **串的幂次结构：若分解得到的因子全部相同，则 s 是该因子的整数次幂；但反过来不成立（`baba` 分解为 `b|ab|ab`，仍是幂串），判幂请用 30.7 的 `n % (n - fail[n])`，别拿 Duval 当判据。**

复杂度：最小/最大表示 O(n) 时间、O(1) 额外空间；Duval O(n) 时间、O(1) 额外空间（结果另计）；两者都对字符集大小不敏感，$\sigma = 26$ 或 $\sigma = 10^9$ 一样快。

易错点：① 下标必须写成 `(i+k) % n`，k 会超过 n，忘了取模直接越界；若改用 `s + s` 省取模，数组要开到 2n 且循环条件里的 n 仍是原长，混用两者是常见崩点。② 循环条件是 `k < n` 而不是 `k <= n`：k 到 n 说明前 n 位全同，两起点等价，再比就死循环。③ 跳跃量是 `i + k + 1` 而不是 `i + k` 或 `i + 1`：跳少了退化成 $O(n^2)$，本题「看起来对但 TLE」的根源几乎都在这里。④ 失配后必须 `k = 0`，只有「相等」分支能保留 k。⑤ `i == j` 时必须 `++j`，否则两指针重合、比较恒等，k 一路涨到 n 才退出，答案虽对但白跑一轮。⑥ 返回

值是 `min(i, j)` 而不是 i：两个指针谁先到 n 不确定。⑦ maxRep 只改比较符号，跳跃方向不能跟着改——「出局的永远是字典序较大的那个起点」，这个语义与求最小还是最大无关，改错会得到随机结果。⑧ 字符比较依赖 `char` 的符号：开 `-fno-strict-aliasing` 不管用，遇到非 ASCII 或高位字节要用 `unsigned char`，或统一先 `- 'a'`。⑨ Duval 内层条件必须是 `s[k] <= s[j]`（带等号）；写成 `<` 会在「相邻字符相等」处提前收束，`abab` 被切成 `ab|ab` 之外的碎块，因子不再满足不升序。⑩ Duval 里 `k` 复位成 `i` 而不是 0：k 是相对本轮因子起点的偏移。⑪ 切块的循环是 `while (i <= k)`，带等号；写成 `i < k` 会漏掉最后一个因子。⑫ 空串不产生任何因子，调用方要能接受空结果；n = 1 时 minRep 与 duval 都应退化为 0 与单因子。⑬ 判循环同构时长度不同必须直接返回 false，先比长度再比最小表示，否则会在 substr 处越界。

自查：本轮用 4000 组随机串（$n \leqslant 9$，$\sigma \leqslant 4$）与暴力对拍——minRep、maxRep 与「枚举全部 n 个移位取字典序极值」完全一致；Duval 的每一项都检查三条（拼接还原原串、每个因子是 Lyndon、因子字典序不升）；另用 3000 组随机串检验「循环同构 $\Leftrightarrow$ 最小表示串相同」，含随机重排的负例，零反例。

30.11 后缀自动机进阶：endpos、link 树与广义 SAM

一句话：30.6 只画了 SAM 的轮廓，真正做题要会三件事——把状态读成 endpos 等价类（于是 `len[v] - len[link[v]]` 是该类里的子串个数）、把 link 边读成一棵树（子树和即出现次数）、把多串情形写成广义 SAM（同一自动机装多个串，插入要特判已有转移）。会了这三点，SAM 就从「能判子串」升级成「能统计一切」。

状态到底是什么：SAM 的一个状态 v 代表一组子串，它们的结束位置集合（endpos）完全相同。这一组子串恰好是「长度从 `len[link[v]] + 1` 到 `len[v]`」的那些后缀，因此个数为 `len[v] - len[link[v]]`，把它们全加起来就是本质不同子串总数。这条等式是 SAM 上一切计数的起点，比「节点数不超过 2n-1」更值得记。

link 树：把 `link[v]` 当父亲，得到一棵以 0 为根的树（有的资料叫 parent 树 / 后缀链接树）。它有两层含义：从串的角度，v 的父亲是「v 的最长真后缀所在的状态」；从集合的角度，`endpos(v)` $\subsetneq$ `endpos(link[v])`，即 v 的 endpos 是父亲的真子集。于是「v 的子树大小（按前缀终止点计数）= $|\text{endpos}(v)|$ = v 所代表子串在原串中的出现次数」，这是 SAM 最常用的性质，几乎所有「出现次数」「出现在哪些位置」的题都归结为在 link 树上做一次自底向上的累加。

四条可背的结论：① 状态数 $\leqslant$ 2n-1，转移数 $\leqslant$ 3n-4（$n \geqslant 2$）；② 每个前缀在插入结束时对应一个状态（`cur`），把这些状态的计数置 1，其余置 0，按 len 降序在 link 树上累加，得到每个状态的出现次数；③ 从根出发沿任意路径走，读出的字符串都是原串的子串，且同一状态内的所有串「走到的终点相同、出现次数相同」；④ 两个串的最长公共子串可以在 A 的 SAM 上跑 B：能走就走，走不动沿 link 跳并令当前匹配长度 = `len[link]`，均摊 $O(|B|)$。

```
struct SAM {
    static const int MAXN = 2000005;              // 状态数 ≤ 2n-1
    int nxt[MAXN][26], link[MAXN], len[MAXN], cnt[MAXN];
    int sz, last;

    void init() {
        sz = 1; last = 0;
        len[0] = 0; link[0] = -1;                 // -1 让重定向循环自然终止
        memset(nxt[0], 0, sizeof(nxt[0]));
        cnt[0] = 0;
    }

    int extend(int c) {                           // 返回新状态编号
        int cur = sz++;
        len[cur] = len[last] + 1; cnt[cur] = 1;   // 前缀终止点，出现次数基数 1
        memset(nxt[cur], 0, sizeof(nxt[cur]));    // clone 会用 memcpy 整行覆盖，这里必须清
        int p = last;
        for (; p != -1 && !nxt[p][c]; p = link[p]) nxt[p][c] = cur;
        if (p == -1) link[cur] = 0;
        else {
            int q = nxt[p][c];
            if (len[p] + 1 == len[q]) link[cur] = q;
            else {                                // 需要分裂：q 的 endpos 被拆成两半
                int clone = sz++;
                len[clone] = len[p] + 1;          // 注意：不是 len[q]
                link[clone] = link[q]; cnt[clone] = 0;
                memcpy(nxt[clone], nxt[q], sizeof(nxt[q]));
                for (; p != -1 && nxt[p][c] == q; p = link[p]) nxt[p][c] = clone;
                link[q] = link[cur] = clone;
            }
        }
        last = cur;
        return cur;
    }

    void build(const string& s) { init(); for (char ch : s) extend(ch - 'a'); }

    long long distinct() {                        // 本质不同子串数
        long long r = 0;
        for (int v = 1; v < sz; v++) r += len[v] - len[link[v]];
        return r;
    }

    void countOcc(int n) {                        // 出现次数 = link 树子树和
        static int buc[MAXN], ord[MAXN];
        for (int i = 0; i <= n; i++) buc[i] = 0;
        for (int i = 0; i < sz; i++) buc[len[i]]++;
        for (int i = 1; i <= n; i++) buc[i] += buc[i - 1];
        for (int i = 0; i < sz; i++) ord[--buc[len[i]]] = i;     // 按 len 升序
        for (int i = sz - 1; i > 0; i--) cnt[link[ord[i]]] += cnt[ord[i]];
    }

    int lcs(const string& t) {                    // t 与已建串的最长公共子串
        int v = 0, l = 0, best = 0;
        for (char ch : t) {
            int c = ch - 'a';
            while (v && !nxt[v][c]) { v = link[v]; l = len[v]; }
            if (nxt[v][c]) { v = nxt[v][c]; ++l; }
            else { v = 0; l = 0; }
            best = max(best, l);
        }
        return best;
```

```
    }
};

// 字典序第 k 小（k 从 1 起）的本质不同子串：先按 len 降序算 f[v] = 1 + Σ f[to]
// 需要额外的 f / buc2 / ord2 三个数组，与 countOcc 里同构，此处略去排序部分
string kth(int k, const int* f) {                 // f 已按 len 降序算好
    if (k >= f[0]) return "";                    // f[0] 含空串，故用 >=
    string res; int v = 0;
    while (true) {
        for (int c = 0; c < 26; c++) if (nxt[v][c]) {
            int u = nxt[v][c];
            if (k > f[u]) k -= f[u];
            else { res += char('a' + c); v = u; --k; break; }
        }
        if (k == 0) return res;
    }
}
```

广义 SAM：多个串共用一个自动机时，`init` 只做一次，每条新串的 `last` 重置为 0。麻烦在于「转移已经存在」的情形——若直接照上面的 `extend` 跑，会为一条已经走过的路径新建 cur，导致 len 偏大、本质不同子串数重复计数。正确做法是把「已存在转移」单独处理：

```
// 广义 SAM 的插入：last 由调用方给出（每串从 0 开始）
int gextend(int last, int c) {
    if (nxt[last][c]) {                          // 转移已存在，不能新建 cur
        int q = nxt[last][c];
        if (len[q] == len[last] + 1) return q;   // 直接复用
        int clone = sz++;                        // 否则仍需分裂
        len[clone] = len[last] + 1;
        link[clone] = link[q]; cnt[clone] = 0;
        memcpy(nxt[clone], nxt[q], sizeof(nxt[q]));
        for (int p = last; p != -1 && nxt[p][c] == q; p = link[p]) nxt[p][c] = clone;
        link[q] = clone;
        return clone;
    }
    // 与普通 extend 相同的后半段，返回 cur
    int cur = sz++;
    len[cur] = len[last] + 1; cnt[cur] = 1;
    memset(nxt[cur], 0, sizeof(nxt[cur]));
    int p = last;
    for (; p != -1 && !nxt[p][c]; p = link[p]) nxt[p][c] = cur;
    if (p == -1) link[cur] = 0;
    else {
        int q = nxt[p][c];
        if (len[p] + 1 == len[q]) link[cur] = q;
        else {
            int clone = sz++;
            len[clone] = len[p] + 1;
            link[clone] = link[q]; cnt[clone] = 0;
            memcpy(nxt[clone], nxt[q], sizeof(nxt[q]));
            for (; p != -1 && nxt[p][c] == q; p = link[p]) nxt[p][c] = clone;
            link[q] = link[cur] = clone;
        }
    }
    return cur;
}
```

复杂度与空间：建树 O(n)（每个字符的均摊操作常数是 2—3 次指针跳转）；countOcc 的计数排序是 O(n + |Σ| 无关)，比每次 `sort` 按 len 排序的 O(n log n) 更稳。空间是硬约束：`int nxt[2·10^6][26]` 就是 208 MB，n = 10^6 必然 MLE。三条出路——把字符集压到实际出现的字符数（0/1 串只要 16 MB 级）、把 26 个 int 换成 `map<int,int>` 或邻接表（时间常数涨 3—5 倍）、只在需要处保存必要状态（例如只做 LCS 时可以用后缀数组替代）。

典型问法对照：

- 本质不同子串数：`Σ(len[v] - len[link[v]])`，无需建 link 树。
- 某子串出现次数：先在自动机上走到它对应的状态 v（走不到即 0 次），答案 `cnt[v]`（须先 countOcc）。
- 最短/最长不重复子串：等价于该状态区间的下界 `len[link[v]]+1` 与上界 `len[v]`。
- 两串最长公共子串：A 建 SAM，B 在上面跑 `lcs`，O(|A| + |B|)。
- 字典序第 k 小子串：DAG 上 DP 求 `f[v] = 1 + Σ f[to]`，按 len 降序算，再沿转移逐位下降。
- 多串公共子串：广义 SAM 建好后，在 link 树上做「每个状态被几条串覆盖」的计数，覆盖数等于串数者即为公共子串。
- 子串出现位置集合：需要线段树合并维护 endpos 集合，属 27 章可持久化/合并那一套，先确认题目是否真的需要位置而不仅是次数。

易错点：① `len[clone]` 要写 `len[p] + 1`，写成 `len[q]` 会让 clone 与被复制的 q 完全重合，答案偏大且难查。② `cnt[clone] = 0` 不可省：clone 不是任何前缀的终止状态，置 1 会让该类的出现次数凭空翻倍。③ 必须先 `memcpy` 转移再改指向；顺序反了会把 clone 的转移清空，随后 `nxt[p][c] == q` 的重定向循环找不到目标。④ 重定向的条件是 `nxt[p][c] == q` 且 `p` 到 -1 停；写成 `!= q` 会跳出循环、漏改一半状态。⑤ `link[0] = -1`：写 0 会让重定向循环在根上自旋，是 SAM 模板最常见的死循环来源。⑥ 计数排序的桶大小按串长 n，不是按状态数 sz——len 的最大值是 n，桶开 n+1 就够，但多测时 n 变化，桶必须按当前 n 清。⑦ 降序累加时跳过根（`i > 0`），否则 `cnt[link[0]]` 会写到下标 -1。⑧ 多测清零：只清 `nxt[0]` 不够，`extend` 里对 `nxt[cur]` 的清零和 clone 的整行 `memcpy` 都是在覆盖旧边，若把这两行删掉「省常数」，第二组数据必然错。⑨ 广义 SAM 直接 `last = 0` 然后跑普通 extend 是错的，必须走 `gextend` 的已有转移分支，否则本质不同子串数偏多。⑩ LCS 失配时先 `v = link[v]` 再把 `l = len[v]` 同步，顺序或写法反了会少算——`l` 永远不能超过 `len[v]`。⑪ 第 k 小里判越界用 `k >= f[0]`（f 含空串的 1），用 `>` 会多输出一个空串。⑫ n = 1 时状态数为 2（根 + 一个），所有下标上界都要按 `sz` 动态取，别硬编码从 1 开始。

自查：本轮用 600 组随机串（n ≤ 12，σ ≤ 3）对拍四项——本质不同子串数与暴力集合大小一致；每个状态的 `cnt` 与该状态内任一代表串的真实出现次数一致；同状态内子串个数恰为 `len[v] - len[link[v]]`；字典序第 k 小与排序后的子串表逐项一致。另用 800 组随机串对拍 LCS、500 组（1—3 串）对拍广义 SAM 的本质不同子串数与并集大小，全部零反例。

30.12 后缀数组进阶：height 的六种用法与单调栈

一句话：30.6 只说了「SA 能干什么」，没给写法。真正把 SA 用起来，全部依赖一条恒等式和一棵 RMQ 结构：

```
LCP(suffix(sa[i]), suffix(sa[j])) = min( ht[i+1], ht[i+2], ..., ht[j] )    (i < j)
```

也就是说，「两个后缀的最长公共前缀」不是字符串问题，而是 height 数组上的区间最小值问题。剩下的六种用法都是这句话的推论：能二分长度的，靠「按 ht ≥ L 分组」；能累加贡献的，靠「ht 作为区间最小值的单调栈」。

复杂度：倍增 + 基数排序构建 SA 是 O(n log n)（n = 10^6 时约 0.4 s）；Kasai 求 ht 是 O(n)；ST 表 O(n log n) 建、O(1) 查；二分分组是 O(n log n)；单调栈 O(n)。空间主要是 `st[20][n]`，n = 10^6 时 80 MB，再大改用线段树或分块。

模板 A：构建 SA 与 height。倍增 + 计数排序的写法，第二关键字由「sa 平移 k 位，越界者排最前」直接得到，不必真的排序。判终止的条件是「不同排名数 ≥ n」，不是循环 log n 次。

```
int sa[MAXN], rk[MAXN], oldrk[MAXN], ht[MAXN], tmp[MAXN], cnt[MAXN];

void buildSA(const char* s, int n) {          // s 下标 0..n-1
    int m = 256;
    for (int i = 0; i < n; ++i) { rk[i] = (unsigned char)s[i]; sa[i] = i; }
    for (int i = 0; i < m; ++i) cnt[i] = 0;
    for (int i = 0; i < n; ++i) ++cnt[rk[i]];
    for (int i = 1; i < m; ++i) cnt[i] += cnt[i-1];
    for (int i = n - 1; i >= 0; --i) sa[--cnt[rk[i]]] = i;      // 单字符先排一次
    for (int k = 1; ; k <<= 1) {
        int p = 0;
        for (int i = n - k; i < n; ++i) tmp[p++] = i;            // 第二关键字为空，最小
        for (int i = 0; i < n; ++i) if (sa[i] >= k) tmp[p++] = sa[i] - k;
        for (int i = 0; i < m; ++i) cnt[i] = 0;
        for (int i = 0; i < n; ++i) ++cnt[rk[tmp[i]]];
        for (int i = 1; i < m; ++i) cnt[i] += cnt[i-1];
        for (int i = n - 1; i >= 0; --i) sa[--cnt[rk[tmp[i]]]] = tmp[i];   // 按第一关键字稳定排序
        memcpy(oldrk, rk, n * sizeof(int));
        rk[sa[0]] = 0;
        for (int i = 1; i < n; ++i) {
            int x = sa[i-1], y = sa[i];
            bool same = oldrk[x] == oldrk[y] &&
                (x + k < n ? oldrk[x+k] : -1) == (y + k < n ? oldrk[y+k] : -1);
            rk[y] = rk[x] + (same ? 0 : 1);
        }
        m = rk[sa[n-1]] + 1;
        if (m >= n) break;
    }
}

void buildHt(const char* s, int n) {          // Kasai：ht[0] 恒为 0
    for (int i = 0, k = 0; i < n; ++i) {
        if (rk[i] == 0) { k = 0; continue; }
        int j = sa[rk[i] - 1];
        while (i + k < n && j + k < n && s[i+k] == s[j+k]) ++k;
        ht[rk[i]] = k;
        if (k) --k;                            // 关键：k 只掉 1，才有 O(n)
    }
}
```

用法一：任意两后缀的 LCP（O(1) 查询）。对 `ht[1..n-1]` 建 ST 表，查询时先把排名排好序、再取区间最小值。区间是 `[min(rk)+1, max(rk)]`，左端点加 1 不能漏。

```
int lg2_[MAXN], st[20][MAXN];
void buildST(int n) {
    lg2_[1] = 0;
    for (int i = 2; i <= n; ++i) lg2_[i] = lg2_[i >> 1] + 1;
    for (int i = 1; i < n; ++i) st[0][i] = ht[i];
    for (int k = 1; (1 << k) <= n; ++k)
        for (int i = 1; i + (1 << k) - 1 < n; ++i)
            st[k][i] = min(st[k-1][i], st[k-1][i + (1 << (k-1))]);
}
int lcp(int i, int j, int n) {                // 后缀起点 i、j
    if (i == j) return n - i;
    int a = rk[i], b = rk[j];
    if (a > b) { int t = a; a = b; b = t; }
    ++a;
    int t = lg2_[b - a + 1];
    return min(st[t][a], st[t][b - (1 << t) + 1]);
}
```

用法二：本质不同子串数。 `n(n+1)/2 − Σ ht[i]`。**直觉：按字典序逐个后缀插入，第 i 个后缀贡献的新子串是「它的全部前缀减去与上一个后缀重复的 ht[i] 个」。O(n)，不需要建树。**

用法三：不可重叠最长重复子串（二分 + 分组）。长度 L 可二分（若存在长度 L 的答案，长度 L−1 也存在）。判定时把 `ht[r] ≥ L` 的相邻后缀划成一组——组内任意两后缀的 LCP 都 ⩾ L；只需检查组内 `max(sa) − min(sa) ≥ L`，即两个出现位置不重叠。O(n log n)。

```
bool ok(int L, int n) {
    int mn = sa[0], mx = sa[0];
    for (int r = 1; r < n; ++r) {
        if (ht[r] >= L) {
            mn = min(mn, sa[r]); mx = max(mx, sa[r]);
            if (mx - mn >= L) return true;
        } else mn = mx = sa[r];              // 断开，重开一组
    }
    return false;
}
int longestRepeat(int n) {                   // 不可重叠
    int lo = 0, hi = n;
    while (lo < hi) {
        int mid = (lo + hi + 1) >> 1;
        if (ok(mid, n)) lo = mid; else hi = mid - 1;
    }
    return lo;
}
```

用法四：多串的最长公共子串。把 m 个串用互不相同的分隔符拼成一个串（分隔符必须比所有字符小且两两不同，否则跨串匹配会假阳性），对拼接串建 SA。仍二分 L，分组后统计「组内出现了几种串」——给每个位置记一个 `owner`（属于第几个原串，分隔符记 −1），组内覆盖数等于 m 即 L 可行。复杂度 O(N log N)，N 为总长。

```
// owner[i]：位置 i 属于第几个原串，分隔符处为 -1
bool okMulti(int L, int n, int m) {
    static int seen[MAXM], stamp = 0;
    int cur = 0;
    for (int r = 1; r < n; ++r) {
        if (ht[r] >= L) {
            for (int idx = r - 1; idx <= r; ++idx) {
                int o = owner[sa[idx]];
                if (o >= 0 && seen[o] != stamp) { seen[o] = stamp; ++cur; }
            }
            if (cur == m) return true;
        } else {
            ++stamp; cur = 0;                 // 清空：用时间戳代替 memset
            int o = owner[sa[r]];
            if (o >= 0) { seen[o] = stamp; cur = 1; }
        }
    }
    return false;
}
```

用法五：字典序第 k 小子串（去重）。SA 已按字典序排好后缀，第 i 个后缀贡献 `n - sa[i] - ht[i]` 个「此前没出现过」的子串，且这些子串恰好是它长度为 `ht[i]+1 … n-sa[i]` 的前缀。按 sa 顺序累减，落到哪个后缀、长度就是 `ht[i] + 剩余 k`。O(n)。

```
string kthSub(const char* s, int n, long long k) {   // k 从 1 开始
    for (int i = 0; i < n; ++i) {
        long long fresh = (long long)n - sa[i] - ht[i];
        if (k > fresh) { k -= fresh; continue; }
        return string(s + sa[i], ht[i] + (int)k);    // 长度 = ht[i] + k
    }
    return string();                                  // k 超过总数
}
```

用法六：单调栈统计 $\Sigma_{i<j}$ LCP。很多题（典型是「差异」型求和）要算所有后缀对的 LCP 之和。由恒等式，每一对 (i,j) 的贡献是 ht 区间的最小值，于是把问题转成「每个 ht[r] 作为最小值被多少个区间包含」：用单调栈求出左右第一个严格更小/更小或相等的位置 L[r]、R[r]，贡献为 `ht[r]·(r-L[r])·(R[r]-r)`，累加即得。O(n)。

```
long long sumLCP(int n) {                    // Σ_{i<j} LCP(suffix i, suffix j)
    static int L[MAXN], R[MAXN], stk[MAXN];
    int top = 0;
    for (int r = 1; r < n; ++r) {
        while (top && ht[stk[top-1]] > ht[r]) --top;    // 左边第一个「严格更小」
        L[r] = top ? stk[top-1] : 0;
        stk[top++] = r;
    }
    top = 0;
    for (int r = n - 1; r >= 1; --r) {
        while (top && ht[stk[top-1]] >= ht[r]) --top;   // 右边第一个「更小或相等」
        R[r] = top ? stk[top-1] : n;
        stk[top++] = r;
    }
    long long ans = 0;
    for (int r = 1; r < n; ++r)
        ans += (long long)ht[r] * (r - L[r]) * (R[r] - r);
    return ans;
}
```

用法选择表

问法	手段	复杂度
两后缀 LCP / 模式串在串中出现次数	ST 表 RMQ（或线段树）	O(1) / O(log n) 每次
本质不同子串数	n(n+1)/2 − Σht	O(n)
最长重复子串（可/不可重叠）	max(ht) / 二分 + 分组	O(n) / O(n log n)
多串最长公共子串	拼接 + 分隔符 + 二分 + 覆盖计数	O(N log N)
字典序第 k 小子串	按 sa 累减 fresh	O(n)
Σ 后缀对 LCP / 区间最小值类统计	单调栈	O(n)
子串出现位置集合	SA + 二分定位排名区间 + 主席树	O(n log n)

易错点

- RMQ 区间左端点忘了 +1。LCP 是 `ht[min(rk)+1 .. max(rk)]`，写成 `[min(rk), max(rk)]` 会把「该后缀与自己在 SA 中的前一个」的 ht 也算进去，答案偏小。本轮对拍：200 组随机串、每组 20 次随机查询，漏 +1 时会大量偏小。
- 倍增的终止条件写成固定 log n 次。必须以「排名数 ≥ n」为止；固定次数的写法在 n 不是 2 的幂时最后一次可能白跑，更糟的是位运算 `k <<= 1` 一直增长到 `n - k` 为负，`for (int i = n-k; ...)` 的下标直接越界。
- 初始没有按单字符排一次。倍增的第一轮假定 `sa` 已按 rk 有序（第二关键字顺序由 sa 平移得到），直接从 `sa = 0,1,2,…` 起步，第一轮得到的顺序是错的，且错得很隐蔽——小数据常偶然正确。

- **Kasai 里的 k 归零时机。只有 `rk[i] == 0`（该后缀字典序最小，没有前驱）时才能把 k 清零；写在别处会让复杂度退化为 $O(n^2)$。另外 `if (k) --k` 不可漏，漏掉同样是 $O(n^2)$。**
- **多串拼接的分隔符相同。m 个串用同一个分隔符，会让「跨串」的假公共子串被算进来，答案偏大。分隔符还必须比所有字符都小，且不出现在原串里。**
- **分组判定的「断点」处理。`ht[r] < L` 时必须把 mn/mx 重置为 `sa[r]` 而不是继续沿用上一组的极值，否则会跨组比较，报出不存在的不重叠对。**
- **第 k 小子的长度算错。长度是 `ht[i] + k`（k 是剩余值），不是 `k` 也不是 `ht[i] - k`；k 超过总数时要显式返回空串，否则会构造出越界的 `string(s + sa[i], 超长)`。**
- **单调栈的严格/非严格混用。左边界取「严格更小」、右边界取「更小或相等」，这样每个区间的最小值只被算一次；两边都取严格会重复计算相等 ht 的贡献，两边都不严格会漏算。**
- **ht 数组下标从 0 还是 1。ht[0] 恒为 0（第一个后缀没有前驱），所有 RMQ 与分组都从 r = 1 起；把 ht 当 0 基数组用会让整段错位一位，且小 n 时可能仍然「看起来对」。**
- **空间与常数。倍增法要 6 个 n 长数组 + ST 表 20n，$n = 10^6$ 时约 100 MB；若内存 256 MB 且还有其他结构，ST 表改成分块 RMQ 或直接上线段树。SA-IS 是 $O(n)$ 但常数与代码量都大，除非 $n \geqslant 5\times10^6$ 否则不划算。**

自查：本轮把上述模板逐行译成 Python 对拍——300 组随机串（$n \leqslant 30$，$\sigma = 3$）的 sa 与「直接排序所有后缀」完全一致，ht 与逐对暴力 LCP 一致；200 组 RMQ 查询与暴力 LCP 一致；200 组本质不同子串数与集合大小一致；200 组字典序第 k 小与排序子串表逐项一致；200 组不可重叠最长重复与 $O(n^3)$ 暴力一致；150 组（2—3 串）多串公共子串与暴力一致；250 组 Σ 后缀对 LCP 与 $O(n^2)$ 暴力一致。全部零反例。

30.13 AC 自动机上的 DP：状态转移、矩阵加速与状压

问题形状：给若干个模式串（禁用词 / 必须包含的关键词），问「长度恰为 `n` 的字符串里，有多少个不含任何模式串」「有多少个至少包含全部关键词」「出现次数总和」等等。这类题的共同点是：判定「当前前缀是否已经违规」只需要看 AC 自动机走到了哪个节点，于是「长度」成了阶段、「自动机节点」成了状态，DP 自然成立。30.4 只给了一句「在自动机上跑 DP」，本节把它拆成四种问法与三套模板。

结论一：转移是固定的，所以这是一个「矩阵幂」型递推。设自动机有 `S` 个节点，`dp[i][u]` 表示「长度为 `i`、走到了节点 `u`、且一路合法」的方案数，则

```
dp[i+1][ v ] += dp[i][ u ]        对每个字符 c，v = go(u, c)，且 v 不是「终止节点」
答案 = Σ_u dp[n][u]
```

`go(u, c)` 是补全过的转移（`nxt[u][c]`，在 build 时把空儿子指向 `nxt[fail[u]][c]`，O(1) 转移）。因为 `dp[i+1] = dp[i] · M` 里的 `M` 与 `i` 无关，`n` 到 `10^18` 时把 `M` 做快速幂即可。

问法	状态	复杂度	n 的上限
不含任何模式串的串数 `S`		`O(n·S·|Σ|)`	10^7 左右（`S` 小）
同上，但 `n` 极大	`S×S` 矩阵	`O(S³ log n)`	10^{18}（要求 `S ≤ 100`）

至少包含 `k` 个关键词 `S · 2^k` `O(n·S·2^k·|Σ|)` `k ≤ 10`，`n ≤ 10^3` 每个模式串的出现次数 `S` 一次扫描 + fail 树 `O(|T| + S)` 与 `n` 无关

结论二：终止标记必须沿 fail 下传，这是四个问法共同的前提。一个节点是「终止节点」，不仅因为它自己是一个模式串的结尾，还因为它的 fail 链上任何一个是结尾——文本匹配到这个节点时，后缀里出现的模式串同样算命中。所以 build 时要写 `term[u] |= term[fail[u]]`（BFS 顺序保证 `fail[u]` 先算完）。漏了这一句，只有「模式串恰好是另一模式串的后缀」时才会出错，这是它难被发现的原因：实测 840 组随机数据只错 18 组（2.1%），且错的全是 `{"ab", "b"}`、`{"abc", "bc"}` 这一类包含关系。

结论三：关键词「全部出现」用位掩码，且掩码要挂在节点上而不是走法上。`term[u]` 本身就是一个位掩码（第 `i` 位表示「第 `i` 个关键词在该节点结束」），DP 时 `mask' = mask | term[v]` 即可，代价是状态数乘 `2^k`。若关键词更多（`k > 12`），这条路走不通，只能改用容斥（29.13 的子集反演）把「全部出现」拆成「至少避开某几个」。

结论四：出现次数 = fail 树上的子树和。扫描文本时每到一个节点给计数 +1，然后把计数按 BFS 逆序（等价于按 fail 深度从大到小）累加到 `fail[u]`，每个模式串的出现次数就是它终点节点的子树和。这一步与「在自动机上 DP」是同一棵树的两种用法：前者自底向上汇总，后者自顶向下计数。

模板 A：AC 自动机 + 计数 DP（不含任何模式串，`n` 可达 10^7）：

```
#include <bits/stdc++.h>
using namespace std;
const int MAXS = 105, AL = 26;
const long long MOD = 1000000007LL;

struct AC {
    int nxt[MAXS][AL], fail[MAXS], term[MAXS], sz;
    void init() { sz = 1; memset(nxt[0], 0, sizeof nxt[0]); fail[0] = 0; term[0] = 0; }
    int newNode() { memset(nxt[sz], 0, sizeof nxt[sz]); fail[sz] = 0; term[sz] = 0; return sz++; }
    void insert(const char *s, int id) {
        int u = 0;
        for (; *s; ++s) {
            int c = *s - 'a';
            if (!nxt[u][c]) nxt[u][c] = newNode();
            u = nxt[u][c];
        }
        term[u] |= 1 << id;               // 同一个节点可能结束多个关键词
    }
    void build() {
        queue<int> q;
        for (int c = 0; c < AL; ++c) { int v = nxt[0][c]; if (v) { fail[v] = 0; q.push(v); } }
        while (!q.empty()) {
            int u = q.front(); q.pop();
            term[u] |= term[fail[u]];      // 关键一句：后缀命中也要标成终止
            for (int c = 0; c < AL; ++c) {
                int v = nxt[u][c];
                if (v) { fail[v] = nxt[fail[u]][c]; q.push(v); }
                else   { nxt[u][c] = nxt[fail[u]][c]; }   // 补全，之后转移 O(1)
            }
        }
    }
};

long long dp[MAXS], nd[MAXS];
long long countAvoid(AC &ac, long long n) {
    fill(dp, dp + ac.sz, 0); dp[0] = 1;
    for (long long i = 0; i < n; ++i) {
        fill(nd, nd + ac.sz, 0);
        for (int u = 0; u < ac.sz; ++u) {
            if (!dp[u]) continue;
            for (int c = 0; c < AL; ++c) {
                int v = ac.nxt[u][c];
                if (ac.term[v]) continue;              // 命中禁用词，剪掉
                nd[v] = (nd[v] + dp[u]) % MOD;
            }
        }
        copy(nd, nd + ac.sz, dp);
    }
    long long r = 0;
    for (int u = 0; u < ac.sz; ++u) r = (r + dp[u]) % MOD;   // 求和也要取模
    return r;
}
```

模板 B：`n` 极大时改矩阵幂。`S` 不超过 100 时，直接做矩阵乘法 `O(S³ log n)` 已经能过；若常数吃紧，只预处理 `M^(2^k)`，再把初始向量逐个乘过去，省掉「矩阵乘向量」的重复构造：

```
struct Mat {
    int n; long long a[MAXS][MAXS];
    Mat(int n_ = 0) : n(n_) { memset(a, 0, sizeof a); }
    Mat operator*(const Mat &b) const {
        Mat c(n);
        for (int i = 0; i < n; ++i)
            for (int k = 0; k < n; ++k) {
                if (!a[i][k]) continue;
                long long aik = a[i][k];
                for (int j = 0; j < n; ++j)
                    c.a[i][j] = (c.a[i][j] + aik * b.a[k][j]) % MOD;
            }
        return c;
    }
};

long long countAvoidBig(AC &ac, long long n) {
    int S = ac.sz;
    Mat M(S);
    for (int u = 0; u < S; ++u)
        for (int c = 0; c < AL; ++c) {
            int v = ac.nxt[u][c];
            if (!ac.term[v]) M.a[u][v] = (M.a[u][v] + 1) % MOD;   // 重边要累加
        }
    // 向量 e0 依次乘 M^(2^k)，避免求整个 M^n
    vector<long long> vec(S, 0); vec[0] = 1;
    while (n) {
        if (n & 1) {
            vector<long long> t(S, 0);
            for (int u = 0; u < S; ++u) if (vec[u])
                for (int v = 0; v < S; ++v)
                    t[v] = (t[v] + vec[u] * M.a[u][v]) % MOD;
            vec.swap(t);
        }
        M = M * M; n >>= 1;
    }
    long long r = 0;
    for (int v = 0; v < S; ++v) r = (r + vec[v]) % MOD;
    return r;
}
```

模板 C：关键词必须全部出现（状压）。`k` 个关键词各占一位，`term[u]` 直接当掩码用：

```
const int K = 8;                                  // 关键词个数，2^K 要能开下
long long f[2][MAXS][1 << K];

long long countAll(AC &ac, int n, int k) {
    int S = ac.sz, full = (1 << k) - 1, cur = 0;
    memset(f, 0, sizeof f);
    f[0][0][0] = 1;
    for (int i = 0; i < n; ++i, cur ^= 1) {
        memset(f[cur ^ 1], 0, sizeof f[cur ^ 1]);
        for (int u = 0; u < S; ++u)
            for (int m = 0; m <= full; ++m) {
                long long v0 = f[cur][u][m];
                if (!v0) continue;
                for (int c = 0; c < AL; ++c) {
                    int v = ac.nxt[u][c], nm = m | ac.term[v];    // 出现哪些关键词由节点决定
                    f[cur ^ 1][v][nm] = (f[cur ^ 1][v][nm] + v0) % MOD;
                }
            }
    }
    long long r = 0;
    for (int u = 0; u < S; ++u) r = (r + f[cur][u][full]) % MOD;
    return r;
}
```

易错点：

- **终止标记不下传。**`term[u] |= term[fail[u]]` **漏写时，只有「某模式串是另一模式串后缀」的数据会错：实测 840 组里错 18 组（2.1%），单看通过率完全不像个 bug。自查办法是专门造** `{"ab", "b"}`、`{"abc", "bc"}` **这种包含关系的样例。**
- `sum` **忘记取模。上面** `countAvoid` **最后一行若写成** `r += dp[u]` **不取模，** `S = 3` **时答案最多超出** `2·MOD`**；与矩阵版对拍会看到「差恰好一个** 10^9+7**」这种极有辨识度的偏差（实测 40 组里 2 组、差值精确等于一个模数）。凡是「对拍两边只差一个模数」，先怀疑这一处，不要怀疑算法。**
- **矩阵里的重边没有累加。**`M.a[u][v] += 1` **而不是** `= 1`**：不同字符可能从同一节点走到同一节点，最典型的是「模式串只用 a、b，其余 24 个字符全部补全到根」——此时** `M[0][0] = 24`**。实测模式串** `{"ab"}`**、字母表 26：正解** `n = 2` **是 675、**`n = 3` **是 17524（与枚举全串一致），改成赋值后分别只有 4 和 8。这个错法不会有任何报错，只是答案小了两三个数量级。**
- **状态数按「模式串个数」估。节点数是所有模式串长度之和 + 1，不是** `n × 个数`**；10 个长度 10 的模式串就可能是 101 个节点。数组一律按「总长 + 5」开，矩阵按** `S×S` **开——**`S = 100` **时一个** `long long` **矩阵就是 80KB，两个临时矩阵就 240KB，别在栈上开。**
- **多组数据不清空。**`sz` **不复位会让上一组的节点留在** `nxt` **里，本组建树时顺着它们继续走，答案偏大且与输入顺序有关。每组数据的第一句必须是** `ac.init()`**。**
- **关键词相同导致位掩码浪费。两个一模一样的模式串在同一个节点结束，** `term` **有两位，但「全部出现」的判定要求两位都在——只要出现一次两位就都置上了，结论正确；不过** `2^k` **会被白白放大一倍。去重后** `k` **才准。**
- `k` **大仍硬上状压。**`k = 20` **时状态数** `S·2^20` **直接爆内存。要么改成容斥（**`2^k` **也在容斥里，但可以配合「至少」转「恰好」降维），要么题目本身就只要求「至少包含其一」，那就把条件并进** `term` **的判空里，根本不需要掩码。**

- 用 `fail` 链跳转代替补全的 `go`。DP 内层每次 `while (u && !nxt[u][c]) u = fail[u]`，最坏把复杂度乘上模式串长度。build 里那句 `nxt[u][c] = nxt[fail[u]][c]` 就是为了让内层变成 O(1)。
- 把「不含任何模式串」写成「不等于任何模式串」。前者要求整个串的所有子串都不命中，后者只是整串不相等。判定必须在自动机上逐字符做，不能拿哈希整串比。
- 在 `n = 0` 或空模式串上翻车。空模式串的终点就是根，`term[0]` 被置位后所有 DP 都是 0（正确，因为空串是任何串的子串）；`n = 0` 时空串本身算 1 个方案，循环不执行、答案取 `dp[0][0] = 1`，别把返回写成 0。

自查：把三份模板逐行译成 Python（同样的 `go` 补全、同样的 `term` 下传、同样的按位或），与「枚举字母表 {a,b,c} 上全部长度 `n` 的串」暴力对拍：(1) 不含模式串计数，840 组（模式串 1—3 个、长度 1—3，`n = 1..7`）0 处不符；(2) 矩阵幂与迭代 DP 对拍 40 组（$n \le 25$，取模 10^9+7）0 处不符，其中修掉的那处差异正是「求和忘记取模」；(3) 状压「关键词全部出现」200 组 0 处不符；(4) fail 树子树和求出现次数，200 组随机文本与朴素 `find` 全等，0 处不符；(5) 错法实测：不下传终止标记 840 组错 18 组，矩阵建图把 `+= 1` 改成 `= 1` 时，模式串 `{"ab"}`、26 字母的答案从 675 / 17524 掉到 4 / 8（与枚举全串的 675 / 17524 对照）。本机无 `g++`，C++ 侧只做了语法与数组上界自查（`MAXS = 105` 对应「总长 $\le$ 104」）。

31 第 31 章　计算几何与杂项

几何题的分值高、失分也高，多数败在精度与退化情况。本章给出向量、凸包、扫描线的标准写法，另附随机化、bitset、根号分治三类「非主流但好用」的技巧；随后补上凸包的加法——闵可夫斯基和，它是凸包相交判定与凹序列 (max,+) 卷积的统一工具；末节是最小圆覆盖，随机增量法用期望 O(n) 解决「最多由三点确定」的经典问题，也是本章精度处理的反面教材集中地；31.9 收在半平面交上——把「每条约束都是一个半平面」的几何题统一成一个 O(n log n) 的单调队列算法，凸多边形交、二维线性规划、二分求最大内切圆都归它管；31.10 收在旋转卡壳上——凸包上最远点对、最小宽度与最小面积外接矩形共用同一条「对踵点单调」的双指针，求包 O(n log n) 之后本体只需 O(n)；31.11 离开几何，收在 min-max 容斥上——它把「全部条件都达成」的 max 拆成「至少一个达成」的 min 的加权和，按位期望、集齐问题、随机游走访问所有点都归它管，且不要求随机变量独立；31.12 最后把「距离」这门账算清——曼哈顿与切比雪夫只差一次 45° 旋转，同一条恒等式同时给出 O(n) 点集直径、O(1) 单次最远查询、菱形计数转方形计数与网格上的两遍距离变换，并且明确区分「直径只需 2 组符号、查询必须 4 组」这个最易混的细节。

31.1 向量、叉积与基础判定

计算几何的一切建立在叉积上。二维向量 `(x,y)` 的叉积 `cross(a,b) = a.x*b.y - a.y*b.x`，其符号表示 b 在 a 的顺时针还是逆时针。

```
struct Pt {
    double x, y;
    Pt operator-(const Pt& o) const { return {x - o.x, y - o.y}; }
    double operator^(const Pt& o) const { return x * o.y - y * o.x; }   // 叉积
    double operator*(const Pt& o) const { return x * o.x + y * o.y; }   // 点积
};
double cross(Pt o, Pt a, Pt b) { return (a - o) ^ (b - o); }
int sgn(double x) { return fabs(x) < 1e-9 ? 0 : (x > 0 ? 1 : -1); }
```

四个基础判定：点在线段上（叉积为 0 且点积投影在范围内）、两线段相交（跨立实验 + 特判共线）、点在多边形内（射线法或转角法）、多边形面积（鞋带公式 `Σ(x_i·y_{i+1} - x_{i+1}·y_i)/2`）。

精度处理原则：能用整数就不用浮点。坐标是整数时，叉积用 `long long` 精确比较，避免 eps。必须用浮点时，eps 取 1e-9（double 有 15 位有效数字，1e-9 在 10^4 量级坐标下是安全的）。

31.2 凸包

凸包是「包含所有点的最小凸多边形」，Andrew 算法（水平扫描）最好写：按 x 排序，正反各扫一遍，用叉积维护凸性。

```
vector<Pt> convexHull(vector<Pt> p) {
    sort(p.begin(), p.end(), [](const Pt&a, const Pt&b){
        return a.x != b.x ? a.x < b.x : a.y < b.y; });
    int n = p.size(), k = 0;
    vector<Pt> h(2 * n);
    for (int i = 0; i < n; ++i) {
        while (k >= 2 && sgn(cross(h[k-2], h[k-1], p[i])) <= 0) --k;
        h[k++] = p[i];
    }
    for (int i = n - 2, t = k; i >= 0; --i) {
        while (k > t && sgn(cross(h[k-2], h[k-1], p[i])) <= 0) --k;
        h[k++] = p[i];
    }
    h.resize(k - 1);
    return h;
}
```

凸包的四个用途：求最远点对（旋转卡壳，O(n)）、判断点是否在凸包内（二分 O(log n)）、半平面交（凸包的对偶问题）、闵可夫斯基和（两个凸包逐边合并）。

易错：判据 `<= 0` 会删掉共线点（得到严格凸包），`< 0` 保留共线点；点集退化（全部共线、只有 1—2 个点）要特判。

31.3 扫描线与离线技巧

扫描线把「二维的静态问题」转成「一维的动态问题」：按某一维排序，用数据结构维护另一维。

矩形面积并：按 x 扫描，线段树维护 y 方向被覆盖的长度，面积 = `Σ 覆盖长度 × Δx`。线段树的节点维护「覆盖次数」与「有效长度」，`pushup` 时若 `cnt>0` 则长度 = 区间全长，否则由儿子合并。

矩形周长并：面积并的变体，额外统计「覆盖段数」与「是否覆盖端点」，周长 = 竖边增量 + 横边增量。

三维偏序：CDQ 分治 + 树状数组，或排序 + CDQ + 归并，$O(n \log^2 n)$。

离线处理的三板斧：按询问的右端点排序（把「区间查询」转成「前缀查询」）、按值域排序（插入即生效）、时间分治（CDQ）。想到离线，往往能把 $O(n\sqrt{n})$ 降到 $O(n \log n)$。

31.4 随机化与模拟退火

随机化不是「碰运气」，而是在无法精确求解时用概率换正确率。三类用法：

1. 随机化哈希：给每个元素随机权值，用异或/和判断集合相等（如「判断两棵子树是否同构」）。

2. 随机化贪心 / 多次重排：把元素随机打乱后跑贪心，重复几十次取最优。对「划分」「调度」类问题常能拿到接近最优的解。

3. 模拟退火（SA）：以概率 `exp(-Δ/T)` 接受更差的解，温度 T 从高到低衰减。适用于连续或离散的「求函数极值」问题。

```
double T = 1e5, dT = 0.999;
while (T > 1e-8) {
    double nx = cur + (rand() * 2.0 / RAND_MAX - 1) * T;
    double delta = f(nx) - f(cur);
    if (delta > 0 || exp(delta / T) > rand() * 1.0 / RAND_MAX) cur = nx;
    T *= dT;
}
```

参数经验：初温取「最差解与最优解之差」的量级，降温系数 0.99—0.999，每个温度迭代 100—1000 次。退火结果不稳定，尽量多跑几次取最优。

爬山 vs 退火：爬山只接受更优解，容易陷局部最优；退火以一定概率接受更差解，能跳出局部最优。两者都用「多次随机起点」加固。

31.5 bitset 优化

`std::bitset` 把 64 个 bool 压进一个 `unsigned long long`，整体位运算由硬件并行完成，相当于常数除以 64。

```
bitset<MAXN> f;
f[0] = 1;
for (int i = 1; i <= n; ++i) f |= (f << w[i]);   // 子集和可行性，O(n·MAXN/64)
```

三个经典应用：

- **背包可行性：`f |= f << w`，把 O(nV) 降到 O(nV/64)。**
- **传递闭包：`reach[i] |= reach[j]`（j 可达 i 时），Floyd 的 $O(n^3)$ 降到 $O(n^3/64)$。**
- **集合交并：统计「有多少对集合有交集」，用 bitset 直接与运算。**

注意：`bitset` 的大小必须是编译期常量；动态长度用 `vector<unsigned long long>` 手写，或 `boost::dynamic_bitset`（NOI Linux 可用，但官方只保证标准库，慎用）。

31.6 根号分治与复杂度平衡

当问题有两种做法、复杂度分别是 $O(n\sqrt{n})$ 与 $O(n^2/\sqrt{n})$ 时，按阈值切换能把总复杂度压到 $O(n\sqrt{n})$。这是「看似无解」的题的常见突破口。

典型模型：多次询问「区间内出现次数 > k 的数」。k 大时暴力枚举，k 小时预处理每个数出现位置。阈值取 $\sqrt{n}$。

另一类：修改与询问的规模不对称。「单点修改 + 区间查询」用树状数组；「区间修改 + 单点查询」用差分。当两类操作次数差异悬殊时，选择与「次数多的一方」匹配的结构。

阈值的选择：令两种做法的复杂度相等，解出阈值。常见的是 $\sqrt{n}$、$n^{2/3}$（带修莫队）、$n/64$（bitset）。

什么时候想到根号分治：数据范围 10^5 但想不出 log 做法；题目同时有「小规模高频」与「大规模低频」两种操作；出现「出现次数」「颜色数」这类与阈值相关的量。

31.7 闵可夫斯基和与凸包相加

定义与结论： `A ⊕ B = { a + b | a ∈ A, b ∈ B }`**。若 A、B 都是凸多边形，则 A⊕B 仍是凸多边形，且它的边序列恰好是两条边序列按极角归并——每一条边只出现一次、方向不变。于是顶点数 ⩽ n + m，归并本身 O(n + m)，加上两次求凸包就是 O(n log n + m log m)。这是「凸包上做加法」唯一需要记住的一句话。**

两条可加性，一真一假：周长可加， `per(A⊕B) = per(A) + per(B)`**；面积不可加，** `area(A⊕B) = area(A) + area(B) + 混合项`**（混合项在归并过程中累计，不要试图写成闭式）。把面积写成两者相加是这套题最常见的失分点。**

什么时候用：① 判定两个凸多边形是否相交；② 求两凸包上的最近/最远距离；③ 凹序列的 (max,+) 卷积；④ 半平面交的对偶形式（凸包边的法向集合相加）。只要题目里出现「两个凸的可行域」「各自取一个点，使某线性函数最优」，先想闵可夫斯基和。

```
struct P { long long x, y; };
P operator+(P a, P b) { return {a.x + b.x, a.y + b.y}; }
P operator-(P a, P b) { return {a.x - b.x, a.y - b.y}; }
long long cross(P a, P b) { return a.x * b.y - a.y * b.x; }

// 输入：A、B 为逆时针、严格凸、无重复点、首尾不重复的点列（Andrew 求凸包的结果）
// 输出：A⊕B 的顶点，逆时针，可能有共线点（不影响面积与点内判定）
vector<P> minkowski(vector<P> A, vector<P> B) {
    int n = A.size(), m = B.size();
    if (n == 1) { for (auto& p : B) p = p + A[0]; return B; }   // 单点：直接平移
    if (m == 1) { for (auto& p : A) p = p + B[0]; return A; }
    // 两条边序列必须从同一极角方向的极点起步：取 y 最小、y 相同取 x 最小
    auto cmp = [](P a, P b) { return a.y != b.y ? a.y < b.y : a.x < b.x; };
    rotate(A.begin(), min_element(A.begin(), A.end(), cmp), A.end());
    rotate(B.begin(), min_element(B.begin(), B.end(), cmp), B.end());

    vector<P> ea(n), eb(m);
    for (int i = 0; i < n; ++i) ea[i] = A[(i + 1) % n] - A[i];
    for (int i = 0; i < m; ++i) eb[i] = B[(i + 1) % m] - B[i];

    vector<P> res; res.reserve(n + m);
    res.push_back(A[0] + B[0]);                  // 起点相加，后面只累加边
    int i = 0, j = 0;
    while (i < n || j < m) {
        if (i < n && j < m) {
            long long c = cross(ea[i], eb[j]);
            if (c > 0)      { res.push_back(res.back() + ea[i]); ++i; }
            else if (c < 0) { res.push_back(res.back() + eb[j]); ++j; }
            else            { res.push_back(res.back() + ea[i] + eb[j]); ++i; ++j; }  // 同向共线合并
        } else if (i < n) { res.push_back(res.back() + ea[i]); ++i; }
        else              { res.push_back(res.back() + eb[j]); ++j; }
    }
    res.pop_back();                              // 最后一步回到起点，去掉重复点
    return res;
}
```

用法一：凸包相交判定。两个凸集 A、B 有公共点 ⇔ `0 ∈ A ⊕ (-B)`（即 A 与 B 的闵可夫斯基差包含原点）。做法：对 B 的点全部取反、求 Minkowski 和、再判原点是否落在结果凸包内（点在凸包内是 O(log n)，实在不想写就 O(n+m) 扫一遍，反正和本身就要 O(n+m)）。相比「枚举 A 的每条边做半平面交」，这条路代码短一个数量级。

用法二：凸包间距离。最远点对：对 A⊕(-B) 求直径（旋转卡壳 O(n+m)）；最近距离：求原点到 A⊕(-B) 的距离，原点在内部时为 0。两块凸多边形的「最短平移量」也在这里——这是碰撞检测与「最少移动多少步才能相接」类题的标准解法。

用法三：凹序列的 (max,+) 卷积。设 f、g 都是凹序列（差分单调不增），则 `h[k] = max_i ( f[i] + g[k-i] )` 仍是凹序列，而且 h 的图像正是点集 `{(i, f[i])}` 与 `{(j, g[j])}` 的上凸壳的闵可夫斯基和——两条上凸壳边归并一次，O(n+m) 出全部 h[k]。这给「`f[i] = max_j (a[j] + b[i-j])`」型转移提供了除分治/单调队列之外的第三条路，前提是能证明 a、b 都是凹的。注意它要求整段卷积，不能只查一个 k；若只要单个 k，二分斜率更快。

易错点：① 输入必须是逆时针严格凸、无重复点、首尾不重复；直接扔未排序的点集进去，边向量乱序，归并立刻失效。② 两条边序列的起点极角必须一致（都用 y 最小、再 x 最小），否则归并错位，结果多边形自交。③ 叉积为 0 表示两边同向（严格凸包下不会出现反向共线），必须合并成一条边一次走完，否则结果里出现重复的共线点，做点数统计时会错。④ 退化：n = 1 是单点（直接平移），n = 2 是线段——归并仍成立，但结果会保留共线顶点，此时面积公式与点内判定都要允许共线。⑤ 坐标 10^9 时叉积到 10^{18}，必须 `long long`；也不要用 double 判叉积符号，10^{18} 已超出 double 的精确整数范围。⑥ 输出点数 $\leqslant n+m$，`reserve(2 * (n + m))` 留余量，最后记得 `pop_back()`。⑦ 面积不可加、周长可加，别照搬。⑧ 判「原点是否在凸包内」时，原点恰好落在边上（叉积为 0）算不算相交，要按题意分别处理，这一步含糊会 WA 在边界数据上。

自查：随机生成两组点集（坐标 −6…6，各 1—7 个点），求凸包→闵可夫斯基和→与「暴力枚举所有 a+b 再求凸包」逐点比对；同时检查周长可加性。本轮 3000 组随机数据（含大量单点、线段退化）全部一致，周长误差 < 10^{-9}。

31.8 最小圆覆盖：随机增量法

定义：给平面点集 P，求半径最小的圆使其覆盖所有点。三个支撑点的情况只有三种——1 个点（退化，半径 0）、2 个点（两点连线作直径）、3 个点（三点外接圆）。这个「最多由 3 个点确定」的性质是全部算法的支点。

判定条件：所求圆必定经过点集中 1、2 或 3 个点，且这三个点不能再由其中更少的点确定（即不存在以两点为直径就覆盖三点的情形）。直接枚举 `C(n,3)` 个外接圆再逐个校验是 $O(n^4)$，$n = 10^5$ 时完全不可用。

随机增量：先把点随机打乱，再按顺序处理。设已处理前 i 个点、得到最小圆 C；加入 p_i 时若它在 C 内则什么都不做，否则 p_i 必在新圆的边界上——于是把问题降一维：以 p_i 为边界点重算。第二层同理，若 p_j 不在圆内，则新圆以 p_ip_j 为直径；第三层若 p_k 还在圆外，则新圆是 $p_ip_jp_k$ 的外接圆。三层循环体量看起来是 $O(n^3)$，但随机顺序下第 i 步「p_i 落在圆外」的概率不超过 $3/i$（最终圆只由 3 个点决定，随机的 i 个点里这 3 个点中最后出现的那个才触发关键更新），内层期望总代价 $\Sigma\, O(i)\cdot(3/i) = O(n)$。

```
struct Pt { long double x, y; };
long double dist2(Pt a, Pt b) {
    long double dx = a.x - b.x, dy = a.y - b.y;
    return dx * dx + dy * dy;
}
// 三点外接圆圆心：以 a 为基准平移，解两条中垂线交点
Pt circumCenter(Pt a, Pt b, Pt c) {
    long double bx = b.x - a.x, by = b.y - a.y;
    long double cx = c.x - a.x, cy = c.y - a.y;
    long double d  = 2 * (bx * cy - by * cx);          // = 2 * cross(b-a, c-a)
    long double b2 = bx * bx + by * by, c2 = cx * cx + cy * cy;
    long double ux = (cy * b2 - by * c2) / d;
    long double uy = (bx * c2 - cx * b2) / d;
    return {a.x + ux, a.y + uy};
}

Pt o; long double r;                                   // 结果：圆心与半径
void minCircle(vector<Pt> p) {
    mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
    shuffle(p.begin(), p.end(), rng);                  // 必须！期望复杂度全靠它
    int n = p.size();
    o = p[0]; r = 0;
    for (int i = 1; i < n; ++i) {
        if (dist2(p[i], o) <= r * r * (1 + 1e-12L) + 1e-18L) continue;
        o = p[i]; r = 0;                               // p[i] 落在边界上
        for (int j = 0; j < i; ++j) {
            if (dist2(p[j], o) <= r * r * (1 + 1e-12L) + 1e-18L) continue;
            o = {(p[i].x + p[j].x) / 2, (p[i].y + p[j].y) / 2};   // 两点为直径
            r = sqrt(dist2(p[i], o));
            for (int k = 0; k < j; ++k) {
                if (dist2(p[k], o) <= r * r * (1 + 1e-12L) + 1e-18L) continue;
                o = circumCenter(p[i], p[j], p[k]);               // 三点定圆
                r = sqrt(dist2(p[i], o));
            }
        }
    }
}
```

复杂度：期望 `O(n)`，最坏 $O(n^3)$（概率随 `n` 增大趋 0）。空间 `O(n)`（存点集）。与两个替代方案的对比：凸包 + 三分（旋转卡壳求最小宽度圆）只在「答案由 2 点决定」时正确，一般情形不成立；模拟退火/梯度下降是启发式，精度依赖步长，评测机用多组极限数据即可卡掉。随机增量是唯一既精确又线性的做法。

易错点

- **忘了 `shuffle`。输入若按极角或坐标有序（出题人很容易造：圆周上按角度给出的点），每次加入的新点都在圆外，三层循环全部跑满，直接退化成 $O(n^3)$，10^5 必然超时。随机打乱是算法的一部分，不是可选优化。**
- **用绝对 eps 判「在圆外」。写成 `dist2 > r*r + 1e-10`，当坐标为 10^6 量级、r^2 达 10^{12} 时，`double` 在该量级的间隔约 10^{-4}，绝对 eps 等于没设——浮点误差会让「本已在圆内的点」被判在圆外，反复重算，甚至死循环在同一组三点上。必须用相对误差，如 `r*r*(1+1e-12)`。**
- **三点共线仍去求外接圆。`d = 2·cross(b−a, c−a) = 0` 时除零得 `inf/nan`，之后所有比较全为 `false`，答案静默变成 `nan`。理论上共线时 p_k 不可能在直径圆外，但浮点误差下仍可能进入；加一句 `if (fabs(d) < 1e-18L) continue;` 兜底。**

- 空点集没判。`p[0]` 越界。`n = 0` 时半径定义为 0，函数开头返回。
- 只算半径不验包含。随机增量的正确性依赖每次更新后圆确实覆盖已处理的点；若 `circumCenter` 的公式写错（例如把 `cy*b2 - by*c2` 写成 `by*c2 - cy*b2`），程序照样跑完并给出一个「看起来合理」的半径。提交前必须加一段 `for (auto&q : p) assert(dist2(q,o) <= r*r*(1+1e-9));` 自查。
- 用 `float` 或早期就 `sqrt`。`float` 只有 7 位有效数字，坐标 10^6 时误差已达 0.1；每次比较都开方既慢又累积误差。全程用平方距离比较、`long double` 运算，只在赋值 `r` 时开方一次。
- `r = dist2(...)` 忘了开方。三层里 `r` 是长度、`dist2` 是平方，混用会让「在圆外」的判断恒为假，答案恒为第一个点。
- 用 `rand()` 打乱。`rand()` 上限 32767，`n = 10^5` 时 `rand() % n` 只有 32768 种取值，`shuffle` 出来的顺序远非均匀，期望复杂度失去保证。用 `mt19937`。
- 把 `p` 按引用传入后又 `shuffle`。原数组顺序被改，若后面还要用它（例如输出原始编号对应的点）就会错位。要么传值，要么在打乱前另存一份。
- 整数坐标却想全程整数。外接圆圆心是有理数不是整数，最小圆覆盖无法在整数域里完成；要么一开始就用 `long double`，要么接受输出误差（多数题允许 10^{-6} 相对误差）。

自查：随机生成 `n ≤ 8`、坐标 −6…6 的点集，与「枚举全部 1 点 / 2 点 / 3 点确定的圆、取覆盖全部点的最小者」对拍。本轮 600 组全部一致（半径误差 < 10^{-7}），并对 `n = 20000` 与 `n = 10^5` 的随机点集验证了包含性（最大相对越界 1.2×10^{-16}，恰在 `double` 机器精度上）与线性耗时。

31.9 半平面交：把「每个约束都是一个半平面」一次解完

问题形式：给 n 条有向直线，每条取它左侧的半平面 `{q : cross(v, q - p) ≥ 0}`，求这些半平面的交。交可能是：凸多边形（有界）、无界凸区域、一条线段、一个点、或空集。绝大多数题目问的是面积，或「交是否非空」。

为什么不能逐条裁剪。用 Sutherland–Hodgman 逐条切割一个初始大矩形，单次切割 `O(当前顶点数)`，n 条线最坏 `O(n^2)`，n = 10^5 直接超时，而且「初始矩形取多大」本身就是个精度陷阱。正确做法是 按方向角排序 + 双端队列（S&I 算法），把「哪些线是有效的」压到 `O(n log n)`。

算法五步

1. 把每条线表示成「锚点 p + 方向 v」，并算出 `ang = atan2(v.y, v.x)`，按 ang 排序。
2. 同角度去重：方向相同的两条线只有一条有效——保留约束更强（更靠左）的那条，即若 `cross(旧线.v, 新线.p - 旧线.p) > 0` 就用新线替换旧线。
3. 依次把线压入双端队列。压入前先弹队尾：若「队尾两条线的交点」不在新线左侧，说明队尾那条线已被新线完全覆盖，弹出。
4. 同样地弹队首：若「队首两条线的交点」不在新线左侧，弹出队首。这一步是处理「环形约束」的关键，漏掉会让首尾自相矛盾。

5. **全部压完后，再用队首去检查队尾、用队尾去检查队首各一轮（首尾各可能残留一条废线），剩下的线数 $\geqslant 3$ 时，相邻两两求交即为答案多边形（逆时针）。**

复杂度：排序 `O(n log n)` 是瓶颈，队列部分每条线最多进队一次、出队一次，均摊 `O(n)`；空间 `O(n)`。与逐条裁剪的 `O(n²)` 相比，$n = 10^5$ 时差三个数量级。

可编译模板

```
#include <bits/stdc++.h>
using namespace std;
const double EPS = 1e-9;

struct Pt {
    double x, y;
    Pt(double x = 0, double y = 0) : x(x), y(y) {}
    Pt operator+(const Pt& o) const { return Pt(x + o.x, y + o.y); }
    Pt operator-(const Pt& o) const { return Pt(x - o.x, y - o.y); }
    Pt operator*(double k) const { return Pt(x * k, y * k); }
    Pt operator/(double k) const { return Pt(x / k, y / k); }
};
double cross(Pt a, Pt b) { return a.x * b.y - a.y * b.x; }

struct Line {
    Pt p, v;                 // 过 p、方向 v；半平面取 v 的左侧
    double ang;
    Line() {}
    Line(Pt p, Pt v) : p(p), v(v), ang(atan2(v.y, v.x)) {}
    bool operator<(const Line& o) const { return ang < o.ang; }
};
// 严格在左侧（留 EPS 余量，同等重要）
bool onLeft(const Line& l, Pt q) { return cross(l.v, q - l.p) > EPS; }

Pt inter(const Line& a, const Line& b) {
    double t = cross(b.p - a.p, b.v) / cross(a.v, b.v);
    return a.p + a.v * t;
}

// 返回交多边形（逆时针）；空集或无界都返回空 vector
vector<Pt> halfPlaneInter(vector<Line> ls) {
    sort(ls.begin(), ls.end());
    vector<Line> u;                                  // 同角度只留最强的一条
    for (const Line& l : ls) {
        if (!u.empty() && fabs(l.ang - u.back().ang) < EPS) {
            if (cross(u.back().v, l.p - u.back().p) > 0) u.back() = l;
            continue;
        }
        u.push_back(l);
    }
    deque<Line> q;
    for (const Line& l : u) {
        while (q.size() >= 2 && !onLeft(l, inter(q[q.size() - 1], q[q.size() - 2])))
            q.pop_back();
        while (q.size() >= 2 && !onLeft(l, inter(q[0], q[1])))
            q.pop_front();
        q.push_back(l);
    }
    while (q.size() >= 3 && !onLeft(q[0], inter(q[q.size() - 1], q[q.size() - 2])))
        q.pop_back();
    while (q.size() >= 3 && !onLeft(q[q.size() - 1], inter(q[0], q[1])))
        q.pop_front();
    if (q.size() < 3) return {};                     // 注意：可能是无界，不一定是空
    vector<Pt> poly;
    for (size_t i = 0; i < q.size(); ++i)
        poly.push_back(inter(q[i], q[(i + 1) % q.size()]));
    return poly;
}

double polygonArea(const vector<Pt>& p) {
    if (p.size() < 3) return 0;
    double s = 0;
```

```
    for (size_t i = 0; i < p.size(); ++i) s += cross(p[i], p[(i + 1) % p.size()]);
    return fabs(s) / 2;
}

// 凸多边形（逆时针）的边直接转成半平面
vector<Line> toHP(const vector<Pt>& poly) {
    vector<Line> ls;
    for (size_t i = 0; i < poly.size(); ++i)
        ls.push_back(Line(poly[i], poly[(i + 1) % poly.size()] - poly[i]));
    return ls;
}
```

三处典型用法

- **两个凸多边形求交：把两个多边形的边全部转成半平面（各自逆时针），跑一次 `halfPlaneInter`，面积即交集。复杂度 `O((n+m) log(n+m))`，比「逐边裁剪」稳。**
- **二维线性规划：每个约束 `a·x + b·y ≤ c` 都是一条半平面，判可行域是否为空；要最大化 `px + qy` 时，在交出的凸多边形上三分或直接枚举顶点（线性目标在凸多边形上的极值必在顶点取到）。**
- **二分 + 半平面交：把每条约束整体平移 `d`（沿内法向），二分最大的 `d` 使交仍非空，就是「最大内切圆半径」类问题；也用于判定「能否用半径 r 的圆盖住给定方向的所有半平面」。**

易错点

- **同角度去重判据写反。写成 `cross(新线.v, 旧线.p − 新线.p) > 0`（把新旧调了个位置）就会留下更弱的那条，交区偏大。这是本节最隐蔽的错误：结果仍是一个合法凸多边形，只是比真解大一圈。本轮实测偏 0.4%—12%，肉眼看不出来，必须靠对拍。**
- **把「无界」当成「空集」。`q.size() < 3` 时返回空 vector，只说明「没有有界交」。若题目允许无界（例如只问可行性、或问面积无穷），必须另外判断；常见补救是额外加四条 `|x| ≤ M, |y| ≤ M` 的边框线，把无界转成有界，M 取题目坐标范围的 2 倍，并确认答案区域确实落在框内。**
- **首尾两轮检查漏写。整个算法处理的是环，压完全部线后队首可能已经不满足队尾约束。少这两轮，常见症状是「面积偏大、多出一个小角」。**
- **用 `vector` 当双端队列。`erase(begin())` 是 `O(n)`，整体退化到 $O(n^2)$。用 `std::deque`，或用数组 + 头尾指针手写。**
- **`atan2` 在 $\pm\pi$ 附近不稳。角度接近 π 与 $-\pi$ 的线本是同一方向，排序后却分居两端，去重失效。稳妥写法是不用 atan2：以「上半平面优先、同半平面按叉积」为比较器（`cmp` 先看 `v` 的 `y` 正负，再判 `cross`），代价是多写十行。**
- **三点共线 / 退化成点或线段。`q.size() == 3` 但三条线共点时面积为 0，输出 `0.000` 即可；有的题要求保留 3 位小数，别输出 `-0.000`（加一句 `if (fabs(ans) < 5e-4) ans = 0;`）。**
- **输入多边形是顺时针。半平面取左侧，顺时针多边形的边给出的全是「外侧」，交出来是空集或整个平面的反面。用有向面积判一下，负的就 `reverse`。**
- **平行时调了 `inter`。两条平行线 `cross(a.v, b.v) = 0`，求交会除以 0 得到 `inf/nan`，之后所有 `onLeft` 全为 false，队列被清空。去重之后理论上不会出现平行，但浮点误差下的「近似平行」仍可能溜进来，`inter` 前加一句 `if (fabs(cross(a.v,b.v)) < 1e-12) return a.p;` 兜底。**

- **EPS 用绝对值。坐标 10^6 量级时 `cross` 的量级是 10^{12}，绝对 `1e-9` 等于没设；要么全程 `long double`，要么把 EPS 按坐标量级缩放（`eps = 1e-9 * max(1, |v|²)`）。**
- **以为面积一定是整数。半平面交的顶点是有理数，面积是半整数或无理数；要输出整数必须先确认题目保证，否则按浮点输出。**

自查：本轮把模板逐行译成 Python 后做了三组对拍——① 随机凸包（n ⩽ 8，坐标 ±20）的边转半平面，交面积与原凸包面积比对，300 组全过；② 在①的基础上追加「同角度但内推 0–0.3」的冗余线并打乱顺序，用于专测平行合并，300 组全过（这一组在修正判据前全错）；③ 随机散线 2000 组，其中 269 组有界，与「大盒裁剪（B = 10^7）」的精确面积比对全过，1731 组无界情形正确返回 0。另单测两条线围成的第一象限（无界）与反向平行（空集），均返回面积 0。

31.10 旋转卡壳：凸包直径、宽度与最小外接矩形

一句话判定：问题问的是凸包上的最值，而且这个最值由「两条平行支撑线」决定——最远点对（凸包直径）、最小宽度（两条最近平行支撑线的距离）、最小面积外接矩形、最大面积内接三角形。这类问题有同一个性质：当支撑线沿凸包逆时针转动时，对踵点（与它配对的顶点）也只会前进、不会后退。于是把所有顶点扫一遍的双指针就是 O(n)，这个技巧叫旋转卡壳（rotating calipers）。

三条支撑它成立的事实：

- **对踵点只有 O(n) 对。凸包上「以某条边为支撑线时最远的顶点」随边单调移动，整圈下来每对顶点至多被访问常数次，所以双指针总移动量 O(n)，不是 $O(n^2)$。**
- **单调性是核心，不是技巧。丢掉它（比如直接在原点集上乱扫），指针会来回跳，复杂度退化成 $O(n^2)$ 且答案可能错——因为内部点根本不是候选。**
- **比较要用叉积/点积，不要开方。`cross(e, pⱼ − pᵢ)` 是「点 j 到直线 i→i+1 的有向距离 × |e|」，它随 j 单调，而 `dist` 在三点近共线时会出现平台，用距离比较会让指针停在错误的位置。**

问题	指针数	推进判据	答案
凸包直径（最远点对）	1	`cross(e, pⱼ₊₁−pᵢ) > cross(e, pⱼ−pᵢ)`	各轮 `max(dist²(pᵢ,pⱼ), dist²(pᵢ₊₁,pⱼ))`
凸包最小宽度	1	同上（找最高点）	`min cross(e, pⱼ−pᵢ) / |e|`
最小面积外接矩形	3	j 最高、k 投影最大、l 投影最小	`min（高 × 宽）`，高用叉积、宽用点积差，同除 `|e|`

复杂度：求凸包 Andrew 单调链 `O(n log n)`（瓶颈在排序），卡壳本体 `O(n)`，空间 `O(n)`。粗看只比 $O(n^2)$ 的暴力少一个量级，但 n = 10^5 时是 10^{10} 与 10^5 的差别。

可编译模板

```
#include <bits/stdc++.h>
using namespace std;
const double EPS = 1e-9;

struct Pt {
    double x, y;
    Pt(double x = 0, double y = 0) : x(x), y(y) {}
    Pt operator+(const Pt& o) const { return Pt(x + o.x, y + o.y); }
    Pt operator-(const Pt& o) const { return Pt(x - o.x, y - o.y); }
    bool operator<(const Pt& o) const { return x != o.x ? x < o.x : y < o.y; }
    bool operator==(const Pt& o) const { return fabs(x - o.x) < EPS && fabs(y - o.y) < EPS; }
};
double cross(Pt a, Pt b) { return a.x * b.y - a.y * b.x; }        // 叉积
double dot(Pt a, Pt b)   { return a.x * b.x + a.y * b.y; }        // 点积
double len(Pt a)         { return sqrt(dot(a, a)); }
double dist2(Pt a, Pt b) { return dot(a - b, a - b); }

// Andrew 单调链：返回逆时针凸包，丢掉共线点（用 <= 0 弹栈）
vector<Pt> convexHull(vector<Pt> p) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    int n = p.size();
    if (n <= 2) return p;
    vector<Pt> h(2 * n);
    int k = 0;
    for (int i = 0; i < n; ++i) {                 // 下凸壳
        while (k >= 2 && cross(h[k - 1] - h[k - 2], p[i] - h[k - 2]) <= EPS) --k;
        h[k++] = p[i];
    }
    for (int i = n - 2, t = k + 1; i >= 0; --i) { // 上凸壳
        while (k >= t && cross(h[k - 1] - h[k - 2], p[i] - h[k - 2]) <= EPS) --k;
        h[k++] = p[i];
    }
    h.resize(k - 1);                              // 末点与首点重合，去掉
    return h;
}

// 1) 凸包直径（最远点对距离）
double diameter(const vector<Pt>& h) {
    int n = h.size();
    if (n <= 1) return 0;
    if (n == 2) return len(h[0] - h[1]);
    double best = 0;
    for (int i = 0, j = 1; i < n; ++i) {
        Pt e = h[(i + 1) % n] - h[i];
        while (cross(e, h[(j + 1) % n] - h[i]) > cross(e, h[j] - h[i]))
            j = (j + 1) % n;                      // 严格更远才前进
        best = max(best, max(dist2(h[i], h[j]), dist2(h[(i + 1) % n], h[j])));
    }
    return sqrt(best);
}

// 2) 最小宽度：所有方向里最小的高
double minWidth(const vector<Pt>& h) {
    int n = h.size();
    if (n <= 2) return 0;
    double ans = 1e100;
    for (int i = 0, j = 1; i < n; ++i) {
        Pt e = h[(i + 1) % n] - h[i];
        while (cross(e, h[(j + 1) % n] - h[i]) > cross(e, h[j] - h[i]))
            j = (j + 1) % n;
        ans = min(ans, cross(e, h[j] - h[i]) / len(e));
```

```
    }
    return ans;
}

// 3) 最小面积外接矩形：三指针（最远点 / 投影最大 / 投影最小）
double minAreaRect(const vector<Pt>& h) {
    int n = h.size();
    if (n <= 2) return 0;
    int j = 0, k = 0, l = 0;
    Pt e0 = h[1] - h[0];
    for (int t = 1; t < n; ++t) {                  // 关键：用第一条边把三指针的初值定死
        if (cross(e0, h[t] - h[0]) > cross(e0, h[j] - h[0])) j = t;
        double d = dot(e0, h[t] - h[0]);
        if (d > dot(e0, h[k] - h[0])) k = t;
        if (d < dot(e0, h[l] - h[0])) l = t;
    }
    double ans = 1e100;
    for (int i = 0; i < n; ++i) {
        Pt e = h[(i + 1) % n] - h[i];
        while (cross(e, h[(j + 1) % n] - h[i]) > cross(e, h[j] - h[i])) j = (j + 1) % n;
        while (dot(e, h[(k + 1) % n] - h[i]) > dot(e, h[k] - h[i]))     k = (k + 1) % n;
        while (dot(e, h[(l + 1) % n] - h[i]) < dot(e, h[l] - h[i]))     l = (l + 1) % n;
        double L = len(e);
        double high  = cross(e, h[j] - h[i]) / L;                 // 垂直于 e 的高
        double width = (dot(e, h[k] - h[i]) - dot(e, h[l] - h[i])) / L;
        ans = min(ans, high * width);
    }
    return ans;
}
```

三处典型用法

- **「平面上最远的两点」：直接 `diameter(convexHull(p))`。$n = 10^5$ 时必须这么做；退化成暴力是 $O(n^2)$ 的 10^{10}。**
- **「用两条平行线夹住所有点，最小间距」：`minWidth`。板宽、传送带宽度一类题；注意若要求整数输出，宽度是无理数，须按题意取整方式处理。**
- **「用矩形框住所有点，最小面积 / 最小周长」：`minAreaRect`。最优矩形的某条边一定与凸包某条边共线——这是能用卡壳枚举的根据，枚举方向时只需枚举凸包的边，不必枚举任意角度。**

易错点：

- **没先求凸包就卡壳。内部点会破坏「最远点随边单调」这一前提，指针来回跳，复杂度退化成 $O(n^2)$ 且可能死循环。顺序永远是：去重 → 求凸包 → 卡壳。**
- **凸包退化成 1 个或 2 个点没特判。`minWidth` 与 `minAreaRect` 会拿零长度边做除数，输出 `inf` 或 `nan`；直径在 $n = 1$ 时应为 0、$n = 2$ 时就是两点距离。**
- **推进判据用 `>=`。凸包上常有等高的共线边（例如矩形四个角），用 `>=` 会让指针在等值点之间一路滑到 `j == i`，答案变成 0 或死循环。必须严格 `>`。**
- **用距离推进而不是叉积。$\texttt{dist}^2(p_i, p_j)$ 在「最远点是一整条共线边」时不严格单调，指针停在平台左端，直径偏小。叉积没有这个问题。**

- 整数坐标直接乘叉积。坐标 10^9 时 `cross` 达 10^{18}，`int` / `long long` 都有溢出风险；本模板统一用 `double`，代价是 10^{-9} 量级的相对误差。
- 矩形模板里 `k` 与 `l` 的方向写反。`k` 求投影最大、`l` 求投影最小；写反后 `width` 恒为负，`min` 一直取负值，最后输出 `-inf` 或 0。
- 三指针初值全取同一个下标（本节模板第一版就是这样写的，`j = k = l = 1`）。此时第一条边上 `k` 与 `l` 的投影差恒为 0，`width = 0`，而 `min` 一旦取到 0 就再也回不来——正方形 `(0,0)(2,0)(2,2)(0,2)` 输出 0 而不是 4。必须先用第一条边做一次 `O(n)` 全扫描，把「最远点 / 投影最大 / 投影最小」三个初值分别定准，之后才能靠单调性只前进。
- 面积漏除边长。`cross` 是「距离 × |e|」，`dot` 差也是「长度 × |e|」，两者相乘须除以 L^2，而本模板各除一次 `L` 恰好抵消。写成 `high * width * L` 或只除一次都会错，n 很小时甚至能过大样例。
- 凸包方向搞错。本模板返回逆时针；若自己写成顺时针或在排序时把 `y` 的比较写反，卡壳的「前进方向」也随之反，指针全程不动，答案恒等于第 0 条边的结果——不报错，只错数。
- 共线点全保留。凸包留有共线点时会出现零长度边，除零得到 `nan`；本模板用 `<= EPS` 弹栈去共线，代价是「输出凸包顶点」类题目要另写一版（判据改 `< -EPS`）。
- 拿凸包点数当候选点数输出。有些题问「最远点对是哪两个」，卡壳过程中要额外记录 `i`、`j` 对应的原始下标；凸包重排后下标已变，得在建包时带上原编号。

自查：本轮把四个模板逐行译成 Python，随机点集（$n \leqslant 60$，坐标 $-50\cdots50$，含重复点与人为压成一条直线的大量共线点）共 400 组，分别与暴力对拍——① 直径与 $O(n^2)$ 全点对最大距离一致；② 最小宽度与「枚举每条凸包边、取所有点到该边最大距离的最小值」一致；③ 最小外接矩形与 $O(n^2)$「枚举每条边方向、用该方向与其法向的投影差相乘」一致，最大相对误差 2.2×10^{-16}。修正前第 ③ 项 400 组里错 241 组，正方形退化用例输出 0（应为 4），错因正是三指针初值同点——这条已写进易错点。另用 $n = 10^5$ 的随机点集确认三者在 0.05 s 内完成，验证了 $O(n)$ 的指针总量。

31.11 min-max 容斥：把 max 换成 min

结论：设 S 是有限非空实数集合，则

$$\max(S) = \Sigma_{\varnothing \neq T \subseteq S} (-1)^{|T|+1} \cdot \min(T)$$

把 max 换成「第 k 大」，系数要改：

$$\mathrm{kthmax}(S) = \Sigma_{T \subseteq S,\ |T| \geq k} (-1)^{|T|-k} \cdot C(|T|-1,\ k-1) \cdot \min(T)$$

为什么成立：按「谁是 T 中的最小值」分类统计每个元素的系数。设元素 x 在 S 中有 r 个元素比它小，则 x 作为 $\min(T)$ 被计入的次数是 $\Sigma_j (-1)^{j+1} C(r, j)$（从比它小的元素里选 j 个一起进 T），等于 `[r = 0]`——只有 S 的真正最大值系数为 1，其余全为 0。k 大版本同理，二项式恒等式换成 $\Sigma_j (-1)^j C(r, j) C(j+\cdot, k-1) = [r = |S|-k]$。证明可以只记结论，但「为什么只有最大值留下」这一句要记住，它是判断公式有没有用反的唯一依据。

期望形式才是它真正的用途：两边同时取期望仍然成立，而且不要求元素相互独立——期望的线性性只要求各项期望存在。$E[\max] = \Sigma_{\varnothing \neq T \subseteq S} (-1)^{|T|+1} E[\min(T)]$。这是全节最容易被误解的一条：很多人以为要先证独立性，于是放弃了本该能做的题。反过来说，公式能用不代表好用——只有当 $E[\min(T)]$ 本身容易算时才有价值。

判定条件（三条同时满足才用）：① 要求的量是「全部条件都满足」的时刻，即一组随机变量的 max；② 「至少一个条件满足」的时刻，即 min，有闭式或能 O(1) 算出；③ 变量个数小（$\leqslant 20$），因为要枚举全部子集。反过来，如果 min 比 max 更难算（例如「所有位都为 1」反而好算），这套工具不适用，直接算原问题。

三个标准场景：

- 按位 min-max：每次操作以概率 p_i 把第 i 位置 1，问「所有位都变成 1」的期望操作次数。这里 max 是「最后一位被置 1」，min 是「mask 中最早被置 1 的那一位」，而 $E[\min(\text{mask})] = 1 / \Sigma_{i \in \text{mask}} p_i$（几何分布的期望，仍是 O(1)）。这是 min-max 容斥出现频率最高的形态，近年的省选与 NOI 里反复以变体形式出现。
- 集齐问题（coupon collector）：n 种物品等概率出现，集齐的期望次数。取 $E[\min(T)] = n/|T|$，代入得 $\Sigma_{j=1}^{n} (-1)^{j+1} C(n,j) \cdot n/j = n \cdot H_n$，与经典结论吻合，可用作公式的自检用例。
- 随机游走访问所有点：把「访问到点 i 的时间」当随机变量，min(T) 是「首次进入 T 中任一点」的期望，用高斯消元或树上期望公式算出，再套容斥。

```cpp
#include <bits/stdc++.h>
using namespace std;

// 已知 g[mask] = E[ min_{i in mask} t_i ]（mask 非空，由题目给出）
// 求 E[ max_{i in S} t_i ]，S = 全集。复杂度 O(2^n)
double expectedMax(int n, const vector<double> &g) {
    int full = (1 << n) - 1;
    double ans = 0;
    for (int sub = full; sub; sub = (sub - 1) & full) {
        int pc = __builtin_popcount(sub);
        double sgn = (pc & 1) ? 1.0 : -1.0;      // (-1)^{|T|+1}
        ans += sgn * g[sub];
    }
    return ans;
}

// 第 k 大（k = 1 退化为 max）。同样 O(2^n)，组合数预处理
double kthMax(int n, int k, const vector<double> &g) {
    int full = (1 << n) - 1;
    static double C[32][32];
    for (int i = 0; i <= n; ++i) {
        C[i][0] = 1;
        for (int j = 1; j <= i; ++j) C[i][j] = C[i-1][j-1] + C[i-1][j];
    }
    double ans = 0;
    for (int sub = full; sub; sub = (sub - 1) & full) {
        int pc = __builtin_popcount(sub);
        if (pc < k) continue;                    // |T| < k 无贡献
        double sgn = ((pc - k) & 1) ? -1.0 : 1.0;   // (-1)^{|T|-k}
        ans += sgn * C[pc - 1][k - 1] * g[sub];
    }
    return ans;
}

// 按位 min-max：p[i] = 一次操作把第 i 位置 1 的概率
// g[mask] = 1 / sum_{i in mask} p[i]
double allBitsOn(int n, const vector<double> &p) {
    vector<double> g(1 << n, 0.0);
    for (int mask = 1; mask < (1 << n); ++mask) {
        double s = 0;
        for (int i = 0; i < n; ++i) if (mask >> i & 1) s += p[i];
        g[mask] = 1.0 / s;
    }
    return expectedMax(n, g);
}
```

复杂度与规模：只求全集 S 的 max，只需枚举 S 的子集，$O(2^n)$；若要每个 mask 的 `E[max(mask)]`（例如做子集 DP 的转移），总代价 $\Sigma_{mask} 2^{|mask|} = 3^n$。n = 20 时 $2^{20} \approx 10^6$ 可过，$3^{20} \approx 3.5\times10^9$ 不可过——所以「按位」类题目通常只问全集，或者用「高维前缀和（SOS DP）」把 $O(3^n)$ 压到 $O(n2^n)$。看到 n 在 20 附近而题面要求所有子集的答案，先想 SOS DP，不要硬上 3^n。

易错点：

- **符号整体取反。`(-1)^{|T|+1}` 在 $|T| = 1$ 时为正（第一项就是 min(S)，必须为正）。写成 `(-1)^{|T|}` 会让答案变成 −max，样例若是「只有一个元素」还能对，多个元素立刻错。**
- **漏掉 $|T| = 1$ 项。有人把单项当成平凡情形删掉，等于把 min(S) 从公式里拔掉，结果偏小且无规律。**

- **把空集算进去。** `g[0]` 无定义（min 为空集是 +∞），代码里循环写 `for (sub = full; sub; ...)` 天然跳过 0，若改成 `sub >= 0` 会读未初始化值。
- **以为需要独立性。不需要。但要注意：需要「每个 `E[min(T)]` 已知或可算」，如果元素强相关且 min 的期望本身要解方程，公式仍成立却不能直接用。**
- **把 OR 的期望当 max 的期望。按位模型里 OR 的期望（至少一位为 1）与 max 的期望（全部位为 1）是两个不同的量，前者 `1/Σp_i` 恰好就是 `E[min(全集)]`。若题目问「所有位都达成」，只有 max 才对；反过来把 max 的展开式当 OR 的答案会偏大数倍。**
- **k 大系数写错。`(-1)^{|T|-k}C(|T|-1, k-1)`，`|T| < k` 的项直接无贡献（不是系数为 0 而是不该出现）。k = 1 时必须退化成 max 的式子：`C(|T|-1, 0) = 1`、`(-1)^{|T|-1} = (-1)^{|T|+1}`，可用这条反向验证代码。**
- **浮点相消。n = 20 时 2^{20} 项、最大系数 C(19,9) ≈ 9.2×10^4，正负项量级相近，`double` 可能只剩 6—7 位有效数字。能解析化简的先化简（如集齐问题的 $n\cdot H_n$），必须数值算时用 `long double` 并从 |T| 小的项开始累加。**
- **把 min-max 容斥与「至少/恰好」容斥混为一谈。后者（二项式反演）处理的是「计数时重复计入」，前者处理的是「max 与 min 的转换」，两者的系数形式相似但适用对象完全不同：看到「恰好 k 个」想二项式反演，看到「全部达成的时间」才想 min-max。**

自查：本轮把三个模板译成 Python 后做了两类对拍。① 确定性版本：随机取 n ≤ 8 个实数（含重复值），用 `expectedMax` 的公式算 max、`kthMax` 算第 k 大，与直接排序结果比对 4000 组，k 从 1 到 n 全覆盖，全部一致；② 期望版本：按位模型随机取 p_i（$\Sigma p \le 0.95$），用状态 DP `E[S] = (1 + Σ_{i∈S} p_i E[S\{i}]) / Σ_{i∈S} p_i` 求精确值，与 min-max 展开式比较 700 组（n ≤ 6，$\Sigma p > 0.95$ 的组合已剔除），最大相对误差 4.4×10^{-16}；等概率集齐问题 n = 1…6 分别得 1、3、5.5、8.333、11.417、14.7，与 $n\cdot H_n$ 逐位相同。

31.12 曼哈顿距离与切比雪夫距离：旋转坐标、直径与距离变换

一条恒等式解决一整类题：曼哈顿距离（L_1）与切比雪夫距离（L_∞）在二维下是同一个东西的两种坐标写法。令

```
u = x + y,   v = x - y;          // 旋转 45°，并把长度放大 sqrt(2) 倍
// 则  |x1-x2| + |y1-y2|  ==  max( |u1-u2|, |v1-v2| )
// 逆变换  x = (u + v) / 2,  y = (u - v) / 2   —— 只为保证整数，实务上整体乘 2 而不是除
```

证明只需按符号分四种情况展开：`|dx|+|dy| = max(dx+dy, dx-dy, -dx+dy, -dx-dy)`，而右式四项恰好就是 `max(|du|, |dv|)`。这条恒等式带来三个直接后果，本节的三份模板分别对应它们：

想要什么	旋转后变成什么	代价
点集内两点最大 L_1（直径）	`max(maxU-minU, maxV-minV)`，只需 2 组极值	O(n)
查询点到点集的最远 L_1	4 组投影 `max(s·p)` 减去 `s·q`，必须 4 组 预处理 O(4n)，每次 O(4)	

「到 q 距离 ≤ r」的点数	轴对齐正方形计数：`|u-uq| ≤ r` 且 `|v-vq| ≤ r`	离线扫描线 + 树状数组	O((n+q) log n)

为什么直径只要 2 组、查询要 4 组——这是最容易记混的一处。直径是 $\max_{p,r} L_1(p,r)$，展开后是 $\max_s(\max_p s\cdot p - \min_r s\cdot r)$；符号 `s` 与 `-s` 给出的差值完全相同（只是把 max 与 min 互换），所以 4 组里有一半是重复的。查询时被减数是固定的 `s·q`，`s` 与 `-s` 不再等价，四组必须都算。实测：直径只用 2 组符号跑 500 组随机点集零错误；把查询也压成 2 组，同样 500 组里错 229 组。

模板一：直径与查询（点坐标 $|x|,|y| \le 10^9$ 必须先想溢出）

```
struct Pt { long long x, y; };

// 曼哈顿直径：O(n)，只维护 u/v 的四个极值
long long manhattanDiameter(const vector<Pt> &p) {
    long long uMax = LLONG_MIN, uMin = LLONG_MAX;
    long long vMax = LLONG_MIN, vMin = LLONG_MAX;
    for (const Pt &q : p) {
        long long u = q.x + q.y, v = q.x - q.y;
        uMax = max(uMax, u); uMin = min(uMin, u);
        vMax = max(vMax, v); vMin = min(vMin, v);
    }
    return max(uMax - uMin, vMax - vMin);
}

// 查询：点 q 到点集的最远曼哈顿距离。预处理 4 组投影，单次查询 O(1)
struct FarQuery {
    static constexpr int S[4][2] = {{1,1},{1,-1},{-1,1},{-1,-1}};
    long long best[4];
    void build(const vector<Pt> &p) {
        for (int k = 0; k < 4; ++k) best[k] = LLONG_MIN;
        for (const Pt &r : p)
            for (int k = 0; k < 4; ++k)
                best[k] = max(best[k], S[k][0] * r.x + S[k][1] * r.y);
    }
    long long ask(Pt q) const {
        long long res = 0;
        for (int k = 0; k < 4; ++k)
            res = max(res, best[k] - (S[k][0] * q.x + S[k][1] * q.y));
        return res;
    }
};

// 反过来：切比雪夫最远距离 = 旋转后的曼哈顿最远距离
// (u,v) -> (u+v, u-v) 后 L1 距离恰为 2 * L∞ 距离，需要真实值时除以 2
```

模板二：网格上的曼哈顿距离变换（多源「到最近标记格的距离」，$n\times m \le 10^7$ 时比 BFS 快一个常数，且不用队列）

```cpp
// g[i][j] = 1 表示该格是源。返回每格到最近源的曼哈顿距离
vector<vector<int>> manhattanTransform(const vector<vector<int>> &g) {
    int n = g.size(), m = g[0].size();
    const int INF = 1e9;
    vector<vector<int>> d(n, vector<int>(m, INF));
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < m; ++j)
            if (g[i][j]) d[i][j] = 0;

    // 第一遍：左、上 两个方向
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < m; ++j) {
            if (i) d[i][j] = min(d[i][j], d[i-1][j] + 1);
            if (j) d[i][j] = min(d[i][j], d[i][j-1] + 1);
        }
    // 第二遍：右、下 两个方向（漏了这一遍就只剩「左上方向」的距离）
    for (int i = n - 1; i >= 0; --i)
        for (int j = m - 1; j >= 0; --j) {
            if (i + 1 < n) d[i][j] = min(d[i][j], d[i+1][j] + 1);
            if (j + 1 < m) d[i][j] = min(d[i][j], d[i][j+1] + 1);
        }
    return d;
}
```

这是标准的二维「距离变换」：曼哈顿距离可以拆成「先横后纵」两段，每段都是个一维 DP，所以两遍扫描就够（第一遍处理来自左上方的源，第二遍处理来自右下方的源）。它是精确的，不是近似——因为任意一条最短路都能被拆成「先往左上走一部分、再往右下走一部分」，两遍分别覆盖这两种情况。八邻域版本只需把第二遍的 `+1` 换成对角线也能走（即 `min(d[i+1][j+1]+1, ...)`），此时算的是切比雪夫距离。

模板三：曼哈顿球计数（「到 `(qx,qy)` 距离 ≤ r 的点有多少个」，`n, q ≤ 2×10^5`）

```
struct Pt { long long x, y; };

struct BIT {
    int n; vector<int> t;
    BIT(int n = 0) : n(n), t(n + 1, 0) {}
    void add(int i, int v) { for (; i <= n; i += i & -i) t[i] += v; }
    int sum(int i) { int s = 0; for (; i > 0; i -= i & -i) s += t[i]; return s; }
};

// 询问 (qx, qy, r)：曼哈顿距离 <= r 的点数
// 旋转后 |u-uq| <= r 且 |v-vq| <= r，uq = qx+qy, vq = qx-qy
struct Ev { long long u, vl, vr; int id, sign; };

vector<int> ballCount(const vector<Pt> &pt, const vector<array<long long,3>> &ask) {
    int n = pt.size(), q = ask.size();
    vector<pair<long long,int>> ord(n);
    vector<long long> u(n), v(n), vy;
    vy.reserve(2 * n + 2 * q);
    for (int i = 0; i < n; ++i) {
        u[i] = pt[i].x + pt[i].y; v[i] = pt[i].x - pt[i].y;
        ord[i] = {u[i], i}; vy.push_back(v[i]);
    }
    sort(ord.begin(), ord.end());

    vector<Ev> ev;
    for (int i = 0; i < q; ++i) {
        long long uq = ask[i][0] + ask[i][1], vq = ask[i][0] - ask[i][1], r = ask[i][2];
        ev.push_back({uq + r, vq - r, vq + r, i, +1});
        ev.push_back({uq - r - 1, vq - r, vq + r, i, -1});   // 前缀和相减
        vy.push_back(vq - r); vy.push_back(vq + r);
    }
    sort(vy.begin(), vy.end());
    vy.erase(unique(vy.begin(), vy.end()), vy.end());
    sort(ev.begin(), ev.end(), [](const Ev &a, const Ev &b) { return a.u < b.u; });

    BIT bit(vy.size());
    vector<int> ans(q, 0);
    int ptr = 0;
    for (const Ev &e : ev) {
        while (ptr < n && ord[ptr].first <= e.u) {
            int id = ord[ptr].second;
            int pos = lower_bound(vy.begin(), vy.end(), v[id]) - vy.begin() + 1;
            bit.add(pos, 1); ++ptr;
        }
        int l = lower_bound(vy.begin(), vy.end(), e.vl) - vy.begin() + 1;
        int r = upper_bound(vy.begin(), vy.end(), e.vr) - vy.begin();
        if (r >= l) ans[e.id] += e.sign * (bit.sum(r) - bit.sum(l - 1));
    }
    return ans;
}
```

曼哈顿最小生成树只需一句提醒：旋转不改变边权排序，所以「按旋转后坐标做 L_∞ 完全图 Kruskal」在数学上是对的，但边数是 $O(n^2)$，$n = 10^5$ 必死。正解是「八分区域最近邻」——对点集做 8 种变换（`(x,y)`, `(y,x)`, `(-x,y)`, `(y,-x)` 与坐标取反的组合），每种变换下只保留「在 `x' ≥ x` 且 `x'+y'` 最小的那个邻居」，用树状数组按 `x'+y'` 做后缀最小即可 $O(n \log n)$ 求出每个点的最近候选，再把这 $O(8n)$ 条边丢给 Kruskal。这一步的常数与离散化细节比上面三份模板都重，比赛里若 $n \le 5000$，直接 $O(n^2)$ 枚举更稳。

易错点：

- **查询最远只枚举两组符号。直径可以省成 2 组，查询不行；两者混用是本节最高频的错误，500 组随机数据错 229 组，且错误答案通常只偏小一点，样例往往还能过。**
- **距离变换只做一遍扫描。只留「左、上」那一遍，等于假设所有源都在查询点的左上方：200 组随机网格（密度 0.12）里错 194 组，且错的格子集中在右下角。**
- **把 L_1 球当矩形。曼哈顿球是菱形，直接拿 `[qx-r, qx+r] × [qy-r, qy+r]` 做二维前缀和会把四个角多算进去（多算的正是 `|dx|+|dy| > r` 的那些点）；必须先在旋转后的坐标系里计数。**
- **旋转后忘了边长是 2r。条件 `|u-uq| ≤ r` 对应的是边长 `2r` 的正方形，写 `< r` 会漏掉边界上距离恰为 r 的点，写 `≤ 2r` 会多算一圈。**
- **切比雪夫转曼哈顿忘了乘 2。`(u,v) → (u+v, u-v)` 后 L_1 距离是原 L_∞ 距离的 2 倍，报答案前要除 2；反过来曼哈顿转切比雪夫时若坐标出现半整数（`(x+y)/2`），统一先把所有坐标乘 2 再算，别用 `double`。**
- **用 `int` 存 `u = x + y`。`|x|, |y| ≤ 10^9` 时 `u` 到 `2×10^9`，已超 `int`；再在 `uMax - uMin` 处相减，`int` 会先溢出成负数再参与 `max`，答案莫名偏小。全程 `long long`。**
- **维度推广时忘了 2^k。k 维 L_1 直径要枚举 2^k 组符号（每组算 `n` 个点的一次点积），k = 4 时 16 组、k = 10 时 1024 组；k 大而 n 也大时要先降维或改用其他结构，别硬枚。**
- **把「旋转后最远」当「原坐标最远」去取点。极值点是在 `u` 或 `v` 上取的，要输出具体是哪两个点，得同时记下标再回代原坐标算距离；直接用 `uMax - uMin` 当坐标差会得到 2 倍误差的值。**
- **网格里混用 L_1 与八邻域。四邻域 BFS 是曼哈顿距离，八邻域 BFS 是切比雪夫距离；「国王走法」类题两遍扫描也要相应改对角线，改了一半会得到既非 L_1 也非 L_∞ 的怪距离。**
- **距离变换的无穷大取小了。`INF` 必须大于 `n+m`，取 `1e9` 安全；若图里一格源都没有，答案全是 `INF`，题目若要求输出 `-1` 要在最后统一替换，而不是让 `INF + 1` 溢出。**

自查：本轮把三份模板译成 Python 后做了五组对拍。① 直径：500 组随机点集（n ≤ 60，坐标 ±50）与 $O(n^2)$ 暴力完全一致；② 旋转恒等式：4000 组随机点对逐一验证 `|dx|+|dy| = max(|du|,|dv|)`，零反例；③ 查询最远：4 组符号版本 500 组零错，压成 2 组的版本错 229 组；④ 距离变换：200 组随机网格（≤18×18，源密度 0.15）与多源 BFS 逐格相同，只做正向一遍的版本错 194 组；⑤ 球计数：600 组（n ≤ 80，r ≤ 20）用「旋转 + 矩形判定」与直接枚举 `|dx|+|dy| ≤ r` 比对，零错。C++ 侧无编译器，只做语法与上界自查（`long long` 贯穿、`vy` 预留 `2n+2q`、树状数组下标 1-based）。

附录 A　资料分布统计

> 统计口径为「标签出现次数」，一条资料可带多个标签，故各行之和大于总数 431。

按竞赛类别

通用	16		4%
CSP-J	116		27%
CSP-S	173		40%
NOIP	156		36%
省选	185		43%
NOI	193		45%
IOI	56		13%

按任务需求

入门自修	53		12%
体系学习	70		16%
专题突破	158		37%
赛前冲刺	76		18%
教练备课	79		18%
查漏补缺	90		21%
实战训练	94		22%

按作用效果

速查	97		23%
深耕	179		42%
实战	140		32%
方法论	68		16%

附录 B　编号速查表

> 按编号顺序排列，供快速定位与勾选。可单独打印作为进度表。

第 5 章　教材与专著

001	48 课搞定信息学奥赛：C++ 趣味编程	CSP-J
002	C++编程与信息学竞赛数学思维	CSP-J/CSP-S
003	CCF CSP 例题解析与实战技巧：编程基础	CSP-J/CSP-S
004	CCF CSP 第一轮认证一本通（第 2 版）	CSP-J/CSP-S
005	CCF GESP 直通车：C++ 六级七级精讲精练	CSP-J
006	CCF 中学生计算机程序设计	CSP-J
007	CCF 中学生计算机程序设计	CSP-J/CSP-S
008	CCF 信息学奥赛基础篇	CSP-J/CSP-S
009	Cracking the Coding Interview	CSP-J/CSP-S
010	Think Like a Programmer	CSP-J/CSP-S
011	中学生计算机程序算法入门：例题精析与训练	CSP-J
012	信奥逐梦之路：家长陪伴与升学规划全阶段指南	CSP-J/CSP-S/NOIP
013	信息学奥赛 CSP-J/S 数学考点大全	CSP-J/CSP-S
014	信息学奥赛1：114道题零基础吃透C++与算法基础	CSP-J
015	信息学奥赛2：121道题零基础吃透高级算法与数据结构	CSP-J
016	信息学奥赛一本通	CSP-J
017	信息学奥赛一本通	CSP-J/CSP-S
018	信息学奥赛初赛全解	CSP-J/CSP-S
019	信息学奥赛大通关·入门篇（上下册）	CSP-J
020	信息学奥赛打通关 · 入门篇（全 2 册）	CSP-J
021	信息学奥赛课课通	CSP-J/CSP-S
022	信息学竞赛教程（初级）	CSP-J/CSP-S
023	啊哈！算法	CSP-J
024	我的第一本算法书	CSP-J
025	我的第一本算法竞赛启蒙书	CSP-J
026	深入浅出程序设计竞赛	CSP-J/CSP-S
027	用 AI 决胜信息学奥赛：C++ 编程基础	CSP-J
028	算法与数学：数学思维与编程实践	CSP-J/CSP-S
029	算法图解	CSP-J
030	算法竞赛从新手到高手	CSP-J/CSP-S
031	算法竞赛入门到进阶	CSP-J/CSP-S
032	算法竞赛入门经典	CSP-J/CSP-S
033	算法竞赛试炼场：洛谷300题精析	CSP-J/CSP-S
034	算法竞赛黄金法则：提高算法和思考力的77项技巧	CSP-J/CSP-S/NOIP
035	算法设计与分析（图解版）	CSP-J/CSP-S
036	聪明人的游戏：信息学探秘（提高篇）	CSP-J/CSP-S
037	计算概论：C++ 编程与信息学竞赛入门	CSP-J
038	高效通关信息学竞赛·初赛篇（入门组）	CSP-J
039	Algorithms Unlocked	CSP-S/NOIP
040	Algorithms, 4th Edition（Sedgewick）	CSP-S/NOIP/省选
041	C++ Primer	CSP-S/NOIP
042	CCF CSP 例题解析与实战技巧：工程实现	CSP-S
043	CCF 中学生计算机程序设计	CSP-S/NOIP
044	CCF 信息学奥赛提高篇	CSP-S/NOIP
045	Competitive Programmer's Handbook	CSP-S/NOIP
046	Competitive Programming in Python	CSP-S
047	Elements of Programming Interviews	CSP-S/NOIP
048	Guide to Competitive Programming	CSP-S/NOIP
049	Guide to Competitive Programming	CSP-S/NOIP/省选
050	信息学奥赛一本通 · 提高篇	CSP-S/NOIP
051	信息学奥赛之数学一本通	CSP-S/NOIP
052	挑战程序设计竞赛	CSP-S/NOIP/省选
053	数据结构与算法分析	CSP-S/NOIP
054	数据结构（C++ 语言版）	CSP-S/NOIP/省选
055	程序设计竞赛进阶：算法与实践	CSP-S/NOIP/省选

056 算法导论（核心篇）（原书第 4 版） CSP-S/NOIP/省选/NOI

057 算法思维训练：信奥大师的算法教与学秘籍 CSP-S/NOIP/省选

058 算法竞赛实战笔记 CSP-S/NOIP

059 算法竞赛核心教程：策略与算法深度解析 CSP-S/NOIP/省选

060 算法竞赛进阶指南 CSP-S/NOIP/省选

061 算法训练营：进阶篇（全彩版） CSP-S/NOIP/省选/NOI

062 算法设计与分析 CSP-S/NOIP

063 算法设计与分析：以 ACM 大学生程序设计竞赛在线题库为例（第 2 版·微课版） CSP-S/NOIP/省选

064 算法（第 4 版） CSP-S/NOIP

065 计算思维训练——动态规划 CSP-S/NOIP

066 ACM 国际大学生程序设计竞赛系列 · 知识与入门 NOIP/省选

067 Algorithms（Dasgupta - Papadimitriou - Vazirani） NOIP/省选/NOI

068 Algorithms（Jeff Erickson） NOIP/省选/NOI/IOI

069 C++，挑战编程——程序设计竞赛进阶训练指南 NOIP/省选

070 Competitive Programming 4（配套代码库） NOIP/省选/NOI

071 Mathematics for Computer Science NOIP/省选/NOI

072 Programming Pearls NOIP/省选/NOI

073 算法详解（Algorithms Illuminated，卷 1—4） NOIP/省选/NOI

074 编程珠玑 NOIP/省选

075 ACM 国际大学生程序设计竞赛系列 · 算法与实现 省选/NOI

076 Algorithm Design（Kleinberg - Tardos） 省选/NOI/IOI

077 Competitive Programming 4 省选/NOI/IOI

078 Introduction to Graph Theory（West） 省选/NOI/IOI

079 Introduction to Probability 省选/NOI

080 The Algorithm Design Manual 省选/NOI/IOI

081 信息学竞赛中的数学 省选/NOI

082 具体数学 省选/NOI

083 挑战程序设计竞赛 2 省选/NOI

084 数据结构与算法（上下册） 省选/NOI

085 离散数学及其应用 省选/NOI

086 算法导论 省选/NOI/IOI

087 算法导论（扩展篇）（原书第 4 版） 省选/NOI

088 算法竞赛中的数论 省选/NOI

089 算法艺术与信息学竞赛 省选/NOI

090 算法设计手册 省选/NOI

091 组合数学 省选/NOI

092 Graph Theory（Bondy & Murty） NOI/IOI

093 Handbook of Data Structures and Applications（2nd Edition） NOI/IOI

094 Linear Algebra Done Right NOI/IOI

095 Structure and Interpretation of Computer Programs NOI/IOI

096 概率论与数理统计 NOI

097 线性代数及其应用 NOI

098 编程原本 NOI

099 计算几何：算法与应用 NOI

100 ACM Transactions on Algorithms（TALG） IOI

101 Algorithmica IOI

102 Journal of Graph Algorithms and Applications（JGAA） IOI

103 计算机程序设计艺术 IOI

第 6 章 官方文档与竞赛规则

104 CCF NOI 系列活动考场纪律 通用

105 NOI 大纲（2025 年修订版） 通用

106 NOI 官网 通用

107 NOI 系列活动和认证日历 通用

108 《中国计算机学会通讯》（CCCF） 通用

109 《信息学奥林匹克竞赛年鉴》 通用

110 2024 年 NOI 活动题目分析报告（CSP-J/S 2024 第二轮 + NOIP 2024 知识构成） CSP-J/CSP-S/NOIP

111 2026 年 WCET & LMCC & NOI 教师发展活动日历 CSP-J/NOIP

112 CCF CSP 认证官方说明 CSP-J/CSP-S

113 CCF CSP 认证官网 CSP-J/CSP-S

114 CCF NOI 教师培训考核管理规则 CSP-J/NOIP/NOI

115 CCF 信息学奥赛入门篇 CSP-J

116 CCF 关于 CSP-J/S 2026 第一轮晋级第二轮的规定 CSP-J/CSP-S

117 CCF 关于 CSP-J/S 2026 第一轮认证评级分数线的公告 CSP-J/CSP-S

118 CCF 关于 NOI 相关权益的声明 CSP-J/CSP-S/NOIP/NOI

119 CSP-J/S 2026 第一轮认证证书申请通知 CSP-J/CSP-S

120 CSP-J/S 2026 第二轮报名通知 CSP-J/CSP-S

121 CSP-J/S 认证须知 CSP-J/CSP-S

122 NOI Linux 2.0 官方发布页 CSP-J/CSP-S/NOIP/省选/NOI

123 USACO 官网 CSP-J/CSP-S/NOIP

124 关于 CCF CSP-J/S 认证等级名称规范的公告 CSP-J/CSP-S

125 关于 NOI 系列活动中编程语言使用限制的补充说明 CSP-J/CSP-S/NOIP/NOI

126 关于参加 CSP-J/S 认证需符合年龄条件的公告（12 周岁门槛） CSP-J/CSP-S/NOIP

127 关于重申在校在职 NOI 指导教师不得校外有偿兼职授课的公告 CSP-J/CSP-S/NOIP/NOI

128 电子学会青少年等级考试 CSP-J

129 蓝桥杯 CSP-J

130 2025 年 CSP-J/S 第二轮认证题目分析报告 CSP-S/CSP-J

131 CCF 全国青少年信息学奥林匹克竞赛教程 CSP-S/NOIP/省选

132 COCI CSP-S/NOIP/省选

133 NOIP 2026 参赛名额分配方案 CSP-S/NOIP

134 2023 全国信息学奥林匹克年鉴（电子版） NOIP/省选/NOI

135 CCF 关于 NOIP 2026 报名的通知 NOIP

136 JOI NOIP/省选

137 NOIP 评奖规则 NOIP

138 NOIP 题目综合分析报告 NOIP

139 第六届女生竞赛报名通知（NOIP 2026 女生赛） NOIP/NOI

140 2022—2023 年国内外信息学主要赛事分析报告 省选/NOI

141 APIO 2026 中国区活动赛事综述与分数线 省选/NOI

142 CCF 关于 NOI 2027 省内选拔的若干规定 省选/NOI

143 CCPC 官网 省选/NOI

144 ICPC 官网 省选/NOI

145 NOI 2027 名额分配测算方案 省选/NOI

146 NOI 2027 联合省选：两试时间与成绩构成对照（26 省市） 省选

147 2024 全国信息学奥林匹克年鉴 NOI/省选

148 APIO 官网 NOI

149 Baltic OI NOI

150 CCF NOI 集训营 2026 综合素质评价机制（附国家集训队行为规范） NOI/IOI

151 CEOI NOI/IOI

152 IOI 2027 中国国家队选拔规则与国家集训队行为规范 NOI/IOI

153 IOI Syllabus NOI/IOI

154 NOI 冬令营 2026 首次 AI 辅助编程测试（人机协同评测） NOI/IOI

155 NOI 参赛资格规定 NOI

156 NOI 试题分析与年鉴 NOI

157 POI NOI

158 USACO 2025-2026 赛季赛制改革：三场线上赛 + 监考制 US Open NOI/IOI

159 关于自 NOI 2029 起取消 A 类选手加分政策的公告 NOI/省选

160 IOI 2026 中国队 3 金 1 银与最终排名 IOI/NOI

161 IOI 2027 官方网站（第 39 届，德国波茨坦 HPI，2027-09-12—19） IOI

162 IOI 官网 IOI

163 IOI 年度报告与命题趋势 IOI

第 7 章　在线平台与题库

164 HydroOJ 通用

165 AcWing CSP-J/CSP-S/NOIP/省选

166 Aizu Online Judge CSP-J/CSP-S

167 Algorithm Visualizer CSP-J/CSP-S

168 AtCoder CSP-J/CSP-S/NOIP/省选

169 HackerRank CSP-J/CSP-S

170 LeetCode CSP-J

171	OpenJudge	CSP-J
172	PTA 拼题 A	CSP-J/CSP-S
173	USACO	CSP-J/CSP-S/NOIP
174	USFCA 数据结构可视化	CSP-J/CSP-S/NOIP
175	Vijos	CSP-J/CSP-S
176	VisuAlgo	CSP-J/CSP-S/NOIP
177	WZOI（温州中学 OJ）	CSP-J/CSP-S
178	一本通在线评测（ybt）	CSP-J/CSP-S
179	梦熊 OJ（Mengxiong Online Judge）	CSP-J/CSP-S/NOIP
180	洛谷	CSP-J/CSP-S/NOIP/省选
181	洛谷主题题库	CSP-J/CSP-S/NOIP/省选/NOI
182	洛谷公开赛与月赛	CSP-J/CSP-S/NOIP/省选
183	洛谷题单	CSP-J/CSP-S/NOIP
184	编程猫 OJ（北大—点猫人工智能教育联合实验室）	CSP-J/CSP-S/NOIP/NOI
185	蓝桥云课	CSP-J/CSP-S
186	计蒜客	CSP-J/CSP-S
187	51Nod	CSP-S/NOIP
188	AtCoder Problems	CSP-S/NOIP/省选/NOI
189	Baekjoon Online Judge	CSP-S/NOIP
190	CSES Problem Set	CSP-S/NOIP
191	CodeChef	CSP-S/NOIP
192	Codeforces	CSP-S/NOIP/省选/NOI
193	Codeforces 题目集（按 rating 筛选）	CSP-S/NOIP/省选/NOI
194	Comet OJ	CSP-S/NOIP
195	DMOJ	CSP-S/NOIP
196	HDU Online Judge	CSP-S/NOIP
197	Lutece	CSP-S
198	POJ	CSP-S/NOIP
199	SPOJ	CSP-S/NOIP
200	UVa Online Judge	CSP-S/NOIP
201	ZOJ	CSP-S
202	e-olymp	CSP-S/NOIP
203	牛客网	CSP-S/NOIP
204	Codeforces Gym	NOIP/省选/NOI/IOI
205	Judge Duck Online	NOIP/省选
206	Kattis	NOIP/省选
207	Timus	NOIP/省选
208	LibreOJ	省选/NOI
209	Nowcoder 多校训练营	省选/NOI
210	OEIS 整数数列百科	省选/NOI/IOI
211	QOJ	省选/NOI/IOI
212	UOJ	省选/NOI
213	Library Checker	NOI
214	Universal Cup	NOI/IOI
215	oj.uz	NOI/IOI

第 8 章　集训队论文与专题讲义

216	信息学竞赛中的思维方法	通用
217	测试数据设计中的三要素	通用
218	ISSEP 2026 论文集：AI 时代的中小学信息学教育	CSP-J/CSP-S
219	OI Wiki 学习路线（C++ 基础 → CSP-J 入门级知识清单）	CSP-J/CSP-S
220	Legend-K 的数据结构与算法笔记	CSP-S/NOIP/省选
221	搜索方法中的剪枝优化	CSP-S/NOIP
222	搜索顺序的选择	CSP-S/NOIP
223	背包九讲	CSP-S/NOIP
224	骗分导论	CSP-S/NOIP
225	字符串算法选讲	NOIP/省选
226	常数优化与卡常技巧	NOIP/省选
227	浅析非完美算法在信息学竞赛中的应用	NOIP/省选
228	浅谈用极大化思想解决最大子矩形问题	NOIP/省选
229	矩阵乘法与线性递推	NOIP/省选
230	NOIpaper：国家集训队论文集（1999—2025）	省选/NOI/IOI
231	OI 论文集精读	省选/NOI/IOI
232	Trinkle23897 的 OI 课件	省选/NOI
233	acm-cheat-sheet	省选/NOI
234	huzecong 的 OI 课件	省选/NOI
235	分块与莫队算法	省选/NOI
236	动态规划优化专题	省选/NOI
237	动态规划的深入讨论	省选/NOI
238	动态规划算法的优化技巧	省选/NOI
239	图论建模专题	省选/NOI
240	强连通分量与 2-SAT	省选/NOI

241 把握本质，灵活运用——动态规划的深入探讨 省选/NOI

242 数学模型的建立、比较和应用 省选/NOI

243 树型动态规划 省选/NOI

244 状态压缩动态规划 省选/NOI

245 离线处理思想 省选/NOI

246 线段树的应用 省选/NOI

247 组合游戏略述——浅谈 SG 游戏的若干拓展及变形 省选/NOI

248 网络流算法的应用 省选/NOI

249 解决动态统计问题的两把利刃 省选/NOI

250 Pólya 原理及其应用 NOI

251 伸展树的基本操作与应用 NOI

252 充分利用问题性质——例析动态规划的「个性化」优化 NOI

253 具体数学讲义 NOI

254 动态树及其应用 NOI/IOI

255 可持久化数据结构 NOI

256 后缀数组 NOI

257 后缀数组——处理字符串的有力工具 NOI

258 多串匹配算法及其启示 NOI

259 容斥原理与反演 NOI

260 左偏树的特点及其应用 NOI

261 平面图在信息学中的应用 NOI

262 提交答案题与交互题 NOI/IOI

263 数值分析在竞赛中的简单应用 NOI

264 数据结构的联合——块状链表 NOI

265 最小割模型在信息学竞赛中的应用 NOI

266 构造题策略 NOI

267 母函数的性质及应用 NOI

268 猜数问题的研究 NOI

269 答案只有一个——浅谈问答式交互问题 NOI/IOI

270 维护森林连通性——动态树 NOI/IOI

271 计算几何讲义 NOI

272 论随机化算法的原理与设计 NOI

第 9 章　历年真题与题解

273 CCF NOI 2026 笔试题库 CSP-J/CSP-S/NOI

274 CSP-J/S 2025 题目及数据（入门组 4 题 + 提高组 4 题） CSP-J/CSP-S

275 CSP-J/S 2026 第二轮认证测试数据（入门组 + 提高组） CSP-J/CSP-S

276 CSP-J/S 历年认证题 CSP-J/CSP-S

277 CSP-J/S 第一轮真题 CSP-J/CSP-S

278 JOI 2026/2027 一次预选（第 26 回日本信息学奥赛第一轮）题目・解说・解答例 CSP-J/CSP-S

279 USACO 历年试题 CSP-J/CSP-S/NOIP

280 《CCF 信息学奥赛入门篇》练习指导 CSP-J

281 信息学奥赛 3：历年真题解析与拉分解题思维训练 CSP-J

282 信息学奥赛 CSP 通关之路：CSP-J/S 第一轮原创全真模拟试卷集（2026） CSP-J/CSP-S

283 信息学奥赛 CSP-J 初赛通关手册：10 年真题 + 10 套模拟精练精讲 CSP-J

284 信息学奥赛一本通 CSP-J/CSP-S

285 洛谷主题题单 CSP-J/CSP-S/NOIP/省选

286 算法竞赛入门经典——习题与解答 CSP-J/CSP-S

287 算法竞赛宝典 CSP-J/CSP-S

288 背包问题专题 CSP-J/CSP-S/NOIP

289 BalkOI 2026 试题与在线镜像（巴尔干信息学奥林匹克，地拉那） CSP-S/NOIP/省选

290 CCF CSP 例题解析与实战技巧：算法应用 CSP-S/NOIP

291 Programming Challenges CSP-S/NOIP

292 eJOI 2026 试题・题解・数据（欧洲初中生信息学奥林匹克，立陶宛考纳斯） CSP-S/NOIP

293 信息学奥赛一本通 · 高手训练 CSP-S/NOIP/省选

294 区间与序列 DP 专题 CSP-S/NOIP/省选

295 单调栈与单调队列 CSP-S/NOIP/省选

296 字符串哈希与 KMP CSP-S/NOIP/省选

297 树形 DP 专题 CSP-S/NOIP/省选

298 树状数组 CSP-S/NOIP/省选

299 算法思维：竞赛真题精选精讲（第 2 版） CSP-S/NOIP/省选

300 算法竞赛入门经典——训练指南 CSP-S/NOIP

301 JOI 2025/2026 二次预选（第 25 回日本信息学奥赛第二轮）题目・解说・数据 NOIP/省选

302 NOIP 2025 题目及数据（糖果店 / 清仓甩卖 / 树的价值 / 序列询问） NOIP/省选

303 NOIP 历年真题（2011 起） NOIP

304 NOIP 历年试题 NOIP
305 二分图匹配（匈牙利 / Hopcroft-Karp / KM） NOIP/省选/NOI
306 博弈论与 SG 函数 NOIP/省选/NOI
431 数位 DP 专题 NOIP/省选
308 状压 DP 专题 NOIP/省选/NOI
309 矩阵与线性递推 NOIP/省选
310 线段树与懒标记 NOIP/省选/NOI
311 2-SAT 与差分约束 省选/NOI
312 AC 自动机 省选/NOI
313 ACM 国际大学生程序设计竞赛系列 · 题目与解读 省选/NOI
314 AHOI 安徽省选 省选
315 APIO 2026 中国区活动题目及数据 省选/NOI
316 BJOI 北京队选拔 省选/NOI
317 BOI 试题库与 2026 年题目（波罗的海信息学奥林匹克，1995—2026 全量题面、数据与题解） 省选/NOI
318 CEOI 2026 试题与数据（第 33 届中欧信息学奥林匹克，斯洛文尼亚卢布尔雅那） 省选/NOI
319 DP 优化专题（单调队列/斜率/决策单调性） 省选/NOI
320 FJOI 福建省选 省选
321 GDOI 广东省选 省选
322 HAOI 河南省选 省选
323 HEOI 河北省选 省选
324 HNOI 湖南省选 省选/NOI
325 JSOI 江苏省选 省选
326 Manacher 与回文自动机 省选/NOI
327 SCOI 四川省选 省选/NOI
328 SDOI 山东省选 省选/NOI
329 SHOI 上海队选拔 省选/NOI
330 WEOI 2026 试题与题解（第 4 届西欧信息学奥林匹克，卢森堡） 省选/NOI
331 ZJOI 浙江省选 省选/NOI
332 十二省联考 省选/NOI
333 可持久化数据结构 省选/NOI
334 后缀数组与后缀自动机 省选/NOI/IOI
335 国家集训队论文配套题 省选/NOI
336 多项式与 NTT 省选/NOI
337 平衡树（Treap / Splay / FHQ） 省选/NOI
338 插头 DP（连通性状态压缩） 省选/NOI
339 树链剖分（HLD） 省选/NOI
340 网络流（最大流 / 最小割 / 费用流） 省选/NOI/IOI
341 联合省选（CCF 统一命题） 省选
342 莫比乌斯反演与杜教筛 省选/NOI
343 虚树 省选/NOI
344 计算几何基础与凸包 省选/NOI
345 APIO 历年试题 NOI
346 EGOI 2026 试题与逐题统计（第 6 届欧洲女子信息学奥林匹克） NOI/IOI
347 IOI 2025 官方赛题包（ioi/task-archive 2025，6 题全量数据） NOI/IOI
348 IOI 2026 中国国家集训队集中培训与精英培训题目及数据 NOI/IOI
349 IOI 2026 试题与题解（第 38 届国际信息学奥林匹克，塔什干） NOI/IOI
350 Min_25 筛与洲阁筛 NOI
351 NOI 2026 题目及数据 NOI
352 NOI 冬令营 2026 题目及数据 NOI/省选
353 NOI 历年真题 NOI
354 NOI 历年试题 NOI
355 生成函数与组合计数 NOI/IOI
356 IOI 历年试题 IOI

第 10 章 课程与视频

357 OI Wiki 专题页 通用
358 AcWing 算法基础课 CSP-J/CSP-S
359 USACO Guide CSP-J/CSP-S/NOIP/省选
360 USACO Training Gateway CSP-J/CSP-S
361 中国大学 MOOC CSP-J/CSP-S/NOIP
362 代码随想录 CSP-J/CSP-S
363 洛谷网校 CSP-J/CSP-S/NOIP
364 董晓算法（B 站） CSP-J/CSP-S/NOIP
365 2026 年第四期 NOI 教师培训（南京站，中级/初级） CSP-S/NOIP/省选
366 AcWing 算法提高课 CSP-S/NOIP/省选
367 Berkeley CS61B（数据结构） CSP-S/NOIP/省选
368 CS 97SI（斯坦福竞赛编程） CSP-S/NOIP
369 Codeforces EDU CSP-S/NOIP

370 Codeforces EDU（算法教程） CSP-S/NOIP/省选
371 Princeton Algorithms（Coursera） CSP-S/NOIP/省选
372 UCF COP 4516 Contest Problem Solving（中佛罗里达大学竞赛解题课） CSP-S/NOIP/省选
373 中国大学 MOOC《算法设计与分析》（北大） CSP-S/NOIP/省选
374 学堂在线 CSP-S/NOIP/省选
375 数据结构（清华） CSP-S/NOIP
376 灵茶山艾府（B 站 / 题解） CSP-S/NOIP/省选
377 AtCoder Library Practice NOIP/省选
378 MIT OCW 6.006（算法导论） NOIP/省选/NOI
379 NOI 2025 冬令营授课与交流活动现场视频（14 场） NOIP/省选/NOI
380 算法设计与分析（MIT 6.006） NOIP/省选
381 APIO 2025 讲者授课现场视频（5 场） 省选/NOI/IOI
382 APIO 2026 中国区活动讲座现场视频（7 场） 省选/NOI/IOI
383 AcWing 算法进阶课 省选/NOI
384 CS 3233（NUS 竞赛编程） 省选/NOI
385 CCF NOI 冬令营 2026 第一课堂（CCF 数字图书馆） NOI/省选
386 CCF NOI 集训营 2026 报到通知（附日程） NOI/IOI
387 CCF 关于举办 IOI 2027 精英培训的通知 NOI/IOI
388 CMU 15-295 Competition Programming and Problem Solving NOI/IOI
389 CMU 15-451（算法设计与分析） NOI/IOI
390 NOI 冬令营 2026 讲座与交流活动现场视频（14 场） NOI/IOI
391 Stanford CS161（算法设计与分析） NOI/IOI
392 蒋炎岩（JYY）课程站 NOI/IOI
393 IOI 官方 YouTube 频道 IOI

第 11 章 工具与环境

394 Arbiter 通用
395 Cena 通用
396 DOMjudge 通用
397 Lemon 通用
398 CP Editor CSP-J/CSP-S
399 Code::Blocks CSP-J/CSP-S
400 Dev-C++ CSP-J
401 LOJ — LeLe Online Judge（开源可自部署的 ACM/OI 双赛制评测系统） CSP-J/CSP-S/NOIP
402 Oracle VM VirtualBox CSP-J/CSP-S/NOIP/省选
403 VMware Workstation Player CSP-J/CSP-S/NOIP/省选
404 洛谷 IDE CSP-J/CSP-S
405 Compiler Explorer (godbolt) CSP-S/NOIP/省选/NOI
406 Desmos 图形计算器 CSP-S/NOIP/省选
407 GeoGebra CSP-S/NOIP/省选
408 NOI Linux 2.0 CSP-S/NOIP/NOI
409 Visual Studio Code CSP-S/NOIP/省选/NOI
410 对拍脚本 CSP-S/NOIP/省选
411 GDB NOIP/省选/NOI
412 CF Polygon（出题系统） 省选/NOI
413 CLion 省选/NOI
414 Wolfram Alpha 省选/NOI
415 testlib.h 省选/NOI
416 Valgrind NOI
417 perf NOI

第 12 章 社区与期刊

418 OI Wiki 通用
419 Stack Overflow 通用
420 GeeksforGeeks CSP-J/CSP-S
421 OI Wiki 英文镜像 / 社区分支 CSP-J/CSP-S/NOIP/省选/NOI
422 洛谷讨论区 CSP-J/CSP-S/NOIP/省选
423 Codeforces Blog（题解与教程） CSP-S/NOIP/省选/NOI
424 演算法筆記 CSP-S/NOIP
425 Matrix67 博客 NOIP/省选/NOI
426 cp-algorithms NOIP/省选/NOI
427 hzwer 的博客 NOIP/省选/NOI
428 Computer Science StackExchange 省选/NOI
429 Mathematics StackExchange 省选/NOI/IOI

430 **《Olympiads in Informatics》第 20 卷（IOI 学术期刊，2026）** NOI/IOI

431 **许淇文 IOI 2026 参赛总结** NOI/IOI

附录 C　版本记录与致谢

版本	日期	说明
v1.0	2026 年 11 月	首版。收录 431 条资料，建立三重索引体系，配套在线检索与可打印 PDF。
v2.0	2026 年 11 月	补录 20 条（含 2026 赛季官方文件与近年新出教材），新增第五编「考场实务」三章：第一次上考场、NOI Linux 2.0 环境调试、爆零清单。封面与版式重做，PDF 加书签与元数据。

本书的资料条目来自公开互联网，评述部分为编者撰写。感谢所有把知识免费公开的作者——OI Wiki 的维护者、开源课件的作者、把论文扫描上传的前辈，以及每一位在题解区写下「这里有个坑」的选手。

特别感谢中国计算机学会及各省组织单位，让这套竞赛体系得以稳定运行三十年。

编者　2026 年 11 月